

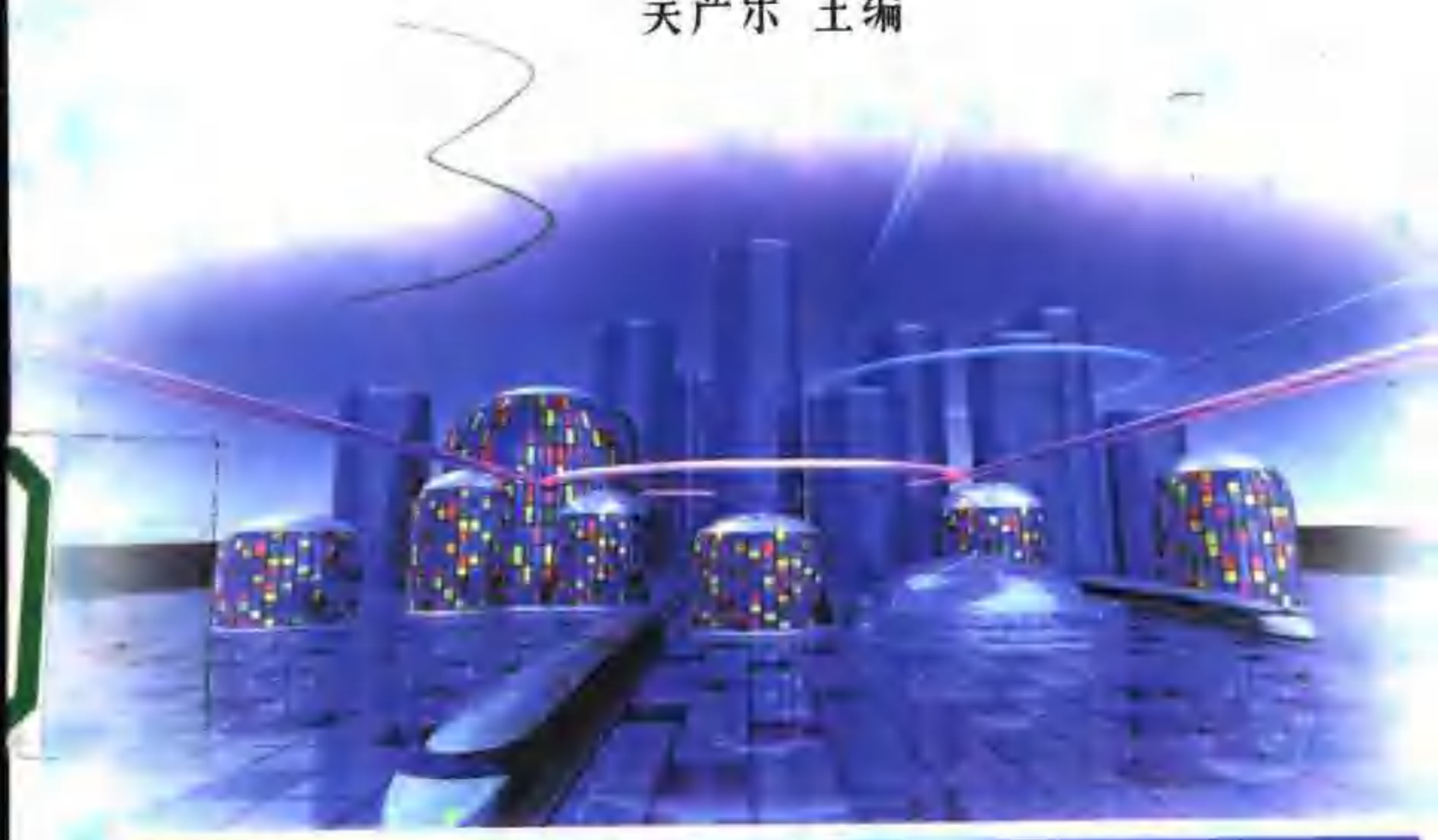
面向 **21** 世纪计算机专业本科系列教材



微机系统与接口技术

学习指导 · 题解 · 实验

吴产乐 主编



华中科技大学出版社

HUAZHONG UNIVERSITY OF SCIENCE AND TECHNOLOGY PRESS

面向 **21** 世纪计算机专业本科系列教材

- 离散数学
- 数字逻辑
- 计算机组成原理
- 微机系统与接口技术
- **微机系统与接口技术 学习指导·题解·实验**
- 计算机系统结构
- 数据结构
 - 用Pascal语言、C++语言对照描述算法
- 汇编语言程序设计
- 操作系统原理
- 数据库系统原理
- 编译原理
- 软件工程
- 计算机图形学
 - 原理、方法与应用
- 通信原理概论
- 计算机网络
- 信息安全与保密
- 电子商务概论

责任编辑：叶见欣

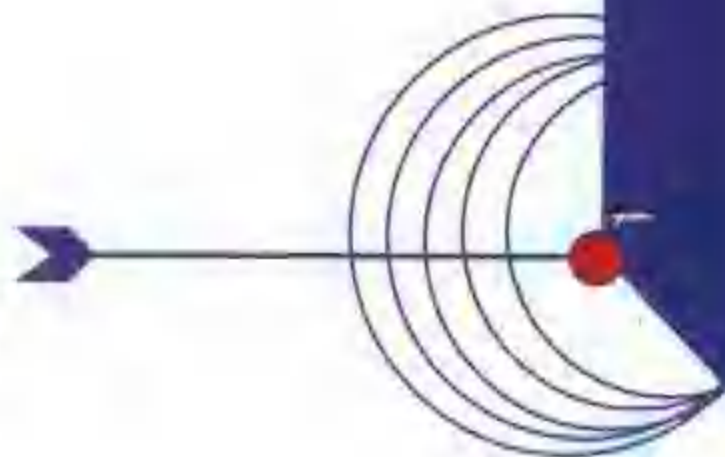
封面设计：刘 卉

ISBN 7-5609-3174-X



9 787560 931746 >

定价：28.80元



面向 21 世纪计算机专业本科系列教材

微机系统与接口技术

学习指导·题解·实验

主编 吴产乐

编者 吕 慧 陶慕柳 彭鸿雁 邱小燕

叶 刚 华 宇 李 薇 徐贵钢

关 钢 杨春贵

华中科技大学出版社

图书在版编目(CIP)数据

微机系统与接口技术 学习指导·题解·实验/吴产乐 主编
武汉:华中科技大学出版社,2004年8月
ISBN 7-5609-3174-X

I. 微…

Ⅱ. 吴…

Ⅲ. 微型计算机-接口-高等学校-教学参考资料

Ⅳ. TP36

微机系统与接口技术 学习指导·题解·实验

吴产乐 主编

责任编辑:叶见欣

封面设计:刘 卉

责任校对:封春红

责任监印:熊庆玉

出版发行:华中科技大学出版社

武昌喻家山 邮编:430074 电话:(027)87557437

录 排:华中科技大学惠友文印中心

印 刷:湖北科学技术出版社黄冈印刷厂

开本:787×960 1/16

印张:22.75

字数:406 000

版次:2004年8月第1版

印次:2004年8月第1次印刷

定价:28.80元

ISBN 7-5609-3174-X/TP·526

(本书若有印装质量问题,请向出版社发行部调换)

内 容 简 介

本书是《微机系统—接口技术》教材的图止续片。继冲照解和应响也已书。本书七教材同

前 言

本书是《微机系统与接口技术》的同步辅导、精选题解和实验指导书。它包含了上述教材的主要内容，并兼蓄了同类经典教材精华，补充了一些虽然不是教学大纲要求但是非常实用、先进的知识、方法和手段，有助于学生的知识立体化。本书可与课程教材同步使用，在广度和深度上拓展了教学内容，帮助学生理解知识要点、掌握重点难点，进行自测自考，帮助教师辅导答疑解惑和指导实验设计。

全书共分三篇 15 章，以及 6 个附录。同步辅导篇（第 1~9 章）为课程同步学习指导，对应教材《微机系统与接口技术》9 章的内容。每章由知识要点、例题精析、学习自评和教学建议组成。每章知识要点包括该章的重点和难点，对本章重点和难点进行了归纳和综合，深入地阐述以及适当地扩展，以加深对教材知识要点的理解；例题精析从本套计算机系列教材、教育部确定的“面向 21 世纪课程教材”、国外流行教材、名校考研和自考试题库中选题，给出了详细的解题方法、思路、步骤和答案；学习自评包括难度适当的自测练习及其答案，帮助学生了解自己对课程内容的理解程度和解题能力；教学建议根据各校的课时安排提出了教师的讲授重点和学生自学章节的建议。为了让读者更系统深入地了解《微机系统与接口技术》教材中微处理器的内容，第 2 章知识要点中针对 32 位微处理器进行了适当讲解和扩充；在例题精析与学习自评中引入了大量有关 32 位系统、保护模式、多任务等内容的例题和试题，并给出了详细的解题思路和方法。然后介绍了一个解析 32 位微处理器的综合程序，包括系统初始化、进入保护模式、建立页表及启用分页机制、多任务调度与管理等内容，并对每一部分的程序段进行了详尽的说明，以便读者能更好地理解 32 位微处理器的功能、特点以及编程方法。综合训练篇（第 10~11 章）为综合试题与模拟试题，各有一章，都附有解题思路和参考答案。综合试题是从历年课程考试、研究生考试、外版教材、产品手册中提炼出来的。模拟试题是根据教学大纲的要求，参考历年国内外名校课程考试试题精心设计的，都是为了测试学生对知识要点的掌握程度和综合应用知识的能力。实验指导篇（第 12~15 章）为实验指导，分为 4 章，介绍了实验的仪器设备、编程环境和调试方法，设计了定时器/计数器、中断控制器、DMA 控制器、并行接口、串行接口、D/A 和 A/D 变换等接口实验，给出了实验目的要求、接线图、操作说明和编程指导。书末的附录包括每章思考题和练习题的详细参考答案，以及便于查阅的 Pentium 微处理器指令集、ASCII 码表、中断类型、MS-DOS 系统功能调用和系统 ROM BIOS 中的驱动程序功能调用。

本书可作为计算机及相关专业微机系统、接口技术类课程教材配套使用的教学参考书和实验指导书，也可作为从事微机系统和接口设计的研究开发人员的参考资料。对于想深入理解 32 位微处理器、接口技术和计算机系统的教师、学生和专业人士都将有所帮助。

本书主编吴产乐教授拟定了编写大纲和章节内容，参与了所有篇章的编写、统阅和审定了全书。李薇编写了第 1 章，陶慕柳编写了第 2 章，叶刚编写了第 3 章、第 9 章，华宇和彭鸿雁编写了第 4 章、第 5 章，邱小燕编写了第 6 章、第 7 章，吕慧和杨春贵编写了第 8 章、第 10 章和第 11 章，徐贵钢和关钢编写了第 12 章、第 13 章、第 14 章、第 15 章和附录 B~F。附录 A 教材各章的思考题与练习题参考答案由第 1~9 章编写人员各自编成。在本书编写过程中，武汉大学计算机学院及计算机工程系的领导和“微机系统与接口技术”课程教学小组的任课老师提出了许多宝贵的意见和具体指导，武汉大学计算机学院张沪寅副教授、博士研究生邢建兵、程伟、吴黎兵、张秀山，硕士研究生吴梦晓等提供了大量可供选择的例题、试题和产品资料，武汉理工大学和华中师范大学计算机学院的老师一直关注本书的内容和进展，华中科技大学出版社的领导和编辑作了许多指导性和服务性工作，编者在此一并表示诚挚的谢意。

由于时间及水平的限制，书中难免有不妥之处，敬请批评指正。

武汉大学 吴产乐

2003 年 11 月于武昌珞珈山

目 录

同步辅导篇

第 1 章 概述	(1)
1.1 知识要点	(1)
1.1.1 微计算机系统组成	(1)
1.1.2 典型微处理器简介	(1)
1.1.3 微机接口技术	(7)
1.2 例题精析	(8)
1.3 学习自评	(10)
1.3.1 自测练习	(10)
1.3.2 自测练习解答	(12)
1.4 教学建议	(13)
第 2 章 微处理器	(15)
2.1 知识要点	(15)
2.1.1 32 位微处理器的内部结构	(15)
2.1.2 寄存器组	(16)
2.1.3 指令系统的发展	(19)
2.1.4 工作模式	(20)
2.1.5 中断与异常	(25)
2.1.6 存储管理	(30)
2.1.7 多任务	(32)
2.1.8 高速缓存 Cache	(33)
2.1.9 Pentium 的引脚及功能	(37)
2.1.10 总线周期	(38)
2.2 例题精析	(41)
2.3 学习自评	(67)
2.3.1 自测练习	(67)
2.3.2 自测练习答案	(72)
2.4 教学建议	(76)

第 3 章 内存储器	(78)
3.1 知识要点	(78)
3.1.1 内存储器	(78)
3.1.2 DRAM 的刷新	(79)
3.1.3 半导体存储器性能	(79)
3.1.4 存储器芯片的组成	(80)
3.1.5 存储器地址空间的硬件组织	(80)
3.1.6 PC/XT 存储器子系统	(81)
3.1.7 奔腾机存储器子系统	(83)
3.2 例题精析	(83)
3.3 学习自评	(88)
3.3.1 自测练习	(88)
3.3.2 自测练习解答	(89)
3.4 教学建议	(91)
第 4 章 输入/输出 (I/O) 接口	(92)
4.1 知识要点	(92)
4.1.1 I/O 接口	(92)
4.1.2 可编程间隔定时器 (PIT) 82C54	(93)
4.1.3 可编程外围接口 (PPI) 82C55A	(95)
4.1.4 串行通信和通用异步收发器 (UART) 8250	(97)
4.2 例题精析	(100)
4.3 学习自评	(112)
4.3.1 自测练习	(112)
4.3.2 自测练习解答	(116)
4.4 教学建议	(127)
第 5 章 微机与外设的数据交换	(128)
5.1 知识要点	(128)
5.1.1 微机与外设的数据交换方式	(128)
5.1.2 8259A 可编程中断控制器 (PIC)	(128)
5.1.3 8237/82C37 DMA 控制器 (DMAC)	(131)
5.2 例题精析	(133)
5.3 学习自评	(140)

5.3.1 自测练习	(140)
5.3.2 自测练习解答	(142)
5.4 教学建议	(146)
第 6 章 总线技术	(147)
6.1 知识要点	(147)
6.1.1 总线技术概述	(147)
6.1.2 系统总线	(151)
6.1.3 局部总线	(154)
6.1.4 外设总线	(156)
6.1.5 通信总线	(159)
6.2 例题精析	(160)
6.3 学习自评	(161)
6.3.1 自测练习	(161)
6.3.2 自测练习解答	(163)
6.4 教学建议	(165)
第 7 章 用户交互接口	(166)
7.1 知识要点	(166)
7.1.1 键盘接口	(166)
7.1.2 打印机接口	(167)
7.1.3 显示器接口	(167)
7.2 例题精析	(173)
7.3 学习自评	(185)
7.3.1 自测练习	(185)
7.3.2 自测练习解答	(187)
7.4 教学建议	(190)
第 8 章 外存储器	(191)
8.1 知识要点	(191)
8.1.1 数据磁记录方式	(191)
8.1.2 数据磁记录编码技术	(191)
8.1.3 增强 IDE 接口	(191)
8.1.4 SCSI 接口	(193)
8.1.5 软盘存储技术的发展	(193)

8.1.6 硬盘存储技术的发展	(194)
8.1.7 磁带存储技术	(196)
8.1.8 光盘存储技术	(196)
8.2 例题精析	(198)
8.3 学习自评	(201)
8.3.1 自测练习	(201)
8.3.2 自测练习解答	(204)
8.4 教学建议	(206)
第9章 微计算机系统	(207)
9.1 知识要点	(207)
9.1.1 IBM PC/XT 系统基本组成	(207)
9.1.2 IBM PC/AT 系统基本组成	(209)
9.1.3 80386 系统基本组成	(211)
9.1.4 80386/80486 EISA 系统	(212)
9.1.5 Pentium 微机系统	(213)
9.2 例题精析	(214)
9.3 学习自评	(215)
9.3.1 自测练习	(215)
9.3.2 自测练习解答	(216)
9.4 教学建议	(216)

综合训练篇

第10章 模拟试题及解答	(217)
模拟试卷一	(217)
模拟试卷一答案	(219)
模拟试卷二	(221)
模拟试卷二答案	(224)
第11章 综合题及解答	(226)
综合题及解答一	(226)
综合题及解答二	(235)

实验指导篇

第 12 章 微机接口实验系统结构及使用说明	(250)
12.1 TPC-H 微机接口实验系统基本组成	(250)
12.2 实验台结构及使用说明	(250)
第 13 章 实验程序的建立与执行	(258)
13.1 编辑和运行汇编源程序所必备的软件	(258)
13.2 建立与执行汇编源程序的步骤	(258)
13.3 调试程序 DEBUG 及其使用	(263)
第 14 章 微机接口电路实验	(268)
14.1 可编程定时器/计数器 8254(8253)的原理及应用	(268)
14.1.1 实验目的	(268)
14.1.2 实验预备知识及学习要求	(268)
14.1.3 实验内容	(269)
14.1.4 编程提示及参考程序流程框图	(270)
14.1.5 实验报告要求	(271)
14.2 中断控制器 8259 的工作原理及应用	(272)
14.2.1 实验目的	(272)
14.2.2 实验预备知识及学习要求	(272)
14.2.3 实验内容	(273)
14.2.4 编程提示及参考程序流程框图	(273)
14.2.5 实验报告要求	(275)
14.3 可编程并行接口 8255 的原理及应用	(275)
14.3.1 实验目的	(275)
14.3.2 实验预备知识及学习要求	(276)
14.3.3 实验内容	(276)
14.3.4 编程提示及参考程序流程框图	(277)
14.3.5 实验报告要求	(279)
14.4 串行通信	(279)
14.4.1 实验目的	(279)
14.4.2 实验预备知识及学习要求	(279)

14.4.3	实验内容	(280)
14.4.4	编程提示及参考程序流程框图	(281)
14.5	数/模 (D/A) 转换器及应用	(283)
14.5.1	实验目的	(283)
14.5.2	实验预备知识及学习要求	(283)
14.5.3	实验内容	(284)
14.5.4	编程提示及参考程序流程框图	(285)
14.5.5	实验报告要求	(286)
14.6	模/数 (A/D) 转换器及应用	(286)
14.6.1	实验目的	(286)
14.6.2	实验预备知识及学习要求	(287)
14.6.3	实验内容	(288)
14.6.4	编程提示及参考程序流程框图	(288)
14.6.5	实验报告要求	(290)
14.7	DMA 传送	(290)
14.7.1	实验目的	(290)
14.7.2	实验预备知识及学习要求	(291)
14.7.3	实验内容	(292)
14.7.4	编程提示及参考程序流程框图	(293)
14.7.5	实验报告要求	(294)
第 15 章 微机硬件应用综合设计		(295)
15.1	控制七段数码管进行静态与动态显示	(295)
15.1.1	实验目的	(295)
15.1.2	实验内容	(295)
15.1.3	编程提示及参考程序流程框图	(296)
15.1.4	实验报告要求	(297)
15.2	继电器控制	(297)
15.2.1	实验目的	(297)
15.2.2	实验内容	(297)
15.2.3	编程提示及参考程序流程框图	(298)
15.2.4	实验报告要求	(298)
15.3	竞赛抢答器	(299)
15.3.1	实验目的	(299)
15.3.2	实验内容	(299)

15.3.3 编程提示及参考程序流程框图.....	(299)
15.3.4 实验报告要求.....	(300)
15.4 交通灯控制.....	(300)
15.4.1 实验目的.....	(300)
15.4.2 实验内容.....	(301)
15.4.3 编程提示及参考程序流程框图.....	(301)
15.4.4 实验报告要求.....	(302)
15.5 数字录音机.....	(302)
15.5.1 实验目的.....	(302)
15.5.2 实验内容.....	(302)
15.5.3 编程提示及参考程序流程框图.....	(302)
15.5.4 实验报告要求.....	(304)
附录 A 《微机系统与接口技术》思考与练习题参考答案.....	(305)
附录 B ASCII 码表及控制符号定义.....	(327)
附录 C 中断类型分配.....	(329)
附录 D MS-DOS 系统功能调用.....	(331)
附录 E ROM BIOS 功能调用.....	(337)
附录 F Pentium 指令集简表.....	(342)
参考文献.....	(350)

1.1 知 识 要 点

1.1.1 微计算机系统组成

微计算机 (Microcomputer, MC) 是指以微处理器为核心, 配上存储器、I/O 接口电路及其他器件, 通过系统总线构成的计算机, 又称微型计算机, 简称微机。

微处理器是微机的核心, 它决定计算机的性能和档次。其位数越多, 处理能力就越强。微机内存储器包括 RAM 和 ROM 两部分, 内存容量的大小与微机的性能有密切的关系。I/O 接口是微机与外设进行数据交换的通道。

微机系统 (Microcomputer system, MCS) 是指以微机为中心, 配以相应的外部设备、电源、辅助电路和机架以及系统软件、支持软件、应用软件所构成的计算机系统。

微机系统组成参见教材^[1]中的图 1.1。

1.1.2 典型微处理器简介

通常所说的 8 位 CPU、16 位 CPU、32 位 CPU 是指数据总线的宽度分别是 8 位、16 位和 32 位。

1. 8 位微处理器

(1) 8 位微处理器比较

3 种典型的 8 位微处理器性能比较如表 1.1 所示。

(2) 8 位微处理器特点

Z80 是 8 位微处理器中功能较齐全的一种, 它具有较多特点。

① 设有两组通用寄存器 (AF、A'F'; BC、B'C'; DE、D'E'; HL、H'L'), 可以分别作为 8 位寄存器或成对组成 16 位寄存器使用; 可由指令相互切换工作, 提高了程序转换及数据处理速度。

表 1.1 典型 8 位微处理器性能比较

名 称	8080	M6800	Z80
生产公司	Intel	Motorola	Zilog
组成芯片	8088、8228、8224	M6800	Z80
采用工艺	NMOS	NMOS	NMOS
工作频率	2MHz	1 MHz	2.5~4 MHz
电源种类	+5 V、-5 V、+12 V	+5 V	+5 V
数据线数	8	8	8
地址线数	16	16	16
I/O 端口	独立编址	存储器映像	独立编址
引脚封装	40DIP	40DIP	40DIP
寄存器组	9×8 位/2×16 位	3×8 位/3×16 位	18×8 位/4×16 位

② 增加了一些专用寄存器。设置 IX、IY 两个 16 位的变址寄存器，增加了寻址灵活性；设置了中断页面寄存器（I），可以在 RAM 的任何区域指定向量中断的入口地址；设置刷新计数器（R），指定刷新行地址，在每个取指令机器周期后的两个状态，输出刷新地址和刷新控制信号。刷新由 CPU 自动管理，一行刷新后 R 自动加 1。

③ 有较强的中断处理能力。设有非屏蔽中断和可屏蔽中断两种功能，可屏蔽中断可由程序设置为 3 种工作方式：IM₀、IM₁、IM₂。Z80 内部还设有链式中断优先权结构。

④ 时钟、电源种类单一。使用单相时钟 ϕ 和 +5 V 电源，使用方便。

⑤ 指令功能比较齐全。Z80 有 158 条指令，寻址方式较多，兼容了 Intel 8080 的指令系统；它还设立了位操作和一些高效指令（如数据块传送、检索功能指令等），大大方便了程序设计。

2. 16 位微处理器

(1) Intel 8086/8088

Intel 8086（以下省略 Intel），有 16 根数据线和 20 根地址线，可寻址的空间达 1 MB。Intel 公司推出准 16 位微处理器 8088 是为了与当时已有的一整套 Intel 外围设备接口芯片直接兼容使用，同时也降低了成本。8088 的内部寄存器、运算部件以及内部操作都是按 16 位设计的，但对外的数据总线只有 8 根。

8086/8088 由两个逻辑部件组成：总线接口部件 BIU 和执行部件 EU。

BIU 由下列部分组成：4 个 16 位的段寄存器 CS、DS、SS 和 ES，1 个 16 位的指令指针寄存器 IP，1 个 16 位的内部通信寄存器，1 个 20 位的地址加法器；1 个 6 B（8086）或 4 B（8088）的 FIFO 指令队列；以及总线控制逻辑。BIU 负责

与存储器及 I/O 芯片接口, 执行所有外部总线周期, 提供总线控制信号, 根据执行部件请求, 完成取指和进行数据交换。

EU 由下列部分组成: 4 个通用寄存器 AX、BX、CX、DX, 4 个专用寄存器 BP、SP、SI、DI, 一个标志寄存器 FR, 算术逻辑单元 ALU。EU 负责指令的执行, 进行数据处理和有效地址计算并管理寄存器。16 位标志寄存器 (FR) 设置了 9 位标志: OF、DF、IF、TF、SF、ZF、AF、PF 和 CF。

8086/8088 EU 操作过程如下: 从 BIU 的指令队列中取出指令操作码, 通过译码电路分析要进行的操作, 发出相应的控制命令, 控制数据进出 ALU 和相应的操作。如果是运算操作, 操作数经暂存寄存器送入 ALU, 运算结果经 ALU 数据总线送到相应寄存器, 同时标志寄存器 FR 根据运算结果改变标志位。如果执行指令时需要从外界存取数据, 则 EU 向 BIU 发出请求, 由 BIU 通过 8086/8088 外部数据总线访问存储器或外部设备, 通过 BIU 的内部通信寄存器向 ALU 数据总线传送数据。

(2) Intel 80286

Intel 80286 是一种高性能 16 位微处理器, 其芯片上共集成了 13.4 万只晶体管, 具有 68 个引脚, 采用四列直插式封装, 分开设置 16 条数据线和 24 条地址线, 时钟频率初始为 6 MHz, 后来逐渐提高到 8 MHz、10 MHz、12 MHz、16 MHz、20 MHz、25 MHz。

80286 具有存储管理和保护机构。它采用分段的方法管理存储器, 每段最大为 64 KB, 且支持虚拟存储器。80286 有两种工作模式: 实模式和保护模式。运行实模式时, 相当于一个快速的 8086, 20 位物理地址的形成与 8086 相同, 物理地址空间为 1 MB。运行保护模式时, 可寻址 16 MB 物理地址, 提供 1 GB 的虚地址空间, 并能实现段寄存器、存储器访问及特权级和任务之间的保护, 因而, 80286 能可靠地支持多用户多任务系统。

80286 内部分为 4 个独立的处理部件: 执行部件 EU、地址部件 AU、指令部件 IU 和总线部件 BU。每个部件都可与其他部件异步并行操作, 因而, 80286 的运行速度较 8086 快。

BU 包括总线接口电路、预取器和 6 B 的预取队列。BU 负责处理 CPU 和系统总线之间的所有通信和数据传送。

IU 包括指令译码器和已译码指令队列。指令部件将指令字节从预取队列中取出, 送入指令译码器。指令译码器将每个指令字节译码变成 69 位的内部码形式, 存入可容纳 3 条指令的已译码指令队列中。

EU 包括 ALU 及标志寄存器、通用寄存器阵列和控制电路等。控制电路接收已译码指令的 69 位内部码并产生控制电位序列送入 EU 及其他部件, 以完成指令执行并影响标志位。ALU 及标志寄存器分别用来进行算术与逻辑运算和保存控制

与状态标志。通用寄存器暂存操作数和运算结果。

AU 实现段式存储管理和虚拟存储管理,用于实模式下 20 位物理地址的形成,用于保护模式下对存储器的保护和形成 24 位物理地址。

3. 32 位微处理器

(1) Intel 80386 系列

1985 年 10 月 Intel 公司推出了第一片全 32 位体系结构微处理器 80386。80386 能直接寻址 4 GB 物理地址空间,64 TB 的虚拟存储空间;支持段式管理和页式管理;支持多任务多用户系统,提供 4 级特权级,对任务之间、任务和操作系统及应用程序进行严密保护隔离。此外,80386 增加了一个 16 位的高速缓冲器 Cache,使得运行速度加快。

80386 共有 6 个功能部件,它们是总线接口部件、指令预取部件、指令译码部件、执行部件、分段部件和分页部件。其中执行部件包括 3 个子部件:控制部件、数据处理部件、保护检测部件。总线接口部件、指令预取部件和指令译码部件构成 80386 指令流。执行部件、分段部件和分页部件形成 80386 的地址流水线,如图 1.1 所示。

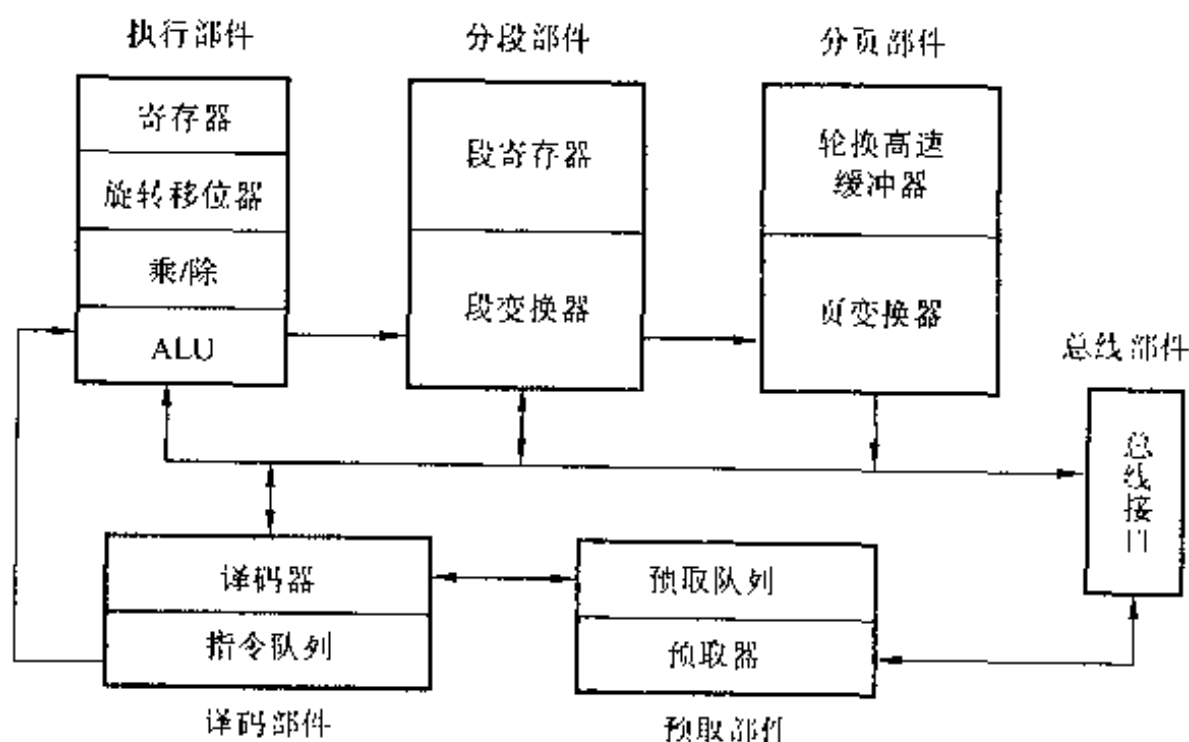


图 1.1 80386 部件的组成

各部件的功能如下。

① 总线接口部件:提供 80386 与其外部环境的接口。它接收预取部件取指和执行部件数据传送的请求,并做请求优先权处理;同时,产生或处理执行现行总线周期信号。另外,它还控制同外部总线主控设备和协处理器的接口。

② 指令预取部件:执行 80386 程序的提前查看功能。指令预取比数据传送优

先权低。如果总线没有请求数据传送,空闲的总线周期预取指令流存入 16 B 的预取队列中。

③ 指令译码部件:从预取队列中取出指令字节并将它们译成内部码存入指令缓冲的已译码指令队列中。

④ 执行部件:执行已译码指令队列中指令,并且与要求完成这条指令的其他部件通信。执行部件还用专用硬件来加速某些操作的执行。

⑤ 分段部件:将逻辑地址按执行部件要求变换成线性地址,并且用段描述符寄存器来加速地址变换。

⑥ 分页部件:当 80386 允许分页机制时,分页部件将分段部件或指令预取部件产生的线性地址变换成物理地址,并且使用 TLB 来加速地址变换。变换后的物理地址送总线接口部件,完成内存和 I/O 的存取。

(2) Intel 80486 系列

Intel 80486 微处理器是由主处理机部件、浮点运算部件、8 KB 的 Intel 高速缓存部件构成的。在 80386 微处理器子系统中,这 3 个部件是各自独立的芯片。在 80486 中,这 3 个部件被集中到一块芯片上。

把浮点运算部件 FPU 集成到 80486 芯片上,并且与 CPU 之间的数据通道是 64 位的,这不仅维持了 80486 与 80387 的软件兼容,而且还使 80486 的计算能力大大提高。80486 芯片内部还集成了一个 8 KB 的数据/指令混合型 Cache,它与片外的二级 Cache 联合起来可提高数据或指令的命中率,进一步提高了 CPU 指令执行的速率。

(3) Intel Pentium 系列

Intel Pentium 系列和 Itanium 系列微处理器性能比较如表 1.2 所示。

4. 64 位微处理器

拥有高时钟频率、整合第二级 Cache 及拥有 64 位带宽的微处理器已成为未来发展趋势。英特尔安腾 (Itanium) 处理器 (简称安腾处理器) 是英特尔公司 64 位处理器家族的第一位成员,它可使客户以更经济的成本获得针对高端 64 位服务器和工作站的更广泛的平台和应用选择。

安腾处理器采用 IA-64 构架,基于 EPIC (显式并行指令) 技术。采用该项技术,安腾处理器可同时执行多达 20 个操作。安腾处理器表现出世界级的浮点性能 (可在一个 CPU 时钟周期内完成 20 次浮点运算),尤其适合业务数据挖掘应用中的快速分析和高性能计算。基于安腾的服务器包含特殊指令,能够加快安全运算。其性能是配置相当的 RISC 系统的 10 倍。

基于安腾的服务器能够直接寻址 16 TB。基于安腾的服务器每秒可处理 2.1 GB。

表 1.2 Intel Pentium 系列和 Itanium 系列微处理器性能比较

	Pentium	Pentium Pro	Pentium MMX	Pentium II	Pentium III	Pentium 4	Intel Itanium II
推出时间	1993 年 3 月	1995 年 11 月	1997 年 1 月	1997 年 5 月	1999 年 2 月	2000 年 11 月	2001 年 4 月
线宽	0.8~0.35 μm	0.6~0.35 μm	0.35 μm	0.35~0.25 μm	0.25~0.13 μm	0.18~0.13 μm	0.13 μm
工艺	4 层金属配线 BiCMOS	4 层金属配线 BiCMOS	4 层金属配线 CMOS	4 层金属配线 CMOS	6 层 CMOS	6 层 CMOS	6 层铜布线
晶体管数	310 万个	550 万个	450 万个	730 万个	950 万个及更多	2800 万个	22000 万个
引脚封装	273 PGA	387 PGA	273 PGA	528 SEPP, slot 1	slot 1/socket 370	slot 1 或 FC-PGA 封装 的 socket370	slot 1
时钟频率	60~233MHz	155~433MHz	160~233MHz	233~450MHz	50MHz~1.4GHz	500MHz~1.3GHz	900MHz~1GHz
指令系统	IA-32 RISC + CISC	IA-32 CISC 指令 RISC 化	IA-32 RISC+CISC	Pentium Pro +MMX	IA-32	IA-32 NetBurst	IA-64(64 位)
主要特点	2 路超标量流水 线结构: U 和 V 结 构: 3 个并行译码 流水线	2 路超标量流水 线结构: U 和 V 结 构: 3 个并行译码 流水线	具有 Pentium III 所 有特性并采用多 媒体扩展 (MMX) 和 SIMD 技术 增加数据类型, 寄 存器 57 条增强 部件与两条流 水线配合: 1 个 指令周期可执 行两条指令	具有 Pentium III 所 有特性并采用多 媒体扩展 (MMX) 和 SIMD 技术 增加数据类型, 寄 存器 57 条增强 部件与两条流 水线配合: 1 个 指令周期可执 行两条指令	在 Pentium III 基 础上增加 70 条 指令, 8 条总 线和 8 个 128 位 寄存器	先进的系统缓冲器, 6 个填充缓冲器, 8 条总 线和 8 个 128 位 寄存器	超级流水线技术 支持双精度 SIMD 处理, SSE2 指令集 共有 114 组指令
	8KB 数据 L1Cache; 8KB 指令 L1Cache; 256KB L2Cache	8KB 数据 L1Cache; 8KB 指令 L1Cache; 256KB L2Cache (CPU 封装在 一起)	16KB 数据 L1Cache; 16KB 指令 L1Cache; 256KB L2Cache	16KB 数据 L1Cache; 16KB 指令 L1Cache; 256KB L2Cache	16KB 数据 L1Cache; 16KB 指令 L1Cache; 主频下 256KB L2Cache 在片内先进的缓存转 换架构	8KB 数据 L1Cache; 12KB 指令跟踪 Cache, 256/512KB L2Cache	128 位整数寄存器 128 位浮点寄存器 4 个整数运算执行单 元, 4 个 MMX 浮点运 算执行单元
	分支预测: 2 个 32 字节的缓冲 区	动态分支预测和推 断执行、乱序执行	分支预测: 2 个 32 字节的缓冲区分 析: 推测执行	动态执行技术, 多 分支预测: 数据流分 析: 推测执行	动态执行技术, 多 分支预测: 数据流分 析: 推测执行	高级动态执行引 擎, 快速执行引擎	1 个超大内存 (VIM) 单元功能以及推测功 能
	64 位外部数据 总线	64 位外部数据 (前 端) 总线内存: 64 位全速同步后 总线 L2Cache	64 位外部数据总 线	双重独立总线: 100MHz 前端总线; 二级高速缓存总线	双重独立总线结 构: 前端总线 100 或 133MHz	400MHz 前端总线 速度, QDR 技术	前端总线 400MHz (四 路 DDR (双铝数据传 输))

1.1.3 微机接口技术

1. 接口

接口 (interface) 就是微处理器与外部世界的连接部件 (电路), 它是 CPU 与外界进行信息交换的中转站。源程序或原始数据要通过接口从输入设备进来, 运算结果要通过接口向输出设备送出; 控制命令通过接口发出, 现场状态通过接口取进, 这些来往信息都要通过接口进行变换与中转。微机接口技术是采用硬件与软件相结合的方法, 研究微处理器如何与外部世界进行最佳耦合与匹配, 以在 CPU 与外部世界之间实现高效、可靠的信息交换的一门技术。外部世界是指除 CPU 以外的所有设备或电路, 包括存储器、I/O 设备、控制设备、测量设备、通信设备、多媒体设备、A/D 和 D/A 转换器等。

2. 微机接口功能

CPU 与外部设备 (简称外设) 之间的接口一般有如下功能。

(1) 数据缓冲功能

为了解决高速主机与低速外设的矛盾, 接口中一般都设置数据寄存器或锁存器, 称之为“数据口”。为了联络, 接口电路还要提供寄存器“空”、“满”或“准备好”、“忙”、“闲”等状态信号, 以便向 CPU 报告接口或外设的工作情况, 称之为“状态口”。

(2) 接收和执行 CPU 命令的功能

CPU 对 I/O 设备的控制命令一般均以代码的形式发到接口的命令控制寄存器中, 称之为“控制口”。接口电路对命令代码进行识别和分析, 并分解成若干个控制信号, 传到 I/O 设备, 使其产生相应的具体操作。

(3) 信号转换功能

由于外设所需的控制信号和所提供的状态信号往往同微机的总线信号不兼容, 尤其是计算机中往往要连接不同公司生产的芯片, 因此, 信号变换就变得不可避免。信号转换 (包括 CPU 的信号与外设的信号的逻辑关系上, 时序配合上以及电平匹配上的转换) 成为接口设计中的一个重要任务。

(4) 设备选择功能

微机系统中一般带有多种外设, 同一种外设也可能有多台, 而 CPU 在同一时间里只能与一台外设交换信息, 这就要借助于接口中的地址译码电路对外设进行寻址。高位地址用于芯片选择, 低位地址进行芯片内部寄存器的选择, 只有被选中的设备才能与 CPU 进行数据交换或通信。

(5) 中断管理功能

当外设需要及时得到 CPU 的服务, 特别是在出现故障需要 CPU 立即进行处理时, 就要求在接口中设置中断控制器, 为 CPU 处理有关中断事务 (提出中断请

求,进行中断优先级排队,提供中断向量等),这样既增加了微机系统对外界的响应速度,又使 CPU 与外设并行工作,提高了 CPU 的效率。

(6) 数据宽度变换的功能

CPU 所处理的是并行数据(8 位、16 位或 32 位),而有的外设只能处理串行数据,在这种情况下,接口就应具有数据“并→串”和“串→并”的变换功能。为此,在接口中设置了移位寄存器。

(7) 可编程功能

现在的接口芯片大都是可编程的,这样在不改动硬件的情况下,只修改驱动程序就可以改变接口的工作方式,大大增加了接口的灵活性和可扩充性。

3. 微机接口类型

微机接口的分类方法有多种,按功能分,有内存储器接口、I/O 接口、键盘接口、鼠标接口、显示接口、外存储器接口和模拟接口等;按应用分,有 3 种基本类型:运行辅助接口、用户交互接口和传感控制接口。

(1) 运行辅助接口

运行辅助接口是使微机实现最基本功能所需的接口。它包括微处理器周围的控制总线、地址总线和数据总线的锁存器、驱动器、接收器、收发器和时钟电路。

(2) 用户交互接口

用户交互接口是把用户指定的数据发送给主机系统或从主机系统接收数据的接口电路。它包括计算机终端接口、键盘接口、鼠标接口、显示接口、打印接口、操纵杆接口、光笔接口、录入笔接口、语音识别和合成接口等。

(3) 传感和控制接口

微机控制系统通过传感接口接收检测对象、控制对象的状态和数据,在进行处理后通过控制接口执行。传感接口具有模拟量到数字量的转换器(A/D 转换器)和数字量到微机系统总线的接口。控制接口将微机运算处理后得到的数字信号转换成适当的电压或电流,直接或通过机电接口驱动执行机构动作,以实现对外部世界的控制。A/D 转换器由于微机内总功率小,速度快、集成度高的元器件使它与外部世界的控制信号电平不相匹配,所以控制接口有时比较复杂,要将微机内部的数据信号转换成合适大小的电压或电流控制外部世界的部件或装置。

1.2 例题精析

例 1.1 Intel 8086/8088 标志寄存器的跟踪标志是什么?当跟踪标志的值为多少时,CPU 为单步操作状态。

解 Intel 8086/8088 标志寄存器的跟踪标志是 TF，当跟踪标志的值为 1 时，CPU 为单步操作状态。

说明 TF 为跟踪标志。TF=1 时为单步操作，CPU 每执行一条指令后进入内部中断；若 TF=0，则 CPU 处于正常的连续执行指令状态。

例 1.2 列出下列 CPU 的数据总线与地址总线的位数：

8080 8086 80286 80386 80486

解 这些 CPU 的数据总线与地址总线的位数如表 1.3 所示。

表 1.3 英特尔公司部分 CPU 数据总线和地址总线位数

	数据总线/位	地址总线/位
8080	8	16
8086	16	20
80286	16	24
80386	32	32
80486	32	32

例 1.3 简述 Z80 中指令“LD (HL), D”的意思。

解 Z80 中，指令“LD (HL), D”表示把 D 寄存器的内容传送到 HL 寄存器对所指定的存储单元中。这是一条单字节指令。指令中的源地址采用寄存器寻址方式，目的地址采用寄存器间接寻址。

例 1.4 简述 8086 指令系统的特点。

解 (1) 为了适当照顾与 8080 的兼容而保留了一些单字节的指令，这使用户可以用较少的代码字节进行编程实现较多的功能。

(2) 设置了功能较强的成组输入/输出指令、成组传送指令、字符串比较以及其他一些面向字节和字的指令。

(3) 在算术运算方面，8086 能够执行 8 位和 16 位带符号的四则运算。

(4) 在逻辑运算方面，8086 除了能执行标准的逻辑操作外，还能对特定的数值进行测试，对标志位置数。

例 1.5 判断指令

ADD A, (IX+50)

执行的寻址方式。

解 该指令执行的是变址寻址方式。

变址寻址是指将指令指定的变址寄存器的内容和指令中给定的偏移量相加，所得的结果作为操作数的地址。这里是把 IX 中的内容与指令给定的位移量 50H 相加作为操作数的地址，由这个地址单元取出的内容 X 与累加器 A 中的内容 Y

相加、结果送至 A 中。

例 1.6 怎样设置和清除跟踪标志 TF? 调试程序怎样能单步跟踪用户程序?

解 (1) TF 的设置和清除的程序段如下:

```
PUSHF
POP  AX
OR   AX,0100H
(或  AND AX,FEFFH)
PUSH AX
POPF
```

```
PUSH [FLAGS 映象]
```

```
PUSH [用户程序 CS]
```

```
PUSH [用户程序 IP]
```

```
⋮
```

```
⋮ 无 POP 操作
```

```
IRET
```

(2) 因为在 TF 设置前将控制转移到用户程序不能产生单步中断, 在 TF 设置后将控制转到用户程序之前将会对自身下一条指令跟踪, 必须同时进行 TF 设置和控制转移。在内存单元中造好 TF=1 的 FLAGS 映像、用户程序 CS 和 IP, 然后执行 IRET 指令实现上述双重功能。

1.3 学 习 自 评

1.3.1 自测练习

一、填空题

1. 80386 支持 (1)、(2) 和 (3) 三种模式。
2. 8088CPU 内部结构按功能可以分成两部分, 即 (1) 和 (2)。
3. 8086/8088CPU 内部共有 (1) 个 (2) 位寄存器。
4. 80386 的 6 大部件分别是: (1)、(2)、(3)、(4)、(5) 和 (6)。
5. Z80CPU 有 (1) 个 8 位寄存器和 (2) 个 16 位寄存器。
6. 8088/8086CPU 的标志寄存器中的 OF 表示 (1)。
7. 微机接口的主要功能有数据缓冲功能、接收和执行 CPU 命令的功能、(1)、

____(2)____、____(3)____、数据宽度变换的功能和可编程功能等。

8. 所谓接口就是____(1)____与____(2)____的连接部件(电路),它是 CPU 与外界进行信息交换的____(3)____。

二、选择题

1. 在一般微处理器中, () 包含在中央处理器(CPU)中。
A. 算术逻辑单元 B. 主内存 C. 输入/输出单元 D. 磁盘驱动器
2. CPU 对 DMA 控制器提出的总线请求响应要比中断请求响应快,其原因是 ()。
A. 只需完成访问内存操作 B. 只需释放总线控制权
C. 无须保留断点现场 D. 有硬件控制器
3. 程序查询 I/O 的流程总是按 () 次序完成一个字符的传输。
A. 读状态端口, 写数据端口, 读控制端口
B. 写数据端口, 读状态端口, 写控制端口
C. 读状态端口, 读/写数据端口
D. 随 I/O 接口具体要求而定
4. DMA 方式传送数据的正确顺序是 ()。
① DMAC 向 CPU 发请求;
② DMA 传送结束;
③ DMAC 发内存地址;
④ DMA 传送开始;
⑤ DMAC 发控制信号;
⑥ DMAC 向 I/O 发响应信号;
⑦ CPU 响应 DMAC 的请求;
⑧ I/O 向 DMAC 发请求。
A. 13564827 B. 81763542 C. 81567342 D. 48176352
5. 字长标志着计算机进行一次运算的精度, 8 位机的精度为 ()。
A. $1/128$ B. $1/256$ C. $1/512$ D. $1/1028$
6. 运算器又称为算术逻辑单元 ALU, 它由两个输入端来连接参加运算的两个参数, 通常一个与内部总线相连, 另一个来自 ()。
A. 控制器 B. 指令寄存器 C. 累加器 D. 存储器
7. 下述电路中 () 不是 DMAC (直接存储器存取控制器) 的组成部分。
A. 状态寄存器 B. 地址寄存器 C. 运算器 D. 字计数器

三、问答题

1. 8086/8088CPU 由哪几个部分组成? 各部分功能如何? 相互之间有什么联系?

2. 80286CPU 由哪 4 个独立部件组成? 各有何功能?
3. 80386CPU 由哪 6 个独立部件组成? 各有何功能?
4. 80486CPU 由哪几个部分组成? 各部分的功能如何?
5. 举例说明接口的不同类型和不同功能。
6. Pentium III 和 Pentium 4 各有何特性?
7. CPU 与接口之间有哪几种传送信息方式, 他们各应用在什么场合?
8. 请简要解释中断方式的过程。

1.3.2 自测练习解答

一、填空题

1. (1)实模式 (2) 保护模式 (3) 虚拟 8086 模式
2. (1) 总线接口单元 (2) 执行单元
3. (1) 14 (2) 16 位
4. (1) 总线接口部件 (2) 指令预取部件 (3) 指令译码部件
(4) 执行部件 (5) 分段部件 (6) 分页部件
5. (1) 18 (2) 4
6. 溢出标志
7. (1) 信号转换功能 (2) 设备选择功能 (3) 中断管理功能
8. (1) 微处理器 (或微机) (2) 外部设备 (3) 中转站

二、选择题

1. A 2. B 3. C 4. B 5. B 6. C 7. C

三、问答题

1~5 题略。

6. Pentium III 和 Pentium 4 基本特性如表 1.4 所示。Pentium III 的一些技术特性如下。

Katmai-Pentium III: 0.25 μ m; 新的SECC2封装SLOT1构架; 32KB L1Cache和512KBL2Cache(运行在芯片核心速度的一半以下); 包含MMX和Intel自己的“3D”指令SSE(Streaming SIMD Extention, 数据流指令多数据扩展功能)。前端总线频率为100MHz。

Coppermine-Pentium III: 0.18 μ m; FC-PGA封装的Socket370架构; 核心电压1.6V; 64KB全速L1和256KB全速L2Cache。前端总线频率为100MHz和133MHz两种。

Pentium 4 的一些技术特性如下。处理器频率: 1.4~2 GHz。制作工艺: 0.18 μ m 铝连接。核心电压: 1.5V。采用 423 针的新型 FC-PGA (反转芯片球状矩阵) 封装; 全新的扩展指令集 SSE2, 包括 144 个 128 位全新 SIMD 浮点管理指令; 8KB

L1 Cache, 256KB 全速 L2 Cache: 400MHz 的前端总线, 使用 Rambus 时带宽高达 3.2GB。

表 1.4 Pentium III 和 Pentium 4 特性比较

	Pentium III Processor	Pentium 4 Processor	Improvement 倍
Frequency	1 GHz	1.4 GHz	1.4
Adder Speed	1 ns	< 36 ns	> 2.8
Adder Bandwidth	2 billion/sec	> 5.6 billion/sec	> 2.8
L1 Cache Size	16 KB	8 KB	0.5
L1 Cache Speed	3 ns	< 1.42 ns	> 2.1
L1 Cache Bandwidth	16 GB/sec	> 44.8 GB/sec	> 2.8
L2 Cache Size	256 KB	256 KB	1
L2 Cache Bandwidth	16 GB/sec	44.8 GB/sec	> 2.8
System Bus	133MHz	400MHz	3
Data Per Second	1.06 GB/sec	3.2 GB/sec	3
ALU	Single	Double	2

7. CPU 与接口之间传送信息的方式有四种: 程序控制方式、中断方式、DMA 方式、处理机方式。其中程序控制方式包括无条件传送和查询传送。

无条件传送只能应用于简单对象控制, 如 LED 显示等; 查询传送在 CPU 不太忙, 传送速度不太高的情况下可以采用; 中断方式用于 CPU 任务比较忙, 传送速度不太高的系统中, 尤其适合实时控制及紧急事件处理; DMA 方式对于高速度大批量数据传送特别有用; 处理机方式比 DMA 方式更有效地减少 CPU 在 I/O 处理中的开销。

8. 首先由外设通过接口向 CPU 提出中断申请, 然后 CPU 中断当前执行的程序, 接着 CPU 响应申请, 保留当前程序指令地址和当前标志, 转到中断服务程序去执行, 即中断服务。服务完毕, 中断返回。

注 其他略。

1.4 教学建议

本章主要介绍微计算机系统组成、典型的微处理器和微机接口技术的概念、类型及其基本功能。重点是 Intel8080、M6800、Z80 等 8 位微处理器, Intel8086/8、80286 等 16 位微处理器, Intel80386、80486、Pentium、Pentium Pro、Pentium MMX、Pentium II、Pentium III、Pentium4 等 32 位微处理器和 Intel Itanium 等 64 位微处理

器和基本组成、体系结构、主要特性和发展过程及趋势。

教学中微处理器的发展过程重点讲述 Z80、Intel 的 8088、80286、80386 和 Pentium 后面的产品。Intel 的 80486 和 Pentium 是本书第 2 章的主要内容，可只作承前启后作用的描述。

如果教学安排的课时较少，本章内容可以作为学生自学，教学仅作概括性的导读。

第2章 微处理器

2.1 知 识 要 点

2.1.1 32位微处理器的内部结构

1. Intel 80486 的内部结构

Intel 80486 内部有如下部件。

① 总线接口部件：与芯片总线、指令预取部件、Cache（高速缓存）部件相连，执行总线访问请求。总线操作包括取指令、读内存、Cache 行填充等读操作和以写缓冲方式进行的写内存、输出、Cache 通写等写操作。

② 指令预取部件：总线空闲时产生存储器地址，向 Cache 或通过总线接口部件向内存取 16B 的代码放入 32B 长度的指令队列中。

③ Cache 部件：80486 片内有 8KB 的数据和指令一级 Cache，通过 128 位的通道与指令预取部件相连。命中 Cache 时，将 16B 的代码放入预取队列中；未命中时，CPU 通过总线接口部件从内存中取指和进行 Cache 行填充。

④ 指令译码部件：通过两步译码，从指令预取队列取机器码，转换成控制信号。

⑤ 控制部件：对整数部件、浮点运算部件和段部件进行控制。

⑥ 整数部件：包括 9 个 32 位寄存器、1 个桶形移位器和 1 个 ALU，在一个时钟周期内完成整数传送、算术运算或逻辑操作。

⑦ 浮点部件：具有增强的 80387 功能的浮点部件通过 64 位宽的通道与 CPU 相连，速度快。

⑧ 段部件与页部件：进行存储器保护和虚存管理。逻辑地址通过段部件转换成线性地址，然后通过页部件转换成物理地址。

2. Intel Pentium 的内部结构

Pentium 微处理器包含了 80486 的全部特性，并且增强和增加了许多功能，包括优化的高速缓存结构、更宽的数据总线、更快的协处理器、双整型处理器以及

分支预测逻辑。

高速缓存采用哈佛结构, 2 个 Cache, 每个 8KB, 一个用于数据缓存, 另一个用于指令缓存, 这样就完全避免了指令预取与数据操作二者之间的冲突。而且, 每组 Cache 都有各自的转换后援缓冲器 (TLB), 能够将代码或数据的线性地址转换成物理地址。

Pentium 的外部数据总线经总线接口部件从 32 位扩展到 64 位, 有效地克服了外部总线上的瓶颈效应。Pentium 有 3 个独立的处理单元: 一个流水线式浮点处理单元和两个被 Intel 标为 U 和 V 的整型处理单元, U 和 V 各有 5 级流水线。这种超标量流水线式体系结构, 利用资源重复和时间重叠, 提高了空间和时间的并行性, 从而提高了指令的平均执行速度。

指令预取缓冲器从指令代码 Cache 或内存中预取指令, 经译码后按类别输入 U 与 V 两条流水线中。U 流水线可以执行全部指令, V 流水线只能执行简单指令, 同一时刻二者可以并行操作。

Pentium 借助分支目标缓冲器等逻辑部件实现了分支转移的动态预测, 分支预测逻辑能够存储转移指令中分支的目标地址与当前分支转移的次数等统计信息。这样, 分支预测逻辑就能够可靠地预测分支走向, 指令预取队列就可以预取较多的指令, 使得流水线管道不会空闲, 从而大大加速了程序的执行。

2.1.2 寄存器组

1. 通用寄存器、指令计数器和标志寄存器

EAX: 可作为 EAX、AX、AH、AL, 用作累加器和寻址存储单元等。

EBX: 可作为 EBX、BX、BH、BL, 用作基址寄存器和寻址存储单元等。

ECX: 可作为 ECX、CX、CH、CL, 用作计数器和寻址存储单元等。

EDX: 可作为 EDX、DX、DH、DL, 用作数据寄存器和寻址存储单元等。

ESI: 可作为 ESI、SI, 用作源变址寄存器, 为串操作指令寻址源数据串。

EDI: 可作为 EDI、DI, 用作目标变址寄存器和通用寄存器等。

EBP: 可作为 EBP、BP, 用作基址指针, 可作为堆栈的一个附加指针。

ESP: 可作为 ESP、SP, 用作堆栈指针, 自动指向栈顶位置。

EIP: 可作为 EIP、IP, 用作指令指针, 保存下一条将要执行的指令的偏移量。

EFLAGS: 可作为 EFLAGS、FLAGS, 用作标志寄存器。80X86~Pentium 系列微处理器 EFLAGS 和 FLAGS 寄存器如图 2.1 所示。

2. 段寄存器和描述符寄存器

共有 6 个段寄存器: CS、SS、DS、ES、FS 和 GS。段寄存器的功能在实模式下和保护模式下是不同的。

31~22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
全0	I D	V I P	V I F	A C	V M	R F	0	N T	IO PL	O F	D F	I F	I F	S F	Z F	0	A F	0	P F	I	C F	
							0 0 0 0			8086/8088 (9个标志)												
							0	80286 (11个标志)														
0 0 0 0					80386 (13个标志)																	
0 0 0				80486 (14个标志)																		
Pentium (17个标志)																						

图 2.1 80X86~Pentium 系列微处理器 EFLAGS 和 FLAGS 寄存器

① CS 作为代码段寄存器，定义存放代码的存储器段的起始地址、长度和属性。在实模式下，它定义一个 64KB 存储器段的起点；在保护模式下，它选择一个描述代码存储器起始地址、长度和属性的描述符。对于 16 位的微处理器，代码段的最大限长为 64KB；对于 32 位微处理器，代码段的最大限长为 4GB。

② SS 作为堆栈段寄存器，定义堆栈用的存储区，由堆栈段基址和堆栈指针确定堆栈段内当前的入口地址。

③ DS 作为数据段寄存器定义程序使用的数据存储区域，可以通过偏移地址或者其他含有偏移地址的寄存器的内容寻址数据段内的数据。同 CS 一样对于 16 位的微处理器，代码段最大限长为 64KB，对于 32 位微处理器代码段最大限长为 4GB。

④ ES 作为附加段是为了一些串指令存放目标数据而附加的一个数据段。

⑤ FS 和 GS 是 32 位微处理器新增的两个寄存器段。

注意 在保护模式下，段地址不再像实模式那样由段寄存器提供。在原来存放段地址的段寄存器里含有一个选择符，用于选择描述表内的一个描述符。描述符描述存储器段的基址、限长和属性。在将选择符装入一个段寄存器时，微处理器自动将内存中对应的描述符透明装入段部件中的描述符寄存器，便于 CPU 在片内访问。

3. 控制寄存器

CR₀: 协处理器仿真 EM (2) 和协处理器类型 ET (4) 仅用于 80386；数字错误 NE (5)、写保护 WP (6)、对齐标志 AM (18)、非通写 NW (29) 和 Cache 禁止 CD (30) 等控制位只对 80486 和 Pentium 有效；其余保护模式允许 PE (0)、监视协处理器 MP (1)、任务转换 TS (3) 和分页允许 PG (31) 等控制位用于所有 32 位微处理器。ET 用于指示 80386 的协处理器的类型，EM 用于 CPU 模拟在没有 FPU 时执行协处理器指令。MP 用于监视协处理器，可与 EM 或 TS 配合使用。由于在任务转换时未切换 FPU 的运行状态，只有在新任务 (TS = 1) 需要 FPU

工作 ($MP = 1$, 遇到 FPU 操作码或 WAIT 指令) 时才进行 FPU 的现场切换。PE 和 PG 分别用于保护模式和分页管理的允许及禁止。NE 控制数字错误是否引起异常 16。WP 控制是否允许核心代码对用户页面进行写。AM 与 EFLAGS 中 AC 一起控制 CPL = 3 的程序的存储器操作是否进行对齐检查和相应处理。CD 和 NW 的组合用于控制 Cache 能否工作和工作时是通写还是回写操作。

CR₂: CR₀ 中 PG = 1 时 CR₂ 有效。当某页不在内存中, 页变换时发生页故障, CPU 将缺页调入内存, 其线性地址保存在 CR₂ 中。

CR₃: CR₀ 中 PG = 1 时 CR₃ 有效。CR₃ 中包含页目录表基址及片外页级 Cache 使能和写属性控制位。

CR₄: Pentium 扩充的控制寄存器, 使用最低的 7 位。虚拟 8086 模式扩展 VME (0) 和保护模式虚拟中断 PVI (1) 用于虚拟 8086 模式和保护模式支持 VIP, 以便改进虚拟 8086 应用程序的性能和使某些在 CPL = 0 才能执行的程序可以在 CPL = 3 时执行。时间标记禁用 TSD (2) 控制 RDTSC 指令是作为特权指令还是可以在任何特权级上执行。调试扩展 DE (3) 允许支持 I/O 断点。页面大小扩充 PSE (4) 和物理地址扩充 PAE (5) 允许采用 4MB 页面和 32 位以上的物理地址。机器检查允许 MCE (6) 使机器检查异常 (异常 18) 功能有效。

4. 系统地址寄存器 GDTR、IDTR、TR、LDTR

全局描述符表寄存器 GDTR: 包含系统唯一的 GDT 的 32 位线性地址和 16 位限长。

中断描述符表寄存器 IDTR: 包含系统唯一的 IDT 的 32 位线性地址和 16 位限长。

任务寄存器 TR: 包含当前任务的任务状态段 TSS 选择符, 指向 GDT 中一个 TSS 描述符。TSS 描述符又被透明装入对应的 TSS 描述符寄存器中, 包含 32 位线性基址、20 位限长和 12 位其他属性。

局部描述符表寄存器 LDTR: 包含当前任务的 LDT 的选择符, 指向 GDT 中一个 TSS 描述符。LDT 描述符透明装入对应的 LDT 描述符寄存器中, 包括 32 位线性基址、20 位限长和 12 位其他属性。

5. 调试寄存器

DR₀~DR₃: 各含一个断点的 32 位线性地址。可以设置数据断点、代码断点 (包括 ROM 断点) 和 I/O 断点 (仅 Pentium 支持)。

DR₇: 调试控制寄存器, 用于指明断点发生的条件和访问类型。R/W 确定发生断点的访问类型: 指令执行、数据写入、I/O 读/写或数据读/写。LEN 确定断点地址开始存放的数据长度是字节、字或双字。G/L 指明断点是全局/局部使能。GE/LE 指明全局/局部断点是精确还是非精确断点。

DR₆: 调试状态寄存器, 用于指示调试异常是否发生 (B₀~B₃ 指示) 和发生

的原因（由 BD、BS、BT 区别）。

6. 测试寄存器和模型专用寄存器

Intel 80486 用 $TR_3 \sim TR_5$ 作 Cache 测试, $TR_6 \sim TR_7$ 作 TLB 测试。Pentium 的模型专用寄存器包含了上述功能,除了 TR_0 未定义外, TR_1 作奇偶检验逆寄存器, TR_2 作指令 Cache 结束位测试, TR_8 是 TR_7 的扩展, $TR_9 \sim TR_{11}$ 作 BTB 测试, TR_{12} 作分支预测和执行跟踪等性能测试。此外就是用于机器检查和性能检测的寄存器。Pentium 的模型专用寄存器必须在 $CPL = 0$ 时用 RDMSR 和 WRMSR 指令读/写,其余系统寄存器如 CR、DR 和 80486 的 TR 则可在 $CPL = 0$ 时用 MOV 指令传送。

2.1.3 指令系统的发展

1. 微处理器指令系统的扩充

Intel 80386 微处理器对 16 位微处理器的指令系统进行了扩充,80486 在 80386 的基础上增加了一些新的指令, Pentium 在 80486 的基础上又进行了一定的扩展,表 2.1 列出了 80486、Pentium 分别在 80386 和 80486 基础上新增加的指令。

表 2.1 80486 和 Pentium 增加的指令

80486 新增加的指令		Pentium 新增加的指令	
CMPXCHG	比较和交换	CMPXCHG8B	比较和交换 8 个字节
XADD	交换与相加	CPUID	返回 CPU 标识码
BSWAP	字节交换	RDTSC	读时间标记计数器
INVD	作废 Cache	RDMSR	读模型专用寄存器
WBINVD	写回并作废 Cache	WRMSR	写模型专用寄存器
INVLPG	作废 TLB 项	RSM	从系统管理中断返回

2. MMX 技术

1997 年 1 月, Pentium MMX 问世,它引入了新的数据类型和增加了 57 条新指令,显著地提高了多媒体数据的处理能力。这其中包括数据传送、比较、移位、转换指令和压缩/解压、饱和运算、积和运算、清除 MMX 状态指令等。

MMX 技术是在原 Pentium 的基础上扩充了面向多媒体信息处理的数据类型和指令,使之能够更加有效地处理视频、音频和高质量图形数据。传统的 80X86 指令系列是单指令单数据 (Single Instruction Single Data, SISD) 体系,一条指令只能处理一个数据。实际上,大量的多媒体数据存在着“对称成组”处理的特点。例如左右双声道的两路音频信号, R、G、B 三路视频信号及三维图形的 X、Y、Z 坐标信号等,可以用一条指令来同时处理多个数据以提高指令执行的效率,这是单指令多数据的体系。后来的 Pentium II 是 Pentium Pro 和 Pentium

MMX 的集成。

3. SSE 技术

MMX 的单指令多数据 SIMD 技术只改善了整数运行的速度,不能提高浮点处理的速度。为此,Pentium III 相对于 Pentium II 增加了适应 Internet 发展的 SIMD 流扩展(Streaming SIMD Extensions, SSE)技术,新增了 KNI(Katmai New Instructions)指令集的 70 条指令,这样就使得浮点数处理也具备了单指令多数据的能力。

(1) SSE 指令集

① 单指令多数据浮点运算指令,共 50 条。Pentium III 芯片内还增加了 8 个专门配套的 128 位浮点寄存器,一条指令可以同时处理 4 对单精度 32 位浮点数的运算,从而使 Pentium III 在语音数据处理以及图形的旋转、翻转和变型等三维空间浮点运算处理方面的整体性能有很大的提高。

② 内存连续数据优先处理指令,共 8 条。典型的存储器连续数据流有音频数据流、视频数据流、数据库访问数据流与图形处理数据流等。通过采用新的数据预存取技术,减少了 CPU 处理连续数据流的中间环节,大大提高了连续数据流处理的效率,也明显改善视频/音频信息处理的速度与质量。

③ 增强的多媒体运算指令,共 12 条。采用了比 Pentium II 更先进的算法,进一步增强和完善了 MMX 指令系统。

(2) SSE 2 指令集

Pentium 4 的指令系统在 Pentium III 的 SSE 的基础上又有进一步的发展,这也就是第二代单指令多数据流扩展(SSE2)。SSE2 由 144 组新指令构成,包括 SIMD 的 4 个单精度浮点或 2 个双精度浮点数处理、16 个字节/8 个字/4 个双字/2 个四字/1 个 128 位长字等多种数据格式的整数运算,还增加了其他 Cache 管理指令等,极大地提升了对多媒体数据的处理能力。

2.1.4 工作模式

1. 三种工作模式及其相互转换

32 位微处理器工作在三种工作模式下:实模式(Real)、保护模式(Protected)和虚拟 8086 模式(Virtual 8086)。

① 实模式的工作原理与 8086 基本相同,主要区别是借助操作数长度前缀,CPU 能处理 32 位数据。另外,在实模式下还可以使用新增的两个段寄存器 FS 和 GS,运行速度更快;扩充了新指令,功能和性能更高。

② 在保护模式下,引入了虚拟存储器的概念和描述符等数据结构,实现存储管理、控制转移和任务切换;用特权保护、存储器保护和操作系统保护等机制支持多任务。

③ 虚拟 8086 模式是一种既能有效利用保护功能又能执行 8086 代码的工作方式，可以说是保护方式的一种子方式。CPU 的工作原理与在保护模式下的相同，但程序指定的逻辑地址解释与 8086 相同，即可以和在实模式下一样执行 8086 的应用程序。另外在虚拟 8086 模式下，各段特权级均为 3，而在实模式下各段特权级为 0。所以在虚拟 8086 模式下执行特权指令、I/O 敏感指令和一些只能在 0 级代码段中执行的指令时将会发生异常。

这三种工作模式在一定条件下是可以相互转换的，如图 2.2 所示。

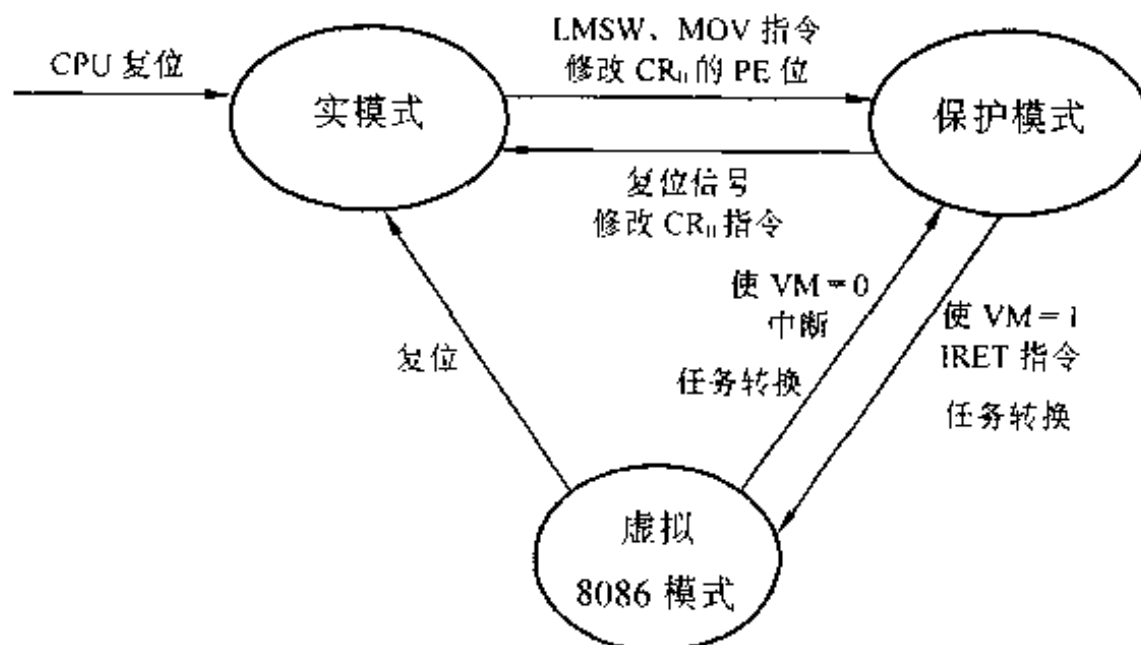


图 2.2 32 位微处理器三种工作模式之间的转换

2. 选择符、描述符和描述符高速缓冲寄存器

选择符用于检索 GDT 或 LDT 中的描述符，描述符用来描述段的基址、限长、属性、访问权限、各种类型等。一类叫做存储段的包括数据段（包括堆栈段）和代码段，其属性如表 2.2 所示；另一类叫做系统控制段的包括系统段（TSS 和 LDT）和各种门，其类型如表 2.3 所示。

在实模式和虚拟 8086 模式下，段寄存器含有段值，为了访问存储器形成物理地址，处理器引用相应的段寄存器并将其值乘以 16，形成 20 位的段基地址。在保护模式下，段寄存器含有段选择符，为了访问存储器形成线性地址，处理器要使用

表 2.2 数据段代码段描述符属性表

	G	D/B	P	DPL	S	E	ED/C	W/R	A
	粒度	尺度	在内存	特权级	描述符类	可执行	扩展/ 证实	读/写	存取性
0	1B	16 位	不在	0 至 3	系统控制	数据段	ED 向上	W 非写	未存取
1	4KB	32 位	在内存		数据代码	代码段	C 证实	R 可读	已存取

表示必须可读,“W”表示必须可写,“P”表示必须存在,否则发生异常;“B”表示以字节为单位,“U”表示向上扩展,“D”表示根据每个描述符中的属性而定,“-”表示此项不用。

3. 保护

(1) 段级保护

当处理器要访问代码段、数据段或系统段时,要进行三种保护检查。

① 类型检查。参见表 2.2 中 ED/C、W/R、A 的说明,描述符中属性位 $S=1$ 时描述的是代码段或数据段, $E=1$ 表明是代码段, C、R、A 位置位分别表示代码段是被证实的、可读的或已存取的; $E=0$ 表明是数据段, ED、W、A 位置位分别表示数据段是向下扩展(对于堆栈)的、可写的、已存取的。根据上述 4 位定义参数,处理器将进行段级第一个保护检查。在把描述符的选择符装入段寄存器,作上述类型检查时,只有可执行段的描述符的选择符才能装入 CS 寄存器,只有可写数据段的描述符的选择符才能装入 SS 寄存器,可执行段描述符属性如果不是可读的,则其选择符不能装入数据段寄存器中等。将不相容段类型的选择符装入段寄存器,或者是某指令试图访问一个类型与其不一致的段时,都会产生异常。如一个可执行代码段不能写入,若需写入,可以重建一个可写数据段描述符描述该段,以便写入代码段。相容的段寄存器和段类型如表 2.5 所示。

表 2.5 相容的段寄存器和段类型

段寄存器	段 类 型			
	只读	读/写	只执行	执行/读
DS、ES、FS、GS	是	是	否	是
SS	否	是	否	否
CS	否	否	是	是

② 限长检查。在实模式和虚拟 8086 模式下,段长度固定为 64KB。在保护模式下,段限长由 20 位的限长值和 G 位属性计算。当 $G=0$ 时,段最大长度为 1MB。当 $G=1$ 时,段最大长度为 4MB;此时如果段限长值为 0,则该段最大长度为 4KB,寻址范围为 0~4095B。

描述符属性中的 $ED=1$ 时,表示向下扩展的段,常用于堆栈段; $ED=0$ 时,表示向上扩展的段,常用于数据段。向下扩展段的限长表示一个段的最小偏移量,其合法的偏移量范围: $\text{段限长}+1 \leq \text{偏移量} \leq \text{FFFFFH}$ 或 FFFFFFFFFH ;当限长为 0 时,向下扩展段具有最大长度。除了向下扩展的段以外,所有的段描述符的限长都表示段的最大偏移量,其合法的偏移量范围: $0 \leq \text{偏移量} \leq \text{段限长}$ 。

限长检查不仅用于代码段、数据段和堆栈段,也用于系统段中描述符表。限

长检查防止寻址超过表的末端而装入非法的段描述符。LDT 描述符中段限长和 G 位含义与段描述符相同, IDT 和 GDT 的限长均为 16 位。描述符表的限长为 $8N-1$, 其中 N 是表中描述符的最大数目。

③ 特权检查。在段级保护中, CPU 与 CPL、DPL 及 RPL 相关。CPL 在 CS 寄存器的低 2 位; SS 寄存器的低 2 位总是复制 CPL 的值; DPL 是创建段描述符时指定的; RPL 在选择符的低 2 位, 如果 RPL 的特权级低于 CPL, CPL 便降级与 RPL 匹配, 结果则是有效特权级 EPL。一个任务的 CPL 是动态随 CS 寄存器值变化, 程序中每个段都有固定特权级, 故一个任务的特权级取决于该任务当前执行到应用程序或系统软件中的哪一个段。

特权级检查对代码段、数据段、门、任务状态段的描述符特权级都要检查, 当段选择符装入段寄存器时, CPU 对段特权级自动检查, 以便决定哪个程序段能够访问此段。

(2) 数据访问

对于段级保护的 4 级特权, 允许较高特权级的程序访问同特权级或低特权级的数据段。在数据访问时, 被访问的数据段 DPL 规定允许访问该段的最低特权级, 即 $DPL \geq CPL$ (或 RPL)。

(3) 控制转移

① 段内转移。段内转移通过近程 JMP 或近程 CALL 及 RET 指令执行。它不会引起特权级变化, 无须装入 CS 寄存器, 仅作限长检查, 限长值已预先装入到段描述符高速缓存中。

② 直接段间转移。直接段间转移通过远程标号的 JMP 或 CALL 及 RET 指令执行。远程标号直接给出一个 48 位远指针: 16 位选择符和 32 位偏移量。16 位选择符装入 CS 寄存器, 所选择的目标段描述符自动装入 CS 段描述符高速缓存寄存器中。接着进行保护检查: 当目标段描述符特权级 DPL 与现行特权级 CPL 相等, 且目标段是一个存在的、可执行的代码段; 或目标段 DPL 高于等于现行段 CPL, 且目标段是一个存在的、可执行的证实代码段时, 控制转移能够实现。

段描述符属性中 $S=1$ 、 $E=1$ 、 $C=1$ 标志可执行的证实代码段, 它用来存放多个特权级程序共享的例程, 并在调用程序 CPL (低于等于可执行证实代码段 DPL) 上执行该例程, 而不要求改变特权级, 不在可执行证实代码段由其 DPL 表示的特权级上执行共享例程, 说明直接段间转移实现的是同一特权级同一任务的控制转移。

③ 通过调用门的段间转移。JMP 指令通过调用门在同一特权级转移控制; CALL 指令通过调用门使程序转移到更高特权级, RET 指令则相反。CALL (或 JMP) 指令的操作数为 32 位偏移量和 16 位选择符, 当选择符装入 CS 寄存器时, CPU 自动检查其选择的描述符。当发现调用门时, 就用门中选择符从 GDT 或 LDT

中得到目标段描述符。该描述符包含目标段基址，调用门中偏移量为目标段偏移量，CALL 中偏移量丢弃。显然这是一种间接的段间转移。通过调用门实现控制转移时，要检查门及目标代码段的特权级。调用门 DPL 应等于低于 CALL 指令所在段的 CPL，进而检查目标段 DPL。当目标段是其 DPL 等于当前 CPL 的非证实代码段，或高于等于当前 CPL 的证实代码段时，门的转移在同一级进行；如果目标段是其 DPL 高于当前 CPL 的非证实代码段，则程序控制向高特权级进行。只有在 CALL 指令通过调用门向高特权级转移时，如果目标段 DPL 低于 CPL，将发生一般保护异常。RET 指令不能使用调用门。

除上述特权级检查外，还要对描述符属性中 P 位及调用门中偏移量是否超越目标代码段中的界限等进行保护检查，所有检查通过后方能完成程序段间转移。

在进行控制转移，CALL 指令通过调用门向高优先级程序转移时，不仅切换特权级，也需要切换堆栈，堆栈的特权级由目标段 DPL 决定，这对保护程序很重要。然而，通常在调用门中规定目标段 DPL = CPL 或目标段为证实代码段，这时在通过调用门进行控制转移时，无须特权级转移和堆栈的切换。CALL 指令通过调用门把控制转移到较高特权级，当高优先级子程执行完成时，RET 指令又把控制转移到较低优先级的 CALL 指令所在段，同时把堆栈切换回原来的栈。

2.1.5 中断与异常

1. Pentium 中断系统

Pentium 中断系统由外部中断和内部中断组成，如图 2.3 所示。

(1) 外部中断

外部中断是由一些异常的外部事件引起的，有通常意义的可屏蔽中断 INTR 和非屏蔽中断 NMI；广义的包括系统初始化 INIT，它使 Pentium 进行内部自测试、复位和进入实模式，但保持片内 Cache、写缓冲器和浮点寄存器内容不变。

更广义的中断还包括：系统管理中断 SMI，它使 Pentium 执行完当前指令后输出有效的 SMI $\overline{\text{ACT}}$ 信号，向外界提示将进入系统管理模式；外界输入的 $\overline{\text{EWBE}}$ 有效使 Pentium 将寄存器内容暂时写入系统管理 RAM 中保存，整个写周期结束后立即进入系统管理模式，执行相应的中断处理程序实现如电源管理等功能；在系统管理模式中执行重新启动的 RSM 指令，可将系统管理 RAM 中暂存的内容重新装入 Pentium 寄存器，使系统恢复运行被中断的程序。

Cache 清除 FLUSH 有效时将数据 Cache 中所有修改行回写到内存并使内部 Cache 无效，在刷新完成时 Pentium 产生一个刷新应答周期。

探针模式 $\text{R}/\overline{\text{S}}$ 有效将中断现行程序，使 Pentium 进入探针模式工作，Pentium 输出有效的 PRDY 信号响应 $\text{R}/\overline{\text{S}}$ 的中断请求，指示 Pentium 准备接收探针方式指令，当执行指令时，PRDY 清 0；当 $\text{R}/\overline{\text{S}}$ 返回高电平时 Pentium 又重新启动程序执行；

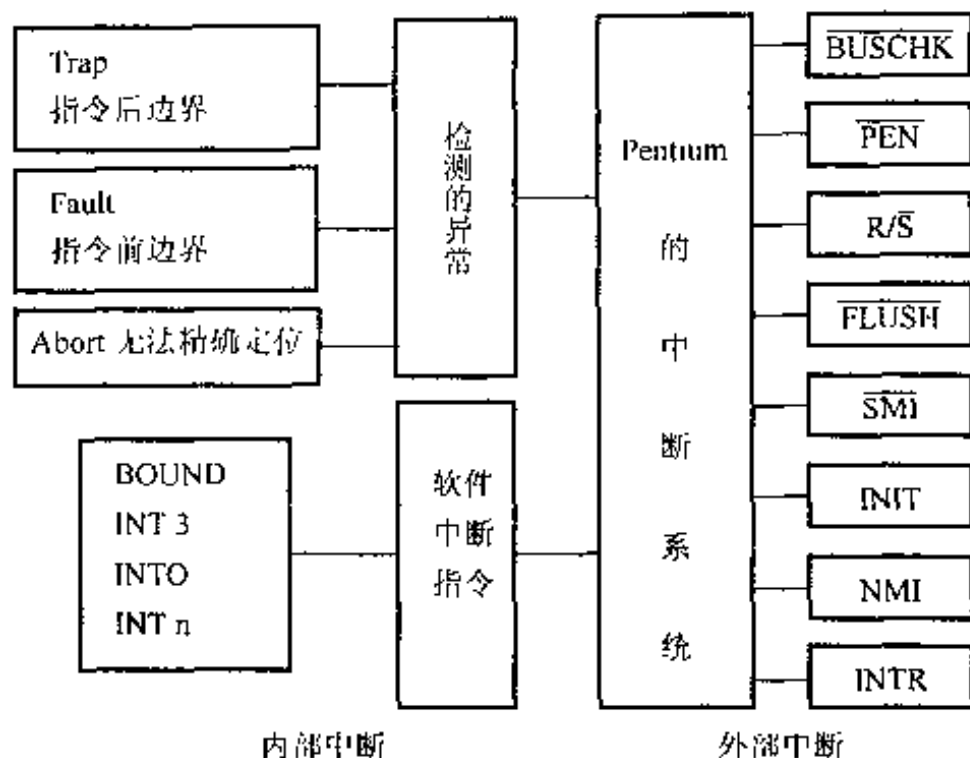


图 2.3 Pentium 的中断系统

探针模式是一种基于硬件的调试功能, Intel 的方案是一个带有 20 个连接点的外接硬件调试器支持系统调试, 便于用户检查并修改 CPU 的状态。奇偶校验允许 $\overline{PEN}$ 或总线周期检测 $\overline{BUSCHK}$ 是外部请求通知 Pentium 发生了读数据奇偶校验错或总线周期错。在控制寄存器 CR_4 的机器检查允许 MCE 为 1 时, 上述错误之一都会使处理器自动执行异常 18 (见本书第 29 页) 处理程序。

(2) 内部中断

内部中断包括指令执行前边界和后边界检测到的异常以及软件中断指令。异常根据异常条件检测是在指令后边界、指令前边界或者是无法精确定位, 可以分为 Trap、Fault 或 Abort。边界检查指令 BOUND、单字节断点中断指令 INT (或 INT 3)、溢出中断指令 INTO、双字节中断指令 INT n 都产生软件中断, BOUND 产生的中断属于 Fault, 其他中断指令产生的中断属于 Trap。

2. Pentium 中断与异常的暂时屏蔽

某些条件和标志的设置会引起处理器禁止一些类的中断和异常, 这时, 被屏蔽的各种中断被挂起, 而异常则被废弃。

- ① 当系统正在处理一个非屏蔽中断时, 屏蔽外部任何新的非屏蔽中断。
- ② 当 EFLAGS 中 $IF=0$ 时, 屏蔽外部可屏蔽中断。
- ③ 当 EFLAGS 中 $RF=1$ 时, 防止一个指令断点故障处理程序执行多次。
- ④ 当 EFLAGS 中 $IF=0$ 时, 在执行 STI 及其下面一条指令时, 屏蔽外部可屏蔽中断。这是因为在连续执行 STI 和 IRET 指令时, 若在 STI 后、IRET 前响应中断, 就会使中断服务程序不能返回, 过多占用堆栈。
- ⑤ 执行以 SS 为目标寄存器的 MOV 和 POP 指令时, 将在该指令及下一条指

令（通常是以 ESP 为目标寄存器的 MOV 及 POP 指令）执行期间屏蔽中断、调试异常和单步陷阱，这样可安全完整地更新一个栈指针。

⑥ 在指令前缀及被锁定的指令之间不允许中断。

3. Pentium 中断和异常的优先级

在指令边界有一个以上的异常和中断请求时，Pentium 处理器以预定的优先级进行相应服务。中断和异常类中优先级如表 2.6 所示。处理器首先服务有较高优先级的异常或中断请求，转移控制执行该处理程序的第一条指令，较低优先级异常被废弃，较低优先级中断被挂起。当较高优先级中断处理完后，再按优先级次序响应和处理挂起的中断。同时当较高优先级异常处理完，重新启动引起较低异常的指令时，任何丢弃的异常均可重新发生。

表 2.6 Pentium 中断和异常的优先级

优先级	类	说 明
最高	1	前条指令的陷阱： ①断点；②调试陷阱异常（TF 标志为 1，TSS 中 T 位为 1，数据/IO 断点）
	2	外部中断： ①NMI 中断；②INTR 中断
	3	预取下条指令故障： ①代码断点故障；②代码段限长违反故障；③预取页故障
	4	译码下条指令故障： ①非法操作码；②指令长度大于 15B；③协处理器不可采用
最低	5	执行一条指令故障： ①一般性检测；②来自前条 FP 指令的错误；③溢出中断； ④BOUND 指令边界检查出故障；⑤无效的 TSS；⑥段不存在； ⑦堆栈异常；⑧一般保护异常；⑨数据页故障；⑩对齐检查

表 2.6 说明在前条指令的后边界到下条指令前边界 Pentium 处理器响应中断和异常的优先级顺序。而对于一条指令来说，其从前边界到后边界响应中断和异常的优先级顺序是 Fault 异常，除调试外其他 Trap 指令，本条指令调试 Trap，下条指令调试 Fault、NMI 和 INTR。

4. 保护模式下的中断和异常

保护模式下的中断和异常，除了包括实模式下的异常 0、1、2、3、4、5、6、7、8、9（对协处理器段越限时，80486/Pentium 不会发生异常 9，而是引起异常 13）、12、13、16 外，Pentium 还有异常 10、11、14、17、18。以下将对保护模式下新增的和 Pentium 增强的异常 10~18 作一说明。

(1) 异常 10，无效 TSS 异常

① 任务转向一个无效 TSS 的段。

② 无效 TSS 状态涉及到 TSS 本身、LDT 段、堆栈段、数据段和代码段。

③ 使用一个任务门的外部中断（错误代码中 EXT 位表示）试图将一个任务转向一个无效 TSS 的段。

④ 当无效 TSS 异常发生转入异常处理程序时，压入栈中的 CS 及 EIP 值指向引起此异常的指令；而当该异常作为任务转换的一部分发生时，则指向新任务的第一条指令。

⑤ 为了确保处理异常的 TSS 是可采用的，无效 TSS 异常的处理程序必须是一个使用任务门调用的任务。

(2) 异常 11，段不存在异常

① 在试图装入除 SS 外其他段寄存器时，CPU 检测到对应的段描述符中 $P=0$ 。

② 在用 LLDT 装入 LDTR 时，LDT 描述符中 $P=0$ ，错误码中 $IDT=1$ 。

③ 使用了一个段不存在的门描述符。

④ 段不存在异常是可以重新启动的故障类异常。

(3) 异常 12，堆栈异常

① 涉及 SS 的任何操作（POP、PUSH、ENTER、LEAVE、LSS 及显式或隐式使用 SS 的存储器操作）的限长违反。

② 装载 SS 时（任务转换、不同特权级的调用和返回）CPU 检测到 $P=0$ 的描述符。

③ 引起堆栈异常的指令是可以重新启动的。

(4) 异常 13，一般保护异常

所有没有引起其他异常的保护违反引起的异常属于一般保护异常；实模式下存储器操作数地址超越限长时，产生异常 13，这时无错误码压入堆栈。保护模式下产生异常 13 举例：

① 使用 CS、DS、ES、FS、GS 或访问描述符表时超越段限长。

② 使用空选择符（全 0）的 DS、ES、FS、GS 访问存储器。

③ 使用段寄存器访问时，被访问的数据段 DPL 高于程序段 CPL。

④ 用系统段或只可执行段描述符的选择符装入 DS、ES、FS、GS。

⑤ 用只读段、可执行段或系统段描述符的选择符装入 SS。

⑥ 对只读代码段或数据段进行写，或对只执行段进行读，或将控制转移到不可执行段。

⑦ 试图用 $PG=1$ 和 $PE=0$ 装入 CR_0 ，或用 1 装入 CR_4 的保留位。

⑧ 切换到正忙的任务。

⑨ 超出处理器指令限长。

⑩ 虚拟 8086 模式通过中断门或异常门的中断服务程序的特权级不是 0。

⑪ 当执行 CLTS、HLT、LGDT、LIDT、LLDT、LMSW、LTR、RDMSR 或 WRMSR 指令时，程序的 $CPL \neq 0$ 。

② 当执行 CLI、STI、LOCK、IN、OUT、INS 或 OUTS 时，CPL>IOPL 或 TSS 的 I/O 位图不允许。

(5) 异常 14, 页故障

当 CR₀ 中 PG = 1, 在线性地址变换到物理地址时, CPU 检测下列状态之一会发生异常 14。页故障是可以重新启动的异常, 一旦引起页故障的原因在处理程序中被排除, 则中断返回可以重新执行原产生页故障的指令。

① 地址变换所需的页目录项或页表项中 P = 0, 指示页表或包含操作数的页不在内存中。

② 程序没有足够的特权访问指定的页。

③ 用不适当的访问类型访问内存中页。

④ 页级保护违反。

(6) 异常 16, 协处理器出错

① 当 CR₀ 中 NE = 1, 一个非屏蔽的浮点异常在下一条非控制的浮点指令或 WAIT 指令执行前将产生异常 16。

② 当 CR₀ 中 NE = 0, 且引脚 $\overline{\text{IGNNE}} = 1$, 一个非屏蔽的浮点异常在下一条非控制的浮点指令或 WAIT 指令执行前将冻结 CPU, 等待由响应 FERR 引脚输出“0”的外部中断带出, 在这种情况下外部中断将调用异常处理程序。而当 CR₀ 中 NE = 0, 且 $\overline{\text{IGNNE}} = 0$, 则 CPU 忽略浮点异常。

(7) 异常 17, 对齐检查异常

对齐检查是在 CR₀ 中 AM = 1、EFLAGS 中 AC = 1 和 CPL = 3 时进行的, 在访问非对齐的操作数时产生对齐检查故障。数据类型有对齐需求 (要求其地址低位为 0、00 或 000), 不符合对齐要求的将引起异常 17。以下为不同数据类型的对齐需求。

① 地址应能被 2 整除的数据类型: 字、选择符, 32 位段指针等。

② 地址应能被 4 整除的数据类型: 双字、短实数、48 位段指针、位串等。

③ 地址应能被 8 整除的数据类型: 长实数、暂实数 (TEMPREAL) 等。

(8) 异常 18, 机器检查异常

当 Pentium 的 CR₄ 中 MCE = 1 时, 允许系统执行异常 18, 处理机器检查异常。下列情况之一都会引起 Pentium 的机器检查异常:

① 在 $\overline{\text{PEN}}$ 引脚输入逻辑 0, 外部电路通知 CPU 在读周期发生数据奇偶校验错。

② 在 $\overline{\text{BUSCHK}}$ 引脚输入逻辑 0, 外部电路通知 CPU 当前总线周期没有正常结束。

引起异常的当前总线周期地址和类型分别锁存在机器检查地址寄存器 MCA (模型专用寄存器 MSR-00H) 和机器检查类型寄存器 MCT (模型专用寄存器

MSR-01H)。64 位的 MCT 最低 5 位是 b_4 : Lock、 b_3 : M/IO、 b_2 : D/C、 b_1 : W/R、 b_0 : CHK。类型锁存到 MCT 时, CHK 位置 1, 读 MCT 时, CHK 位清零。异常 18 处理程序可以访问 MCT 中信息, 以确定引发异常的原因, 进行处理。

利用 RDMSR 指令可以读 MCA 和 MCT 的内容。如:

MOV ECX, Imm ; Imm 代表 MSR 的编号, 如 00H, 01H 等

RDMSR ; EDX: EAX ← MSR[ECX]

利用 CPU ID 指令在 EAX = 1 时执行后, 由特性标志寄存器 EDX 的位 7 可以确定机器检查异常的出现。

2.1.6 存储管理

1. 实模式存储管理

32 位微处理器可工作于实地址模式、保护虚拟地址模式或虚拟 8086 模式。在实模式下, 所有存储单元的 20 位地址都由段值乘以 16 加偏移量组成。装在段寄存器的段值定义任何 64KB 存储器段的起始地址, 偏移量用于在 64KB 存储器段内选择任一单元。实模式操作方式只允许微处理器寻址第一个 1MB 存储空间, 实模式段只能起始于存储器系统内 16B 整数倍的边界, 这个 16B 边界通常称为分段。

实模式下页部件不工作, 其线性地址就是物理地址, 如上述方式形成, 其段长度固定为 64KB。实模式下代码段的 CPL、数据段和堆栈段的 DPL 都为 0, 具有最高特权级。

2. 保护模式存储管理

在保护模式下, 从虚拟(逻辑)地址到物理地址变换, 要经过分段和分页两级变换。第一级使用段机制的段选择符、段描述符、描述符表和段寄存器、段描述符高速缓冲寄存器, 将虚拟地址转换成线性地址。第二级使用分页机制的两级页表(页目录表和页表)和转换后援缓冲器(TLB), 把线性地址转换成物理地址。在两级地址变换过程中, 段机制是必用的, 分页机制则根据需要允许或禁止。如果禁用分页机制, 则经过段机制的线性就是物理地址。

段机制使用大小可变的块管理存储器, 有利于复杂系统的逻辑分区。它使段的大小可按需确定及段可作为独立单位处理, 使段的保护和共享得到简化。分页机制使用固定大小的块来管理内存和外存, 使虚拟存储器的实现变得方便、容易。

对于保护模式下的页式存储管理系统, 80486 固定采用 4KB 页面和 32 位物理地址。对于大系统而言, 4KB 的页面显得太小, 系统运行效率不高。Pentium 增加了一个 4MB 的大页面, 因而形成了两种页面功能结构。当使用 4MB 页功能时, 页内偏移量要占用 22 位, 因而线性地址仅分割为页目录号与偏移量两个部分, 取消了页号。图 2.4 所示的简单比较了两种情况的线性地址分割方法。

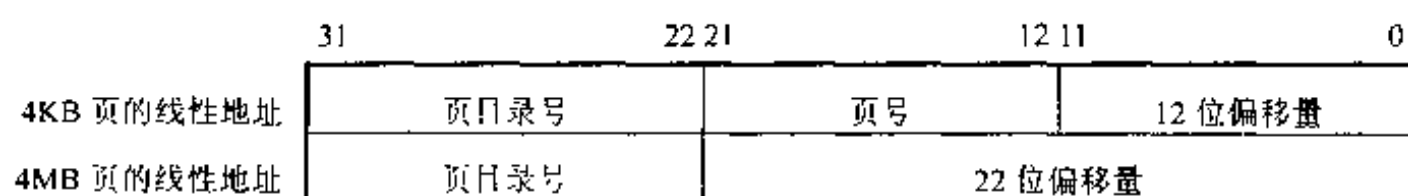


图 2.4 页式管理的线性地址分割

为了达到支持两种页面功能的目的，在页目录项的属性中增设了页面尺度选择位 (SIZ)。当 SIZ=1 时为 4MB 页，SIZ=0 时仍为 4KB 页。页目录项的结构如图 2.5 所示。

31	22	21	12	11	9	8	7	6	5	4	3	2	1	0
页表地址	0	AVAIL 可用性	0	SIZ 页尺度	D 改写性	A 访问性	PCD 页 Cache	PWT 页通写	U/S 特权级	R/W 读写性	P 页存在			
属性	1	可用		4MB	已改写	已访问	禁止	通写	用户级	读写	在内存			
	0	不可用		4KB	未改写	未访问	允许	回写	监控级	只读	不在			

图 2.5 页目录项结构

Pentium 的内部控制寄存器 CR₀、CR₂、CR₃ 和 CR₄ 与分页管理都有关。对于 CR₀，PG 位为页管理功能选择，该位为 1 表示使用分页功能，为 0 则禁止分页功能；WP 位为写保护，该位为 1 表示只读页，不容许管理员写入，为 0 则可写入。对于 CR₂，仅当出现分页管理故障时，该寄存器才会发挥作用，它将近期故障的指令的线性地址保存下来。对于 CR₃，高端 20 位为页目录表基地址的 A₃₁~A₁₂，A₁₁~A₀ 则除 D₄、D₃ 位外，其余位都为 0。PCD 位是页目录表的页级外部 Cache 禁止位，PWT 位是页目录表的页级外部 Cache 通写位。对于 CR₄，PSE 位为页尺度扩展，该位为 1 时是 4MB 页，为 0 时是 4KB 页。

保护模式下地址变换机制对虚拟存储器的支持是对驻留在内存中的虚拟存储器部分采用有效标记标志和给出使用统计信息。当处理器访问这部分虚拟空间时，便将相应虚拟地址转换为物理地址。在地址变换过程中，可以利用 TLB 替代两级页表变换，提高地址变换的处理速度。如果程序访问的是未驻留在内存的虚拟地址空间，则标记为无效状态，并引起异常。在异常处理程序中，操作系统会把未驻留内存部分从外存调入内存，并可根据需要更换地址变换表，异常处理后程序重新执行引起异常的指令。对驻留在内存中虚拟存储器部分使用统计信息，可在内存空间紧缺时，由操作系统决定并将哪些部分调回外存。处理器硬件和操作系统的这种支持可使应用者会感到有一个比实际内存大得多的存储空间可被利用。

3. 虚拟 8086 模式存储管理

在虚拟 8086 模式中，一个或多个 8086 实模式程序可执行于保护模式环境中，这是一个由处理器能力和称为虚拟机监控程序的操作系统软件组合而创建的环

境, 这样虚拟 8086 任务执行起来便感觉它是执行在 8086 上的任务。虚拟 8086 模式实行段页式存储管理, 它的段式存储管理类似于实模式存储器分段和物理地址形成, 属性中除段描述符特权级为 3 外, 其他均相同; 它的页式存储管理使用类似于保护模式的分页机制, 把多个虚拟 8086 任务的 1MB 线性地址映射到不同的物理地址。同样它也可以产生大于物理存储空间的虚拟存储空间, 构成页请求虚拟存储系统。

2.1.7 多任务

1. 任务设置

任务状态段 (TSS) 描述符和其他任何描述符一样, 包含了任务状态段的位置、大小和优先级别等属性。他们之间的区别在于 TSS 描述符所描述的 TSS 不包括数据和代码, 但包含任务状态和链接, 以使任务可以被嵌套。多任务系统与全局描述符表 GDT、局部描述符表 LDT 和中断描述符表 IDT 联系非常密切, Pentium 在执行任务前必须在存储器中定义 GDT、IDT、LDT 和 TSS, 并装载相应的寄存器 GDTR、IDTR、LDTR 和 TR, 图 2.6 形象地描述了它们之间的关系。

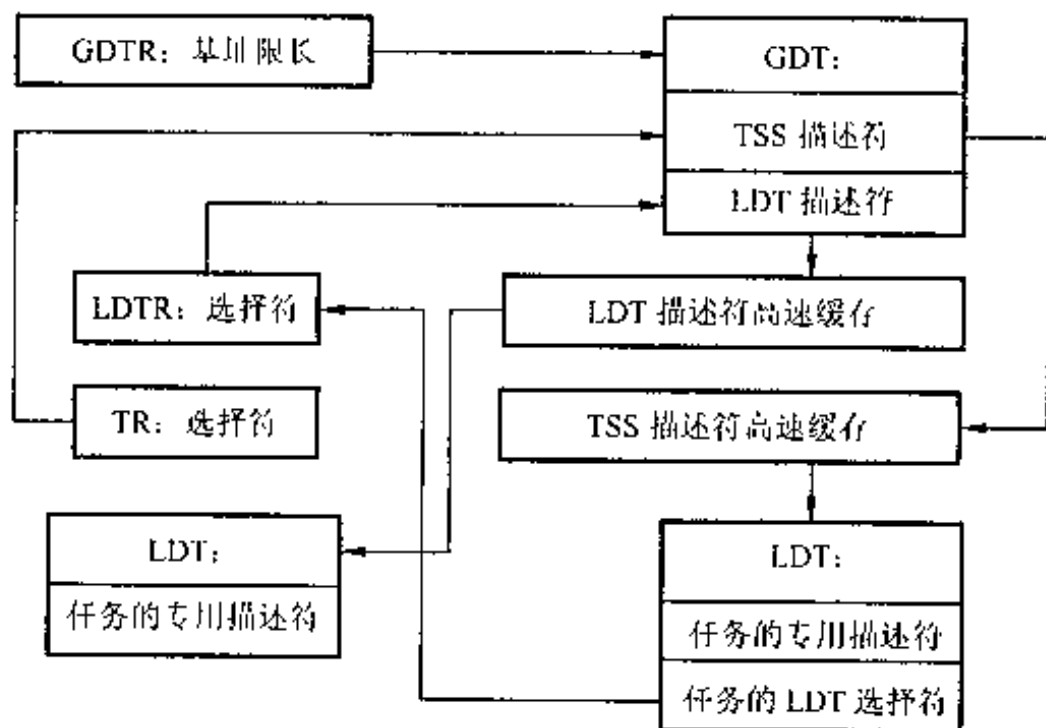


图 2.6 任务设置示意图

2. 任务转换过程

由装入 GDTR 可获得 GDT 的起始地址和限长, TSS 描述符和 LDT 描述符在 GDT 中。由段间 JMP 或 CALL 指令直接、或通过 GDT、IDT、LDT 中任务门间接选择 GDT 中 TSS 描述符; TSS 描述符通过特权级检查或由任务门通过 DPL 检查以及存在和限长检查; 在保存转出任务的状态后, 转入任务的 TSS 选择符加载到 TR, TSS 描述符加载到 TR 对应的描述符高速缓存中; 由 TSS 描述符确定转入

任务的 TSS 起始地址, 计算偏移量查获相应内容装入 CPU 寄存器, 转入任务的 LDT 选择符装入 LDTR 中, 在 GDT 中查找转入任务的 LDT 描述符, 将其加载到 LDTR 对应的描述符高速缓存中; 由高速缓存中的 LDT 描述符查找到转入任务的 LDT 中, 其中包含该任务专用的各种描述符。

任务返回时, EFLAGS 中 NT=1, 返回指令将处理器当前状态存入现行任务的 TSS 中, 将现行任务 TSS 的 LINK 字段中返回任务的 TSS 选择符装入 TR 中, 处理器可返回到任务嵌套链上的前一个任务去执行。

3. 任务转换检查

表 2.7 说明了任务转换期间所作的检查。其中, NP 表示段不存在异常, GP 为一般保护异常, TS 为无效 TSS 异常, SF 为堆栈异常。其中的第二步是要检查 TSS 描述符中的“忙”位, 如果不可采用, 则对 IRET 指令引起的异常是 TS, 对 JMP、CALL 或 INT 指令引起的异常是 GP。

表 2.7 任务转换期间所作的检查

步骤	检查的状态	可能引起的异常	错误码存在位置
1	TSS 描述符在内存中	NP	转入任务 TSS
2	TSS 描述符是可用的	TS/GP	任务返回链接的 TSS
3	TSS 段限长 ≥ 103	TS	转入任务 TSS
4	TSS 中值装入相应的寄存器		
5	转入任务的 LDT 选择符是有效的	TS	转入任务 LDT
6	代码段 DPL 与选择符 RPL 匹配	TS	转入代码段
7	堆栈段选择符是有效的	TS	转入堆栈段
8	堆栈段在内存中	SF	转入堆栈段
9	堆栈段 DPL 与 CPL 相匹配	TS	转入堆栈段
10	转入任务的 LDT 在内存中	TS	新任务 LDT
11	代码段选择符是有效的	TS	转入代码段
12	代码段在内存中	NP	转入代码段
13	堆栈段 DPL 与选择符 RPL 匹配	TS	转入堆栈段
14	DS、ES、FS、GS 段选择符是有效的	TS	转入数据段
15	DS、ES、FS、GS 段是可读的	TS	转入数据段
16	DS、ES、FS、GS 段在内存中	NP	转入数据段
17	DS、ES、FS、GS 段 DPL $\geq$ CPL (除非是证实代码段)	TS	转入数据段

2.1.8 高速缓存Cache

CPU 频率的提高, 必然要求系统中存储器的存取速度提高和容量增大。内存存储器 DRAM 容量提高比较快, 但读取时间的提高很慢, 从而在速度上与 CPU 主

频的提高极不相配,这样会影响整个系统的性能。20年来,CPU设计的问题之一就是解决高速CPU和低速DRAM之间的平衡或匹配问题,以求系统性能的整体提高。在它们之间加入Cache是解决方案之一。Cache位于CPU和存储器之间,它的容量比存储器小但存取速度快,其内容为存储器的部分拷贝。在程序运行过程中需要取指令或取数据时,先检查Cache中是否有此内容,若有就从Cache中取出,若没有再从存储器取出。

80486时代,由于CPU的时钟频率不断增加,外部Cache的SRAM芯片速度也要相应提高,这样会增加系统成本。为此在设计80486时采用了内部8KB的Cache存放指令和数据。同时,80486也可以使用处理器外部的第二级Cache,用以改善系统性能并降低总线带宽。

到Pentium时,分别设立了各8KB的指令Cache和数据Cache,L2Cache容量为256KB。英特尔公司推出的PentiumMMX将L1Cache容量增加到32KB,L2Cache容量增加到512KB。PentiumII同样有2级Cache,L1Cache为16KB,L2Cache为512KB。PentiumIII具有32KB L1Cache和512KB L2Cache,L2Cache可扩充到1~2MB。可以看出,Cache的出现是对CPU的高速度与内存的相对低速度的一种折衷,而这种折衷对提高计算机系统的性能起了至关重要的作用。

1. 80486的Cache

80486的内部Cache是一个4路组相连存储器,在内存存储器中给定单元的数据能够存储在Cache内4个单元的任何一一个之中。这种4路相连方式是高命中率的全相连Cache和快速的直接映像Cache的一种折衷,因而能进行快速查找并获得较高命中率。80486的外部Cache可以工作在80486所有的工作模式:实地址模式、保护模式和虚拟8086模式。对Cache的操作是由系统自动进行的,对程序员透明。而在多处理器系统中,可能要求系统软件的干预。对于一般的计算机,在系统CMOS设置中均有Cache使用模式的设置。

2. Pentium的高速缓存

为提高计算机操作的并行度及简化设计,PentiumCPU将存放指令的Cache和存放数据的Cache分开,分别称为指令Cache(I-Cache)和数据Cache(D-Cache)。这两个Cache可以同时被访问。指令和数据Cache独立的结构减少了争用Cache所造成的冲突,提高了处理器效能。Pentium的Cache采用回写与通写相结合的工作方式。回写方式可以减少总线上的写操作,通写方式容易保证Cache一致性。Pentium灵活地使用这两种方式。

这两个Cache均为双路组相连结构,每一路为4KB,每个Cache行为32B。Cache控制器将CPU的32位地址分割成标签、索引与字节选择三个域。标签占有地址 $A_{31} \sim A_{12}$ 共20位,索引占用地址 $A_{11} \sim A_5$ 共7位,字节选择占用地址最低的 $A_4 \sim A_0$ 共5位。一个完整的Cache记录由Cache目录记录与相应的Cache存储

器记录组成,前者指定数据存放的标签位置,即标签和有关属性;后者才是标签访问的相对应的数据。7位索引地址($A_{11} \sim A_5$)确定了128组可能的Cache行。5位的字节($A_4 \sim A_0$)选择可确定Cache行中32B之一。此外采用了一种称为MESI高速缓存一致性协议,为确保多处理器环境下的数据一致性提供了保证。Pentium数据Cache的目录记录中用2位放置MESI,指令Cache只需1位放置SI两个状态。两路合在一起共用一个表明最近最少使用的LRU位。Cache存储器记录对应的Cache行是32B宽,也就是4个64位字,它们对应一次突发模式的4个传输周期。图2.7所示的是Pentium的Cache结构。

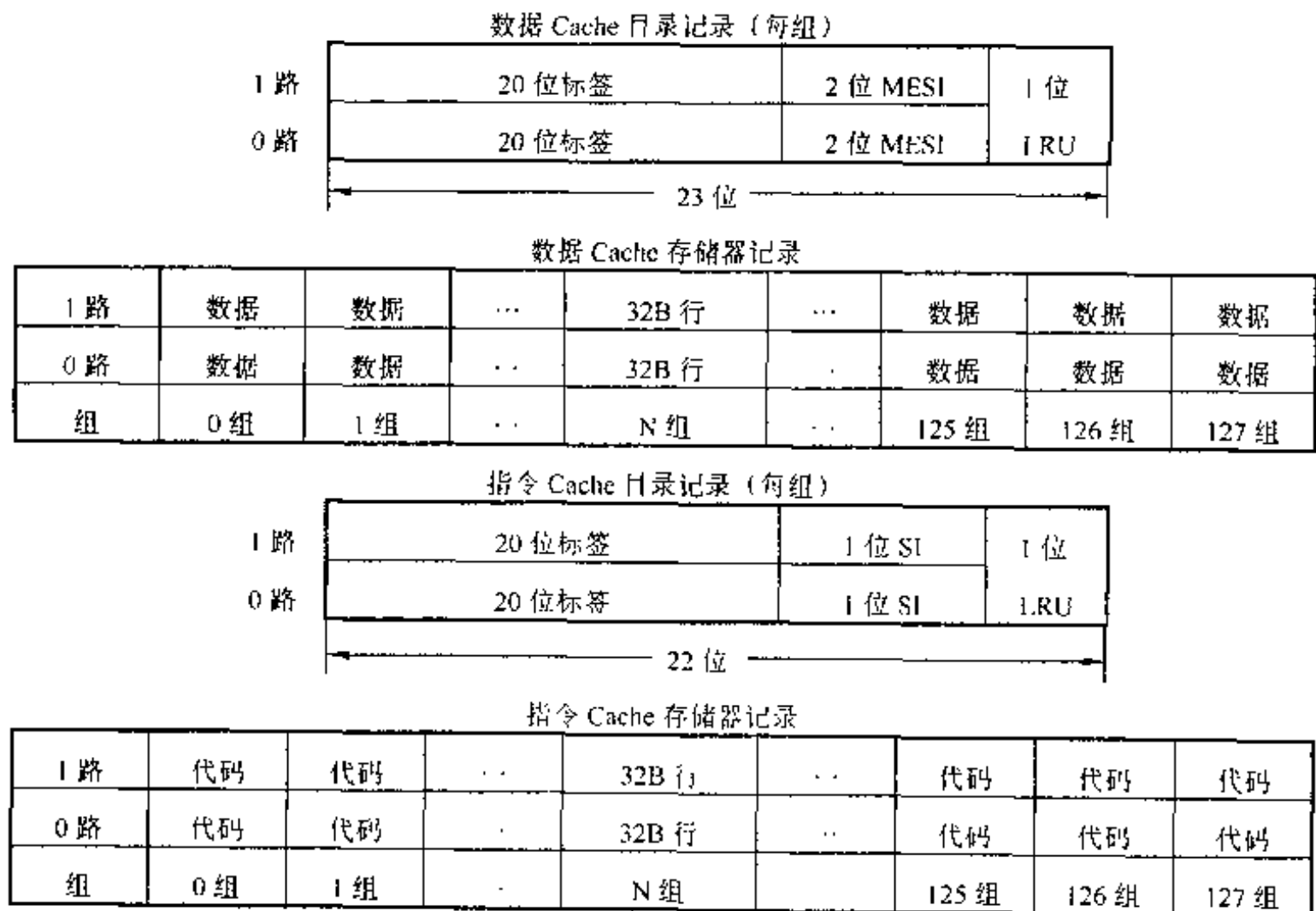


图2.7 Pentium的Cache结构

3. Pentium MMX 的 Cache

Pentium MMX 的片内 L1 数据和指令的 Cache, 每个增加到 16KB, 4 路相连。较大的独立的内部 Cache, 减少了平均内存存取时间, 同时提供了对近期所用指令和数据的快速存取, 性能因此得到提高。数据 Cache 支持采用回写方式更新内存。

4. Pentium II 的 Cache

Pentium II 的 L2 Cache 不在内嵌芯片上, 而是与 CPU 通过专用 64 位 Cache 总线相连, 与其他元器件一起被组装在同一基板上, 即“单边接触盒”上。双重

独立总线结构是指处理器至 L2 Cache 的 Cache 总线和处理器至内存的系统总线。Pentium II 处理器可以同时使用这两条总线,与单一总线结构的处理器相比,该处理器可以处理两倍多的数据,Pentium II 的 L2 Cache 比 Pentium 的 L2 Cache 要快 1 倍。随着 Pentium II 主频的提高,L2 Cache 的速度也将加快,并且流水线型系统总线可允许多个数据同时并行传输,而不是单个顺序传输。改进型的双重独立总线结构,可以产生超过单总线结构三倍的带宽。另外,在 Pentium II 中采用了 ECC 技术,此技术应用到二级 Cache 中大大提高了数据的完整性和可靠性。

5. Pentium III 的 Cache

Pentium III 具有 32KB 的 L1 Cache 和 512KB 的 L2 Cache。L2 可扩充到 1~2MB,具有更合理的内存管理,可以有效地对大于 L2 Cache 的数据块进行处理,使 CPU、Cache 和内存存取更趋合理,提高了系统整体性能。在执行视频回放和访问大型数据库时,高效率的 Cache 管理使 Pentium III 避免了对 L2 Cache 的不必要的存取。由于消除了缓冲失败,多媒体及其他对时间敏感的操作的性能得到提高。对于可缓存的内容,Pentium III 通过预先读取期望的数据到高速缓存里来提高速度,这一特色提高了 Cache 的命中率,减少了存取时间。

为进一步发挥 Cache 的作用,改进内存性能并使之与 CPU 发展同步来维护系统平衡,英特尔公司增加了缓存控制指令。英特尔在 Pentium III 处理器中增加了 70 条 3D 及多媒体的 SSE 指令集,其中很重要的一组指令是缓存控制指令。Pentium III 处理器的缓存控制指令用于优化内存连续数据流处理。针对数据流的应用需要对以前的 Cache 运作方式进行改进,减少一些不必要的中间环节,节省时间,增加 CPU 数据总线的实际可用带宽,提高了 Cache 的效率。采用 Pentium III 处理器的内存流优化处理指令后,应用程序就能让数据流直接到达内存。当数据流直接到达内存时,处理器负责维护 Cache 的一致性。这种方式避免了为数据流留出空间而清空 Cache 的当前内容,从而提高了 Cache 的利用率。Cache 控制指令也改进了进出处理器的数据流,使处理器保持其高速率运作。通过这些指令(同时还需要一些专为其设计以使其发挥优势的软件),商业用户可以在操作系统和图形设备驱动程序中感受其性能优势。

有两类缓存控制指令。一类是数据预存取(Prefetch)指令,能够增加从内存到 Cache 的数据流;另一类是内存流优化处理(Memory Streaming)指令,能够增加从处理器到内存的数据流。这两类指令都赋予了应用开发人员对缓存内容更大控制能力,使他们能够控制 Cache 操作以满足其应用的需求。

数据预存取指令允许应用识别出所需的信息,并预先将其从内存中取出存入 Cache。这样一来,处理器可以更快地获取信息,从而改进应用性能。为了进一步削减内存延迟,内存访问还可以与计算机周期保持流水操作。内存流优化处理指令允许应用越过 Cache 直接访问内存。通常情况下,处理器写出的数据都将暂

时存储在缓存中以备处理器稍后使用。如果处理器不再使用它,数据最终将被移至内存。然而,对于多媒体应用来说,通常不再需要使用这些数据。因此,这时将数据尽快地移到内存中则显得至关重要。Pentium III 芯片有数个 Prefetch 指令,不只是从 L1 Cache 中取得缓存数据,还可以从所有 Cache 中选择缓存数据,所以速度很快。

6. Pentium 4 的高速缓存

Pentium 4 配备了存取时间低于 1.42ns 的 8KB 的 L1 Cache,拥有极低的寻找时间,能迅速地找到并且命中目标指令,大大提高了 CPU 的工作效率。Pentium 4 使用了执行跟踪 Cache 和数据 Cache 相结合的技术,更进一步地提升处理器的效率。借助于执行跟踪 Cache 可以从主执行循环中消除译码器的译码延迟。另外,对于一个 1.5GHz 的 Pentium 4 来说,256KB 二级 Cache 可以 48GB/s 的速度向处理器传输指令和数据,并且还将随着处理器主频的提高而提高。需要说明的是动态跟踪 Cache 与传统的 L1 Cache、L2 Cache 不同,它直接和执行单元和动态跟踪引擎相连,通过动态跟踪引擎可以很快地找到所执行的指令,并且将指令的顺序存储在跟踪 Cache 里,减少了主执行循环的解码周期,提高了处理器的运算效率,据称可以提高 33% 的运算效率。执行跟踪 Cache 的大小约为 4KB,位于指令译码器和处理器内核第一级执行流水线之间。指令在译码单元内译码之后,微操作首先经过执行跟踪 Cache 存储和输出才能到达内核第一级执行流水线。执行跟踪 Cache 可以存储 1200 条以上微操作,处理器在出现分支预测错误后可以从跟踪 Cache 中重新获得所需的微操作,降低了分支预测错误对处理器性能的影响。

2.1.9 Pentium 的引脚及功能

1. 引脚及功能

Pentium 处理器引脚及功能如表 2.8 所示。

表 2.8 Pentium 引脚的功能

功 能	引脚及其输入 (IN) 输出 (OUT) 方向
时钟	IN: CLK
初始化	IN: RESET, INIT
地址总线	I/O: A ₃₁ ~A ₁ ; OUT: $\overline{BE}_7 \sim \overline{BE}_0$
地址屏蔽	IN: $\overline{A20M}$
地址奇偶	I/O: AP; OUT: $\overline{APCHK}$
数据总线	I/O: D ₆₃ ~D ₀
数据奇偶	I/O: DP ₇ ~DP ₀ ; IN: $\overline{PEN}$; OUT: $\overline{PCHK}$
内部奇偶错	OUT: $\overline{IERR}$
系统错	IN: $\overline{BUSCHK}$

续表

功 能	引脚及其输入 (IN) 输出 (OUT) 方向
浮点错报告	OUT: $\overline{\text{FERR}}$; IN: $\overline{\text{IGNNE}}$
总线周期定义	OUT: $\text{M}/\overline{\text{IO}}$, $\text{D}/\overline{\text{C}}$, $\text{W}/\overline{\text{R}}$, $\overline{\text{CACHE}}$, SCYC , $\overline{\text{LOCK}}$
总线控制	OUT: $\overline{\text{ADS}}$; IN: $\overline{\text{BRDY}}$, $\overline{\text{NA}}$
总线仲裁	IN: $\overline{\text{BOFF}}$, HOLD ; OUT: BREQ , HLDA
中断	IN: INTR , NMI
页 Cache	OUT: PCD , PWT
Cache 控制	IN: $\overline{\text{KEN}}$, $\text{WB}/\overline{\text{W1}}$
Cache 窥探/一致性	IN: AHOLD , $\overline{\text{EADS}}$, INV ; OUT: $\overline{\text{HIT}}$, $\overline{\text{HITM}}$
Cache 清洗	IN: $\overline{\text{FLUSH}}$
写序	IN: $\overline{\text{EWEB}}$
系统管理模式	IN: $\overline{\text{SMI}}$; OUT: $\overline{\text{SMIACT}}$
探针模式	IN: $\text{R}/\overline{\text{S}}$; OUT: $\overline{\text{PRDY}}$
功能冗余检测	IN: $\overline{\text{FRCMC}}$ ($\overline{\text{IERR}}$)
测试访问口	IN: TCK , TMS , TDI , $\overline{\text{TRST}}$; OUT: TDO
断点/性能监控	OUT: PM_0/BP_0 , PM_1/BP_1 , $\text{BP}_2 \sim \text{BP}_7$
执行跟踪	OUT: $\text{BT}_3 \sim \text{BT}_0$, IU , IV , IBT

2. 引脚输出的三态特性

所有输出和 I/O 双向引脚在三态测试模式和校验器模式 ($\overline{\text{IERR}}$ 除外) 都是浮置的。当总线保持或 $\overline{\text{BOFF}}$ 时, 浮置的输出引脚有: $\overline{\text{ADS}}$ 、 $\overline{\text{BE}}_7 \sim \overline{\text{BE}}_0$, $\overline{\text{CACHE}}$ 、 $\overline{\text{LOCK}}$ 、 $\text{M}/\overline{\text{IO}}$ 、 $\text{D}/\overline{\text{C}}$ 、 $\text{W}/\overline{\text{R}}$ 、 PCD 、 PWD 、 SCYC ; 浮置的双向引脚有: $\text{A}_{31} \sim \text{A}_3$ 、 AP 、 $\text{D}_{63} \sim \text{D}_0$ 、 $\text{DP}_7 \sim \text{DP}_0$ 。 $\text{A}_{31} \sim \text{A}_3$ 和 AP 在地址保持时也是浮置的。

2.1.10 总线周期

Pentium 处理器具有良好的总线接口, 支持多种数据传送总线周期。与 80486 类似, Pentium 有单数据传送和突发式数据传送总线周期, 有非缓存式和缓存式传送总线周期。与 80386 类似, Pentium 有非流水线式和流水线式总线周期。Pentium 处理器初始的总线周期如表 2.9 所示, Pentium 处理器特殊总线周期译码如表 2.10 所示。

1. 非流水线式读/写总线周期

无论芯片外部存储器系统的读/写速度如何, Pentium 都是执行相同的时钟周期。唯一的差别在于若存储器系统比较慢, 则 $\overline{\text{BRDY}}$ 信号有效时间较晚一点, 因而 Pentium 要插入若干个等待状态。

表 2.9 Pentium 处理器初始的总线周期

$M/\overline{IO}$	$D/\overline{C}$	$W/\overline{R}$	$\overline{CACHE}^*$	$\overline{KEN}$	总线周期描述	传送次数
0	0	0	1	×	中断响应 (两个锁定的周期)	每周期 1
0	0	1	1	×	特殊总线周期 (见表 2.9)	1
0	1	0	1	×	I/O 读, 不超过 32 位, 非高速缓存	1
0	1	1	1	×	I/O 写, 不超过 32 位, 非高速缓存	1
1	0	0	1	×	代码读, 64 位, 非高速缓存	1
1	0	0	×	1	代码读, 64 位, 非高速缓存	1
1	0	0	0	0	代码读, 256 位突发式行填充	4
1	0	1	×	×	Intel 保留	
1	1	0	1	×	存储器读, 不超过 64 位, 非高速缓存	1
1	1	0	×	1	存储器读, 不超过 64 位, 非高速缓存	1
1	1	0	0	0	存储器读, 256 位突发式行填充	4
1	1	1	1	×	存储器写, 不超过 64 位, 非高速缓存	1
1	1	1	0	×	256 位突发式回写	4

*: 在 $M/\overline{IO}$ 为低电平或 $\overline{PCD}$ 为高电平时, $\overline{CACHE}$ 信号不会有效。

表 2.10 Pentium 处理器特殊总线周期

$\overline{BE}_7$	$\overline{BE}_6$	$\overline{BE}_5$	$\overline{BE}_4$	$\overline{BE}_3$	$\overline{BE}_2$	$\overline{BE}_1$	$\overline{BE}_0$	特殊总线周期
1	1	1	1	1	1	1	0	关机
1	1	1	1	1	1	0	1	清除 (INVD、WBIVD 指令)
1	1	1	1	1	0	1	1	暂停
1	1	1	1	0	1	1	1	回写 (WBINVD 指令)
1	1	1	0	1	1	1	1	清除响应 ($\overline{FLUSH} = 0$)
1	1	0	1	1	1	1	1	分支跟踪消息

Pentium 的非流水线式读/写周期在第一个时钟周期 T_1 期间, Pentium 发送地址信号 $A_{31} \sim A_3$, $\overline{BEX}$ 信号则由访问的数据宽度确定, $\overline{ADS}$ 信号为低电平说明地址与总线控制信号有效。在读周期中 $W/\overline{R}$ 保持低电平, 而在写周期中则呈现高电平。依据存储器子系统的速度, 过一段时间子系统就会输出访问到的数据并激活 $\overline{BRDY}$ 信号, 并且还输出奇偶校验位信号。如果数据与奇偶位是立即有效的, 即没有等待状态, 则 $\overline{BRDY}$ 在 T_2 周期结束前就由存储器子系统置为有效。如果存储器或外部设备在 T_1 和 T_2 两个时钟周期内还不能完成一次读/写请求, 那么存储器控制器就会将 $\overline{BRDY}$ 信号保持为高电平, 并且向 Pentium 请求增加一个 T_2 时钟周期, 这就是等待状态。如果在附加的 T_2 时钟周期后, $\overline{BRDY}$ 仍为高电平, 处理器会继续插入一个等待状态, 如此下去。写数据和读数据的等待状态一般是不一样的, 内存储器写数据比读数据更快。其原因在于写操作只是传给存储器控制器地

址与数据字节信息,然后存储器控制器就能独立执行写操作,Pentium 不用等待这个过程,可以执行其他操作。对于读操作而言,CPU 只能等待内存存储器送出数据并完成读过程,在此期间不能执行其他操作。

2. 突发式读/写总线周期

对于大量数据的传输,Pentium 处理器执行突发式操作。在这种模式下,传输数据的时间减小到只用一个时钟周期,传输率则提高了一倍。这种模式是直接面向 Cache 的 Cache 行填充与回写操作。在这种模式中, $\overline{\text{BRDY}}$ 、 $\overline{\text{CACHE}}$ 、 $\overline{\text{KEN}}$ 信号扮演了重要的角色。当读周期开始时,Pentium 使用低电平有效的 $\overline{\text{CACHE}}$ 信号通知存储器子系统寻址片内 Cache,执行 Cache 操作,如果存储器子系统发出 $\overline{\text{KEN}}$ 低电平有效信号,Pentium 就会进入 Cache 行填充操作。由于传输的长度固定,在接受到起始地址后,存储器子系统就能够独自计算出后面的 3 个突发模式地址,Pentium 就只需在第一个周期内发出起始地址和 $\overline{\text{BEX}}$ 信号,且在后面的 3 个周期内保持不变。

突发写周期操作基本与突发读周期操作类似,需要注意的是, $\overline{\text{KEN}}$ 信号一直是无效的,因为片内 Cache 的数据将被更新且不会出现重新填充的现象。在数据 Cache 中对于一行的修改,突发写周期总是伴随着一个回写周期。对于代码 Cache,则没有回写现象,因为代码数据是不允许覆盖写入的。所以,突发写周期总是针对数据 Cache 的,而突发读周期则对二者都有效。

3. 流水线式读/写总线周期

当存储器的访问是一个一个地顺序执行时,Pentium 通过 $\overline{\text{NE}}$ 信号形成流水线式访问,在流水线式总线周期中,下一个总线周期地址在前一个总线周期的数据传送时产生,由于各总线周期部分重叠,所以处理器总体执行速度加快了。Pentium 处理器的单数据传送和突发式数据传送总线周期均可以是流水线式的。突发式读总线周期是高速缓存式的,而高速缓存式读总线周期也是突发式的。

4. Intel 80486 和 Pentium 处理器总线的区别

① 不同于 80486 的 32 位数据总线,Pentium 有 64 位数据总线 $\text{D}_{63} \sim \text{D}_0$ 、8 位数据奇偶线 $\text{DP}_7 \sim \text{DP}_0$ 和 8 位字节允许线 $\overline{\text{BE}}_7 \sim \overline{\text{BE}}_0$ 。

② Pentium 用 $\overline{\text{NA}}$ 引脚支持地址流水线提供同时执行 2 个总线周期的能力。

③ Pentium 在 $\overline{\text{NA}}$ 信号或第一个 $\overline{\text{BRDY}}$ 信号最先有效时采样一次 $\overline{\text{KEN}}$ 信号确定是否允许高速缓存,而 80486 在 Cache 行填充周期则要在第一个和最后一个 $\overline{\text{RDY}}$ 或 $\overline{\text{BRDY}}$ 有效前各采样一次 $\overline{\text{KEN}}$ 。

④ Pentium 通过 $\overline{\text{CACHE}}$ 和地址一起控制突发传数长度,80486 是用 $\overline{\text{BLAST}}$ 控制突发数据传送长度。

⑤ Pentium 处理器 1 个总线周期就可进行 8B 写操作,因而不需要 $\overline{\text{PLOCK}}$ 信

号引脚。

⑥ Pentium 处理器在突发总线周期不改变低位地址和字节允许信号。

⑦ Pentium 的请求回写和行填充作为突发周期运行，突发传送不能像 80486 那样利用 $\overline{\text{RDY}}$ 和 $\overline{\text{BLAST}}$ 信号来中断。

⑧ Pentium 不支持非缓存式突发周期和非突发式缓存周期。Pentium 的可缓存就意味着可突发。

⑨ Pentium 用新的引脚 $\overline{\text{CACHE}}$ 、 $\overline{\text{HIT}}$ 、 $\overline{\text{HITM}}$ 、 $\overline{\text{INV}}$ 和 $\overline{\text{WB/WT}}$ 支持回写 Cache 协议。

⑩ Pentium 不用 80486 的 $\overline{\text{BS}}_8$ 和 $\overline{\text{BS}}_{16}$ 来支持动态总线宽度。

⑪ Pentium 不存储由 $\overline{\text{BOFF}}$ 中断的总线周期中返回的数据，重启周期从头开始；80486 存储 $\overline{\text{BOFF}}$ 有效前返回的数据，重启周期从中断点开始。

⑫ Pentium 处理器在相邻的锁定周期之间产生一个空闲的时钟。

⑬ Pentium 处理器用 SCYS 引脚指示锁定操作期间的分割 (Split) 周期。

⑭ Pentium 处理器非缓存的指令是预取 8B，80486 是预取 16B。

⑮ Pentium 处理器用 INIT 引脚在维持内部 Cache 和浮点机器状态不变情况下执行系统复位的功能。

⑯ Pentium 处理器用 $\overline{\text{EWBE}}$ 引脚支持处理器与外部系统之间的强存储定序。

⑰ Pentium 处理器新的引脚 $\overline{\text{APCHK}}$ 、 $\overline{\text{BUSCHK}}$ 、 $\overline{\text{PEN}}$ 、 $\overline{\text{IERR}}$ 和 AP 支持内部奇偶错检测、数据奇偶错检测和地址奇偶错检测。

⑱ Pentium 处理器用 TDI、TDO、TMS、 $\overline{\text{TRST}}$ 和 TCK 引脚支持边界扫描。

⑲ Pentium 处理器用 IU、IV 和 IBT 引脚和分支跟踪消息特殊周期支持执行跟踪。

⑳ Pentium 处理器用 $\overline{\text{FRCMC}}$ 、 $\overline{\text{IERR}}$ 支持功能冗余检测，用 BP_3 、 BP_2 、 PM_1/BP_1 和 PM_0/BP_0 支持断点匹配和性能监控，用 $\overline{\text{SMI}}$ 中断输入和 $\overline{\text{SMIACT}}$ 响应输出实现系统管理模式。

㉑ Pentium 的 $\overline{\text{FLUSH}}$ 是边沿触发的输入，只在其每个下沿边识别一次，它是在指令边界作为中断实现的。

2.2 例题精析

例 2.1 程序在数据段中定义的数组如下：

```
NAMES DB 'TOM.',  
        DB 20  
        DB 'ROSE.'
```

DB 30

DB 'KATE.'

DB 25

请指出下列指令是否正确？为什么？

(1) MOV BX, OFFSET NAMES

MOV AL, [BX+5]

(2) MOV AX, NAMES

(3) MOV AX, WORD PTR NAMES + 1

(4) MOV BX, 6

MOV SI, 5

MOV AX, NAMES[BX][SI]

(5) MOV BX, 6*2

MOV SI, 5

MOV AX, OFFSET NAMES[BX][SI]

INC [AX]

(6) MOV BX, 6

MOV SI, 5

LEA DI, NAMES[BX][SI]

MOV AL, [DI]

解 (1) 两条指令都是合法指令， $(AL) = 20$ 。

第一条指令执行 $(BX) = \text{NAMES}$ 的偏移地址：

第二条 MOV 指令执行源操作数有效地址 $EA = (BX) + 5$ ；

所以， $(AL) = (\text{NAMES} + 5) = 20$ 。

(2) 指令不合法。变量 NAMES 的类型为字节，而目的寄存器是 AX，所以源操作数与目的操作数类型不一致。

(3) 指令正确。此指令使用属性运算符将源操作数 $(\text{NAMES} + 1)$ 单元的类型强制为字类型，与目的操作数类型一致。

指令执行源操作数的 $EA = \text{NAMES}$ 的偏移地址 + 1，而 $(\text{NAMES} + 1) = 'O' = 4FH$ $(\text{NAMES} + 2) = 'M' = 4DH$ ，所以 $(AX) = 4D4FH$ 。

(4) 前两条正确；第三条错误，源操作数与目的操作数类型不一致。第三条指令中基址加变址加位移形成的源操作数类型属性为字节，与目的操作数 AX 类型不一致。

(5) 前两条指令正确，后两条错误。OFFSET 运算符只能作用于直接的存储变量；在 16 位寻址中，AX 不能作为间接寻址寄存器。

(6) 均为合法指令。 $(AL) = 30$ 。依题意， $(BX) = 6$ ， $(SI) = 5$ ， $(DI) =$

$(BX) + (SI) + \text{OFFSET NAMES} = \text{OFFSET NAMES} + 11$, 所以 $(AL) = (\text{NAMES} + 11) = 30$ 。

小结 判断指令合法性, 可以从如下几个方面考虑:

(1) 在双操作数指令中, 看源操作数和目的操作数的地址是否同时为存储器地址, 除串操作指令外, 若是则非法。

(2) 看操作数和目的操作数的类型是否一致, 除少数指令外 (如移位、扩展传送指令), 大部分指令要求一致。

(3) 段超越前缀可修改操作数所在的段。应注意: 段超越前缀不能使用 CS, 串操作指令的目的操作数所在段 (ES) 不能超越。

(4) 移位指令使用立即数作为源操作数时, 8086 指令只能是 1, 而 80486 指令可为 1~31, 若大于 31, 则只使用后 5 位移位计数。

(5) 代码段寄存器 CS 不能作指令的目的寄存器。段寄存器间不能直接传送, 也不能直接送入立即数。

(6) 源操作数或目的操作数使用存储器寻址时, 看寻址用的基址、变址寄存器使用是否正确。

(7) 堆栈指令的操作数只能为字或双字。

(8) 注意标号与变量的区别: 标号可通过跳转、调用指令引用, 但变量不能; 标号不能用作存储器基址/变址寻址的位移量, 但变量可以。

(9) I/O 指令使用直接寻址的寻址范围为 0~255; 使用间接寻址时, 间址寄存器只能为 DX。I/O 端口地址大于 255 时, 只能使用 DX 间接寻址。

(10) OFFSET 运算符只能作用于直接的存储器变量, 若作用于用作基址/变址寻址位移量, 则出错。

例 2.2 已知在指令执行前, 有 $DI = 0300H$, $BX = 0010H$, $CF = 0$, $DS = 2000H$, $SS = 3000H$, $BP = 0100H$, $SP = 0020H$, $[DS: 310H] = 0AAH$, $[DS: 311H] = 55H$ 。

(1) 执行指令 $RCL \text{ WORD PTR } [BX + DI], 1$ 后, 目的操作数值为_____。

(2) 指令 $MOV AL, [BP + 100H]$ 的源操作数物理地址为_____。

(3) 指令 $POP 300H[DI]$ 的源操作数物理地址为_____。

解 (1) $2AD5H$ 。目的操作数 $EA = (BX) + (DI) = 0300H + 0010H = 0310H$, BX 、 DI 约定的段均为 DS 。所以, 此指令功能是将 $[DS: 310H]$ 开始的字数据带进位循环右移一位。移位前目的操作数为 $55AAH$, 移位后为 $2AD5H$ 。

(2) $30200H$ 。源操作数 $EA = (BP) + 100H = 0100H + 100H = 0200H$, 基址寄存器 BP 约定的段为 SS 。所以, $PA = (SS) \times 16 + EA = 3000H \times 16 + 0200H = 30200H$ 。

(3) $30020H$ 。此指令源操作数为栈顶字数据, 所以: 源操作数 $EA = (SP) = 0020H$, $PA = (SS) \times 16 + EA = 3000H \times 16 + 0020H = 30020H$ 。

小结 80X86CPU 在实模式或虚拟模式下, 计算用存储器寻址的源操作数或目的操作数的物理地址时, 关键是

(1) 搞清指令使用的寻址方式并根据寻址方式计算操作数的有效地址 (偏移地址)。

(2) 若操作数缺省段超越前缀时, 要搞清寻址方式约定的段寄存器, 并注意寻址过程中同时使用基址和变址寄存器且二者约定的段冲突时, 使用基址寄存器约定的段寄存器。

(3) 物理地址的计算方法:

$$PA = \text{段寄存器} \times 16 + EA$$

例 2.3 说明如果段的基地址为 10000000H, 界限为 001FFH, G 位为 0 和 1 时, 段的起始地址和结束地址。

解 G=0 时, 起始地址为

$$10000000H$$

结束地址为

$$10000000H + 001FFH = 100001FFH$$

G=1 时, 起始地址为

$$10000000H$$

结束地址为

$$10000000H + 001FFFFFFH = 101FFFFFFH$$

分析 32 位微处理器描述符中用到了一个以前没有的特征位: G 位, 即粒度位。如果 G=0, 说明段的界限为 00000H 到 FFFFFH。如果 G=1, 界限值要被 4KB 乘 (在尾部添上 FFFH)。界限为 00000000H 到 FFFFFFFFH。

例 2.4 假设段管理部件已获得 32 位线性地址为 04834056H, 控制寄存器 CR₃ 中保存的页目录表基地址的高 20 位为 00005, 试分析将线性地址转换为物理地址的寻址全过程。

解 (1) 将线性地址 04834056H 转换成 32 位二进制数, 并按图 2.8 所示的结构分割成“页目录号—页号—偏移量”3 段。

(2) 将目录表基地址的低 12 位补 0, 从而获得页目录表基地址为 00005000H。线性地址 31~22 位是页目录索引值, 现为 12H, 先将其乘以 4 得 48H 再与页目录基地址相加得到 00005048H, 即在页目录中找到当前的页目录项, 从该页目录项中获得当前页表的首地址为 0000B000H。

(3) 线性地址 21~12 是页表索引值, 现为 34H, 先将其乘以 4 得 0D0H 再与页表基地址相加得到 0000B0D0H, 即在页表中找到当前的页表项, 从该页表项中获得当前页帧的基地址为 03000000H。

(4) 线性地址 11~0 是偏移量, 现为 056H, 将其与页基地址相加, 所得

03000056H 即为当前寻址的目标存储器的物理寻址。

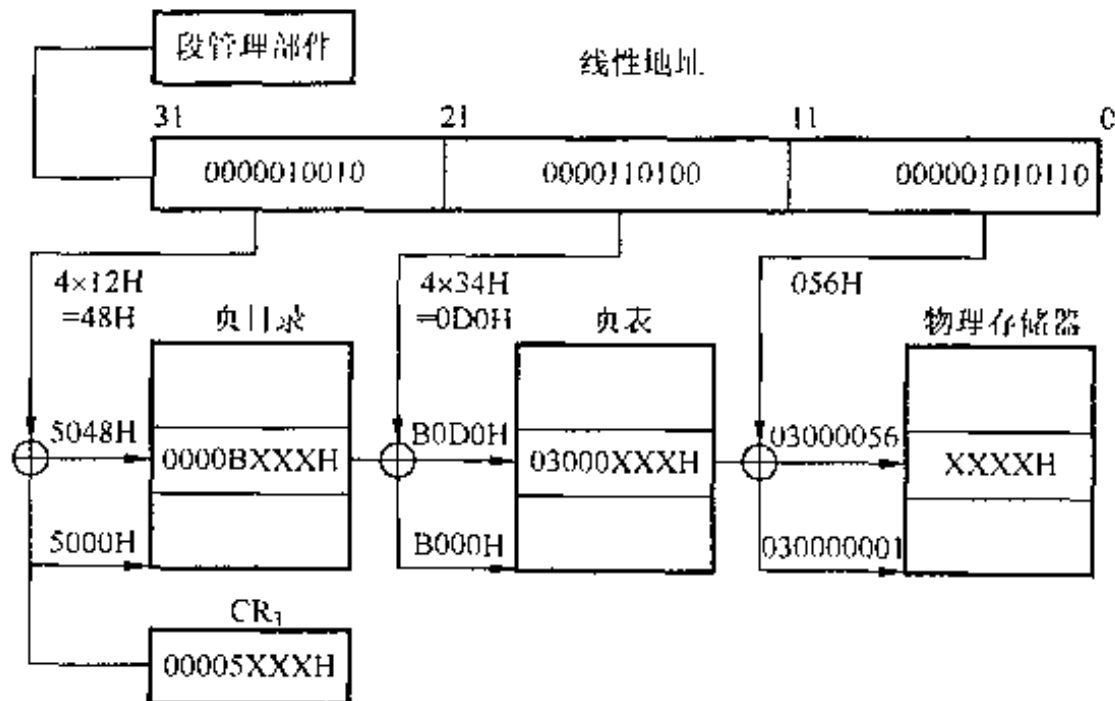


图 2.8 例 2.4 图

例 2.5 将选择符 0224H 装入段寄存器，该值指向局部描述符表中从地址 00100220H 开始的段描述符。如果该描述符的字为：(00100220H) = 0110H、(00100222H) = 0000H、(00100224H) = 1A20H、(00100226H) = 0001H，那么段的基址和限长以及其他属性将怎样变化？描述符规定的存储器段是否已装入物理存储器？是代码段还是数据段？如果 EIP 的当前值为 00000226H，那么从代码段取回的下一条指令的物理地址是多少？

解 首先了解图 2.9 所示内容。由图及题意可知，界限的 0~15 位为 0110H，基址前 15 位全为 0，16~23 位为 02H，所以段的基址为 00200000H，限长为 10110H，访问权限为 1AH，将其分为 8B，可以得到 P=0，DPL=0，S=1，E=1，所以存储器段没有装入物理存储器，为代码段。如果 EIP 的当前值为 00000226H，则依据 CS: EIP 的组合定位微处理器执行的下一条指令，所以下一条指令的物理地址为 002000226H。

基地址 (31~24)							G	D	O	AVL	界限 (19~16)	字节 6
P	DPL	S	E	ED/C	W/R	A	基地址 (23~16)					字节 4
基地址 (15~0)												字节 2
界限 (15~0)												字节 0

图 2.9 例 2.5 图

例 2.6 某中断程序入口地址为 23456H，放置在中断向量表中的位置为 00020H，问此中断向量号为多少？入口地址在向量表中如何放置？

解 向量号为 08H；入口地址在 00020H 开始的 4 个单元中的存放顺序是 56H，

04H, 00H, 23H。

分析 当 32 位微处理器工作在实模式下时, 和 8086 一样, 其中断向量表页将存放在内存的最低 1KB 中, 因此中断向量地址与中断类型号之间的关系是: 中断向量表地址 = 中断类型号 $\times$ 4。每个中断向量由 4B 组成, 高二位字节存放在 CS 中, 低二位字节存放在 IP (EIP) 中。

对此题, 有中断向量号 = 中断向量地址 $\div$ 4 = 00020H $\div$ 4 = 08H; 又 23456H = 2300H $\times$ 16 + 0456H。此题中断程序入口地址 23456H 对应的二维地址是不唯一的, 所以答案也不是唯一的, 假定划分为二维地址 2300H: 0456H, 则为该答案给出的存放结果。

例 2.7 陷阱标志 (TF) 在中断期间标志寄存器的内容被压入栈堆后将被清除。当 TF = 1 时, 它在每条指令执行之后产生一个陷阱中断 (类型 1), 这就是为什么我们常称陷阱为单步的原因。当 TF = 0 时, 程序正常执行。试编写 TF 置 1 和置 0 的程序段。

解 置位 TF, 以便允许跟踪的过程。

TRON PROC NEAR

PUSH AX ; 保护寄存器

PUSH BP ;

MOV BP, SP ; 取 SP

MOV AX, [BP+8] ; 从堆栈获得标志, 在标志寄存中 TF 处于第 8 位

OR AH, 1 ; 置位 TF

MOV [BP+8], AX ; 保存标志

POP BP ; 恢复寄存器

POP AX

IRET

TRON ENDP

清除 TF, 禁止跟踪的过程

TROFF PROC NEAR

PUSH AX ; 保护寄存器

PUSH BP ;

MOV BP, SP ; 取 SP

MOV AX, [BP+8] ; 从堆栈获得标志, 在标志寄存中 TF 处于第 8 位

AND AH, 0FEH ; 清 TF

MOV [BP+8], AX ; 保存标志

POP BP ; 恢复寄存器

POP AX

IRET

TROFF ENDP

在这两个程序段中，利用 BP 寄存器从堆栈中取出标志寄存器，默认情况下 BP 寻址堆栈段。找到标志后，在从中断服务程序返回前将 TF 位置位或者清除。IRET 指令以新的陷阱标志状态恢复标志寄存器。

例 2.8 当 $(IDT + 8H) = 1000H$ ， $(IDT + AH) = B000H$ ， $(IDT + CH) = AE00H$ ， $(IDT + EH) = 0000H$ 时，试说明此门。

解 门描述符是一种特殊格式的系统描述符。门描述符包含了一个 32 位的偏移地址、一个字计数器、一个选择符和一个属性字节。32 位的偏移地址指向中断服务程序或其他程序入口。字计数字段仅对调用门有效，指示已有多少个参数从调用者堆栈传送到所调用子程序的堆栈上。这个从调用者堆栈中传送数据的特性对于实现高级语言非常有用。选择符用来指示中断服务程序或子程序的代码段描述符、或任务状态段 (TSS) 的描述符在 GDT 或 LDT 中的位置。图 2.10 所示的是 32 位微处理器的门描述符示意图。

偏移地址 (31~16)							IDT + EH
P	DPL	0	类型 (4 位)	0	00	字计数 (4~0)	IDT + CH
选择符 (15~0)							IDT + AH
偏移地址 (15~0)							IDT + 8H

图 2.10 例 2.8 图

由上表可知，此门类型为 1110，即 32 位微处理器的中断门。门中选择符为 B000H，及其 RPL=00，在 GDT 基地址偏移量为 B000H 处找到中断服务程序代码段描述符，由其中基地址和中断门中偏移量找到中断服务程序。

例 2.9 某 I/O 许可位图为 11111111 11101100 00001100 10111100，试分析 8 位、16 位、32 位 I/O 端口的访问属性。

解 如果当前任务的 CPL 与 IOPL 允许对 I/O 地址空间进行访问，Pentium 就会检查 I/O 许可位图，确定其端口可否操作。位图的每一位对应一个 8 位的 I/O 地址，如果该位置 1，则当访问该位对应的端口时 Pentium 会送出一一般保护错误异常中断；若该位置 0，则可对相应的端口进行 I/O 操作。

依题意，其中的 11111111B 仅仅是一个结束标志，实际的 I/O 位图只有 3B，它表示有 24 个 I/O 端口受到保护，从左至右一一对应的端口是 23, 22, ..., 1, 0，于是有以下三种访问格式：

(1) 对于 8 位端口的访问。

锁定的端口（即不能被访问的）为 2、3、4、5、7、10、11、18、19、21、22、23；未锁的端口（即可以访问的）为 0、1、6、8、9、12、13、14、15、16、17、20。

(2) 对于 16 位端口的访问。

锁定的端口为 2、4、6、10、18、20、22；未锁的端口为 0、8、12、14、16。

(3) 对于 32 位端口的访问。

锁定的端口为 0、4、8、16、20；未锁的端口为 12。

例 2.10 对于一个外部频率为 66MHz 的 Pentium 微处理器，在没有等待状态的情况下，试分析在非流水线读/写周期、突发读/写周期和流水线读/写周期下的传输速率。

解 从教材上我们知道了三种总线周期的流程图。首先是非流水线读/写周期，对于这种情况来说，一个无等待状态的数据传输至少占用 2 个 CLK 时钟，于是最高的数据传输速率为 $8B \times 66MHz / 2 = 254MB/s$ 。对于突发周期而言，由于操作是直接面向 Cache，传输的长度固定为 32B，在这种情况下，Pentium 只需在第一个周期内发出起始地址和 $\overline{BEx}$ 信号，后面的三个周期则保持不变即可，所以传输 32B 需要 $2 + 3 = 5$ 个时钟周期。因此，传输速率为 $32B \times 66MHz / 5 = 423MB/s$ 。

最后是流水线式总线周期，由于运用了重叠原理，使得第一个总线周期减少到只要一个单一的 CLK 周期，故传输 32B 只需要 4 个时钟周期，所以传输速率为 $32B \times 66MHz / 4 = 528MB/s$ 。

例 2.11 假定任务 A 的 CPL=2，在某一时刻为当前任务，其任务状态段和 TSS 寄存器的状态如图 2.11 所示。现在任务 A 要通过图 2.12 所示的任务门调用任务 B，试分析其过程。

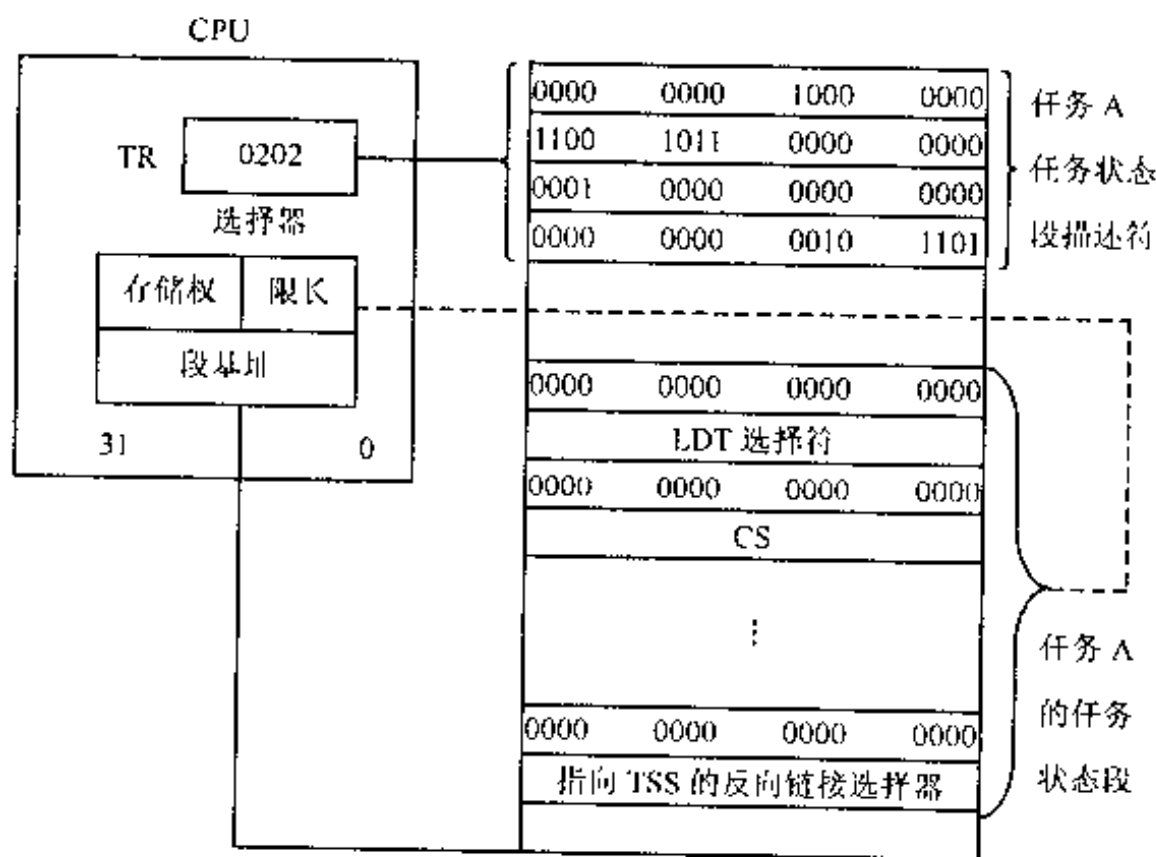


图 2.11 例 2.11 图 (1)

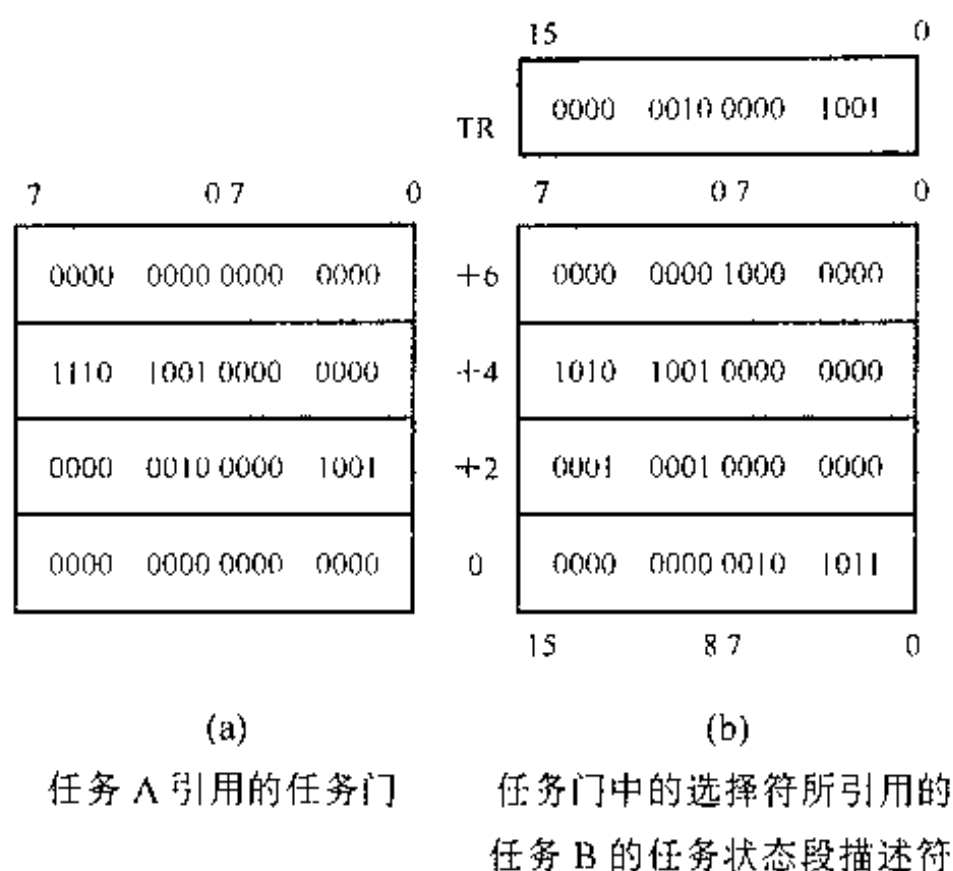


图 2.12 例 2.11 图 (2)

解 处理器先用图 2.12 (a) 所示的任务门所给出的信息 (即目的选择符), 从 GDT 中找到索引值为 41H 的项, 在这一项中存放着如图 2.12 (b) 所示的任务 B 的任务状态段的描述符。操作系统中的任务转换机制, 把任务 A 的各个寄存器的当前值, 保护到任务 A 的 TSS 中; 然后把任务门中的任务 B 的 TSS 选择符装入 TR 寄存器中, 并把该选择符所指描述符的 TYPE 置为 0BH。由于装入了 TR 寄存器, 任务 B 的 TSS 的描述符中的相应信息如 TSS 的基址、限长以及访问权字节等就装入相应的高速缓冲寄存器 Cache 中。CALL 指令把任务 A 的 TSS 选择符, 装入至任务 B 的 TSS 的 link 字段, 并把任务 B 的标志寄存器中的 NT 置位 1, 实现任务嵌套。CPU 的各个寄存器由任务 B 的 TSS 中的值装入, 就可以开始执行任务 B。这时任务转换机制和存储器中的内容, 如图 2.13 所示。

在这个例子中, 任务 A 的 $CPL=2$, 它引用任务门的 $DPL=3$, 满足 $CPL \leq DPL$ 的条件, 故访问是允许的。任务 B 的任务状态段描述符 $DPL=1$, 表示当直接调用任务状态段时, 要求 $CPL \leq 1$, 任务 A 不能满足这个要求, 所以要通过任务门来间接调用。

在任务切换过程中, 从任务 B 的 TSS 中恢复 CS 寄存器值时, 就建立了新的当前任务特权级。这样, 既实现了任务切换, 同时也通过门描述符实现了特权级的改变。当任务 B 完成后, 通过执行一条 IRET 指令返回。由于此时 $NT=1$, 所以返回操作会通过任务 B 的 TSS 中的反向链接信息, 把控制返回任务 A。

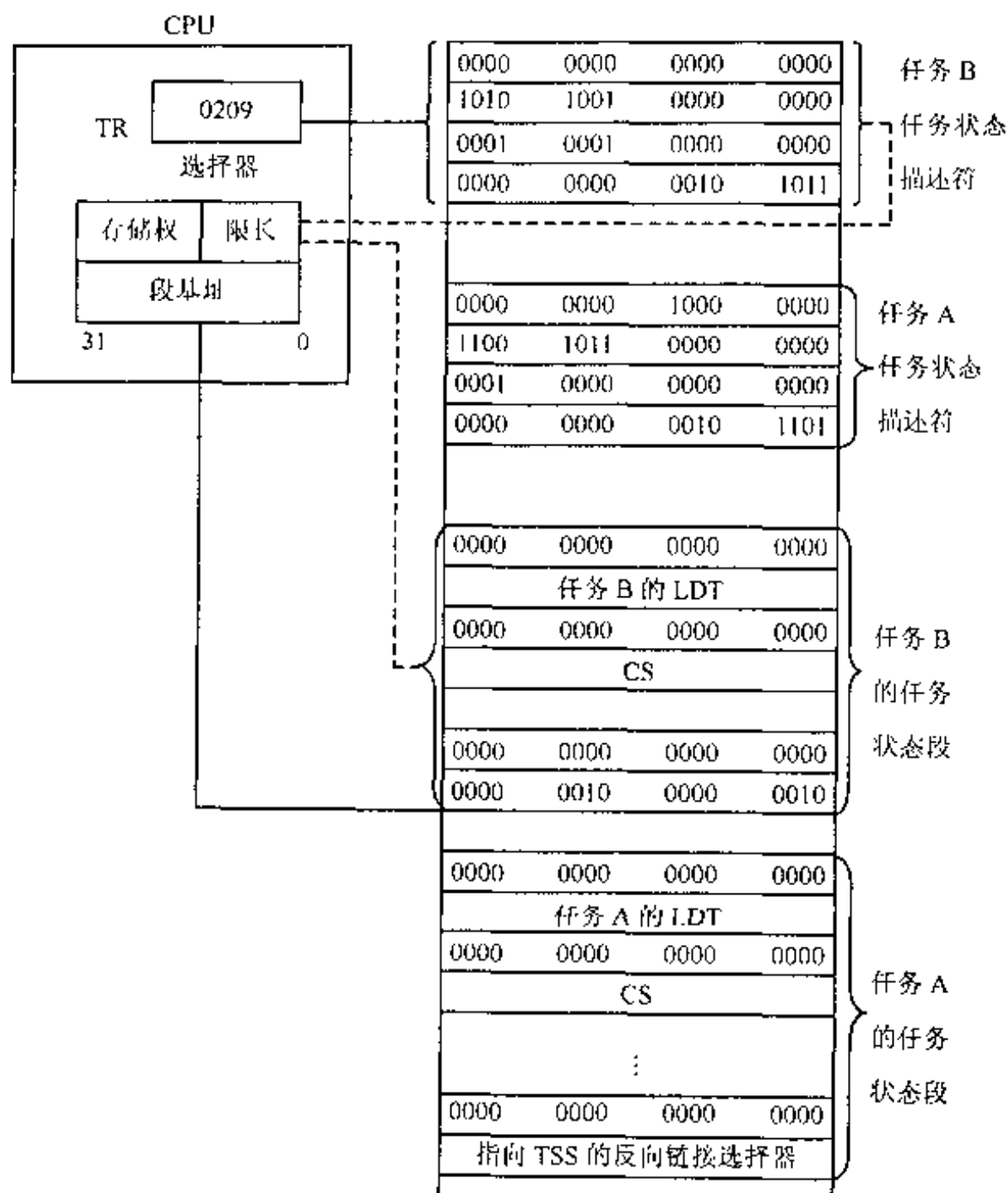


图 2.13 例 2.11 图 (3)

例 2.12 有一个 32 位微处理器初始化实例。该实例介绍系统从初始化状态开始转变为一个 32 位系统的过程, 转变后的系统具有分页功能, 可运行多个任务, 这个初始化实例由若干个程序段组成。

(1) Equates

这是实例程序的第一部分, 该部分进行如下定义:

- ① 定义助记符。
- ② 定义 GDT 及 LDT 的选择符。
- ③ 定义页的物理地址。

(2) 段定义

定义实例中的四个段 (即 OS_Data_Seg、User_Code_Seg、User_Data_Seg 及 OS_Code_Seg)。

(3) 代码 1—冷启动

系统复位时, 32 位微处理器以实模式开始操作。这种模式下最大的段为 64KB, 选择符左移四位与段内偏移量相加形成物理地址, 因此, 最大的物理地址为 FFFFFH, 即整个地址空间为 1MB。

系统复位时, 将 CS 初始化为 F000H, IP 初始化为 FFF0H。在实模式下由 CS 和 IP 形成的 20 位地址为: $F000H \times 16 + FFF0H = FFFF0H$ 。也就是说复位后执行的第一条指令的地址在 F000H 段的 FFF0H 偏移处。通常, 我们总是把系统复位后要执行的程序固化在 ROM 中, 并占用内存空间的一个存储段, 在这种情况下, 应占用的段地址为 F0000H。由于 32 位微处理器的数据和地址都是 32 位的, 如果把固件所占用的段地址定在 F0000H, 则系统的内存地址空间被固件所在地址分成不连续的两个部分, 会造成对内存资源有效管理的困难。因此, 32 位微处理器硬件规定系统复位时, 最高 12 位地址线($A_{31} \sim A_{20}$)用 1 个触发器的“1”状态强制为高电平, 从而使固件所在的段从 F000H 平移至 FFFF000H, 第一条要执行的指令的地址也由 FFFF0H 变为 FFFFFFF0H。由于 FFFF000H 段是 32 位系统地址空间的最后一个 64KB 存储段, 所以内存地址空间的其余部分构成一个完整的连续区域。最高 12 位地址线强制为“1”的状态持续至从 FFFFFFF0H 开始执行的程序中, 遇到第一个段间转移指令为止。这是硬件设计中每当执行段间转移指令时, 都将前述触发器置为“0”的缘故。这一方法是所有机器在系统初起时, 总是从定位在最高存储区域的固件程序的预定地址开始执行的常用方法, 从而也要求 Boot ROM 总是置于存储器的最高端。

(4) 代码 2—置初值

这段程序开始于标号 Start16 (程序清单中 BootROM16 段的开始处)。首先将 IF 位置 1, 接着再将 IDT 的界限置 0。前者用于屏蔽任何可屏蔽中断, 后者用于万一发生并接收了不可屏蔽中断后能使系统马上处于关机状态。为使 IDT 的界限为 0, 需使用 LIDT 指令, 而该指令要求有一存储器操作数, 因此需将 DS 寄存器初始化, 并指向 OS_Data_Seg。在本例中, 因为 DS 寄存器初始化为 0, 而 OS_Data_Seg 的偏移量也恰好为 0, 所以初始化 DS 并指向 OS_Data_Seg 变得没有必要。然而, 在多数情形下, 要使用一个段必须先初始化段寄存器, 所以在实例中沿用常规办法。

(5) 代码 3—建立和装入 GDT

代码的第三部分建立 GDT, 并将 GDT 伪描述符装入处理器。

首先, 要用适当的信息填入 GDT。本例中, 我们有五个 GDT 表项 (NULL、OS Code、OS Date、LDT 及 TSS)。表 2.11 列出了这五个描述符的内容, 表 2.12 列出这里使用的描述符类型。程序代码就是要将描述符的内容进行适当分离, 使其形成标准的描述符格式。

表 2.11 例 2.12 中所使用的描述符内容

描述符名	所在描述符表	界限 (LIMIT)	基地址 (BASE)	存储段 D 属性或者系统 D 类型	描述符类型 (S)	特权级 (DPL)	存在位 (P)	操作尺度 (D/B)	粒度位 (G)
NULL	GDI	00000	00000000	0	0	0	0	0	0
OS 代码	GDT	0000E	FFFF0000	A	1	0	1	1	1
OS 数据	GDT	FFFFFF	00000000	2	1	0	1	1	1
用户代码	LDT	000FF	01000000	A	1	3	1	1	1
用户数据	LDT	FEFEF	01100000	2	1	3	1	1	1
TSS1	GDT	000D7	000000B0	9	0	0	1	0	0
LDT1	GDT	0000F	00000188	2	0	0	1	0	0

表 2.12 例 2.12 中使用的描述符类型

描述符类型 (S)	属性 (E ED/C W/R A) 或类型	描述符说明
1	属性 2: 0 0 1 0	存储段—可读/写数据
1	属性 A: 1 0 1 0	存储段—可执行可读代码
0	类型 9	系统段—可用 32 位 TSS
0	类型 2	系统段—LDT

然后在存储器中建立伪描述符。伪描述符的低位字为 GDT 的界限，其值为描述符表项数的 8 倍减 1，即 GDT 内的最大偏移量。伪描述符的高位双字是 GDT 的线性基址。

最后，用 LGDT 指令将伪描述符装入处理器。

(6) 代码 4—进入保护模式

这一步是要设置机器状态字的 PE 位。在 PE 位置位之后，要立即执行一条跳转指令，清除芯片上的预取指令及译码序列。这一点之所以重要，是因为在 PE 位置位后，机器从实模式转入了保护模式，而已预取及译码的指令则从属于不再有效的实模式。至此，机器已进入了保护模式，但数据和偏移量还都是 16 位的。然后，再执行一条段间 JMP 指令，该指令转向 OS Code 段的 Start 32。由于 GDT 中定义 OS Code 段为 32 位的代码段，所以在 JMP 指令执行完毕之后，机器进一步转变为一个 32 位机。

(7) 代码 5—装入段描述符

本段程序从标号 Start 32 开始（在清单中 32 位 OS 段的前部）。由于机器已处于保护方式，因此有必要重新装入段寄存器使得数据段变为可寻址的段。在前面的代码 2 中曾用选择符装入过 DS，但在那里是实模式，通过 DS 及偏移量形成的物理地址最多只寻址 1MB。在启用保护方式之后 DS 装入选择符，是为了在对应的段 Code 描述符寄存器中得到描述符。通过描述符不仅建立起各种保护信息，

还使偏移量及基地址都扩展为 32 位，寻址范围扩大为 4GB。装入 DS 的选择符还装入 ES，以供串操作指令使用。

(8) 代码 6—建立和装入 LDT

任务的局部地址空间由 LDT 进行映射。LDT 包含两个表项，一个用户代码段，另一个为用户数据段。建立 GDT 时，GDT 中应有映射 LDT 的表项。LDT 本身也作为一个数据段存在，并定义为 OS Data 段的一部分。初始化实例中，当 32 位微处理器使用实际的 LDT，用 OS_Data 段中的别名建立 LDT 时，要用适当的信息填入 LDT。在实例中，也就是用户代码、用户数据两个段的有关信息。这些信息也列在表 2.11 中。程序代码将表 2.11 中 User Code 及 User Data 两行内容进行适当分离，使其符合描述符的格式，再写入 LDT 中。在 LDT 建立后，再把访问 LDT 的选择符装入 LDTR。此后，LDT 即可被访问。

(9) 代码 7—建立 IDT

初始化中的一个很重要的部分是建立 IDT。在我们的实例中，当进入用户方式之后，用户任务访问操作系统服务的唯一途径就是借助于异常处理机制，借助门可以在低特权级访问高特权级的程序，事实上，IDT 表项本身即是一个门——中断门。由于整个操作系统在实例中对当前任务是可见的，所以在 IDT 表项中也不必使用任务门。实例中中断门的基本内容见表 2.13。如果一个异常或中断在特权级 3 被响应，则要进行特权级的转换，相应地还要改变所使用的栈。因此，响应中断之后，代码特权级从 3 级切换到 0 级，使用的堆栈也从原来的 3 级栈改变为 0 级栈。

表 2.13 例 2.12 中中断门的基本内容

字 段	值
Offset	默认中断处理程序代码段偏移量
Selector	中断处理程序代码段选择符
Dword Count	0，不拷贝参数
Type	E，32 位微处理器中断门类型
DPL	0，描述符优先级

OS_Data 段中的别名用来访问 IDT。开始时，所有 17 个中断门都填入相同的信息，这 17 个中断级是由 32 位微处理器预先定义的。中断门中的信息是指向处理程序入口的，在这里就指向系统默认的中断处理程序，该处理程序实际上是不做任何处理即返回。完成这一步之后，再对那些在系统中有实际意义的中断，由操作系统提供特别的处理程序，将转入该处理程序所需的信息填入对应的 IDT 表项中，修改并替换原来已填入的默认信息。对于所有有意义的中断，替换相应 IDT 表项中的信息都完成之后，IDT 的建立即告结束。

(10) 代码 8—建立页目录表

段机制建立起来以后,即可着手建立分页机制。页机制有两级转换表:页目录表及页表。每一任务有一页目录表,最多可有 1024 个页目录表项,各指向一个页表;每一页表最多可有 1024 个页表项,各指向一页;每一页为 4KB。每个页目录表项除含有指向对应 4MB 线性地址空间的页表基址外,还含有目录表项本身的若干属性。如属性中的存在位为 0,则该表项可视为空,对应的 4MB 线性地址空间页就不存在。如果某一任务实际使用了 N 个页表(注意,不一定每一页表都用满 4MB),则应在页目录中填入 N 个目录项。填入的信息有:存在位、用户/监控特权级、读/写方式、所指向页表的物理基址等。因此,建立页目录表的程序分为两个部分:第一部分将页目录表的所有 1024 个表项的存在位都置 0;第二部分将映射有效线性地址空间的表项用上述几个方面的信息填入。

实例中实际填入了 6 个页目录表项,页表地址可从前面的 EQU 指令中查到。需要注意的第二个表项,即 entry1,页目录项指向的页表是页目录表本身。这一点看起来容易使人发生混淆,但是,这的确是把页目录表和页表映射到线性地址空间中的一种很方便的方法。

(11) 代码 9—建立页表

建立页表与建立页目录表有完全类似的过程。首先将所有页表项标志为不存在,然后将有效表项填入相应的信息。因为一个任务只有一个页目录表,而一个任务的页表则可能有多个(本实例中即有 5 个页表),因此页表的建立就有多个这样的过程。使线性地址等于物理地址在建立页表中是一个很重要的考虑。在代码 9 建立的页表中,下列线性地址等同于物理地址:

① 操作系统代码在 ROM 之外执行时,操作系统在以物理地址 FFFF0000H, FFFF1000H, ..., FFFFF000H 开始的 16 个页应该赋予等同的线性地址,也就是使操作系统代码的线性地址等于物理地址;否则在分页机制启用之后,就会发生莫名其妙的跳转,产生难以预料的后果。

② GDT、IDT 及 TSS 存储在物理地址开始的第一页。对于该页,也赋予相同的线性地址,以避免分页机制启用之后,要重新装入相应寄存器及重建相联系的表。

表 2.14 列出的是代码 9 建立的页表对应的页目录项,页表项及访问属性。

(12) 代码 10—启用分页机制

建立了页目录表及页表之后,即可启用分页机制。这一工作只需用一条指令将 CR₀ 的 PG 位置 1 即可。但在启用分页机制之后,要用一条跳转指令清指令预取队列及指令译码队列。分页机制被禁用的情况下,物理地址总是等于线性地址。然而分页机制启用之后,这一关系一般来说是不成立的,所以预取的指令队列及译码队列在 CR₀ 的 PG 位置 1 之后即变为无效,故做一次跳转是完全必需的。

表 2.14 已分配页的具体内容

页目录项	页表项	用户/监控	写权限	页内容
0	0	监控	无	IDT、GDT、TSS
1	0	监控	无	页表 0
1	1	监控	无	页目录
1	3	监控	无	页表 3
1	4	监控	无	页表 4
1	3FEH	监控	无	页表 3FE
1	3FFH	监控	无	页表 3FF
3	0	监控	无	OS 专用数据
3	3FFH	监控	无	OS 栈
4	0	用户	无	用户代码
4	100H	用户	有	用户数据
3FEH	3FFH	用户	有	用户栈
3FFH	3F0H~3FFH	监控	无	OS 代码

(13) 代码 11—装入操作系统栈指针

不同特权级程序有不同的堆栈。本例使用两个特权级，故只需要两个堆栈。LSS 指令一次装入 SS 及 ESP 两个寄存器，其间不会由于中断而使 SS 及 ESP 的内容不一致而造成错误。我们的程序到此为止，尚未开中断，因而使用 LSS 指令也并非必需。但是，用 LSS 装入 SS 及 ESP，而不是用两条指令完成同样的装入操作，这是一个好的编程习惯。这样往往可以避免发生看起来非常简单然而又难以排除的编程错误。

(14) 代码 12—建立 TSS

在特权级改变或任务切换期间，需访问 TSS 以检索或存储信息。例如，系统中的第一次异常要求从用户特权级 3 转移到监控特权级 0，作为特权转移的一部分，使用的栈也要加以改变。所以，期望能在 TSS 中的适当位置找到 0 级栈的选择符及偏移量。像这样的信息必须预先给出，而其他若干信息则是切换过程中自动装入的。代码 12 放入 TSS 中的信息有：0 级栈偏移量、0 级栈选择符、页目录表基址、LDT 选择符、I/O 位图偏移量、调试陷阱位等。I/O 位图偏移量在程序中置为 8000，该值超过了 TSS 的界限，因而指的是一个空位图。然后装入 TR 使 TSS 对处理机可见。保留区的最高字节设置为 0，指示该任务尚未使用协处理器。至此，初始化的所有任务几乎都已完成。下面是装入用户任务。

(15) 代码 13—装入用户任务

实例中有一个名为 Loader（）的过程，该过程为用户装入任务，功能有：

① 将用户任务从虚拟空间读入内存空间。事实上只先读入一页，其他各页（如

果存在的话)在以后发生页故障处理时读入。

② 返回任务的开始地址,即第一条指令的偏移量,并存放在 EAX 寄存器中,此后应将此值放入 TSS 中,以便调度到此任务后可从 TSS 中找到执行地址。

③ 计算代码长度,并适当调整 User_Code_Seg 的界限及 User_Data_Seg 的基址。

(16) 代码 14—为用户任务调用做准备

前面已经讨论过,要从内层级通过调用切换到外层级是不可能的。系统初启时,工作于 0 级,而任务处于 3 级。为了能从 0 级转到 3 级执行用户任务,需要制造一个假的 IRET 环境,产生一个当前在 0 级运行是由于中断了用户程序而正在做中断处理的假象。这样,用 IRET 启动系统的第一个用户任务就很自然了。代码 14 就制造这一假的 IRET 环境,在堆栈中压入如下信息:栈选择符、栈偏移量、标志、代码选择符、代码偏移量等。

代码段选择符与偏移量应为中断时的断点,但每个任务第一次执行时都在选择符及偏移量处写入程序要执行的第一条指令的地址,这就是 Loader() 返回在 EAX 中的地址。注意使堆栈中标志寄存器映像中的 IF 位置位而允许中断。

(17) 代码 15—调用用户任务

实例的最后一步是执行 IRET 指令,该指令将把刚压入栈中的信息恢复至用户任务中,从而改变保护级、栈、CS: EIP 及标志。IRET 执行完毕后,即完成了对第一个用户任务的调用,全部工作也到此结束。

解 下面是程序清单。

; 各种常数

IDT_Entries EQU 17

GDT_Entries EQU 5 ; Null, OS Code, OS Data, TSS, LDT 五个 GDT 表项

LDT_Entries EQU 2 ; User Code, User Data 两个 LDT 表项

Page_Size EQU 4096

Page_entries EQU 1024

Not Present EQU 0

PE EQU 1

PG EQU 80000000H

Int_Flag EQU 0200H

; GDT 表项的选择符

Null EQU 0H ; 空描述符

OS_Code_Sel EQU 8H ; 1, GDT, RPL = 00

```

OS_Data_Sel    EQU    10H    ; 2, GDT, RPL = 00
LDT1_Sel       EQU    18H    ; 3, GDT, RPL = 00
TSS1_Sel       EQU    20H    ; 4, GDT, RPL = 00
; LDT 表项的选择符
User_Code_Sel  EQU    7H     ; 0, LDT, RPL = 11
User_Data_Sel  EQU    0FH    ; 1, LDT, RPL = 11
; 物理地址常数, 用于初始化页表
P_OS_Gdata     EQU    00000H
P_Page_Dir     EQU    01000H
P_Page_Tab0    EQU    02000H
P_Page_Tab3    EQU    03000H
P_Page_Tab4    EQU    04000H
P_Page_Tab3FE  EQU    05000H
P_Page_Tab3FF  EQU    06000H
P_OS_Pdata     EQU    07000H
P_OS_Stack     EQU    08000H
P_User_Code    EQU    09000H
P_User_Data    EQU    0A000H
P_User_Stack   EQU    0B000H
TSS_Size       EQU    104    ; TSS 段的最小段长
FP_Save        EQU    112    ; 浮点保存区域共 108B。另 4B 存放 FP 已被使用
                        标志

```

OS 数据段

; 包含段表及页表, 该数据段的线性地址为 0

```
OS_Data_Seg    SEGMENT    RW    USER32
```

; IDT 表在线性地址为 0 的位置接下去是 GDT、TSS、LDT

```
IDT_Table      db  IDT_Entries*8 DUP(?)
```

```
GDT_Table      db  GDT_Entries*8 DUP(?)
```

```
TSS1_Table     db  (TSS_Size+FP_Save) DUP(?)
```

```
LDT1_Table     db  LDT_Entries*8 DUP(?)
```

```
Pdescr        db  6 DUP(?)    ; 为 IDTR、GDTR 建立伪描述符
```

****物理地址空间中的页目录表及页表区域, 用于在启用分页机制之前初始化页表****

```
ORG 00001000H    ; 物理地址
```

```
Page_Dir       dd  Page_Entries DUP(?)
```

```

Page_Tab0      dd  Page_Entries DUP(?)  ; OS 公共数据 (IDT、GDT、TSS)
Page_Tab3      dd  Page_Entries DUP(?)  ; OS 专用栈
Page_Tab4      dd  Page_Entries DUP(?)  ; 用户代码及数据
Page_Tab3FE    dd  Page_Entries DUP(?)  ; 用户栈
Page_Tab3FF    dd  Page_Entries DUP(?)  ; OS 代码
**线性地址空间中的页目录表及页表区域, 4M 线性地址空间供所有页目录表及页表使用**
                ORG 00400000H
L_Page_Tab0    dd  Page_Entries DUP(?)  ; OS 公共数据 (IDT、GDT、TSS)
L_Page_Dir     dd  Page_Entries DUP(?)  ; 页目录
                ORG 00403000H
L_Page_Tab3    dd  Page_Entries DUP(?)  ; OS 专用栈
L_Page_Tab4    dd  Page_Entries DUP(?)  ; 用户代码及数据
                ORG 007FE000H
L_Page_Tab3FE  dd  Page_Entries DUP(?)  ; 用户栈
L_Page_Tab3FF  dd  Page_Entries DUP(?)  ; OS 代码
**OS 数据, 4M 线性地址空间, 例子中未使用**
                ORG 08000000H
**OS 数据, 4M 线性地址空间, OS 专用的数据和栈**
ORG 0C000000H      ; 线性地址
Unshared_OS_Data   dd      ?
temp      db      10 DUP(?)      ; 不被共享的 OS 数据区域
fptss     dw      ?
**OS 栈**
ORG 0FFFFFFCH      ; 栈顶
Top_OS_Stack dd      ?
OS_Data_Seg  ENDS
**用户代码段**
; 此段安排在线性地址 1000000H 处
User_Code_Seg  SEGMENT      ER      USE32
; 线性地址空间中, 用户代码段的开始地址, 定义一个 1MB 的分区
User_Code_Seg  ENDS

**用户数据段**
; 此段安排在线性地址 1100000H 处
User_Data_Seg  SEGMENT      RW      USE32

```

；线性地址空间中，用户数据段的开始地址，存储器的其余部分定义为用户的数据

```
ORG 0FEEFFFC
```

```
Top_User_Stack dd ?
```

```
User_Data_Seg ENDS
```

****32 位 OS 代码段****

；32 位的 OS 代码安排在线性地址 0FFFF0000H 处

```
OS_Code_Seg SEGMENT ER USE32 ; Locate AT 0FFFF0000H
```

Default_Hand:

；<默认处理程序代码>

```
IRET
```

DNA_Hand:

；<DNA 处理程序代码：协处理器异常的处理程序>

```
IRET
```

Page_Fault_Hand:

；<页故障处理程序代码>

```
IRET
```

Debug_Hand:

；<排错处理程序代码>

```
IRET
```

Start32: 【代码 5】

；**将 OS 数据段的选择符装入 DS、ES，使 OS_Data 可被寻址**

```
MOV AX, OS_Data_Sel
```

```
MOV DS, AX
```

```
MOV ES, AX
```

```
ASSUME DS: OS_Data_Seg
```

****建立 LDT ** 【代码 6】**

```
MOV EAX, offset LDT1_Table
```

；表项 0，用户代码段描述符

；Base=01000000H，Limit=000FFH

；G=1，D/B=1，P=1，DPL=11，S=1，E=1，ED/C=0，W/R=1，A=0

```
MOV word ptr [EAX], 000FFH ; Limit[15~0]=00FFH
```

```
MOV word ptr [EAX+2], 0000H ; Base[15~0]=0
```

```
MOV byte ptr [EAX+4], 000H ; Base[23~16]=0
```

```

MOV byte ptr [EAX+5], 0FAH      ; P = 1, DPL=3, S = 1, E = 1
; ED/C = 0, W/R = 1, A = 0
MOV byte ptr [EAX+6], 0C0H      ; G=1, D/B=1, Limit[19~16]=0
MOV byte ptr [EAX+7], 01H      ; Base[31~24]=01H
ADD EAX, 8
; 表项 1, 用户数据段描述符
; Base = 01100000H, Limit = FEEEFH
; G = 1, D/B = 1, P = 1, DPL=3, S = 1, E = 1, ED/C = 0, W/R = 1, A = 0
MOV word ptr [EAX], 0EEEFH      ; Limit[15~0]=0EEEFH
MOV word ptr [EAX+2], 0000H      ; Limit[15~0]=0
MOV byte ptr [EAX+4], 10H      ; Base[23~16]=10H
MOV byte ptr [EAX+5], 0F2H      ; P = 1, DPL=11, S = 1, E = 1
; ED/C = 0, W/R = 1, A = 0
MOV byte ptr [EAX+6], 0CFH      ; G=1, D/B=1, Limit[19~16]=0FH
MOV byte ptr [EAX+7], 01H      ; Base[31~24]=01H
MOV AX, LDT1_Sel
LLDT AX

```

建立 IDT 【代码 7】

：先将 IDT 各个表项填入默认处理数据，再对需处理的中断的表项进行修改

```

MOV EAX, offset IDT_Table
MOV EBX, offset Default_Hand
MOV word ptr [EAX], BX          ; 默认处理程序入口的低 16 位
MOV word ptr [EAX+2], OS_Code_Sel
MOV byte ptr [EAX+4], 00H      ; 字计数为 0
MOV byte ptr [EAX+5], 8EH      ; 类型为 32 位微处理器中断门, 特权级为 0, P=1
SHR EBX, 16
MOV word ptr [EAX+6], BX      ; 默认处理程序入口的高 16 位
; 用默认处理表项填满全部 IDT 表
MOV ESI, offset IDT_Table
MOV EDI, (offset IDT_Table+8)
MOV ECX, (IDT_Entries - 1) * 2
REP MOVSD
; 修改非默认处理的中断表表项
; 表项 1: 排错

```

```

MOV EAX, (offset IDT_Table + (1*8))
MOV EBX, offset Debug_Hand
MOV word ptr [EAX], BX           ; 处理程序入口低 16 位
SHR EBX, 16
MOV word ptr [EAX + 6], BX       ; 处理程序入口高 16 位
; 表项 7: 设备不可用
MOV EAX, (offset IDT_Table + (7*8))
MOV EBX, offset DNA_Hand
MOV word ptr [EAX], BX           ; 处理程序入口低 16 位
SHR EBX, 16
MOV word ptr [EAX + 6], BX       ; 处理程序入口高 16 位
; 表项 14: 页故障
MOV EAX, (offset IDT_Table + (14*8))
MOV EBX, offset Page_Fault_Hand
MOV word ptr [EAX], BX           ; 处理程序入口低 16 位
SHR EBX, 16
MOV word ptr [EAX + 6], BX       ; 处理程序入口高 16 位
; 装入 IDTR
MOV word ptr pdescr, (IDT_Entries*8 - 1)
MOV dword ptr pdescr [2], offset IDT_Table
LIDT pword ptr pdescr

```

****建立页目录表** 【代码 8】**

```

MOV EAX, Not_Present
MOV ECX, Page_Entries
CLD
MOV EDI, offset Page_Dir
REP STOSD           ; 标记整个目录不存在
; 表项 0: 共享的 OS 数据, 用户不可访问, 安排在地址 0, 所以线性地址 = 物理地址,
; 而且装入的, GDT、IDT 及 TSS 在分页机制被启用后仍然有效
MOV Page_Dir[0], P_Page_Tab0 OR 001H
; 表项 1: 页目录表指针, 把页目录表及页表映射到线性地址空间, 用户不可访问
MOV Page_Dir[4], P_Page_Dir OR 001H
; 表项 3: OS 专用数据, 用户不可访问
MOV Page_Dir[12], P_Page_Tab3 OR 001H

```

：表项 4：用户代码及数据，用户可读/写

MOV Page_Dir[16], P_Page_Tab4 OR 007H

：表项 1022：用户栈，用户地址空间的最高端，用户可读/写

MOV Page_Dir[4088], P_Page_Tab3FE OR 007H

：表项 1023：共享的 OS 代码，用户不可访问

MOV Page_Dir[4092], P_Page_Tab3FF OR 001H

****建立页表** 【代码 9】**

：页表 0：OS 公共数据，首先清除整个表

MOV EAX, Not_Present

MOV ECX, Page_Entries

CLD

MOV EDI, offset Page_Tab0

REP STOSD

：为 IDT、GDT 及 TSS 等分配存储页

MOV Page_Tab0[0], P_OS_Gdata OR 001H

MOV EAX, Not_Present

MOV ECX, Page_Entries

CLD

MOV EDI, offset Page_Tab3

REP STOSD

：为专用数据和 OS 栈分配存储页

MOV Page_Tab3[0000H], (P_OS_Pdata OR 001H)

MOV Page_Tab3[0FFCH], (P_OS_Stack OR 001H)

：页表 4：用户代码、数据和栈

MOV EAX, Not_Present

MOV ECX, Page_Entries

CLD

MOV EDI, offset Page_Tab4

REP STOSD

：为用户代码及数据分配存储页

MOV Page_Tab4[0000], (P_User_Code OR 005H)

MOV Page_Tab4[1024], (P_User_Data OR 007H)

：页表 1022：用户栈

MOV EAX, Not_Present

MOV ECX, Page_Entries

CLD

MOV EDI, offset Page_Tab3FE

REP STOSD

: 为用户栈分配存储页

MOV Page_Tab3FE[4092], (P_User_Stack OR 007H)

: 页表 1023: OS 代码。所有页面对用户可读, 存储器最高 64KB 地址的映射, 使物理地址与线性地址相匹配

MOV EAX, 0FFFFFF005H : 最高页表项

MOV ECX, Page_Size

LoopTop: SUB ECX, 4

MOV Page_Tab3FF[ECX], EAX

SUB EAX, 1000H

CMP ECX, 4032

JNZ LoopTop

****启用分页机制** 【代码 10】**

MOV EAX, P_Page_Dir

MOV CR3, EAX

MOV EAX, CR0

OR EAX, PG

MOV CR0, EAX

JMP Pflush

Pflush:

****装入 OS 栈指针** 【代码 11】**

MOV dword ptr temp[0], offset Top_OS_Stack

MOV word ptr temp[4], OS_Data_Sel

LSS ESP, pword ptr temp

****建立 TSS** 【代码 12】**

: 只装入由 OS 访问的部分, 其余部分不装入或在第一次向内层转移时装入

MOV dword ptr TSS1_Table[4], offset Top_OS_stack : 0 级栈偏移

MOV dword ptr TSS1_Table[8], OS_Data_Sel : 0 级栈选择符

MOV EAX, CR3 : CR3

MOV dword ptr TSS1_Table[1CH], EAX


```

MOV dword ptr TSS1_Table[60H], 0          ; LDT
MOV word ptr TSS1_Table[66H], 8000H       ; I/O 位图偏移
MOV word ptr TSS1_Table[64H], 0          ; 调试陷阱位
MOV AX, TSS1_Sel
LTR AX
MOV byte ptr TSS1_Table[TSS_Size + FP_Save - 1], 0 ; 初始化浮点状态字节
MOV fptss, 0

```

****装入用户任务** 【代码 13】**

```

MOV EAX, User_Code_Sel      ; 用户代码段选择符
MOV ECX, User_Data_Sel      ; 用户数据段选择符
; loader 从盘上取出第一个用户任务, 并将装入到传递给该任务的选择符所指定的位置。
; loader 将用户任务的入口地址偏移量置入 EAX 寄存器返回
CALL loader

```

****为调用用户任务做准备** 【代码 14】**

```

; 用户的 SS: EIP、FLAGS、CS: EIP 压栈, 使 IRET 调用用户任务
MOV ECX, User_Data_Sel      ; 用户栈
PUSH ECX
MOV ECX, offset Top_User_Stack ; 用户栈指针
PUSH ECX
PUSHF
OR SS: dword ptr[ESP], Int_Flag ; 启用中断
MOV ECX, User_Code_Sel      ; 用户任务的代码段
PUSH ECX
PUSH EAX                    ; 用户任务开始地址的偏移

```

****调用用户任务** 【代码 15】**

```

用 IRET 转入用户程序的执行
IRET

```

OS_Code_Seg_ENDS

; 16 位的引导程序安排在存储器的最高 4KB 地址, 系统复位时形成的地址 0FFFFFF000H 即指向此段程序

BootRom16 SEGMENT EO USE 16 ; 段地址 0FFFFFF000H

ORG 0F000H ; 偏移 0F000

Start 16:

****其他: CLI, DS, 空 IDT** 【代码 2】**

CLI ; 关中断

MOV AX, 0

MOV DS, AX ; 实模式下初始化数据段

ASSUME DS: OS_Data_Seg ; 置 IDT 为空, 使任何中断都引起系统停机

MOV word ptr pdescr[0], 0 ; 16 位表界限为 0

MOV dword ptr pdescr[2], 0 ; 32 位表界限为 0

LIDT pword ptr pdescr

****建立 GDT** 【代码 3】**

MOV EAX, offset GDT_Table

; 表项 0: 空描述符

MOV dword ptr [EAX], 0

MOV MOV dword ptr [EAX + 4], 0

ADD EAX, 8

; 表项 1: OS 代码段描述符

; Base = FFFF0000H, Limit = 0000EH

; G = 1, D/B = 1, P = 1, DPL=00, S = 1, E = 1, ED/C = 0, W/R = 1, A = 0

MOV word ptr [EAX], 0000EH ; Limit[15~0]

MOV word ptr [EAX + 2], 0000H ; Base[15~0]

MOV byte ptr [EAX + 4], 0FFH ; Base[23~16]

MOV byte ptr [EAX + 5], 09AH ; P = 1, DPL=00, S = 1, E = 1, ED/C = 0,
W/R = 1, A = 0

MOV byte ptr [EAX + 6], 0C0H ; G, D, Limit[19~16]

MOV byte ptr [EAX + 7], 0FFH ; Base[31~24]

ADD EAX, 8

; 表项 2: OS 数据段描述符

; Base = 00000000H, Limit = 0FFFFFFH

; G = 1, D/B = 1, P = 1, DPL=00, S = 1, E = 0, ED/C = 0, W/R = 1, A = 0

; 把整个数据空间映射到 OS 地址空间

MOV word ptr [EAX], 0FFFFFFH ; Limit[15~0]

MOV word ptr [EAX + 2], 0000H ; Base[15~0]

```

MOV byte ptr [EAX + 4], 00H          ; Base[23~16]
MOV byte ptr [EAX + 5], 092H        ; P = 1, DPL=00, S = 1, E = 0, ED/C = 0,
                                     W/R = 1, A = 0
MOV byte ptr [EAX + 6], 0CFH        ; G, D, Limit[19~16]
MOV byte ptr [EAX + 7], 00H        ; Base[31~24]
ADD EAX, 8
; 表项 3: LDT
; Base = (offset LDT1_Table), Limit = (LDT_Entries*8)
; G = 0, Type = LDT, DPL = 00
MOV word ptr [EAX], (LDT_Entries*8 - 1) ; Limit[15~0]
MOV EBX, offset LDT1_Table
MOV word ptr [EAX + 2], BX          ; Base[15~0]
SHR EBX, 16
MOV byte ptr [EAX + 4], BL          ; Base[23~16]
MOV byte ptr [EAX + 5], 082H        ; Type = LDT, P = 1, DPL = 00, S = 0
MOV byte ptr [EAX + 6], 000H        ; Limit[19~16]
MOV byte ptr [EAX + 7], 0BH         ; Base[31~24]
ADD EAX, 8
; 表项 4: 任务的 TSS
; Base = (offset TSS1_Table), Limit = (TSS_Size + FP_Save)
; G = 0, Type = 可用的 32 位 TSS, DPL = 00
MOV word ptr [EAX], (TSS_Size + FP_Save - 1) ; Limit[15~0]
MOV EBX, offset TSS1_Table
MOV word ptr [EAX + 2], BX          ; Base[15~0]
SHR EBX, 16
MOV byte ptr [EAX + 4], BL          ; Base[23~16]
MOV byte ptr [EAX + 5], 089H        ; Type = 可用的 32 位 TSS, P = 1, DPL = 00, S = 0
MOV byte ptr [EAX + 6], 000H        ; Limit[19~16]
MOV byte ptr [EAX + 7], 0BH         ; Base[31~24]
; 建立 GDT 伪描述符, 并将其装入 GDTR
MOV word ptr pdescr[0], (GDT_Entries*8 - 1)
MOV dword ptr pdescr[2], offset GDT_Table
LGDT pword ptr pdescr

```

****进入保护方式** 【代码 4】**

SMSW AX

OR AX, PE

LMSW AX ; 设置保护启用位

JMP Flush

Flush: JMP far ptr Start 32 ; 通过 GDT 跳转到 32 位的 OS 代码

****冷启动代码**【代码 1】**

ORG 0FFF0H

JMP Start 16

BootRom16 ENDS

END

2.3 学习自评

2.3.1 自测练习

一、填空题

1. 从 80286 到 Pentium 所有微处理器工作在保护模式时, 都允许操作第一个 (1) 以上的存储器。如同实模式一样, 这个扩展内存系统也通过 (2) 加 (3) 访问。不同的只是段地址不存放在段寄存器中。在保护模式下, 段地址存放在由段寄存器选择的 (4) 中。

2. 保护模式下, 描述符包含 (1)、(2) 和 (3) 等。80386 及更高档型号的微处理器使用 (4) 位基地址, 允许存储器段起始于 (5) 存储区域的任何地方。

3. 标志位控制微处理器的 INTR 引脚的有效请求在指令后边界能否得到响应。

4. 堆栈存储器由 (1) 段寄存器和 (2) 寄存器的组合寻址。

5. 如果保护模式下的 DS = 0103H, 则请求优先级是 。

6. 在 Pentium 微处理器中, 全局描述符表的最大长度是 。

7. 32 位微处理器用 位的位移量转移到 4GB 代码段内的任何位置。

8. 如果 JMP 指令存储在当前代码段内的 100H 地址, 那么, 要将它转移到当前代码内的 200H 地址, 不可以用 转移。

9. 80486 微处理器中采用 校验。

10. 80486 只有一条流水线, 而 Pentium 具有 (1)、(2) 两条流水线,

能够执行 2 条 (3) 指令, 同一时刻二者可以 (4)。

11. 80486 片内有 8KB Cache, Pentium 则有 (1) Cache, 且将 (2) Cache 与 (3) Cache 完全分开, 各为 (4) Cache。

12. Pentium 的非屏蔽中断 (1) 通常由 (2) 引发的, 而可屏蔽中断 (3) 则可通过 (4) 中的中断允许标志 IF 实施封锁。

13. 与处理器外部引发的硬件中断不同, (1) 与 (2) 是由处理器内部产生的。前者又可分为三种类型 (3)、(4) 和 (5); 而后者是由 (6) 指令引起的中断, 可分为 (7) 与 (8) 两种。

14. Pentium 的段选择符中有 (1) 位具有地址属性, 供选择不同的段。若偏移量有 32 位地址属性, 则虚拟目标地址共有 (2) 位地址属性, 虚拟地址空间可以达到 (3)。

15. 保护模式下段选择符具有 (1) 位, 用 (2) 位作为 GDT 或 LDT 中描述符索引值, 用 (3) 位作为表指示器, 当值为 (4) 时表示 GDT, 为 (5) 时表示 LDT。最低的 (6) 位是请求特权级 RPL。

16. 当切换到保护模式时, 操作系统或者 ROM BIOS 必须组织存储器中的表格, 至少也得对 (1) 寄存器与 (2) 寄存器进行初始化。

17. Pentium 可以通过 (1) 和 (2) 两种不同的策略来实现对 I/O 地址空间的保护。

18. 虚拟 8086 模式本质上不是一种独立的工作模式, Pentium 此时实际上是操作于 (1) 下, 只是 (2) 与 (3) 的线性地址适应 8086 或者 Pentium 的 (4)。

19. 当控制寄存器 CR₄ 中的 VME 标志位为 (1) 时, Pentium 的虚拟 8086 模式是完全兼容 80386/80486 的一种模式。当为 (2) 时, Pentium 操作于一种新的增强型虚拟 8086 模式, 此时标志寄存器中的 (3) 与 (4) 两个标志位就会起作用。

20. Pentium 主要是通过 (1) 和 (2) 来实现多处理器系统的。

二、单项选择题

1. 80386/80486 工作在保护虚拟地址方式时, 所具有的虚拟地址空间是由 () 决定的。

- A. 地址总线
- B. 页式存储单元是否启动
- C. 段式存储管理单元产生的线性地址空间

2. 80486 与 80386 相比, 其主要改进之一是 ()。

- A. 指令执行单元采用了 RISC 结构, 且指令流水线增加到 6 级
- B. 片内含 Cache, 且指令执行单元也采用了流水线结构
- C. 增加了页式存储管理部件, 虚拟存储空间更大

3. 下列指令中, 唯一不含有非法操作数寻址的指令是 ()。
 - A. ADD [BX], [2000H]
 - B. ADC [SI][DI], AX
 - C. SUB [2000H], 2000H
 - D. SBC AX, EX
4. 若 $AX = 65ACH$, $BX = B79EH$, 则下列对指令执行结果叙述正确的是 ()。
 - A. 执行 ADD AX, BX 后, $CF = 1$, $OF = 1$
 - B. 执行 SUB AX, BX 后, $SF = 1$, $OF = 0$
 - C. 执行 TEST BX, AX 后, $OF = 0$, $CF = 0$
 - D. 执行 XOR AX, BX 后, $PF = 1$, $IF = 0$
5. 如果有多个中断同时发生, 系统将根据中断优先级响应优先级最高的中断请求。若要调整中断事件的响应次序, 可以利用 ()。
 - A. 中断响应
 - B. 中断屏蔽
 - C. 中断向量
 - D. 中断嵌套
6. 当 80486 工作在实模式下时, 已知中断类型为 14H, 则它的中断向量存放在存储器的向量单元 () 中。
 - A. 00051H~00054H
 - B. 00056H~00059H
 - C. 0000: 0050H~0000: 0053H
 - D. 0000: 0056H~0000: 0059H
7. 80386/80486 在保护模式下工作时, 虚拟地址向物理地址的转换都是 () 完成的。
 - A. 先经段表转换, 再经两级页表转换
 - B. 经段表转换
 - C. 经段表转换或先后经段表和两级页表转换
8. 一个 PC 机, 其 CPU 采用 80486, 系统总线采用具有 16 位数据线, 24 位地址线的 ISA, 它可安装的物理存储器的最大容量为 ()。
 - A. 1MB
 - B. 4MB
 - C. 16MB
 - D. 4096MB
9. 80486 总线周期定义信号主要有三个: $\overline{W/R}$ 、 $\overline{D/C}$ 、 $\overline{M/\overline{IO}}$ 。在存储器数据读取周期中, 应有 ()。
 - A. $\overline{M/\overline{IO}} = 1$ 、 $\overline{D/C} = 1$ 、 $\overline{W/R} = 0$
 - B. $\overline{M/\overline{IO}} = 1$ 、 $\overline{D/C} = 0$ 、 $\overline{W/R} = 0$
 - C. $\overline{M/\overline{IO}} = 1$ 、 $\overline{D/C} = 0$ 、 $\overline{W/R} = 1$
10. 80486 的引脚 $\overline{BE}_3 \sim \overline{BE}_0$ 用于指明总线操作时访问 $A_{31} \sim A_2$ 所指 4B 中的哪些字节。对于字对准的 16 位数操作, 这 4 个引脚信号的状态为 ()。
 - A. $\overline{BE}_3 \sim \overline{BE}_0 = 0$
 - B. $\overline{BE}_3 \sim \overline{BE}_0 = 1$
 - C. $\overline{BE}_3 = \overline{BE}_2 = 1$, $\overline{BE}_1 = \overline{BE}_0 = 0$ 或 $\overline{BE}_3 = \overline{BE}_2 = 0$, $\overline{BE}_1 = \overline{BE}_0 = 1$

三、问答题

1. 微处理器在实模式下, 对于下列 CS: IP 组合, 找出要执行的下条指令的

存储器。

- (a) CS = 1000H 和 IP = 2000H
 - (b) CS = 2000H 和 IP = 1000H
 - (c) CS = 2300H 和 IP = 1A00H
 - (d) CS = 1A00H 和 IP = B000H
 - (e) CS = 3456H 和 IP = ABCDH
2. 在 32 位微处理器中, 哪个或哪些寄存器被用于存放串指令的目标地址?
3. 假定 32 位微处理器工作在实模式下, (DS) = 1000H, (SS) = 2000H, (SI) = 007FH, (BX) = 0040H, (BP) = 0016H, 变量 TABLE 的偏移地址为 0100H。请指出下列指令的源操作数字段是什么寻址方式? 它的有效地址 (EA) 和物理地址 (PA) 分别是多少?
- (a) MOV AX, [1234H]
 - (b) MOV AX, [TABLE]
 - (c) MOV AX, [BX+100H]
 - (d) MOV AX, TABLE [BP] [SI]
4. 一个 Pentium 微处理器的描述符中含有基址 00280000H, 界限 00010H, 并且 G=1, 由这个描述符寻址的起始地址和结束地址是什么?
5. 编写一个 32 位微处理器用的描述符, 用于描述从 03000000H 单元开始至 05FFFFFFH 单元结束的存储器段, 该段是向上增长并且可写的数据段。
6. 如何访问在存储器中的局部描述符表?
7. 什么是程序不可见寄存器?
8. 如果微处理器将线性地址 00200000H 送到分页机制, 哪个页目录项被访问? 哪个页表项被访问?
9. 32 位微处理器中 TLB 的用途是什么?
10. 列出 32 位微处理器寄存器寻址使用的 32 位寄存器。
11. 为下面的每个任务选择指令:
- (a) EBX 复制到 EDX。
 - (b) BL 复制到 CL。
 - (c) SI 复制到 BX。
 - (d) DS 复制到 AX。
 - (e) AL 复制到 AH。
12. 指令 MOV [BX], [DI] 的错误是什么?
13. 说明 MOV BX, DATA 和 MOV BX, OFFSET DATA 指令之间的区别。
14. MOV AL, [BX][SI] 指令中有错误吗? 如果有, 请说明它的错误是什么?
15. 给定 DS = 1100H, BX = 0200H, LIST = 0250H 和 SI = 0500H。假设按实

模式操作，确定下面每条指令寻址存储器的地址。

(a) MOV LIST[SI], EDX

(b) MOV CL, LIST[BX+SI]

(c) MOV CH, [BX+SI]

16. 段间转移和段内转移之间的区别是什么？

17. 在 Pentium 微处理器中哪些指令把 EFLAGS 存放到堆栈中？

18. 为了使用 16 位指令模式，Pentium 微处理器中的指令是怎么构造的？

19. 写出求 EAX、EBX、ECX 和 EDX 之和的过程。如果出现进位，将逻辑 1 放入 EDI，否则将 0 放入 EDI。程序执行以后，结果要放在 EAX 中。

20. 简要描述 8080~Pentium 4 指令系统的发展。

21. 解释 Pentium 的两个整型单元如何同时执行两个不相关的指令。

22. 什么是 Pentium 的系统存储器管理工作模式。

23. 将 Pentium 新增加的超标量结构和分支预测功能与 80486/80386 比较，有什么具体的改进？

24. 描述 Pentium 如何访问 4MB 页？

25. Pentium 新增加了哪些寄存器？在 Pentium 寄存器中又增加了哪些新的功能位？

26. Pentium 的中断输入引脚是什么？它们分别有哪些功能？优先级如何？

27. 描述 Pentium 在实模式下执行中断操作的具体过程。

28. 就寻址方法而言，试述保护模式和实模式的区别。

29. 说明实模式、保护模式、虚拟 8086 模式之间的区别，并描述三者之间的转换过程。

30. 假设执行指令将寄存器 EAX 中的值写入主存储器：

MOV FS: ABCDEF, EAX

这里的 16 位段寄存器 FS 中存放着段选择器，ABCDEF 则是 32 位的偏移量，试描述寻址的具体步骤（结合 GDT，LDT，并考虑越界的情况）。

31. 相对于实模式来说，保护模式具有更多的异常中断，试简单描述在保护模式下新增加的异常。

32. 32 位微处理器在 16 位微处理器描述符和选择符的基础上做了哪些改进？其主要区别是什么？

33. 阐述保护模式中断与实模式中断之间的区别。

34. Pentium 在保护模式下主要的保护机制有哪些？试分别加以说明。

35. 试比较存储器读/写数据等待周期的差别。

36. TSS 段中的 I/O 许可位图为 11111111 10101101 01001100 10001001 10100000，试分析 8 位、16 位、32 位 I/O 端口的访问属性。

37. 在有 Cache 的系统中, 通写法、回写法是针对什么问题而提出来的? 他们的含义是什么?

38. 在 Pentium 突发周期中起主要作用的是哪几种信号? 它们分别有什么功能?

39. 为什么会产生等待状态? Pentium 是如何解决这个问题的?

40. 流水线访问的优势是什么? 它在什么情况下的效率更高? 请分析其原理。

2.3.2 自测练习答案

一、填空题

1. (1) 1MB (2) 段地址 (3) 偏移量 (4) 段描述符
2. (1) 基地址 (2) 界限 (3) 访问权限字节 (4) 32 (5) 4GB
3. EFLAGS 寄存器中 TF
4. (1) 堆栈 (2) 堆栈指针
5. 3
6. 64KB
7. 32
8. 短
9. 偶
10. (1) U (2) V (3) 整数 (4) 并行操作
11. (1) 16KB (2) 指令 (3) 数据 (4) 8KB
12. (1) NMI (2) 严重的硬件错误 (3) INTR (4) 复位 EFLAGS 寄存器
13. (1) 异常中断 (2) 软件中断 (3) 陷阱 (4) 故障 (5) 中止
(6) INT n (7) BIOS 中断 (8) DOS 中断
14. (1) 14 (2) 46 (3) 64TB
15. (1) 16 (2) 13 (3) 1 (4) 0 (5) 1 (6) 2
16. (1) GDTR (2) IDTR
17. (1) 标志寄存器中的 IOPL 标志
(2) 任务状态段 TSS 中的 I/O 位图措施
18. (1) 保护模式 (2) 段寄存器 (3) 偏移量 (4) 实模式
19. (1) 0 (2) 1 (3) VIP (4) VIF
20. (1) MESI 协议 (2) 锁定的总线周期

二、单项选择题

1. B 2. B 3. C 4. C 5. B 6. C 7. C 8. C 9. A 10. C

三、问答题

1. (a) 12000H (b) 21000H (c) 24A00H (d) 25000H (e) 3F12DH
2. ES:DI 寄存器组合被用于存放串指令的目标地址。
3. (a) 直接寻址, EA = 1234H, PA = 11234H
(b) 直接寻址, EA = 0100H, PA = 10100H
(c) 基址寻址, EA = 0140H, PA = 10140H
(d) 带位移的基址加变址寻址, EA = 0195H, PA = 20195H
4. 基地址为 00280000H, 结束地址为 00290FFFH。
- 5.

03	C0H	6
92H	00	4
00	00	2
2F	FF	0

6. 通过局部描述符寄存器 LDTR 定位 LDT 定位 LDT, 通过选择符中 TI = 1 和 Index 值来访问 LDT 和其中的描述符。
7. 对程序不可见的寄存器是指段寄存器、任务寄存器和局部描述符表寄存器在段 Cache 区的段描述符寄存器、TSS 描述符寄存器和 LDT 描述符寄存器。
8. 页目录 000H 和页表项 200H。
9. 利用 TLB 有可能实现近期访问过的程序或数据的线性地址到物理地址的快速变换。
10. 这样的 32 位寄存器有 EAX、EBX、ECX、EDX、ESP、EBP、ESI 和 EDI。
11. (a) MOV EDX, EBX (b) MOV CL, BL
(c) MOV BX, SI (d) MOV AX, DS (e) MOV AH, AL
12. 存储器到存储器的传输是非法的, 除了 MOVS 指令以外。
13. MOV BX, DATA 将位于 DATA 存储单元的内容复制到 BX, 而 MOV BX, OFFSET DATA 指令则将 DATA 的偏移地址复制到 BX。
14. 没有这种指令的替换形式, 应为: MOV AL, [BX + SI]。
15. (a) 11750H (b) 11950H (c) 11700H
16. 段间转移是一个远转移, 允许转至存储器系统内的任意位置, 而段内转移是一个近转移, 只允许访问当前代码段内的任何位置。
17. PUSHFD
18. 在完整的段定义 SEGMENT 伪指令后放一 USER16 伪指令, 在其他的存储模型语句后加一个 .486 开关来选择 16 位指令模式。
19. MOV EDI, 0
ADD EAX, EBX

```
.IF CARRY?
    MOV EDI, 1
.ENDIF
ADD EAX, ECX
.IF CARRY?
    MOV EDI, 1
.ENDIF
ADD EAX, EDX
.IF CARRY?
    MOV EDI, 1
.ENDIF
```

注 以句点开始的控制程序流程的语句为 MASM 指示特殊的汇编语言命令，仅适合于 MASM 6.X 版本，不能用于早期版本。

20. 见本书的 2.1.3 小节。

21. Pentium 有 U 和 V 两条整型流水线，每条都由 5 级组成。主流水线 U 可以执行 80X86 的全部指令，包括微代码形式的复杂指令；而 V 流水线则只能执行简单的整型指令与浮点部件指令 FXCHG。为了使两条流水线中同时执行的两条指令能够同步协调操作，这两条指令必须配对，也就是说两条流水线同时操作是有条件的，指令的配对必须符合一定的规则。

22. 系统存储管理模式是由 Pentium 存储器管理中断输入访问的一个特殊模式。

23. 运用超标量结构的 Pentium 微处理器由三个执行单元组成，一个执行浮点指令，而另两个（U 流水线和 V 流水线）执行整型指令，这意味着 Pentium 可同时执行 2 条指令。而采用分支预测逻辑可以减少由于分支导致的时间消耗。

24. 见本书的 2.1.6 小节的 (2)。

25. 见本书的 2.1.2 小节和图 2.1。

26. 见本书的图 2.3。

27. 见教材《微机系统与接口技术》2.5.4 小节的图 2.21。

28. 实模式只是简单地将段基址乘以 16，然后加上偏移量地址。而保护模式则要复杂很多，必须通过段管理部件将段寄存器和有效地址组成的逻辑地址转换成线性地址和多种保护性检查。在允许分页的情况下，段部件转换后的线性地址还不是物理地址，还必须经过页变换。

29. 见本书图 2.2。

30. 第一步：通过 FS 段寄存器中段选择器的 T1 位可以判定是访问全局描述符表还是局部描述符表。

第二步：通过 GDTR 或 LDT 描述符寄存器可以读出相应描述符表的 32 位基地址。

第三步：将段选择符中的索引值乘以 8 后与描述表的基地址相加即指向所需的段描述符的首地址，判断该地址是否超出了界限，如果超出，则发 13 号异常中断，表示出现“一般保护错误”。

第四步：如果未越界，则从描述表中读出由 8B 组成的段描述符，从而获取了该段的基地址、界限与属性。

第五步：通过地址加法器将这个 32 位的基地址与 32 位的偏移量相加得到 32 位的存储器地址，判断该地址是否超出了指定的界限，如果超出，则发 13 号异常中断表示出错。

第六步：如果未越界，则该地址就是寻址目标的线性地址。当未使用页功能时，该线性地址就是存储器的物理地址。

31. 双重故障（异常 8）、非法的任务状态段（异常 10）、段不可用（异常 11）、页故障（异常 14）、对齐检查（异常 17）、机器检查（异常 18）。

32. 16 位和 32 位微处理器之间的主要区别在于后者增加两个字节：一个字节用于扩展段基址到 32 位；另一个字节定义了一些属性如 G 和 D/B，增加了 4 位段界限，真正的段界限由 G 位属性和 20 位段界限组成。32 位微处理器的描述符使用 32 位的段基址和 32 位或 20 位的界限，而不是 16 位微处理器中的 24 位段基址和 16 位界限。

33. 主要区别是中断向量的位置不同，实模式中中断使用的向量存于存储器底部 1KB 的中断向量表中，而保护模式下中断使用的描述符可以处在存储器系统中任意指定的中断描述符表 IDT 中。另一个区别是保护模式中中断服务程序受各种保护机制保护。

34. Pentium 在保护模式下的保护机制主要用来读取与控制 CPU 状态，或者控制访问代码段与数据段的指令。一般采用以下三种保护机制：限制对段的访问；限制对段的使用；特权指令。

35. 写数据的等待周期与读数据的有所不同。写操作只是传给存储器控制器地址与数据字节信号，然后存储器控制器就从数据总数上获取数据而独立执行写操作。Pentium 则不用等待与监视低速存储器写操作的过程，可以执行其他的操作；而读数据时，CPU 只能等待内存存储器送出数据并完成读取过程，不能执行其他的操作。

36. 对于 8 位端口访问：在 $CPL > IOPL$ 时不能访问的端口有 5, 7, 8, 11, 15, 18, 19, 22, 24, 26, 27, 29, 31；对于端口 0, 1, 2, 3, 4, 6, 9, 10, 12, 13, 14, 16, 17, 20, 21, 23, 25, 28, 30 则任何特权的程序都可以访问。

对于 16 位端口访问：在 $CPL > IOPL$ 时不能访问的端口有 4, 6, 8, 10, 14,

18, 22, 24, 26, 28, 30; 对于端口 0, 2, 12, 16, 20 则任何特权的程序都可以访问。

对于 32 位端口访问: 在 $CPL > IOPL$ 时不能访问的端口有 4, 8, 12, 16, 20, 24, 28; 对于端口 0, 任何特权的程序都可以访问。

37. 通写法是多数 Cache 采用的一种最简单的方法, 这种策略下的 CPU 写操作总会导致向内存存储器中传送数据, 相当于 CPU 与内存存储器之间是穿透直通的。回写法则只写 Cache, 而不写内存存储器, 仅当 Cache 数据要被替换时才写内存存储器。

38. Pentium 依据 $\overline{BRDY}$ 、 $\overline{CACHE}$ 、 $\overline{KEN}$ 信号就可以决定是否要执行突发周期。 $\overline{BRDY}$ 是突发式操作的 $\overline{RDY}$ 信号, Pentium 缓存式总线周期都是突发式周期。Pentium 使用低电平有效的 $\overline{CACHE}$ 信号通知存储器子系统寻址片内 Cache, 执行 Cache 操作, 如果存储器子系统发出 $\overline{KEN}$ 低电平有效信号, 则 Pentium 就会进入 Cache 行填充操作; 当 $\overline{CACHE}$ 有效、 $\overline{KEN}$ 无效时进行突发式回写操作。

39. 如果存储器或外部设备在 T_1 和 T_2 两个时钟周期内还不能完成一次读/写请求, 那么存储器控制器就会将 $\overline{BRDY}$ 信号保持为高电平, 通知 Pentium 还要增加一个 T_2 时钟周期, 以便给存储器或外部设备更多的时间去完成操作。这就是通常所说的等待状态。

40. 流水线访问能增加总线带宽和数据传输量。当存储器的访问是一个接一个地顺序执行时, 流水线访问才具有优势。微处理器在数据传输逻辑电路忙于数据读/写的同时将下一个地址传送至译码器, 这样译码逻辑与地址缓冲器就能预先工作, 其子系统的访问就是交叠的, 即在当前总线周期结束之前地址逻辑就已接收的新的地址。

2.4 教学建议

本章是全书的重点和难点, 在课时分配上约占全学期课时的 1/3 左右。为了完整地、系统地讲述微机系统与接口技术, 全书分为 9 章。其中第 1 章概论和第 9 章微计算机系统内容对普通高等教育本科专业学生可以采用自学方式, 不在课堂上讲授; 需要讲授的是 80486/Pentium、8254、8255、8250、8259、8237 和内存存储器、总线技术、交互接口、外存储器。对于本章 2.3 节的内容, 指令系统是供学习时使用的, 可不在课堂上讲授。

对于 32 位微处理器的内部结构, 80486 要讲清各部件的功能和相互关系, Pentium 主要说明它在体系结构上新的特点。对寄存器组要说明 32 位 CPU 和 16 位 CPU 在寄存器种类、数据、长度和功能等方面的区别, 重点说明用于保护特性

的系统地址寄存器和控制寄存器。中断异常着重说明保护模式下各种中断和异常。存储管理讨论在实模式、保护模式和虚拟 8086 模式下存储管理的情况。多任务应阐明任务设置、任务转换过程和任务转换检查等方面的内容。高速缓存讨论从 80486 到 Pentium 4 高速缓存的发展以及结构和功能的变化。80486 和 Pentium 的引脚应详细介绍 80486 各组引脚的功能后说明 Pentium 引脚对应 80486 各组引脚的保留、增加、除去或增强功能。总线周期说明 80486 和 Pentium 的主要总线周期时序、总线状态变化和它们的区别。

第 3 章 内存存储器

3.1 知 识 要 点

3.1.1 内存存储器

1. 只读存储器(ROM)

在线运行时其存储内容是非易失性的、只读的，信息通常在脱机状态下用电气方式写入。根据不同的信息写入方式，ROM 分为如下 4 种。

① 掩膜 ROM(Masked ROM, 简称 ROM)。信息已由生产厂家用掩膜技术写好，固化在存储器中，只能读出而无法改写。一般用于存放系统程序 BIOS 和用于微程序控制。

② 一次可编程 ROM(Programmable ROM, 简称 PROM)。由用户使用专用电子设备烧断微小的镍-铬丝或硅氧化物熔丝进行一次写入操作。

③ 可擦除可编程 ROM(Erasable Programmable ROM, 简称 EPROM)。存储内容既可以读出，也可以写入，但在每次写操作之前须用紫外线照射 15~20min 以擦去所有信息；可以多次写。

④ 电可擦除可编程 ROM(Electrically Erasable Programmable ROM, 简称 EEPROM)。信息可以读出和写入，写操作之前不需要将以前存储的内容擦去，能直接对寻址的字节或块进行修改，但其写操作所需时间远大于读操作所需时间；集成度较低。

2. 随机读写存储器(RAM)

运行时既可以读出信息，也可以写入信息，但断电后将丢失存储内容。根据存储信息机构的原理，它又可分为如下 2 种。

① 静态随机读/写存储器(Static RAM, 简称 SRAM)。利用数字电路中触发器的导通和截止状态来表示信息 0 和 1，其基本存储电路实际上是一种半导体双稳态触发器。在不断电的情况下存储信息能够一直保持而不会丢失。SRAM 存取速度快，集成度低。

(c) 动态随机读/写存储器(Dynamic RAM, 简称 DRAM)。利用 MOS 电容充电成高电位后表示 1 而放电后表示 0, 以此实现信息存储。由于电容存在漏电现象, 为保持信息, 必须进行周期性刷新。DRAM 的存储密度比 SRAM 高而成本低、功耗小、易于封装, 但存取速度比 SRAM 慢。目前出现许多新型的 RAM 技术, 如扩展数据输出 RAM(Extended Data Out RAM, 简称 EDORAM)、同步 RAM(Synchronous DRAM, 简称 SDRAM)、高速缓存 DRAM(Cached DRAM)。

3. 闪存(Flash Memory)

属于电可擦写的非易失性存储器, 采用从 EPROM 演化而来的 ETOX(EPROM Tunnel Oxide)技术, 其性能介于 EPROM 和 EEPROM 之间。闪存有 3 种标准的阵列结构: 整体擦除、自举块和快擦写文件。

3.1.2 DRAM的刷新

动态存储器采用“读出”方式进行刷新。MOS 存储单元在读出过程中恢复了 MOS 管栅极电容电荷, 保持原单元的内容, 所以读出过程就是刷新过程。通常, 在刷新过程中只改变行选地址, 每次刷新一行。依次对存储器的每一行进行读出, 就可以完成对整个 DRAM 的刷新。从上一次对整个存储器刷新结束到下一次对整个存储器全部刷新一遍为止, 这一段时间间隔称为刷新周期, 一般为 2 ms 或 4 ms。

常用的刷新方式有以下 3 种。

1. 集中式刷新

在整个刷新间隔内, 前一段时间为重复读/写或维持周期, 后一段时间暂停读/写或维持操作而逐行进行刷新。这种刷新方式较适用于高速存储器。

2. 分散式刷新

将一个存储系统周期分为两部分, 前一部分时间用于读/写操作或维持信息, 后一部分时间作为刷新操作时间。此刷新方式不存在停止读/写操作的死时间。

3. 异步式刷新

这是前两种方式的结合, 即在刷新周期内分散地把全部行刷新一遍。

3.1.3 半导体存储器性能

1. 存储容量

存储容量是指存储器可以容纳的存储单元总数, 一般以位或字节为单位。

存储器芯片容量 = 芯片的地址单元数 × 数据线位数

2. 存取时间

存取时间又称为存储器访问时间, 是指从启动一次存储器操作到完成该操作所经历的时间, 即从存储器接收到 CPU 发出的地址信息到读出的信息稳定地出现在数据总线上所需要的时间。

3. 存储周期

存储周期指连续启动两次独立的存储器操作所需的最小时间间隔。通常,存储周期略大于存取时间。存取时间和存储周期反映了存储器的速度指标。

3.1.4 存储器芯片的组成

存储器芯片通常由地址译码器、存储矩阵、读/写控制、片选控制以及三态数据缓冲器组成。

地址译码器的输入信息来自 CPU 子系统的地址锁存器。地址锁存器用来存放所要访问的存储单元的地址。中央处理器要选择某一存储单元,就在地址总线上输出此单元的地址信号给地址译码器。地址译码器将用二进制代码表示的地址转换成输出端的电位,来驱动相应的读/写电路,以便选择所要访问的存储单元。

根据存储矩阵的排列方式,可将存储芯片的组成为字结构和位结构两种。

① 字结构方式。在用这种方式组织的存储器中,一个存储单元存放一个位,每个字的各位都存储于同一个存储芯片上。此方式适用于小容量存储器。

② 位结构方式。将一个存储器芯片的各个基本存储单元排列成不同字的同一位,同一个字的不同位有相同的地址码。位结构方式的主要优点是引脚少且成本低,适合用于大容量存储器。

三态数据缓冲器用作 RAM 的数据 I/O 控制电路,以使 RAM 的数据输入/输出端能方便地挂接到数据总线上。

存储器控制电路通过 RAM 的外引线端,接收来自 CPU 或外部电路的控制信号,经过组合变换后,对存储矩阵、地址译码器及三态数据缓冲器进行控制。

3.1.5 存储器地址空间的硬件组织

1. 地址交叉编址方式

存储器实际上都是由多个存储模块组成的,可实现重叠与交叉存取。在 M 个模块上交叉编址($M=2^m$),称为模 M 交叉编址。设存储器包括 M 个模块,每个模块容量为 L ,各存储模块进行低位交叉编址,连续的地址分布在相邻的模块中,则第 i 个模块 M_i 的地址编号依次为 $M \times j + i$,其中 $j=0,1,2,\dots,L-1$; $i=0,1,2,\dots,M-1$ 。

这种编址方式使用地址码的低位字段经过译码选择不同的存储模块,而高位字段指向相应的模块内部的存储字。连续地址分布在相邻的不同模块内,而同一模块内的地址都不是连续的。对任何交叉存储器来说,使用效率与所产生的存储器地址的顺序密切相关。如果程序段和数据块连续地在存储器中存放和读取,则此编址方式将大大提高存储器的有效访问速度。 M 个交叉模块的实际使用率大约在 M 和 M 之间。

8086 系统的存储空间用奇偶地址单元交叉方式(模 2 低位交叉)编址:80386DX

系统的存储空间由 4 个独立字节宽度的存储体组成, 用模 4 低位交叉编址。

2. 32 位系统中字及双字的数据交换方式

(1) 对准字及对准双字方式

访问的字和双字数据的高 30 位地址是相同的。以此方式访问存储器只需要一个总线周期。

(2) 非对准字及非对准双字方式

访问的字和双字数据的高 30 位地址是不同的。以此方式进行存储器存取需要两个总线周期。

设所要访问字的字节号为 $a, a+1$ 。因 $(32-30)^2=4$, 如果 $a \text{ div } 4 = (a+1) \text{ div } 4$, 则该字为对准字, 否则为非对准字。式中, div 为取整除符号。

设所要访问双字的字节号为 $b, b+1, b+2, b+3$ 。如果 $b \text{ div } 4 = (b+1) \text{ div } 4 = (b+2) \text{ div } 4 = (b+3) \text{ div } 4$, 则该双字为对准双字, 否则为非对准双字。

3.1.6 PC/XT 存储器子系统

1. 行列地址选通信号生成电路

(1) 产生行选通信号的 3-8 译码器工作条件

① 存储器周期中读命令 $\overline{\text{MEMR}}$ 为低电平, 或者在存储器写周期中写命令 $\overline{\text{MEMW}}$ 为低电平, 使得译码器 G_1 端为高电平。

② 非刷新操作时, 8237 的 0 通道未执行 DMA 操作, $\overline{\text{DACK}_0\text{BRD}}$ 使译码器 $\overline{G_{2A}}$ 为低电平。

③ 当访问系统板上的 RAM 时, PROM 电路 24S10 的 Q_0 输出高电平, 在非刷新操作时, $\overline{G_{2B}}$ 成为低电平。

上述 3 个条件都满足时, 对输入信号译码, 产生行选通信号。

在刷新操作期间, $\overline{\text{DACK}_0\text{BRD}}$ 为低电平, 当 $\overline{\text{MEMR}}$ 信号来到时, 输出低电平, 使 4 个行选通信号同时有效, 对全部 DRAM 芯片执行刷新。

(2) 产生列选通信号的 3-8 译码器工作条件

① 不是刷新操作, 高电平的 $\overline{\text{DACK}_0\text{BRD}}$ 送到 G_1 端。

② 非刷新操作期间, 访问系统板上的 RAM, 使 $\overline{G_{2A}}$ 成为低电平。

③ 存储器读/写命令使 $\overline{G_{2B}}$ 成为低电平。

在存储器读周期里, 存储器读信号经过延迟线 TD_1 延迟 100 ns, 因而列选通信号落后行选通信号 100 ns。在存储器写周期里, 存储器写命令 $\overline{\text{MEMW}}$ 使用的是 8288 发出的超前写命令 $\overline{\text{AMWC}}$, 该命令比滞后的写命令 $\overline{\text{MWTC}}$ 大约提前 200 ns, 因此列选通信号落后行选通信号 200 ns。

RAM 读/写时序关系参见表 3.1。RAM 的读/写控制如图 3.1 所示。

表 3.1 读/写时序关系

0 ns	60 ns	100 ns 或 200 ns
$\overline{\text{MEMR}} / \overline{\text{MEMW}}$ 有效		$U_{69}-2$ 有效 (R/DMA 等) 或 $U_{69}-1$ 有效 (W)
$U_{56}-G_1=1$		$U_{42} \sim \overline{G_{2B}}=0$
$\text{RAS}_1=0$		$\overline{\text{CAS}}_1=0$
$\text{ADDRSEL}=0$	$\text{ADDRSEL}=1$	$\text{ADDRSEL}=1$

U_{19}/U_{40} — 选

74 LS158 $S=0$ $\overline{G}=0$ 锁存 A 组

74 LS158 $S=1$ $\overline{G}=0$ 锁存 B 组

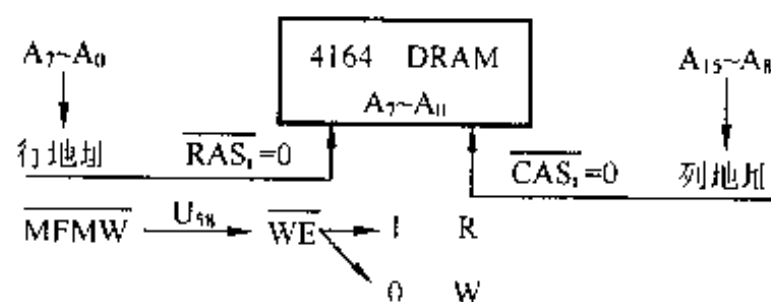


图 3.1 RAM 的读/写控制

2. RAM 电路

RAM 电路如图 3.2 所示。

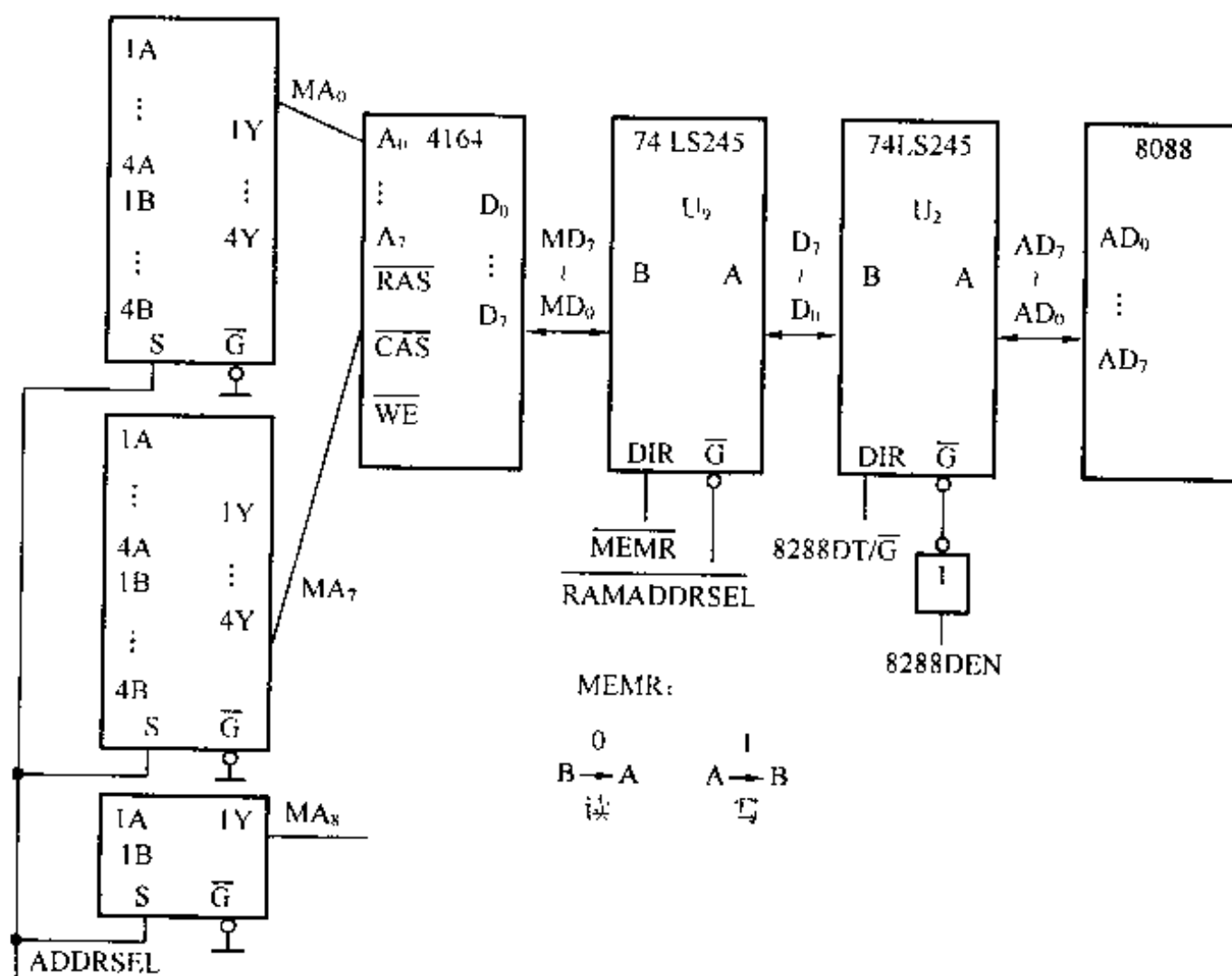


图 3.2 RAM 的数据读写

3. 奇偶校验电路

奇偶发生/校验器 74S280 有 9 个数据输入端, 当输入的 9 位数据中“1”的个数是奇数时, 输出端 ODD 为高电平, EVEN 端为低电平; 当输入的 9 位数据中“1”的个数是偶数时, 输出端 EVEN 为高电平, ODD 端为低电平。EVEN 端接到奇偶校验 RAM 芯片的数据输入端 D, 控制写入奇偶校验位的数据。ODD 端作为奇偶校验的结果输出, 出现奇偶校验错时 ODD 端输出“0”, 访问系统板 RAM 时 $\overline{\text{RAMADDRSEL}}$ 为低电平, 当 $\overline{\text{MEMR}}$ 信号结束时, 电平由低跳到高, 把奇偶校验的结果打入奇偶校验触发器, 输出的 PCK 信号传送到 8088 的 NMI 控制电路, 在允许 NMI 时向 8088 申请非屏蔽中断。PCK 信号送到 8255A-5 的 PC₇ 端, 8088 可以通过读 8255A-5 的 PC 口判断系统板上 RAM 的奇偶校验状态。

3.1.7 奔腾机存储器子系统

奔腾机存储器子系统由 DRAM 数据存储阵列、 $\overline{\text{RAS}}/\overline{\text{CAS}}$ 地址多路转换电路、数据总线收发器以及控制逻辑电路组成, 支持流水线式和突发式总线周期, 可以进行页式和存储体交替式访问, 支持对高速缓存的回写修改。

系统中的 DRAM 存储阵列采用交替方式组织, 分为两个相互独立的存储体, 分别存储偶地址和奇地址, 可实现对两个存储体的交替式访问, 从而使系统获得较高的效率。

地址多路转换电路由透明锁存器、2 路到 1 路转换器、缓冲器、突发式地址逻辑等部件构成。地址位 $A_5 \sim A_{12}$ 用于产生列地址, 地址位 $A_{14} \sim A_{21}$ 用于产生行地址, 由行选信号决定输出到缓冲器中的是行地址还是列地址。

数据总线收发器电路所用器件为双向寄存式收发器。CPU 的数据线 $D_0 \sim D_{63}$ 和校验线 $DP_0 \sim DP_7$ 分别通过一组收发器与存储体 bank₀、bank₁ 相连。

存储器子系统的控制逻辑电路接收 CPU 的总线周期标志信号和其他控制信号, 输出多路转换电路、数据总线收发器和 DRAM 阵列的控制信号以及 CPU 的存储器总线控制信号。

3.2 例题精析

例 3.1 判断下列说法是否正确。

- (1) 只要地址译码正确, CPU 就能正确地访问存储器。
- (2) 部分地址译码法的电路复杂, 但没有浪费地址空间; 全译码法的译码电路简单, 却要浪费地址空间。
- (3) CPU 与存储器连接时, 只需要考虑负载、译码、控制线问题就足够了。

(4) 在 RAM 与 CPU 连接时, RAM 的存取速度高于 CPU 的存取速度, 因此构成系统时, 任意一块 RAM 芯片都可以直接与 CPU 相连。

(5) CPU 的驱动能力足够大, 因此考虑负载问题是多余的。

(6) 存储器的最大存取时间是从数据线上出现有效数据信息到数据总线上的有效数据信息消失这段延迟时间。

(7) 在选择存储器时, CPU 的存取时间要小于存储器的最大存取时间。

解 (1) 错。CPU 能否正确访问存储器除了要求地址译码正确外, 还需要二者工作速度相匹配等条件。

(2) 错。应为部分译码法电路简单, 浪费地址空间; 全译码法电路复杂, 但不浪费地址空间。

(3) 错。除了负载、译码、控制线外, 还需要考虑二者工作速度匹配与否。

(4) 错。只有存取速度高于 CPU 给定存取速度的 RAM 方可与该 CPU 直接连接。

(5) 错。8086CPU 的总线负载能力有限, 一般为一个 TTL 器件或 20 个 MOS 器件。所以当总线上挂接的器件较多时, 一定要考虑是否超载。

(6) 错。存取时间为存储器从接收到 CPU 发出的地址信息到读出的信息稳定地出现在数据总线上所经历的时间。

(7) 错。CPU 的存取时间要大于存储器的最大存取时间, 否则它将无法进行正常工作。

例 3.2 某一 RAM 芯片内部采用两个 64 选 1 的地址译码器, 并且有一个数据输入端和一个数据输出端。试问该 RAM 芯片的容量及内部存储矩阵的阵列格式。

解 根据题意, 该 RAM 芯片可寻址的范围为 $64 \times 64 = 4096$, 数据为 1b (bit, 位), 因此, 其芯片容量为 4096×1 位, 芯片内部应排列成 64×64 的矩阵。

例 3.3 现有 SRAM 芯片若干, 芯片容量为 512×4 位, 欲组成 $16K \times 8$ 位的静态存储器, 需要多少芯片? 需要多少位地址作为片内地址选择端? 多少位地址作为芯片选择端?

解 设已知芯片容量为 $n_1 \times m_1$, 其中 n_1 为单片寻址范围, m_1 为数据线位数。待求存储器总容量为 $N \times M$ (N 为存储器寻址范围, M 为存储器数据线位数), 则先要组成一个芯片组 $n_1 \times M$, 使数据线位数达到要求, 所需芯片数为 M/m_1 , 此时组成 $N \times M$ 容量存储器所需芯片组数为 N/n_1 。因此, 所需芯片总数应为 $(N/n_1) \times (M/m_1)$ 。

单片容量给定后, 片内地址线数目应为 $a_1 = \log_2 n_1$, 而 RAM 总的地址线数目为 $a_0 = \log_2 N$, 则用于片选信号的地址线数为 $a_2 = a_0 - a_1$ 。

欲组成题中存储器, 需要芯片数为 $(16K/512) \times (8/4) = 64$ 。RAM 地址线数目

为 $a_0 = \log_2 16K = 14$, 片内地址线数目为 $a_1 = \log_2 512 = 9$, 片选地址线数目为 $a_2 = a_0 - a_1 = 5$ 。

例 3.4 一台 8 位微机的地址总线为 16 条, 其 RAM 存储器容量为 32KB, 采用全译码法, 其首地址为 4000H, 且地址是连续的。问可用的最高地址是多少?

解 32KB 存储空间占用地地址线条数为 $\log_2 32K = 15$, 如果存储地址起始单元为 0000H, 则其范围应为 0000H ~ 7FFFH ($2^{15} - 1 = 7FFFH$)。题中首地址为 4000H, 则所求最高地址应为 $4000H + 7FFFH = BFFFH$ 。

例 3.5 现有 $16K \times 1$ 位的动态存储器芯片若干, 构成 $64K \times 8$ 位的存储器。该存储器读/写周期均为 $0.5 \mu s$, CPU 在 $1 \mu s$ 内至少访问一次。试问采用哪种刷新方式比较合理? 两次刷新的最大时间间隔是多少? 对全部存储单元刷新一遍所需的实际刷新时间是多少?

解 根据题目条件, CPU 在 $1 \mu s$ 内至少访问一次内存, 所以整个存储器的平均读/写周期与单个存储芯片的读/写周期相接近, 应采用集中式刷新比较合理, 因为集中刷新适合于高速存储器。

动态存储器单元两次刷新的最大时间间隔时间不应超过 2ms。

DRAM 芯片为 $16K \times 1$ 位, $16K = 2^{14} = 2^7 \times 2^7 = 128 \times 128$, 内部存储矩阵为 128 行 $\times$ 128 列, 刷新工作是按行进行的, 故对存储器内所有芯片的 128 行进行刷新, 刷新时间需要 128 个读/写周期, 所需的刷新时间为 $0.5 \mu s \times 128 = 64 \mu s$ 。

例 3.6 有一个 $16K \times 16$ 位的存储器, 由 $1K \times 4$ 位的动态 RAM 芯片构成(芯片内结构是 64×64 结构)。问:

(1) 所需 RAM 芯片总数是多少?

(2) 若采用异步刷新方式, 单元刷新周期不超过 2ms, 则刷新信号周期是多少?

(3) 若采用集中刷新方式, 存储器刷新一遍最少用多少读/写周期, 死时间率是多少?

解 (1) 所需 RAM 芯片总数为 $(16K/1K) \times (16/4) = 64$ 。

(2) 采用异步刷新方式, 在 2ms 内分散地把所有存储单元的 64 行刷新一遍, 故刷新信号的时间间隔为 $2ms/64 = 31.25 \mu s$, 可取刷新信号周期为 $30 \mu s$ 。

(3) 采用集中刷新方式, 设读/写周期为 T (单位: μs), 所有存储单元刷新一遍(64 行)需要 $64T$, 则死时间率 $= 64T / 2ms = 100\% \times 64T / 2000$ 。

例 3.7 图 3.3 所示的为 SRAM 的写入时序图, 其中 $R/\overline{W}$ 是读/写命令控制线, 当 $R/\overline{W}$ 线为低电平时, 存储器按给定地址把数据线上的数据写入存储器。请指出写入时序中的错误, 并画出正确的写入时序图。



图 3.3 例 3.7 图 (1)

解 写入存储器的时序信号必须同步。通常,当 $\overline{R/\overline{W}}$ 线加负脉冲时,地址线和数据线的电平必须是稳定的。当 $\overline{R/\overline{W}}$ 线达到低电平时,数据立即被存储。因此,当 $\overline{R/\overline{W}}$ 线处于低电平时,如果数据线改变了数值,则存储器将存储新的数据。同样,当 $\overline{R/\overline{W}}$ 线处于低电平时,如果地址线发生了变化,数据将被存储到新的地址。题中的错误在于 $\overline{R/\overline{W}}$ 处于低电平写有效时,地址线发生了两次变化,数据线发生了一次变化。正确的时序如图 3.4 所示。

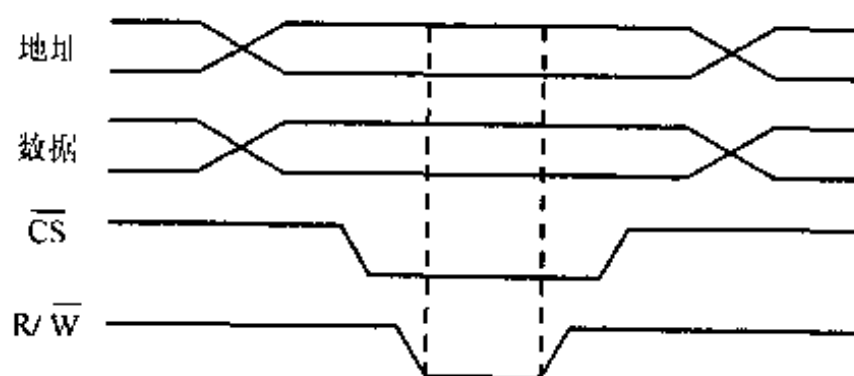


图 3.4 例 3.7 图 (2)

例 3.8 在由 80386DX 微处理器构成的系统中,访问首地址为 2001 的双字需要多少个总线周期?

解 被访问的双字数据由地址为 2001、2002、2003、2004 的 4 个存储单元组成:

$$2001 \div 4 = 2002 \div 4 = 2003 \div 4 = 500$$

而

$$2004 \div 4 = 501$$

由此可知该双字为非对准双字,所以存取该双字需要 2 个总线周期。一个总线周期用来访问地址为 2001、2002 和 2003 的 3 个字节,另一个总线周期访问地址为 2004 的字节。

例 3.9 某 8 位机采用单总线结构,地址总线 16 根($A_{15} \sim A_0$),双向数据总线 8 根($D_7 \sim D_0$),控制总线中与主存有关的有 $\overline{MREQ}$ (允许访存,低电平有效), $\overline{R/\overline{W}}$ 。主存地址空间分配如下: 0~8191 为系统程序区,由只读存储器芯片组成;

8192~32767 为用户程序区；最后 2K 地址空间为系统程序工作区。上述地址为十进制，按字节编址。现有如下存储器芯片：ROM $8K \times 8$ 位（控制端仅有 $\overline{CS}$ ），SRAM $16K \times 1$ 位、 $2K \times 8$ 位、 $4K \times 8$ 位、 $8K \times 8$ 位。

请从上述芯片中选择适当芯片，设计该计算机内存存储器，画出主存储器逻辑框图。

解 根据给定条件，选用 ROM $8K \times 8$ 位芯片 1 片；RAM $8K \times 8$ 位芯片 3 片， $2K \times 8$ 位芯片 1 片；3-8 译码器使用 $\overline{Y}_0$ ， $\overline{Y}_1$ ， $\overline{Y}_2$ ， $\overline{Y}_3$ ， $\overline{Y}_7$ 输出端；最后 $2K \times 8$ 位芯片需加门电路译码。

主存地址空间分布如表 3.2 所示。逻辑框图如图 3.5 所示。

表 3.2 例 3.9 表

8KB(ROM)	30KB(空)
24KB(RAM)	2KB(RAM)

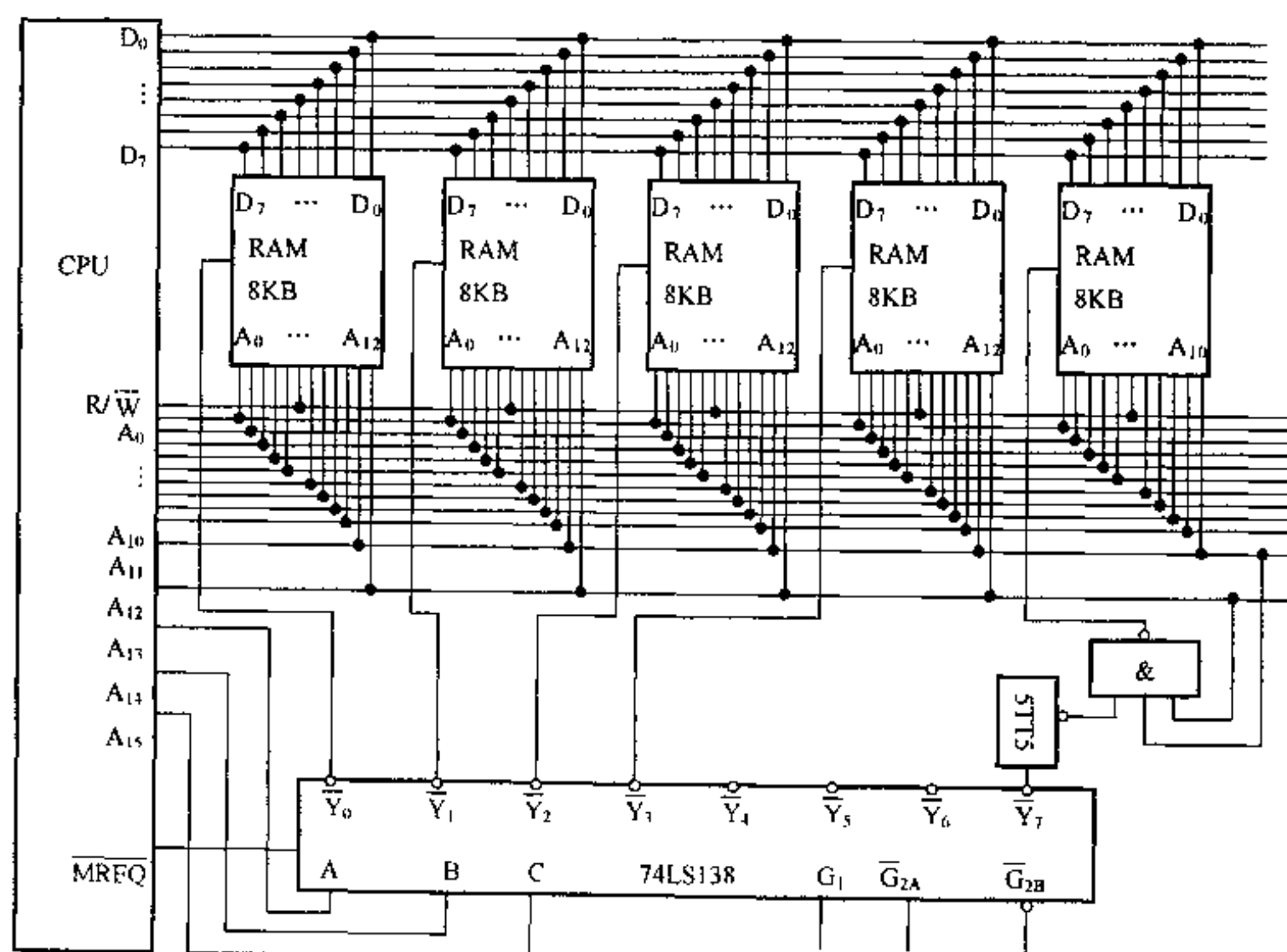


图 3.5 例 3.9 图

3.3 学 习 自 评

3.3.1 自测练习

一、填空题

1. 在下表所列典型技术特性栏目的空格中填入适当的术语。

序号	技 术	读写方式	信息可保存性	存储物理介质
①	双极性半导体			
②	金属-氧化物-半导体			
③	磁盘			

2. _____是指若干存储结构组成的机器字的基本存储单位。
3. _____是指存储器所能容纳的最大二进制信息字节数。
4. _____是指存储单元中对应一个字节数据的地址编号。
5. 半导体静态存储器是靠_____(1)_____存储信息的, 半导体动态存储器是靠_____(2)_____存储信息的。
6. 半导体动态存储器必须刷新的原因是_____(1)_____, 刷新方式一般有_____(2)_____, _____(3)_____, _____(4)_____三种。
7. 存储器的读出时间通常称为_____(1)_____, 它定义为_____(2)_____, 为便于读/写控制, 存储器的_____(3)_____应和读出时间相等, 而事实上它小于读出时间。
8. 为提高经数据总线进行数据传送的可靠性, 需在数据存储器接口增加_____。
9. 若设置为奇校验的 74S280 奇偶发生/校验器电路的输入为 IHGFEDCBA=11111111₂, 则其输出应为_____。

二、选择题

1. 存储器是计算机系统记忆设备, 它主要用来 ()。
 - A. 存放数据
 - B. 存放程序
 - C. 存放数据和程序
 - D. 存放微程序
2. 存储周期是指 ()。
 - A. 存储器的读出时间
 - B. 存储器的写入时间
 - C. 存储器进行连续读和写操作所允许的最短时间间隔
 - D. 存储器进行连续写操作所允许的最短时间间隔
3. 和外存储器相比, 内存储器的特点是 ()。
 - A. 容量大、速度快、成本低
 - B. 容量大、速度慢、成本高

- C. 容量小、速度快、成本低 D. 容量小、速度快、成本高

三、问答题

1. ROM、PROM、EPROM 和 EEPROM 分别用在什么场合?
2. 用存储器件组成内存时,为什么总是采用矩阵形式?试举例说明。
3. 为了节省存储器的地址译码电路,通常采用哪些方法?
4. 在选择存储器件时,首要考虑的因素是哪些?
5. 何谓静态 RAM?它的使用特点是什么?
6. 在对静态存储器读/写时,地址信号分为几个部分?分别产生什么信号?
7. 动态 RAM 的工作特点是什么?与静态 RAM 相比,动态 RAM 有什么长处和不足?说明动态 RAM 的适用场合。
8. 试述动态 RAM 刷新过程和正常读/写过程的区别。
9. 试比较存储器读周期和存储器写周期的差别。
10. 如何产生存储器芯片的片选择信号?产生片选择信号有几种方法?各有何特点?适用于什么场合?

3.3.2 自测练习解答

一、填空题

1.

序号	技 术	读/写方式	信息可保存性	存储物理介质
①	双极性半导体	随机存取	非破坏性读出,断电后信息丢失	电子
②	金属-氧化物-半导体	随机存取	非破坏性读出或破坏性读出	电子
③	磁盘	随机半顺序读出	非破坏性读出,永久保存	磁

2. 存储单元
3. 存储容量
4. 存储地址
5. (1) 触发器 (2) 电容
6. (1) 电荷泄漏,需要定期补充 (2) 集中式 (3) 分散式 (4) 异步式
7. (1) 存取时间
 - (2) 从存储器接收读出请求到所需要的信息出现在输出端上的时间
 - (3) 写入时间
8. 奇偶发生/校验电路
9. $\sum \text{even} = 0$; $\sum \text{odd} = 1$

二、选择题

1. C 2. C 3. D

三、问答题

1. ROM 一般由生产厂家将系统程序固化好, 配备到计算机上。PROM 的典型应用是用于高速计算机中的微程序存储器, 或作专用函数库。EPROM 是在微机应用与开发中运用较广泛的芯片, 常用于存储用户已开发成功的程序。EEPROM 摒弃了 EPROM 须脱机写入和擦除的缺点, 可在线进行改写, 因而, 在用户开发过程中程序尚未定型时较多采用。

2. 采用矩阵形式存储信息主要是为了节省地址译码电路, 提高半导体器件的集成度。例如, 要组成 4KB 的内存, 若不采用矩阵形式来组织存储单元, 则需要 4096 条译码线才能实现对 4KB 的寻址。如果用 64×64 的矩阵来排列, 则只需要 64 条行选择线和 64 条列选择线就可以了。存储容量越大, 矩阵形式的优越性就越明显。

3. 通常采用存储器的模块结构并在一个存储模块内将存储器分组, 用地址的高位对模块进行选择, 其余位对存储器进行选择。

4. 选择存储器件首要的指标是存取速度及芯片的存储容量, 同时还应考虑芯片的集成度、功耗、价格等。

5. 静态 RAM 是触发器存储信息的半导体存储器。静态 RAM 是非破坏性读出, 与动态 RAM 相比, 容量小, 功耗大(总有半导体元件处于导通状态)。

6. 对静态 RAM 进行读/写操作时, 地址信号被分为片内地址和片选地址两个部分, 分别用于片内存储单元的选择和芯片组的选择。

7. 动态 RAM 利用电容存储电荷的原理保存信息。与静态 RAM 相比, 动态 RAM 电路结构简单, 集成度高, 存储容量大。但由于电容漏电, 使用动态 RAM 必须配以复杂的刷新控制电路, 以便不断地对其补充电荷, 因此其连线较静态 RAM 复杂, 常用在要求速度快、存储容量大的场合。

8. 动态 RAM 维持信息的时间仅为 2ms, 因此必须在 2ms 内对动态 RAM 进行刷新。刷新工作是任一动态 RAM 存储器都需要的, 刷新操作是对所有芯片按行进行的。正常的读/写操作则是根据程序运行的要求, 对某一部分单元进行读/写操作。

9. 存储器读周期和存储器写周期的主要差别在于数据总线上出现数据的时间。在存储器读周期内, 地址线和选通控制线稳定后, 被读出的数据才出现在数据总线上; 而在存储器写周期内, 则是往存储器内写入新的信息, 故在所有选通控制信号出现之前, 数据线上应有待写的稳定数据。

10. 存储器芯片的片选择信号通过地址译码方法产生, 实现片选的方法有如下 3 种。

① 线选法。它直接将某地址高位接到某存储芯片的片选端,当该地址线信息为1时选中该存储器,然后再依地址低位对其进行片内寻址。线选法的优点是线路简单,选择芯片不需外加逻辑电路,故可用于较小的微机系统中。但是,线选法不能充分利用系统的存储器空间,且把地址空间分成了相互隔离的区段,给编程带来一定的困难。

② 全译码法。它把 N 条低位地址线直接接到所有存储芯片的地址输入端(N 由片内存储容量 2^N 决定),以进行存储芯片的片内寻址,再将地址总线中剩下的高位地址线全部接到地址译码器的输入端,把经译码器译码后的输出作为各芯片的片选信号。全译码法寻址不浪费可利用的地址范围,并可使存储器互相邻接,不产生地址码重叠的存储区,但全译码法寻址对译码电路的要求较高。

③ 部分译码法,是线选法和全译码法的结合。在部分译码法中,利用地址线高位进行译码产生片选信号时,有的地址线未参加译码,这些地址线在需要时可直接与芯片片选信号相连,以对芯片进行线选。当 CPU 的寻址范围大于所设计的微计算机系统要求的存储容量时,常采用此种译码法。

3.4 教学建议

本章的教学应着重于存储器地址空间的组织和存储器子系统两部分内容。其中,存储器子系统中行选列选电路、RAM 电路和奇偶校验电路是本章的难点,应结合书中给出的电路框图介绍,以帮助学生掌握存储器读/写的时序关系、控制原理以及整个存储器子系统的工作原理。

第4章 输入/输出（I/O）接口

4.1 知识要点

4.1.1 I/O接口

1. I/O 接口的功能和结构

I/O 接口的功能包括数据缓冲、接收和执行 CPU 的命令、信号转换、设备选择、中断管理、数据宽带交换和可编程等。接口是介于主机和外设之间的缓冲电路和相应软件。在学习时,应注重理解主机和外设之间加入接口的必要性以及 CPU 和外设之间速度、信号和数据不匹配的实际情况。

主机和外设之间交换信息都是通过操作接口电路的 I/O 端口来实现的。一个典型的 I/O 接口结构是由数据端口、状态端口和控制端口组成的。每个 I/O 端口对应一个或多个 I/O 地址。I/O 接口中的数据端口可以是双向的,状态端口只作输入操作,控制端口只作输出操作。

注意 一个端口往往对应接口芯片内部的一个或一组寄存器。学习时,应注重掌握接口芯片各端口的地址与功能尤其是扩展端口和状态端口信息的格式。

2. I/O 接口的端口编址方式

CPU 对 I/O 端口的编址方式分为端口统一编址方式和端口独立编址方式两种。在设有专用的 I/O 指令的 CPU 中, I/O 端口可以独立编址, I/O 地址空间完全独立于存储器空间, 于是一个 I/O 端口的地址可以与一个存储单元的地址重叠, CPU 通过控制总线来确定要访问哪一个空间。在统一编址的系统中, CPU 对 I/O 端口的 I/O 操作如同对存储单元的读/写操作一样, 对存储器的各种寻址方式也同样适用于 I/O 端口, 这种方式又常称为存储器编址映像。

显然, 独立编址要求 CPU 具有专用的 I/O 指令, 及相应的控制信号控制寻址空间。并且, 具有独立编址能力的计算机系统也可以采用统一编址。表 4.1 所示的是两种方式的比较。

表 4.1 端口统一编址和端口独立编址比较

编址方式	基本思想	存储空间	CPU 对端口的访问
端口统一编址	端口被视为存储单元,并予以相应的存储器地址	I/O 端口地址和存储器地址在同逻辑地址空间中	CPU 访问端口如同访问存储器一样,所有的内存指令同样适合 I/O 端口
端口独立编址	所有的 I/O 端口看作是一个独立于存储器空间的 I/O 空间	计算机系统内有 2 个存储空间:一个是存储器地址空间,一个是 I/O 端口地址空间	访问 I/O 地址空间必须用专门的 I/O 指令及相应控制信号

4.1.2 可编程间隔定时器 (PIT) 82C54

1. 82C54 的内部结构

82C54 由数据总线缓冲器、读/写逻辑和 3 个独立的 16 位计数器组成。每个计数器包括如下寄存器。

① 控制字寄存器:保存该计数器的 6 位控制字,控制字决定计数器的工作方式、计数形式和输出方式,以及如何装入计数初值;8254 的 3 个控制字寄存器共用一个端口地址,用控制字最高二位确定写入哪一个计数器。

② 装入器、计数器和锁存器:16 位装入器也叫计数值寄存器 (CR),用于装入或读出初值;16 位计数器也称计数单元 (CE),计数器在门控信号允许、输入脉冲作用下进行减 1 计数,计数值为 0 时输出编程的信号;输出锁存器 (OL) 根据锁存命令或读回命令锁存计数器的当前值供 CPU 读出。

③ 状态寄存器和状态锁存器:状态寄存器保存计数器的状态,状态寄存器的内容可由读回命令锁存在状态锁存器中供 CPU 读出。

2. 82C54 的控制字、读回命令和状态字

(1) 82C54 的控制字

控制字规定计数器选择、读/写方式选择、工作方式选择和数制选择。其中,读/写方式选择中的计数器锁存命令将当前计数单元的内容锁存到相应的输出锁存器中,供 CPU 读出。

(2) 82C54 的读回命令

读回命令将指定计数器中计数单元的当前内容锁存到输出锁存器中,或将状态寄存器的内容锁存到状态锁存器中。不同于锁存命令,读回命令能够同时锁存几个计数器的计数值和状态值。CPU 可用输入指令读出锁存器的内容并对锁存器“解锁”。当同时锁存计数值和状态值时,CPU 使用第一条输入指令先读出状态锁存器的内容,再用第二条输入指令读出输出锁存器的内容。

(3) 82C54 的状态字

状态字低 6 位为最后写入的控制字的低 6 位,最高位为该计数器输出引脚

OUT 的状态, 次高位指示最后写入计数值寄存器 CR 中的内容是否装入到计数单元 CE 中。在控制字装入控制字寄存器或计数初值装入计数值寄存器 CR 时, 计数单元被锁存的计数值不是最后写入的新计数值; 仅当次高位状态为 0, 锁存的计数值才是有效的。

3. 82C54 的工作方式

82C54 有 6 种工作方式, 学习时需注意从门控信号 GATE 的作用、计数值和启动计数的条件、输出信号 OUT 波形等方面比较它们的功能时序。

(1) 门控信号 GATE 的作用

对于方式 0 和方式 4, GATE 高电平允许计数, 低电平停止计数, 上升沿成为高电平后允许继续计数。

对于方式 1 和方式 5, GATE 高/低电平都不影响计数, GATE 上升沿启动计数, 每次 GATE 上升沿都触发重新计数。

对于方式 2 和方式 3, GATE 高/低电平分别是允许/禁止计数。有软、硬两种方式启动计数: 当 GATE 为高电平, 软件装入计数值启动计数; 在写计数值后, GATE 上升沿启动计数。

(2) 计数过程

① 计数值。最小计数值: 方式 0、1、4 和 5 为 1; 方式 2 和 3 为 2。最大计数值: 所有方式均为 0, 它代表二进制计数时的 65536 和十进制计数时的 10000。

② 计数启动条件。方式 0 和方式 4 写计数值启动计数; 方式 1 和方式 5 用门控信号启动计数; 方式 2 和方式 3 可用软、硬两种方式启动计数。

③ 计数重复条件。方式 0 和方式 4 通过写计数值重新计数; 方式 1 和方式 5 用 GATE 触发重新计数; 方式 2 和方式 3 是自动重复、连续计数。

④ 改变计数值。方式 0 和方式 4 是改变计数值后下一个 CLK 立即有效; 方式 1 和方式 5 是改变计数值后 GATE 上升沿触发后有效; 方式 2 是改变计数值后计数到 1 时有效或 GATE 触发时有效, 方式 3 改变计数值后计数到 0 时有效或 GATE 触发时有效。

(3) 输出波形

设输入时钟为 CLK, 写计数值 N, 输出为 OUT。

方式 0 写控制字时 OUT 变低, 写计数值 N 后 (N+1) CLK 时 OUT 输出变高; 方式 1 写控制字时 OUT 变高或保持为高电平, 写计数值 N, 待 GATE 触发后输出 NCLK 宽度的低电平 OUT。

方式 2、方式 4、方式 5 都是输出宽度为 1CLK 的低电平 OUT, 方式 2 的周期是 NCLK, 方式 4 和方式 5 的 OUT 是 (N+1) CLK 后输出 1CLK 的低电平。但这 3 种方式启动计数的触发条件不同。

方式 3 的输出 OUT 是周期为 NCLK 的连续方波。对于方式 2、3、4、5, CPU

写入控制字后, 它们的输出 OUT 立即变高或保持为高。

4.1.3 可编程外围接口(PPI) 82C55A

1. 82C55A 的控制字

82C55A 的各种工作方式由控制字设定, 在对 82C55A 的初始化编程过程中, 控制字由 CPU 指令送到 A 组或 B 组的控制电路的控制寄存器中。CPU 送控制寄存器的控制字有两种。

(1) 方式控制字($D_7=1$)

82C55A 的方式控制字如图 4.1 所示。

D_7	D_6D_5	D_4	D_3	D_2	D_1	D_0
1	A 组 工作方式	A 口 I/O	$C_7 \sim C_4$ I/O	B 组 工作方式	B 口 I/O	$C_3 \sim C_0$ I/O

图 4.1 82C55A 的方式控制字

(2) 端口 C 位设置字($D_7=0$)

82C55A 的端口 C 位设置字如图 4.2 所示。

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	x	x	x	写入位编码		写入内容	

图 4.2 82C55A 的端口 C 位设置字

2. 82C55A 的工作方式

① A 口有 3 种工作方式, B 口有 2 种工作方式, C 口只有基本输入/输出方式, 但却具有按位操作的功能。

② 当 A 口或 B 口工作于方式 1、方式 2 时, C 口的某些位自动指定为 PA 和 PB 的控制联络线, 这些控制联络线的输入/输出方向也就自动指定, 在方式控制字中对 C 口的高低 4 位部分的输入/输出方向的指定将不会影响这些控制联络线, 而只能指定除这些控制联络线以外的 C 口中的普通输入/输出线方向。

③ 当 A 口或 B 口工作于方式 1, 方式 2 时, 若采用中断方式与 CPU 交换信息, 一定要 82C55A 片内的相应中断允许触发器 INTE 置“1”。

注意 此时用 C 口的按位置/复位控制字操作 PC_4 (A 口的输入中断允许控制位) 或 PC_2 (B 口的输入或输出中断允许控制位), 或 PC_6 (A 口的输出中断允许控制位) 时, 只影响 INTE 触发器的状态, 不影响对应的引脚状态。

④ 当 A 口或 B 口工作于方式 1, 方式 2 时, 82C55A 通过对 C 口地址进行读操作, 将获取一个状态字。注意, 该状态字的某些位不反映引脚的状态。例如, 状态字的 D_2 位表示中断允许触发器的状态, 而不是表示 PC_2 引脚上 $\overline{STB}_B$ 或

$\overline{\text{ACK}}_{\text{B}}$ 输入信号的状态。

82C55A 的 3 个端口、3 种工作方式可以相互组合，其数据输入/输出以及控制联络信号如图 4.3 所示。

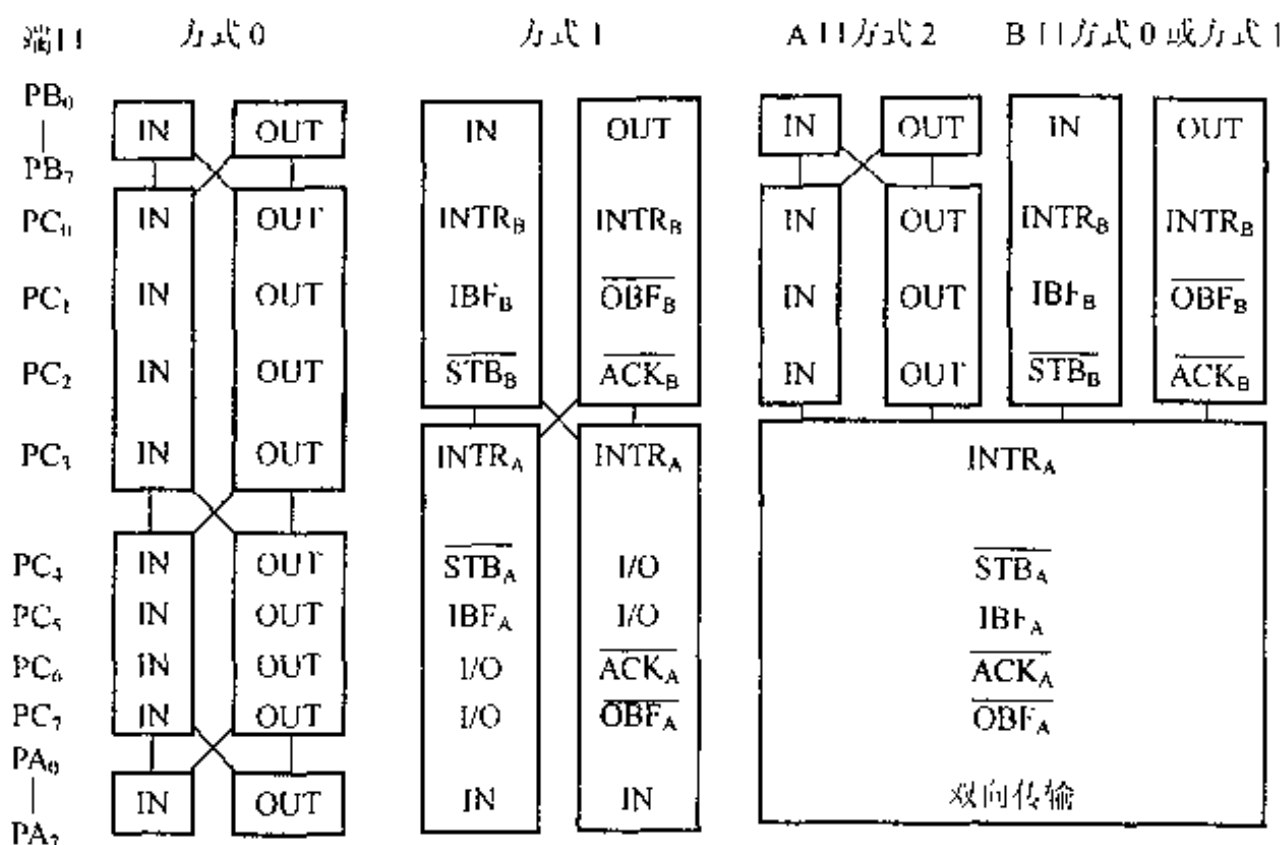


图 4.3 82C55 工作方式的组合

3. 82C55A 的状态字

82C55A 通过读 C 口的状态可以了解 3 个端口在 3 种不同的工作方式下的状态，如同工作方式的组合一样，工作状态也是组合的，如图 4.4 所示。

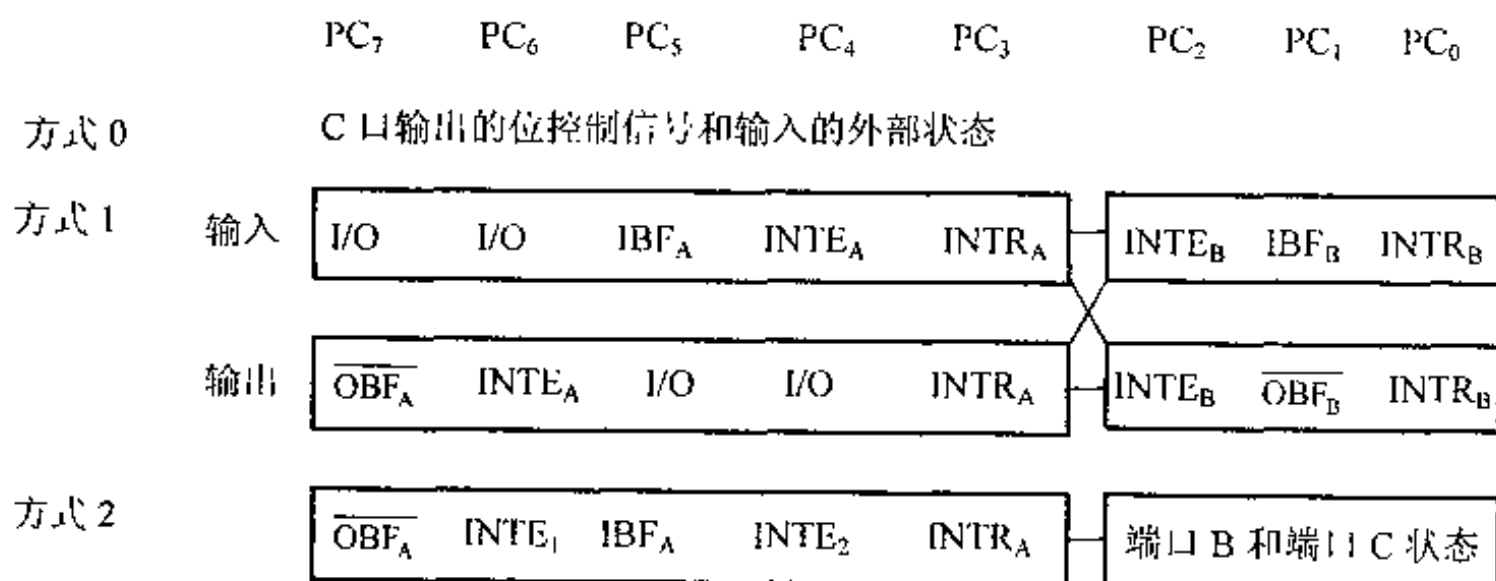


图 4.4 82C55A 状态字的组合

4.1.4 串行通信和通用异步收发器(UART) 8250

1. 串行通信概述

(1) 异步串行通信和同步通信

① 异步串行通信。收发双方可以没有共同时钟；采用起止同步控制方式；字符间是异步的，字符内位是同步的；主要通信规程为 ASYNC 等。

② 同步串行通信。收发双方使用同一时钟源；采用同步字符或硬件信号进行同步；字符间和字符内都是同步的；通信规程有面向字符的 BSC、BISYN 和面向比特的 SDLC、HDLC 和 ADCCP 等。

(2) 串行通信的工作方式

根据是单向通信还是双向传送，工作方式分为单工和双工两大类，双工方式中又根据是否能够同时进行双向传送分为半双工和全双工。此外还有复用和分用方式，常用的有频分复用 FDM、时分复用 TDM 和波分复用 WDM，WDM 又分为稀疏波分复用 CWDM 和密集波分复用 DWDM。

(3) 基波(带)和载波传输

基带传输是利用数字信号直接进行近距离传输。由于线路电阻、电感、电容、漏电流及漏感分布参数的影响，矩形波会发生畸变和延迟，所以基带传输速率与距离成反比关系。

载波传输要利用调制器可以将数字信号调制成模拟信号，能够进行远距离传输，然后再用解调器把模拟信号变换为数字信号。调制常用的方法有调幅、调频和调相。幅移键控 ASK、频移键控 FSK 和相移键控 PSK 可分别使正弦波的幅值、频率和相位随着数字信号 1 和 0 变化。

(4) 数字传输率和调制速率

数字传输率，又称比特率，为单位时间内传送的二进制位数，单位是 b/s，用 C 表示。

调制速率，又称波特率，为单位时间内信号(或线路)状态变化数量，单位是 baud，用 B 表示。

当用 N 表示调制信号(或线路)状态数，比特率与波特率的关系为 $C = B \log_2 N$ 。若单位时间内调制或解调 1 个信号(2 种状态)，则两种速率在数值上是相等的。

2. EIA RS-232 串行接口标准

RS-232 是与 CCITT 的 V 系列建议基本相似的数据终端设备 DTE 和数据通信设备 DCE 之间的接口。规定了按位串行传输的机械、电气、功能和规程特性。

(1) 机械特性

RS-232 使用 25 针 D 型插头座 DB-25，其他使用 DB-9 或 DB-15 的插头座可用连接器转换。

(2) 电气特性

RS-232 按负逻辑工作, 逻辑 0 为+3V 或+5~+15V 之间的任一电压, 逻辑 1 为-3V 或-5~-15V 之间的任一电压。RS-232 使用的 EIA 电平与计算机使用的 TTL 电平需要用集成电路芯片或分立电路进行转换。

RS-232 标准规定若不使用 Modem, 则在码元畸变小于 4% 的情况下, DTE 和 DCE 之间最大传输距离为 15m(50 英尺)。可见这个最大距离是在码元畸变小于 4% 的要求, 接口标准在电气特性中规定, 驱动器的负载电容应小于 2500pF。例如, 采用每 0.3m (约 1 英尺) 的电容量为 40~50pF 的普通非屏蔽多芯电缆作传输线, 则传输电缆的长度, 即传输距离为

$$L = \frac{2500\text{pF}}{170\text{pF/m}} \approx 15\text{m}$$

然而, 在实际应用中, 码元畸变超过 4%, 即使为 10%~20% 时, 也能正常传输信息, 这意味着驱动器的负载电容可以超过 2500pF, 因而传输距离可大大超过 15m, 这说明了 RS-232 标准所规定的直接传输最大距离为 15m, 是偏于保守的。

(3) 功能特性和规程特性

RS-232 接口用于异步通信的引脚大约有 10 根, 用于计算机 (相当于 DTE) 通过 RS-232 接口与 Modem (相当于 DCE) 相连。下面叙述中, ○中数字是引脚号, () 中字母是代号, 然后是简称、信号名及主要功能。

地线 ①, (AA), 保护地; ②, (AB), 信号地。

数据信号 ③, (BA), TxD: 发送数据, DTE→DCE; ④, (BB), RxD: 发送数据, DTE←DCE。

Modem 控制和状态信号。

计算机 (DTE) 输出给 Modem (DCE) 的信号如下。

⑤, (CD), DTR: 数据终端准备好。程序控制下, 告诉 Modem, 计算机已准备好。

⑥, (CA), RTS: 请求发送。程序控制下, 告诉 Modem, 计算机请求发送数据。

计算机 (DTE) 从 Modem 输入的信号如下。

⑦, (CC), DSR: 数据装置准备好。告诉计算机, Modem 已准备好。

⑧, (CB), CTS: 允许发送。告诉计算机, Modem 已准备好接收, 允许发送。

⑨, (CF), DCD: 数据载波检测。告诉计算机, Modem 已建立与电话线另一端 Modem 的连接, 检测到线路上的载波。

⑩, (CE), RI: 振铃指示。告诉计算机, 连到 Modem 上的电话响铃了, 说明接收到对方 Modem 拨号后通过电话网送来的信号。

3. 8250 的内部寄存器和编程

(1) 8250 的内部寄存器

8250 内部共有 10 个可编程的寄存器,在组成异步通信适配器时,可通过跳线选择串行端口。COM₁ 选择 3F8H~3FFH 地址和 IRQ₄ 中断请求输入,COM₂ 选择 2F8H~2FFH 地址和 IRQ₃ 中断请求输入。由于端口地址数量不够,采用寄存器中指定位参与寻址。在线路控制寄存器 LCR 中最高有效位被指定为除数锁存器访问位 DLAB。当 DLAB=0 时,XF8H 对应接收缓冲寄存器 RBR 和发送保持寄存器 THR,读/写信号分别选通 RBR 和 THR;XF9H 对应中断允许寄存器 IER。当 DLAB=1 时,XF8H 对应除数锁存器低 8 位 DLL;XF9H 对应除数锁存器高 8 位 DLH。其他各寄存器寻址与 DLAB 无关,端口地址与可编程寄存器一一对应。XFAH、XFBH、XFCH、XFDH 和 XFEH 分别对应中断标识寄存器 IIR、线路控制寄存器 LCR、Modem 控制寄存器 MCR、线路状态寄存器 LSR 和 Modem 状态寄存器。

(2) 8250 初始化

- ① 测试 8250 即异步适配器的存在: IIR 的高 5 位为全 0。
- ② 设置波特率除数: 在 LCR 中 DLAB=1 时设置 DLL 和 DLH。
- ③ 设置通信线路控制: 在 LCR 中 DLAB=0 时编制 LCR 中各位,设置串行通信的数据格式。
- ④ 设置 Modem 的控制: 输出引脚 $\overline{\text{DTR}}$ 、 $\overline{\text{RTS}}$ 、 $\overline{\text{OUT}}_1$ 和 $\overline{\text{OUT}}_2$ 的状态。注意在 PC 机的异步适配器上用 $\overline{\text{OUT}}_2$ 控制中断请求信号是否能够送出,如果是允许中断, $\overline{\text{OUT}}_2$ 应输出有效电平。
- ⑤ 设置中断允许: 在 LCR 中 DLAB=0 时设置 IER,分别允许接收数据出错、接收数据就绪、发送保持寄存器空和 Modem 状态变化中断。

(3) 8250 中断功能测试

在允许中断情况下,直接向 LSR 的低 6 位 THRE、BI、FE、PE、OE 和 DR 写 1,向 MSR 低 4 位 IRLSR、RI、IDSR 和 ICTS 写 1(或者向 MSR 高 4 位 RLSR、RI、DSR 和 CTS 写 1 再写 0,状态变化引起 MSR 低 4 位中相应位自动置 1),可以测试 8250 中 10 个中断源的功能。

在回送(Loop)方式下,8250 的发送、接收和 Modem 状态变化的中断仍然工作。但 Modem 中断源为 MCR 低 4 位,非状态输入端,并受 IER 控制。改变 MCR 低 4 的值,可产生 Modem 状态变化中断。

一般 8250 中断测试是编程在回送方式下进行的。

(4) 中断方式异步通信驱动程序编写要点

- ① 中断初始化。修改中断向量表中 COM₁/COM₂ 对应的中断类型 OCH/OBH 的向量,指向中断服务程序。

- 初始化 8250: 设置 IER 相应位、MCR 中的 OUT_2 位。使中断请求信号能够送到 8259 的 IR_4 或 IR_3 。根据波特率设置分频的除数。根据通信规程设置数据帧格式。

- 开放异步中断: 使用 8259OCW₁ 的 D_4 或 D_3 清除对 IR_4 或 IR_3 的中断请求屏蔽, 开放异步中断。

② 中断主程序。根据应用需求自行编制。

③ 中断子程序。

- 辨别发生中断的中断源: 读 IIR, 根据其中的 $D_2D_1D_0$ 和状态转入相应子程序。

- 中断处理: IIR 的 $D_2D_1D_0$ 表示 ID_1 、 ID_2 、IP。

当 $D_2D_1D_0 = 110$ 时, 接收数据出错, 读 LSR 中 $D_4D_3D_2D_1$, 判断是间断还是何种错误, 转到相应的子程序。

$D_2D_1D_0 = 100$, 接收数据就绪, 从 RBR 中读取数据存入收缓冲区。

$D_2D_1D_0 = 010$, 发送保持寄存器空, 从发送缓冲区取数据, 并送以 THR。

$D_2D_1D_0 = 000$, Modem 状态变化, 读 MSR 的 $D_3D_2D_1D_0$ 位, 判断是哪些 Modem 引脚改变了状态, 转相应处理子程序。

判断有无尚未处理的中断请求: 每种中断源的中断处理完之后, 再次读取 IIR 的 $D_2D_1D_0$ 位, $D_0 = 0$, 根据 D_2D_1 转入相应子程序; 若 $D_1 = 1$, 说明 8250 所有中断处理结束, 对 8259 发 EOI 命令返回被中断的程序。

4.2 例题精析

例 4.1 接口是介于_____和_____之间的_____电路, 这两个部件之间通过操作接口电路的_____来实现信息的交换。

解 接口是介于主机和外设之间的缓冲电路, 这两个部件之间通过操作接口电路的 I/O 端口来实现信息的交换。

例 4.2 根据端口接收和输出信息的不同, 可将端口分为_____, _____和_____。

解 根据端口接收和输出信息的不同, 可将端口分为数据端口, 控制端口和状态端口。

例 4.3 连接通信双方设备的线路一般可以选择的工作方式包括: _____, _____, _____和_____。RS-232 标准的内容包括 DTE 和 DCE 之间的_____, _____, _____和_____。

解 连接通信双方设备的线路一般可以选择的工作方式包括: 单工传输方式,

半双工传输方式，全双工传输方式和多工传输方式。RS-232 标准的内容包括 DTE 和 DCE 之间的机械特性，功能特性，电气特性和规程特性。

例 4.4 设 8255A 的 4 个端口地址为 0060H~0063H，请编写下列情况下的初始化程序。

(1) 将 A 组和 B 组设置成方式 0，A 口、B 口为输入，C 口为输出。

(2) 将 A 组工作方式设置为方式 2，B 组为方式 1，A 口、B 口作为输出。

(3) 将 A 口、B 口均设置为方式 1，均为输入，PC₆和 PC₇为输出。

(4) A 口工作在方式 1，输入；B 口工作在方式 0，输出；C 口高 2 位为输出，低 3 位为输入。

解 (1) MOV AL,10010010B

OUT 63H,AL

(2) MOV AL,11000100B

OUT 63H,AL

(3) MOV AL,10110110B

OUT 63H,AL

(4) MOV AL,10110001B

OUT 63H,AL

例 4.5 8254 的端口地址为 40H~43H，计数器 2 工作在方式 2，按二进制计数，计数值为 1110H，现要在计数过程中读取该计数器的计数值。请对其进行初始化编程。

解 MOV AL,0B4H

OUT 43H,AL ; 设置计数器 2 方式字

MOV AL,10H

OUT 42H,AL ; 送计数值低字节

MOV AL,11H

OUT 42H,AL ; 送计数值高字节

MOV BH,20H ; 设置中间计数次数

W: DEC BH

CMP BH,0

JNZ W ; 判断延迟时间是否到

MOV AL,80H ; 计数值锁存命令

OUT 43H,AL

IN AL,42H ; 读取计数值低字节

MOV AH,AL

IN AL,42H ; 读取计数值高字节，结果在 AX 中

讲评 为了读取计数过程中某值,应先将输出锁存器锁存,可通过计数器锁存命令,如本例;也可通过读回命令,读回命令一般用于同时锁存多个计数器的计数值和状态值。

例 4.6 在一控制系统中,8251A 作为 CPU 的全双工异步串行通信接口,采用中断方式工作,波特率因子是 16,字符长度是 8,偶校验,1 位停止位,发送和接收数据的波特率均为 1200baud,8254 的计数器 2 为 8251A 提供发送、接收时钟 $\overline{T_xC}$ 和 $\overline{R_xC}$,其 CLK₂ 输入为 8MHz 信号。已知 8251A 的端口地址为 0F0H 和 0F1H,8254 的端口地址为 230H~233H。

(1) 编写 8251A 的初始化程序段;

(2) 编写 8254 的计数器 2 的初始化程序段。

解 (1) 8251A 是通用同步/异步串行接口,其初始化流程如图 4.5 所示,包括工作方式字和工作命令字,均写入奇地址。

工作方式字各位定义如下:

S ₂	S ₁	EP	PEN	L ₂	L ₁	B ₂	B ₁
----------------	----------------	----	-----	----------------	----------------	----------------	----------------

B₂B₁ 为 00 设置同步方式,为 01/10/11 设置波特率因子等于 1/16/64 的异步方式; L₂L₁ 为 00/01/10/11 设置字符长度为 5/6/7/8 位; PEN 为 0/1 用来禁止/允许奇偶校验; EP 为 0/1 设置奇/偶校验位; 异步方式下, S₂S₁ 为 00 无效,为 01/10/11 设置停止位为 1/1.5/2 位,同步方式下, S₁ 为 0/1 设置内同步/外同步检测, S₂ 为 0/1 设置双同步字符/单同步字符。

工作命令字各位定义如下:

EH	IR	RTS	ER	SBRK	R _x E	DTR	T _x EN
----	----	-----	----	------	------------------	-----	-------------------

T_xEN 为 0/1 用来禁止/允许发送; DTR 为 1 使引脚 $\overline{DTR}$ 输出低电平; R_xE 为 0/1 用来禁止/允许接收; SBRK 为 0/1 设置正常工作/送间隔字符; ER 为 1 使出错标志复位; RTS 为 1 使引脚 $\overline{RTS}$ 输出低电平; IR 为 1 进行内部复位; EH 为 1 开始搜索同步字符。

8251A 的初始化程序段如下:

MOV AL,0

MOV DX,0F1H

: 向命令端口送任意数

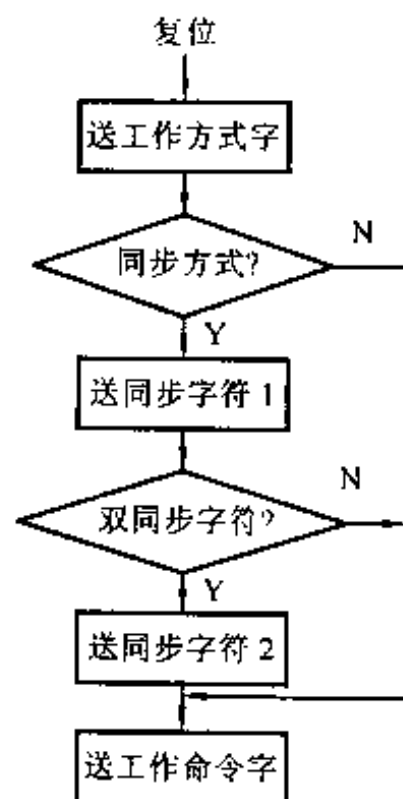


图 4.5 8251A 初始化流程

```

OUT    DX,AL
MOV    AL,40H                ; 内部复位,命令字中 IR=1
OUT    DX,AL
MOV    AL,7EH                ; 工作方式字 01111110
OUT    DX,AL
MOV    AL,37H                ; 工作命令字 00110111
OUT    DX,AL

```

(2) 8254 的计数器 2 作为串行通信的波特率发生器, 所以选择方式 3-N 分频方波。又因为输入 CLK₂ 为 8MHz, 要求输出方波为 16 (波特因子) × 1200 baud = 19.2kHz, 所以计数器 2 的计数初值为 8MHz/19.2kHz ≈ 417

8254 的计数器 2 的初始化程序段如下:

```

MOV    AL,10110110B          ; 计数器 2: 先低后高写计数值、方式 3、二进制计数
MOV    DX,233H
OUT    DX,AL
MOV    DX,232H
MOV    AX,417
OUT    DX,AL                 ; 初值低 8 位
MOV    AL,AH                 ; 初值高 8 位
OUT    DX,AL

```

例 4.7 利用 Ins 8250 的 LOOP 方式测试其发送、接收和 Modem 状态变化的中断功能。

解 设数据发送波特率为 1200baud, 8 位数据位, 1 位停止位, 采用偶校验。

```

IMR_REG    EQU    21H
INT_EOI    EQU    20H
EOI         EQU    20H
DATA_IN     EQU    3F8H
DATA_OUT    EQU    3F8H
DLAB_DLL    EQU    3F8H
DLAB_DLH    EQU    3F9H
IER_ADDR    EQU    3F9H
IIR_ADDR    EQU    3FAH
LCR_ADDR    EQU    3FBH
LSR_ADDR    EQU    3FDH
MCR_ADDR    EQU    3FCH
MSR_ADDR    EQU    3FEH

```



```

CR          EQU      0DH
LF          EQU      0AH
STACK SEGMENT PARA STACK' STACK'           ; 定义堆栈
STAPN      DB          100      DUP(?)
TOP        EQU      LENGTH STAPN
STACK      ENDS
DATA       SEGMENT                       ; 定义数据区
Mess1      DB          'No ASYNC Communication Adapter!', CR, LF, '$'
Mess2      DB          'No Interrupt! Time Out ', CR, LF, '$'
Mess3      DB          'Delta CTS Interrupt!', CR, LF, '$'
Mess4      DB          'Delta DSR Interrupt!', CR, LF, '$'
Mess5      DB          'Ring Indicate!', CR, LF, '$'
Mess6      DB          'Test Successful!', CR, LF, '$'
Mess7      DB          'Error During Testing!', CR, LF, '$'
SOUR       DB          'ASYNC Communication Adapter Test Program ', CR, LF
           DB          'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz' CR, LF
           DB          '1234567890!@#%^&*()-+=_:"';<?./{}[]:$' , CR, LF
LEN        DB          $-SOUR
DEST       DB          LEN      DUP(0)       ; 接收字符缓冲区
END_FLAG   DB          1        DUP(0)       ; 发送, 接收结束标志暂存
IMR_BUF    DB          1        DUP(0)       ; 8259 原中断屏蔽字暂存
DATA       ENDS
CODE       SEGMENT
           ASSUME CS : CODE, SS: STACK, DS: DATA, ES: DATA
DISPLAY    PROC NEAR                       ; 显示子程序, 字符串地址在 DS: DX
           MOV     AH, 09H
           INT     21H
           RET
DISPLAY    ENDP
SEND       PROC NEAR                       ; 发送子程序, 发送字符地址在 DS: DI
           LODS    SOUR                     ; 取字符
           MOV     DX, DATA-OUT
           OUT     DX, AL                     ; 字符送 ADAPTER 输出
           RET
SEND       ENDP

```

```

RECEIVE PROC    NEAR                                ; 接收子程序
    MOV         DX, DATA_IN
    IN          AL, DX                               ; 从串口读字符
    CMP         AL, '$'                             ; 判字符是否等于'$'
    JNE         RECE1                                ; 转字符存储
    MOV         DX, IER_ADDR
    MOV         AL, 00H                             ; 禁止任何中断
    OUT         DX, AL
    MOV         END_FLAG, 0AAH                      ; 置接收完标志
    MOV         AL, '$'
RECE1: STOS     DEST                                ; 存字符
    RET
RECEIVE ENDP
; Modem 状态变化引起中断的处理子程序
Modem PROC      NEAR
    MOV         DX, MSR_ADDR
    IN          AL, DX                               ; 读 MSR
    AND         AL, 07H                             ; 取低 3 位: TERI、DDSR、DCTS
    CMP         AL, 01H                             ; 值等于 1
    JE          DCTS                                ; 转 DCTS 处理程序段
    CMP         AL, 02H                             ; 值等于 2
    JE          DDSR                                ; 转 DDSR 处理程序段
    CMP         AL, 04H                             ; 值等于 4
    JE          TERI                                ; 转 TERI 处理程序段
    JMP         SHORT MO_RET
DCTS: MOV       DX, OFFSET Mess3
    CALL        DISPLAY                             ; 显示 CTS 引起中断的信息
    JMP         SHORT MO_RET
DDSR: MOV       DX, OFFSET Mess4
    CALL        DISPLAY                             ; 显示 DSR 引起中断的信息
    JMP         SHORT MO_RET
TERI: MOV       DX, OFFSET Mess5
    CALL        DISPLAY                             ; 显示 RI 引起中断的信息
MO_RET: RET
Modem ENDP

```

；设置 8259A 子程序

```
SET 8259 PROC    NEAR
    IN      AL, IMR_REG      ; 读 8259 中断屏蔽字
    MOV     IMR_BUF, AL      ; 保存
    AND     AL, 0EFH         ; 允许 IRQ4 中断请求
    OUT     IMR_REG, AL
    PUSH    DS               ; 建立 IRQ4 中断向量
    MOV     AX, CS
    MOV     DS, AX
    MOV     AL, OCH          ; 中断类型 OCH 送 AL
    MOV     DX, OFFSET SERVICE ; 中断服务程序
    MOV     AH, 25H          ; 设置中断向量的 DOS 调用
    INT     21H
    RET
```

SET 8259 ENDP

；设置 Ins 8250 子程序

```
ADAPTER PROC    NEAR
    MOV     AL, 00H
    MOV     END_FLAG, AL      ; 释放接收结束标志
    MOV     IMR_BUF, 00H      ; 允许 8259 所有中断
    MOV     DX, LCR_ADDR
    MOV     AL, 80H           ; 置 LCR 的最高有效位
    OUT     DX, AL            ; DL AB=1
    MOV     DX, DLAB_DLL
    MOV     AL, 60H           ; 除数锁存器低 8 位
    OUT     DX, AL
    MOV     AL, 00H           ; 除数锁存器高 8 位
    MOV     DX, DLAB_DLH
    OUT     DX, AL            ; 1200Baud
    MOV     DX, LCR_ADDR
    MOV     AL, 1AH           ; 设置数据格式：偶校验，1 停止位，7 数据位
    OUT     DX, AL
    MOV     DX, MCR_ADDR
    MOV     AL, 1FH           ; 设置 MCR，回送方式，其他 4 位控制输出低有效
    OUT     DX, AL
```

```

        RET
ADAPTER    ENDP
; 主程序
START:  MOV     AX, DATA
        MOV     DS, AX
        MOV     ES, AX
        MOV     AX, STACK
        MOV     SS, AX
        MOV     AX, TOP
        MOV     SP, AX
        MOV     DX, IIR_ADDR
        IN      AL, DX                ; 读 8250 中断标识寄存器
        TEST    AL, 0F8H              ; 判断 D7~D3 是否都为 0
        JNZ     ERR1                  ; 异步通信口不存在, 转 ERR1
        CALL    SET_8259               ; 调用设置 8259 和中断向量程序
        CALL    ADAPTER                ; 调用设置 8250 子程序
        MOV     SI, OFFSET SOUR        ; 发送字符首地址 → SI
        MOV     DI, OFFSET DEST        ; 接收字符首地址 → DI
        MOV     DX, IER_ADDR           ; 中断允许寄存器
        MOV     AL, 02H                ; 允许发送数据保持寄存器空中断
        OUT     DX, AL                 ; 设置发送中断允许
        STI                                ; 开中断
        NOP
        MOV     DX, IER_ADDR
        MOV     AL, 03H
        OUT     DX, AL                 ; 置发送、接收中断允许
        MOV     CX, 000FH              ; 延时等待
        MOV     BX, 0FFFFH
LQ:     DEC     BX                      ; 等待中断发生
        JNZ     LQ
        CMP     END_FLAG, 0AAH         ; 判结束标志
        JE      COUT2                  ; 结束, 转下一个处理
        LOOP    LQ                     ; 未结束, 转继续等待
        JMP     ERR2                   ; 未产生中断, 转 ERR2
COUT2:  MOV     DX, OFFSET DEST

```

```

        CALL    DISPLAY
; 测试 Modem 状态变化引起的中断
        MOV     DX, MSR_ADDR
        IN      AL, DX                ; 读使 MSR 低 4 位清 0
        MOV     AL, 08H
        MOV     DX, IER_ADDR
        OUT     DX, AL                ; 设置允许 Modem 状态变化中断
        MOV     DX, MCR_ADDR
        MOV     AL, 1EH ; 初始化设为 1FH, LOOP 方式  $\overline{\text{DTR}}$  连向  $\overline{\text{DSR}}$ 
        OUT     DX, AL                ; 使 MSR-DSR 变化, 引起中断
        MOV     CX, 0FFFFH
LL0:    LOOP    LL0                  ; 等待中断
        MOV     DX, MCR_ADDR
        MOV     AL, 1CH ; 前面设置为 1EH, LOOP 方式  $\overline{\text{RTS}}$  连向  $\overline{\text{CTS}}$ 
        OUT     DX, AL                ; 使 MSR-CTS 变化, 引起中断
        MOV     CX, 0FFFFH
LL1:    LOOP    LL1                  ; 等待中断
        MOV     DX, MCR_ADDR
        MOV     AL, 18H ; 前面设置为 1CH, LOOP 方式  $\overline{\text{OUT1}}$  连向  $\overline{\text{RI}}$ 
        OUT     DX, AL                ; 使 MSR-RI 变化, 引起中断
        MOV     CX, 0FFFFH
LL2:    LOOP    LL2                  ; 等待中断
        MOV     DX, OFFSET Mess6
        CALL    DISPLAY                ; "Test successful!"
RETURN:  MOV     DX, IER_ADDR
        MOV     AL, 00H                ; 不允许 8250 中断
        OUT     DX, AL
        MOV     AL, IMR_BUF
        OUT     IMR_REG, AL            ; 恢复 8259 屏蔽字
DOSRET:  MOV     AH, 4CH                ; 调用 DOS 的 4CH 功能, 返回
        INT     21H
ERR1:    MOV     DX, OFFSET Mess1 ; 出错处理 "No ASYNC Communication Adapter!"
        CALL    DISPLAY                ; 显示适配器不存在
        JMP     SHORT DOSRET
ERR2:    MOV     DX, OFFSET MESS2      ; "No Interrupt!"

```

```

CALL    DISPLAY                ; 显示中断无法产生信息
JMP     SHORT RETURN          ; 结束处理
ERRPROC: MOV    DX, OFFSET mess7 ; "Error During Testing!"
        CALL    DISPLAY        ; 测试过程中出错
        JMP     SHORT RETURN
SERVICE: CLI                  ; 中断服务程序, 开始关中断
        MOV     DX, IIR_ADDR
        IN      AL, DX         ; 读 IIR
        CMP     AL, 02H        ; 发送中断?
        JE      JSEND
        CMP     AL, 04H        ; 接收中断?
        JE      JRECE
        CMP     AL, 00H        ; Modem 中断?
        JNE     ERRPROC
        CALL    Modem          ; 调用 Modem 处理子程序
        JMP     SHORT IRETURN
JSEND:   CALL    SEND          ; 调用发送子程序
        JMP     SHORT IRETURN
JRECE:   CALL    RECEIVE       ; 调用接收子程序
IRETURN: MOV     AL, EOI
        OUT     INT_EOI, AL    ; 设置 8259 中断结束
        IRET                  ; 中断返回
CODE     ENDS
        END START

```

例 4.8 对 8254 定时计数器 3 个通道进行初始化编程的顺序有没有要求?已知 8254 中计数器 0、1、2 及控制寄存器地址分别为 340H, 342H, 344H, 346H, 试对 8254 三个计数器进行编程, 使计数器 0 设置为方式 1, 计数初值为 2050H; 计数器 1 设置为方式 2, 计数初值为 3000H; 计数器 2 设置为方式 3, 计数初值为 1000。

解 对 8254 定时计数器 3 个通道进行初始化的顺序没有特定要求, 对每个计数器都应该先写入控制字然后再写初值。初始化顺序灵活, 可以先写 3 个计数器的控制字, 再写它们的计数初值; 也可以写一个计数器的控制字, 接着写它的计数初值, 逐个进行。

本题 8254 的 4 个地址均为偶数, 这是系统地址总线的 $A_0=0$, A_2 、 A_1 分别连接 8254 的 A_1 、 A_0 所致。初始化编程如下:

```

MOV  DX, 346H
MOV  AL, 32H           ; 计数器 0 控制字: 00110010
OUT  DX, AL
MOV  AL, 64H           ; 计数器 1 控制字: 01100100
OUT  DX, AL
MOV  AL, 0A7H          ; 计数器 2 控制字: 10100111
OUT  DX, AL
MOV  DX, 340H          ; 写计数器 0 初值
MOV  AL, 50H
OUT  DX, AL
MOV  AL, 20H
OUT  DX, AL
ADD  DX, 2              ; 写计数器 1 初值
MOV  AL, 30H
OUT  DX, AL
ADD  DX, 2              ; 写计数器 2 初值
MOV  AL, 10H
OUT  DX, AL

```

例 4.9 现要采用一片 8254 产生近似为 1Hz 的对称方波，已知时钟频率为 2MHz，8254 地址为 350H~353H，试编写 8254 初始化程序，并说明通道引脚的连接方法。

解 在时钟频率为 2MHz 时，要得到 1Hz 的方波，分频系数为 2×10^6 。8254 的每个计数器都是 16 位的，分频系数最大为 65536，所以必须用多个计数器串联起来工作。该题的答案不唯一。可以将 2MHz 的时钟送 CLK_0 ，计数器 0 分频系数设为 10000， OUT_0 连接 CLK_1 ，使得计数器 0 输出的 200Hz 脉冲作为计数器 1 的时钟输入，计数器 1 的分频系数设为 200， OUT_1 就可输出 1Hz 的方波。

```

MOV  DX, 353H
MOV  AL, 34H           ; 计数器 0 方式 2 的控制字: 00110100
OUT  DX, AL
MOV  AL, 77H           ; 计数器 1 方式 3 的控制字: 01110111
OUT  DX, AL
SUB  DX, 3
MOV  AX, 10000          ; 计数器 0 初值
OUT  DX, AL
MOV  AL, AH
OUT  DX, AL

```

```

INC    DX
MOV    AL, 00H           ; 计数器1初值
OUT    DX, AL
MOV    AL, 02H
OUT    DX, AL

```

8254 计数器级联时, 应注意常数装入时引起的计数误差。当用输出指令向 8254 某通道写入初值时, 必须经过一个时钟上升沿和一个时钟下降沿之后, 数值才装入计数寄存器, 在此之前读出的计数器内容均无效。本题是产生周期波形, 常数写入的误差只在产生第一个周期时存在, 对使用无影响。若计数器级联用作单次计数则必须注意初值装入引起的误差。

例 4.10 82C55A 的方式控制字和 C 口置 1 / 置 0 控制字都是写入控制端口的, 它们是怎样区分的? 要求 82C55A 的 A 口工作于方式 1 输入, B 口工作于方式 1 输出, 允许 A 口中断管理, 禁止 B 口中断管理; PC₇ 和 PC₆ 作基本输出用。试写出相应的控制字。其控制口地址为 243H。

解 82C55A 控制字的最高位(b₇)用以区分方式控制字和 C 口置 1 / 置 0 控制字。当 b₇ = 1 为方式控制字, b₇ = 0 为 C 口置 1 / 置 0 控制字。

```

MOV    DX, 243H
MOV    AL, 0B4H           ; B4H = 10110100
OUT    DX, AL             ; 方式控制字, A 口方式 1 输入, B 口方式 1 输出
MOV    AL, 09H           ; INTEA 置 1, 允许 A 口中断管理, 由 PC4 写入
OUT    DX, AL
MOV    AL, 04H           ; INTEB 置 0, 禁止 B 口中断管理, 由 PC2 写入
OUT    DX, AL

```

例 4.11 模/数转换器 ADC0809 通过并行接口芯片 82C55A 与系统总线相连, 如图 4.6 所示。设 82C55A 的端口地址为 80H~83H, 试编写程序段用查询方式完成通道 IN₁ 的模数转换。

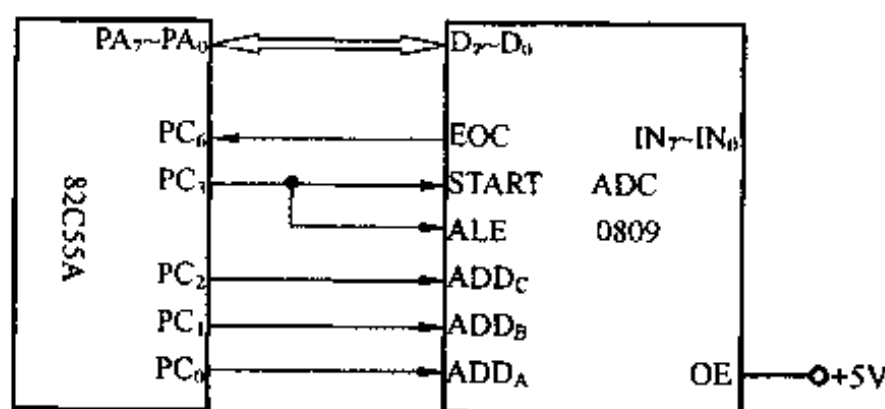


图 4.6 82C55A 与 ADC0809 连接图

解 ADC 0809 是 CMOS 的 8 位 A/D 转换器, 有 8 个模拟量输入通道 IN₇~IN₀,

8 位的数字量输出 $D_7 \sim D_0$ 。本例中 $D_7 \sim D_0$ 与 82C55A 的 A 口相连, 所以可通过读 A 口获得数字量。

ALE 为地址锁存信号, 高电平有效时, ADD_C 、 ADD_B 、 ADD_A 信号被锁存, 通过这 3 个引脚输入的信号选通 8 路模拟量输入中的一路进行模/数转换。本例中要求完成通道 IN_1 的模/数转换, 所以 ADD_C 、 ADD_B 、 ADD_A 应该分别输入低电平、低电平和高电平, 即 82C55A 的 PC_2 、 PC_1 、 PC_0 分别输出逻辑 0、逻辑 0 和逻辑 1。

START 是模/数转换的启动信号, 高电平有效, 也就是说可通过 82C55A 的 PC_3 来启动转换。

EOC 是转换结束的输出信号, 高电平有效, 那么通过查询 82C55A 的 PC_6 , 可以知道转换是否结束。

OE 是输出允许端, 当输入高电平时, $D_7 \sim D_0$ 输出有效, 否则输出高阻态, 本例中 OE 直接接高电平。

MOV	AL, 98H	: 82C55A 方式控制字, A 组和 B 组方式均设方式 0
OUT	83H, AL	: A 口输入, C 口高 4 位输入, 低 4 位输出
MOV	AL, 06H	: 82C55A 的 PC_3 置 0
OUT	83H, AL	
MOV	AL, 09H	
OUT	82H, AL	: 选择通道 IN_1 , 并启动模数转换
MOV	AL, 01H	
OUT	82H, AL	
L: IN	AL, 82H	
TEST	AL, 01000000B	: 测试 PC_6
JZ	L	: 为 0 等待
IN	AL, 80H	: 读 A 口获得数字量

4.3 学 习 自 评

4.3.1 自测练习

一、填空题

1. 8255A 具有 (1)、(2) 和 (3) 3 种工作方式。
2. 从 8254 计数器中读出的计数值 是读出瞬间的计数单元内容。
3. 8254 共有 个地址。
4. 串行通信方式分为 (1) 和 (2)。

5. RS-232C 是 (1) 与 (2) 间的接口标准。
6. 根据传输线路不同, 串行通信可分为 (1)、(2)、(3) 和 (4) 4 种方式。
7. 串行通信中“每秒传送的二进制位数”称为 (1)，“单位时间内线路状态变化的数目”称为 (2)。
8. I/O 端口寻址方式分两种, 一种是存储器映射方式, 即把 (1) 统一编址; 另一种是 I/O 映射方式, 即把 (2) 分别进行独立编址。
9. 8255A 的 C 口使用比较特殊, 除作数据端口外, 当工作在方式 1 和方式 2 时, 它的部分信号线被分配作 (1); 在 CPU 取 8255A 状态时, C 口又作为 (2) 用。
10. 串行通信比较适合于 (1) 微机系统之间的数据通信, 微机的串口 1 的 I/O 地址为 (2)。

二、选择题

1. 8255A 工作在方式 2 时, A 口 ()。
 - A. 只能作输入口
 - B. 只能作输出口
 - C. 作输入口或作输出口
 - D. 同时作输入口和输出口
2. 可编程定时器/计数器 8254 有 () 工作方式。
 - A. 2 种
 - B. 4 种
 - C. 6 种
 - D. 8 种
3. 计数器与定时器的工作原理是 ()。
 - A. 不完全相同的
 - B. 根本不同的
 - C. 相同的
 - D. 互不相关的
4. 可编程定时器/计数器 8254 的控制字为 () 个字节。
 - A. 1
 - B. 2
 - C. 3
 - D. 4
5. 8254 计数器启动计数的方式 ()。
 - A. 仅有软件方式
 - B. 仅有硬件方式
 - C. 有软件和硬件方式
 - D. 仅有门控信号触发
6. 向 8254 写入的计数初值写到了 () 中。
 - A. 计数值寄存器 CR
 - B. 输出锁存器 OL
 - C. 减 1 计数单元 CE
 - D. 控制字寄存器
7. 在使 8254 工作前, 须先向 8254 写入一个 (), 以确定 8254 的工作方式。
 - A. 控制字
 - B. 计数值
 - C. 状态字
 - D. 读回命令

- ### 三、问答题

1. 初始化命令字的作用是什么?
2. 为什么要对 8254 写控制字? 控制字是否可以读出? 计数初值是否可以读出? 可以从 8254 读出什么内容?
3. 定时器和计数器有什么相同和不同?
4. 串行通信与并行通信有什么不同?
5. 调制解调器在串行通信中有什么作用?
6. 什么是串行通信中的异步方式?
7. 输入缓冲的作用是什么? 输出锁存的作用是什么?
8. 统一编址和独立编址的优缺点各是什么?

四、编程题

1. 说明 PC/XT 定时器/计数器工作的地址范围, 各端口地址各为多少? 设置计数器 0 为 BCD 码计数方式, 工作于方式 3, CLK_0 输入端每来 5 个计数脉冲, 输出端 OUT_0 的高电平与低电平反相一次, 写出设置控制字的指令序列。
2. 8255A 的 A 口与共阴极的 LED 显示器相连, 若片选信号 $A_9 \sim A_2 = 11000100$, 问 8255A 的地址范围是多少? A 端口应工作在什么方式? 写出 8255A 的初始化程序。
3. 如图 4.7 所示, 分析各端口的地址, 并写出 8255A 的初始化程序段。
4. 如图 4.8 所示, 8255A 同开关和 LED 显示器相连, 要求 LED 显示器反映开关状态, 开关闭合对应于发光二极管发光。分析各端口地址, A 口、B 口工作在什么方式, 并写出相应程序段。

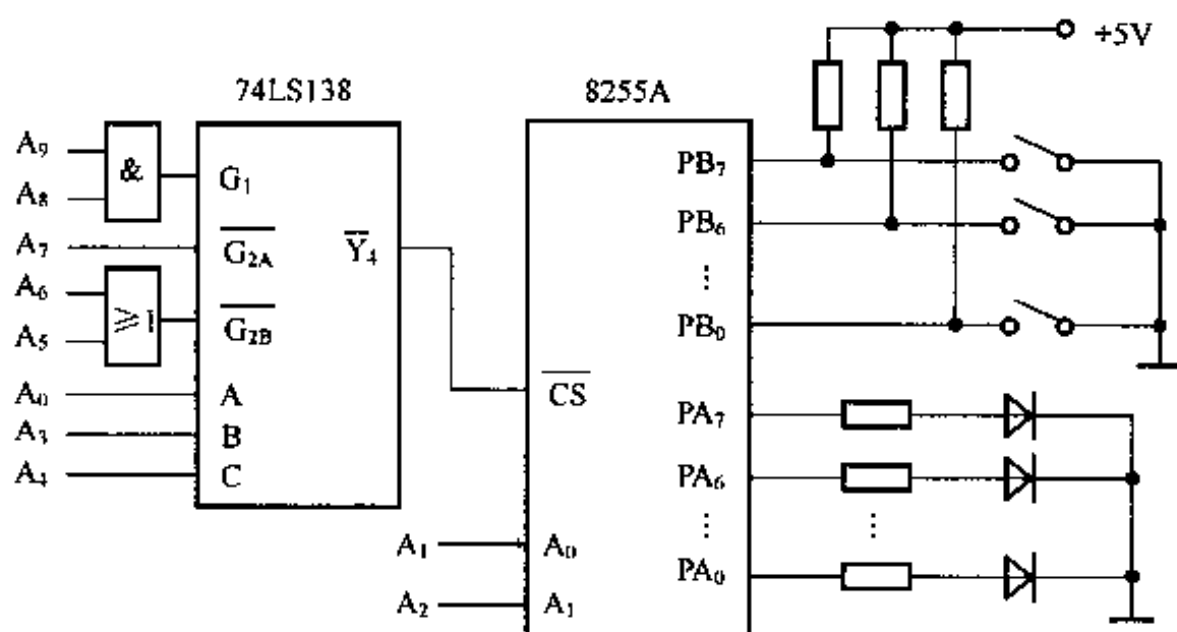


图 4.7

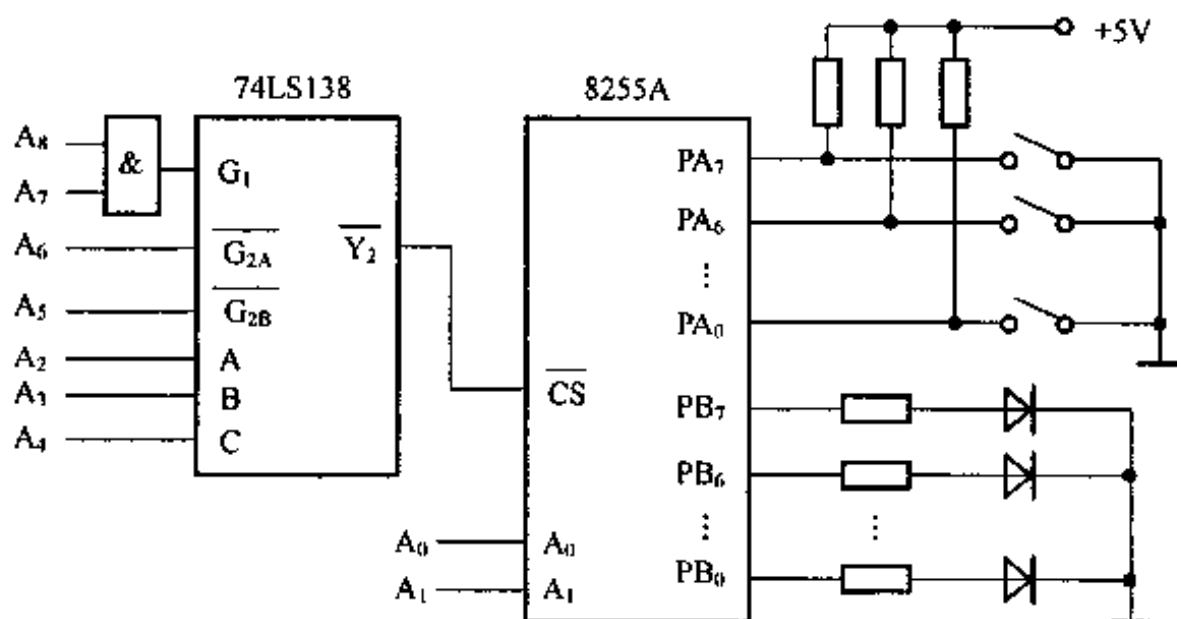


图 4.8

5. 利用 8255A 检测 8 个外部开关量的状态, 请画出接口电路图, 写出初始化程序段。

6. 在 IBM PC/XT 系统中, 使用 Ins 8250 作串行接口时, 若要求以 1200baud 的波特率发送一个字符, 字符格式为: 7 个数据位, 一个停止位, 一个奇校验位。试编写 8250 的初始化程序 (设 8250 的地址为 2F8H~2FFH)。

7. 在 IBM PC/XT 系统中, 要用 Ins 8250 以 300baud 的速率发送汉字编码信息, 试编写相应的初始化程序段。

8. 在 PC/XT 上做一个自发自收的异步串行通信实验, 要求通信波特率为 4800baud, 每帧由 1 位起始位、6 位数据位、1 个偶校验位、2 个停止位组成。实验前先要检查串口 1 异步适配板的存在性, 并填入板的基地址到基值区。试完成对 Ins 8250 的初始化编程。

9. 假设有 A/D 和 D/A 转换器与炉温控制设备相连接, A/D 转换器特性为:

当输入控制端出现下降沿时启动 A/D 转换，当转换完一个数据时，在输出控制端输出脉冲。要求采用 8255A 作为 PC 机与它们之间的接口，A 口连 A/D 转换器，B 口连 D/A 转换器，PC₄ 连 A/D 转换器的输出控制端，PC₃ 连 8259A 的 IRQ₉。试编写程序，采用中断方式将 A/D 转换的数据 X 进行 F(X) 运算，然后将结果从 D/A 转换器输出。假设 8255A 端口地址为 300H~303H，F(X) 运算的子程序已存在，其入口参数和出口参数均在 AL 中。

10. 对 IBM PC 串口 1 编制一个简易的仿终端程序，使其实现：当按键时，将该键的 ASCII 码送串口输出；并将从串口 1 收到的字符送显示器显示，假设接收用中断方式，发送用程序查询方式。设置串口参数为：数据长 8 位，无校验，2 位停止位，波特率为 9600baud，程序要使用 3 个 ROM BIOS 的驱动程序。

11. 8255A 作为 ADC0809 与 CPU 的接口，如图 4.9 所示。设当 CPU 送出的地址为 80H~83H 时， $\overline{Y_0}$ 输出低电平，当送出的地址为 84H~87H 时， $\overline{Y_1}$ 输出低电平。试编写程序段用查询方式完成通道 IN₇ 的模数转换。

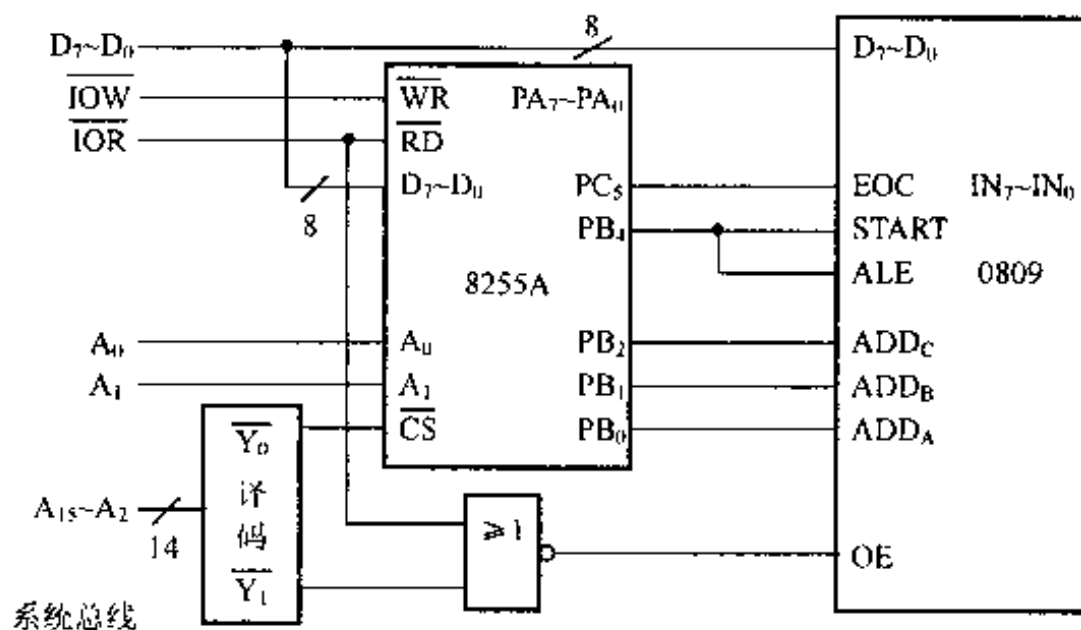


图 4.9

4.3.2 自测练习解答

一、填空题

- (1) 基本 I/O
 - (2) 选通 I/O
 - (3) 双向选通 I/O
- 不一定
- 4
- (1) 同步通信
 - (2) 异步通信
- (1) DTE
 - (2) DCE
- (1) 单工传输方式
 - (2) 半双工传输方式
 - (3) 全双工传输方式
 - (4) 多工传输方式

7. (1) 数据传输率 (2) 波特率
8. (1) 端口地址与存储器地址 (2) 端口地址与存储器地址
9. (1) 专用联络应答信号 (2) 状态口
10. (1) 远距离 (2) 3F8H~3FFH

二、选择题

1. D 2. C 3. C 4. A 5. C 6. A 7. A 8. D 9. C 10. C

三、问答题

1. 初始化命令字的作用是使芯片处于一个规定的基本工作方式,并在此方式下进行工作。

2. 8254有多种工作方式供选择,而且其内部寄存器是16位,数据线是8位,因此8254在使用前必须先向其写控制字,以决定以哪一种工作方式及读/写指示。8254的控制字寄存器只能写不能读,计数寄存器CR也只能写不能读。可从输出锁存器OL读出的当前计数值,从状态寄存器读出计数器的当前状态。

3. 定时器和计数器是同一器件——计数器件,其共同的特点是都有一个计数脉冲输入端,每输入一个脉冲,计数器就进行加1或减1计数。若计数器件的计数脉冲的频率固定,则可利用计数实现定时,这就是定时器。若计数器件的作用仅仅是记录输入脉冲的多少,则称为计数器。

4. 在串行通信中,数据在单条一位宽的传输线上一位一位地分时传送出去,数据传输速度较慢,但成本低,适用于远距离传输。

在并行通信中,数据在多条一位宽的传输线上同时传输出去,数据传输速度较高,但成本较高,适用于近距离通信。

5. 计算机中的数字信号在长距离传输中会严重失真,发送方使用Modem将数字信号调制成适合于在线路上传输的模拟信号,然后传输出去,接收方收到模拟信号后,再使用Modem将模拟信号解调为数字信号,送主机处理。

6. 异步传输方式允许发收双方不用共同时钟,采用起止同步控制方式,每个字符设有起始位,表示字符开始,在字符结束时有可选的奇偶校验位和停止位。

7. 输入缓冲的作用是为了防止外设数据干扰数据总线,以便于CPU打开三态缓冲器接收;输出锁存的作用是为了使CPU通过数据总线输出给外设的数据能保持,以便于外设接收。

8. 统一编址不设专门的I/O指令,内存指令均可用于外设,能提供极大的方便;但I/O端口占用了内存地址,就相对减少了内存可用范围,而且对内存操作的指令执行时间比I/O指令的执行时间长。

独立编址中内存地址和I/O端口地址相对独立,可分开设计,而且易于辨认内存和I/O端口的指令,I/O指令执行速度也较快;但用于I/O端口的指令功能较弱,在I/O操作中必须通过CPU寄存器进行中转才能完成。

四、编程题

1. (1) 地址范围: 80H~83H。计数器 0: 80H。计数器 1: 81H。计数器 2: 82H。控制寄存器: 83H。

(2) CLK₀ 输入端每接收 5 个计数脉冲, 输出端 OUT₀ 的高电平与低电平反相一次(对称方波), 则 CLK₀ 每输入 10 个脉冲, 对应于 OUT₀ 输出方波的一个周期, 即计数初值应为 10, BCD 码计数, 为 10H。初值小于 256, 工作于方式 3, 所以方式控制字为 00010111B=17H。

(3) 8254 初始化程序如下:

```
MOV AL, 17H
OUT 83H, AL
MOV AL, 10H
OUT 80H, AL
```

2. (1) 地址范围: 310H~313H。

(2) A 端口工作在方式 0 输出。

(3) 初始化程序如下:

```
MOV AL, 80H
MOV DX, 313H
OUT DX, AL
```

3. (1) A 端口: 310H, B 端口: 312H, C 端口: 314H, 控制寄存器: 316H。

(2) 初始化程序如下:

```
MOV AL, 82H          ; A 口方式 0 输出, B 口方式 0 输入
MOV DX, 316H
OUT DX, AL
```

4. (1) A 端口: 188H。B 端口: 189H。C 端口: 18AH。控制口: 18BH。

(2) A 端口方式 0 输入, B 端口方式 0 输出。

控制字: 10010000 (90H)。

(3) 初始化程序如下:

```
MOV AL, 90H
MOV DX, 18BH
OUT DX, AL
MOV DX, 188H
IN AL, DX
NOT AL
MOV DX, 189H
OUT DX, AL
```

5. (1) 硬件连接图, 如图 4.10 所示。

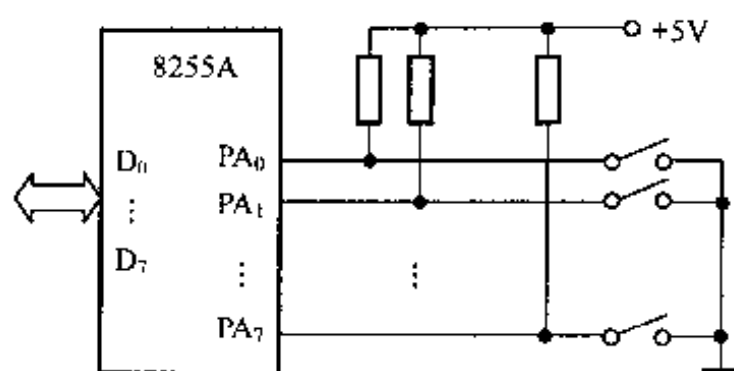


图 4.10

(2) 初始化程序段如下:

```

MOV AL, 90H          ; A 口控制字 (A 口方式 0, 输入)
OUT PORT, AL         ; 设控制寄存器的端口地址为 PORT
6. MOV AL, 80H
MOV DX, 2FBH         ; 使 DLAB 位 = 1
OUT DX, AL
MOV DX, 2F8H         ; 写入除数寄存器低字节, 对应 1200b
MOV AL, 60H
OUT DX, AL
INC DX
MOV AL, 00H          ; 写入除数寄存器高字节
OUT DX, AL
MOV AL, 0AH          ; 写线路控制寄存器, 定数据格式
MOV DX, 2FBH
OUT DX, AL

```

7. 每个汉字占用 2B, 1B=8b(机内码)。所以数据位取 8 位, 加上 1 位奇校验位, 一个停止位, 一个起始位, 共 11 位组成, 假设用 COM2, 地址为 2F8H~2FFH, 初始化程序如下:

```

MOV AL, 80H
MOV DX, 2FBH
OUT DX, AL
MOV DX, 2F8H
MOV AL, 80H
OUT DX, AL
INC DX
MOV AL, 01H

```



```

OUT  DX, AL
MOV  AL, 0BH
MOV  DX, 2FBH
OUT  DX, AL

```

8. 初始化程序如下:

```

MOV  DX, 3FAH
IN   AL, DX
TEST AL, 0F8H
JNZ  ERROR           ; 不存在则转显示错误
AND  DX, 0FFF8H      ; 串口 1 异步适配板存在则初始化
MOV  AX, 0040H
MOV  ES, AX
MOV  ES:RS-232-BASE, DX ; 填入板的地址到基址区
MOV  AL, 80H
MOV  DX, 3FBH
OUT  DX, AL          ; 设置 DLAB 的标志位为 1
MOV  AX, 0018H       ; 写除数寄存器, 4800baud 对应 0018H
MOV  DX, 3F8H
OUT  DX, AL
MOV  AL, AH
MOV  DX, 3F9H
OUT  DX, AL
MOV  AL, 1DH         ; AL=00011101
MOV  DX, 3FBH
OUT  DX, AL          ; 写 LCR, 6 位数据, 偶校验, 两个停止位
MOV  AL, 13H         ; 写 Modem 控制寄存器, 自发自收(LOOP=1)
MOV  DX, 3FCH        ; 使请求发送和数据终端就绪信号有效
OUT  DX, AL
MOV  AL, 0           ; 屏蔽中断
MOV  DX, 3F9H
OUT  DX, AL

```

9. 源程序如下:

```

STACK  SEGMENT  PARA  STACK 'STACK'
        DB      256 DUP(?)
STACK  ENDS

```

```

DATA    SEGMENT
        :

DATA    ENDS

CODE    SEGMENT

        ASSUME    CS: CODE, DS: DATA, SS: STACK

START   PROC FAR

        PUSH    DS
        XOR     AX, AX
        PUSH    AX                ; 保存 PSP 首地址, 以便返回 DOS
        MOV     AX, DATA
        MOV     DS, AX            ; 将 DATA 送 DS
        CLI                     ; 关闭中断
        MOV     AX, 0             ; 中断向量表段基址
        MOV     ES, AX
        MOV     DI, 1C4H          ; IRQ9 的中断号 71H×4=1C4H
        MOV     AX, OFFSET  INTP
        CLD
        STOSW                    ; 将中断服务程序偏移量送 1C4H 单元
        MOV     AX, SEG  INTP
        STOSW                    ; 将中断服务程序段基址送 1C6H 单元
        IN      AL, 21H
        AND     AL, 111111011B
        OUT     21H, AL           ; 开放主片 8259A 的中断 IRQ2
        IN      AL, 0A1H
        AND     AL, 111111011B
        OUT     0A1H, AL          ; 开放从片 8259A 的中断 IRQ9
        MOV     DX, 303H
        MOV     AL, 10110000B      ; 8255A 控制字: A 端口方式 1 输入, B 端口
                                   ; 方式 0 输出
        OUT     DX, AL
        MOV     AL, 00001001B
        OUT     DX, AL            ; 开放 A 端口中断: INTEA=1
        STI                     ; 开中断
        :                       ; 执行主程序

        RET                      ; 返回 DOS

```

```

START      ENDP
INTP       PROC FAR
            STI                      ; 开中断
            PUSH AX
            PUSH BX
            PUSH CX
            PUSH DX
            PUSH SI
            PUSH DI
            PUSH BP
            PUSH ES
            PUSH DS                  ; 保存程序现场
            MOV DX, 300H
            IN AL, DX                ; A/D 输入
            CALL FN                  ; 调用控制算法子程序
            MOV DX, 301H
            OUT DX, AL               ; D/A 输出
            MOV AL, 20H
            OUT 0A0H, AL             ; 向从 8259A 发 EOI
            MOV AL, 00001011B       ; OCW3: 读 ISR
            OUT 0A0H, AL
            IN AL, 0A0H
            XOR AL, AL
            JNZ EXIT1               ; ISR 不为全 0
            MOV AL, 20H
            OUT 20H, AL
EXIT1:     POP DS
            POP ES
            POP BP
            POP DI
            POP SI
            POP DX
            POP CX
            POP BX
            POP AX                  ; 恢复现场

```

```

                IRET                ; 中断返回
INTP            ENDP
FN              PROC    NEAR        ; 控制算法子程序
                :
                RET
FN              ENDP
CODE            ENDS
                END    START

```

10. 仿终端程序:

```

BUFSIZE    EQU    100
STACK      SEGMENT PARA  STACK 'STACK'
            DB          256 DUP(0)
STACK      ENDS
DATA       SEGMENT PARA  PUBLIC 'DATA'
ELFLAG     DB          0           ; 接收错误标志
BFFLAG     DB          0           ; 接收缓冲区满标志
BUFFER     DB          BUFSIZE DUP(0) ; 定义接收缓冲区
BUFPTR0    DW          0           ; 接收缓冲区头指针
BUFPTRIN   DW          0           ; 接收缓冲区尾指针
ELINE      DB  'THE LINE STATE ERROR!','$' ; 接收错误提示信息
BUFFILL    DB  'THE BUFFER FILLED!','$' ; 接收缓冲区满提示信息
DATA       ENDS
CODE       SEGMENT PARA  PUBLIC 'CODE'
START      PROC  FAR
            ASSUME  CS: CODE, DS: DATA, SS: STACK
            PUSH    DS
            MOV     AX, 0
            PUSH    AX                ; 标准程序首部
            MOV     AX, DATA
            MOV     DS, AX
            CLI                ; 关闭中断
            MOV     AX, 0
            MOV     ES, AX
            MOV     DI, 4*12        ; 分配给通信口 1 的中断类型码为 12
            MOV     AX, OFFSET  RINT

```

```

CLD
STOSW                                ; 设置中断服务程序的入口地址
MOV  AX, CS
STOSW
IN   AL, 21H
AND  AL, 0EFH
OUT  21H, AL                        ; 开 8259A 的 IRQ4 串口 1 中断
MOV  DX, 3FBH
MOV  AL, 80H                        ; DLAB 置 1
OUT  DX, AL
MOV  DX, 3F8H                       ; 设置除数锁存器
MOV  AL, 0CH                        ; 9600Baud
OUT  DX, AL
MOV  DX, 3F9H
MOV  AL, 0
OUT  DX, AL
MOV  DX, 3FBH
MOV  AL, 07H                        ; 设置数据格式: 8 个数据位, 2 个停止位, 无校验
OUT  DX, AL
MOV  DX, 3FCH
MOV  AL, 0BH
OUT  DX, AL                        ; 置 DTR、RTS 和 OUT2 有效, 允许请求中断
MOV  DX, 3F9H
MOV  AL, 01
OUT  DX, AL                        ; 只允许接收数据就绪中断
STI
FOREVER: MOV DX, 3FDH
IN   AL, DX                        ; 读线路状态寄存器
TEST AL, 20H                      ; 测试发送保持寄存器是否为空
JZ   ELDISP                        ; 不空, 转显示接收字符; 未按键转
MOV  AH, 1
INT  16H                          ; 测试是否有键入
JZ   ELDISP
MOV  AH, 0
INT  16H                          ; 获得按键 ASCII 码送 AL

```

```

        MOV    DX, 3F8H
        OUT    DX, AL                ; 数据送串口输出
ELDISP: MOV    BL, ELFLAG
        CMP    BL, 0                ; 判断接收是否发生错误
        JE     BFDISP
        MOV    AH, 9
        LEA    DX, ELINE
        INT    21H
BFDISP: MOV    BL, BFFLAG
        CMP    BL, 0                ; 判断接收缓冲区是否满
        JE     RDISP
        MOV    AH, 9
        LEA    DX, BUFFILL
        INT    21H
RDISP:  MOV    BX, BUFPTRO
        CMP    BX, BUFPTRIN        ; 判断缓冲区内有无字符
        JNZ    BDISP              ; 有, 转 BDISP
        JMP    FOREVER            ; 无, 转 FOREVER
BDISP:  MOV    AL, BUFFER[BX]      ; 取缓冲区字符
        PUSH   BX
        MOV    BX, 0
        MOV    AH, 14
        INT    10H                ; 显示接收数据
        POP    BX
        INC    BX
        CMP    BX, BUFSIZE        ; 超过缓冲区底部吗
        JC     BDISP1             ; 未, 转
        MOV    BX, 0              ; 转向缓冲区顶部
BDISP1: MOV    BUFPTRO, BX        ; 修改接收缓冲区头指针
        JMP    FOREVER
START  ENDP
; 接收数据中断服务程序
RINT   PROC    FAR
        STI
        PUSH   BX

```

```

        PUSH    AX
        PUSH    DX
        PUSH    DS
        MOV     AX, DATA
        MOV     DS, AX
        MOV     DX, 3FDH
        IN      AL, DX                ; 取线路状态
        TEST    AL, 1EH              ; 测试接收数据是否有错
        JNZ     LERROC               ; 错, 转错误处理
        MOV     DX, 3F8H
        IN      AL, DX                ; 输入串口字符
        MOV     BX, BUFPTRIN
        MOV     [BUFFER+BX], AL      ; 放入缓冲区
        INC     BX
        CMP     BX, BUFSIZE          ; 超过缓冲区底部吗
        JC      RINT3                ; 未, 转
        MOV     BX, 0                ; 已超过, 转向顶部
RINT3:  CMP     BX, BUFPTRO          ; 缓冲区已满否
        JZ      BF                   ; 满, 转 BF 处理
        MOV     BUFRTRIN, BX
RINT2:  MOV     AL, 20H
        OUT     20H, AL              ; 向 8259A 发 EOI 命令
        POP     DS
        POP     DX
        POP     AX
        POP     BX
        IRET
BF:     MOV     AL, 1
        MOV     BFLAG, AL            ; 置缓冲区满标志
        JMP     RINT2
LERROR: MOV     DX, 3F8H
        IN      AL, DX                ; 清除接收缓冲寄存器
        MOV     AL, 1
        MOV     ELFLAG, AL           ; 置接收错误标志
        JMP     RINT2

```

```

RINT ENDP
CODE ENDS
END START

11. MOV     AL,88H           ; 8255A 方式控制字: 10001000
      OUT    83H,AL         ; A、B 方式 0 输出, PC7~PC4 输入
      MOV     AL,07H
      OUT    81H,AL
      ADD     AL,10H        ; 选择通道 IN7, 启动转换
      OUT    81H           ; 给 START、ALE 送一正脉冲
      SUB     AL,10H
      OUT    81H,AL
L:    IN      AL,82H        ; 读 C 端口
      TEST    AL,00100000B  ; 测试 PC5
      JZ      L            ; EOC 输出低电平则继续查询
      IN      AL,84H        ; 使 OE 输入高电平, 获得数字量

```

4.4 教学建议

本章教学要抓住理论与实际两个方面。一方面是微机 I/O 接口的基础知识、异步串行通信和同步串行通信的基本原理、EIA RS-232 串行接口标准等基础理论, 另一方面是 8254 定时器/计数器、8255 并行接口、8250 串行接口等常用接口芯片的主要功能、内部结构、工作方式、控制字和状态字。并行 I/O 接口芯片和串行 I/O 接口芯片怎样与系统和外设连接; 怎样编写初始化程序和外设驱动程序是理论联系实际、微机应用技术的关键。为加强芯片互联和编程的训练, 在本章例题精析中给出了较多的实例, 供教学参考。

第 5 章 微机与外设的数据交换

5.1 知 识 要 点

5.1.1 微机与外设的数据交换方式

微机与外设进行数据交换时一般采用查询、中断、DMA 和通道方式，表 5.1 列出了简要的说明和性能比较。查询、中断、DMA 是 CPU 控制 I/O 方式，通道是 IOP（IO 处理机）控制 I/O 方式。

表 5.1 微机与外设的数据交换方式

	请求方式	数据交换	硬件支持	性能特点
查询方式	软件查询 外设状态	软件进行 数据传输	独占 CPU 无其他硬件	定时协调好 CPU 效率低
中断方式	硬件发生 中断请求	中断程序 传送数据	中断控制器 8259 等	CPU 分时操作、 中断处理费时
DMA 方式	DMAC 转发 总线请求	DMAC 控制 数据传送	DMA 控制器 8237 等	DMA 硬件控制需 CPU 参与
通道方式	CPU 建立 I/O 操作信息表	IOP 读信息表 控制 I/O 操作	IO 协处理器 8089 等	IOP 控制传数 CPU 进行其他操作

5.1.2 8259A可编程中断控制器（PIC）

8259A 是一个可编程的中断控制器，用于实现中断优先权管理、中断屏蔽、中断向量提供。单片的 8259A 可管理 8 级硬件中断，多片级联时，可用 9 片构成 64 级主从式中断系统，利用查询中断可以把中断扩展到 64 级以上。芯片共有 28 个引脚，内部结构包括如下 3 大部分。

① 总线和级联缓冲机构，包括数据总线缓冲器、读/写控制逻辑、级联/缓冲比较器。

② 中断管理机构，包括中断请求寄存器 IRR、中断服务寄存器 ISR、判优电

路,主要用于中断管理。

③ 中断控制逻辑,包括4个中断初始化命令字 $ICW_1 \sim ICW_4$ 和3个操作命令字 $OCW_1 \sim OCW_3$ 。在使用8259A时,首先要进行初始化编程,一般情况下,初始化命令字一旦设定,工作过程中就不再改变。操作命令字则由应用程序设定,用于对中断处理过程作动态控制。在一个系统运行过程中,可多次改写操作命令字。

8259A 通过编程可选择多种工作方式。

1. 设置优先级的方式

优先级设置方法可分为以下4种。

(1) 一般全嵌套方式

这是8259A最常用的方式。若在对8259A初始化时 ICW_4 的 $SFUM=0$,且没有设置其他优先级方式,则自动按一般全嵌套方式工作。在此方式下中断请求按0~7优先级处理。当一个中断被响应,即把中断类型码置于DB上时,中断服务寄存器ISR的对应位 IS_n 置为1,并保持到CPU发出中断结束命令(EOI)后才复位,以此为判优电路提供判优的依据。因为判优电路要将新接收的中断请求与当前的 IS_n 位比较,以决定中断优先级,优先响应较高级中断,它可形成中断嵌套。

(2) 特殊全嵌套方式

特殊全嵌套方式多用于多片级联且必须将响应的中断优先级保存在各从片的大系统中,此时特殊全嵌套方式仅设置主片中 ICW_4 的 $SFUM$ 为1。它和全嵌套方式基本相同,所不同的是在特殊全嵌套方式下,当主片处理某一级中断时,可响应同级的中断请求,从而实现从片对主片同级中断请求的特殊嵌套。这种方式是专门为多片8259A系统提供的,用来确认从片内部优先级的工作方式。

(3) 优先级自动循环方式

这种方式一般用在系统中多个中断源优先级相等的场合。在这种方式下,优先级队列不是固定不变的,一个IR受到中断服务后,其优先级自动降为最低。初始化优先级队列规定为 $IR_0, IR_1, \dots, IR_7$,由 OCW_2 的 $R=1, SL=0$ 决定系统是否采用优先级自动循环方式。

(4) 优先级特殊循环方式

这种方式和优先级自动循环方式唯一的区别是,优先级是通过编程确定的,既不是固定 IR_0 为最高级,也不是刚服务过的IR为最低级。它是当 OCW_2 的 $R=1, SL=1$ 时, $L_2L_1L_0$ 指定的IR优先级最低。

2. 屏蔽中断源的方式

(1) 一般屏蔽方式

一般屏蔽方式通过 OCW_1 ,使8259A中的屏蔽寄存器IMR中的一位或若干位置1来屏蔽IRR对应位的中断源。一个中断源的屏蔽不影响其他中断源的请求。

当某一中断请求被响应时,ISR 中相应位置 1,屏蔽了同级(一般全嵌套方式)和较低级中断请求。

(2) 特殊屏蔽方式

在某些希望一个中断服务程序能动态改变系统优先级结构的场合,常采用特殊屏蔽方式。即在此中断服务程序中,用 OCW_1 将屏蔽寄存器中本级中断的对应位置 1,即将本级中断屏蔽;然后写入 OCW_3 ,使 $ESMM=1$ 、 $SMM=1$,这样使中断服务寄存器中当前对应位自动清 0,为开放较低级中断请求提供可能。特殊屏蔽方式总是在中断处理程序中使用。采用这种方式后,本级中断在中断服务程序中将被屏蔽,对外界来说,好像 CPU 未处理任何中断。这样即使是最低级中断请求,也会得到响应。需要特殊屏蔽方式复位时,给 8259A 送 OCW_3 ,使 $ESMM=1$ 、 $SMM=0$;然后送 OCW_1 ,使本级中断的屏蔽位清除,最后向 8259A 送中断结束命令结束服务。

3. 结束中断处理的方式

在 8259A 中,当一个中断请求得到响应时,会将芯片内部的 ISR 的相应位置位;而当中断服务结束时,应使相应位复位,即为结束中断处理,否则 8259A 的中断控制功能就会不正常。8259A 有 2 种结束中断处理的方式,即自动结束方式和非自动结束方式,而后者又有一般中断结束方式和特殊中断结束方式之分。自动中断结束方式 AEOI 仅用于多个中断源不发生嵌套的单片 8259A 系统中,在 8259A 初始化时,使 ICW_4 的 AEOI 置为 1,即会在中断响应的第 2 个周期将 ISR 相应 IS_n 位清 0。

非自动结束方式中一般中断结束方式用在全嵌套的情况下,当用输出指令往 8259A 发一般中断结束命令时,8259A 会把当前 ISR 中最高的非零 IS_n 位复位,因为最高的 IS_n 位对应着最后一次被响应和处理的中断。具体操作是 OCW_2 中置 $SL=0$ 、 $R=0$ 、 $EOI=1$ 。

特殊中断结束方式用在非全嵌套方式下,因为此时只根据 ISR 无法确定哪一级中断是最后响应和处理的。此时在程序中要发一条特殊的中断结束命令,指出要清除当前 ISR 的哪一个 IS_n 位。这也是通过设置 OCW_2 来实现的,此时 $R=0$ 、 $SL=1$ 、 $EOI=1$ 、 $L_2L_1L_0$ 的状态指明了要复位的 IS_n 位。

4. 引入中断请求方式

(1) 边沿触发方式

边沿触发方式以上跳沿向 8259A 请求中断,上跳沿后可一直维持高电平,边沿触发方式是通过 ICW_1 的 $LTIM=0$ 来设置的。

(2) 电平触发方式

电平触发方式以高电平来申请中断。但在响应中断后必须及时清除高电平,

以防止引起第二次误中断。电平触发方式是通过 ICW₁ 的 LTIM=1 来设置的。

(3) 中断查询方式

这是一种用软件确定中断请求位的方式,一般在一个中断服务程序可为几个中断设备服务的场合使用。此时可将 IF 复位,禁止 CPU 用硬件响应中断请求,但是外设仍通过 8259A 申请中断(可为边沿触发或电平触发,由 ICW₁ 来设置),8259A 不使用 INT 信号向 CPU 申请中断。CPU 可用软件查询确定中断源,从而实现对设备的中断服务。CPU 的查询命令是通过 OCW₃ 的 P=1 设置来实现的,CPU 随后的 IN 指令($\overline{CS}=0, \overline{RD}=0$)作为中断响应。若有中断请求,则识别出最高级的中断请求,使 ISR 中相应位置位,转入相应的中断服务程序。

5.1.3 8237/82C37 DMA控制器(DMAC)

1. DMAC 总线主控和 I/O 接口的双重功能

DMAC 是总线主控设备,可以像 CPU 那样占用系统总线,以 DMA 方式控制存储器与 I/O 接口或存储器之间的高速数据交换;DMAC 又是 I/O 接口芯片,像其他总线的设备一样,接受 CPU 对它进行编程、读/写操作。

2. 8237 有 4 个独立的 DMA 通道

8237 有 4 个独立的 DMA 通道。每个通道可独立编程、自动预置;每个通道有 64 KB 的地址空间和计数功能,如若在超过 64 KB 地址空间的系统中进行 DMA 传送时,则需将超过的地址位通过 I/O 端口或页面寄存器来实现。8237 的 DMA 通道可以通过级联扩充,使用下级 HRQ 连上级 DREQ 传递 DMA 请求,使用上级 DACK 连下级 HLDA 传递 DMA 响应;没有 DMA 嵌套,也不限制级联的级数。

3. 8237 的内部结构

8237 内部功能分为定时和控制逻辑、优先权控制逻辑、命令控制逻辑、内部寄存器、数据和地址缓冲器组等。

(1) 定时和控制逻辑

该功能包括时钟、片选、复位、就绪输入信号,地址允许、地址选通输出信号,存储器和 I/O 读/写双向信号以及过程结束双向信号。

(2) 优先权控制逻辑

该功能包括 4 个通道的 DMA 请求输入和 DMA 响应输出信号以及总线请求输出和总线响应输入信号;两种优先权管理方式(固定优先权方式和循环优先权方式)。优先权表示多个通道在同时进行总线请求时,哪一个通道优先获得响应,一旦某个通道占有总线控制权,其他通道的总线请求都得不到响应,直到该通道服务结束。总线请求和响应没有类似中断嵌套的机制。

(3) 命令控制逻辑

CPU 用 $\overline{CS}, A_3 \sim A_0, \overline{IOW}, \overline{IOR}$ 信号预置内部寄存器和发布软件命令,在 DMA

操作期间对控制字译码,选择 DMA 通道和传送方式,执行特定的操作。

(4) 内部寄存器和软件命令

8237 命令控制逻辑可以预置内部寄存器和发布软件命令。8237 四个通道共用的寄存器有命令寄存器、状态寄存器、暂存寄存器和先/后触发器;每个通道都有工作方式寄存器、当前地址寄存器和基地址寄存器、当前字计数寄存器和基字计数寄存器、1 个请求标志位和 1 个屏蔽标志位。8237 有 3 条软件命令:清除先/后触发器命令、主清除命令和清除屏蔽寄存器命令。

(5) 数据及地址缓冲器组

缓冲器组中包括有 1 个 8 位 I/O 缓冲器——8 位数据总线和高 8 位地址总线分时复用;1 个 4 位 I/O 缓冲器——CPU 控制总线时与 $\overline{CS}$ 、 $\overline{IOW}$ 或 $\overline{IOR}$ 信号一起读/写内部寄存器等,DMAC 控制总线时,送出 8237 产生的最低 4 位地址;1 个 4 位输出缓冲器——DMAC 控制总线时,送出 8237 产生的次低 4 位地址。

4. 8237 的工作方式

(1) 操作类型

检验:8237 控制总线、产生地址、响应 $\overline{EOP}$ 、不发读写信号、不传输数据。

DMA 读:用 $\overline{MEMR}$ 从存储器读数据到总线上,用 $\overline{IOW}$ 直接写入 I/O 口。

DMA 写:用 $\overline{IOR}$ 从 I/O 口读数据到总线上,用 $\overline{MEMW}$ 直接写入存储器。

(2) 传送方式

单次传送:每次 1B,两个单次传送之间总线至少释放 1 个周期。

成组传送:连续传送,直至计数终止或 $\overline{EOP}$ 有效时为止。

请求传送:连续传送,是 DREQ 一直有效的成组传送,DREQ 无效时暂停。

级联传送:如前述级联用于增加 DMA 通道,而其他 3 种方式的不同在于释放总线的条件。

(3) 自动预置

8237 可以设置为自动预置和非自动预置方式。自动预置时,在 $\overline{EOP}$ 有效时,基寄存器内容重新进入当前寄存器中,屏蔽位不变,8237 继续工作。非自动预置时,在 DMA 传送终止时,清请求位和置屏蔽位。

(4) 存储器之间传送

只有存储器之间传送的数据要进入 8237,在暂存寄存器中暂存。8237 没有 I/O 读/写周期,而有 2 个存储器周期:1 个读和 1 个写,共 8 个状态。

5. 8237 的 DMA 请求和屏蔽

(1) DMA 请求

每个通道都有一个请求标志位,当其置 1 时可以产生总线请求。请求位有规定的优先级,它不能被屏蔽。使请求标志位置 1 有 2 个方法:①软件请求,用程

序使其置 1；②硬件请求，通过未屏蔽的 DREQ 输入使其置 1。

(2) DMA 屏蔽

每个通道有一个屏蔽标志位，当其置 1 时禁止 DREQ 有效请求使请求标志置位。4 个通道的屏蔽标志位可以组成一个屏蔽寄存器，有 3 种方法操作：①写屏蔽寄存器单个位；②写屏蔽寄存器所有位；③清屏蔽寄存器所有位。

6. 8237 的 DMA 工作过程

① 初始化 8237：起始地址、数据长度、工作方式、操作类型等。

② 外设 $\rightarrow$ 8237 $\overline{\text{DREQ}}_i = 1 \rightarrow$ 8237 $\overline{\text{HRQ}} = 1 \rightarrow$ CPU $\text{HOLD} = 1 \rightarrow$ CPU $\text{HLDA} = 1 \rightarrow$ 8237 $\text{HLDA} = 1 \rightarrow$ 8237 $\overline{\text{DACK}}_i = 1 \rightarrow$ 外设。

③ DMA 周期

- 通道 CH_i 当前地址寄存器输出高 8 位地址 $\rightarrow \text{DB}_7 \sim \text{DB}_0 \rightarrow \overline{\text{ADSTB}} \downarrow$ 时存入地址锁存器 $\rightarrow \text{AEN} = 1$ 送上 $\text{A}_{15} \sim \text{A}_8$ ； CH_i 当前地址寄存器输出低 8 位地址 $\rightarrow \text{A}_7 \sim \text{A}_0$ 。

- 8237 发 $\overline{\text{MEMR}}/\overline{\text{IOR}}$ ，读出数据送到 $\text{D}_7 \sim \text{D}_0$ ；8237 发 $\overline{\text{IOW}}/\overline{\text{MEMW}}$ ， DB 上数据送入外设/存储器。

- 一个 DMA 周期后，通道 CH_i 当前地址寄存器 ± 1 ，当前字计数器寄存器 -1 ，根据设定的工作方式，重复或终止。

- 当前字计数器寄存器计数终止 (TC)，发出 $\overline{\text{EOP}}$ 有效，终止 DMA 传送。

自动预置：基寄存器 $\rightarrow$ 当前寄存器，请求位、屏蔽位不变。

非自动预置：复位请求位、置屏蔽位、 $\overline{\text{HRQ}} = 0$ 、释放总线。

- 存储器之间传送需要 2 个总线周期 8 个状态，必须由软件置源通道 CH_0 的请求位启动，也可以对外部 $\overline{\text{EOP}}$ 作出响应。这种方式可用于块搜索，当一个数据比较器检测到匹配时，向 8237 发有效的 $\overline{\text{EOP}}$ 来终止搜索。

5.2 例题精析

例 5.1 微机和外设的 4 种数据交换方式是_____、_____、_____和_____。前 3 种可以称为_____，是因为对数据的处理更需要 CPU 的参与。

解 微机和外设的 4 种数据交换方式是软件查询方式、中断方式、DMA 方式和通道方式。前 3 种可以称为CPU 控制的 I/O 方式，是因为对数据的处理更需要 CPU 的参与。

例 5.2 8237A 芯片内部有 4 个独立的 DMA 通道，4 个通道在请求 DMA 传送时的优先权可通过编程设定为_____和_____。每个通道均可通过编程选择_____、_____、_____和_____4 种传送方式之一进行 DMA 传送，每次允许传

送的最大长度为_____KB。

解 8237A 芯片内部有 4 个独立的 DMA 通道, 4 个通道在请求 DMA 传送时的优先权可通过编程设定为固定方式和循环方式。每个通道均可通过编程选择单字节、数据块、请求和级联方式 4 种传送方式之一进行 DMA 传送, 每次允许传送的最大长度为 64KB。

例 5.3 对 8237A 4 个通道基地址和当前地址寄存器、基址计数寄存器和当前字计数寄存器进行测试检查, 先写入 FFFFH 后再读出, 比较读/写操作是否正确。若正确, 则再写入 0000H, 同样读出校验, 若仍正确, 则认为 8237A 工作正常, 就可以对其初始化; 若有错, 则执行暂停命令。

解 程序中用标号 DMA 表示其首地址。

初始化程序如下:

```

        MOV    AL,04H                ; 写命令寄存器
        OUT    DMA+08H,AL            ; 禁止 DMAC 工作
        OUT    DMA+0DH,AL           ; 送主清除命令
        MOV    AL,0FFH
C16:    MOV    BL,AL
        MOV    BH,AL
        MOV    CX,8
        MOV    DX,DMA
C17:    OUT    DX,AL
        OUT    DX,AL
        IN     AL,DX                  ; 读入低字节
        MOV    AH,AL
        IN     AL,DX                  ; 读入高字节
        CMP    BX,AX                  ; 判断是否正确
        JE     C18
        IHLT
C18:    INC     DX                      ; 地址加 1, 指向下一个寄存器
        LOOP   C17
        INC     AL                      ; 使 AL=0, 再循环一次
        JZ     C16

```

评述 对 8237A 进行初始化编程时, 首先利用硬件产生 RESET 信号, 或软件复位命令对 8237A 复位, 使 8237A 进入空闲状态。这样才可使 8237A 处于从属地位, 进行编程。本题是对 8237A 的 4 个通道的基地址寄存器及当前地址寄存器、基址计数寄存器和当前字计数寄存器进行检测。端口地址分别为 DMA+(0~7)

共 8 个, 故设置计数指针 CX 为 8。这样两次循环即可完成全部题目要求。

例 5.4 利用 8237A 的通道 2, 由磁盘输入 32KB 的一个数据块, 传送至内存 68000H 开始的区域, 采用地址增变化、成组方式, 传送完不自动预置, 磁盘的 DREQ 和 DACK 都是高电平有效。请编写初始化程序。

解 设首地址设为 DMA

```

DMA EQU    0000
OUT  DMA+0DH,AL      ; 送主清除命令
MOV  DX,DMA+4        ; DMA+4 是通道 2 的基地址和当前地址 R
MOV  AL,0
OUT  DX,AL           ; 写入地址低 8 位, 先/后触发器已清 0
MOV  AL,80H
OUT  DX,AL           ; 写入地址高 8 位, 先/后触发器现为 1
MOV  DX,0081H        ; 0081H 是页面寄存器的地址输出端口
MOV  AL,6
MOV  DX,AL           ; 输出地址最高 4 位 A19~A16, 这样 20 位地址为 68000H
MOV  DX,DMA+5        ; DMA+5 是通道 2 基址计数和当前字计数 R
MOV  AL,0FFH
OUT  DX,AL           ; 写入初值低 8 位
MOV  AL,7FH
OUT  DX,AL           ; 写入初值高 8 位, 字节总长为 32KB
MOV  DX,DMA+0BH      ; DMA+0BH 是方式寄存器的端口
MOV  AL,10001010     ; 对通道 2 进行方式选择, 成组传输、DMA 读, 地
                        ; 址增 1 变化, 禁止自动预置功能
OUT  DX,AL
MOV  DX,DMA+0AH      ; DMA+0AH 是屏蔽寄存器的端口
MOV  AL,02H
OUT  DX,AL           ; 使通道 2 清除屏蔽
MOV  DX,DMA+08H      ; DMA+08H 是命令寄存器的端口, DREQ 和 DACK
                        ; 均为高电平有效, 固定优先级, 正常时序, 启动工作
MOV  AL,80H
OUT  DX,AL

```

评述 8237A 设置有 3 个软件复位命令: 主清、清先/后触发器和清屏蔽寄存器命令。注意, 这些命令只要对某个适当的地址执行写操作就会自动执行, 而不管写入的是什么数。

这是一个典型的 8237A 初始化编程步骤:① 对 8237A 进行复位;② 将 DMA 传送中内存的起始地址的低 16 位写入基地址和当前地址寄存器,从 I/O 端口输出最高 4 位地址(或从页面寄存器输出最高 4 位地址);③ 将传送字节总长写入基字计数和当前字计数寄存器;④ 写入正确的方式字;⑤ 写入正确的屏蔽字;⑥ 写入正确的命令字。

在为通道选择方式字时,请注意 DMA 写和 DMA 读的含义。所谓 DMA 写是指 DMAC 在申请到总线控制权后,将当前地址寄存器的内容送到地址总线上,发出 $\overline{\text{IOR}}$ 信号和 $\overline{\text{MEMW}}$ 信号,从 I/O 端口读数据写入内存,通常又称为 DMA 输入;而在 DMA 读时,发出 $\overline{\text{MEMW}}$ 信号和 $\overline{\text{IOW}}$ 信号,从内存读数据写入 I/O 端口,通常又称为 DMA 输出。

DMA 传送可以由外设发出有效的 DREQ 而引发,同时也可以由软件设置 DMA 请求标志而引起。清除屏蔽寄存器命令使 DMAC 的 4 个通道的全部屏蔽位被清除,使各通道均能接受 DMA 请求。

例 5.5 8237A 只有 8 位数据线,请问它是如何支持 16 位 RAM 与 16 位 I/O 接口之间的 DMA 传送的,同时 DMA 控制器选择存储器到存储器的传送操作应具备什么条件?

解 DMA 控制器取得总线控制权后,向系统总线发出控制存储器和 I/O 设备操作需要的读/写信号。传送数据的途径是 I/O 设备接口→系统数据总线→存储器,不经过 DMA 控制器。DMA 控制器只起控制作用,通道的全部 DMA 操作必须在存储器的偶地址即字边界上进行。

DMA 选择存储器到存储器的传送操作必须具备通道 0、通道 1 均未被占用的条件。通道 0 产生源地址,通道 1 产生目标地址和进行减 1 计数。

例 5.6 DMA 控制器的初始化应包括哪些内容?设 8237A 的端口地址为 00H~0FH,通道 0 的页面寄存器地址为 87H。使通道 0 工作在成组方式,地址增变化,自动预置功能,把从内存 25000H 开始的 1024 字节传送给外设端口。DACK 为高电平有效, DREQ 为低电平有效,固定优先级,正常时序,不扩展写信号,非存储器到存储器传送。试设计 8237A 的初始化程序。

解 8237A DMAC 初始化编程一般包括以下内容:

- (1) 输出主清除命令;
- (2) 写入基地址与当前地址寄存器;
- (3) 写入基字计数与当前字计数寄存器;
- (4) 写入页面寄存器;
- (5) 写入方式寄存器;
- (6) 写入相关屏蔽位;
- (7) 写入命令寄存器;

(8) 若需软件请求启动(如存储器至存储器传送), 则需写入请求标志位, 一般是外设端口发请求开始 DMA 过程。

8237A 初始化程序如下:

```

OUT  0DH,AL          ; 发主清除命令
MOV  AL, 00H          ; 写基地址和当前地址寄存器
OUT  00H, AL
MOV  AL, 50H
OUT  00H, AL
MOV  AX, 03FFH        ; 写基字计数和当前字计数寄存器
OUT  01H, AL
MOV  AL, AH
OUT  01H, AL
MOV  AL, 02H          ; 写页面寄存器
OUT  87H, AL          ; 通道 0 的页面寄存器地址
MOV  AL, 98H          ; 写方式寄存器
OUT  0BH, AL
MOV  AL, 0C0H         ; 写命令寄存器
OUT  08H, AL
MOV  AL, 00H
OUT  0AH, AL          ; 写屏蔽标志位

```

例 5.7 IBM PC/XT 的 8259A 工作于单级方式、程序 EOI 方式、缓冲器方式, 编写 8259A 的初始化程序。

解 初始化程序如下:

```

CLI                      ; 关闭中断
MOV  AL, 13H
OUT  20H, AL             ; 写 ICW1 的内容
MOV  AL, 08H
OUT  21H, AL             ; 写 ICW2 的向量类型号
MOV  AL, 09H
OUT  21H, AL             ; 写 ICW4 的内容
MOV  AL, 00000100B
OUT  21H, AL             ; 除 IR2 外, 其余清屏蔽
STI                      ; 开中断

```

例 5.8 利用 PC/AT 机器中 8259A 的 IRQ₉ 中断写出采用中断传送数据的程序框架。

解 程序框架如下:

```

STACK SEGMENT PARA STACK 'STACK'
        DB                                256 DUP(?)
STACK ENDS                                ; 定义堆栈段
DATA SEGMENT
        ;                                ; 定义数据段
DATA ENDS
CODE SEGMENT
        ASSUME CS:CODE, DS:DATA, SS:STACK
START PROC FAR
PUSH    DS
XOR     AX, AX
PUSH    AX                                ; 保存 PSP 首地址, 以便返回 DOS
MOV     AX, DATA
MOV     DS, AX                            ; 将 DATA 送入 DS
CLI                                           ; 关闭中断
MOV     AX, 0                               ; 开始初始化向量表中的 IRQ9
MOV     ES, AX
MOV     DI, 1C4H                            ; IRQ9 的中断类型码为 71H
MOV     AX, OFFSET INTP
CLD
STOSW                                       ; 中断服务程序偏移量送到 1C4H 处

MOV     AX, SEG INTP
STOSW                                       ; 中断服务程序段地址送 1C6H 处
IN      AL, 21H
AND     AL, 11111011B                       ; 从 8259 连至主 8259IR2
OUT     21H, AL                             ; 开放从 8259A 的中断
IN      AL, 0A1H
AND     AL, 11111101B                       ; IRQ9 连在从 8259IR1
OUT     0A1H, AL                           ; 开放 IRQ9 中断
        ;                                ; 其他的初始化
STI
        ;                                ; 完成开中断
        ;                                ; 开始主程序

```

```

RET                                ; 返回 DOS
START  ENDP
INTP   PROC  FAR
    STI                                ; 开中断
    PUSH  AX
    PUSH  BX
    PUSH  CX
    PUSH  DX
    PUSH  SI
    PUSH  DI
    PUSH  BP
    PUSH  ES
    PUSH  DS                        ; 保存服务程序用的现场
    :
    :                                ; 中断传送
    :
    MOV  AL, 20H
    OUT  0A0H, AL                  ; 向从 8259A 发 EOI, OCW2
    MOV  AL, 00001011B
    OUT  0A0H, AL                  ; 取从 8259A 的 ISR, OCW3
    IN   AL, 0A0H
    JNZ  EXIT1                     ; 从 8259A 还有中断要处理
    MOV  AL, 20H
    OUT  20H, AL                   ; 向主 8259A 发 EOI
EXIT1:  POP  DS
        POP  ES
        POP  BP
        POP  DI
        POP  SI
        POP  DX
        POP  CX
        POP  BX
        POP  AX                    ; 恢复现场
        IRET                       ; 中断返回
INTP   ENDP

```

CODE ENDS ; 代码段结束

END START ; 程序结束

例 5.9 写出把 AL 中字符送给打印机的子程序。

解 子程序如下:

```
PRINT PROC NEAR
    PUSH AX
    PUSH DX ; 保存寄存器的内容
    MOV DX, 378H
    OUT DX, AL ; 传送字节到打印机端口
    MOV DX, 379H
WAIT: IN AL, DX
    TEST AL, 80H ;  $\overline{\text{BUSY}}=0$ , 打印机忙
    JZ WAIT ; 查询并等待打印机空闲
    MOV DX, 37AH
    MOV AL, 0DH
    OUT DX, AL ; 输出控制信号 00001101
    MOV AL, 0CH
    OUT DX, AL ; 输出 00001100, 有效的选通脉冲将数据送打印机
    POP DX
    POP AX ; 恢复寄存器的内容
    RET
PRINT ENDP
```

5.3 学 习 自 评

5.3.1 自测练习

一、填空题

1. DMA 方式的中文意义是 (1) , 它适用于 (2) 数据传送。
2. 8237 的 4 种传送方式包括 (1) 、 (2) 、 (3) 和 (4) 。
3. 通常的数据传送方式控制有 4 种, 分别为 (1) 、 (2) 、 (3) 和 (4) 。
4. 采用 DMA 方式时, 完成存储器和外部设备之间的数据传送需要 (1) 个总线周期。数据传送速度受 (2) 和 (3) 的限制。
5. DMAC 通过 (1) 、 (2) 和 (3) 3 种方式来占有总线控制权。
6. DMA 控制器启动的总线周期有: (1) 周期、 (2) 周期。

二、选择题

1. 8086CPU 的 I/O 地址空间为 ()。
A. 64KB B. 1MB C. 256B D. 1024B
2. 向 8237A 主清除命令地址 (DMA+0DH) 输出 FFH 值后, 8237A 所处的状况是 ()。
A. 允许 8237A 操作, 允许各通道的 DMA 请求
B. 允许 8237A 操作, 禁止各通道的 DMA 请求
C. 禁止 8237A 操作, 允许各通道的 DMA 请求
D. 禁止 8237A 操作, 禁止各通道的 DMA 请求
3. 利用 8259A 芯片把中断扩大到 64 级以上时, 对与从 8259A 相联的 8259A 上某一 IR 的请求的响应方式只能是 ()。
A. 向量中断方式 B. 查询中断方式
C. 显式中断方式 D. CALL 调用方式
4. 级联的从 8259A 上较高级的中断请求能够中断该片上较低级的中断服务, 则主 8259A 不能工作在 ()。
A. 特殊全嵌套方式 B. 一般全嵌套方式
C. 固定优先级方式 D. 一般屏蔽方式
5. CPU 在执行 OUT DX, AL 指令时, () 寄存器的内容送到数据总线上。
A. AL B. DX C. AX D. DL
6. 若 8259A 工作在优先级自动循环方式, 则 IRQ₄ 的中断请求被响应并且服务完毕后, 优先级最高的中断源是 ()。
A. IRQ₃ B. IRQ₅ C. IRQ₀ D. IRQ₄
7. 8086 中断系统可以管理 () 种中断。
A. 16 B. 1K C. 256 D. 128
8. 两片 8259A 级联后可管理 () 级中断。
A. 15 B. 16 C. 32 D. 64
9. 当 8237A 芯片设置为存储器到存储器操作, 通道 0 开始读存储器的启动条件是 ()。
A. 源存储器提出 DREQ B. 通道 0 屏蔽位清除
C. 目标存储器提出 DREQ D. 通道 0 请求位置位
10. 为实现 DMA 传输, 对 DMA 通道的初始化不能在 () 进行。
A. DMA 控制器取得总线控制权之前
B. 上电启动过程中
C. DMA 控制器取得总线控制权之后
D. CPU 访存操作完成之后

11. 在每一次进行 DMA 传输前,有必要对通道实施初始化,其原因是()。

- A. 字计数器和地址寄存器初值不同
- B. 通道传送方式和操作类型不同
- C. 上次服务结束通道被屏蔽
- D. 上述一个或多个原因组合

12. 虽然某个 DMA 通道是开放的,但 CPU 对其编程期间,8237A 芯片对优先级较高的 DREQ 的有效请求是()。

- A. 肯定会响应的
- B. 可能会响应,取决于 HLDA 状态
- C. 肯定不会响应
- D. 可能会响应,取决于 HRQ 状态

三、问答和编程题

1. 8259A 的中断服务寄存器 ISR 的某一位置 1,其含义是什么?

2. 8259A 可编程中断控制器中断系统应具有的功能?

3. 8259A 优先级方式选择有哪些?

4. 什么是 DMA 传送方式?可编程 DMA 控制器有哪些基本功能模块?简要叙述 DMA 的工作过程。

5. DMAC 只有 16 位地址线,它是怎样实现 PC/XT 机器 1MB 空间的寻址功能的?

6. CPU 和外设之间的数据传送方式有哪几种?无条件传送方式通常用在哪些场合?

7. 相对于查询方式,中断方式有什么优点?和 DMA 方式比较,中断方式有什么缺点?

8. DMA 控制器的低 4 位地址线为什么是双向的?什么时候往 DMA 控制器传送地址?什么时候 DMA 控制器往地址总线传输地址?

9. 试说明 DMA 读传送数据的过程。在 PC/XT 机上单字节传送一次要几个时钟周期?加上两次传送中间至少要间隔一个总线周期,问在 XT 机上最高 DMA 传输速率为多少?在读传输时,外设接口的地址不送上地址总线,DMAC 怎样识别数据传向哪个外设端口?

10. 编写一个子程序,使其功能实现为每批数据传送设置 DMA 通道 2 的参数(8237 的命令寄存器已经设置)并启动通道 2。

5.3.2 自测练习解答

一、填空题

1. (1) 直接存储器存取

(2) 批量高速

2. (1) 单次 (2) 成组 (3) 请求 (4) 级联
3. (1) 查询方式 (2) 中断方式 (3) DMA 方式 (4) I/O 处理机方式
4. (1) 1 (2) 存储器的存取速度 (3) 外部设备传输特性
5. (1) CPU 子系统释放总线处于高阻状态 (2) 暂停 CPU 时钟脉冲
(3) 利用 CPU 不访问总线的时间间隔
6. (1) 存储器读和 I/O 写总线周期 (2) I/O 读和存储器写总线周期

二、选择题

- | | | | | | |
|------|------|------|-------|-------|-------|
| 1. A | 2. B | 3. B | 4. B | 5. A | 6. B |
| 7. C | 8. A | 9. D | 10. C | 11. D | 12. B |

三、问答和编程题

1. 说明相应的中断服务程序正在被 CPU 执行或挂起, 若 ISR 中多位是 1, 则说明是中断嵌套。

2. 中断系统应具有如下功能:

- ① 对多中断源请求, 软件可禁止与允许每个请求;
- ② 判别中断优先级, 响应优先级别最高的请求;
- ③ 可中断嵌套, 较高级的中断请求可中断较低级的中断服务;
- ④ 响应中断后, 能自动转向中断处理程序, 处理结束后自动返回主程序。

3. 8259A 优先级方式选择有如下几种。

- ① 一般全嵌套: 固定优先级, IR_0 最高, IR_7 最低。
- ② 特殊全嵌套: 与上基本相同, 响应同级中断请求。
- ③ 优先级自动循环: 某级服务结束时, 优先级自动循环使刚服务过的级为最低。
- ④ 优先级特殊循环: 某级服务结束时, 优先级循环至特指的 IR 为最低。

4. DMA 的概念: DMA 传送方式不需处理器干预可完成 M 与 I/O 间的数据传送。DMA 期间系统总线由其他主控模块驱动, 该主控模块要为系统提供地址及控制信号。DMA 控制器芯片是一个主控模块, 与处理器配合可实现系统的 DMA 操作。

可编程 DMA 控制器有如下的基本功能模块: 地址寄存器、字计数寄存器、工作方式寄存器、命令寄存器、数据和地址缓冲器、定时和控制逻辑、优先权控制逻辑和命令控制逻辑等。

DMA 的工作过程: ① I/O 向 DMAC 发 DMA 请求; ② DMAC 向 CPU 发总线请求; ③ CPU 响应 DMAC 的总线请求; ④ DMAC 向 I/O 发 DMA 响应信号; ⑤ DMAC 发内存地址; ⑥ DMAC 发读/写控制信号; ⑦ DMA 传送开始; ⑧ DMA 传送结束。

5. 系统地址总线为 20 位, 而 8237 只能输出 16 位地址。为了能访问所有存

储单元,引入 DMA 页面寄存器,存放 4 个 DMA 通道的高 4 位地址 $A_{19} \sim A_{16}$ 。当 $\overline{\text{DMAAEN}}$ 为低电平时,页面寄存器的输出允许端 $\overline{\text{READ}}$ 有效,送出选中通道的高 4 位地址。它与 8237 送出的 16 位地址一起形成 20 位地址 $A_{19} \sim A_0$ 。这 20 位地址信息能寻址全部 1MB 存储空间。

6. CPU 和外设之间的数据传送方式通常有无条件方式、程序查询方式、中断方式、直接存储器存取(DMA)方式和通道方式。无条件传送方式适合于外设总是处于准备好的情况,例如,读取开关状态或设置开关状态,控制 LED 显示器的显示等。

7. 中断方式可以使 CPU 与外设并行工作,所以比起查询方式来,CPU 的工作效率提高了。但中断方式仍然要由 CPU 通过指令来传送数据,对高速设备来说速度太慢了。DMA 方式是在 DMA 控制器控制下进行数据传送的,速度上限主要取决于存储器的存取速度和外设的传输特性,中断方式传输速度不及 DMA 方式快。

8. DMA 控制器是一种总线控制器件,在 DMA 方式时,它替代 CPU 控制系统总线,完成外设与内存单元之间或者内存单元与内存单元之间的数据传送,所以它必须能向地址总线发出地址。在 DMA 控制器接管总线之前,CPU 必须对它进行初始化编程,以确定 DMA 的工作方式、传送计数、存储单元起始地址等等,为此 CPU 需要对 DMAC 内的寄存器寻址,DMAC 必须能接受地址信息。由于 DMAC 内有 16 个寄存器,所以它的低 4 位地址线是双向的。

9. DMA 读传输是把系统内存中的数据传送给接口板。DMA 读周期被启动后,8237A 首先把存储器的地址送上系统地址总线,接着 $\overline{\text{MEMR}}$ 信号有效,将存储器数据送上数据总线,然后在 DMAC 的 $\overline{\text{IOW}}$ 信号的控制下,数据总线上的数据写入 I/O 端口。在 PC/XT 机上单字节传送一次要 6 个时钟周期,加上两次传输中间至少要间隔一个总线周期(4 个时钟周期),时钟周期为 210ns,则 XT 机上最高 DMA 传输速率为

$$\frac{1}{(6+4) \times 210 \times 10^{-9}} \text{KB} = 476 \text{KB/s}$$

在读传输时,外设接口的地址不送上地址总线,DMAC 是靠 DMA 响应信号 $\overline{\text{DACK}_i}$ 来选择数据传向的外设端口的。在设计接口电路时,让 $\overline{\text{IOW}}$ 和 $\overline{\text{DACK}_i}$ (对应于该 I/O 口的)同时有效时,数据被锁存。

10. 程序如下:

入口参数:

AL: DMA 方式控制字

ES: BX: 内存起始地址

DH: 传送的扇区数

出口参数: CF 为 1, 跨页; CF 为 0, 不跨页

DMA EQU 0

DMA_SET_UP PROC NEAR

PUSH CX

CLI

OUT DMA+12, AL ; 先/后触发器清 0

OUT DMA+11, AL ; 设置通道 2 的方式字

MOV AX, ES ; 开始将 ES:BX 地址转换为 20 位地址

MOV CL, 4

ROL AX, CL

MOV CH, AL

AND AL, 0F0H

ADD AX, BX

JNC J33

INC CH

; (AX) 送通道 2 的基地址寄存器和当前地址寄存器

J33: PUSH AX

OUT DMA+4, AL

MOV AL, AH

OUT DMA+4, AL

MOV AL, CH

AND AL, 0FH ; 取出地址高 4 位

OUT 81H, AL ; 送页面寄存器

; 求传送字节数, 并送到 AX 中, 字节数 = 扇区数 × 512 字节/扇区

MOV AH, DH

SUB AL, AL

SSHR AX, 1

POSH AX ; 扇区数乘 128 送 AH

MOV AH, ES:[SI+3] ; 从软盘参数块取每扇区字节参数送 AH

MOV CL, AH

POP AX

SHL AH, CL ; AX 中得到传送字节总数

DEC AX ; AX 为要传送字节总数

; 向通道 2 基字节计数寄存器和当前字节计数寄存器送初值

PUSH AX

```

OUT    DMA+5, AL
MOV    AL, AH
OUT    DMA+5, AL
    II                                ; 开中断, 以下判断是否跨页
POP    CX                            ; 弹出传送字节数减 1 送 CX
POP    AX                            ; 弹出传送地址初值低 16 位送 AX
ADD    AX, CX                        ; 检测是否跨页, 跨页时另作处理
; 恢复 CX, 启动通道 2 工作并返回
POP    CX
MOV    AL, 2
OUT    DMA+10, AL                    ; 清通道 2 的屏蔽位
RET
DMA_SET_UP    ENDP

```

5.4 教 学 建 议

本章主要介绍了微机与外设的数据交换方式、可编程中断控制器 8259/82C59、8237/82C37 控制器、多功能 I/O 接口芯片 82380 的结构、编程以及级联等方面的内容。对于本章的教学, 应侧重于两个方面: 一方面要对教材中所介绍芯片的工作原理、组成结构有清晰的认识和比较全面的了解; 另一方面要加强编程方面的训练, 做到能够针对题目的要求根据具体的芯片编写出可执行的汇编语言程序。在内容和时间安排上, 可着重讲授 8259 和 8237 芯片, 对于 82380 可作一般介绍以辅导自学。需要讲清 82380DMA 控制器中两组通道的优先权及两组优先权结合时的排列; 以及 82380 可编程中断控制器的内部结构。

第6章 总线技术

6.1 知 识 要 点

6.1.1 总线技术概述

1. 总线的定义

总线是模块之间或者设备之间传送信息、相互通信的一组公用信号线，是系统在总线主控制器的控制下，将发送器发出的信息准确地传送给某个接收器的信号载体或通路。

2. 总线的特点

公用性，即可以同时挂接多个模块设备。

3. 总线的优点

- ① 简化了系统结构，便于系统设计制造。
- ② 大大减少了连线数目，便于布线，减小体积，提高系统可靠性。
- ③ 便于接口设计，所有与总线连接的设备均采用类似的接口。
- ④ 便于系统的扩充、更新与灵活配置，易于实现系统模块化。
- ⑤ 便于设备的软件设计，所有接口的软件就是对不同的口地址进行操作。
- ⑥ 便于故障诊断和维修，同时也降低了成本。

总线技术已被现代的计算机系统和许多电子设备广泛采用。

4. 总线的分类

(1) 按照传输信息的性质分类

可以将总线分为三类：数据总线 DB 是双向总线，用于在各模块之间交换信息；地址总线 AB 是微处理器发出的单向或双向总线，用于指定将数据送往何处或数据来自何处；控制总线 CB，整体而言是双向总线，但在内部有些是单向的，有些是双向的。控制总线包括用于存储器和 I/O 读/写操作控制的基本控制线以及数据传输握手线、总线判决线、中断控制线等。其中，数据总线和地址总线比较简单，控制总线因微处理机型号的不同而相差很大。不同的微处理器控制总线上

的信号线的种类、时序关系、有效电平和边沿都不同。

(2) 按照信号线的功能分类

可以将总线分为：基本信息总线——包括数据线、地址线及内存和 I/O 的读/写控制信号线等；数据联络总线——控制启动和停止总线操作、实现传送同步的信号线，是为保证总线上能容纳各种存取速度的设备而设计的信号线；判决总线——包括总线判决（总线请求、总线确认线）和中断判决（中断请求线、中断响应线）等几类。

(3) 按照层次分类

可以将总线分为：片内总线——位于微处理器或 I/O 芯片内部，用于片内各功能单元之间的互连；芯片总线——用于单板计算机或一块 CPU 插件板的电路板内部，即芯片一级的连接；系统总线（内总线、板级总线、部件总线）——用于微机系统内各插件板之间的连接，使得系统配置灵活、简单并便于开发；设备总线（外总线、通信总线）——用于系统之间的连接，如各微机系统之间或微机系统与仪器或其他电子设备之间的连接；局部总线——其概念有多种含义，它是从多处理机系统中引出的，存取速度比系统总线快，从逻辑上是芯片总线，但是用于多个模块的连接，是介于芯片总线和系统总线之间的一种总线。一般将具有总线仲裁能力的多主控模块间总线称为系统总线，不具有总线仲裁能力的模块间总线称为局部总线。

5. 总线的标准化与总线规范

在计算机系统的各部件之间采用总线连接的情况下，各部件间的通用性为用户安装和使用带来了方便。而且，微机系统制造厂家在设计系统时，总是设法使产品的总线能够连接更多的设备以得到广泛的市场。这样就有规范化的可通用的总线。

采用通用总线标准可以为不同模块、设备的互联提供一个标准的界面。这个界面为两端的模块和设备提供透明性，界面的任何一边只需要根据总线标准的要求来实现接口的功能，不必考虑另一边的接口方式。

每种总线标准都有详尽的规范说明，一般包括下列内容。

- ① 机械结构规范。规定模块尺寸、总线插头、连接器的形状和尺寸等规格。
- ② 功能结构规范。规定每个引脚信号的名称和功能及其相互作用的协议、时序、信息流向、信息管理规则等。
- ③ 电气规范。规定信号工作时的逻辑电平（高/低）、最大额定负载能力以及动态转换时间等。

6. 总线的性能指标

总线能否保证模块间的通信通畅是衡量总线性能的关键指标。主要有以下几个方面的性能指标：

(1) 总线宽度

即数据线的条数，以及总线传输信息的串并行性。并行总线在不同的信号线上同时传输一个数据字的不同位，而串行总线则在同一根信号线上分时传输同一数据字的不同位。

(2) 总线定时协定

即总线上采用同步还是异步定时，取决于传输数据的两个模块之间约定的协议。一般有同步协定、异步协定、半同步协定和分离式协定。

(3) 总线传输率

即系统在给定工作方式下所能达到的数据传输率，在给定方式下单位时间内能够传输的字节数或者比特数。

(4) 总线频宽

即总线本身所能达到的最高传输率，又称为标准传输率或最大传输率。

7. 总线判决和握手技术

(1) 总线的操作过程

总线数据传输周期可分为4个阶段：①总线请求和仲裁阶段；②寻址阶段；③传输阶段；④结束阶段。

(2) 总线使用权的分配

为保证在任何时刻最多只有一个主控设备占用总线，通常采用的方法有：①令牌总线 ②冲突检测 ③总线仲裁。

(3) 总线仲裁技术

① 集中仲裁。

(a) 菊花链式查询方式。这种方式为每个使用总线的部件设置一定的优先级，在逻辑连接上离总线控制部件（仲裁器）越近的部件拥有最高优先级。菊花链式总线判决方式如图6.1所示。为分配总线使用权，在控制总线中增加三根信号线作为总线控制线，它们是总线请求BR、总线允许BG和用于轮询判决的BB。

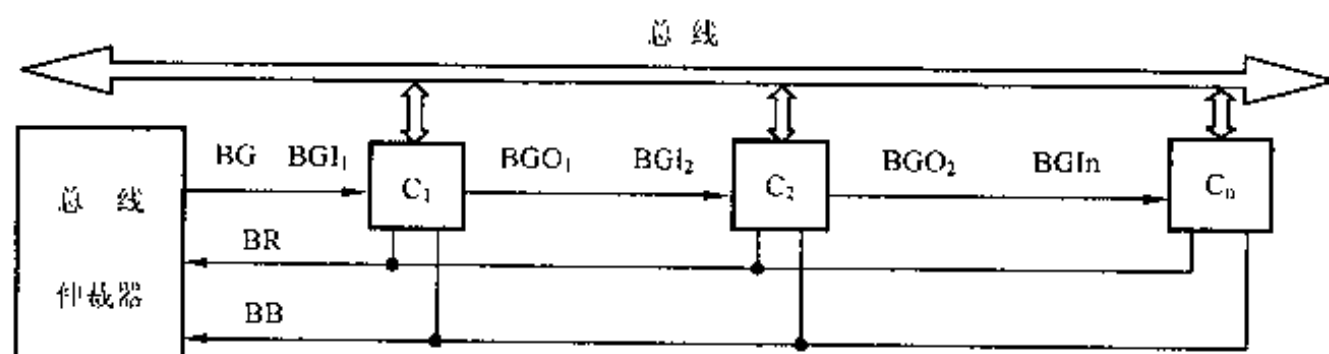


图 6.1 菊花链式总线判决方式

菊花链式结构实现简单，只用几根控制线即可实现总线的优先级控制，便于增减总线上的设备，总线控制器的结构与部件数量无关。但是，它对电路故障敏

感,若某部件发生故障,其后的所有部件甚至整个菊花链均不能正常工作;仲裁速度慢,总线响应信号串行传送,主控设备越多仲裁时间越长;优先级不能改变,逻辑连接上远离控制部件的设备有可能会“饿死”,因为没有控制机制保证获得总线使用权的设备不长期占用总线。

(b) 计数器定时查询方式(轮询判决)。在这种方式中,若总线上有 N 个部件,则在总线控制部件内设置一个计数器,可以从 0 计数至 $N-1$,每个值对应一个部件。不论哪个部件要使用总线,均通过 BR 提出申请。主控制部件通过地址总线定时送出计数器的当前值。提出申请的部件检查地址总线,若发现其上的值与自身的编号相等,则取得总线使用权,并通过置 BB 有效通知控制部件。若控制部件在一定时间内未收到 BB 有效通知,则令计数器加 1 (或减 1),发下一个地址。

若计数器从 0 开始递增,则优先级顺序与部件编号一致。若每次计数从上次计数终止的值开始,则从统计效果上看,各部件的优先级基本相同。

(c) 独立请求方式(并行裁决)。该方式由控制部件对申请进行排队和管理,并可改变或屏蔽某些部件的总线请求。它的申请和响应信号线本身是以专线而非总线形式传递信息的,如图 6.2 所示。因此,它速度快、可使用软件灵活控制、对故障不敏感,但是电路复杂,不易增加设备。

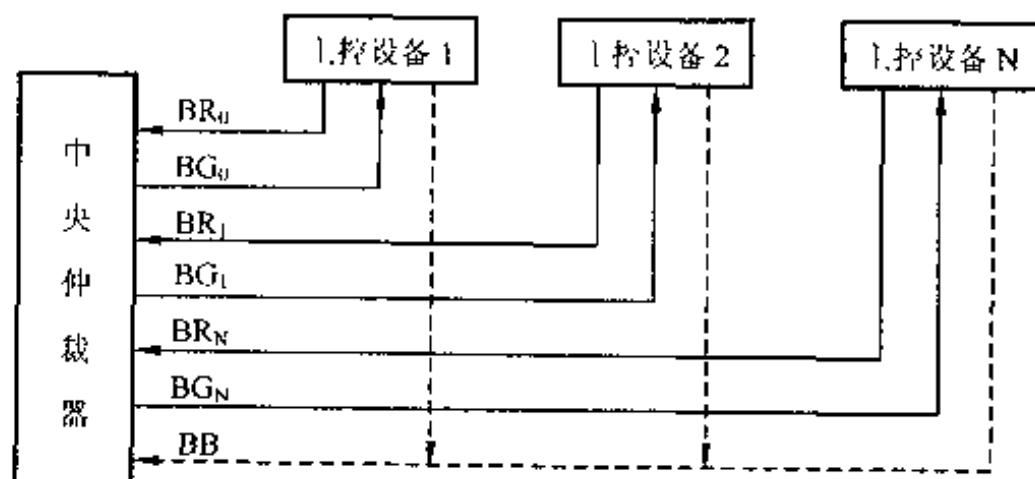


图 6.2 独立请求总线判决方式

(d) 二维判决方式。这种方式综合了前几种方式的优点,在一个系统中综合使用菊花链和并行判决,如图 6.3 所示。总体而言是并行判决,但并行判决通道中的部分或全部通道可以使用串行判决。二维判决适合于主控设备很多的场合。

② 分布仲裁。

分布仲裁的仲裁器分布于各个主控设备中,没有独立的中央仲裁器。

图 6.4 所示的是一种基于优先级的分布仲裁方法。为实现分布仲裁,应能在各个主控设备之间传递必要的总线控制信息。每个设备都事先设定一个优先级,用一个 n 位二进制数表示。最高优先级的主控设备优先数为 n 位全 1,而最低优先级的主控设备优先数为 n 位全 0。

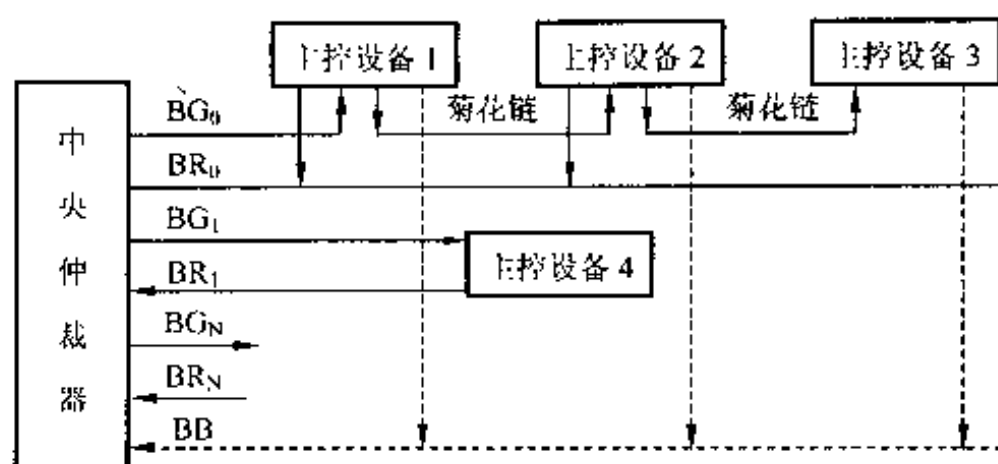


图 6.3 二维总线判决方式

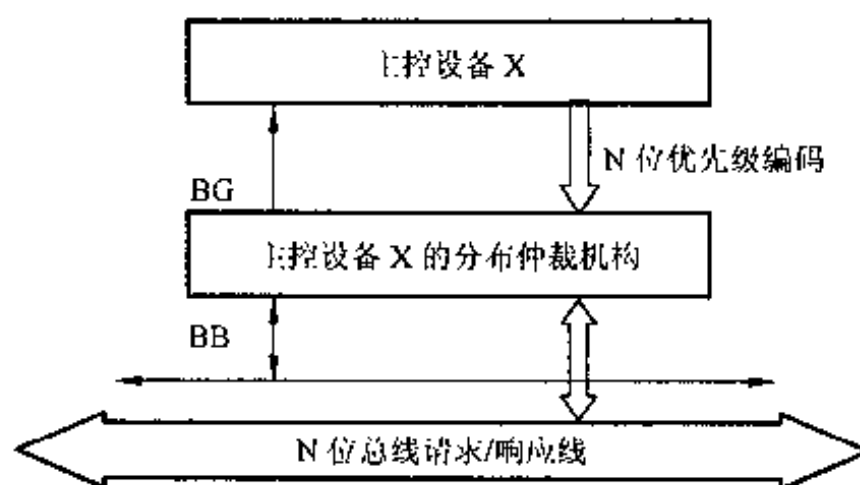


图 6.4 分布总线判决方式

(4) 总线传输联络技术

共享总线的各部件间进行数据传输需要收发双方通过联络实现同步，以便进行高速而可靠的寻址和数据传输。总线联络的作用是控制每个总线周期中数据传输的开始和结束，以及整个总线周期中的每个子周期的开始和结束。这需要靠总线上的定时协定解决。定时协定主要有4种：同步总线协定；异步总线协定；半同步总线协定；分离式总线协定。

6.1.2 系统总线

1. IBM PC 总线

1981年，IBM定义了一套用于在PC机的主机板上增加扩展卡的系统总线，在PC系统中称为扩展槽或I/O通道，一般称为PC总线或XT总线。该总线是为Intel 8088（CPU）的IBM PC及其带硬盘的增强型号IBM PC/XT定义的。8088是准16位微处理器，所以该总线是8位系统总线。它由8088CPU芯片总线经驱动器、锁存器、数据收发器、总线控制器、中断控制器、DMA控制器以及其他逻辑重新组合扩展而成。

IBM PC总线的主要特点如下：共有62根引脚，采用边缘连接方式，即靠电

路板的边缘进行电气连接,并起到一定的机械固定作用。在系统板上双列插槽,插槽由塑料基座构成,基座中央有金属簧片。扩展卡上为双面“金手指”端片。

IBM PC 总线的主要性能如下:8 位数据线;支持 8 级硬件申请的 6 个中断请求;支持 4 个 DMA 通道;存储器和 I/O 通道数据存取能力;最高时钟频率 5MHz;最大数据传输率 4MB/s。

2. ISA 总线

ISA 总线是在 XT 总线上进行了扩展,构成与 XT 总线向上兼容、具有更宽的数据通道、更多的中断线和 DMA 通道的 16 位系统总线,一般称为 AT 总线。

ISA 总线的主要特点如下:8 位 ISA 扩展 I/O 插槽由 62 个引脚组成,用于 8 位的插卡;8/16 位的扩展插槽除了具有一个 8 位 62 线的连接器外,还有一个附加的 36 线连接器,这种扩展 I/O 插槽既可支持 8 位的插卡,也可支持 16 位的插卡。

ISA 总线的主要性能如下:24 位地址线,可直接寻址的内存容量为 16MB;8/16 位数据线;62+36 引脚;最大位宽 16b;中断请求扩大到 15 级;DMA 通道增加到 7 个;开放式总线结构,允许多个 CPU 共享系统资源,具有总线仲裁功能;最高时钟频率 8MHz;最大数据传输率 16MB/s。

3. EISA 总线

EISA 总线在结构上与 ISA 总线有良好的兼容性,保护了厂商和用户巨大的软硬件投资;同时又充分发挥和利用 32 位微处理机的功能,使之在图形技术、光存储器、分布处理、网络通信、数据处理等需要高速处理能力的地方发挥作用。EISA 总线的推出打破了 IBM MCA 结构对微计算机发展的垄断。

EISA 插槽既与 ISA 插卡兼容,又与 EISA 插卡兼容。在 EISA 插卡中使用 32 位数据线,能达到 33MB/s 的传输率。

EISA 的主要性能指标与 ISA 相比,有以下优点:开放式结构,EISA 和 ISA 兼容,现有的 ISA 扩充板可以用于 EISA 总线上;32 位地址线,直接寻址范围为 4GB;16/32 位数据线;支持突发的传送周期;最高时钟频率 8.3MHz;最大数据传输率 33MB/s。

EISA 插槽由于要与 ISA 和 EISA 插卡兼容,因此在结构上和以往的插槽不同,采用了双层结构,EISA 插卡的金手指也是双层结构。

4. STD 总线

STD 总线是美国 Prolog 公司于 1978 年宣布的 8 位工业标准微机总线。它采用公共母板结构:其总线布置在一块母板上,板上安装若干个 56 线双列插座,插座对应的引脚都连到同一根信号线上。系统采用模块式结构,各种功能模块都按照标准插件尺寸制造,可插入任意插座,只要模块的信号、引脚都符合 STD 规范,就可以在 STD 总线上运行。它适用于工业现场监测与控制,工作环境适应范围宽、实时性高、组态灵活、开发手段简易。

STD 总线上, 多个中断源的中断优先级控制有两种方法实现: 串行中断优先链和并行中断优先链。STD 总线支持多微处理器系统, 即支持多总线控制器系统, 有两种总线优先级控制: 串联总线优先级控制和并联总线优先级控制。

该总线是以 Zilog 公司的 8 位 CPU Z80 为基础制定的, 适用于大部分 8 位微处理器, 因而取名标准总线。它具有逻辑电源线和辅助电源线、数据线、地址线和控制线等。

STD 经过扩展后, 依靠地址、数据线复用可以支持 16 位 CPU。为了能应用于 32 位处理机, 近年又定义了 STD32 标准, 并与 8 位的模块保持兼容。

5. 微机和其他常用系统总线

(1) PC/104 总线

PC/104 总线是应用于超小型嵌入式 PC 系统的总线。这种 PC 机由于体积小、结构紧凑而在各种工业控制中很受欢迎, 并被嵌入到对体积和功耗要求都很高的产品中, 例如, 医疗仪器、实验仪器、通信装置、商用终端 (POS 电子收款机等)、军用电子设备、机器人等。目前使用 PC/104 总线的嵌入式 PC 机在工业控制等领域已相当流行, 很多厂家生产了系列化的功能模块。

PC/104 总线有两个总线插头, 其中 P_1 有 64 脚, P_2 有 40 脚, 共 104 脚。这种微机系统无论是主机还是总线, 在机械结构上均非常小, 但在硬件和软件上与 PC 总线完全兼容 (物理尺寸和机械结构不兼容, 但电气和功能兼容), 实际上是为了更好地满足工业控制或小型化要求而开发出来的小型化 PC 机型。PC/104 总线和 PC 总线使用的引脚数目不一样, 多出的引脚均接地。

使用 PC/104 总线的嵌入式 PC 机系统的特点如下。

① 使用超小尺寸的模块, 全部模块包括 CPU 模块在内尺寸均为 $90\text{mm} \times 96\text{mm}$, 而一般 PC 机总线板尺寸要大得多。

② PC/104 总线采用自堆结构, 取消了底板和插槽, 利用模块上的堆装总线插座将各模块堆叠连接在一起, 组装紧凑而灵活。

③ 总线驱动电流小, 功耗低。为适应小型化要求, 各模板均采用 VLSI 器件、门阵列、ASIC 芯片以及大容量固态盘。

(2) Multibus (多总线)

多总线是 Intel 公司在 1977 年推出的用于单板机与其扩展板之间以及多处理器之间相互通信的 16 位微机系统的总线。1982 年, IEEE 在多总线的基础上制定了支持 8 位/16 位微机、有 24 位寻址能力并适用于多处理器结构设计的 IEEE-796 总线标准, 后来称为 Multibus I。

多总线是一种异步的多重处理系统总线。多总线是由并行系统总线、扩充的局部总线、串行系统总线、多通道 DMA 总线和 I/O 扩充总线五种不同的总线组成的。多个 CPU 通常在各自的局部总线上运行, 并有自己的局部存储器和局部 I/O

系统, 当需要访问公用存储器或公用 I/O 时, 才通过总线仲裁器占用系统总线。

多总线信号线连接在 P_1 、 P_2 两个插头上, P_1 有 86 线, P_2 有 60 线 (仅用 4 根线作为地址扩展)。从功能上多总线分为: 控制信号线、地址和禁止线、数据线、中断线、总线控制线等。

Multibus I 的升级版本 Multibus II 以支持 32 位微机和多机系统为宗旨, 并向 OSI(开放式系统互联)的 7 层模型靠拢。

(3) VME 总线

VME (Versa Module Eurocard) 总线是 Motorola 等公司开发的、经欧洲广泛使用之后改进而成的高速、高性能 32 位总线。1985 年定为 IEEE-P1041 标准。VME 总线是异步总线、主从结构, 具有多个主控设备的仲裁逻辑。它支持具有优先级的矢量中断。

该总线支持多计算机/多处理器结构。为避免系统总线成为多个处理机相互通信的瓶颈, VME 在系统中增加了信息通道。它由局部总线 VME、串行总线 VMS 以及系统总线组成。另外还有 4 个子总线: 数据传送总线、仲裁总线、中断总线和公用总线。

VME 总线有 P_1 、 P_2 两个连接器, 每个连接器分 3 列, 每列 32 脚, 共 96 脚。 P_2 只用中间 32 脚, 两侧 64 脚由用户定义, 只支持板内, 不连总线。

6.1.3 局部总线

局部总线 (Local BUS) 是 PC 体系结构的重大发展, 它打破了数据 I/O 的瓶颈, 使高性能 CPU 的功能得以充分发挥。从结构上看, 所谓局部总线是在系统总线和 CPU 芯片总线之间增加的一级总线。由于独立于 CPU 的结构, 使总线形成了一种独特的中间缓冲器的设计, 从而与 CPU 及时钟频率无关, 因此用户可以将一些高速外设, 如网络适配卡、图形卡、硬盘控制器等从系统总线上卸下而通过局部总线直接挂接到 CPU 芯片总线上, 使之与高速的 CPU 芯片总线相匹配, 而不需要担心在不同时钟频率下会引起性能上的分歧。

1. PCI 总线

PCI 总线有严格的规范, 能够支持多个外围设备。具有高性能、兼容性强、高可靠性和可扩展性等特点。

(1) 高性能

① 高吞吐量 (高总线频宽)。PCI 总线的时钟频率最高为 33MHz (最新版本可以支持最高 66MHz), 它与 CPU 是异步工作的, 与 CPU 的时钟频率无关。总线宽度为 32 位, 并可扩展到 64 位, 所以其带宽可达 $33 \times 4 = 132\text{MB/s}$ 或者 $33 \times 8 = 264\text{MB/s}$, 甚至更高。

② 支持突发读/写方式。新型的高速微处理器均支持突发读/写操作, PCI

总线能够很好地支持总线的这种操作。这种线性（顺序）寻址方式可以从内存某一地址起连续读/写数据，但只需传送一次地址。每次传送时，总线控制设备会通过地址总线传送本次突发的开始地址，并进行一次数据的读/写。然后每次由被访问的存储器或外设自动地将地址加 1 而不需要传送下一个地址，便可读出或写入数据流内的下一个数据。线性突发传送非常有效地利用总线的带宽传送数据，减少不必要的地址传送。这种数据传送方式特别适合于多媒体数据传输和数据通信。

③ 支持并发工作。PCI 总线上的外设可以与 CPU 并发工作。一般设计良好的 PCI 控制器具有多级缓冲，例如，CPU 在向 PCI 总线上设备执行写操作时，只需将一批数据快速写入缓冲，数据从缓冲器传送到 PCI 外设的过程可以完全在 PCI 控制器的控制下自动执行而无需 CPU 的任何干预，CPU 可以去执行其他操作。这种并发性提高了整体性能。

④ 总线时延低。总线时延由仲裁时延、总线使用权获取时延和从设备时延构成。PCI 桥路提供很短的延迟通道，时延长度的最坏情况可以预测。每个设备上的延时器规定设备使用总线的最长时间，以便 CPU 优化系统性能。

⑤ 支持总线主控技术。允许智能设备在需要时取得总线控制权，以加速数据传输。

(2) 独立于处理器的结构

PCI 总线是一种不依附于某个具体处理器的局部总线。从结构上看，PCI 是在 CPU 和系统总线之间插入的一级总线，具体控制由 PCI 桥路实现。PCI 桥路提供了信号缓冲，使 PCI 总线能支持 10 台外设，并能在很高的时钟频率下保持高性能。

PCI 总线的结构与处理器的结构无关，它采用独特的中间缓冲器设计，把处理器子系统与外设分开。通过缓冲器的设计，用户可以随意增设多种外设扩展系统，而不必担心在不同时钟频率下会导致性能下降。这种独立性也使该总线有可能适应将来的处理器，改变 CPU 后，也只需要改变相应的 PCI 桥路就可以使用，从而延长了总线的寿命。

(3) 基本操作特点

① PCI 总线是同步总线，总线上的信号由时钟信号 CLK 同步。

② 每次总线传输都从帧同步信号有效开始。

③ 每个总线周期由 1 个地址期和 1 个或多个数据期组成。每个总线周期开始首先是地址期，在此期间主控部件送出地址和总线命令。地址期后为 1 个或若干个数据期，传输数据和字节有效信号。例如，1 个 PCI 总线读周期有 1 个地址期和 3 个数据期。

④ 只有当总线的主控设备和目标设备都发出就绪信号之后，当前的这个数据

期才能够结束, 否则进入等待状态。

(4) 即插即用 (自动配置)

传统的扩展卡插入系统时需要由用户使用开关、跳线或软件设置扩展卡需要占用系统内存空间、I/O 端口、系统中断和 DMA 通道。通过 PCI 总线连接外设时, 主机可按外设的增删情况自动配置系统资源。

(5) 高效益和低成本

① 高集成度。将大量的系统功能集成在 PCI 芯片内部, 可以减少部件间相互连接的逻辑电路, 减小电路板空间并降低成本, 同时也提高了可靠性。

② 管脚多路复用。地址线 and 数据线以及许多控制线共用管脚, 减少了管脚个数和 PCI 部件的封装尺寸。

(6) 兼容性和可扩展性好

通过 PCI 总线控制器, PCI 可以与其他总线 (如 ISA, EISA, MCA, VL 等) 保持很好的兼容性。

2. VL-BUS

VL-BUS 作为 32 位高速总线具有 132MB/s 的数据传输率。其优点是采用了开放性结构、协议简单、传输率高、价格低廉、支持多种硬件。但 VL-BUS 的规范性、兼容性和扩展性均较差, 结构偏简单、无缓冲器, 只能支持 3 个外设插接板。VL-BUS 只是对 CPU 总线的直接扩充, 其规范只发挥了 i486CPU 的功能。VL-BUS 是非多路复用体系, 信号庞杂, 它只规定了信号线的定义, 但对时间关系、负载情况等没有精确的规定, 实现时差别较大。总之, VL-BUS 局部总线只是 i486 微机上的一个经济实用的高性能总线。

6.1.4 外设总线

1. USB 总线

USB(Universal Serial Bus)总线是康柏、惠普、Intel、IBM、NEC、微软、北方电讯 7 大公司联合推出的基于 PC 平台的即插即用型的串行总线。

(1) USB 的特点

- ① 即插即用, 可以自动识别总线上的设备并为其配置软件和硬件资源。
- ② 可以动态连接和重新配置外设, 支持热拔插功能。
- ③ 总线上的设备的传输带宽可以从几 Kb/s 到几百 Mb/s, 速率最高可达 480Mb/s。
- ④ 通信协议能够灵活支持等时数据传输和异步消息传输的混合模式。
- ⑤ 允许多台设备同时工作, 最多可接 127 台物理设备。
- ⑥ 协议开销小, 总线利用率高。
- ⑦ 支持实时的语声、音频和视频数据传输。

⑧ 可以向 USB 总线上的设备供电，总线上的设备也可以自备电源。

(2) 总线拓扑

USB 总线将 USB 设备与 USB 主机相连，总线的物理连接是一种层叠式的星形拓扑。USB 总线上的设备有两种类型。

① 功能单元 (Functions)。指具有 USB 接口，实现某种功能的外设，例如，带有 USB 接口的便携式接口的硬盘、打印机、扫描仪、局域网集线器等。

② USB 集线器 (Hub)。为 USB 总线提供额外的连接点的设备，可以连接其他 USB 集线器或功能单元。虽然总线从物理上是分层的，但在实际通信过程中，所有 USB 设备对 USB 主机而言地位都是平等的，即 USB 总线的逻辑拓扑结构是不分层的星形拓扑。

(3) 电气特性

USB 总线通过一条 4 芯电缆传送电源和数据，电缆以点到点方式在设备之间连接。

(4) 机械特性

USB 连接器分为 A 系列和 B 系列两种，A 系列用于和主机连接，B 系列用于和 USB 设备的连接。USB 连接器的排列如表 6.1 所示。

表 6.1 USB 连接器的管脚分配

端子号	信号	典型电缆颜色	端子号	信号	典型电路颜色
1	V _{BUS}	红色	4	GND	黑色
2	D ⁻	白色	外皮	屏蔽	管线
3	D ⁺	绿色			

(5) 总线协定

USB 总线是一种轮询总线，由主机控制器控制所有的数据传输。大多数传输包含 3 个 USB 分组。

① 主机控制器首先发出一个“权标 (Token) 分组”，指明传输的类型、方向（每次传输的方向是确定的）、USB 设备的地址以及终点编号。USB 设备对相应的地址字段进行译码，选中被寻址的设备。

② 如果本次传输的源端能够提供数据，它发出数据分组；否则发出一个指示分组，指明它没有数据可以传输。

③ 一般目的端回送一个联络分组指明本次传输是否成功。

(6) 传输类型

USB 有 4 种基本的传输类型：①控制传输 ②批量传输 ③中断数据传输 ④等时传输。

2. IEEE1394

IEEE1394 接口又称火线 (FireWire) 接口, 是 IEEE 标准化组织制定的一项具有多媒体数据传输速度的高速串行接口标准。

消费电子方面已经有了采用 1394 接口的便携式摄像机、数字相机、数字卫星接收装置等。将来, 数字电视、数字机顶盒、DVD 播放机也采用 1394 接口。1394 接口可能是消费电子和个人计算机之间的物理桥梁。用户将能够通过他们的 PC 来控制消费电子设备和 PC 外设, 编辑音频或视频数据, 连接外设到 Internet 等等。

IEEE 1394 的特性如下。

① 实时数据传输(Real-Time Data Transfer)。IEEE 1394 具有两种数据传输模式, 等时 (Isochronous) 传输与异步传输。在同一总线下, 等时传输及异步传输连线可能同时存在。

② 驱动程序安装简单、扩展性好。

③ 内存映射的架构(Memory Mapping Architecture)。所有 IEEE 1394 总线上的资源, 都可以映射到某段内存地址, 并依此方式来存取数据。

④ 1394 接线可提供电源。对无自用电源的设备而言, 可以通过 IEEE 1394 6-Pin 的连接头来供给电源。

⑤ 通用 I/O 连接头: 整合各种 PC 的连接头成为一种万用的连接头, 使用者就不用花时间辨认不同外围设备要接到那个接头, 同时也降低系统的成本。

⑥ 点对点的通信架构(Peer-to-Peer Communication Architecture): IEEE 1394 外围设备间互传数据时, 不需主机监控, 因此, 不会增加主机的负载。CPU 资源占用率低。

⑦ 多种数据传输率: 在相同的总线上可以有数种不同的数据传输速率, 100Mb/s、200Mb/s、400Mb/s 甚至 1Gb/s, 如此高的速率使得它可以作为硬盘、DVD、CD-ROM 等大容量存储设备的接口。

⑧ 最理想的多媒体设备的接口。IEEE1394 支持等时传输模式, 同步传输模式, 并确保某一连线的频宽。对于即时影像而言这是相当重要的。因为影像数据都会有其时间上的限制, 无法接受过久的延迟。

IEEE1394 适用于声音、图像和视频多媒体产品、高速打印机和扫描仪产品、硬盘等存储设备、数码摄影机、显示器和影音录放设备等。

⑨ 支持热插拔。IEEE1394 可以自动侦测设备的加入与移出动作并对系统做重新整合, 无须人工干预。

⑩ 连接方便, 易于扩展, 不必设定标识号和连接终端负载, 可采用菊花链或树形方式连接, 所有连接的设备是平等关系, 不需个人计算机介入也可形成系统。

⑪ 单根线缆最长为 4.5m, 最大可进行 15 级级联, 连接最大距离为 72m。

⑫ 采用 6 股铜芯线缆, 两股用于供电, 另外 4 股分为两对双绞线, 接头小巧

耐用。

IEEE1394 将来有望取代现有的 SCSI 接口和 IDE 接口,但由于成本较高和技术上还不够成熟等,目前仍然只有少量使用 IEEE1394 接口的产品。

IEEE1394 接口的优点在于所传输的图像不必经过压缩再解压缩,在分辨率 640×480 的条件下,可以达到动态传输的需求,同时,1394 接口的电路设计较 USB 简单。

IEEE1394 技术由于具备高速数据传输速率、点对点的传输模式等特点,使信息家电或周边产品之间不需通过电脑主机就可以进行串接及数据传输,所以 IEEE1394 接口早已发展到信息家电市场,用来作为计算机和信息家电、智能家电的链接的一个桥梁,达到信息家电智能一体化的目的。

IEEE1394 使用户能够得到丰富的数字化应用,使得 PC 更具吸引力。IEEE1394 模块使制造商大大地简化了制造过程,降低成本。对外设供应商来说,消费类电子和计算机工业支持单一的工业标准将使得产品可以推向两个市场。

IEEE1394 未来发展方向可分为 3 部分:家庭网络及 SOHO 族应用、信息家电、无线传输等。

6.1.5 通信总线

1. RS-232 总线

参见本书 4.1.4 小节。

2. RS-485 总线

在通信距离为几十米到上千米时,广泛采用 RS-485 串行总线标准。RS-485 采用平衡发送和差分接收,因此,具有抑制共模干扰的能力。加上总线收发器具有高灵敏度,能检测低至 200mV 的电压,故在千米以外传输信号都能得到恢复。RS-485 采用半双工工作方式,任何时候只能有一点处于发送状态,因此,发送电路必须由使能信号加以控制。RS-485 用于多点互联时非常方便,可以省掉许多信号线。应用 RS-485 可以联网构成分布式系统,其允许最多并联 32 台驱动器和 32 台接收器。

3. IEEE-488 总线

IEEE-488 总线是一种并行总线接口标准。IEEE-488 总线用来连接系统,例如,微计算机、数字电压表、数码显示器等设备及其他仪器仪表均可用 IEEE-488 总线装配起来。它按照位并行、字节串行双向异步方式传输信号,连接方式为总线方式,仪器设备直接并联在总线上,总线上最多可连接 15 台设备,设备间最大距离为 20m,整个系统的电缆总长不超过 220m。信号传输速度一般为 500KB/s,最大传输速度为 1MB/s。

IEEE-488 总线组成的系统中每台设备可以按控制器、发送器或者接收器的方式工作,这由各种设备具备的不同功能确定。IEEE-488 总线采用负逻辑工作,使

用 24 针组合插头座。其信号线中有 8 根地线, 8 根双向数据线, 3 根字串传送控制线用于异步联络方式传输数据, 5 根接口管理线用于控制总线的工作。

6.2 例题精析

例 6.1 下列陈述不正确的是 ()。

- A. 总线结构传送方式可以提高数据的传输速度
- B. 与独立请求方式相比, 菊花链式查询方式对电路的故障更敏感
- C. PCI 总线采用同步时序协议和集中式仲裁策略
- D. 总线的带宽制约总线本身所能到达的最高传输速率

解 总线结构传送方式可以简化结构, 提高可靠性, 便于故障诊断和维修, 降低成本, 但不一定能够提高数据的传输速度, 因此 A 选项不正确。其他三项的说法正确。

例 6.2 下列各项中, () 不是同步总线协定的特点。

- A. 不需要应答信号
- B. 各部件的存取时间比较接近
- C. 各部件有统一的系统时钟
- D. 总线周期长度可变

解 同步总线协定方式中, 连在总线上的所有部件在一个统一的时钟信号控制下进行信息传送, 没有联络信号, 总线上各部件的访问时间比较接近, 以访问周期最长的部件的访问时间作为总线周期。因此, 总线周期固定, 一旦设计完成, 就不能在系统中加入速度更低的设备。所以答案为 D。

例 6.3 ISA 总线支持多少位数据传递?

解 早期 ISA 总线只支持 8 位数据传输, 最新的 ISA 支持 8/16 位的数据传输。ISA 总线是由 Intel 公司, IEEE 和 EISA 集团联合开发的与 IBM/AT 原装机总线意义相近的总线, 即 8/16 位的“工业标准结构”(ISA-Industry Standard Architecture) 总线。8 位 ISA 扩展 I/O 插槽由 62 个引脚组成, 用于 8 位的插卡; 8/16 位的扩展插槽除了具有一个 8 位 62 线的连接器外, 还有一个附加的 36 线连接器, 这种扩展 I/O 插槽既可支持 8 位的插卡, 也可支持 16 位插卡。

例 6.4 在要求通信距离为几十米到上千米时, 广泛采用哪个串行总线标准, 为什么?

解 RS-485 标准。RS-232 标准规定, 驱动器允许有 2500pF 的电容负载, 通信距离将受此电容限制。例如, 采用 150pF/m 的通信电缆时, 最大通信距离为 15m; 若每米电缆的电容量减小, 则通信距离还可以增加。

RS-422 和 RS-485 采用平衡发送和差分接收, 因此, 具有抑制共模干扰的能力。加上总线收发器具有高灵敏度, 能检测低至 200mV 的电压, 故在千米以外传输信号都能得到恢复, 通信距离可以达到千米。RS-422 标准只允许电路中有一个发送器, 而 RS-485 标准允许电路中有多个发送器, 因此应用更广泛。

例 6.5 简要说明 ISA 总线、EISA 总线、MCA、VESA 总线以及 PCI 总线的区别与联系。

解 上述总线的比较如表 6.2 所示。

表 6.2 总线比较表

	ISA	EISA	MCA	VESA	PCI
数据路径宽度/b	8/16	32	16/32/64	32/64	32/64
总线时钟频率/MHz	5 33/8 33	8 33	10 00	33/66	33/66
数据传输速率/MB/s	5 33/8 33	33 00	20/40/80/160	132	132/264
插槽数量/个	0~8	0~8	0~8	0~2	0~4
是否支持总线主控	否	是	是	是	是
数据/地址奇偶校验	否	否	是	否	是
同步、信道检验	否	否	是	否	是
卡 ID/自动配置	否	是	是	是	是
与 PC/ISA/EISA 相配合	无	无	无	是	是

6.3 学习自评

6.3.1 自测练习

一、填空题

1. EISA 总线可以传输的数据在宽度上为_____位。
2. 某系统总线的—个存取周期最快为 3 个总线时钟周期, 在一个总线周期中可以存取 32 位数据。若总线的时钟频率为 8.33MHz, 则总线的带宽为_____MB/s。
3. 计算机系统与外部设备之间相互连接的总线称为_(1)_; 用于连接微型机系统内各插件板的总线称为_(2)_; CPU 内部连接各寄存器及运算部件之间的总线称为_(3)_。
4. 一次总线的信息传送过程大致可以分为 4 个阶段, 依次为_(1)_、_(2)_、_(3)_、_(4)_。
5. VESA 总线是一种可直接与 CPU 通信的_____总线。
6. 主要有两种局部总线_(1)_总线和_(2)_总线。
7. VL 总线支持_(1)_MHz 到_(2)_Hz 的时钟频率, 数据带宽从_(3)_位

扩展到 (4) 位, 总线最大传输速率为 (5) MB/s。

二、单选题

1. 在目前流行的大多数奔腾机中, 硬盘一般通过硬盘接口电路连接到 ()。
A. CPU 芯片总线 B. PCI 总线
C. ISA 总线 (AT 总线) D. 存储器总线
2. 现行 PC 机的主要局部总线是 ()。
A. ISA 总线 (AT 总线) B. STD 总线
C. EISA 总线 D. PCI 总线
3. 现行 PC 机中打印机与主机的接口标准大多采用 ()。
A. RS-232 B. Centronics C. IDE D. IEEE-488
4. PC 机系统板上 I/O 扩充插槽引脚上的信号是 ()。
A. 控制器信号的延伸和再驱动 B. 外总线信号的延伸和再驱动
C. CPU 引脚信号的延伸和再驱动 D. 系统总线信号的延伸和再驱动
5. 微型计算机中数据总线的信号状态是 ()。
A. 单向双态 B. 单向三态 C. 双向三态 D. 双向双态
6. 执行输出指令 OUT 时, 向相应的 I/O 接口芯片产生的有效控制信号是 ()。
A. $\overline{AEN}$ B. $\overline{IOR}$ C. $\overline{IOW}$ D. ALE
7. 属于 PC 机局部总线的是 ()。
A. IEEE488 B. EISA C. PCI D. USB
8. 现行 PC 机中普遍采用的 ISA 总线(AT 总线), 其 I/O 插槽能够支持的存储器地址线有 ()。
A. 32 条 B. 24 条 C. 20 条 D. 16 条
9. 在使用 PCI 总线的奔腾计算机中, CPU 访问(读/写)内存储器通过 ()。
A. ISA 总线(AT 总线) B. PCI 总线
C. VESA 总线 D. CPU
10. STD 总线、ISA 总线和 Multibus 总线都属于 ()。
A. 局部总线 B. 系统总线 C. 外部总线 D. 通信总线
11. 测控系统计算机的工业标准总线中, 其信号线总数最少的是 ()。
A. IBMPC/XT 总线 B. ISA 总线
C. STD 总线 D. EISA 总线
12. 现行 PC 机普遍采用的 ISA 总 (AT 总线) 扩充插槽可供插卡使用的硬中断有 ()。
A. 16 级 B. 15 级 C. 11 级 D. 8 级
13. 目前使用奔腾处理器的 PC 机, 图形加速卡最好连接在 () 上。

- A. ISA 总线 B. PCI 总线 C. EISA 总线 D. MCA 总线
14. 目前采用奔腾处理器的 PC 机, 其局部总线大多数是 ()。
- A. VESA 总线 B. ISA 总线 C. EISA 总线 D. PCI 总线
15. PCI 接口包含了一个 () 的配置内存空间。
- A. 256B B. 512B C. 1024B D. 2048B
16. () 支持热拔插功能。
- A. EISA B. PCI 和 VL
C. USB 和 IEEE1394 D. MCA

三、问答题

1. 什么是总线? 什么是接口?
2. 总线规范的基本内容是什么?
3. 简要说明 PC 总线、ISA 总线和 EISA 总线的区别和联系。
4. 描述 EISA 总线接口如何能够容纳 ISA 接插件?
5. PCI 总线接口上的配置内存的作用是什么?
6. PCI 总线是如何寻址的?
7. STD 总线主要用于什么场合? 各信号线的主要功能是什么?
8. 同步总线有哪些优点和缺点? 主要用在哪些 CPU 中?
9. 异步总线怎样实现总线联络? 它有哪些优缺点?
10. 半同步总线怎样实现总线联络? 什么是零等待?

6.3.2 自测练习解答

一、填空题

1. 32
2. 11,11
3. (1) 设备总线或者通信总线 (2) 系统总线 (3) 芯片总线
4. (1) 总线请求和总线仲裁阶段 (2) 寻址阶段 (3) 数据传输阶段 (4) 结束阶段
5. 局部

6. (1) VL (2) PCI
7. (1) 16 (2) 66 (3) 32 (4) 64 (5) 132

二、选择题

1. B 2. D 3. D 4. B 5. C 6. C 7. C 8. A
9. B 10. B 11. C 12. C 13. B 14. D 15. A 16. C

三、问答题

1. 总线是计算机中各个模块间传递信息的信号线的集合。它为模块内和模块

间的各部件提供了标准的信息通道。利用不同的总线,可以实现芯片之间、部件之间、主机与外设之间、系统与系统之间的连接与通信。

接口是介于主机和外设之间的缓冲电路,解决由于信息格式不同、数据总线不允许某个设备长期占用、CPU 与外设之间速度不匹配等问题。

2. 每种总线都有详细的规范,基本的规范如下。

(1) 机械结构规范,规定模块尺寸、总线插头、边沿联接器等规格。

(2) 功能结构规范,确定引脚名称与功能,以及其相互作用的协议。功能结构规范是总线的核心。总线功能结构规范包括如下内容:

① 数据线、地址线、读/写控制线、时钟线和电源线、地线等;

② 中断机制;

③ 总线主控仲裁;

④ 应用逻辑,如握手联络线、复位、自启动、休眠维护等。

(3) 电气规范,规定信号逻辑电平、负载能力及最大额定值、动态转换时间等。

3. 它们分别是 8 位、16 位和 32 位的系统总线,其控制、中断、DMA、总线仲裁、突发传送等功能是增强和增加的,并且是向上兼容的。

4. EISA 总线是在 ISA 总线基础上扩展而成的,引脚由原来 ISA 总线的 62 加 36 个(共 98 个)扩展到 198 个。EISA 总线的信号分成上、下两层。上层包含 ISA 的全部信号,信号的排列、引脚间的距离以及定义规约与 ISA 完全一致;下层包含全部新增的 EISA 信号,这些信号在横向位置上与 ISA 信号线错开。为保证 ISA 标准的适配板只能和上层 ISA 信号相连接,在下层的某些位置设置了卡键,用来防止 ISA 适配器滑入深处的 EISA 层;而在与 EISA 的适配器板相应卡键的位置上,制作了大小相匹配的凹槽,从而保证 EISA 标准的适配器板能够畅通无阻地插到深处层,和上下两层信号相连接。

5. PCI 接口包含了一个 256B 的配置内存空间,允许计算机来访问 PCI 接口,这个特性允许系统自动地为 PCI 插板配置,也就是“即插即用”特性。

6. PCI 的地址线标识 $AD_0 \sim AD_{31}$ 为双向三态数据/地址复用线。在地址期是 32 位地址;在数据期是数据,数据宽度可变,可以是 8 位、16 位、32 位。总线是 32 位的,在地址期 AD_0 和 AD_1 不作为地址而表示某些控制信息。在一些系统中,有一条 64 位的数据总线,使用 $AD_0 \sim AD_{63}$ 来传送数据。将来这条总线将被用来把地址扩展到 64 位。

7. STD 总线主要用于工业控制标准微机的总线。STD 的 56 根信号线分为 4 类。电源线(引脚 1~6, 53~56): 决定总线使用电源的种类以及地线分布和用法。数据总线/地址扩展总线(引脚 7~14)是双向三态数据总线,若是 8 位 CPU,则当存储器的寻址范围在 64KB 内时不需要对数据总线进行复用;若是 16 位 CPU,地址范围超过 64KB,就需要复用数据总线。地址总线和地址总线/数据扩展总线

(引脚 15~30), 当寻址范围超过 64KB 时, 需要将数据总线作地址总线使用, 当传送的数据超过 8 位 (如 16 位) 时, 必须用地址线传送数据。控制总线 (引脚 31~52), 存储器和 I/O 设备的读/写控制、控制外围设备的定时控制、中断和总线控制、时钟和复位控制、优先级控制。

8. 优点: 总线时间固定, 模块之间的配合简单。缺点: 要求所有模块在同一时间内完成, 使得系统的组成缺乏灵活性。主要用在总线上各部件的访问时间比较接近的系统中。

9. 异步总线采用“应答式”传输技术, 用“请求”和“应答”两根信号线来协调传输而不依赖公共时钟信号。

优点: 可以根据模块的速率自动调整相应时间, 挂接外设而不需要考虑该设备的速度, 从而避免同步传输的缺点。缺点: 不管从模块存取时间的长短, 每次都要经过 4 个步骤——请求、响应、撤销请求、撤销响应。

10. 半同步总线设置了一条“等待”或者“准备就绪”信号线, 对于快速模块, 就像同步传输一样, 按严格时钟沿一步步地传送地址、命令和数据; 对于慢速设备, 要借助 READY 线, 强制使主模块延迟数个系统时钟间隔。“零等待”就是不需要 REDAY 线强制使得延迟。

6.4 教 学 建 议

本章的内容包括了主要的总线技术, 许多流行总线都给出了详细引脚信号作为完整的资料, 便于查找比较。授课时的重点是系统总线、局部总线和外设总线。在系统总线中, 建议着重讲述 PC 总线、ISA 总线和 EISA 总线这 3 种兼容的 8 位、16 位和 32 位总线的引脚、定义、性能特点、功能结构和主要用途, 区分它们的不同, 并注意它们功能的增强; 在局部总线中, 应说清局部总线在计算机系统中的作用并讲清现在常用的 PCI 总线; 在外设总线中的 USB 总线和 IEEE1394 总线除了教材中的内容外, 尽可能扩展内容, 讲透这两种串行通信总线的性能特点、工作原理、相互区别和发展前景。

第7章 用户交互接口

7.1 知识要点

7.1.1 键盘接口

1. 键盘的工作原理

键盘由一组排列成矩阵方式的按键开关组成, 按键开关有触点式的, 利用机械使得两导体断通; 还有非触点式的, 分为磁场变化的霍尔效应开关和利用电压或电流变化的电容开关。按键盘的编码功能, 通常分为编码键盘和非编码键盘两种类型。编码键盘中的某一个键被按下后, 能够提供与该键相对应的编码信息。编码键盘的缺点是硬件设备会随着键数的增加而增加。非编码键盘不直接提供按键的编码信息, 而是用较为简单的硬件和一套专用程序来识别按键的位置, 并产生与之相对应的位置码, 然后把位置码转换成对应的编码。非编码键盘主要用软件的方法进行矩阵扫描、识键和译键, 一般有3种方法: 行扫描法、行列扫描法和行列翻转法。

键盘相对于主机而言是一个独立的智能外设。键盘盒内有一片 Intel 8048/8748 或 8242/8742 单片机负责键盘的扫描、消除抖动和输入缓冲等。当敲击键盘时, 单片机识别被按下的键的位置, 并将代表该键位置的一个单字节数字编码送给主机, 这个编码称为被按下键的接通扫描码。当释放被按下的键时, 单片机也会向主机送入一个代表被释放键的断开扫描码。该扫描码是在键被按下时送出的扫描码加上 80H 构成的。标准键盘按键的位置码和扫描码是等值的, 而扩展键盘对于同样的键符, 其位置码与扫描码均不相同, 也不等值。扩展键盘的断开扫描码是在接通扫描码前加一个前缀字节“F0H”。

键盘送出的扫描码经过并/串转换后以串行方式送至主机, 由主机的键盘接口电路进行串/并转换, 而后通过中断控制器的 IRQ_1 向 CPU 申请中断。

2. 键盘接口电路及其工作原理

系统板上的键盘接口电路按照键盘代码串行传送的应答约定, 接收键盘发送的扫描码, 同时向键盘发送主机的控制信号。

标准键盘的接口电路一般设在主机系统板上,通过一个 5 芯 DIN 插座与键盘螺旋状电缆的 5 芯 DIN 插头相连。一旦键盘上有键被按下,键盘就将按键的扫描码通过键盘数据线和键盘时钟以串行方式送到键盘接口电路。

键盘输入对于主机 CPU 是异步的,主机系统以中断请求方式支持键盘的随机输入。每当键盘接口电路从键盘收到串行的按键扫描码,并转换为并行的按键扫描码后,键盘接口电路就向主机 CPU 发出硬件中断请求,主机响应该中断并在该中断程序的控制下从键盘接口电路读取按键扫描码,转换为相应的 ASCII 码,保存在键盘缓冲区或仅仅设置某种键盘状态。

标准键盘接口电路主要由串/并转换移位寄存器 74LS322、中断请求触发器 74LS74、两个 D 触发器、8255A-5 的 $PA_7 \sim PA_0$ 和 $PB_7 \sim PB_0$ 端口以及一些门电路组成。

扩展键盘的接口电路使用单片机作为键盘接口电路的核心部件,除了具备标准键盘接口电路的所有功能之外,还应将按键扫描码转换为兼容的系统扫描码。保证使用扩展键盘和使用标准键盘的主机系统利用键盘中断程序接收到的按键扫描码是相同的。

7.1.2 打印机接口

为了把计算机处理的信息记录在纸上,需要使用打印机。CPU 和打印机之间的数据传送既可以并行传送,也可以串行传送。但是,由于并行接口线路简单,工作速度快,因而广泛使用并行传送的打印机。微机上常用的打印机接口是通用的并行接口 Centronics,主要由命令译码器、数据收发器、8 位并行 I/O 端口、4 位并行 I/O 端口和 5 位输入口组成,并且有中断允许和请求的功能。

命令译码器根据输入译出写控制信号 WPA 和 WPC 或者读控制信号 RPA、RPB 和 RPC。数据收发器处于系统和内部的数据总线之间,由它控制数据的输入/输出、控制信号的输出和状态信号的输入。由锁存器、单向驱动器、缓冲器等电路组成 8 位 I/O 接口,4 位 I/O 接口和 5 位输入接口。

7.1.3 显示器接口

1. MC6845 CRT 控制器

MC6845 CRT 控制器是一个通用低成本的光栅扫描式 CRT 控制芯片,具有较灵活的编程能力,是一种常用的图形时序控制芯片。在 PC 机前期的显示卡产品,如 MDA、CGA、Hercules、Color 400 和长城 0250CH 显示卡中,MC6845 CRT 控制器都得到广泛的采用。

MC6845 CRT 控制器由可编程的水平与垂直定时电路、线性地址产生电路、光标控制、光笔控制以及处理器接口控制电路等组成。它具有以下技术性能。

① 字母数字和图形显示功能。

② 可以通过处理器数据总线进行全部编程,可以为任一种字符、图形产生定时信号,也可以产生水平同步和垂直同步信号。

③ 有硬件滚屏能力,并提供光笔接口。

④ 有光标寄存器和比较电路,光标的格式和闪烁可以编程。

⑤ 可以通过编程选用逐行扫描方式或隔行扫描方式。

⑥ 能够提供 14 位刷新地址和 5 位扫描线地址。

2. 显示标准

在 PC 系列微机系统中形成了多种标准的显示系统。这些显示系统的核心就是显示适配器。应用较为广泛的标准显示系统有如下几种。

(1) MDA(Monochrome Display Adapter)

MDA 单色显示适配器,分辨率为 720×350 ,支持 80×25 字符方式的黑白显示,它的显示控制逻辑是围绕 MC 6845 控制芯片设计的,显存起始地址为 B0000H,大小为 4 KB。

(2) CGA(Color Graphics Adapter)

CGA 是彩色图形适配器,比 MDA 增加了彩色和图形显示功能。分辨率为 320×200 时支持 16 色的 40×25 字符显示和图形显示,分辨率为 640×200 时支持单色的 80×25 字符显示和图形显示。其逻辑结构与 MDA 类似,核心也是 MC6845 芯片。显存起始地址为 B8000H,大小为 16 KB。

(3) EGA(Enhanced Graphics Adapter)

EGA 是增强图形适配器,其结构与 CGA 不同,但可以与 CGA 兼容。它最明显的特点是采用了位平面技术,即同一个像素在各个平面上占同一地址位,而不同位平面同一像素地址中的位形成像素值,可通过查彩色表将其转换成 R、G、B 亮度值。

(4) VGA(Video Graphics Array)

视频图形阵列,它兼容 EGA 的所有标准,还增加了新的图形显示标准:模式 11H、12H、13H。它的结构特点之一是采用了数/模转换器(DAC-Digital Analog Converter),增强了色彩的显示能力。DAC 将 VGA 的二进制彩色信息转换为模拟信号,并输出信号驱动模拟 RGB 显示器。

(5) TVGA

TVGA 是 Trident 公司开发的超 VGA 产品,是一种高性能的彩色图形适配器。它兼容 VGA 全部标准模式,还增加了若干字符显示和图形显示的新模式,可以得到更高的分辨率和更多的色彩显示。

3. CGA 的显示模式和逻辑结构

(1) CGA 的逻辑结构与寄存器

CGA 显示器由 CGA 显示控制卡和 CD(Color Display)彩色监视器组成。显示

控制卡以 MC6845 为核心。主机 CPU 通过总线可以直接访问 CGA 的显存和寄存器。通过 CPU 将需要显示的字符和图形数据写入显存，也可以通过读/写 CGA 的寄存器设置显示模式，选择彩色以及了解 CGA 的工作状态。6845 将显存中的数据送到图形移位器，再经过彩色编码器将视频信号送显示器显示。

CGA 上除了 MC6845 内部有 19 个可编程寄存器外还有 3 个寄存器：1 个模式寄存器 MR，用于指定显示模式；1 个状态寄存器 SR，表示显示器当前的状态；1 个彩色寄存器 CR，用于规定不同显示模式的像素颜色。CGA 还有 1 个与光笔配合工作的光笔触发器。

(2) CGA 的显示模式

CGA 共有 7 种显示模式，其中 4 种是字符模式，3 种是图形模式，如表 7.1 所示。

表 7.1 CGA 的显示模式

模式	类型	分辨率	颜色	页面数	MR 的值	显示模式属性	
0	字符	40×25	2	8	2Ch	40×25	单色字符
1	字符	40×25	16	8	28h	40×25	彩色字符
2	字符	80×25	2	4	2Dh	80×25	单色字符
3	字符	80×25	16	4	29h	80×25	彩色字符
4	图形	320×200	4	1	2Ah	320×200	彩色图形
5	图形	320×200	2	1	2Eh	320×200	单色图形
6	图形	640×200	2	1	1Eh	640×200	单色图形

4. 光笔结构

光笔是一种交互式作图设备，既能指定显示屏上图形的位罝，又能往计算机里输入新的数据。光笔为一笔形圆管，笔尖处有一个接收光的小孔，显示屏上的光可以通过此孔进入光笔。工作时，光笔的拾光孔对着显示屏，当电子光束从孔前扫过时，此光束通过孔后的光学透镜聚焦到光敏感器件上，经过光电转换、放大和整形，变成一个中断信号送到计算机，然后由系统调用程序完成图形处理操作。这样，光笔通过拾取显示屏上动态变化的光就能实现图形数据的输入。

光笔有两种结构：一种在笔体内完成光电转换，另一种在笔体外完成光电转换。第一种笔因为体积有限，只能用光敏二极管或光敏三极管作为光电转换器件，转换后的电信号经过微电子放大器后由屏蔽电缆送出。第二种笔可用光电倍增管作为光电转换器件，光笔拾取到的光经过隔光光导纤维束引到光敏元件输入端。光电倍增管的响应比光敏管快，适用于快速显示。

5. EGA/VGA 工作原理

(1) EGA/VGA 字符显示

EGA/VGA 显存有 4 个位平面，占有相同的地址空间，位平面的选择通过 I/O

端口寻址。字符显示时,由存放于位平面 0 的 ASCII 码指向位平面 2 中相应的字符点阵,字符点阵由位于位平面 1 的属性码控制在 CRT 指定的位置显示。字符属性决定了字符的背景颜色、前景颜色和是否闪烁等。

(2) EGA/VGA 图形显示

EGA/VGA 显存中 4 个位平面相同地址处各有 1 位组合成 4 位像素值,选择 16 个调色寄存器之一。对于 EGA 和 VGA,调色寄存器中 6 位值各有不同的用途。

① EGA 图形显示。EGA 使用的增强彩色显示器是数字 RGB 监视器,红、绿、蓝 3 色各有一引线传送低亮度电子束信号,一引线传送中亮度电子束信号,可混合产生高亮度、中亮度、低亮度及全暗颜色。3 种颜色、4 种亮度可产生 64 种色彩。调色寄存器中 6 位值就是 EGA 的彩色值,作为数字信号直接驱动监视器。

② VGA 图形显示。VGA 使用模拟 RGB 监视器,只有 3 根导线,用导线上的电压控制信号亮度。调色寄存器的 6 位值加上彩色选择寄存器的 2 位值组合的 8 位调色值作为指针选择 256 个彩色 (DAC) 寄存器之一。彩色寄存器长 18 位,红、绿、蓝 3 色各 6 位彩色值。红、绿、蓝的彩色值分别送对应的数模转换器 (DAC),输出驱动模拟 RGB 监视器。

6. EGA/VGA 编程基础

(1) 通用寄存器 (REG) 组

通用寄存器的寻址方式是直接寻址。

① 辅助输出 REG。R, 3CCH; W, 3C2H。水平和垂直回扫信号极性、钟频和 I/O 地址。

② 特色控制 REG。R, 3CAH; W, 3DAH(彩)/3BAH(单)。选择垂直同步信号。

③ 输入状态 REG0。R, 3CZH。判断 CRT 类型和中断请求是否由垂直回扫引起。

④ 输入状态 REG1。R, 3DAH (彩) /3BAH(单)。用于诊断、判断垂直回扫和显示状态。

⑤ VGA 允许 REG。R/W, 3C3H。控制 VGA 是否联机。

(2) 操作定序寄存器组

操作定序寄存器的寻址方式是间接寻址,可读/写,组地址端口为 3C4H,组数据端口为 3C5H。

① 复位 REG(00H)。同步或者异步复位。

② 时钟模式 REG(01H)。控制屏幕关闭、移位加载、点时钟等。

③ 位平面屏蔽 REG(02H)。决定是否允许写位平面 3、2、1、0。

④ 字符映象选择 REG(03H): 选择一套或者两套工作字符集。

⑤ 存储器模式 REG(04H)。CPU 访问位平面的模式和扩展存储器容量。

(3) CRT 控制寄存器组

CRT 控制寄存器的寻址方式为间接寻址, 组地址端口为 3D4H/3B4H, 组数据端口为 3D5H/3B5H。

25 个寄存器分别用于设置水平扫描总值(索引号 00H)、水平显示结束(01H)、水平消隐开始(02H)和结束(03H)、水平回扫开始(04H)和结束(05H)、垂直扫描总值(06H)、溢出(07H)、预置行扫描(08H)、最大扫描线(09H)、光标起始(0AH)和结束(0BH)、显存起始地址高位(0CH)和低位(0DH)、光标位置高位(0EH)和低位(0FH)、垂直回扫开始(10H)和结束(11H)、垂直显示结束(12H)、地址偏移量(13H)、下划线位置(14H)、垂直消隐开始(15H)和结束(16H)、CRTC 模式控制(17H)、行比较(18H)。

(4) 图形控制寄存器组

图形控制寄存器的寻址方式为间接寻址, 组地址端口和数据读/写端口为 3CEH 和 3CFH。

该组寄存器与 ALU、位平面锁存器等构成图形控制器, 管理 CPU 与显存间数据传送。9 个寄存器分别是: 复位/置位(索引号 00H)、复位/置位允许(01H)、彩色比较(02H)、数据移位/功能选择(03H)、位平面读出选择(04H)、图形控制模式(05H)、显存地址和容量辅助(06H)、彩色(位平面)忽略(07H)、显存中位屏蔽(08H)。

(5) 属性控制寄存器组

属性控制寄存器的寻址方式为间接寻址, 组地址读/写端口为 3C0H; 组数据读出端口为 3C1H, 写入端口为 3C0H。21 个寄存器包括: 调色(索引号 00H~0FH, 共 16 个)、模式控制(10H)、边框彩色(11H)、彩色位平面允许(12H)、水平像素平移(13H)、彩色选择(14H)。

(6) DAC 接口寄存器组

DAC 接口寄存器有 256 个彩色寄存器及索引地址寄存器、状态寄存器等。

- ① DAC 状态 REG。只读, 3C7H。DAC 读/写状态。
- ② DAC 读数据索引 REG。只写, 3C7H。彩色寄存器索引号为 00~FFH。
- ③ DAC 写数据索引 REG。读/写, 3C8H。彩色寄存器索引号为 00~FFH。
- ④ DAC 数据 REG。读/写, 3C9H。CPU 写/读 6 位彩色值。
- ⑤ 像素屏蔽 REG。读/写, 3C6H。BIOS 初始化为 FFH, 不能修改。

7. 显示程序设计

一般 PC 机上的应用程序需要对显示器操作时, 可以采用以下 4 种方法。

(1) 使用高级语言提供的 I/O 函数或输入/输出语句

用户使用这些函数或者语句访问显示器, 不需要了解硬件的工作原理、工作过程和相关细节, 实现最简单、最可靠、最安全, 但是高级语言的效率低, 对硬

件设备的支持较少,应用不灵活。

(2) 调用操作系统提供的支持

这种方法是访问 I/O 设备的常用方法。在 DOS 环境下,在 ROM BIOS 中断和 DOS 中断两个层次上为许多外设提供了支持,并对应用程序屏蔽了硬件细节。

① 使用 ROM BIOS 中断调用。它有以下特点:在对程序执行时间非常敏感的系统中更有益;ROM BIOS 例程更接近硬件,能够提供更大的灵活性和通用性;有些 ROM BIOS 功能没有等价的 DOS 例程。

② 使用 DOS 中断调用。DOS 软中断比 ROM BIOS 软中断层次更高,通常需要调用 BIOS 去完成实际的操作,比 BIOS 对硬件的依赖性更低。它有以下特点:功能更强,使用更方便;提供了更多的测试,使系统容错能力得以提高;可移植性好;DOS 例程有更好的兼容性。

(3) 直接程序设计访问显示硬件

在一些对处理机时间极为敏感或者实时性要求非常高的系统(如高速的串行通信系统)中,必须自行开发设备驱动程序,满足系统的特殊要求;在没有操作系统支持的嵌入式计算机系统中,直接访问硬件几乎是 I/O 的唯一方法。由于直接 I/O 操作不受操作系统的限制与管理,难以进行完整的错误检查,容易导致系统的不稳定乃至崩溃;而且直接对硬件进行控制使程序的兼容性降低,因此除非确有必要,应尽量避免使用直接的 I/O 操作。

(4) 自定义虚拟图形设备并配置若干驱动程序进行程序设计

利用直接程序设计方法自行编制 CGA 的图形驱动程序,扩充 INT 10H 的功能,提高画图速度,同时保留程序的兼容性。

(5) CGA 程序设计

① CGA 的 BIOS 级程序设计。CGA 的 BIOS(INT 10H)驻留在主机板的系统 BIOS ROM 中,主要由一组可供用户程序调用的子程序组成。INT 10H 提供的功能分为状态与控制(设置显示模式 00H、选择当前页 05H 和取当前显示状态 0FH)、字符处理(设置光标类型 01H,设置和查询字符光标位置 02H 和 03H,字符读写功能 08H、09H、0AH、0EH)、像素处理(写点 0CH 和读点 0DH)、滚动(对屏幕进行上滚 06H 和下滚 07H)、图形和光笔控制(读入光笔位置 04H 和配置配色器 0BH)等 6 组。用户程序还可以通过 BIOS 通信区了解显示器的工作状态。

② CGA 的直接程序设计。直接程序设计通常仍使用 INT 10H 的 00 号功能设置显示模式,在实际中主要是访问彩色寄存器和显存。

由 CGA 显存的结构可以看出,各显示模式的地址计算方法不完全一样。

字符模式的地址计算公式为

$$\text{地址} = \text{页首址} + (\text{每行字符数} \times \text{行号} + \text{列号}) \times 2$$

式中,每个字符占 2B,前一个字节为编码,后一个字节为属性。

图形模式的字节地址计算公式如下。

对模式 6

$$\text{地址} = \text{B8000H} + 2000\text{H} \times (y \bmod 2) + 80 \times (y/2) + x/8)$$

对模式 4 和 5

$$\text{地址} = \text{B8000H} + 2000\text{H} \times (y \bmod 2) + 80 \times (y/2) + x/4)$$

式中, y 、 x 分别为像素在屏幕上位置的行、列号。

③ 图形显示驱动程序。显示器 BIOS(INT 10H)的图形功能较弱,而直接程序设计方法兼容性差。为了给高层图形软件提供一个良好的与设备无关的接口,用户可以自行编写 CGA 配置图形显示驱动程序。

利用配置图形显示驱动程序的方法和 INT 8AH 的程序框架,修改与设备有关的常数并将设备有关的图形子程序换入 INT 8AH 程序,可以得到 CGA 各种图形模式下的图形显示驱动程序。

7.2 例题精析

例 7.1 说明标准键盘电路、标准键盘接口电路及键盘中断程序的工作原理。

解 标准键盘电路中键盘矩阵由键盘内的 8048 单片机控制。当某个键被按下时,检测电路产生一个高电平有效的按键状态信号 KEY DEPRESSED,并送到 8048 的测试端口 T_1 ,则 8048 停止扫描并以当前计数输出值作为按键的位置码(即按键扫描码)。将在 8048 的 P_{21} 端串行输出的扫描码送往键盘接口电路。

标准键盘接口电路一般设在主机板上,用一个 5 芯的 DIN 插座与键盘螺旋状电缆的 5 芯 DIN 插头相连。一旦键盘上有键按下,键盘就将按键的扫描码通过键盘数据线(KBD DATA, DIN2)和键盘时钟(KBD DATA, DIN1)以串行方式送到键盘接口电路。键盘接口电路从键盘收到串行的按键扫描码,并转换为并行的按键扫描码时,由键盘接口电路向主机发出硬件中断请求,主机响应该中断请求,并在该键盘中断程序的控制下从键盘接口电路读取按键扫描码,转换为相应的 ASCII 码,保存在键盘缓冲区或仅设置某种键盘状态。

例 7.2 PC 机上的应用程序需要访问硬件时,可以采用几种方法?每种方法各有何特点?

解 有以下 3 种方法。

(1) 使用高级语言提供的 I/O 函数或 I/O 语句。这是最高级的、最简便、最可靠、最安全的方法。但是,效率低,目标代码的长度和程序执行时间都比较长,而且由于高级语言侧重在不同的平台上的通用性,对硬件的支持较少,对许多设备没有的高级语言一级提供支持,或是应用不灵活。

(2) 调用操作系统提供的功能调用,例如,DOS 软中断或 ROM-BIOS 软中断。这种访问 I/O 设备的中级方法,效率较高,在操作系统一级的调用更加灵活,可以设置工作方式或修改某些标准设置。

(3) 跳过操作系统直接访问硬件。用于一些对处理机时间极为敏感或实时性要求非常高的系统中。但是,容易导致系统运行的不稳甚至崩溃,而且直接对硬件进行控制也使程序的兼容性变差。

例 7.3 如何编写程序通过调用 BIOS 的 INT 10H 功能设置 CGA 的显示模式?

解 用汇编语言实现。本例子将输入的命令中给出的显示模式参数取出,再将显示器设置为指定的模式。若该例的文件名为 SETMODE,为了进入 CGA 的模式 $n(0\sim6)$,在 DOS 的命令提示符下输入下列命令: SETMODE n 。程序如下:

```
CODE    SEGMENT
        ASSUME CS:CODE
MAIN    PROC
        MOV     AL,DS:[5DH]    ;取命令行中的显示模式
        AND     AL,07H        ;将显示模式值转换成二进制数
        CALL    SETMODE       ;设置显示模式
        MOV     AH,4CH        ;返回 DOS
        INT     21H
MAIN    ENDP
SETMODE PROC                    ;设置显示模式,AL 中为模式值
        XOR     AH,AH
        INT     10H
        RET
SETMODE RET
CODE    ENDS
END     MAIN
```

例 7.4 如何编写程序通过调用 BIOS 的 INT 10H 功能写像素?

解 用汇编语言实现。该例子中的子程序 setpix 利用 INT 10H 写一个像素。CX 和 DX 中放像素坐标,AL 中放彩色号。当 AL 的最高位为 1 时,写方式为异或方式。在运行该例之前必须进入图形模式。程序如下:

```
CODE    SEGMENT
        ASSUME CS:CODE
MAIN    PROC
        MOV     AL,1          ;像素值
```

```

        MOV     CX,200        ;X 坐标
        MOV     DX,100        ;Y 坐标
        CALL    SETPIX        ;写像素
        MOV     AH,4CH        ;返回 MS-DOS
        INT     21H
MAIN     ENDP
SETPIX   PROC                ;写像素，像素值在 AL 中
        MOV     AH,0CH
        INT     10H
        RET
SETPIX   ENDP
CODE     ENDS
END      MAIN

```

例 7.5 用直接程序设计方法显示字符。

解 该例是在字符模式下显示字符的程序。在入口参数中，DX 为字符位置的行列号，BL 为字符的编码，BH 为属性。字符写在第 0 页。内存 44AH 单元中存放着每行的字符数目，这是在设置模式时决定的。程序如下：

```

BIOS_COLUMNS EQU 44AH          ; 每行字符数的指针
WRITECHAR    PROC              ; 写字符/属性
        XOR     AX,AX
        MOV     ES,AX
        MOV     AL,ES:[BIOS_COLUMNS] ; 取每行字符数
        MUL     DH
        XOR     DH,DH
        ADD     AX,DX
        SHL     AX,1
        MOV     DI,AX
        MOV     AX,0B800H        ; 第 0 页首址(段地址)
        MOV     ES,AX
        MOV     AX,BX            ; BX=字符/属性
        STOSW                    ; 写入显存
        RET
WRITECHAR    ENDP

```

例 7.6 利用 INT 10H 设置显示模式和 CGA 直接程序设计编程模式 6 的 4 种写模式下的写像素程序，其中 CX=列坐标、DX=行坐标、AL=颜色值。写像素即

用 AL 中颜色值操作 CRT 上显示点对应的 VRAM 中像素值(写模式 1: 取代; 1: 或; 2: 与; 3: 异或)。

解 程序如下:

```

DRMODE      EQU      4C4H                ; 写模式(0: 取代; 1: 或; 2: 与;
                                           ; 3: 异或)
DSPMEMSEG    EQU      0B800H             ; 显存段地址
SETPIX
    PROC
        PUSH      BP
        PUSH      AX
        PUSH      BX
        PUSH      DX
        PUSH      CX
        XOR        AH,AH
        MOV        BP,AX                  ; BP = 画图色
        MOV        AX,DSPMEMSEG          ; 取显存段地址
        TEST       DL,01H
        JZ         EVLINE
    EVLINE:
        ADD        AX,200H                ; 奇数行加 200H
        MOV        ES,AX                  ; ES=B800 + 200 * (Y MOD 2)
        SHR        DX,1                   ; Y/2
        MOV        AX,DX
        MOV        CL,4
        SHL        AX,CL
        MOV        BX,AX
        SHL        AX,1
        SHL        AX,1
        ADD        AX,BX
        POP        BX
        PUSH       BX
        PUSH       BX
        MOV        CL,3
        SHR        BX,CL
        ADD        BX,AX                  ; BX=80 * (Y/2) + X/8
        POP        CX
        AND        CL,07H                ; CL=X MOD 8
        MOV        AL,80H
        ROR        AL,CL
        MOV        DL,AL                  ; DL 中为像素值的位址
    
```

```

MOV     DH,ES:[BX]      ; 读出显存中的像素字节
AND     DH,DL           ; DH 中为需要修改的位的值
OR      BP,BP           ; BP 中为画图色
JNZ     SPNT0
NOT     DL
SPNT0:  MOV     AH,DS:[DRMODE0] ; 写模式
        CMP     AH,1
        JNZ     SPNT1
        OR      DH,DL
        JMP     SPNT4
SPNT1:  CMP     AH,2          ; 写模式 1: 或
        JNZ     SPNT2
        AND     DH,DL
        JMP     SPNT4
SPNT2:  CMP     AH,3          ; 写模式 2: 与
        JNZ     SPNT3
        XOR     DH,DL
        JMP     SPNT4
SPNT3:  MOV     DH,DL          ; 写模式 3: 异或
SPNT4:  AND     DH,DL          ; 写模式 4: 取代
        JZ      SPNT5
        OR      ES:[BX],DH
        JMP     SPDONE
SPNT5:  NOT     AL
        AND     ES:[BX],AL
SPDONE: POP     CX
        POP     DX
        POP     BX
        POP     AX
        POP     BP
        RET
SETPIX  ENDP

```

例 7.7 利用 INT 10H 设置模式和 CGA 直接程序设计实现模式 4 的取代和异或两种写模式下的写像素程序，其中 CX=列坐标、DX=行坐标、AL=颜色值。写像素即用 AL 中颜色值操作 CRT 显示点对应的 VRAM 中像素值(写模式 0: 取代; 3: 异或)。

解 程序如下:

```
DMWIDTH EQU (640/8)          ; 显存宽度
```

```

DSPMEMSEG EQU    0B800H           ; 显存段地址
DRMODE0 EQU    4C41H             ; 画图模式(0: 取代; 3: 异或)
SETPIX PROC
    PUSH    AX
    PUSH    BX
    PUSH    CS
    PUSH    DX
    AND     AL,03H
    CALL    GETADDRESS           ; 计算显存地址, ES:BX=字节地址,
                                ; CL=位地址

    MOV     AH,0FCH
    ROL     AH,CL
    ROL     AL,CL
    CMP     DS:BYTE PRT 'DRMODE0',2
    JNZ     ORMODE
    MOV     CL,ES:[BX]           ; 异或模式
    MOV     CH,AH
    NOT     CH
    AND     CH,CL
    NOT     CH
    AND     CH,CL
    XOR     AL,CH
ORMODE:  AND     ES:[BX],AH       ; 写显存
    OR      ES:[BX],AL
    POP     DX
    POP     CX
    POP     BX
    POP     AX
    RET
SETPIX ENDP
GETADDRESS PROC
                                ; 计算显存地址: CX=X 坐标,DX=Y 坐标
    PUSH    AX
    MOV     AX,DSPMEMSEG        ; 取显存段地址
    TEST    DL,01H
    JZ      EVLINE
    ADD     AX,200H             ; 奇数线加 200H
EVLINE:  MOV     ES,AX           ; ES=B800 + 200 * (Y MOD 2)
    SHR     DX,1
    PUSH    CX

```

```

MOV     AX,DX
MOV     CL,4
SHL     AX,CL
MOV     BX,AX
SHL     AX,1
SHL     AX,1
ADD     AX,BX
SHR     CX,1
SHR     CX,1
ADD     CX,AX
MOV     BX,CX
POP     CX
AND     CL,03H
SHL     CL,1
MOV     AL,6
SUB     AL,CL
MOV     CL,AL
POP     AX
RET

```

GETADDRESS ENDP

例 7.8 显示页切换。将屏幕设置成彩色文本方式，并分别在显示页 0 到显示页 7 上依次按 7 种不同颜色显示一个字符串。然后把这 8 个显示页逐页设定为当前显示页。每切换一页，等待用户敲入一个字符；若为 Esc 键则返回 DOS 并恢复原来的显示模式，否则继续切换到下一个显示页。

解 程序如下：

```

.MODEL  SMALL                ;采用小模式
.DATA                                ;定义数据段
CR      EQU      0DH          ;定义回车
LF      EQU      0AH          ;定义换行
ESC_KEY EQU      1BH          ;定义 Esc 键
STRING  DB      'TEXT MODE DISPLA PAGE#' ;定义要显示的字符串
LENG    EQU      $-STRING      ;字符串长度
.STACK  200                    ;定义堆栈段
.CODE                                ;定义代码段
START:  MOV      AX,@DATA      ;初始化数据段寄存器
        MOV      DS,AX
        MOV      AH,0FH        ;取当前的显示模式
        INT      10H

```

```

        PUSH    AX                ;入栈保存
        MOV     AH,00H            ;设置显示模式
        MOV     AL,01H            ;40×25 文本模式
        INT     10H
        ;在各显示页中写入字符串
        MOV     BH,0              ;显示页号初值
        MOV     BL,08H            ;显示属性初值
        MOV     CX,SEG STRING     ;缓冲区段基址
        MOV     ES,CX
        MOV     BP,OFFSET STRING  ;缓冲区偏移量
        MOV     DX,0808H          ;显示起始位置:第 8 行,第 8 列
THEN:    MOV     CX,LENG           ;字符串长度
        MOV     AH,13H            ;显示字符串
        MOV     AL,1              ;字符串缓冲区结构
        INT     10H
        MOV     AH,09H            ;显示字符子功能
        MOV     AL,BH             ;显示当前页号
        ADD     AL,30H            ;将数字转换为 ASCII 码
        MOV     CX,1              ;字符个数
        INT     10H
        INC     BH                ;增加页号
        INC     BL                ;增加颜色
        CMP     BH,8              ;未满 8 个显示页则继续
        JNE     THEN
        ;切换显示页
OK:      MOV     AH,05H            ;当前显示页初值
        MOV     AL,DH             ;页号
        INT     10H
        MOV     AH,0              ;读入一个按键
        INT     16H
        CMP     AL,ESC_KEY        ;是 Esc 键吗
        JE      EXIT              ;是,则退回 DOS
        INC     DH                ;否则,增加显示页号
        AND     DH,00000111B      ;将显示页号模 7
        JMP     OK                ;切换到下一个显示页
EXIT:    POP     AX                ;恢复原来的显示模式
        MOV     AH,0
        INT     10H
        MOV     AH,4CH            ;返回 DOS

```

```

INT      21H
END      START

```

例 7.9 图形显示输出。显示多彩斜线。在屏幕上连续画出 45° 角的斜线。当斜线遇到屏幕边界时，向屏幕内侧旋转前进方向 90° 折返，如同一个质点碰到墙壁反弹一样。每次折返都变换一种颜色。开始时屏幕的背景是黑色的，在屏幕上看到的图像好像一个四处碰壁的乒乓球留下的轨迹。随着这些斜线逐渐地填充到屏幕上并规则地排列成相互交错的条纹，整个显示页就好像一幅正在编织的图案，运动的质点像一把梭子来回穿插，使图案不停地变化。

解 程序如下：

```

.MODEL      SMALL                ;定义小模式
.DATA                          ;数据段
    INC_X    DW 1                ;保存 X 坐标的增量
    INC_Y    DW 1                ;保存 Y 坐标的增量
.STACK      200                  ;堆栈段
.CODE                          ;代码段
START:      MOV     AX,@DATA      ;初始化数据段寄存器
            MOV     DS,AX
            MOV     AH,0FH        ;获取当前的显示模式
            INT     10H
            PUSH    AX            ;入栈保存
            MOV     AH,00H        ;设置显示模式
            MOV     AL,12H        ;640×480×16 色图形方式
            INT     10H
            MOV     AH,0CH        ;写像素例程
            MOV     AL,01H        ;颜色值属性初值
            MOV     BH,0          ;页号
            MOV     CX,0          ;X 坐标初值
            MOV     DX,0          ;Y 坐标初值
DO:          INT     10H          ;画一个点
            ADD     CX,INC_X      ;改变 X 坐标
            CMP     CX,639        ;超出右边界了吗
            JNE     X1
            NEG     INC_X         ;是,则修改 X 坐标的增量方向
            INC     AL            ;改变颜色属性
            JMP     MODIFY_Y
X1:          CMP     CX,0          ;超出左边界了吗
            JNE     MODIFY_Y
            NEG     INC_X         ;是,则修改 X 坐标的增量方向

```

```

                INC     AL                ;改变颜色属性
MODIFY_Y:ADD    DX,INC_Y                ;改变 X 坐标
                CMP     DX,479          ;超出下边界了吗
                JNE     Y1
                NEG     INC_Y            ;是,则修改 Y 坐标的增量方向
                INC     AL                ;改变颜色属性
                JMP     TEST_KEY
Y1:             CMP     DX,P             ;超出上边界了吗
                JNE     TEST_KEY
                NEG     INC_Y            ;是,则修改 Y 坐标的增量方向
                INC     AL                ;改变颜色属性
TEST_KEY:PUSH   AX                      ;保存 AX 寄存器
                MOV     AH,01H          ;读键盘缓冲区
                INT     16H
                JNZ     EXIT             ;有键按下则继续
                POP     AX              ;恢复 AX 寄存器
                JMP     SHORT DO         ;无键按下则继续
EXIT:           POP     AX              ;恢复 AX 寄存器
                POP     AX              ;弹出保存的显示模式
                MOV     AH,00H          ;恢复原来的显示模式
                INT     10H
                MOV     AH,4CH           ;返回 DOS
                INT     21H
                END     START           ;程序结束

```

例 7.10 图形方式下的字符显示。该程序将显示模式切换到图形方式并显示一个单词，在输出单词的每个字母时都用写像素例程描绘一个方框将该字母围起来。从显示的效果可以看出，在图形方式下显示字符的好处之一是可以实现文本和图形的混合输出。

解 程序如下：

```

.MODEL SMALL                ;定义小模式
.DATA                       ;数据段
X1 DW ?                    ;方框左上角的 X 坐标
Y1 DW ?                    ;方框左上角的 Y 坐标
X2 DW ?                    ;方框右下角的 X 坐标
Y2 DW ?                    ;方框右下角的 Y 坐标
CUR_X DB 15                ;当前显示字符位置的 X 坐标
CUR_Y DB 13                ;当前显示字符位置的 Y 坐标
CUR_ATTR DB 1              ;当前显示字符的颜色属性

```

	STRING	DB 'HELLO'	;欲显示的字符串
	LAST	EQU \$	;字符串缓冲区的结束地址
.STACK	200		;堆栈段
.CODE			;代码段
START:	MOV	AX,@DATA	;初始化数据段寄存器
	MOV	DS,AX	
	MOV	AH,0FH	;取当前的显示模式
	INT	10H	
	PUSH	AX	;入栈保存
	MOV	AH,00H	;设置显示模式
	MOV	AL,13H	;320×200×256 色图形方式
	INT	10H	
	LEA	SI,STRING	;SI 寄存器指向字符串首地址
DO:	MOV	AH,02H	;设置逻辑光标位置
	MOV	DL,CUR_X	;X 坐标
	MOV	DH,CUR_Y	;Y 坐标
	MOV	BH,0	;页号
	INT	10H	
	MOV	AH,09H	;写单个字符属性
	MOV	BH,00H	;页号
	MOV	BL,CUR_ATTR	;颜色属性
	MOV	CX,1	;字符个数
	MOV	AL,[SI]	;取一个字符
	INT	10H	;计算边框的位置,每个字符为 8×8 像素
	MOV	AL,CUR_X	;AX=字符位置的 X 坐标
	MOV	AH,0	
	MOV	CL,3	
	SHL	AX,CL	;AX×8→AX
	SHB	AX,2	;留出边框的间隙(2 个像素)
	MOV	X1,AX	;X1 为左边线的 X 坐标
	ADD	AX,10	;加字符宽度和间隙(8+2 个像素)
	MOV	X2,AX	;X2 为右边线的 X 坐标
	MOV	AL,CUR_Y	;AX=字符位置的 Y 坐标
	MOV	AH,0	
	MOV	CL,3	
	SHL	AX,CL	;AX×8→AX
	SUB	AX,2	;留出边框的间隙(2 个像素)
	MOV	Y1,AX	;Y1 为左边线的 X 坐标
	ADD	AX,10	;加字符宽度和间隙(8+2 个像素)


```

MOV     Y2,AX           ;Y2 为右边线的 X 坐标
CALL    DRAW_BOX        ;调用画边框子程序
INC     CUR_ATTR        ;颜色属性值加 1
INC     CUR_X            ;当前字符位置 X 坐标加 2
INC     CUR_X
INC     SI              ;字符指针加 1
CMP     SI,LAST         ;是最后一个字符吗
JNE     DO              ;不是则重复操作
MOV     AH,00H          ;等待用户敲入任意键
INT     16H
POP     AX              ;弹出保存的显示模式
MOV     AH,00H          ;恢复显示原来的模式
INT     10H
MOV     AH,4CH          ;返回 DOS
INT     21H

DRAW_PIX PROC           ;写像素子程序
;INPUT:                ;入口参数:
;CX=X COORDINATE        ;CX=水平像素坐标
;DX=Y COORDINATE        ;DX=垂直像素坐标
MOV     AH,0CH          ;写像素子功能
MOV     AL,CUR_ATTR     ;像素颜色属性
MOV     BH,0            ;页号
INT     10H
RET

DRAW_PIX ENDP

DRAW_BOX PROC           ;画边框子程序
;INPUT:                ;入口参数
;(X1,Y1)=COORDINATE OF TOP-LEFT CORNER ;(X1,Y1)为边框左上角坐标
;(X2,Y2)=COORDINATE OF BOTTOM-RIGHT CORNER ;(X2,Y2)为边框右下角坐标
MOV     CX,X1           ;取 X 坐标
MOV     DX,Y1           ;取 Y 坐标
L1:                                           ;画上边线
CALL    DRAW_PIX        ;写一个像素
INC     CX              ;X 坐标加 1
CMP     CX,X2           ;到最右边了吗
JNE     L1              ;未到则继续
L2:                                           ;画右边线
CALL    DRAW_PIX        ;写一个像素
INC     DX              ;Y 坐标加 1

```

```

        CMP     DX,Y2           ;到最下边了吗
        JNE     L2             ;未到则继续
L3:                               ;画下边线
        CALL    DRW_PIX        ;写一个像素
        DEC     CX             ;X 坐标减 1
        CMP     CX,X1          ;到最上边了吗
        JNE     L3             ;未到则继续
L4:                               ;画左边线
        CALL    DRAW_PIX       ;写一个像素
        DEC     DX             ;Y 坐标减 1
        CMP     DX,Y1          ;到最上边了吗
        JNE     L4             ;未到则继续
        RET
DRAW_BOX      ENDP
END          START           ;程序结束

```

7.3 学 习 自 评

7.3.1 自测练习

一、填空题

1. 微机系统中用于用户交互的设备有 (1) 、 (2) 、 (3) 以及多媒体设备和虚拟现实设备。
2. 显示设备主要包括 (1) 、 (2) 和 (3) 显示设备。
3. 键盘由一组排列成 (1) 的按键开关组成，通常有 (2) 和 (3) 两种类型。
4. 非编码键盘主要用软件的方法识键和译键，一般有 3 种方法： (1) 、 (2) 、 (3) 。
5. 标准键盘上键的位置号和按键扫描码是 (1) ，均用 (2) 表示。
6. Centronics 是一个并行口，采用控制和状态线联络，用于 (1) 与 (2) 的联络。
7. 在文本方式下，把屏幕设置成 132 列 25 行时，显示一整屏文本所需显存的容量是 。
8. 光笔式是一种 设备，既能指定显示屏上图形的位置，又能往计算机里输入新的数据。
9. 目前微机上使用的显示系统有 (1) 和 (2) 两种显示方式。

10. CGA 显示器由__ (1) __和__ (2) __组成。整个显示控制卡以__ (3) __为核心。
11. MC 6845 具有__ (1) __、__ (2) __和__ (3) __显示功能。
12. EGA 最明显的特点是采用____技术, 同一像素点在各位平面上占同一地址, 不同位平面上同一像素地址中的位形成像素彩色值。
13. VGA 的结构特点之一是采用了____, 增强了色彩的显示能力。
14. VGA 对不同的显示模式, 分别采用____或____的显示缓存。
15. EGA/VGA 的各种图形和字符显示功能是通过卡上的____来控制的, 直接对这些寄存器编程, 可以得到很高的效率, 但是兼容性不好。
16. VGA 12H 显示方式规定其显示卡可从 256K 种颜色中挑选的任意 16 色在一幅画面中显示, 则显示卡的 DAC 的颜色寄存器为____位。

二、单选题

1. CPU 和打印机之间传送数据是 ()。
A. 既可以并行传输也可以串行传输
B. 串行传输
C. 并行传输
D. 同步传输
2. 显存的写入操作一般按 () 进行。
A. 字节 B. 字 C. 双字 D. 位
3. 标准键盘字符“A”的键号为 30, 先按下该键后松开, 则先输出 (), 后输出 ()。
A. 1EH B. 80H C. 9EH D. 不确定
4. 扩展键盘字符“A”键号为 31, 则接通扫描码为 (), 断开扫描码为 ()。
A. 1FH B. FOH C. FOH 和 1CH D. 不确定
5. 光笔触发器的 () 将显示缓存当时的偏移地址锁存到 CRTC6845 的 R16、R17。
A. 正脉冲 B. 负脉冲 C. 低电平 D. 高电平
6. 对光笔, 写 () 端口, 复位光笔触发器, 为下次光笔工作做准备; 写 () 端口可以置位光笔触发器, 用于测试。
A. 3DAH B. 3DBH C. 3DCH D. 3DDH
7. EGA/VGA 显示字符时, 字符的 ASCII 码放在 (), 属性码放在 (), 字符集放在 ()。
A. 位平面 0 B. 位平面 1 C. 位平面 2 D. 位平面 3
8. () 用于管理 CPU 和 VRAM 之间传送的数据, 它们与算术逻辑部件、位平面锁存器等构成图形控制器。
A. 图形控制寄存器 B. 通用寄存器
C. 操作定序寄存器 D. CRT 控制寄存器组
9. 配备标准键盘的 PC 机, 键盘向主机发送的代码是 ()。

A. 扫描码 B. BCD 码 C. 格雷码 D. ASCII 码

10. 一台激光打印机,可使用 A4 纸(约 8 英寸×11 英寸)以 300dpi 的分辨率打印输出,它所配置的页面存储器的容量应为 ()。

A. 2MB B. 256KB C. 512KB D. 1MB

11. 显示器的主要参数之一是分辨率,其含义为 ()。

- A. 显示屏幕的水平和垂直扫描频率
- B. 显示屏幕上光栅的列数和行数
- C. 可显示不同颜色的总数
- D. 同一幅画面允许显示不同颜色的最大数目

12. 一台 PC 机的显示器具有 1024(点)×768(线)的分辨率,并可同时显示 64K 色,则其图形卡上 VRAM 的容量应配置为 ()。

A. 256KB B. 512KB C. 1MB D. 2MB

三、问答题

1. 常用的键盘结构形式有哪些?它们是如何判别被按下的键的?
2. PC 机的键盘是如何工作的?键盘缓冲区的作用是什么?
3. 说明打印机接口电路中的地址译码、数据和控制信号输出锁存、数据和状态信号输入缓冲和中断的工作原理。
4. 试述 CRT 控制器 MC6845 的功能。
5. 简述显示器的显示方式和显示标准。
6. 在文本方式下,显示缓冲存储器和字符 ROM 分别有何作用?
7. 在图形方式下,像素与显示缓冲区之间应如何进行映射?
8. 设显示器的分辨率为 1280×1024 ,如果希望每个像素可以显示 8 种颜色,并安排 2 个显示页,则显示缓冲区的容量应为多大?

四、程序设计

1. 用汇编语言编制一段程序,利用 INT 16H 功能调用检查当前键盘输入是处于插入状态,还是改写状态,若是改写状态,则改其为插入状态,若是插入状态,则改其为改写状态。
2. 调用 INT 10H 为 GW0520CH 汉字屏幕打间隔为 18 的红色横格。
3. 用汇编程序实现:将显示存储器中位移位置为 0000H 的字节,坐标在 5/0 和 7/0 之间的像素点赋予 1011B 的色彩。

7.3.2 自测练习解答

一、填空题

1. (1) 输入 (2) 输出 (3) 显示设备
2. (1) 字符 (2) 图形 (3) 图像

3. (1) 矩阵方式 (2) 编码键盘 (3) 非编码键盘
4. (1) 行扫描 (2) 行列扫描 (3) 行列翻转
5. (1) 等值的 (2) 1B
6. (1) 打印机 (2) 主机
7. 6600B
8. 交互式作图
9. (1) 文本方式 (2) 图形方式
10. (1) CGA 显示控制卡 (2) CD(Color Display)彩色监视器
(3) MC6845
11. (1) 字母 (2) 数字 (3) 图形
12. 位平面
13. 数/模转换器
14. (1) 位平面结构 (2) 线性结构
15. 寄存器
16. 18

二、选择题

- | | | | | | |
|----------|------|--------|--------|-------|--------|
| 1. A | 2. D | 3. A、C | 4. A、C | 5. A | 6. B、C |
| 7. A、B、C | 8. A | 9. A | 10. A | 11. B | 12. D |

三、问答题

1. 常用的键盘结构形式有编码键盘和非编码键盘两种类型。编码键盘中的某一个键被按下后, 能够提供与该键相对应的编码信息。非编码键盘不直接提供按键的编码信息, 而是用较为简单的硬件和一套专用程序来识别按键的位置, 并提供与按键相对应的中间代码(即键码), 然后把键码转换成对应的编码。

非编码键盘主要用软件的方法识键和译键, 一般有 3 种方法: 逐行扫描、逐列扫描、行列翻转。

2. 键盘缓冲区的作用是保存按键扫描码。

3. 见知识要点。

4. MC6845 的功能有:

① 字母数字和图形显示功能。

② 可以通过处理器数据总线进行全部编程, 可以为任一种字符、图形产生定时信号, 也可以产生水平同步和垂直同步信号。

③ 有硬件滚屏能力, 并提供光笔接口。

④ 有光标寄存器和比较电路, 光标的格式和闪烁可以编程。

⑤ 可以通过编程选用逐行扫描方式或隔行扫描方式。

⑥ 能够提供 14 位刷新地址和 5 位扫描线地址。

5. 见知识要点。

6. 文本方式下, 显示缓冲区用来存放字符 ASCII 码和属性码, 字符 ROM 用来存放字符集。

7. 在图形方式下, 像素与显示缓冲区之间的映射与显示模式有关。具体的映射关系见知识要点。

8. 每个像素显示 8 种颜色, 则每个像素需要 3b, 每页的大小为: $1280 \times 1024 \times 3b = 480KB$, 2 个显示页的大小就为 960KB。

四、程序设计

```

1.  MOV    AH,2           ;设置子功能号
    INT     16H           ;读标志
    MOV     AH,AL         ;AL 中为特殊键标志
    AND     AH,80H        ;AL 最高位为 1 表示 INS 键状态已变
    NOT     AH            ;取 AL 最高位,将其取反
    OR      AH,7FH
    AND     AH,AL

2.  CODES  SEGMENT
        ASSUME CS:CODES

    IP0     DW      0
    IP1     DW      0
    IP2     DW      0
    IP3     DW      0
    IP4     DW      0

    MAIN    PROC
        MOV     AX,6                ;设置显示模式为 6
        INT     10H
        MOV     CX,0
    LP:     MOV     WORD    PTR CS:IP0,4    ;设置直线颜色
        MOV     WORD    PTR CS:IP1,CX    ;起点(X0,Y0)
        MOV     WORD    PTR CS:IP2,0
        MOV     WORD    PTR CS:IP3,CX    ;终点(X1,Y1)
        MOV     WORD    PTR CS:IP4,640
        MOV     BP,CS
        LEA     BX,IP0
        MOV     AH,30H
        INT     10H
        ADD     CS,12H
        CMP     CX,450
        JLE     LP

```

```

        MOV     AH,4CH
        INT     21H
MAIN     ENDP
CODES    ENDS
END      MAIN

```

3. 因为在坐标之外的像素点的色彩不应该被修改,它们的内容将通过对显示存储器的访问而被装入到索引寄存器中。坐标为(7/0)位和(5/0)位的 2 位将会被操作,其余位将不变地回送到位平面。因而,数值 00000101B(05H)首先送到位屏蔽寄存器,数值 00 送到功能选择寄存器。将色彩值 1011B=0BH 写入“设置/清除”寄存器,有 2 位将应接收这个代码。为了使这个色彩可以从寄存器中取出,应该在“允许设置/清除寄存器”中写上数值 1111B(0FH)。接着,执行对显示存储器的访问,而传输的处理器字节并不起任何作用。其程序如下:

```

MOV     AX,0A000H
MOV     DS,AX           ; 把显示存储器地址放入 DS
XOR     BX,BX           ; 装入位移地址 0
MOV     AL,[BX]         ; 装入 0 字节到 AL 寄存器中
MOV     DX,3CEH         ; 送图形控制器的地址寄存器口地址到 DX
MOV     AX,0005H        ; 记入读方式 0 和写方式 0 到方式寄存器
OUT     DX,AX           ; 写 0 到功能选择寄存器
MOV     AL,03H
OUT     DX,AX
MOV     AX,0508H
MOV     DX,AX
OUT     AX,0B00H        ; 写一个新的彩色码到设置/清除寄存器
MOV     DX,AX
MOV     AX,0F00H        ; 写 1111B 到“允许设置/清除寄存器”
OUT     DX,AX
MOV     [BX],AL         ; 操作并重写寄存器

```

7.4 教学建议

本章重点是键盘和键盘接口工作原理、打印机接口工作原理、MC6845 CRT 控制器、CGA 的显示模式和逻辑结构、显示程序设计。上述重点在课堂上应详细讲解,让学生能够深入了解用户交互接口的工作原理,并能够用程序实现一些常见的有关画图的程序。在字符显示和图形显示 EGA 和 VGA 的工作原理也应该讲解,对于 EGA 和 VGA 程序设计基础的各种寄存器应分类进行介绍,举例说明。

第 8 章 外存储器

8.1 知 识 要 点

8.1.1 数据磁记录方式

数据磁记录方式是在磁头线圈中形成的电流在磁头周围产生磁场，使磁头下运动的磁盘介质磁化，把二进制数据序列转换成有规律的脉冲序列记录方式。

记录二进制数据“1”和“0”两种状态，可以用磁化强度的大和小、饱和与不饱和磁化状态、饱和磁化的不同方向表示。大多数磁盘存储器是通过介质磁层不同方向的饱和磁化翻转来记录二进制数据的。

根据介质磁化方向是平行或者是垂直于介质平面分为：水平磁记录方式及垂直磁记录方式。利用磁头磁场水平分量在介质上写入信息的水平磁记录方式存在着自退磁效应和脉冲拥挤效应，限制了记录密度的提高；利用磁头磁场垂直分量在介质上写入信息的垂直磁记录方式可使相邻的退磁场几乎为零，适合于高密度记录。

8.1.2 数据磁记录编码技术

对有用的数据序列和同步的时钟序列进行编码，或对数据序列的位置进行重新编排，将其变换成记录脉冲序列，称为磁记录编码。

数据磁记录编码中常用的概念和术语有位单元、读出分辨力、自同步能力、编码效率和磁化翻转间隔比等。

位单元：编码后形成脉冲序列，其位单元正中位置包含是“1”或“0”的数据位，起始位置包含同步用的时钟位；解码时电路可从脉冲序列中分离出时钟和数据。非编码和编码数据磁记录如表 8.1 所示。

8.1.3 增强IDE接口

IDE 接口又称为 AT 嵌入式接口，是系统级接口。它综合了 ST506、ESDI 和 SCSI 接口的特点，低价易用，是 PC 工业标准。IDE 接口是 AT I/O 通道总线(ISA

表 8.1 非编码和编码数据磁记录

记录	码制	时钟位 e_1	数据位 e_2	特 点
非 编 码	三电平 RZ	无	正、负电平	数据位之间为零电平
	RZ	无	正/负、零电平	脉冲电平与数据位对应
	NRZ-I	无	逢 1 翻转	编码制记录的基础
	NRZ-C	无	导码翻转	
编 码	FM	$e_1 = 1$	$e_2 = d_0$	编码效率 50%，自同步能力强，存储密度低
	MFM	$e_1 = \overline{d_{-1}} \overline{d_0}$	$e_2 = d_0$	编码效率 100%，综合指标优良，应用广泛
	M ² FM	$e_1 = \overline{e_{-1}} \overline{d_{-1}} \overline{d_0}$	$e_2 = d_0$	编码效率 100%，自同步能力差，可靠性好
	GCR(4/5)	二元记录序列中不出现 2 个以上连续的“0”，编码效率为 80%，有一定的自同步能力，高密度编码		
	(d,k)RLL	d 为“1”之间插入“0”的最少个数，决定记录密度 k 为“1”之间插入“0”的最多个数，决定定时程度		
		FM、MFM、M ² FM、GCR(4/5)可表示为 0,1RLL、1,3RLL、1,4RLL、0,2RLL		

总线)的扩充，只有芯片、驱动器之间通信的 5 个信号是 IDE 总线仅有的。

为了适应设备容量大、速度快、数量多和种类多的需求，主要可从以下 4 个方面改进使 IDE 接口，使之成为增强 IDE 接口。

1. 扩大容量

IDE 驱动器容量受限于 BIOS 和 IDE 的 CHS 中较小值组合，仅为 528MB。对于不用 BIOS 存取磁盘的操作系统采用自动配置方式旁路 BIOS，使驱动器的容量可以扩大到 IDE 接口本身支持的大小。对于使用 BIOS 存取磁盘的操作系统采用自动转换方式，或者将 BIOS CHS 信息转换成 IDE CHS 信息，或者将 BIOS CHS 信息转换成 IDE LBA 信息，使增强 IDE 能够访问 BIOS 支持的最大容量。不过这种方法要求对 BIOS 作较大改进以对自动转换提供支持。

2. 加快速度

主机从硬盘缓冲区存取数据的速度称为最大突发速率，IDE 接口采用 PIO 方式传输数据，增强 IDE 接口采用 DMA 方式传输数据，把速率提高到 33MB/s 和 66MB/s。Ultra DMA33 改善了驱动程序，用时钟上下两相传输数据和由硬盘产生选通信号送数据到总线；Ultra DMA66 进一步提高了速率，采用新型 CRC 校验和 80 根引线等措施来提高可靠性。

3. 增加驱动器数量和种类

通过系统、BIOS、OS 厂商对 IDE 接口的两个通道提供硬、软件支持，增强 IDE 接口能够在同一系统中同时支持 4 个 IDE 驱动器。通过扩充 ATA 协议，提供 ATAPI 建议，增强 IDE 接口扩充了设备类型，为 CD-ROM 和磁带机规定了命令集和寄存器，可以支持符合 ATAPI 接口的 CD-ROM 驱动器和磁带机。

8.1.4 SCSI接口

SCSI 是美国国家标准协会(ANSI)公布的接口标准,它是系统级接口,逻辑接口;它定义了一种主机适配器和智能控制器之间双向对等的并行 I/O 总线。SCSI 总线上的设备没有主从之分,是对等关系。启动器和目标器之间采用高级命令通信,不涉及外设特有的物理性质。SCSI 总线上的设备最大数量在 8 位数据宽度时为 8 个,16 位数据宽度时是 16 个。

SCSI 是一个多任务接口,具有总线仲裁功能,同一 SCSI 控制器控制速率为 1.5MB/s(异步)和 5MB/s(同步)。通过提高时钟频率、加宽数据宽度、低压差分方式(LVD)和双边界时钟方案等技术,同步数据速率可从 5MB/s 逐步提高到 10MB/s、20MB/s、40MB/s、80MB/s 和现在的 160MB/s(Ultra 160)。

ANSI 在 1986 年公布 SCSI-1,1990 年推出 SCSI-2,20 世纪 90 年代开始制定 SCSI-3 规范。SCSI-3 是一个多层结构,其协议层除了原有的并行协议外,新增了 3 个协议:串行协议、块传输协议和光纤信道协议。SCSI-3 共有 4 个接口:SCSI-3 并行接口、IEEE1394 接口、SCSI-3 串行接口和 SCSI-3 光纤信道接口。所有的新型接口都是以 PCI 插卡出现,64 位 PCI 总线设计的初衷就是为连接 LAN 卡、SCSI 卡等外设接口,以满足外设高性能工作站和服务器的传输需求。

8.1.5 软盘存储技术的发展

1. 改进盘片介质、磁头性能和记录方式

为了提高记录密度、扩大软盘容量,采用了高矫顽力的存储介质、MIG 复合磁头或硬盘非接触式磁头、垂直磁记录方式,位密度可达 80KB/in(1in=2.54cm),3.5in 软盘容量从 4MB 到 50MB。

2. 充分发挥 ECC 纠正技术的作用

软盘由磁头和磁盘接触方式工作于非密封环境,为了保证读/写可靠性,充分发挥 ECC 校正技术的作用,扩大软盘驱动器中可纠正突发码的位数和允许的个数,并用控制器内置 CRC 电路保证 ECC 的正确性。

3. 采用先进的伺服定位技术

扇区伺服、埋入伺服和光伺服技术可使磁盘定位误差减小,道密度提高,从而提高软盘容量。

4. 使用 SCSI 接口连接主机适配器和软盘控制器

按 SCSI 标准设计嵌入式控制器,把与驱动器接口的部分留在驱动器中,把与主机接口的部分移到适配器中,驱动程序固化在主机适配器的 BIOS 内,速度快、连接方便、具有通用性。

软盘控制器采用 82077AA 芯片。82077AA 可以控制 4 个 5.25in 或 3.5in 的软

盘驱动器,具有4种数据传输率。片内具有可以编程的数据、状态和控制寄存器,有接口信号的缓冲和驱动电路,并有磁带驱动器控制电路。82077AA有与系统接口的数据线、片选地址线、读/写控制线、中断及DMA控制线以及复位、晶振、速率选择、系统表示信号线;有与驱动器接口的驱动器选择、主轴电机选择、磁头选择、步进方向与脉冲等输出信号,有0磁道、写保护索引、磁盘改变、第2驱动器等输入信号。写预补偿和串口电路输出写允许、写数据和码制信号,数据分割电路输入软盘数据和锁相环信号。它是一个通用性好的LSI芯片。

5. 高密度大容量产品

(1) 光磁软盘 (Floptical)

磁记录存储光伺服定位的光磁软盘(3.5in, 21MB)具有容量大、速度快、兼容性好、可靠性高和安装使用方便的特点。它采用(1, 7) RLL编码方式,道密度为1245tpi、位密度为24000bpi、平均寻道时间为80ms、突发数据速率为1.5M/s、2个磁头和753个柱面形成25MB的非格式化容量和21MB的格式化容量。

(2) Zip 软盘系统 (Zipdrive)

结合使用标准硬盘驱动器非接触式和软盘磁介质的相关技术,容量可达100MB、平均存取时间为29ms、盘转速为3000r/min、突发数据速率为1.4MB/s,具有外置并口、内外置SCSI接口,与普通3.5in软盘驱动器无互换性。

Zip 250MB USB驱动器使用Zip 250软盘、可向下兼容Zip 100MB盘片。它使用USB接口,也可通过PCMCIA卡与笔记本机相连。它具有高速、大容量、轻便易用的特点及一机多用的功能,为不同机型之间的数据交换与共享提供了便捷的途径。

8.1.6 硬盘存储技术的发展

硬盘存储技术正朝着高密度、大容量、高速度、高可靠性方向发展。

1. 温彻斯特技术

将磁头、小车、导轨、主轴、盘片等组成的头盘组件全部封装在一个腔体内。采用质轻易浮的磁头、接触式启停、浮动时读/写工作方式,以及主轴电机制动机构,盘面润滑等温彻斯特技术提高了硬盘驱动器的道密度、位密度和可靠性,适用于大气环境下工作。

2. 盘片驱动和磁头定位技术

驱动盘片的主轴电机由滚珠轴承改为黏膜液油轴承,有吸震、减噪、降温、寿命长等特点;音圈电机驱动、光伺服闭环控制的磁头定位系统使硬盘道密度提高到5000~17000tpi。

3. 磁盘和磁头的技术进步

采用电镀和溅射工艺制成的厚度极薄的高磁性能金属薄膜硬盘,具有高记录

密度、高制磁和高矫顽力的特点。

使用窄道宽的磁阻 (MR) 磁头和巨磁阻 (GMR) 磁头容易测出窄道宽磁盘上的读出信号, 可以提高面密度及容量。用于读的磁阻磁头和用于写的磁感应磁头可以分别优化以得到最好的性能。

4. 局部响应最大相似 (PRML) 通道技术

利用 PRML 通道技术可以提高记录密度、读取速度和读取可靠性, 与 MR 磁头技术相结合, 可以大幅度提高存储容量和数据传输率。

5. 自监测、分析及报告技术 (SMART) 和数据保护功能

SMART 根据历史记录及预设安全值对被监测的磁头、磁盘、电机、电路的运行情况进行分析、比较, 发现问题可以发出警告、自动降速并把重要数据文件备份到其他安全扇区、硬盘或存储设备中。

SMART 只能发现问题而无修复功能, 新一代硬盘嵌入的 Data Lifeguard 系统在硬盘闲置时间离线执行错误修复, 可以减少在使用时执行错误修复, 改善硬盘的整体性能。

6. 磁盘阵列技术

具有海量存储、并行处理、高可靠性和便于维护等特点的 RAID 技术的核心是系统冗余、数据容错和提高存取性能。在结构上 RAID 采取 ECC 电路、奇偶校验盘和镜像盘等冗余措施来实现多级容错。

RAID-0 数据分拆 (Striping), 条块交叉存储, 高速高性能。

RAID-1 数据分拆, 扇区交叉存储, 镜像盘备份。冗余量大, 可靠性高。

RAID-10 2 个 RAID-0 互为镜像构成一个 RAID-1。性能最好, 代价较高。

RAID-2 海明码校验, 位交叉存储, 有冗余。适于大量顺序 I/O 请求的计算。

RAID-3 数据分拆, 位/字节交叉存储, 奇偶盘校验。适于超级计算, 大数据量传输。

RAID-4 数据分拆, 扇区交叉存储, 奇偶盘校验。适于事务处理、小数据量传输。

RAID-5 数据位与奇偶位组合分拆, 按扇区交叉存储。可靠性高, 适于小数据量传输。

RAID-6 数据分拆, 块交叉存储, 双盘存放及检错纠错冗余码。可靠性很高, I/O 传输率较高。

RAID-7 具有智能化实时 OS、存储管理软件工具、多 I/O 通道的非同步访问、奇偶检验盘、磁盘 Cache、多重热备份和电源冗余功能的存储计算机, 具有很快的速度和很高的性能。

8.1.7 磁带存储技术

1. 螺旋扫描读/写磁带机

螺旋扫描读/写磁带机通过磁头旋转、螺旋扫描（磁头在磁带上的运动轨迹是螺旋线），达到较高的记录密度。

① DAT 磁带机：即数字音频磁带机。它使用 4mm 磁带，有 DDS-1、DDS-2、DDS-3 等品种。DDS-3 单盘容量为 12GB（未压缩），数据传输率为 1.2MB/s，适用于工作站、中小型服务器作备份。

② 视频盒式磁带机：使用 8mm 磁带。8mm 磁带有两个不兼容的品种，Exabyte 公司的 Mammoth 和 Sony 公司的 AIT。它们的单盒容量为 20GB 或 25GB（未压缩），数据传输率都为 3MB/s，都是采用蒸发金属工艺制成，适用于中大型服务器作备份。

2. 多磁头平行读/写磁带机

具有磁头固定、多磁头读/写、读/写轨迹与磁带运动方向平行的特点。

① QIC 磁带机，使用 1/4in 盒式磁带，有 Travan 标准的 TR-1、TR-3、TR-4 等品种。TR-4 的单盒容量为 4GB，数据传输率为 300KB/s，适用于单机备份。

② DLT 磁带机，采用将数据置于纵向磁道上的线性记录方式，具有更高的容量和速度，磁头寿命可达 10000 小时。Quantum 公司的 DLT-7000 磁带单盒容量为 35GB，数据传输率为 5MB/s，适用于高档服务器和大型机备份。

3. 磁带库

① 磁带自动加载机（Autoloader）。它是小型自动化备份装置，由一个磁带机和 7 盒及以上的磁带组成，可备份 100~200GB 的数据。主要产品有 Quantum 公司的 DLT 等。

② 磁带库(Library)。它是柜式封闭机构，由磁带机、机械手和十至几十盘磁带构成。大型磁带库可有 1000~6000 盘磁带，容量可达几百 PB，适用于中大型数据中心。

8.1.8 光盘存储技术

1. 光盘存储原理

① 写入。写入时由记录信息调制的具有一定能量的激光束聚焦到光存储介质上形成直径为 $1\mu\text{m}$ 以下的细微光点，利用激光对光存介质的光效应或热效应，形成凹坑、起泡、相变、色变或偏振态变化的信息点，使其反射率发生变化。根据激光照射前后的两种不同状态及相应的反射率强弱，记录为“1”和“0”信息。

② 读出。读出时用低功率的激光束扫描信息轨道，由光电检测器检测其反射

率的变化,经过光电转换解调出存储信息。

2. 光盘驱动器工作原理

光盘驱动器工作原理框图如图 8.1 所示。

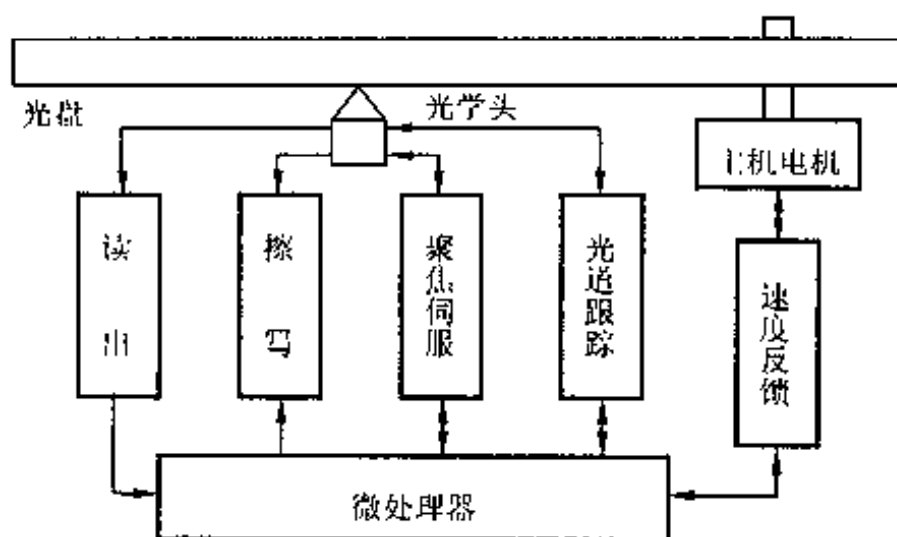


图 8.1 光盘驱动器工作原理

擦写和读出时要调节激光器发射的激光束功率。

聚焦伺服与光道跟踪系统根据光电检测的读/写光点与信息轨道的跟踪误差信号,控制光学头的二维力矩器在与光盘垂直方向上移动聚焦透镜,在光盘径向方向上移动透镜,使物镜光束正确聚焦在信息轨道中央的信息点上。

主轴电机速度反馈系统利用旋转编码器产生伺服信号,控制光盘以恒线速度(clv, 读视音频数据)或恒角速度(cow, 随机访问数据)旋转,以标准格式读/写数据。

3. CD-ROM 的规范及其格式

1985 年发表了 CD-ROM 黄皮书,后来成为 ECMA-119 标准。它定义了 CD-ROM 的物理格式,为误码率要求严格的计算机数据定义了存储模式 1,为误码率要求不高的数据定义了存储模式 2。

1988 年制定了 CD-ROM/XA 规范,1991 年又制定了 CD-ROM/XA II 规范,它们是扩展的音频、视频、文本、图形和图像的混合格式,也有两种模式。对应于上述 CD-ROM 的物理格式,数据记录的逻辑格式是 ISO9660 和 ISO9660 II。它们定义了 CD-ROM 卷和文件结构,一片 CD-ROM 为 1 卷,1 组卷构成卷集。文件系统由逻辑块和逻辑扇区、记录、文件、卷、卷集多级分层结构定义。

4. 操作系统对 CD-ROM 的支持

DOS 对 CD-ROM 的支持使 MSCDEX 程序能使用 DOS 系统网络重定向接口实现 CD-ROM 扩展,使 DOS 应用不需改动就可以直接应用到光盘上。其结构如图 8.2 所示。

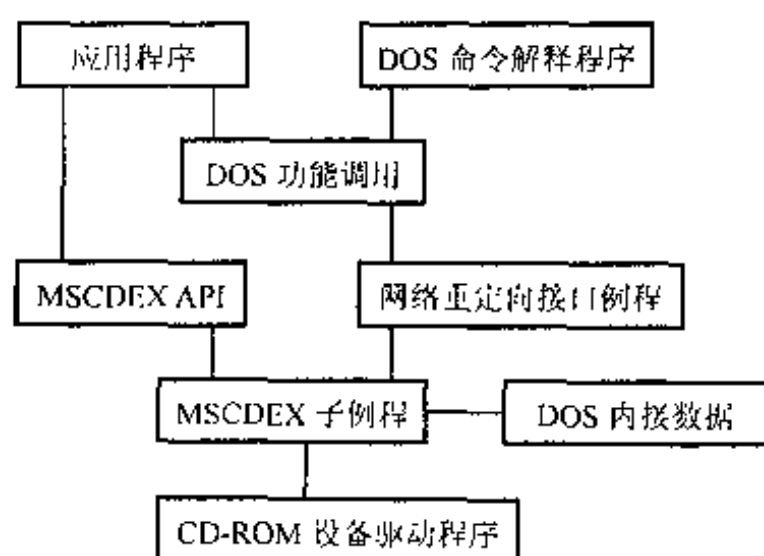


图 8.2 操作系统对 CD-ROM 的支持

8.2 例题精析

例 8.1 画出数据序列 1011001000100001 在 RZ、NRZ-I、NRZ-C、FM、MFM、M²FM 磁记录方式下的脉冲波形。

解 三电平归零制 RZ 分别用正、负脉冲表示二进制数据 1、0。

归零制用正脉冲或负脉冲表示二进制数据 1，无脉冲表示二进制数据 0，本题选用正脉冲表示数据 1。

逢 1 翻转不归零制 NRZ-I 中，数据序列的二进制位为 1 时写电流翻转一次，为 0 时写电流不翻转。

异码翻转不归零制 NRZ-C 中，相邻数据位不同时写电流翻转。

在 FM 制中，每个位单元的起始位置是为 1 的时钟位，每个位单元的中心位置是数据位，数据位为 1 出现脉冲，数据位为 0 不出现脉冲。

在 MFM 制中，数据位仍在位单元中间，数据位为 1 出现脉冲，数据位为 0 不出现脉冲，但是，仅当相邻两个位单元中的数据都是 0 时，才在位单元之间插入时钟位。

在 M²FM 制中，数据位仍在位单元中间，数据位为 1 出现脉冲，数据位为 0 不出现脉冲，但是仅当数据序列中连续 0 的数目不小于 2 时，才在头两个 0 的位单元之间插入时钟脉冲，以后每隔两个 0 在下一个数据 0 的位单元插入一个时钟脉冲。

根据这几种磁记录方式的特点，脉冲波形如图 8.3 所示。

例 8.2 在 NRZ-I 制中，写电流的 () 对应数据位 1。

A. 上升沿 B. 上升沿和下降沿 C. 高电平 D. 低电平

解 答案为 B。NRZ-I 制中，数据序列的二进制位为 1 时写电流翻转一次，可能由 1 翻转到 0，也可能是由 0 翻转到 1，所以选择 B。

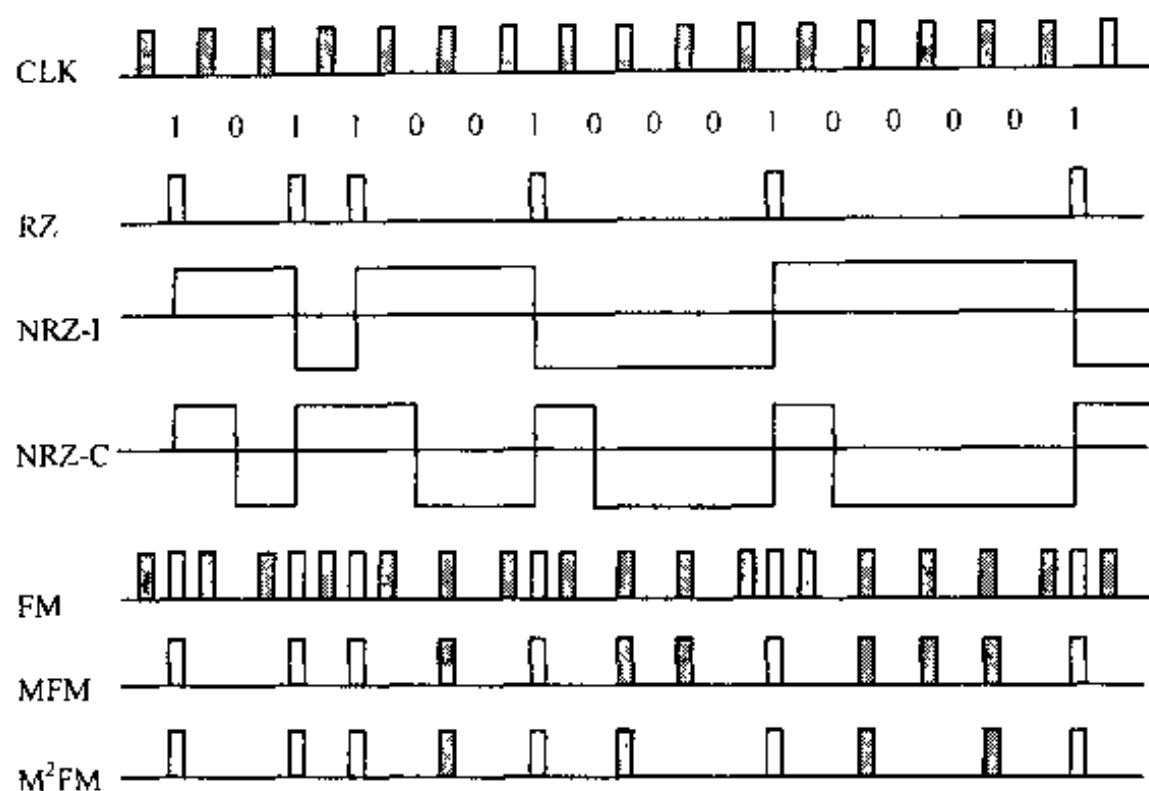


图 8.3 几种磁记录方式的脉冲波形

例 8.3 在编码方式中,每个位单元的起始位置是_____,中间位置是_____。

解 在编码方式中,每个位单元的起始位置是时钟位,中间位置是数据位。

位单元是经过编码后形成的脉冲序列的基本单位。一个位单元包含有用的数据位和同步用的时钟位,时钟位位于每个位单元的起始位置,数据位无论是 1 或 0 都位于每个位单元的正中位置。

例 8.4 将数据序列 1011001000100001 按 GCR(4/5)编码方式转换成的记录序列为 ()。

A. 1011001000100001

B. 11011100101001010001

C. 01011100101001011011

D. 11011100101001001011

解 答案为 C。GCR(4/5)是把数据按 4 位编成一组,再与之对应变换成 5 位的记录序列,每组以及各组记录连接后的记录序列中都不允许出现连续两个以上的 0。

首先将数据序列按 4 位分组,得到 4 组分别为: 1011、0010、0010、0001,然后根据配套教材 318 页的 GCR(4/5)编码规则表,依次将这 4 组变换成 01011、10010、10010、11011,最后合并得到纪录序列为 01011100101001011011。

例 8.5 按照(d,k)RLL 的特点可知, MFM 编码为_____RLL, M^2 FM 编码为_____RLL。

解 按照(d,k)RLL 的特点可知, MFM 编码为 1,3 RLL, M^2 FM 编码为 1,4 RLL。

(d,k)RLL 编码是使记录序列中两个 1 之间 0 的个数受限制的编码,两个 1 之间插入 0 的最少个数和最多个数分别为 d 和 k。

在例 8.1 的波形图中,列出了 MFM 编码和 M^2 FM 编码的所有情况。在为 1 的第三个数据位和为 1 的第四个数据位之间是插入 0 个数最少的情况,仅有一位

为 0 的时钟位。对于 MFM 制, 在为 1 的第一个数据位和为 1 的第三个数据之间是插入 0 个数最多的情况, 有一个为 0 的数据位和两个为 0 的时钟位, 共有 3 个 0。对于 M^2 FM 制, 在为 1 的第九个时钟位和为 1 的第十一个数据位之间是插入 0 个数最多的情况, 有两个为 0 的数据位和两个为 0 的时钟位, 共有 4 个 0。

例 8.6 当 IDE 驱动器要请求 DMA 传送时, IDE 接口向 DMAC 发出 () 信号。

- A. DMARQ B. DMACK- C. INTRQ D. IORDY

解 答案为 A。在进行 DMA 传送前, IDE 接口通过 DMARQ 向 DMAC 发送 DMA 请求信号 DMARQ, 当 DMAC 获得了总线控制权后, 向 IDE 接口发送 DMA 响应信号 DMACK-, 允许 DMA 传送。INTRQ 是向中断控制器发送的中断请求。IORDY 是为了使驱动器的数据传送和主机速度匹配而发送的 I/O 通道就绪信号。

例 8.7 以下关于 SCSI 接口说法正确的是 ()。

- A. 启动器是主设备, 目标器是从设备
B. 在同步方式下, REQ 和 ACK 信号操作在锁步方式下
C. 系统初启总是先进入总线空闲阶段
D. 在异步方式下, 目标器准备好接受数据时使 ACK 信号有效

解 答案为 C。在 SCSI 总线上, 每种设备可以是启动器也可以是目标器, 不是主从关系, 而是双向对等关系, 所以 A 错。在异步方式下, REQ 和 ACK 信号操作在锁步方式下, 当目标器准备好接收数据时使 REQ 信号有效, 然后主机使 ACK 有效并把数据放到总线上, 所以 B 和 D 错。

例 8.8 在硬件复位操作中, 当 82077AA 工作于 AT 模式时, IDENT 和 MFM 引脚 ()。

- A. 均输入 1 B. 均输出 1
C. 分别输入 1 和 0 D. 分别输入 1 和 0

解 答案为 A。AT 模式下的主要特征是: DMA 门控逻辑电路有效, TC 信号高电平有效, 状态寄存器 A 和 B 无效。PS/2 模式下主要特征是: 无 DMA 门控逻辑电路, TC 信号低电平有效, 状态寄存器 A 和 B 有效。PS/2 30 模式的主要特征是: DMA 门控逻辑电路有效, TC 信号高电平有效, 状态寄存器 A 和 B 有效。

例 8.9 以下关于磁盘阵列的说法正确的是 ()。

- A. 镜像盘又称为冗余盘
B. RAID-0 容错结构中无冗余数据
C. RAID-10 的硬盘使用率同 RAID-0 一样
D. RAID-4 将数据按位或字节交叉存在几个硬盘中, 并将奇偶校验位单独存在一个硬盘中

解 答案为 B。镜像盘又称为热备份盘, 不管向主盘存入什么数据, 系统都

要将数据完整、实时地保存在镜像盘中；冗余盘又称奇偶校验盘，存放对应数据位的奇偶信息。所以 A 错。

RAID-0 通过 2 个以上的硬盘组成磁盘阵列，数据被均匀等量地分成数据块分别放在几个硬盘中，没有奇偶校验位，也没有镜像，故无冗余数据，利用率达到了 100%。所以 B 正确。

RAID-1 整个系统会有一个主硬盘和至少一个做实时备份用的副硬盘，通过镜像提高容错性。RAID-10 中两组驱动器都采取 RAID-0 结构，而两组 RAID-0 阵列采用磁盘镜像技术构成一个 RAID-1 阵列，故 RAID-10 的硬盘使用率同 RAID-1 一样。所以 C 错。

RAID-3 和 RAID-4 都是将奇偶校验位单独存在一个硬盘中，但是 RAID-3 将数据按位或字节交叉存在几个硬盘中，RAID-4 将数据以扇区为单位交叉存在硬盘中。所以 D 错。

例 8.10 在 CD-ROM 模式 1 中，用户数据占光盘容量的（ ）。

A. 100% B. 87.8% C. 87.1% D. 37%

解 答案为 C。CD-ROM 模式 1 中，每块 2352B，其中 2048B 的用户数据，所以用户数据所占比例约为 $2048 \div 2352 = 87.1\%$ 。

8.3 学 习 自 评

8.3.1 自测练习

一、填空题

1. 数据磁记录方式是指利用某种规律将 (1) 转换成与介质磁层相应的 (2) 的方式。
2. 举出 4 种非编码的磁记录方式：(1)，(2)，(3)，(4)。
3. 在编码磁记录方式中，二进制数据串不仅包含有用的 (1) 位，还包含作为 (2) 的时钟位。
4. 在 FM 制、MFM 制和 M^2 FM 制中分别有 (1) 种、(2) 种和 (3) 种频率。
5. 将数据序列 1001010001000001 按 GCR(4/5)编码方式转换成的记录序列为_____。
6. 微机系统中，硬盘控制器与存储设备间的接口属于 (1) 级接口，主机适配器与硬盘控制器间接口属于 (2) 级接口。
7. 在 ESDI 接口中，驱动器和控制器之间传输的是 (1) 制数据，支持 (2)

和 (3) 两种工作方式。

8. EIDE 有 (1) 和 (2) 两种数据传输方式。

9. SCSI 总线上的设备分为启动器和目标器,前者是 (1) 的设备,后者是 (2) 的设备。

10. 82077AA 芯片的 FIFO 缓冲区为 _____ 字节。

11. 光磁软盘将 (1) 技术和 (2) 技术结合在一起,采用前者 (3), 采用后者 (4)。

12. 磁带机按读/写方式可分为 (1) 和 (2) 两大类。

13. 蛇形记录方式是指记录信息时 (1) 磁道从磁带首端到磁道末端, (2) 磁道则相反,且首尾相接。

14. DVD 的系统码流主要由 (1)、(2) 和 (3) 3 部分组成。

15. DVD 具有 _____ 个独立的音频码流。

二、单选题

1. 不属于编码磁记录方式的为 ()。

A. NRZ B. FM C. GCR D. 1,3RLL

2. 在归零制中,利用 () 来表示数据 1。

A. 正脉冲 B. 负脉冲 C. 有脉冲 D. 无脉冲

3. 按照(d,k)RLL 的特点可知,GCR(4/5)编码为 () RLL 编码。

A. 0,1 B. 0,2 C. 1,2 D. 1,3

4. 以下关于 ST506/412 接口的说法正确的是 ()。

A. 它适用于闭环控制的磁头定位系统

B. 它在驱动器和控制器之间传送的是从 MFM 制编码数据中分离出的有用数据

C. 它最多只能选择 8 个磁头

D. 它有菊链连接和星形连接两种方式,这两种方式最多都只能连接 4 台驱动器

5. 在 IDE 接口中,当系统初启时若从驱动器存在,则向主驱动器发送 () 信号。

A. SPSYNC- B. DASP C. PDIAG- D. RESET-

6. 基于 BIOS 的 IDE 磁盘驱动器的最大容量为 ()。

A. 136.9GB B. 8.4GB C. 528MB D. 512MB

7. 当 82077AA 接受 TC 输入的信号时,正在进行 () 方式的数据传送。

A. 查询 B. 中断 C. DMA D. 通道

8. 82077AA 芯片有 () 个端口。

A. 3 B. 8 C. 10 D. 13

9. 82077AA 芯片最多能控制 () 个软盘驱动器。
A. 1 B. 2 C. 3 D. 4
10. 当软盘驱动器磁头在 0 磁道时, 一定为逻辑 1 的信号是 ()。
A. TRK_0 B. WP C. IND X D. PLL_0
11. 以下关于硬盘存储器说法正确的是 ()。
A. 温彻斯特盘采用非接触启停工作方式
B. 硬盘控制器将从硬盘接受的并行数据送往系统总线
C. 硬盘驱动器能对写操作进行预补偿
D. 磁阻磁头读出幅度与道宽无关, 可提高盘片道密度, 但不能进行写操作
12. 在容错结构中, 使用分块交叉存储数据技术和奇偶校验盘的有 ()。
A. RAID-0 B. RAID-2 C. RAID-5 D. RAID-6
13. EVENODD 结构用 () 个冗余盘。
A. 0 B. 1 C. 2 D. 3
14. 磁带机有 () 个带盘。
A. 1 B. 2 C. 3 D. 4
15. QIC 磁带机用 () 磁记录编码方式记录数据。
A. GCR B. FM C. NRZ D. RZ
16. 使用螺旋扫描读/写方式的磁带机有 ()。
A. QIC B. 视频盒式磁带机 C. DLT D. TR-4
17. 光盘存储器根据激光束照射后 () 的强弱来读出二进制“0”和“1”。
A. 激光束能量 B. 磁场 C. 反射率 D. 折射率
18. CD-ROM 光盘记录数据的总容量为 ()。
A. 2048B B. 153.6KB C. 9.216MB D. 552.96MB
19. CD-ROM 规范中光道被分为 () 两种。
A. 音频和数据 B. 音频和视频 C. 视频和数据 D. 图像和数据
20. 光盘和磁盘文件的格式不同, DOS 为了支持光盘使用扩展文件 ()。
A. CD-ROM.EXE B. MSCDEX.EXE
C. CONFIG.SYS D. SYS.COM

三、问答题

1. 比较 FM 制、MFM 制和 M^2 FM 制的优缺点。
2. 何谓脉冲拥挤效应?
3. 试述光磁软盘的特点。
4. 试述硬盘驱动器的读/写原理。
5. 试述硬盘存储器的特点及其发展。
6. 试述磁带机的读/写工作原理。

7. DVD 是用了哪些技术来提高光盘记录密度的?

四、作图题

画出数据序列 1001010001000001 在 RZ、NRZ-1、NRZ-C、FM、MFM、M²FM 磁记录方式下的脉冲波形。

8.3.2 自测练习解答

一、填空题

- (1) 二进制数据序列 (2) 磁化翻转
- (1) 三电平归零制 (2) 归零制
(3) 逢 1 翻转不归零制 (4) 异码翻转不归零制
- (1) 数据 (2) 同步
- (1) 2 (2) 3 (3) 4
- 01001111011110111011
- (1) 设备 (2) 系统
- (1) 不归零 (2) 步进 (3) 串行
- (1) PIO (2) DMA
- (1) 发出命令 (2) 接受并执行命令
- (1) 16
- (1) 磁记录 (2) 光学 (3) 读/写数据 (4) 伺服定位
- (1) 螺旋扫描读/写磁带机 (2) 多磁头平行读/写磁带机
- (1) 偶数 (2) 奇数
- (1) 主视频码流 (2) 子图像码流 (3) 声频码流
- (1) 8

二、单选题

- | | | | | | | |
|-------|-------|-------|-------|-------|-------|-------|
| 1. A | 2. C | 3. B | 4. D | 5. B | 6. C | 7. C |
| 8. B | 9. D | 10. A | 11. D | 12. D | 13. C | 14. B |
| 15. A | 16. B | 17. C | 18. D | 19. A | 20. B | |

三、问答题

1. 在 FM 制编码中, 在每个位单元里必定有一个非数据的时钟位, 所以编码效率为 50%, 限制了其存储密度, 但这使它具有很强的自同步能力。

MFM 制编码效率提高到 100%, 比 FM 制记录密度高很多, 而且保留了 FM 制同步能力强的优点, 是一种优良的码制。

M²FM 制编码效率也是 100%, 记录密度高, 且可靠性高, 但自同步能力稍差。

2. 在饱和磁记录方式中, 连续脉冲相互影响引起的读出幅度降低、峰值偏移的现象称为脉冲拥挤效应。

3. 容量大, 兼容性好, 可靠性高, 传输率高, 安装使用方便。

4. 工作时盘片在主轴电机驱动下匀速旋转, 通过磁头臂安装在滑动小车上的读/写磁头浮动在盘片上, 在磁头驱动和定位机构控制下沿盘面做径向运动。磁头机构根据柱面地址把磁头向目标磁道移动, 所选的磁头定位在目标磁道上, 待指定扇区旋转到磁头下, 再根据命令进行读/写操作。若是写操作, 则把要写入的二进制数码经编码电路变换成相应的电流信号, 送到磁头线圈, 在磁道上形成一串微小的磁化单元。读出时, 磁道上记录的磁化单元高速掠过磁头, 在磁头线圈中感应出电压信号, 再经放大、鉴别和整形电路后, 还原成数字信号。

5. 采用非接触的头、盘结构, 采用高精度、轻型的磁头驱动和定位系统, 采用全密封结构, 采用液态轴承马达, 使用连续的金属薄膜磁盘, 使用窄道宽的磁阻磁头提高面密度, 采用局部响应最大相似通道, 具有数据保护功能, 提高了驱动器的速度。

6. 执行写操作时, 磁带控制器发出正向走带命令, 磁头开始正方向运动, 在记录区到达读/写磁头下面时, 抹除磁头清除原有信息, 然后读/写磁头把控制器发来的数据写入磁带。写入同时还要读出数据与控制器送来的内容比较, 如果有错就反向走带退回去, 重新写入, 若多次重复执行仍不成功, 则报告“出错”信息。

读出操作分正读、反读, 读出信号经放大、整形和校验后送控制器, 如果经校验发现有错, 则自动重复执行, 多次执行均失败后报告故障。

7. 提高物镜数值孔径, 减小纠错编码冗余度, 采用 8/16 通道码调制方式, 减小激光波长, 减小光斑直径, 减小道间距, 减小凹坑最小长度, 减小凹坑密度。

四、作图题

脉冲波形如图 8.4 所示。

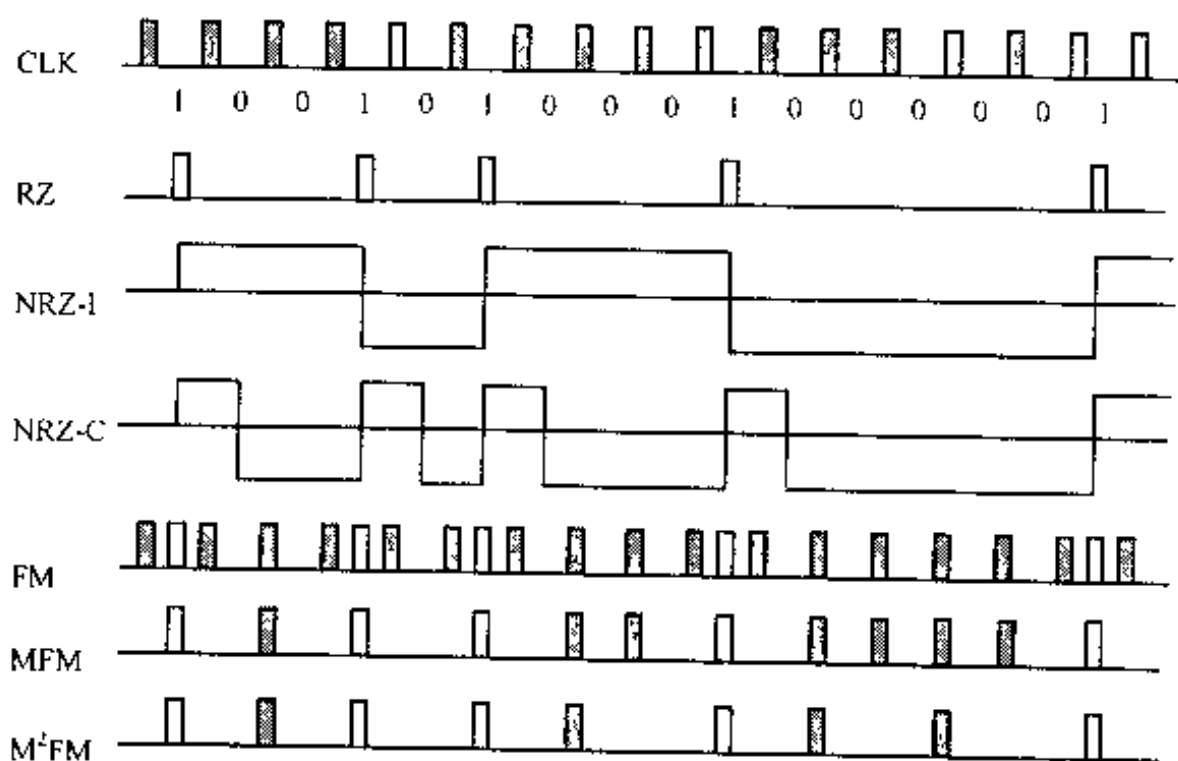


图 8.4

8.4 教 学 建 议

本章重点是数据磁记录方式和编码技术、EIDE 接口和 SCSI 接口、软盘控制器、温彻斯特技术、RAID 技术、光盘存储原理和光盘驱动器原理。上述重点在课堂上应讲授比较详细,着重讲述它们的基本原理和关键技术;课后要适当布置一些作业巩固和扩大所学知识。

软盘、磁盘、光盘、磁带存储器中的内容都有存储介质、控制器、驱动器、设备驱动程序等基本知识,这些外存的详细内容可以在外部设备等专业书籍中找到,本章讲授时应简要说明这些外存储器现有的先进技术和未来的发展趋势。

其他内容则由学生自学,教师可介绍相关资料以扩大知识面。

第9章 微计算机系统

9.1 知 识 要 点

9.1.1 IBM PC/XT系统基本组成

1. PC/XT 系统板

PC/XT 系统板分为 5 个功能模块：处理器子系统(CPU 及其支持器件)，只读存储器(ROM)子系统，随机存储器(RAM)子系统，I/O 接口和 I/O 通道。

(1) 处理器子系统

该子系统由 8088 处理器及支持芯片组成。支持芯片包括 8087 协处理器、8259A 中断控制器、8284 时钟发生器/驱动器、8288 总线控制器、74LS373 地址锁存器、74LS245 数据总线驱动器。

8088 的内部结构是 16 位的，而外部数据总线是 8 位的；有 20 位存储器地址，直接寻址 1MB 存储器空间；I/O 端口地址为 16 位，可寻址 64 K 个 I/O 端口；8088 管理的中断源最多可达 256 个，具有软件中断、非屏蔽中断、可屏蔽中断和单步执行等中断方式。

CPU 工作频率为 4.77 MHz，占空比为 1/3 的系统时钟 CLK_{88} 是由 8284 对 14.31818 MHz 频率的晶振三分频得来的。对 CLK_{88} 进行二分频所得的 2.385 MHz 的外围时钟 PCLK 供一些接口和外设使用。

8088 的基本总线周期为 4 个时钟周期。在 T_1 周期，地址/数据复用线 $AD_7 \sim AD_0$ 输出 $A_7 \sim A_0$ ，地址/状态复用线 $A_{19}/S_6 \sim A_{16}/S_3$ 输出地址 $A_{19} \sim A_{16}$ ，地址线 $A_{15} \sim A_8$ 输出 $A_{15} \sim A_8$ ，这 20 位地址经过锁存驱动之后形成系统地址总线 $A_{19} \sim A_0$ 。在 T_2 、 T_3 、 T_4 周期， $A_{19}/S_6 \sim A_{16}/S_3$ 是状态线 $S_6 \sim S_3$ ， $AD_7 \sim AD_0$ 经过双向驱动形成系统数据总线 $D_7 \sim D_0$ 。

8087 插入系统板工作时，8088 在执行数值运算指令时用 ESC 指令交出控制权，用 $\overline{TEST}$ 引脚输入 8087 的 BUSY 引脚状态，测试其运算是否结束。两个处理器通过请求/允许引脚互连，用 3 个一个时钟周期宽度的负脉冲进行联络，使 8087 请求总线、8088 释放总线、8087 占用总线工作后交回总线控制权。

时钟发生器驱动器 8284 产生系统复位信号 RESET 和准备好信号 READY。电源正常后送出 $50\mu\text{s}$ 低电平 PWRGOOD 信号,经 8284 控制与 CLK_{88} 同步输出 RESET,使系统复位;工作时复位只需 RESET 保持 4 个时钟周期高电平进行内部初始化。8284 准备好控制电路分别接收 DMA 控制电路和等待控制逻辑送来的等待请求信号,在内部经时钟同步后从 READY 输出给 8088,当 READY 为无效低电平时,CPU 在操作周期中插入等待状态 T_w ,使较慢的存储器和 I/O 接口能在系统中工作。

8288 总线控制器接收 8088、8087 的总线周期状态信号,在 DMA 控制电路输出的 DMA 信号无效时,8288 控制总线,译码状态信号产生命令和控制输出信号;DMA 信号有效时,8288 停止工作,DMA 控制器接管系统总线控制权。

(2) ROM 子系统

系统板上具有 64 KB 的 ROM 空间,但实际上只安装了 40 KB。地址为 $\text{F6000H} \sim \text{FDFFFH}$ 的 32 KB 的 ROM 用于存储固化的 BASIC 程序代码。地址为 $\text{FE000H} \sim \text{FFFFFH}$ 的 8 KB 的 ROM 存储上电自检程序、BIOS、磁盘引导装入程序及字符点阵图。

(3) RAM 子系统

系统板上具有 $256\text{KB} \times 9$ 的 RAM 容量,占据的地址空间是 $00000\text{H} \sim 3\text{FFFFH}$ 。RAM 存储器是 9 位宽的,第九位是奇偶校验位。奇偶位的内容产生于每个存储器写周期。系统对全部 RAM 进行奇偶校验,每次存储器读周期要根据 8 个数据位重新产生奇偶位信息,并与上次写周期预先存入的奇偶位作比较。若相同,则认为该数据正确;若不同,则产生奇偶校验错信号,该信号向 8088 申请非屏蔽中断使 8088 进入中断服务程序并在显示器上显示出错提示信息。

(4) I/O 接口

系统板上配置了几种可编程的高性能接口芯片:8237DMA 控制器、8253 定时/计数器、8255 并行 I/O 接口。

DMA 控制器 8237A-5 的 4 个 DMA 通道中,3 个通道用在 I/O 通道上,支持无处理器干预的 I/O 设备和存储器之间的高速数据传输;另一个通道被编程用来刷新系统动态存储器,可编制一个定时器/计数器通道程序来使它周期性地产生一个 DMA 请求,这个请求产生一个存储器读周期,用来刷新系统板上和插在系统扩展槽中的动态存储器。所有 DMA 传送除了刷新通道外,在处理器准备好信号有效的条件下使用 5 个处理器时钟周期($1.05\mu\text{s}$)。刷新系统 RAM 的 DMA 周期用 4 个时钟周期(840ns)。8237 的 4 个 DMA 通道中, CH_0 为动态存储器刷新; CH_1 为用户使用保留; CH_2 为软盘适配器; CH_3 为硬盘适配器。

定时器/计数器 8253-5 具有 16 位定时计数能力,用一个独立的 1.19MHz 时钟驱动通道的时钟输入。3 个可编程定时器/计数器通道在系统中的作用如下:通道 0 用作通用定时器,为日期/计时电子钟提供一个恒定的时间基准;通道 1 用于

定时向 DMA 通道请求对动态 RAM 进行刷新,即在 BIOS 中对通道 1 编程,使其每隔 $15.12\mu\text{s}$ 输出一个信号,产生一个 DMA 请求,实现 DMA 传输方式的存储器虚拟读;通道 2 用作驱动扬声器。

可编程并行 I/O 接口芯片 8255A-5 有 3 个 8 位并行 I/O 接口: A 口、B 口、C 口。A 口在上电自检时工作于输出方式,用来输出检测的标志信号;正常工作时它工作于输入方式,用来读取键盘的输入数据。系统板上设置有键盘接口电路,当该电路接受到一个完整的键盘扫描码时产生一个中断请求信号,8088 响应中断请求后转去执行键盘读入程序,通过 A 口读入扫描码完成键入工作。B 口用来输出系统内部的某些控制信号。C 口用来输入系统配置开关 DIP 的设置状态和若干系统内部的状态信号。

(5) I/O 通道

I/O 通道即 I/O 扩充插槽。系统板上有 8 个 62 线的印刷板插座,用于插入不同功能的 I/O 适配器用来扩充 IBM PC/XT 的功能。

2. PC/XT 显示子系统

PC/XT 的显示子系统由彩色图形适配器和彩色监视器组成。CGA 使用 MC6845 CRT 控制器、控制和状态寄存器、16 KB 的显示缓存、128 个 ASCII 字符发生器、图形和字符移位及彩色编码器。

3. PC/XT 软盘子系统

软盘子系统包括软盘控制器和软盘驱动器。软盘控制器电路由 μPD765 控制芯片、数据缓冲器、译码器、控制寄存器、时钟产生电路、写预补偿电路、数据分离电路组成。系统分配给软盘子系统 1 个 DMA 通道、1 个中断请求。

4. PC/XT 硬盘子系统

硬盘子系统包括硬盘控制器和硬盘驱动器。硬盘控制器由处理机接口、智能控制电路和驱动器接口组成。处理机接口分配给硬盘子系统 4 组(选 1)4 个 I/O 口、1 个 DMA 通道和 1 个中断请求。

9.1.2 IBM PC/AT 系统基本组成

1. IBM PC/AT 系统

IBM PC/AT 由 80286 CPU 和通用的外围芯片组组成。主板核心是 80286 CPU,程序指令均在 CPU 中执行。芯片地址线(A)通过锁存和缓冲方式转换成 24 位系统地址总线(SA),芯片数据线(D)通过缓冲方式转换成 16 位系统数据总线(SD)。

系统地址总线与 ROM/IO 地址缓冲器相连,扩展地址总线(XA)由 ROM 存储器和 I/O 外围电路共用。系统数据总线经 RAM/ROM 数据总线转换电路转换成存储器数据总线(MD),MD 是 ROM 和 RAM 共用的数据通路。系统数据总线的低 8 位经 I/O 数据总线转换电路转换成 8 位扩展数据总线(XD),扩展数据总线是所有

I/O 设备的数据通路。芯片总线中的状态线和控制线经 82288 总线控制器译码转换成控制总线，包括存储器和 I/O 读/写控制信号及数据总线控制信号。在这些信号控制下，可以决定数据传送方向。

时钟信号使微处理器和微机内其他设备之间的操作同步。开机加电时，时钟电路产生加电复位信号用于初始化微处理器和外围电路。时钟电路和等待逻辑电路一起用于使 CPU 与外围慢速电路同步。

为了支持高速存储器数据传送和 I/O 数据传送，PC/AT 系统板上配有两个 8237A DMA 控制器。DMA 控制器 1 包括 4 个 DMA 通道：通道 1 用于 SDLC 同步通信，其余 3 个通道作为 ISA 总线与外部设备进行数据传送的 I/O 通道。DMA 控制器 2 包括 DMA 通道 4~7，其中通道 4 用作 DMA 控制器 1 的级联输入，通道 5~7 用于 ISA 总线。

PC/AT 系统中，基于 8254 的定时器电路模块提供与时钟有关的功能与信号。在 8254 PIT 中含有 3 个 16 位定时器，其输入信号为 1.19 MHz 的时钟信号。定时器 0 提供日期时钟中断，作为系统定时器；定时器 1 每隔 $15.12\mu\text{s}$ 为动态 RAM 提供刷新请求信号；定时器 2 为驱动扬声器提供可编程的音频信号。

PC/AT 的并行 I/O 模块含 1 个 8 位输入口和 1 个 4 位输出口。

当读/写存储器奇偶校验(PCK)和 I/O 通道检查(IOCHCK)2 个信号中任一有效时，非屏蔽中断(NMI)逻辑模块在 NMI 信号线上对 80286 微处理器发出一个服务请求，80286 通过并行 I/O 接口读入 PCK 和 IOCHCK 有关状态并依此确定 NMI 请求原因。

PC/AT 系统中另一中断请求信号由中断控制(PIC)模块输出给 80286 的 INTR。系统使用 2 个 8259A 中断控制器，每个 8259A 提供 8 个优先级中断输入，中断控制器的输入信号由时钟、键盘、磁盘控制器、打印机、通信设备等外设提供。8254 定时器 0 中断具有最高优先级。

PC/AT 系统也是通过 I/O 通道(即 ISA 总线)来实现对系统扩展的支持的。系统主板上 8 个扩展槽，其中 6 个既可用于 98 线插卡也可用于 62 线插卡，另 2 个扩展槽只能用于 62 线插卡。

2. IBM PC/AT 相对 PC/XT 的主要改进

(1) 对系统板内几个主要 I/O 控制模块的改进

① 系统配置选择方法的更改。PC/XT 中系统配置的选择是依靠开关来设置的，而 PC/AT 将配置的选择方法改为通过 CMOS 记忆系统来实现，使得配置选择可以通过软件设置来完成，更加灵活。

② 键盘接口部分的改进。PC/XT 使用 8255 芯片接收键盘的输入数据，监测系统主板状态并发出相应控制信号。PC/AT 中则采用一片 8042 单片机来代替 PC/XT 中 8255 的功能。XT 系统中由 8255 和移位寄存器、移位时钟触发器、中

断请求触发器等完成的键盘接口功能由 8042 控制完成,并增加了使用软件对 CPU 进行复位、地址线 A_{20} 选择等功能。

③ 增加了实时时钟。PC/XT 的标准配置中没有实时时钟。PC/AT 中使用了直流电池及 CMOS 可记忆型集成电路芯片,增加了实时时钟计时和定时功能。由一个电池组给 CMOS 芯片供电,停机后仍能保存系统的配置和时间。

(2) 对控制器和外设的改动和扩充

① 对软盘和硬盘控制器的改动。PC/AT 的软盘、硬盘控制器安装在同一块控制卡上。软盘控制器除支持 PC/XT 的 360 KB 驱动器外,还增加了对 1.2 MB 驱动器的支持。硬盘控制器与系统内存之间的数据传输不再使用 DMA 方式而采用中断方式,硬盘驱动程序不再安装在硬盘控制器上而改装在系统主板上,控制器内部数据编码不再使用 NRZ 编码。

② 对串/并口适配器的改动。PC/AT 用 16450 芯片替代 PC/XT 中的 8250 接口处理芯片。

③ 对键盘的改动。键盘内部使用 8042 单片机代替 PC/XT 使用的 8048。

④ 对内存扩充卡的改动。PC/AT 主板的基本内存为 512 KB,需要增加内存时,增加部分被安装在 1 MB 地址区域。

⑤ 使用增强图形适配器 EGA。EGA 除了兼容 CGA 之外,还支持 640×350 的分辨率,显示缓冲区扩展为 128 KB,可选择色彩增加到 64 种。

9.1.3 80386系统基本组成

80386 系统基本组成如图 9.1 所示。图中, a 表示 CPU 局部总线; b 表示系统

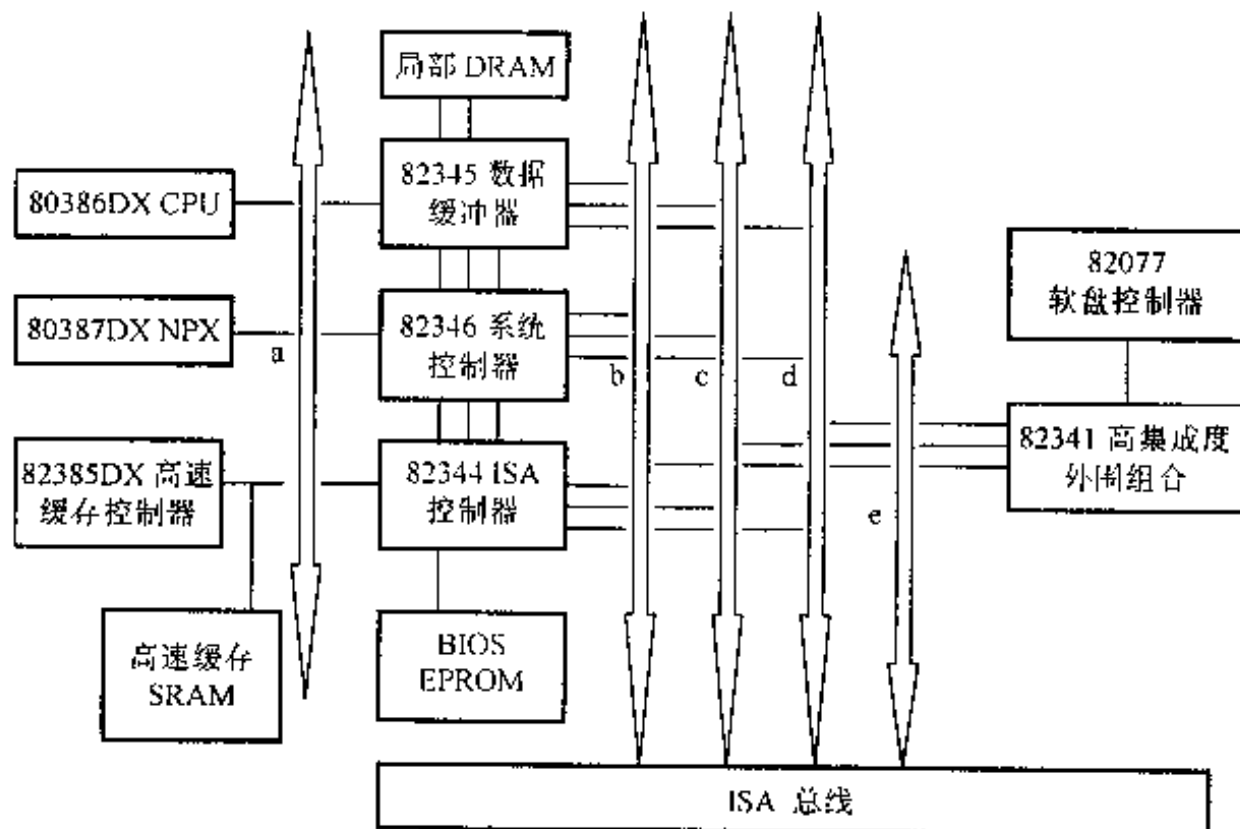


图 9.1 80386 系统的基本组成

控制状态总线；c 表示系统地址总线；d 表示系统数据总线；e 表示外围扩展总线。

1. 82345 数据缓冲器

82345 芯片不仅可对数据总线上的数据进行缓冲，还可进行总线切换，即在 CPU 数据总线 $D_{31} \sim D_0$ ，BIOS ROM/DRAM 数据总线 $MD_{31} \sim MD_0$ ，I/O 数据总线 $XD_7 \sim XD_0$ 及系统数据总线 $SD_{15} \sim SD_0$ 中自动为数据选择合适的去向。82345 芯片还包括奇偶校验码生成电路和奇偶校验电路，该电路仅对存储器数据总线(MD)的数据进行校验。

2. 82346 系统控制器

82346 芯片与 80386DX 局部总线、ISA 总线、82345 数据缓冲器以及 82344 ISA 控制器相连，其功能包括：产生时钟信号，产生 DRAM 阵列的操作地址，为所有主板上的存储器提供控制信号，为 CPU 提供复位信号，提供数字协处理器与 80386DX 的接口。

3. 82344 ISA 控制器

82344 芯片通过 CPU 局部总线与 80386DX 的地址缓冲输入端相连，同时通过 82344/82346 通信接口线与 82346 系统总线控制器相连。82344 有 4 个主要部分：数据转换及等待状态控制电路、82284/82288 准备好信号及总线控制电路、外围控制电路和地址译码器。其主要功能是提供 ISA 总线的大部分信号，在系统使用 BIOS EPROM 芯片及驱动扬声器发声时根据数据宽度选用 8 位或 16 位数据总线。

4. 82341 高集成度外围组合

82341 芯片可完成大量的 PC/AT 所需外设功能，芯片中有两个 16C450 异步串行通信口、一个并行打印机端口、一个实时时钟、一个暂存 RAM、一个键盘/鼠标控制器、一个 IDE 硬盘接口及可编程片选电路。

9.1.4 80386/80486 EISA系统

EISA 总线是一种开放型总线，它与 IBM PC 总线及 ISA 总线兼容，大多 80386/80486 微机采用 EISA 总线。EISA 微机系统采用 82350 芯片组，包括下列 3 种芯片。

(1) 82352 EISA 总线缓冲器

主板上使用两片 82352，分别用于对 CPU 引脚上数据线及地址线缓冲。

(2) 82358 EISA 总线控制器

用于对 CPU 引脚上控制总线的控制。

(3) 82357 集成系统外围电路 (包括 4 种 I/O 外围电路)

① 中断控制器。82357 内部有 2 个与 8259A 功能相当的、连成主从结构的中断控制器。从片的中断请求输出 INT 接到主片的 IRQ_2 输入端，级联成 15 个中断输入的中断系统。EISA 系统的外部中断源分配与 ISA 系统相同，不同之处在于

EISA 系统中 IRQ₁₃ 带两个中断源(原有的协处理器中断和 DMA 控制器连接中断)。

② 定时/计数器。82357 内部有 3 个与 8254 功能相当的定时/计数器。第一个定时/计数器与 ISA 系统中的 8254 兼容。第二个定时/计数器中, 计数器 0 为故障保护定时器, 当该计数器工作时应在确定时间内使其复位, 否则便产生故障保护定时器的 NMI 请求信号; 计数器 1 未使用; 计数器 2 用于控制系统的工作速度, 即该计数器的输出在不降低 CPU 时钟频率的情况下降低系统的工作速度。第三个定时/计数器在系统中未使用。

③ DMA 控制器。82357 内部有 2 个与 8237A 功能相当的、级联成 7 通道的 DMA 控制器, 用来实现高速数据大批量传输。EISA 系统中 DMA 控制器对 8237A 的功能进行了扩充, 可支持 4 种 DMA 周期, 提高了数据的传输速率。

④ NMI 信号产生电路。82357 内部包含了 5 个 NMI 中断源, 其中系统板中存储器奇偶错信号和扩展板 I/O 检验错信号产生的 2 个 NMI 中断源与 PC/AT 系统兼容。EISA 系统新增的 3 个 NMI 中断源为故障保险定时器定时输入 NMI 请求、总线定时输出 NMI 请求和向特定端口地址写任意数据时产生的 NMI 请求。

9.1.5 Pentium微机系统

Pentium 微机系统主要的集成芯片有: Pentium CPU、BIOS ROM 以及与 Pentium CPU 配套的外围芯片组。

外围芯片组包括一片 PCMC 芯片、两片 LBX 芯片和一片 SIO 芯片。

① PCMC 芯片, 其主要功能是管理 PCI 总线、外部 Cache(L2)及内存(DRAM)。PCMC(PCI Cache and Memory Controller)内部集成了外部 Cache 控制器和 Cache 标签存储器, 此外还集成了 DRAM 控制器, 用于控制 DRAM 的刷新和读/写操作等。

② 局部总线加速器芯片 LBX。与 PCMC 一起将 CPU 总线变为 PCI 总线。当 CPU 与内存、CPU 与 PCI 总线、内存与 PCI 总线进行数据读/写操作时, LBX(Local Bus Accelerator)芯片提供了缓冲功能。采用 LBX 芯片可以提高数据传输速度, 实现 DRAM 的 64 位数据操作, 支持 DRAM 的突发式读/写。

③ 系统 I/O 芯片(System I/O)。用来将 PCI 总线变为标准 ISA 总线, 并负责多个 ISA 设备的仲裁。SIO 芯片还集成了诸多 I/O 芯片的功能, 主要包括: 2 个 82C59 中断控制器, 提供 15 级可用中断; 2 个 DMA 控制器, 支持多种 DMA 功能; 1 个 82C54 定时/计数器, 产生系统时钟、内存刷新定时、一定频率的定时脉冲以驱动扬声器发声; 多功能 I/O 芯片, 提供 1 个并口、2 个串口及软盘驱动器接口等。

Pentium 系统采用数据传输率高达 264Mb/s 的同步型总线 PCI 以适应 Windows 环境下对提高图形显示速度的要求, 适应大容量硬盘对高速数据传输的要求。系

统中的 ISA 总线用来支持打印机、声卡及 CD-ROM 等外设的工作。

9.2 例题精析

例 9.1 82385DX 的工作模式是直接映像方式还是两路组相连方式? (见教材图 9.4, p370)

解 $1W/\overline{D}$ 引脚的输入信号为 PUD, PUD 信号由 +5V 经过 1 个 $10k\Omega$ 上拉电阻产生, 因而 $2W/\overline{D}$ 输入为逻辑 1, 82385DX 工作模式为两路组相连方式。

例 9.2 82385DX 高速缓存控制器的片选信号 $\overline{CS}_2$ 和 $\overline{CS}_3$ 会选中高速缓存阵列中的哪一片 SRAM? (见教材图 9.4, p370)

解 82385DX 的输出 $\overline{CS}_2$ 和 $\overline{CS}_3$ 分别与 SRAM U_{17} 的输入引脚 $\overline{CS}_0$ 和 $\overline{CS}_1$ 相连, 所以 $\overline{CS}_2$ 和 $\overline{CS}_3$ 会选中 U_{17} 。

例 9.3 在 82345 数据缓冲器读入数据通路上有哪些多路转换器和锁存器? (见教材图 9.5, p372)

解 读入数据要从 D 总线收发器送到 CPU 数据总线($D_0 \sim D_{31}$)上, 这条输入线路上的多路转换器和锁存器有 D 总线多路转换器、16/32 位锁存器、通道多路转换器和 MD 锁存器等。

例 9.4 通道多路转换器的输入源是什么? (见教材图 9.5, p372)

解 其输入源分别是 XD 总线、SD 总线和 MD 总线。

例 9.5 如果 $\overline{CACHE}$ 信号为逻辑 1, 则 CPU 的 D 总线上数据的输出途径是什么? (见教材图 9.5, p372)

解 CPU 数据总线上的数据可由收发器的 OUT 端口输出, 后写入操作无效时, 高速缓存多路转换器输入 A 端口的数据, 这些数据不必像后写入操作那样要进行数据锁存, 而是直接送到奇偶检验码生成电路的输入端以及 XD 总线多路转换器、SD 总线多路转换器、MD 总线多路转换器的输入端。

例 9.6 82346 芯片 $\overline{ADS}$ 输入信号的功能是什么? (见教材图 9.6, p374)

解 $\overline{ADS}$ 信号为逻辑 0 表示 CPU 接口信号中的地址和控制信号均为有效可用。

例 9.7 82344 芯片的 INTR 信号和 NMI 信号输出到哪里?

解 82344 芯片的 INTR 信号和 NMI 信号直接送入 80386DX CPU 的输入端。

例 9.8 要使 82345 芯片将系统数据总线上的数据从低字节向高字节交换, $\overline{SDLH}/\overline{HL}$ 信号应为哪种逻辑电平?

解 数据从低字节向高字节交换时, $\overline{SDLH}/\overline{HL}$ 信号应为逻辑 1。

例 9.9 打印机中断级是多少?

解 82341 芯片的 IRQ₀ 输出送入 82344 芯片的 IRQ₇ 输入, 因此, 打印机中断为第七级中断。

9.3 学习自评

9.3.1 自测练习

一、填空题

1. IBM PC/XT 系统板大致可分为 (1)、(2)、(3)、(4) 和 (5) 5 个功能块。
2. 80286 微处理器会产生地址、数据、状态和控制信号, 这些信号称为 (1)。其中, 地址线通过 (2) 方式转换成 24 位系统地址总线, 数据线通过 (3) 方式转换成 16 位系统数据总线。
3. IBM PC/AT 中, (1) 是 ROM 和 RAM 存储体共用的数据通路。 (2) 是所有 I/O 设备的数据通路。
4. IBM PC/AT 系统中, DMA 页面寄存器的地址范围是_____。
5. PC/AT 系统板上的大规模集成外围电路主要包括: (1)、(2)、(3)。
6. PC/AT 系统 NMI 中断源为 (1) 和 (2)。
7. 在 IBM PC/AT 系统中, 通过_____实现对系统扩展的支持。
8. 82345 数据缓冲器处于微机的 (1) 总线和 (2) 总线之间。
9. 如果 80386DX 微机系统的工作频率为 25MHz, 那么 TCLK2 输入的时钟频率应该是_____。

二、选择题

1. I/O 设备的数据通路是 ()。
A. MD B. SD C. XD D. 局部数据总线
2. IBM PC/AT 系统中用于 DMA 控制器级联的是 ()。
A. DMA 控制器 1 通道 0 B. DMA 控制器 2 通道 4
C. DMA 控制器 1 通道 3 D. DMA 控制器 2 通道 7
3. ISA 总线可使用的地址空间大小为 ()。
A. 2^8 B. 2^{16} C. 2^{24} D. 2^{32}

三、问答题

1. 传统 PC/AT 机的 3 条系统总线是什么?
2. 时钟产生电路有什么功能?
3. 主中断控制器上哪个中断请求信号的优先级最高?

4. 82345 数据缓冲电路中多路转换器的作用是什么?
5. 82345 芯片的哪些数据总线是 ISA 总线的组成部分?
6. 试述 IBM PC/AT 系统对 IBM PC/XT 的改进? IBM PC/AT 完全兼容 IBM PC/XT 吗?

9.3.2 自测练习解答

一、填空题

1. (1) 处理器子系统 (2) ROM 子系统 (3) RAM 子系统
(4) I/O 接口 (5) I/O 通道
2. (1) 局部总线 (2) 锁存和缓冲 (3) 缓冲
3. (1) 存储器数据总线(MD) (2) 扩展数据总线(XD)
4. 080H~09FH
5. (1) DMA 控制器 8237A (2) 可编程定时/计数器 8254
(3) 可编程中断控制器 8259A
6. (1) 读/写存储器奇偶校验信号 (2) I/O 通道检查信号
7. (1) I/O 通道
8. (1) 局部 CPU (2) ISA 扩展
9. 50MHz

二、选择题

1. C 2. B 3. C

三、问答题

1. 系统地址总线, 系统数据总线, 系统控制总线。
2. 提供系统时钟, 开机复位操作, 产生等待态。
3. 实时时钟。
4. 用于在各数据总线中为数据选择合适的总线进行输出。
5. 扩展数据总线, 系统数据总线。
6. 略。

9.4 教学建议

本章可以在教师指导下让学生自学, 了解 20 世纪末流行的微计算机系统。本章的重点在于微机系统中芯片集的组成以及各部件的功能结构。自学过程中应注意结合教材相应结构框图加以理解, 明确各部分之间的联系、结构特点和功能集成, 以期建立微机系统的完整概念。

综合训练篇

第 10 章 模拟试题及解答

模拟试卷一

一、填空题（每空 1 分，共 15 分）

1. 段寄存器是 (1) 位，描述符寄存器是 (2) 位。

2. 执行以下两条指令

MOV AX,0B715H

ADD AX,0E33BH

后，标志寄存器的 CF=(1)，PF=(2)，AF=(3)，ZF=(4)，OF=(5)，SF=(6)。

3. 80486 工作在保护虚拟模式时，对存储空间分段和分页，每页大小为 (1)，故页目录表基址的低 (2) 位全为 0，其余的高位存放在寄存器 (3) 中。

4. 80486 的中断响应周期包括 个总线周期。

5. EPROM2764 工作于编程方式时，引脚 V_{pp} 输入 (1) 电压， $\overline{OE}$ 为 (2) 电平， $\overline{PGM}$ 为 (3) 电平。

二、选择题（每题 2 分，共 30 分）

1. Intel 公司生产的第一片能支持虚拟存储技术的微处理器是 ()。

A. 8086 B. 80286 C. 80386 D. 80486

2. GDTR、LDTR 和 IDTR 分别是 ()。

A. 32 位、32 位和 32 位 B. 32 位、32 位和 16 位
C. 48 位、16 位和 16 位 D. 48 位、16 位和 48 位

3. Pentium 微处理器是 () 位微处理器。

A. 8 B. 16 C. 32 D. 64

4. 已知 BUF 是数据段定义的字节变量，以下指令正确的是 ()。

A. LEA AX,BUF B. MOV BX,[AX]
C. INC [BX] D. MOV BUF,1000

5. 当发生存储器奇偶校验错时，PC/XT 将产生 ()。

A. 可屏蔽中断请求 B. 不可屏蔽中断请求

- C. 软件请求 D. 异常
6. CPU 响应 INTR 引脚上中断请求的条件是 ()。
A. IF=1 且完成当前指令周期 B. IF=0 且完成当前指令周期
C. IF=1 仅完成第 1 个总线周期 D. IF=0 仅完成第 1 个总线周期
7. 通过 IN 指令访问 I/O 端口时, 源操作数的寻址方式为 () 寻址。
A. 寄存器 B. 寄存器直接 C. 寄存器间接 D. 基址变址
8. 在 8254 的工作方式中, 计数结束输出一个时钟周期负脉冲的有 ()。
A. 方式 0 B. 方式 1 C. 方式 2 D. 方式 3
9. 当对 8255A 端口 C 写入位设置字时, 引脚 A₁ 和 A₀ 的输入电平是 ()。
A. 低电平和低电平 B. 低电平和高电平
C. 高电平和低电平 D. 高电平和高电平
10. RS-232 标准定义的表示数据信号逻辑“1”的电平为 ()。
A. +5 ~ +15 V 间任一电压 B. -15 ~ -5 V 间任一电压
C. +5 V D. 0 V
11. 某系统中有一片 8259A 采用固定优先级, 其 IR₃ 对应的中断类型码为 ABH, 则 IR₂ 对应的中断类型码为 ()。
A. A9H B. AAH C. ACH D. ADH
12. 当 8237 的 DREQ₁ 未输入有效信号, 但希望请求通道 1 执行 DMA 传送时, 可通过置 () 寄存器的通道 1 请求位来完成。
A. 命令 B. 工作方式 C. 请求 D. 屏蔽
13. 在 IBM PC/XT 系统中, 当 8237 处于等待状态时, CPU 处于 ()。
A. 空闲状态 B. 复位状态 C. 等待状态 D. 停机状态
14. CPU 的引脚是 ()。
A. 片总线 B. 内总线 C. 系统总线 D. 外总线
15. IBM PC 系列中, 扩展键盘接口使用的主要芯片是 ()。
A. 8255A B. 8042 C. 8048 D. 8088

三、判断题（每题2分，共10分）

1. 32 位微处理器 LOCK 引脚输出有效低电平一定是由 LOCK 指令前缀设置。 ()
2. 异步通信格式中起始位后接着的一位是字符信息的最高有效位。 ()
3. 在 32 位微处理器系统中, 访问非对准字一定需要两个总线周期。 ()
4. 指令 LGDT 和 LIDT 只能在保护模式下使用, 不能在实模式下使用。 ()
5. SDRAM 工作时与 CPU 的内频时钟同步。 ()

四、名词解释(9分)

1. 微计算机系统。
2. 逻辑地址。
3. 波特率。

五、简答题(6分)

1. 简述光磁软盘的工作原理。
2. 简述微机与外设数据交换方式。

六、综合应用题(30分)

1. 8254 的 CLK_0 输入 1 MHz 的脉冲, $GATE_0$ 接 +5 V 电压, 要求 OUT_0 输出 100 Hz 的方波。设 8254 的端口地址为 40H~43H。

- (1) 计算计数初值, 并编写计数器 0 的初始化程序段;
- (2) 延迟一段时间, 读出当前计数器的计数值, 保存到 BX。

2. 某系统用 8255A 作为打印机的接口, 如图 10.1 所示, 将存放在缓冲区 BUF 的 100 个字节以查询的方式送打印机。设 8255A 的口地址为 0C0H~0C3H, 编写 8255A 的初始化及输出数据的程序段。

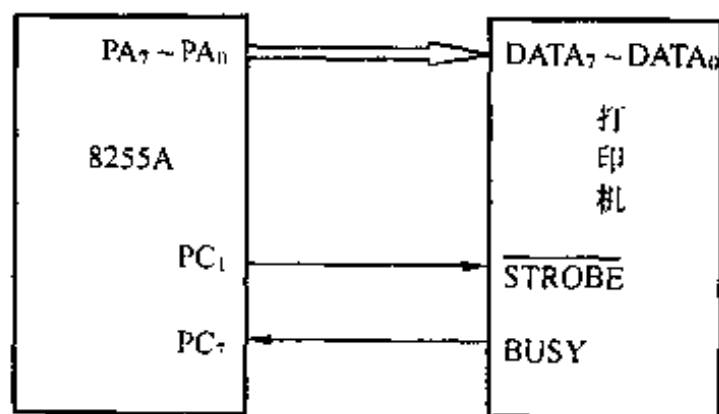


图 10.1

模拟试卷一答案

一、填空题

1. (1) 16 (2) 64
2. (1) 1 (2) 1 (3) 1 (4) 0 (5) 0 (6) 1
3. (1) 4KB (2) 12 (3) CR_3
4. 8
5. (1) +25 V (2) 高 (3) 低

二、选择题

1. B 2. D 3. C 4. A 5. B 6. A 7. C 8. C
9. D 10. B 11. B 12. C 13. D 14. A 15. B

三、判断题

1. 错 2. 错 3. 对 4. 错 5. 错

四、名词解释

1. 微计算机系统是指由微计算机配以相应的外设（如打印机、显示器、磁盘驱动器和磁带机等）和其他专用电路、电源、面板、机架，以及必要的系统软件、支撑软件和应用软件构成的系统。

2. 在实模式下，逻辑地址由 16 位段基址和 16 位偏移地址构成；在保护模式下，逻辑地址由 16 位段选择符和 32 位偏移地址构成。

3. 波特率是指调制解调器将数字信号调制成模拟信号的速率，即单位时间内线路状态变化的数目，以波特为单位。

五、简答题

1. 光磁软盘采用磁记录技术读/写数据、光学技术伺服定位。工作时，由红外发光二极管发射的光束通过读/写磁头块中间的小孔照到伺服道上，从光伺服道反射的光信号再通过磁头块中间的小孔，经过透镜、光监测器转换成电信号控制伺服电机，实现读/写磁头在软盘上的精确定位。

2. 微机与外设的数据交换方式有查询方式、中断方式、DMA 方式和通道方式。

六、综合应用题

1. (1) 计数初值 = $1\text{ MHz} / 100\text{ Hz} = 10\,000$ ，选择方式 3

```
MOV  AL,00110110B
OUT  43H,AL
MOV  AX,10000
OUT  40H,AL
MOV  AL,AH
OUT  40H,AL
```

或者：MOV AL,00100111B

```
OUT  43H,AL
MOV  AL,00H
OUT  40H,AL
```

- (2) MOV AL,0

```
OUT  43H,AL      ; 计数器锁存命令
IN   AL,40H
```

```

MOV    BL,AL
IN      AL,40H
MOV     BH,AL           ; 当前计数值在 BX 中
2.  MOV  AL,10001000B    ; A、B 组方式 0, PC7~PC4 输入, 其余输出
MOV     DX,0C3H
OUT     DX,AL           ; 方式选择控制字
MOV     AL,03           ; C 端口位设置字: 00000011
OUT     DX,AL           ; PC1=1
MOV     BX,0
T:  MOV  DX,0C2H
IN      AL,DX
TEST    AL,10000000B    ; 测试 PC7
JNZ     T
MOV     AL,BUF[BX]
MOV     DX,0C0H
OUT     DX,AL           ; 数据送 A 端口
MOV     AL,02H
MOV     DX,0C3H
OUT     DX,AL           ; PC1=0
INC     AL
OUT     DX,AL           ; PC1=1
INC     BX
CMP     BX,100
JB      T

```

模拟试卷二

一、填空题 (每空 1 分, 共 15 分)

1. Intel 公司生产的第一片集成了浮点运算部件和高速缓存的微处理器是_____。
2. 工作在保护模式下时, 控制寄存器_(1)的_(2)位置_(3)。
3. I/O 端口的编址方式主要有_(1)和_(2)两种, Pentium 系列微处理器构成的微机一般采用后者。
4. 在保护模式下, GDT、LDT 和 IDT 各有_(1)个、_(2)个和_(3)个。

5. 若选用 $64\text{K} \times 1\text{b}$ 的芯片构成 256KB 的 DRAM, 共需 (1) 片, 至少需要 (2) 位地址作为片内地址选择端, (3) 位地址作为芯片选择端。

6. 信号调制技术主要有 (1)、(2) 和 (3) 三种。

二、选择题 (每题 2 分, 共 30 分)

1. 当 80486 工作于实模式下且 $\overline{\text{A20M}}$ 引脚输入高电平时, 可访问的存储空间为 ()。

- A. $0 \sim 0\text{FFFFH}$ B. $0 \sim 0\text{FFFFFFH}$ C. $0 \sim 10\text{FFEFH}$ D. $0 \sim 1\text{FFFFFFH}$

2. 执行以下三条指令

MOV AX, 8032H

SAR AX, 1

RCR AX, 1

后, 标志寄存器的 CF 位和 AX 的内容分别为 ()。

- A. 0, 4019H B. 1, 200CH C. 0, 0C019H D. 1, 600CH

3. 程序员不能直接读/写的 32 位微处理器的寄存器有 ()。

- A. GDTR B. 描述符寄存器 C. TR D. CS

4. 写入 EEPROM 的内容, 可通过 () 擦除。

- A. 紫外线照射 B. 电信号 C. 口令 D. DOS 命令

5. 在 8254 的工作方式中, GATE 输入低电平对计数过程无影响的方式有 ()。

- A. 方式 2 B. 方式 3 C. 方式 4 D. 方式 5

6. 芯片 8255A 中包含一个 8 位数据输入缓冲器和一个 8 位数据输出锁存器/缓冲器的端口是 ()。

- A. A 端口 B. B 端口 C. C 端口 D. 控制器寄存器

7. 8255A 工作于方式 1 输入时, 从端口 C 的 $\text{PC}_7 \sim \text{PC}_3$ 读入的状态是 ()。

- A. I/O I/O IBF_A $\overline{\text{STB}}_A$ INTR_A
 B. I/O I/O IBF_A INTE_A INTR_A
 C. I/O I/O IBF_A $\overline{\text{STB}}_A$ INTE_A
 D. IBF_A INTE_A I/O I/O INTR_A

8. 8250 组成的异步通信适配器工作时逻辑“1”的电平是 ()。

- A. $+5 \sim +15\text{V}$ 间任一电压 B. $-15 \sim -5\text{V}$ 间任一电压
 C. TTL 的“1”电平 D. TTL 的“0”电平

9. 在 IBM PC/XT 系统中, 通过测试 () 寄存器的高 5 位是否全为“0”, 来判断系统中是否存在异步通信适配器。

- A. 中断识别 B. 中断允许 C. Modem 状态 D. 线路状态

10. 某系统中的一片 8259A 设置为循环优先级方式, 则 IR_4 对应的中断服务

程序执行完结束中断时, 中断优先级最高的是 ()。

- A. IR_0 B. IR_4 C. IR_5 D. IR_7

11. 在 IBM PC/XT 系统中, 通过 DMA 方式在硬盘和内存间传送数据, 则最高 4 位的物理地址由端口 () 送出。

- A. 80H B. 81H C. 82H D. 83H

12. 某 8237 采用固定优先级, 屏蔽寄存器内容为 0100B, 当 $DREQ_2$ 和 $DREQ_3$ 同时有效时, CPU 将响应 () 的 DMA 请求。

- A. 通道 3 B. 通道 2 C. 通道 1 D. 通道 0

13. 32 微处理器的虚拟存储空间是 ()。

- A. 46TB B. 32MB C. 64TB D. 4GB

14. IDE 接口和 SCSI 接口的数据线分别为 ()。

- A. 32 根和 16 根 B. 16 根和 32 根
C. 8 根和 16 根 D. 16 根和 8 根

15. 采用 VGA 作为图形适配器, 一屏上最多显示的颜色数为 () 种。

- A. 16 B. 256 C. 1K D. 256K

三、判断题 (每题 2 分, 共 10 分)

1. 在 $IF=1$ 且 $INTR$ 输入高电平时, CPU 立即执行相应的中断服务程序。

()

2. 半导体随机存储器需要定时刷新。

()

3. 在传输率相同的情况下, 同步串行通信的字符信息传输率一定比异步串行通信的高。

()

4. 每一个任务都有一个任务寄存器。

()

5. 若 8259A 被设置为 AEOI 方式, 当对某个中断请求对应的中断响应周期结束时, 其 ISR 中相应位为 0。

()

四、简答题 (15 分)

1. 试述 Pentium 4 内核架构的特点。

2. 何谓非对准双字? 试述 80486 访问对准双字和非对准双字时 $\overline{BE}_3 \sim \overline{BE}_0$ 的输出情况。

3. 简述 USB 接口的主要性能。

五、综合应用题 (30 分)

在某个 16 位系统中有一片采用边沿触发、非缓冲方式、非自动结束中断方式、循环优先级工作的 8259A 和一片用来接收速率为 3600Baud 的数据的 8250, 其中 8250 的 $INTRPT$ 引脚和 8259A 的 IR_6 相连。8250 接收到的数据格式为: 8 位数据, 1 位停止位, 偶校验, 每当它接收到数据就产生中断类型码为 36H 的中断, 将数据保存到字节变量 BUF 处。设 8259A 的口地址为 20H 和 21H, 8250 的口地址为

2F8H~2FFH。

- (1) 初始化 8259A 和 8250;
- (2) 编写 36H 的中断服务程序。

模拟试卷二答案

一、填空题

1. (1) 80486
2. (1) CR₀ (2) 保护模式允许 PE (3) 1
3. (1) 统一编址 (2) 独立编址
4. (1) 1 (2) 多 (3) 1
5. (1) 32 (2) 16 (3) 2
6. (1) 调幅 (2) 调频 (3) 调相

二、选择题

- | | | | | | | | |
|------|-------|-------|-------|-------|-------|-------|------|
| 1. C | 2. D | 3. B | 4. B | 5. D | 6. C | 7. B | 8. C |
| 9. A | 10. C | 11. C | 12. B | 13. C | 14. D | 15. B | |

三、判断题

1. 错 2. 错 3. 错 4. 错 5. 对

四、简答题

1. Pentium IV 内核架构包括一系列新技术: 超级流水线, 执行跟踪缓存, 高级动态执行引擎, 快速执行引擎, SSE2 指令集, 400MHz 系统总线速度。

2. 如果双字的起始物理地址不是 4 的倍数, 则称为非对准双字。当 80486 访问对准双字时, $\overline{BE}_3 \sim \overline{BE}_0$ 同时输出低电平。当 80486 访问非对准双字时, 若该双字物理地址最低 2 位为 01B, 则第一个总线周期 $\overline{BE}_0$ 输出低电平, 第二个总线周期 $\overline{BE}_3 \sim \overline{BE}_1$ 输出低电平; 若该双字物理地址最低 2 位为 10B, 则第一个总线周期 $\overline{BE}_1 \sim \overline{BE}_0$ 输出低电平, 第二个总线周期 $\overline{BE}_3 \sim \overline{BE}_2$ 输出低电平; 若该双字物理地址最低 2 位为 11B, 则第一个总线周期 $\overline{BE}_2 \sim \overline{BE}_0$ 输出低电平, 第二个总线周期 $\overline{BE}_3$ 输出低电平。

3. 具有即插即用特性, 为 USB 接口设计的驱动程序和应用程序可自动启动, 成本低, 节省空间, 为开放性的不具专利版权的理想的工业标准。

五、综合应用题

- (1) MOV AL,00010011B : 初始化 8259A, ICW₁
 OUT 20H,AL
 MOV AL,30H : ICW₂
 OUT 21H,AL

```

MOV    AL,1                ; ICW4
OUT    21H,AL
MOV    AL,10100000B        ; OCW2 设置 EDI 时优先级循环
OUT    20H,AL
MOV    AL,80H              ; 初始化 8250
MOV    DX,2FB1H            ; LCR 地址
OUT    DX,AL
MOV    AL,0                ; 除数锁存器高位
MOV    DX,2F9H
OUT    DX,AL
MOV    AL,20H              ; 除数锁存器低位
MOV    DX,2F8H
OUT    DX,AL
MOV    AL,00011011B        ; 设置数据格式
MOV    DX,2FBH
OUT    DX,AL
MOV    AL,3
MOV    DX,2FCH            ; MCR 地址
OUT    DX,AL
MOV    AL,1                ; 允许接收数据就绪中断
MOV    DX,2F9H            ; IER 地址
OUT    DX,AL
(2) I36H PROC FAR
    CLI
    MOV    DX,2FDH
    IN     AL,DX           ; 读 LSR
    TEST   AL,1EH
    JNZ    EXIT
    MOV    DX,2F8H        ; RBR 地址
    IN     AL,DX
    MOV    BUF,AL
EXIT: MOV    AL,00100000B   ; OCW2, EOI 命令
    OUT    201H,AL
    STI
    IRET
I36H ENDP

```

第 11 章 综合题及解答

综合题及解答一

1. 对 8255A 的控制寄存器依次写入 A0H、01H、02H、05H、08H、0BH、0DH 后, 端口 C 的值为 ()。(x 表示无法确定)

- A. 10101101B B. 1x101101B C. 1x10x101B D. x110x101B

解 答案为 D。送 8255A 控制寄存器的控制字有方式控制字 ($D_7=1$) 和口 C 位设置字 ($D_7=0$) 两种。

A0H 是方式控制字, 将 A 组设置为方式 1 输出, B 组设置为方式 0 输出, 此时 PC_7 、 PC_6 、 PC_3 分别为联络控制信号 $\overline{OBF_A}$ 、 $\overline{ACK_A}$ 、 $INTR_A$;

01H 是 C 端口位设置字, 使 $PC_0=1$;

02H 是 C 端口位设置字, 使 $PC_1=0$;

05H 是 C 端口位设置字, 使 $PC_2=1$;

08H 是 C 端口位设置字, 使 $PC_4=0$;

0BH 是 C 端口位设置字, 使 $PC_5=1$;

0DH 是 C 端口位设置字, 在方式 1 输出时, 对 PC_6 输入无影响, 而是使 $INTE_A=1$ 。

综上所述, 可知答案为 D。

2. 当 CPU 工作于实模式下且内存的部分内容如图 11.1 所示时, 产生除法出错中断, 则该中断对应的中断服务程序起始地址是多少?

解 中断服务程序的起始地址存放在中断向量表中。在实模式下, 中断向量表存放在存储器的最低端, 共占 1024B, 每 4B 存放一个中断服务程序的起始地址, 较高地址的 2B 存放起始地址的段基址, 较低地址的 2B 存放起始地址的偏移地址。中断服务程序的起始地址按中断向量号从小到大的顺序依次存放, 故中断向量号乘以 4 就得到对应中断服务程序起始地址存放处的地址。

物理地址	存储单元内容
00000H	00H
00001H	00H
00002H	A0H
00003H	01H
00004H	00H
00005H	01H
00006H	21H
00007H	10H

图 11.1

除法出错中断的中断向量号为 0, $0 \times 4 = 0$, $IP = (00000H) = 0000H$, $CS = (00002H) = 01A0H$, 物理地址 $= CS \times 10H + IP = 01A00H + 0000H = 01A00H$, 所以除法出错中断服务程序起始地址为 01A00H。

3. 当 80486 工作于实模式下, $\overline{A20M}$ 输入高电平时, 能访问的存储空间为多大?

解 当 80486 工作于实模式下, 如果 $\overline{A20M}$ 输入有效低电平, 则屏蔽 A_{20} 及以上地址线, 仅使用地址线 $A_{19} \sim A_0$, 仿真 8086 的 1MB 存储空间。

在实模式下, 最大段基址为 FFFFH, 最大偏移地址为 FFFFH, 故所能表示的最大地址为 FFFF0H + FFFFH = 10FFEFH, 即实模式下能访问的最大地址空间比 1MB 多。当 $\overline{A20M}$ 输入高电平时, 则允许使用地址线 $A_{20} \sim A_0$, 可访问 1MB 以上的高端内存 HMA, 能访问的存储空间为 0~10FFEFH, 即 10FFF0H 字节。

4. 在以 PC/XT 机为主机的某系统中扩展一块 8254, 其地址为 304H~307H, 要求从 OUT_0 输出频率为 1000Hz 的方波, 从 OUT_1 输出频率为 100Hz 的方波, 从 OUT_2 输出频率为 1Hz 的方波。 CLK_0 的时钟是 4.77MHz, 请画出 8254 的 $GATE_0 \sim GATE_2$ 及 $CLK_1 \sim CLK_2$ 的接线图。计算各计数器的计数初值, 并编制初始化程序段。

解 要求从 OUT_0 、 OUT_1 、 OUT_2 输出方波, 选择方式 3, 计数初值 = CLK 时钟频率 / OUT 输出频率。因为计数器是 16 位, 所以计数初值小于或等于 65535 ($2^{16} - 1$)

给定的时钟脉冲是 4.77MHz, 如果将此脉冲送 CLK_2 , 则计数器 2 的计数初值 $= 4.77MHz / 1Hz = 4770000 > 65535$, 无法完成指定分频, 故要寻找其他时钟脉冲源作为 CLK_2 的输入。从题目中可看到, OUT_0 和 OUT_1 的输出是方波, 均可作为时钟脉冲, 所以当分频次数大于 65535 时, 可用多个计数器完成分频要求。该题的答案不唯一, 不同的连线方式有不同的初始化程序段, 现按图 11.2 所示的连线编程如下。

计数器 0 初值 $= 4.77MHz / 1000Hz = 4770$

计数器 1 初值 $= 1000Hz / 100Hz = 10$

计数器 2 初值 $= 100Hz / 1Hz = 100$

MOV DX, 307H ; 计数器 0

MOV AL, 00110110B

OUT DX, AL

MOV DX, 304H

MOV AX, 4770

OUT DX, AL

MOV AL, AH

OUT DX, AL

MOV DX, 307H

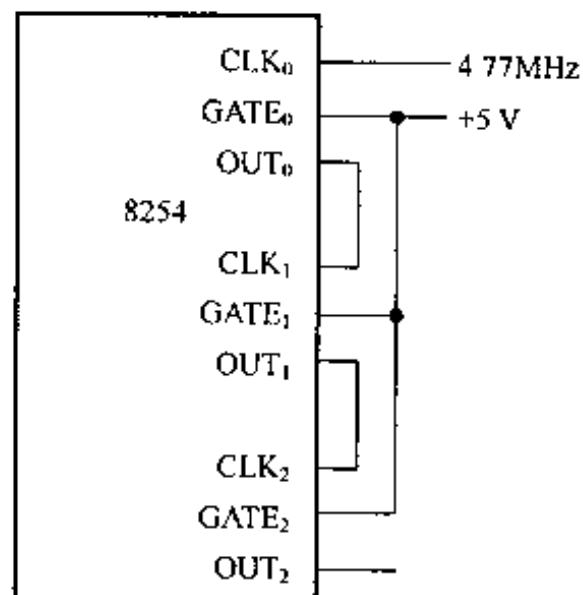


图 11.2

```

MOV    AL,01010110B      ; 计数器 1
OUT     DX,AL
MOV     DX,305H
MOV     AL,10
OUT     DX,AL
MOV     DX,307H
MOV     AL,10010110B      ; 计数器 2
OUT     DX,AL
MOV     DX,306H
MOV     AL,100
OUT     DX,AL

```

5. 某个系统中, 由主 8259、从 8259、第三级 8259 构成可屏蔽中断控制系统, 它们的 I/O 地址分别是 20H~21H、C0H~C1H、E0H~E1H。主 8259 的 INT 连至 8086 的 INTR, 从 8259 的 INT 连至主 8259 的 IR₄, 第三级 8259 的 INT 连至从 8259 的 IR₀。所有 8259 都用边沿触发、非缓冲方式、程序结束中断、固定优先级, 主 8259 的中断类型码为 08H~0FH, 从 8259 的中断类型码为 A0H~A7H。

(1) 编写初始化程序段;

(2) 编写从 8259 上 IR₀ 的中断服务程序。

解 当多个 8259 级联时, CAS₂~CAS₀ 为级联信号线。一个 8259 作为主片, 其 CAS₂~CAS₀ 在中断响应期间输出从 8259 的标志。对于从 8259, 其 CAS₂~CAS₀ 在中断响应期间接收主片送出的标志, 因此仅通过硬件的管理只能实现两级 8259 级联。本题中需要实现三级 8259 级联, 所以第三级 8259 只有通过软件的方式来管理。8259 提供查询命令, 使用该命令可知当前请求服务的最高中断优先级代码, 这样不需要执行中断响应周期, 就能通过软件方式去调用对应的中断服务程序。

```

(1)  MOV    AL,11H          ; 主 8259 初始化: 00010001
      OUT     20H,AL
      MOV     AL,08H         ; 主 8259 的中断类型码
      OUT     21H,AL
      MOV     AL,10H         ; ICW3: 00010000
      OUT     21H,AL
      MOV     AL,11H         ; ICW4: 00010001
      OUT     21H,AL
      OUT     0C0H,AL        ; 从 8259 初始化
      MOV     AL,0A0H        ; ICW2: 从 8259 中断类型码

```

```

OUT    0C1H,AL
MOV     AL,4                      ; ICW3: 00000100
OUT     0C1H,AL
MOV     AL,01H                   ; ICW4: 00000001
OUT     0C1H,AL
MOV     AL,13H                   ; 第三级 8259 初始化: ICW1, 作单片处理
OUT     0E0H,AL
OUT     0E1H,AL                  ; 通过软件管理, 中断类型码的设置无关紧要
MOV     AL,01H                   ; ICW4, 非缓冲, EOI, 8086 系列
OUT     0E1H,AL

```

```

(2) IO PROC FAR
      MOV     AL,0CH              ; 对第三级 8259 的查询命令: OCW3
      OUT     0E0H,AL
      IN      AL,0E0H             ; 获得当前请求服务的最高中断优先级编码
      TEST    AL,80H
      JZ      EXIT
      AND     AL,00000111B
      ADD     AL,ICODE            ; 设 ICODE 为第三级 8259IR0 的中断类型码
      MOV     BL,4
      MUL     BL                  ; AX 为中断向量地址
      MOV     BX,AX
      MOV     DS,0
      CALL    DWORD PTR [BX]     ; 调用中断服务程序
      MOV     AL,20H              ; OCW2
      OUT     0E0H,AL            ; 清第三级 8259 最高优先级的 ISR 位
      MOV     AL,0BH              ; OCW3
      OUT     0E0H,AL            ; 读第三级 8259 的 ISR 命令
      IN      AL,0E0H             ; 获得第三级 8259 ISR 的内容
      CMP     AL,0
      JNZ     EXIT
      MOV     AL,20H
      OUT     0C0H,AL            ; 清从 8259 最高优先级的 ISR 位
EXIT: IRET
IO ENDP

```

6. 某系统中用 8255A 作为一个 3×4 的矩阵键盘与 CPU 的接口, 硬件连接

如图 11.3 所示。8255A 采用行扫描法完成按键的识别，试编写该程序段。设 8255A 的地址为 40H~43H。

解 行扫描法的工作原理如下。

(1) 判断是否有键按下。使所有行线为 0，然后检查列线是否全为 1，若不是，表明有键按下，使行线和列线接通。

(2) 去抖动。

(3) 识别按键。从第 0 行开始逐行扫描，每扫描一行，令该行对应的行线为 0。其余行线为 1，然后读入列线状态，检查是否有列线为 0。若无，则行号加 1，继续扫描下一行，否则，查出状态为 0 的列号。

(4) 产生键码。根据扫描得到的行号和列号，查找键盘编码表，获得按键对应的编码。

由以上所述工作原理可知，本题中 A 端口设置为输出，B 端口设置为输入，A 组和 B 组选择方式 0 即可。

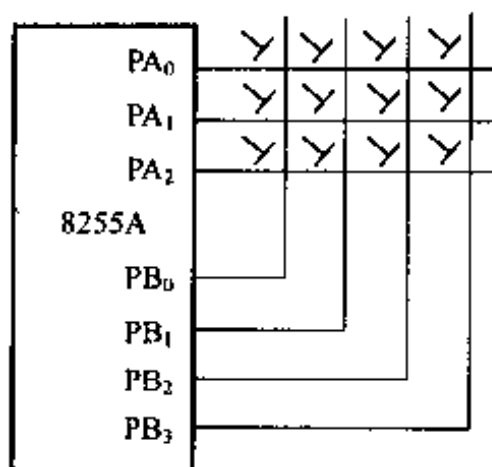


图 11.3

```

MOV    AL,82H           ; 设置 8255A 工作方式
OUT     43H,AL
BEGIN: MOV    AL,0       ; 判断是否有键按下
OUT     40H,AL
WAIT:  IN     AL,41H
AND     AL,0FH          ; 屏蔽高 4 位
CMP     AL,0FH
JZ      WAIT
MOV     CX,7FFFH        ; 延时去抖动
L:     LOOP   L
MOV     BL,3            ; 识别按键
MOV     BH,0            ; 扫描行号
MOV     AL,0FEH
MOV     CH,0FFH         ; 置列号初值
LI:    OUT     40H,AL
ROL     AL,1
MOV     AH,AL
IN      AL,41H
AND     AL,0FH
CMP     AL,0FH

```

```

JNZ    L2
INC     BH
MOV     AL,AH
DEC     BL
JNZ     L1
JMP     BEGIN
L2:     INC     CH
        RCR     AL,1
        JC      L2
        CALL    KEYTABLE      : 根据存放在 BH 和 CH 中的行号和列号查找键盘
                                : 编码表获得键码

```

7. 某系统用 8254 来控制数据采样频率和采样持续时间, 用 8255A 获得采样数据并输出中断请求 IRQ_2 , 在中断服务程序 $INT0AH$ 中把数据存入内存缓冲区, 硬件连接如图 11.4 所示。当 A/D 转换器的 CI 输入下降沿, 启动一次 A/D 转换, 转换完成后, CO 输出一负脉冲。时钟为 1MHz, 8254 的计数器 0、计数器 1、计数器 2 的工作方式分别为方式 2、方式 1、方式 3, 初值分别为 4000、1000、2000。

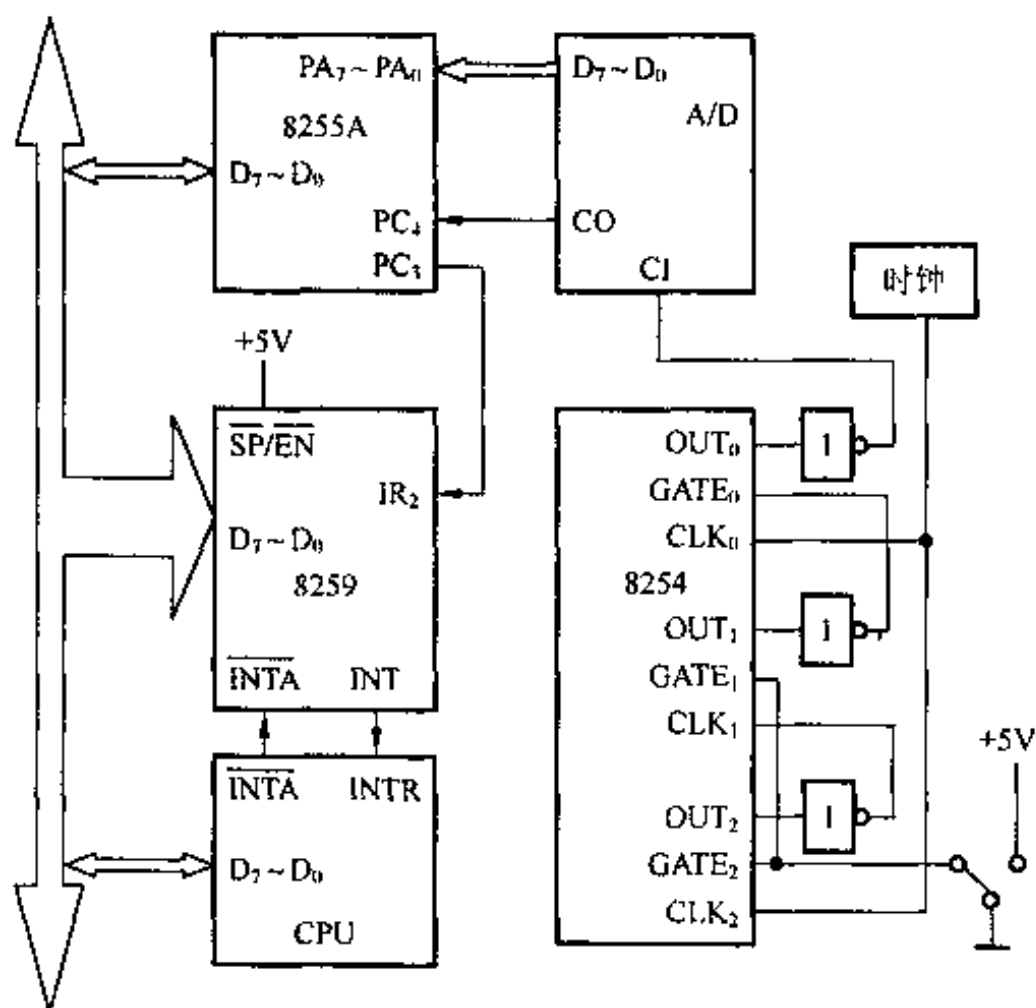


图 11.4

(1) 试分析 A/D 转换器的采样频率和采样持续时间。

(2) 编写 8254、8255A、8259 (边沿触发、固定优先级、程序结束中断) 的初始化程序段。设 8254、8255A、8259 的起始地址分别是 A0H、B0H、C0H。

(3) 编写中断服务程序 INT 0AH。

解 (1) 8254 计数器 0 工作于方式 2, 其 OUT_0 输出负脉冲经反相后, 送 A/D 转换器的 CI 启动 A/D 转换, 控制数据采样频率。计数器 0 初值为 4000, CLK_0 的时钟脉冲为 1MHz, 所以采样频率 $= 1\text{MHz}/4000 = 250\text{Hz}$ 。

OUT_0 的输出受 $GATE_0$ 的控制, 即能从 OUT_0 输出脉冲的必要条件是 $GATE_0$ 为高电平, 故 $GATE_0$ 高电平的持续时间, 就是数据采样的持续时间。而 $GATE_0$ 的输入为计数器 1 的 OUT_1 的输出反相。计数器 1 工作于方式 1, 初值为 1000, 因此, 当 $GATE_1$ 从 “0” 变为 “1” 后, 触发 OUT_1 输出宽度为 $1000/CLK_1$ 的脉冲, 经反相后送给 $GATE_0$ 为正脉冲, 因此, 采样的时间是 OUT_1 输出单脉冲的宽度。而 CLK_1 的输入来自于 OUT_2 的输出反相。计数器 2 工作于方式 3, 初值为 2000, CLK_2 的时钟脉冲为 1MHz, 当 $GATE_2$ 输入高电平时, OUT_2 输出的方波频率 $= 1\text{MHz}/2000 = 500\text{Hz}$, 所以采样的持续时间 $= 1000/500\text{s} = 2\text{s}$ 。

(2) 当 A/D 转换完成时, 要求将采样数据送 8255A 的 A 端口, 同时送负脉冲到 PC_4 , 并使 PC_3 送中断请求。8255A 的 A 组设置为方式 1 输入可满足该要求, 此时 PC_4 是 $\overline{STB_A}$, 当它输入负脉冲时, 将 $PA_7 \sim PA_0$ 所存入 8255A 的输入数据锁存, 若 $INTE_A = 1$, 则将使 PC_3 发出 $INTR_A$ 信号。

CLI

```
MOV    AL,00110100B           ; 8254 初始化
OUT     0A3H,AL                ; 计数器 0, 方式 2
MOV     AX,4000
OUT     0A0H,AL
MOV     AL,AH
OUT     0A0H,AL
MOV     AL,01110010B           ; 计数器 1, 方式 1
OUT     0A3H,AL
MOV     AX,1000
OUT     0A1H,AL
MOV     AL,AH
OUT     0A1H,AL
MOV     AL,10110110B           ; 计数器 2, 方式 3
OUT     0A3H,AL
MOV     AX,2000
```

```

OUT    0A2H,AL
MOV    AL,AH
OUT    0A2H,AL
MOV    AL,10110000B      ; 8255A 初始化
OUT    0B3H,AL           ; A 口方式 1 输入
MOV    AL,09H            ; PC4=1, 使 INTEA=1
OUT    0B3H,AL
MOV    AL,13H            ; 8259A 初始化
OUT    0C0H,AL           ; ICW1: 边沿触发, 单片, IC4
MOV    AL,8
OUT    0C1H,AL           ; ICW2: 中断类型码 08H~0FH
MOV    AL,1
OUT    0C1H,AL           ; ICW4: 非缓冲, EOI, 8086 系列
STI

```

(3) IA PROC FAR

```

IN      AL,0B0H
MOV     [SI],AL          ; SI 指向内存缓冲区
MOV     AL,00100000B     ; 中断结束命令
OUT     0C0H,AL
INC     SI
IRET

```

IA ENDP

8. 在某 16 位微处理器系统中, 利用两片 8255A 的 A 端口作为一个 16 位的并行接口, 将内存缓冲区 BUF 的 256 个字送到一个 16 位的输出设备。8255A 送一低电平给输出设备, 通知它取数据, 当输出设备获得数据后将送回一个负脉冲作为应答信号。

(1) 试设计 8255A 与系统总线的硬件连接。设端口地址为 0H~7H。

(2) 编写利用查询方式完成数据传送的程序段。

解 (1) 16 位系统对存储器的管理是通过将其分成奇存储体和偶存储体两部分, 奇存储体的数据线连系统数据总线的高 8 位, 偶存储体的数据线连系统数据总线的低 8 位, 然后用 A_0 控制偶存储体的片选, $\overline{BHE}$ 控制奇存储体的片选。对 16 位外设接口的管理也可通过类似的方式进行, 一片 8255A 分配偶地址 0H、2H、4H 和 6H, 其数据线连系统数据总线的低 8 位, 用 A_0 控制其片选。另一片 8255A 分配奇地址 1H、3H、5H 和 7H, 其数据线连系统数据总线的高 8 位, 用 $\overline{BHE}$ 控制其片选, 硬件连接设计如图 11.5 所示。

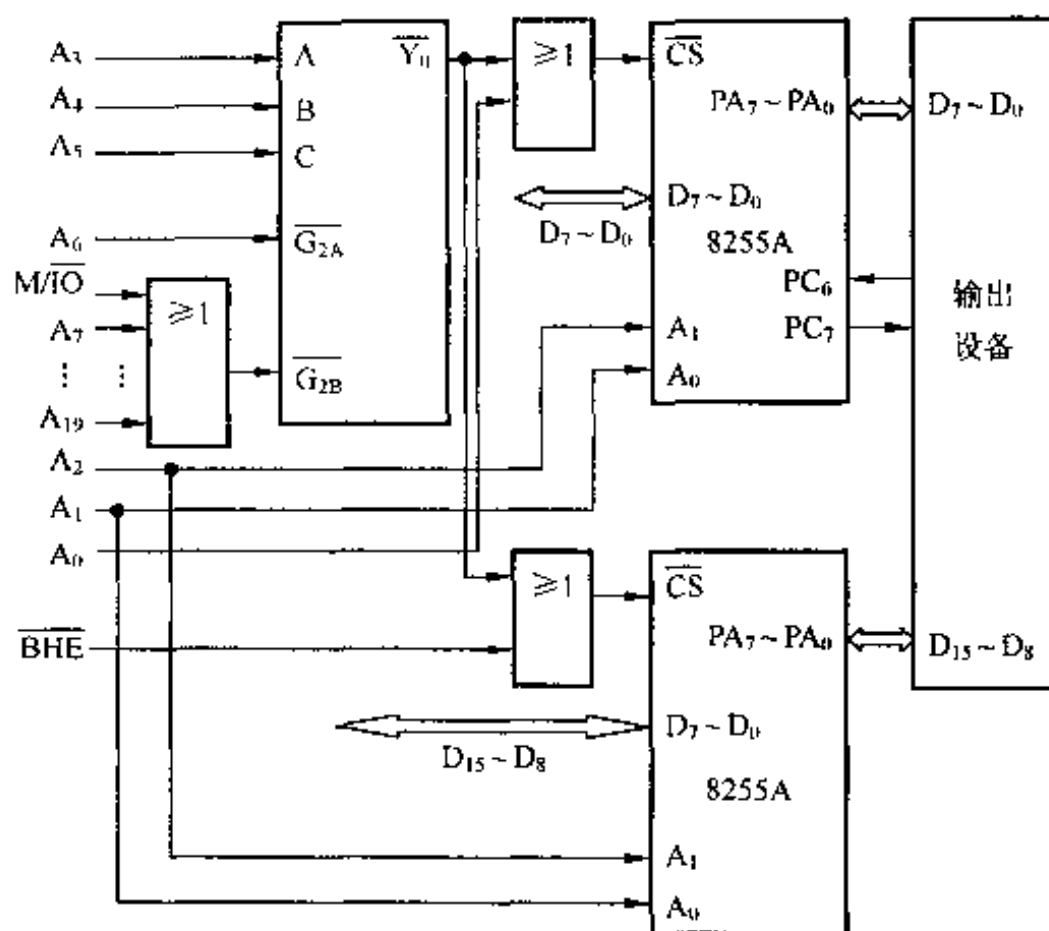


图 11.5

(2) 在 8255A 与输出设备传输过程中, 可直接利用 A 组方式 1 输出模式时定义的联络信号, 当 A 端口满时 $\overline{PC_7}$ ($\overline{OBF}$) 输出低电平, PC_6 获得响应信号 $\overline{ACK}$ 。该题中有两片 8255A, 我们可以任选一片的 C 端口作为联络控制信号, 比如说偶地址的 8255A。

```

MOV     AL,10100000B           ; 偶地址 8255A 初始化: A 端口方式 1 输出
OUT     6H,AL
MOV     AL,10000000B           ; 奇地址 8255A 初始化: A 端口方式 0 输出
OUT     7H,AL
MOV     CX,256
MOV     BX,0
L:      MOV     AX,BUF[BX]
OUT     0H,AX                  ; 传输一个字从端口经 00H、01H 输出
ADD     BX,2
L0:     IN      AL,4H
TEST    AL,10000000B
JZ      L0
LOOP    L

```

综合题及解答二

1. CPU 工作于保护虚拟模式下, 存储管理不分页, $GDTR=00001000FFFFH$, $IDTR=000020000800H$, 内存的部分内容如图 11.6 所示, 当发生非屏蔽中断时, 对应中断服务程序的起始地址为多少?

解 在保护虚拟模式下, 中断服务程序的中断描述符, 包括程序的逻辑起始地址按中断向量号从小到大的顺序依次存放在中断描述符表 IDT 中, IDT 可放在内存的任何地方, 其基址和限长则存放在 CPU 内部的中断描述符寄存器 IDTR 中。IDTR 内容以及中断描述符格式如图 11.7 所示。

IDTR 的高 32 位为中断描述符表的基地址, 故本例中中断描述符表的基地址为 $00002000H$ 。

物理地址	存储单元内容
00001018H	4000FFFFH
0000101CH	01C09800H
	⋮
00002010H	00180000H
00002014H	00008E00H
	⋮
00015010H	00030021H
	⋮
00030010H	001B0021H

图 11.6

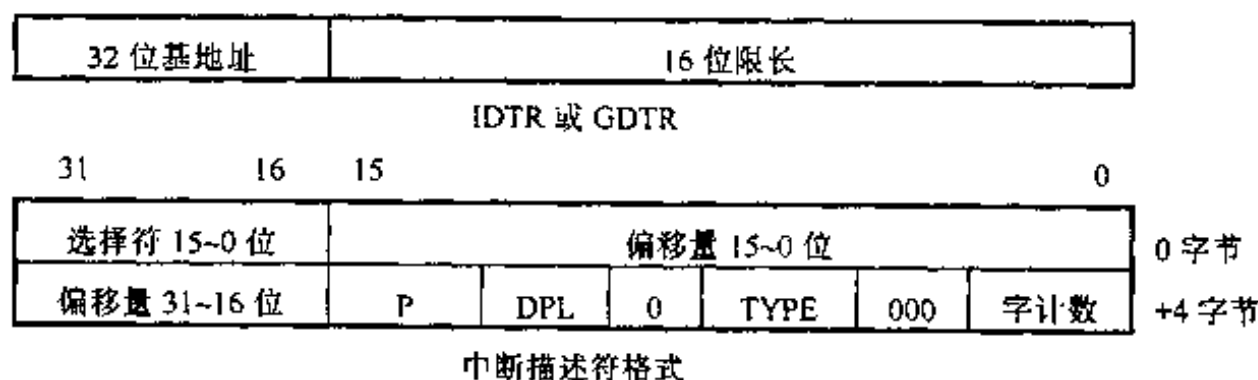


图 11.7

中断描述符占 8 个字节, 非屏蔽中断的中断向量号为 2, 故

对应的中断描述符的地址 = 中断描述符表基址 + 中断向量号 $\times 8$

$$= 00002000H + 2 \times 8 = 00002010H$$

所以, 对应的中断描述符为 $00008E0000180000H$, 据此可知对应中断服务程序的选择符为 $0018H$, 偏移量为 $00000000H$ 。因 $P=1$, 故该描述符有效, 为 $DPL=00$ 的 32 位中断门。

选择符的格式如图 11.8 所示, $TI=0$ 表示描述符在 GDT 中, $TI=1$ 表示描述符在 LDT 中, 所以选择符 $0018H$ 对应的段描述符在 GDT 中第 3 项。

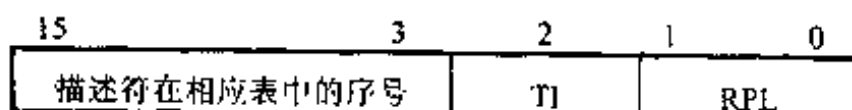


图 11.8

GDT 的基地址存放在 GDTR 中, 每个段描述符占 8 个字节, 故

$$\begin{aligned}\text{对应的段描述符的地址} &= \text{GDTR 中基地址} + \text{序号} \times 8 = 00001000\text{H} + 3 \times 8 \\ &= 00001018\text{H}\end{aligned}$$

所以对应的段描述符为 01C098004000FFFFH, 其中存放的中断服务程序段基址为 01004000H, 则

$$\text{线性地址} = \text{段基址} + \text{偏移量} = 01004000\text{H} + 00000000\text{H} = 01004000\text{H}$$

因为存储管理不分页, 所以线性地址即为物理地址。非屏蔽中断对应的中断服务程序起始地址为 01004000H。

2. CPU 工作于保护虚拟模式下, 存储管理分页进行, GDTR=00001000FFFFH, IDTR=000020000800H, CR₃=00015000H, 内存的部分内容如图 11.6 所示, 当发生非屏蔽中断时, 对应的中断服务程序的起始地址为多少?

解 页目录表基址的低 12 位全为 0, 高 12 位保存在 CR₃ 的高 20 位中, 所以页目录表基址为 00015000H。

由第 1 题可知, 非屏蔽中断服务程序的线性地址为 01004000H, 则页目录号为 4, 页号为 4, 页内偏移量为 000H。

页目录项占 4 个字节, 故

对应的页目录项基址 = 页目录表基址 + 页目录号 × 4 = 00015000H + 4 × 4 = 00015010H
从此地址取出的页目录项为 00030021H, 属性 P=1, 则其高 20 位为页表基址的高 20 位, 页表基址的低 12 位为全 0, 所以页表基址为 00030000H。

页表项占 4 个字节, 故

对应的页表项基址 = 页表基址 + 页号 × 4 = 00030000H + 4 × 4 = 00030010H
从此地址取出的页表项为 001B0021H, 属性 P=1, 则其高 20 位为页基址高 20 位, 页基址低 12 位为全 0, 所以

$$\text{非屏蔽中断服务程序的起始地址} = \text{页基址} + \text{页内偏移量} = 001B0000\text{H}$$

3. 在 PC 系列微机系统中键盘如何复位?

解 (1) PC 标准键盘接口电路通过一个五芯插座与键盘的五芯插头相连, 其中五芯插头的 3 号脚为复位信号输入引脚。在 IBM PC/XT 系统中, 硬件复位信号并未接键盘接口一侧的五芯插座的 3 号脚, 表明系统未用系统硬件复位信号去复位键盘, 系统在上电自检期间采用复位命令软件复位键盘, 步骤如下。

① 设置 PB₆=0, 强制键盘时钟线为低电平, 并保持 20ms 之后又允许键盘工作, 且仅开放键盘中断。

② 若在规定时间内未发生键盘中断, 则表明键盘未回送应答数据, 转错误处理, 否则表明键盘控制器内的单片机 8048 已经判断出主机发送的是软件复位命令, 向主机返回应答数据。

③ 主机读取应答数据, 若此扫描码为 “AAH”, 则说明键盘复位成功, 否则

转错误处理, 显示键盘故障代码。

(2) PC 扩展键盘接口电路选用单片机 (如 PC/AT 选用 Intel 8024) 为核心电路实现键盘接口全部功能, 接口将键盘扫描码转换为与标准键盘相同的系统扫描码, 实现键盘与 CPU 之间的双向数据传送。键盘复位是 CPU 向 8042 输入缓冲器端口 60H 写一字节命令 FFH 后 20ms 内得到键响应信息完成的。系统通过此软件复位命令使键盘进入程序复位和内部自测试, 称为基本保证测试 (BAT), 过程如下。

① 键盘收到复位命令 FFH 后立即回送 ACK (FAH) 响应。

② 键盘接口收到 ACK 后, 使键盘时钟线和数据线置为高电平。

③ 键盘检测到此状态后, 开始 BAT 操作。

④ 若 BAT 正确完成, 则键盘发送 AAH 表示复位结束, 否则以 FDH (或任何其他值) 表示诊断有误。

4. PC 系列微机系统中, 在进行 DMA 传送时, 如何寻址存储单元和外设?

解 PC/XT 中的 DMA 系统采用单片 8237 和一片页面寄存器。在初始化阶段, 将存储单元的低 16 位物理地址送到 8237 的当前地址寄存器, 高 4 位物理地址送到页面寄存器。

在 DMA 期间, 由 8237 的 16 根引脚 $DB_7 \sim DB_0$ 、 $A_7 \sim A_4$ 、 $A_3 \sim A_0$ 输出存放在当前地址寄存器中的低 16 位物理地址, 由页面寄存器送出高 4 位物理地址, 共 20 位物理地址送到存储器选中一个存储单元。同时, 8237 的引脚 DACK 输出有效电平, 作为外设的选通信号。

PC/AT 中的 DMA 系统采用两片 8237 级联, 支持 7 个通道的 DMA 传输, 通道 4 作为主从片级联。从片通道 0~3 的 DMA 按字节传送, 主片通道 5~7 按字传送, 主从片都可寻址 16MB 存储空间。8237 本身提供 16 个地址 $A_{15} \sim A_0$, 每个通道都由一个 8 位页面寄存器来提供高 8 位地址, 这是是非 DMA 期间由 CPU 写入相应通道的页面寄存器的。

DMA 操作时, 从片 $CH_0 \sim CH_3$ 的地址低 16 位由 8237 提供, 高 8 位由页面寄存器提供。在寻址高位字节时, 用 A_0 代替高字节允许信号 $\overline{BHE}$ 。主片 $CH_5 \sim CH_7$ 在 DMA 操作时存储器地址与从片稍有不同。主 8237 提供 16 位地址 $A_{16} \sim A_1$, A_0 和 $\overline{BHE}$ 强制为 0, 相应的页面寄存器提供高 7 位地址 $A_{23} \sim A_{17}$, 对页面寄存器编程时只写高 7 位存储器地址。由于主 8237 传送 16 位数据, 所以在 DMA 周期内是对 16 位 I/O 接口和存储器存取, 在存储器偶地址即字边界上操作。

5. 8259 的 IR_2 中断服务程序原在全嵌套方式下执行, 要求在执行过程中某处能响应 8259 上除 IR_2 之外的任何级别的中断请求, 编写程序段框架。

解 IR_2 中断服务程序开始时在全嵌套方式下执行, 只有 IR_0 、 IR_1 上的中断请求能够中断 IR_2 的中断服务。而 IR_2 至 IR_7 上的中断服务请求都会被屏蔽。现要求程序能够响应除 IR_2 之外的 $IR_0 \sim IR_1$ 、 $IR_3 \sim IR_7$ 上 (只要没有被 IMR 屏蔽) 任

何级别的请求,则需要使 8259 进入特殊屏蔽方式。在程序中某处,首先向 8259 送 OCW_1 ,使 $IMR_2 = 1$,以屏蔽 IR_2 的再次请求;然后向 8259 送 OCW_3 ,使 $ESMM = 1$ 、 $SMM=1$,使 8259 进入特殊屏蔽方式。在 IR_2 中断服务程序继续执行过程中,可以响应比 IR_2 高和低级别的中断请求。因为 8259 在特殊屏蔽时是在一种打乱全嵌套方式下工作的,在中断结束时,必须送出特殊中断结束命令 $SEOI$,让 $L_2L_1L_0$ 指定要复位的 ISR 级别位。在特殊屏蔽方式下工作的中断服务程序在结束中断之前可以送出 OCW_3 使 $ESMM = 1$ 、 $SMM=0$;送出 OCW_1 ,使 $IMR_2 = 0$,回到一般全嵌套方式,可以用一般中断结束命令 EOI 结束中断。 IR_2 中断服务程序框架如下。

```

:                                     ;  $IR_2$  中断服务程序入口,保护现场,开中断,中断服务
CLI
MOV  AL, 04H                       ;  $OCW_1$  的  $D_2=1$  屏蔽  $IR_2$ 
MOV  DX, PORT+1                    ;  $OCW_3$  的口地址 ( $A_0=1$ )
OUT  DX, AL
MOV  AL, 68H                       ;  $OCW_3$ :  $ESMM=SMM=1$ 
MOV  DX, PORT+0                    ;  $OCW_3$  的口地址 ( $A_0=0$ )
OUT  DX, AL                         ; 进入特殊屏蔽方式
STI

:                                     ;  $IR_2$  中断服务程序继续执行,可响应高、低级中断
CLI
MOV  AL, 48H
MOV  DX, PORT+0
OUT  DX, AL
MOV  AL, 00H
MOV  DX, PORT+1
OUT  DX, AL
STI

:                                     ;  $IR_2$  中断服务程序回到全嵌套方式下继续执行
MOV  AL, 20H
MOV  DX, PORT+0
OUT  DX, AL
IRET

```

6. 主从两台微机双向并行传送 256B。主机方 8255 工作在方式 2 并且用中

断方式传送数据，从机方 8255 工作在方式 0，用查询方式传送数据。试画出接口框图和编写程序框架。

解 (1) 接口框图。

主机 8255 的 A 端口作双向传送，中断请求线连到中断控制器 8259 的 IR_2 上，输入/输出操作通过查询 IBF 和 OBF 状态区分。从机 8255 的 A 口和 B 口分别作输出/输入，用状态驱动数据传送。接口电路框图如图 11.9 所示。

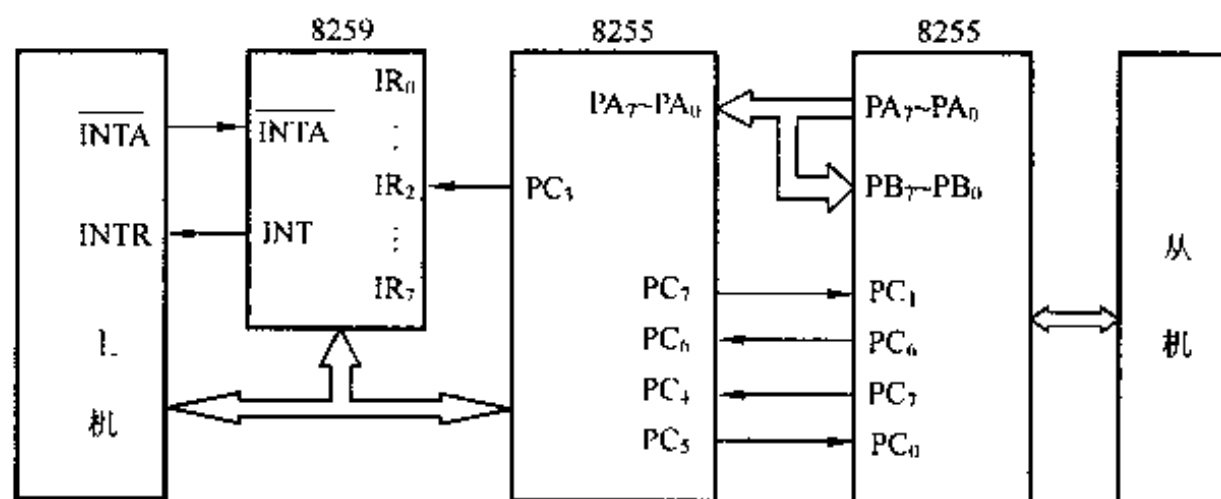


图 11.9

(2) 程序框架。

设主机方 8255 的端口地址为 $A0H \sim A3H$ ，从机方 8255 的端口地址为 $B0H \sim B3H$ 。主机方 8255 的 A 端口设置为方式 2；从机方 A 端口方式 0 输出，B 端口方式 0 输入，C 端口方式 0 中 $PC_7 \sim PC_4$ 为输出和 $PC_3 \sim PC_0$ 为输入。以下是主机方 8255 的初始化程序、主程序和中断服务程序框架。从机方的编程略。

```

:
MOV  AL,0C0H           ; 8255 初始化
OUT  0A3H,AL           ; 8255 控制端口
MOV  AL,09H            ; 使  $PC_4 = 1$ , 则  $INTE_2 = 1$ 
OUT  0A3H,AL           ; 允许输入中断
MOV  AL,0DH            ; 使  $PC_6 = 1$ , 则  $INTE_1 = 1$ 
OUT  0A3H,AL           ; 允许输出中断
LEA  SI, TRBUF          ; 发送数据块首地址
LEA  DI, RVBUF          ; 接收数据块首地址
MOV  CX,0FFH           ; 发送与接收字节数
:
AGAIN:STI              ; 主程序
      HLT              ; 等待中断
      CLI

```


DEC CX	: 通过中断传输了 1B
JNZ AGAIN	: 数据块未传送完
MOV AX,4C00H	
INT 21H	
INT PROC FAR	: 中断服务程序
MOV AL,08H	: 复位 PC ₄ , 使 INTE ₂ =0
OUT 0A3H,AL	
MOV AL,0CH	: 复位 PC ₆ , 使 INTE ₁ =0
OUT 0A3H,AL	
CLI	: 关中断
IN AL,0A2H	: 读 C 端口, 查状态
MOV AH,AL	: 保存状态字
AND AL,20H	: 查 IBF=1, 是否是输入
JZ OUTP	: 不是输入, 则是输出
INP:IN AL,0A0H	: 是输入, 从 A 端口读数
MOV [DI],AL	: 存入接收缓冲区
INC DI	: 内存地址加 1
JMP RETURN	: 返回
OUTP:MOV AL,[SI]	: 发送数据送 AL
OUT 0A0H,AL	: 从 A 端口输出
INC SI	: 内存地址加 1
RETURN:MOV AL,0DH	: 允许输出中断
OUT 0A3H,AL	
MOV AL,09H	: 允许输入中断
OUT 0A3H,AL	
MOV AL,62H	: OCW ₂ , 结束 IR ₂ 上的中断
OUT 020H,AL	
IRET	: 中断返回
INT ENDP	

7. 利用中断方式, 两台 PC/XT 机进行异步串行通信传送 256B。发送和接收缓冲区分别为 TRBUF 和 RVBUF, 8250 的中断请求线连到 8259 的 IR₄, 中断类型为 0CH。串行通信数据格式为 8 位数据、1 个停止位、1 个奇偶位、9600Baud。说明硬件接口和编写通信程序。

解 (1) 硬件接口。

PC/XT 用异步通信适配器与 Modem 连接实现异步串行通信, 2 个 PC/XT 机

器之间进行近距离数据传送, 可以不用 Modem, 只需把 2 台机器异步通信适配器发送数据端 2 和接收数据端 3 相互交叉连接、信号地 7 对接成为需要 Modem 方式即可。

(2) 通信程序。

```

STACK  SEGMENT  STACK 'STACK'
        DW  256  DUP(0)
STACK  ENDS
        DRG  XXXXH
DATA    SEGMENT  'DATA'
        INT-SEG  DW  ?
        INT-OFF  DW  ?
        TRBUF   DW  XXXXH
        RVBUF   DW  XXXXH
DATA    ENDS
CODE    SEGMENT  'CODE'
        ASSUME  CS:CODE,DS:DATA,SS:STACK
START:  MOV  AX, DATA                ; 建立数据段寻址
        MOV  DS, AX
        MOV  AL, 0CH                  ; 将中断类型 0CH 的向量通过 ES:BX 保存
        MOV  AH, 35H                  ; 指定的内存单元
        INT  21H
        MOV  INT-OFF, BX
        MOV  BX, ES
        MOV  INT-SEG, BX
        CLI
        MOV  AX, 250CH                ; 修改串行中断 0CH 的向量为本题串行通
        MOV  DX, SEG INTSEV           ; 信中断服务程序入口地址
        MOV  DS, DX
        MOV  DX, OFFSET INTSEV
        INT  21H
        MOV  AX, DATA                ; 恢复数据段寻址
        MOV  DS, AX
        MOV  DX, 3FBH                 ; 8250 重新初始化
        MOV  AL, 80H                  ; LCR 中 DLAB 置 1
        OUT  DX, AL                   ; 访问 DLI 和 DLH

```

```

MOV  DX, 3F8H
MOV  AL, 0CH                ; 送除数低字节
OUT  DX, AL
MOV  DX, 3F9H
MOV  AL, 00H                ; 送除数高字节
OUT  DX, AL
MOV  DX, 3FBH                ; LCR 地址
MOV  AL, 0BH                ; 串行通信数据格式
OUT  DX, AL
MOV  DX, 3FCH                ; MCR 地址
MOV  AL, 08H                ; OUT2置1, 允许 COM1 中断
OUT  DX, AL
MOV  DX, 3F9H                ; IER 地址
MOV  AL, 03H                ; 允许接收和发送中断
OUT  DX, AL
IN   AL, 21H                ; 读 8259 的 IMR
AND  AL, 0EFH                ; 允许 IR4 上的中断请求
OUT  21H, AL
MAIN: STI
      HLT                    ; 等待中断发生
      CLI
      MOV  CL, 256            ; 传送 256B
      DEC  CL
      CMP  CL, 00H            ; 检查接收/发送完成
      JNZ  MAIN                ; 未完, 继续
      MOV  AX, 250CH            ; 完成, 恢复中断向量
      MOV  DX, INT-OFF
      MOV  BX, INT-SEG
      MOV  DS, BX
      INT  21H
      IN   AL, 21H            ; 屏蔽 8259 的 IR4 的中断请求
      OR   AL, 10H
      OUT  21H, AL
      STI
      MOV  AX, 4C00H            ; 终止退出, 返回 DOS

```

```

        INT 21H
INTSEV  PROC FAR                                ; 串行通信中断服务程序
        STI                                     ; 开中断
        PUSH AX                                ; 保护现场
        PUSH CX
        PUSH DX
        PUSH SI
        PUSH DI
        PUSH DS
        PUSH ES
        MOV DX, 3FAH                            ; IIR 地址
        IN AL, DX
        CMP AL, 02H                            ; 是否发送中断
        JZ TRAN                                ; 是则转发送子程序
RECE:   MOV DX, 3F8H                            ; 接收子程序, RBR 口地址
        IN AL, DX                              ; 从 RBR 读 1B
        MOV DI, RVBUF
        MOV [DI], AL                          ; 读入字节送接收缓冲区
        INC DI                                ; 修改接收缓冲区指针
        MOV RVBUF, DI
        JMP RETURN
TRAN:   MOV SI, TRBUF                          ; 发送子程序
        MOV DX, 3F8H                            ; THR 口地址
        MOV AL, [SI]
        OUT DX, AL                            ; 发送 1B
        INC SI                                ; 修改发送缓冲区指针
        MOV TRBUF, SI
RETURN: POP ES
        POP DS
        POP DI
        POP SI
        POP DX
        POP CX
        POP AL
        MOV AL, 20H                            ; 向 8259 送 EOI

```

```

        OUT  20H, AL          ; OCW2 端口地址
        IRET
INTSEV  ENDP
CODE    ENDS
        END  START

```

8. 建立一个程序框架使 CPU 进入保护模式：在内存最低端建立一个临时 GDT，装入 GDTR，转入保护模式，把 GDT、IDT 和 TSS 从 ROM 移到 RAM 指定处，转向初始任务。

解 设内存的底部有 64KB 的暂存空间用于本程序模块，并有充足的 RAM 空间能拷贝初始 GDT、IDT 和 TSS。

```

CS_BASE    EQU  0FFFF0000H      ; STARTUP_CODE 段线性地址
RAM_START  EQU  400H            ; 线性地址空间可用空间内存起始地址
TSS_INDEX  EQU  10              ; 程序运行后第一个任务的 TSS 索引值
TASK_STATE STRUC                ; TSS 结构
    LINK    DW  ?
    LINK_H  DW  ?
    ESP0    DD  ?
    SS0      DW  ?
    SS0_H    DW  ?
    ESP1    DD  ?
    SS1      DW  ?
    SS1_H    DW  ?
    ESP2    DD  ?
    SS2      DW  ?
    SS2_H    DW  ?
    CR3_REG DD  ?
    EIP_REG  DD  ?
    EFLAGS_REG DD  ?
    EAX_REG  DD  ?
    ECX_REG  DD  ?
    EDX_REG  DD  ?
    EBX_REG  DD  ?
    ESP_REG  DD  ?
    EBP_REG  DD  ?
    ESI_REG  DD  ?

```

```

    EDI_REG    DD ?
    ES_REG     DW ?
    ES_H       DW ?
    CS_REG     DW ?
    CS_H       DW ?
    SS_REG     DW ?
    SS_H       DW ?
    DS_REG     DW ?
    DS_H       DW ?
    FS_REG     DW ?
    FS_H       DW ?
    GS_REG     DW ?
    GS_H       DW ?
    LDT_REG    DW ?
    LDT_H      DW ?
    TRAP_REG   DW ?
    IO_MAP_BASE DW ?

TASK_STATE    ENDS

DESC STRUC
    LIM_0_15   DW ?
    BAS_0_15   DW ?
    BAS_16_23  DB ?
    ACCESS     DB ?
    GRAN       DB ?
    BAS_24_31  DB ?
DESC ENDS

TABLE_REG STRUC
    TABLE_LIM      DW ?
    TABLE_LINEAR    DD ?
TABLE_REG ENDS

GDT_DESC_OFF EQU 1xSIZE(DESC) ; GDT 和 IDT 描述符在 GDT 中的偏移量
IDT_DESC_OFF EQU 2xSIZE(DESC)
LINEAR_SEL    EQU 1xSIZE(DESC) ; 内存中建立临时 GDT 线性地址别名
LINEAR_PROTO_LO EQU 0000FFFFH
LINEAR_PROTO_HI EQU 00CF9200H

```

；描述符的基本结构

； LGDT 和 LIDT 指令使用内存定义

```

PE_BIT EQU 1B ; CR0 中 PE 位
STARTUP_DATA SEGMENT RW
    FREE_MEM_LINEAR_BASE LABEL DWORD
    TEMP_GDT LABEL BYTE ; 段中第 1 个
    TEMP_GDT_NULL_DESC DESC <>
    TEMP_GDT_LINEAR_DESC DESC <>
    TEMP_GDT_SCRATCH TABLE_REG <> ; LGDT 和 LIDT 指令使用的
    APP_GDT_RAM TABLE_REG <> ; 临时区域
    APP_IDT_RAM TABLE_REG <>
    FILL DW ? ; 对准 END_DATA
    END_DATA LABEL BYTE ; 对准双字边界
STARTUP_DATA ENDS
STARTUP_CODE SEGMENT ER PUBLIC USE16
    PUBLIC GDT_EPROM
GDT_EPROM TABLE_REG <> ; 编程者填写
    PUBLIC STARTUP
IDT_EPROM TABLE_REG <> ; 编程者填写
    PUBLIC STARTUP
; 进入 STARTUP 码段入口点, 引导程序将通过一个近 JMP 指令转到此处, 这个标
; 号必须在线性空间的顶部 64KB.
STARTUP: ASSUME DS:STARTUP_DATA,ES:STARTUP_DATA
    LEA EBX,TEMP_GDT ; 在内存低端构造临时 GDT
    MOV DWORD PTR [EBX],0
    MOV DWORD PTR [EBX]+4,0
    MOV DWORD PTR [EBX]+8,LINEAR_PROTO_LO
    MOV DWORD PTR [EBX]+12,LINEAR_PROTO_HI
    MOV TEMP_GDT_SCRATCH.TABLE_LINEAR,EBX
    MOV TEMP_GDT_SCRATCH.TABLE_LIM,15
    DB 66H ; 执行 32 位的 LGDT
    LGDT TEMP_GDT_SCRATCH ; 用临时 GDT 装入 GDTR
    MOV EBX,CR0 ; 进入保护模式
    OR EBX,PE_BIT
    MOV CR0,EBX
    JMP CLEAR_LABEL ; 清预取队列
CLEAR_LABEL: MOV CX,LINEAR_SEL ; DS 和 ES 有 4G 线性地址

```

```

MOV DS,CX
MOV ES,CX

; ; 主要初始化

MOV EAX, RAM_START ; 拷贝 EPROM GDT 到 RAM 中
ADD EAX, OFFSET(END_DATA) ; RAM_START+SIZE(STARTUP_DATA)
MOV EBX, RAM_START
MOV ECX, CS_BASE
ADD ECX, OFFSET(GDT_EEPROM)
MOV ESI, [ECX].TABLE_LINEAR
MOV EDI, EAX
MOVZX ECX, [ECX].TABLE_LIM
MOV APP_GDT_RAM[EBX].TABLE_LIM, CX
INC ECX
MOV EDX, EAX
MOV APP_GDT_RAM[EBX].TABLE_LINEAR, EAX
ADD EAX, ECX
REP MOVS BYTE PTR ES:[EDI], BYTE PTR DS:[ESI]
MOV ECX, EDX ; 在描述符中装入 GDT 基址
MOV [EDX].BASE_0_15+GDT_DESC_OFF, CX
ROR ECX, 16
MOV [EDX].BASE_16_23+GDT_DESC_OFF, CL
MOV [EDX].BASE_24_31+GDT_DESC_OFF, CH
; 拷贝 EPROM IDT 到 RAM 中 RAM_START+SIZE(STARTUP_DATA)
; +SIZE(EEPROM GDT)
MOV ECX, CS_BASE
ADD ECX, OFFSET(IDT_EEPROM)
MOV ESI, [ECX].TABLE_LINEAR
MOV EDI, EAX
MOVZX ECX, [ECX].TABLE_LIM
MOV APP_IDT_RAM[EBX].TABLE_LIM, CX
INC ECX
MOV APP_IDT_RAM[EBX].TABLE_LINEAR, EAX
MOV EBX, EAX

```



```

    ADD    EAX,ECX
REP    MOVS    BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
    MOV    [EDX].BASE_0_15+IDT_DESC_OFF,BX    ; 安装 GDT 中 IDT 指针
    ROR    EBX,16
    MOV    [EDX].BASE_16_23+IDT_DESC_OFF,BL
    MOV    [EDX].BASE_24_31+IDT_DESC_OFF,BH
    MOV    EBX,RAM_START                        ; 装载 GDTR 和 IDTR
    DB     66H                                  ; 执行 32 位的 LGDT
    LGDT    APP_GDT_RAM[EBX]
    DB     66H                                  ; 执行 32 位的 LIDT
    LIDT    APP_IDT_RAM[EBX]
    MOV    EDI,EAX                              ; 移动 TSS
    MOV    EBX,TSS_INDEX.SIZE(DESC)
    MOV    ECX,GDT_DESC_OFF                    ; 为 TSS 构造线性地址
    MOV    GS,CX
    MOV    DH,GS:[EBX].BASE_24_31
    MOV    DL,GS:[EBX].BASE_16_23
    ROL    EDX,16
    MOV    DX,GS:[EBX].BASE_0_15
    MOV    ES,EDX
    LSL    ECX,EBX
    INC    ECX
    MOV    EDX,EAX
    ADD    EAX,ECX
REP    MOVS    BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
    MOV    GS:[EBX].BASE_0_15,DX                ; 装 TSS 指针
    ROL    EDX,16
    MOV    GS:[EBX].BASE_24_31,DH
    MOV    GS:[EBX].BASE_16_23,DL
    ROL    EDX,16
    ; 存空间 RAM 的起始地址在 RAMSTART 的线性地址
    MOV    FREE_MEM_LINEAR_BASE+RAM_START,EAX
    ; 设没有初始任务的 LDT, 如有需要增加移动 LDT 的代码, 并且类似于
    ; 移动 TSS 的编码
    LTR    BX                                  ; 无任务转换, 仅描述符装入

```


实验指导篇

第 12 章 微机接口实验系统 结构及使用说明

12.1 TPC-H 微机接口实验系统基本组成

TPC-H 微机接口实验系统是由清华大学同方集团教学仪器设备公司生产的。该系统由接口卡、实验台两部分组成，二者之间通过 50 线扁平电缆相连。接口卡插入到 PC 系列微机中的 ISA 总线插槽（62 线插槽）上，它的主要功能是将与实验有关的总线信号经驱动后引到实验台上，供相关实验使用，同时引出的信号还有 +5V、+12V、-12V 电源。另外，接口卡上设有一个跳线开关（JP）用于选择 IRQ₂、IRQ₃、IRQ₄、IRQ₇ 中的一个作为中断请求信号（该卡在出厂时已将 4 与 5 短接选择 IRQ₇ 作为中断请求信号）。接口卡的结构如图 12.1 所示。

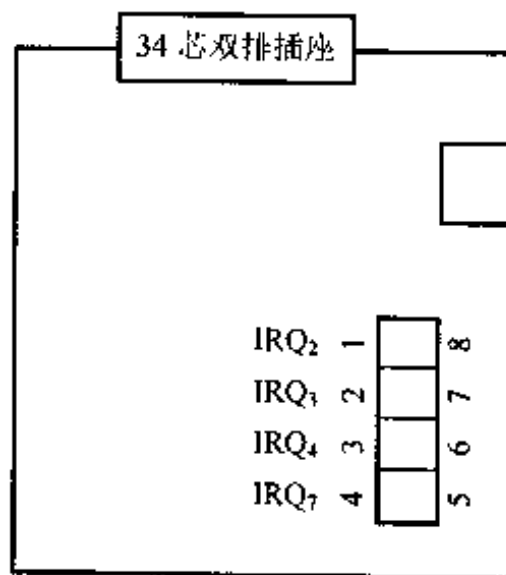


图 12.1 接口卡结构示意图

12.2 实验台结构及使用说明

实验台由 I/O 地址译码电路、总线插孔、接口实验常用集成电路、外围电路及通用 IC 插座等部分组成。外围电路包括逻辑电平开关电路、LED（发光二极管）显示电路、LED（七段数码管）显示电路、时钟电路、单脉冲电路、基本逻辑门电路、复位电路、继电器及步进电机和小直流电机的驱动电路以及逻辑测试笔电路等。

实验台结构布置如图 12.2 所示。下面介绍实验台上的主要通用电路、开关和插座的功能及使用方法。

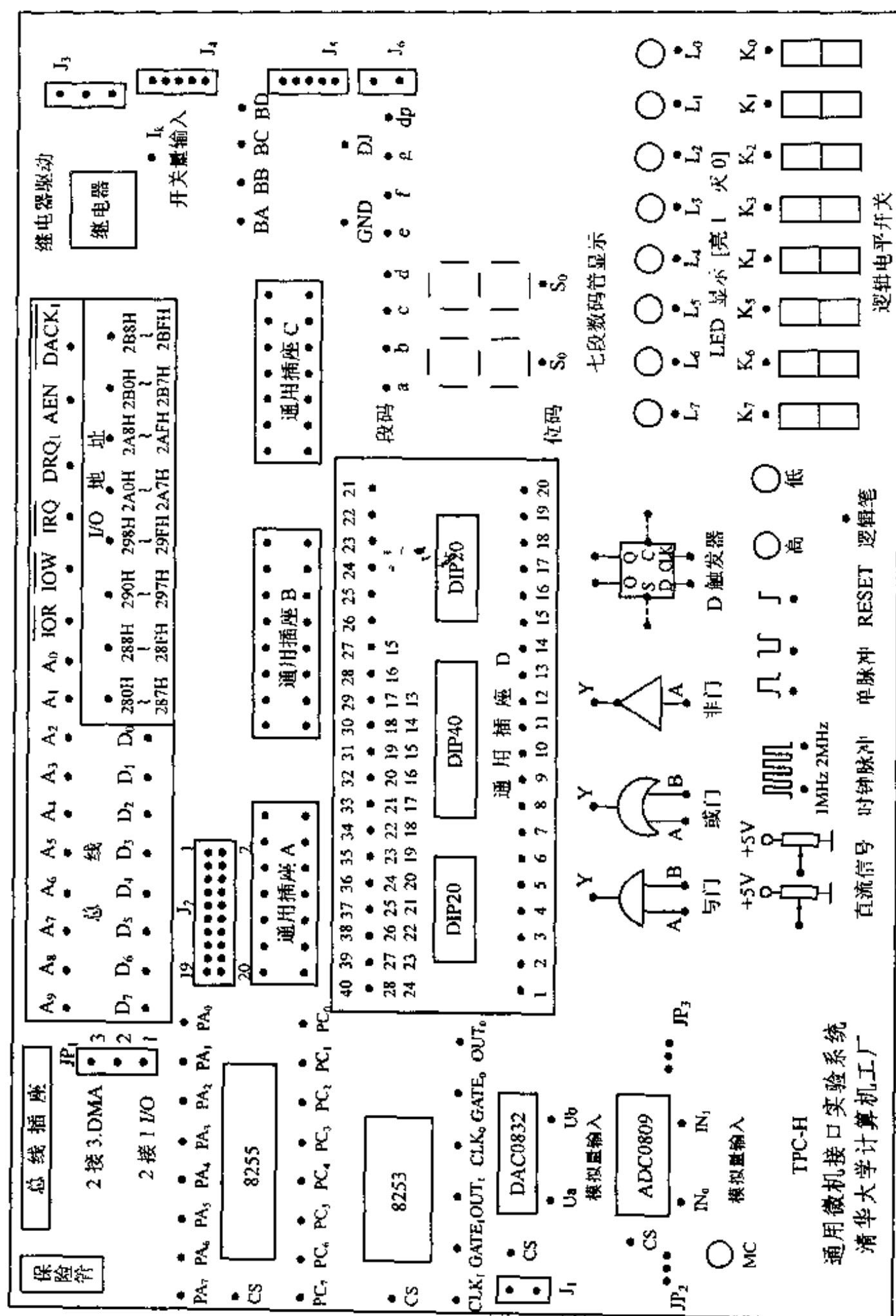


图 12.2 实验台结构

1. I/O 地址译码电路

I/O 地址译码电路如图 12.3 所示。实验台的 I/O 地址选用 PC 系列微机未使用的 I/O 地址空间：280H~2BFH，共分为 8 条译码输出线：Y₀~Y₇。其地址分别为 280H~287H；288H~28FH；290H~297H；298H~29FH；2A0H~2A7H；2A8H~2AFH；2B0H~2B7H；2B8H~2BFH。这 8 根译码输出线在实验台上的“I/O 地址”处分别由“自锁紧”插孔引出，供实验选用。

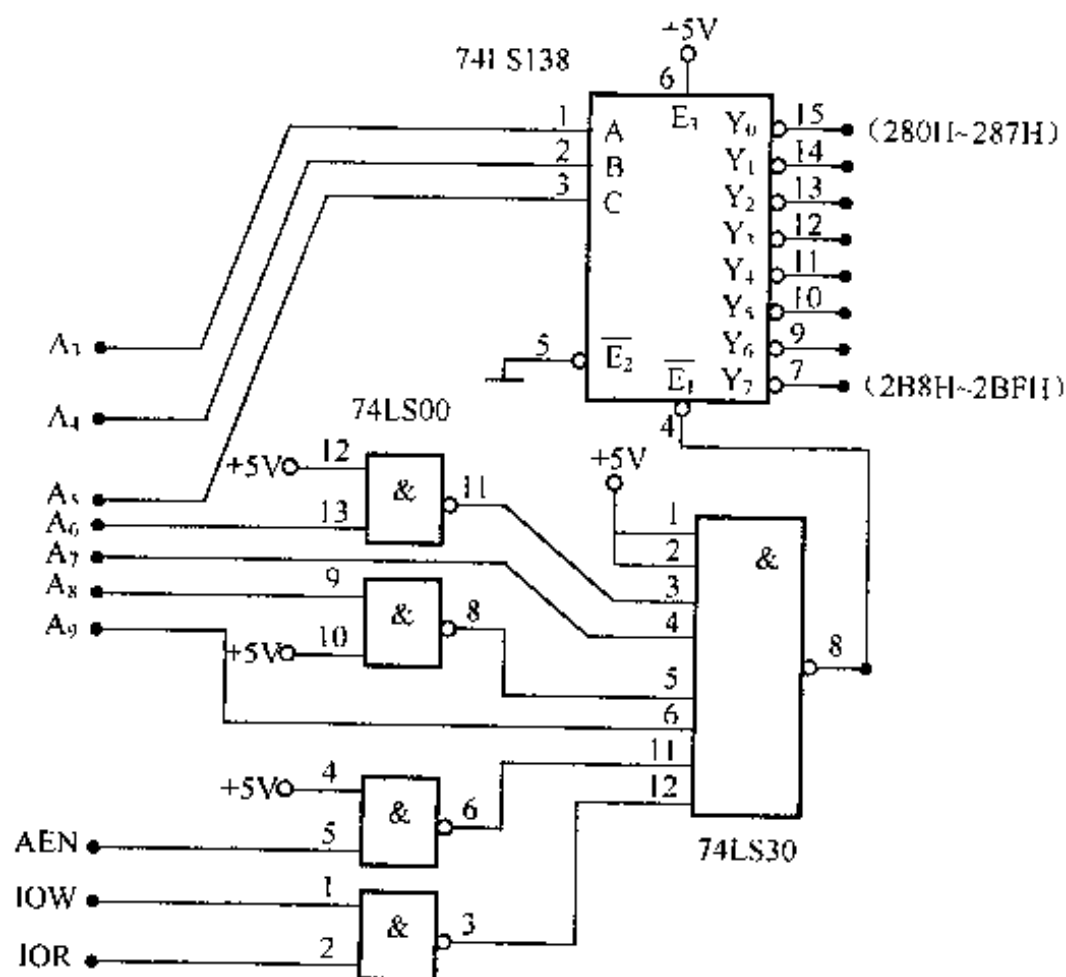


图 12.3 I/O 地址译码电路

2. 总线插孔

总线区引出了经 PC 总线接口卡驱动后的 39 个总线信号（分别采用“自锁紧”插孔引出，供实验者搭接各种接口实验电路使用），包括数据信号 D₇~D₀；地址总线信号 A₁₉~A₀；存储器读/写信号 $\overline{\text{MEMR}}$ 、 $\overline{\text{MEMW}}$ ；输入/输出读、写信号 $\overline{\text{IOR}}$ 、 $\overline{\text{IOW}}$ ；中断请求信号 IRQ；DMA 请求信号 DRQ₁；DMA 响应 $\overline{\text{DACK}}$ 、地址锁存信号 ALE；地址允许信号 AEN；时钟 CLK 和 DMA 传送终止信号 T/C 等。

3. 时钟电路

时钟电路如图 12.4 所示，4MHz 的晶体振荡器通过两个触发器的分频，分别产生输出频率为 2MHz、1MHz 的连续脉冲，供“定时器/计数器、A/D 转换器、串行接口”等实验使用。

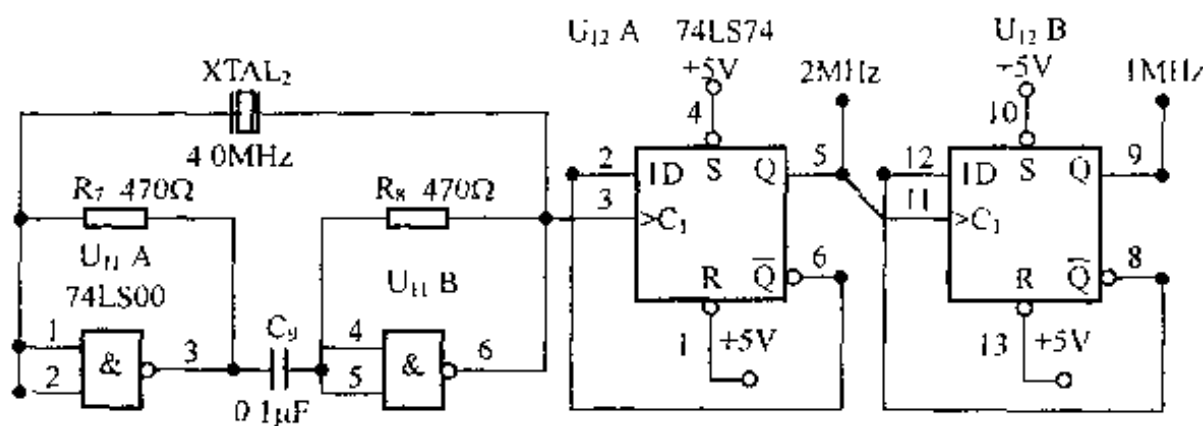


图 12.4 时钟电路

4. 逻辑电平开关电路

逻辑电平开关电路如图 12.5 所示，实验台右下方设有 8 个开关 $K_7 \sim K_0$ ，开关拨向上为“1”状态（即开关断开，输出高电平）；开关拨向下为“0”状态（即开关接通，输出低电平）。电路中串接了保护电阻，使接口电路不直接与 +5V、GND 相连，可有效地防止因误操作、误编程损坏集成电路。

5. LED 发光二极管显示电路

该电路如图 12.6 所示，实验台上设有 8 个发光二极管及相关驱动电路（输入端为 $L_7 \sim L_0$ ），当输入信号为“1”时发光二极管亮，为“0”时发光二极管灭。

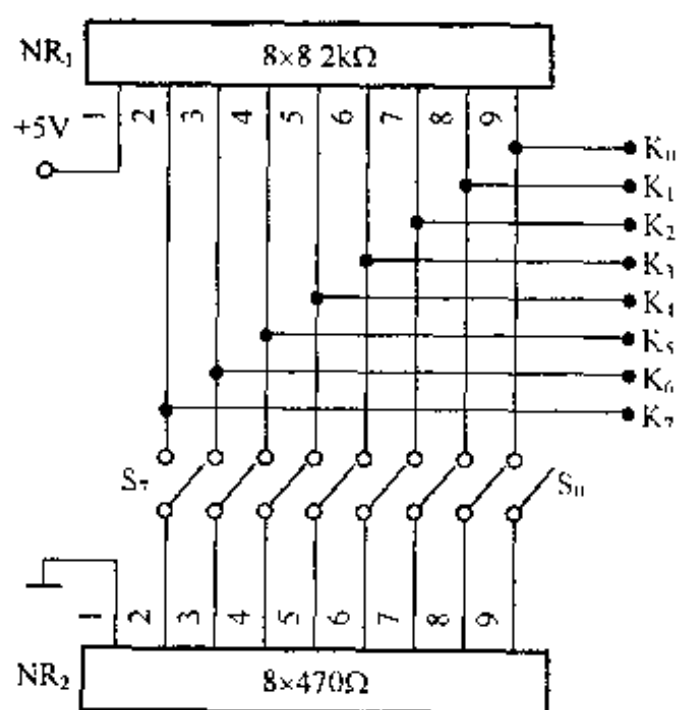


图 12.5 逻辑电平开关电路

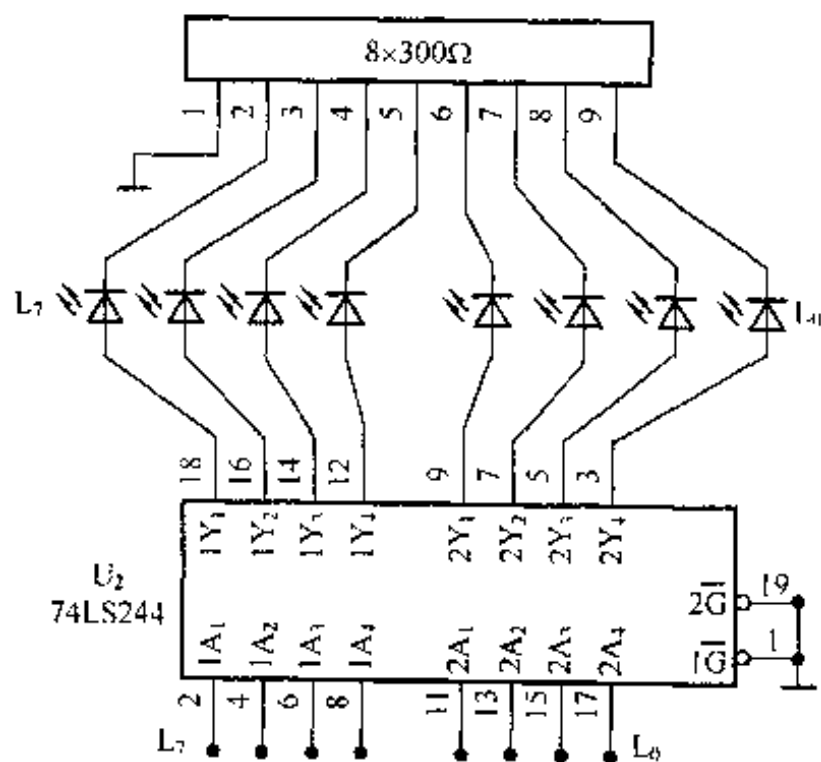


图 12.6 LED 发光二极管显示电路

6. 七段数码管显示电路

该电路如图 12.7 所示，在实验台上设有两个共阴极七段数码管及驱动电路。段码为同相驱动器，位码为反相驱动器，从段码与位码的驱动器输入端（段码输入端：a、b、c、d、e、f、g、dp。位码输入端： S_0 、 S_1 ）输入不同的代码即可显

示不同数字或符号。

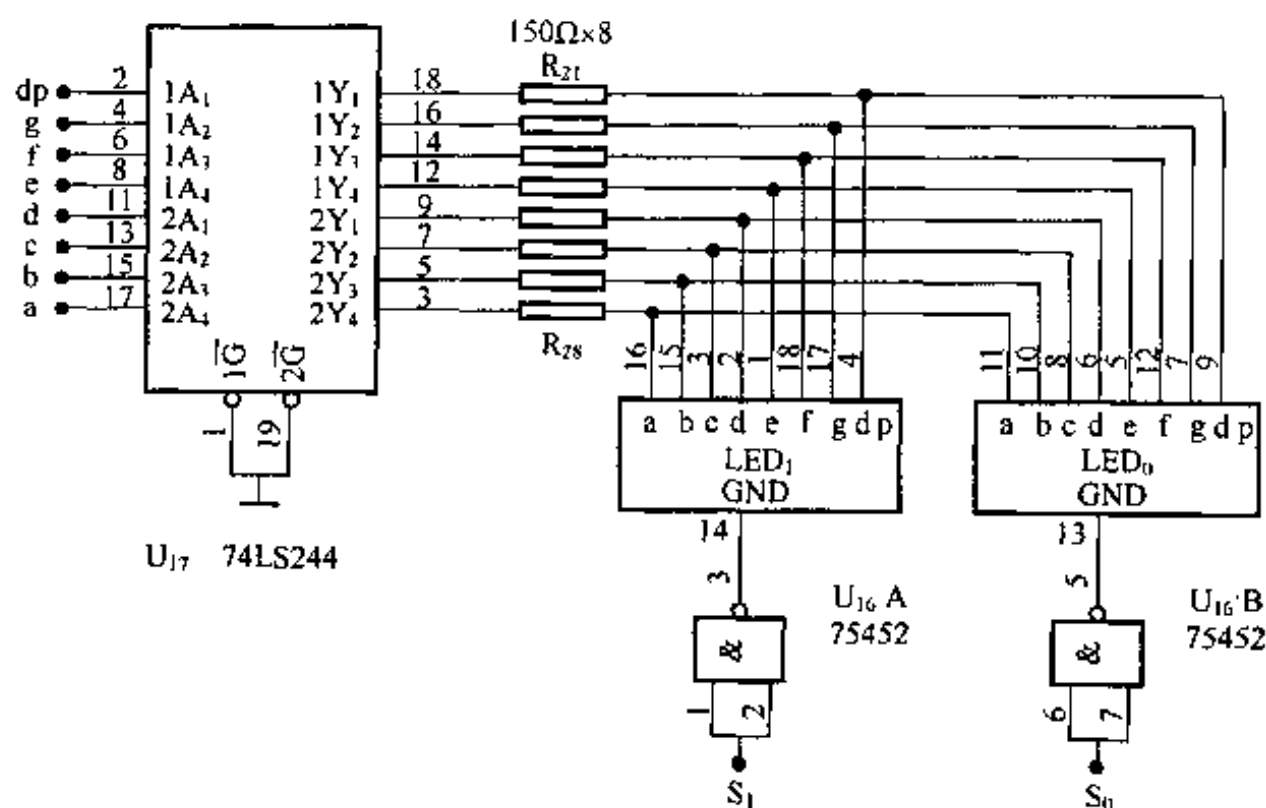


图 12.7 七段数码管显示电路

7. 单脉冲产生电路

该电路如图 12.8 所示，采用 R-S 触发器构成。实验者每按动一次微动开关即可从两个插孔上分别输出一个正脉冲和一个负脉冲，供“中断、DMA、定时器/计数器”等实验使用。

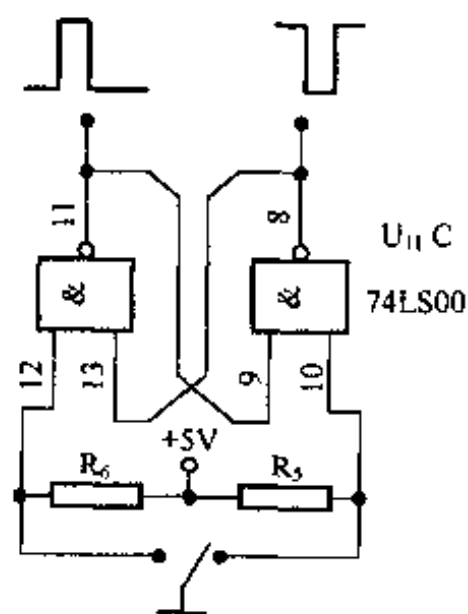


图 12.8 单脉冲产生电路

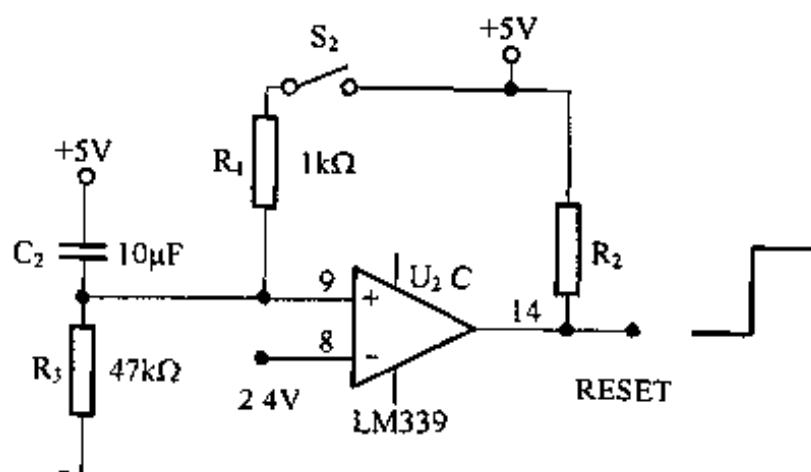


图 12.9 复位电路

8. 复位电路

复位电路如图 12.9 所示，该电路能在上电时或按下复位开关后产生一个高电平的复位信号供 8255、8251 等接口芯片使用。

9. 继电器及驱动电路

该电路如图 12.10 所示, 实验台右上角设有一个+5V 直流继电器及相应的驱动电路。当其开关量输入端 I_k 输入数字量“1”时, 继电器动作: 常开触点闭合, 常闭触点断开。通过相应的实验使实验者了解开关量控制的一般原理。

10. 逻辑笔测试电路

该电路如图 12.11 所示, 当输入端(V)接高电平时红灯亮; 接低电平时绿灯亮。逻辑笔可以替代万用表的部分功能, 检测实验电路中各点的高/低电平, 为调试实验电路提供方便。

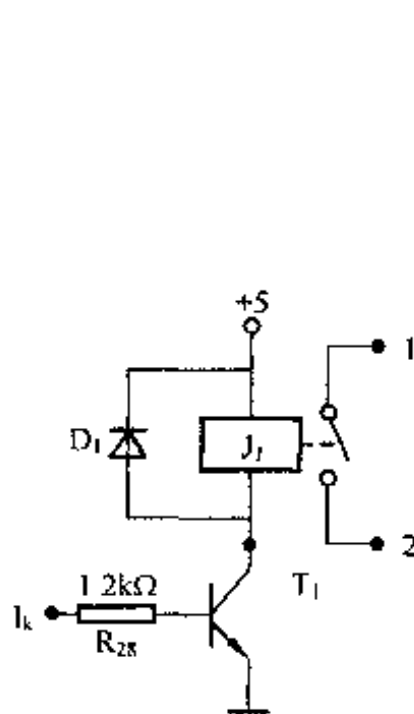


图 12.10 继电器驱动电路

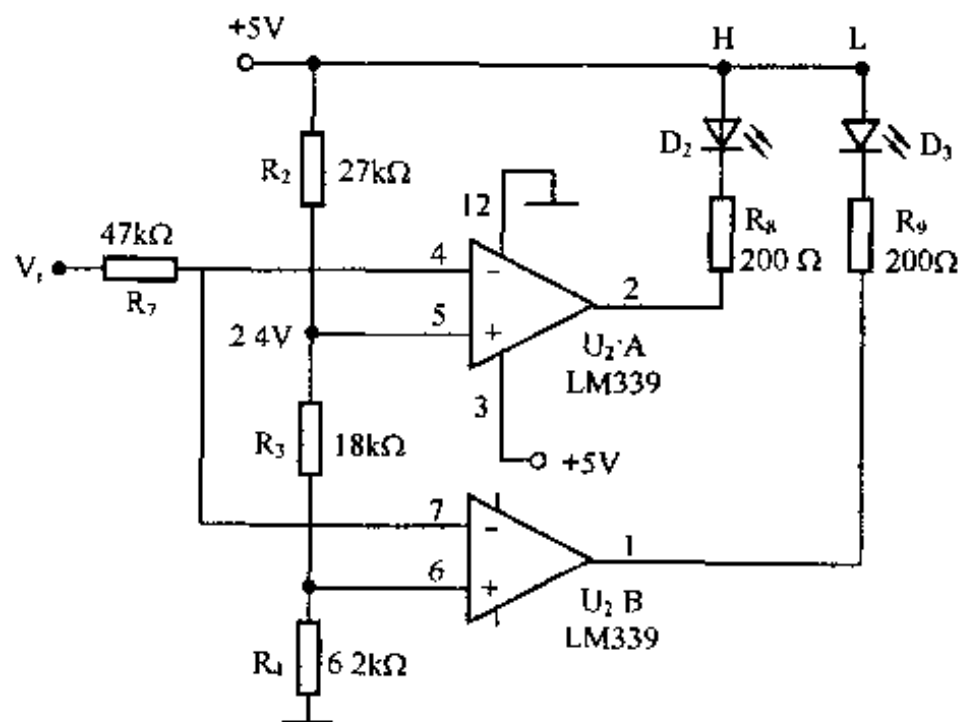


图 12.11 逻辑笔测试电路

11. 接口集成电路

实验台上还配有微机接口实验所需要的最常用的接口电路芯片。包括: 可编程定时器/计数器 (8254)、可编程并行接口 (8255)、数/模转换器 (DAC0832)、模/数转换器 (ADC0809)。这些芯片与 CPU 相连的引线, 除片选信号 $\overline{CS}$ 外在实验台内部都已连接好。此外, 与外界连接的关键引脚在芯片周围用“自锁紧”插孔引出, 供实验者实验时使用。在本实验系统中未用到的引脚, 如 8255 的 $PB_0 \sim PB_7$; 8254 的 CLK_2 、 $GATE_2$ 、 OUT_2 ; ADC0809 的 $IN_3 \sim IN_7$ 未专门引出。另外, 数/模转换器附有双极性输出插孔, 模/数转换器附有双极性输入插孔。具体电路可见后面各实验指导。

12. 基本逻辑单元电路

实验台上设有一块基本逻辑单元电路实验区 (即与门、或门、非门和 D 触发器), 供实验者在做接口实验或数字电路实验时直接使用。

13. 跳线开关

实验台上设有 5 个跳线开关, 其中 JA、JB、JC 在实验台的左上角、50 线总

线插座的左侧和下方。JA 用来选择用主机电源还是用外接附加电源。当用主机电源时,应将 JA 上+5V、+12V、-12V 三个短路片插好。用户若想使用外接附加电源,则必须先将 JA 上的三个短路片全部拔掉才能将外接电源加入。JB 和 JC 用来选择实验类型。做 I/O 实验时,JB、JC 上的短路片都应该插在标有“I/O”的位置上;做存储器实验时,JB、JC 上的短路片都应该插在标有“MEM”的位置上;做 DMA 实验时,JB 应插在“I/O”位置上,JC 应插在“DMA”位置上。实验台出厂时短路片插在“I/O”位置上。

实验台上另两个跳线开关为 JP₂ 和 JP₃。它们位于实验台的左下角,分别用于模/数转换器模拟量输入极性选择。将 JP₂ 的 1、2 短路时,IN₂ (内部已接好 J₂) 可输入双极性电压 (-5~+5V); 2、3 短接时,为单极性电压 (0~+5V)。JP₃ 用来选择 IN₁ 的输入极性,方法同 JP₂。

14. +5V 电源插针

为了减少主机+5V 电源的负担和各主要芯片的安全,在主要接口芯片的左上角都设有相应的电源连接插针(标记为+5V)。当需要用到该芯片时,就用短路片短接该芯片的+5V 插针,即可接通+5V 电源。对于本次实验用不到的芯片一般要将该芯片的+5V 电源短路片拔掉以确保芯片的安全。

15. 通用集成电路插座

实验台上设有 5 个通用数字集成电路插座。其中通用插座 A、B 为 20 脚;通用插座 D 区中有 14 脚、16 脚和 40 脚的插座各一个。每个插座引脚附近都有对应的“自锁紧”插孔。部分实验(DMA、串行双机通信)就是利用这些插座搭接电路的。此外,这些插座还可用于开发新实验。

16. 接线端子

实验台上设有 8 个接线端子,标号为 J₀~J₇。J₀ 用于外接电源;J₁ 用于外接喇叭;J₂ 是一个立体声插孔,用于外接话筒;J₃ 为实验台上的继电器触点;J₄ 用于外接继电器,其中 1 接+5V,2、3 为常闭触点,3、4 为常开触点,5 接地;J₅ 用于外接步进电机;J₆ 用于外接小直流电机;J₇ 是一个 20 芯通用插座,用于外接用户开发的实验板。J₇ 各引脚信号如表 12.1 所示。

表 12.1

19	17	15	13	11	9	7	5	3	1
D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	IRQ	CS
RES	+5V	+5V	$\overline{\text{IOR}}$	$\overline{\text{IOW}}$	A ₀	A ₁	CLK	GND	GND
20	18	16	14	12	10	8	6	4	2

17. 50 线总线插座信号

50 线总线插座信号具体安排如表 12.2 所示。

表 12.2

引脚	信号	引脚	信号	引脚	信号	引脚	信号	引脚	信号
1	+5V	11	E245	21	A ₇	31	A ₁	41	ALE
2	D ₋	12	$\overline{\text{IOR}}$	22	A ₆	32	GND	42	T/C
3	D ₀	13	$\overline{\text{IOW}}$	23	A ₅	33	A ₁₁	43	A ₁₆
4	D ₁	14	AEN	24	+12V	34	GND	44	A ₁₇
5	D ₄	15	$\overline{\text{DACK}}$	25	A ₁	35	$\overline{\text{MEMW}}$	45	A ₁₅
6	D ₃	16	DRQ1	26	GND	36	$\overline{\text{MEMR}}$	46	A ₁₄
7	D ₂	17	IRQ	27	A ₁	37	CLK	47	A ₁₃
8	D ₁	18	+5V	28	-12V	38	RST	48	A ₁₂
9	D ₀	19	A ₉	29	A ₂	39	A ₁₉	49	A ₁₀
10	+5V	20	A ₈	30	GND	40	A ₁₈	50	A ₁₁

18. 直流电源开关 (K₁) 和存储器地址选择开关 (K₂)

实验台上有一个直流电源开关 (K₁)，在实验台的左上角。先打开主机电源开关，再打开实验台上的开关 K₁，实验台才会有电。另外，实验台上还有一个 4 位拨动开关 K₂，在 8255 芯片的右边。K₂ 是在做存储器实验中选择存储器地址时使用的。

19. 使用外接直流电源注意事项

① 用户可以使用主机电源，也可以使用外接直流电源。主机电源是通过总线接口卡引入到实验台的 50 芯插座上，再经过 JA 接到电源开关上的。本实验台在出厂时是按使用主机电源连接的。

② 用户若想使用外接电源，则必须先将 JA 上的 3 个短路片全部拔掉，将主机电源断开后，才能将外部电源接入。

③ 用户电源通过外接电源插座 J₀ 接到实验台上。插入前应仔细检查电源插头上各路电源与 J₀ 上标注的是否相同，如不相同，则应修改电源插头。不然会损坏实验台。

④ 外接电源插入前应检查各路电源输出是否正确，如不正确，则应修改或更换电源。

⑤ 以上各项都正常后，打开电源即可进行实验。

20. 使用自锁紧导线注意事项

将自锁紧导线的插头插入自锁紧插孔时，应稍加用力并沿顺时针方向旋转一下，才能保证接触良好。拔出时，应用手捏住插头先逆时针方向旋转一下，待完全松开后再向上拔出。如果在插头没有完全松开之前强行拔出，很容易损坏导线，造成实验故障。

第 13 章 实验程序的建立与执行

13.1 编辑和运行汇编源程序所必备的软件

在 PC 机上运行汇编源程序，必须具备以下软件：

- ① DOS 2.0（或 CCDOS）以上版本操作系统。
- ② EDIT 文本编辑程序（或其他文本编辑软件）。
- ③ MASM 宏汇编程序。
- ④ LINK 连接程序。
- ⑤ CREF 交叉引用文件处理程序（可选）。
- ⑥ LIB 库管理程序（可选）。
- ⑦ DEBUG 调试程序。
- ⑧ QBASIC 编辑程序。

13.2 建立与执行汇编源程序的步骤

在 TPC-H 微机实验系统中进行各项接口实验，一般都要用汇编语言（或 C 语言）来编写相应的实验程序。以下简要介绍汇编语言程序从建立到执行的过程。

用汇编语言编写的程序（称为汇编源程序）要在机器上运行，必须经过以下几个步骤：

- ① 在 DOS 环境下，调用任一编辑程序（如 EDIT）建立与修改汇编源程序（扩展名必须为.ASM）。
- ② 用宏汇编程序（MASM）对汇编源程序进行汇编，生成相应的目标文件（扩展名为.OBJ）。
- ③ 用连接程序（LINK）对目标文件进行再定位和链接，生成可执行文件（扩展名为.EXE）。
- ④ 运行可执行文件。

经过步骤①~③之后,可执行文件已生成并存放在磁盘上。此时,在 DOS 提示符下直接键入文件名(可不带扩展名.EXE),即可将该文件从磁盘上装入内存立即执行。

然而,设计一个较复杂的汇编源程序,不出现一点错误是不大可能的。如果出现错误,可调用 DOS 支持下的 DEBUG 程序(调试程序),对可执行文件进行动态跟踪调试运行,找出错误的地方。当发现错误后,要重复上述①~④步,即修改源程序中的错误,重新汇编、连接、运行程序,直至程序运行正确(或符合设计要求)为止。

下面将结合一个简单实例对编辑、汇编、连接、运行和调试程序的使用过程作一简要介绍。

在显示器上显示一行“HOW ARE YOU!”信息的汇编语言源程序如下:

```
STACK  SEGMENT  STACK
        DB  200 DUP (0)

STACK  ENDS

DATA    SEGMENT
        BUF  DB  'HOW ARE YOU!','$'
DATA    ENDS

CODE    SEGMENT
        ASSUME  CS:CODE, DS:DATA, SS:STACK

START:  MOV  AX, DATA
        MOV  DS, AX
        LEA  DX, BUF
        MOV  AH, 9
        INT  21H
        MOV  AH, 4CH
        INT  21H

CODE    ENDS

        END  START
```

按上述步骤,应首先用编辑程序在磁盘上建立汇编源程序文件(假设源文件名为 Display.ASM),然后对该文件进行汇编、连接、运行。各命令格式如下(假定所有文件都在 C:\MASM>目录下)。

1. 用 EDIT 编辑汇编源程序

命令格式为:

C:\MASM>EDIT [文件名]

其中,文件名为可选项。如果要带文件名,则必须是带有.ASM 扩展名的文件名

全称。例如：

C:\MASM>EDIT Display.ASM 或 C:\MASM>EDIT

如果启动 EDIT 时不带文件名，而该文件又存在，则在启动后，可选择 File 菜单中的 Open 命令打开并调入该文件，然后对它进行编辑，最后选择 Save 命令保存该文件。

如果是新文件，则在启动 EDIT 之后，可先用 EDIT 编辑程序中的相应命令编辑写好的汇编源程序，编辑完后选择 File 菜单中的 Save As 命令，并按提示回答保存该文件。

如果启动 EDIT 时带有文件名，而且该文件也存在，则启动后即马上将该文件调入编辑器内；否则，说明该文件不存在，则启动后可作为新文件名处理。

在 EDIT 下，源程序是以 ASCII 码字符的形式存放在内存缓冲区内的。若输入过程有错误或对源程序进行汇编、连接、运行过程中发现错误，都可以利用 EDIT 的一些命令对输入的源程序文件进行修改，修改完成后，再选择 File 菜单中 Save As 命令保存。编辑完成后，选择 File 菜单中的 Exit 命令退出 EDIT，返回 DOS 系统。

2. 用 MASM 对源程序进行汇编

源程序建立后，可以用 Microsoft 宏汇编程序（MASM.EXE）对它进行汇编。通过汇编生成可重定位的二进制代码目标文件。

汇编过程中汇编程序一般采用两遍扫描的方法。第一遍扫描源程序产生符号表、处理伪指令等；第二遍扫描产生机器指令代码、确定数据等。如果源程序中有语法错误，则汇编结束时汇编程序将指出源程序中的错误。编程者根据出错提示信息修改源程序，直到通过为止。应该指出的是，汇编正确通过，只能说明源程序没有语法错误，但不能说明算法上或其他方面没有问题。

汇编完成后便建立扩展名为.OBJ 的目标文件，并且根据选择也可同时建立.LST 列表文件和.CRF 交叉索引文件。

对源程序进行汇编的过程有以下两种形式。

(1) 提问方式

在 DOS 状态下键入 MASM 命令，调入宏汇编程序之后即在屏幕上显示如下信息：

C:\MASM>MASM

Microsoft® Macro Assembler Version 5.00

Copyright © Microsoft Corp 1981—1985,1987.All rights reserved.

Source filename [.ASM]:Display ; 输入源程序文件名 Display.ASM

Object filename [Display.OBJ]: ; 汇编后得到目标文件名 Display.OBJ

Source Listing [NUL,LST]:Display ; 输出列表文件（可选）

Cross-reference [NUL,CRF]:Display ; 输出交叉索引文件（可选）

0 Warning errors ; 警告性错误数目

0 Severe errors ; 严重错误数目

汇编程序运行后首先显示版本号, 然后依次有 4 个提示。其中, 第一个提示, 询问要汇编的源程序文件名。用户根据提示输入源文件名 Display (可不带 .ASM 扩展名) 后, 出现第二个提示, 询问目标程序文件名, 括号内的信息为系统规定的默认文件名 (即 Display.OBJ), 通常直接按回车键, 表示采用默认文件名。接着出现第三个提示, 询问是否要建立列表文件 (默认为空), 若要, 则键入文件名。若不要, 则直接按回车键。最后发出第四个提示, 询问是否要建立交叉索引文件 (默认为空), 若要, 则键入文件名; 若不要, 则直接按回车键。在回答了第四个询问之后, 汇编程序便对汇编源程序进行汇编。若汇编过程中发现源程序中有语法错误, 则给出错误的行号和错误信息的提示, 最后列出警告错误数及严重错误数。此时, 应分析错误, 然后再调用 EDIT 加以修改, 改正后重新汇编, 直至汇编后无错误为止。

(2) 命令行的方式

命令行格式为:

C: \MASM>MASM 源文件名, 目标文件名, 列表文件名, 交叉索引文件名;

格式中扩展名都可不给出, 汇编程序会按照默认情况使用或产生。若只想对部分提示给出回答, 则在相应位置用逗号隔开; 若不想对剩余部分回答, 则用分号结束。例如, 以下的命令与前面的分行回答是等效的:

C: \MASM>MASM Display, , Display, Display;

该命令行表示调用 MASM 程序对源文件 Display.ASM 进行汇编, 生成目标文件 Display.OBJ, 列表文件 Display.LST 和交叉索引文件 Display.CRF。

再如:

C: \MASM>MASM Display; (后面各项均不列出)

该命令行表示调用 MASM 程序对源文件 Display.ASM 进行汇编, 仅生成目标文件 Display.OBJ。

3. 执行链接程序 (LINK)

用宏汇编程序 MASM 对源程序进行汇编后产生的二进制目标文件 (.OBJ 文件) 还不能直接在 PC 机上运行, 因为在该目标文件中有可浮动的相对地址 (逻辑地址), 必须经过链接程序 (LINK) 连接后才能执行。链接程序 (LINK) 的功能是把一个或多个独立的目标程序模块装配成一个可重定位的可执行文件, 即扩展名为 .EXE 的文件。此外, 还可产生一个内存映像文件, 扩展名为 .MAP。

执行链接程序 LINK 的过程同执行汇编程序 MASM 类似, 也有两种方式。

(1) 提问方式

在 DOS 状态下键入 LINK 回车 (或 LINK Display 回车), 调入链接程序 LINK, 在屏幕上显示版本信息后, 依次提出 4 个问题。以 Display.OBJ 为被连接

的目标文件为例，显示如下：

```
C: \MASM>LINK
```

```
Microsoft ® Overlay Linker Version 3.60
```

```
Copyright © Microsoft Corp 1983-1987. All rights reserved.
```

```
Object Modules [.OBJ]:Display           : 输入目标文件 Display.OBJ
```

```
Run File[DISPLAY.EXE]:                  : 建立 DISPLAY.EXE 文件
```

```
List File [NUL.MAP]:Display             : 建立 Display.MAP 文件
```

```
Libraries [.LIB]                        : 连接其他程序库文件
```

LINK 程序运行后首先询问要连接的目标文件名，操作员键入文件名作为回答（此处为 Display），如果有多个要连接的目标文件，应一次键入。各目标文件名之间用“+”号隔开，最后按回车键。第二个提示，询问要产生的可执行文件的文件名，一般直接按回车键表示采用方括号内规定的默认文件名。第三个提示，询问是否要建立内存地址映像文件，键入文件名再按回车键表示要建立；直接按回车键表示不要建立。最后询问是否用到库文件，如果没有库文件，则直接按回车键即可（如果用户自己建立了库文件，则键入库文件名）。

回答以上询问后，LINK 便开始进行连接。若连接过程有错，则显示错误信息。此时，应根据错误性质，重新调用 EDIT 编辑程序修改源程序，重新汇编、连接，直至无错误为止。

(2) 命令行方式

命令行格式为

```
C: \MASM>LINK 目标文件名, 执行文件名, 内存映象文件名, 库文件名;
```

格式中扩展名都可不给出，LINK 程序会按默认情况使用和产生。若只想对部分提示给出回答，则在相应位置用逗号隔开；若不想对剩余部分作答，则用分号结束。例如：下列命令行与前边的分行回答操作是等效的。

```
C: \MASM>LINK Display, , Display;
```

该命令行表示执行 LINK 程序对目标程序文件 Display.OBJ 进行连接，生成可执行文件 DISPLAY.EXE（默认值）和内存映像文件 Display.MAP。

再如：C: \MASM>LINK Display;

该命令行表示执行 LINK 程序对 Display.OBJ 文件进行连接，仅生成一个可执行文件 DISPLAY.EXE。如果除 DISPLAY.EXE 文件外，还要产生 Display.MAP 文件，则在分号前加两个逗号。如：

```
C: \MASM>LINK Display, , ;
```

运行 LINK 程序后，即产生两个输出文件：一个是可执行文件 DISPLAY.EXE；另一个是扩展名为.MAP 的内存映像文件，它指出每个段在内存中的地址分配情况及长度。通常，在 DOS 状态下用 TYPE 命令显示打印出来。例如：

```
C: \MASM>TYPE Display.MAP
```

Start	Stop	Length	Name
00000H	0000FH	0010H	DATA
00010H	0004FH	0040H	STACK
00050H	0005FH	0010H	CODE

```
Origin      Group
```

```
Program entry point at 0005:0000
```

从内存映像文件可知，源程序 Display 中定义了 3 个段：数据段（DATA）起始地址为 00000H，终止地址为 0000FH，长度为 0010H 个字节；堆栈段（STACK）起始地址为 00010H，终止地址为 0004FH，长度为 0040H 个字节；代码段（CODE）起始地址为 00050H，终止地址为 0005FH，长度为 0010H 个字节。

4. 运行程序

当用链接程序 LINK 将目标程序文件（.OBJ）连接定位成功后，在磁盘上就已生成一个可执行文件（.EXE）。此时，即可在 DOS 状态下运行该文件。其执行操作如下：

```
C: \MASM>DISPLAY    或    C: \MASM>DISPLAY.EXE
```

在源程序 Display 中如果有显示结果的指令，则执行程序后可以在屏幕上或打印机上看到执行结果；如果没有显示结果的命令，要想看到结果，则只有通过 DEBUG 程序来查看了。

13.3 调试程序 DEBUG 及其使用

在编写和运行汇编源程序的过程中，会遇到一些错误和问题，需要对源程序进行分析和调试。DEBUG 是一种交互式的机器语言程序的调试程序，是专为汇编语言设计的一种调试工具。它在调试汇编语言程序时有较强的功能，能使程序设计者接触到机器内部，可以单步执行程序，也可以在程序中设置断点，能观察和修改寄存器和内存单元的内容，并能监视目标程序的执行情况。这样将便于寻找程序的错误。

1. DEBUG 程序的调用

当前以 Windows 为操作系统的 PC 机都带有 DEBUG 调试程序。当 PC 机进入 MS-DOS 工作方式后，就可以调用 DEBUG 调试程序了。

(1) 调用格式

```
C: \MASM>DEBUG [d:] [Path] [filename.ext]
```

其中，[]的内容为可选项；[d:]为驱动器号，指要调入 DEBUG 状态下的可执行文

件所在的驱动器; [Path]为路径, 指要调入 DEBUG 状态下的可执行文件所在的目录或子目录; [filename.ext]指要调入 DEBUG 状态下的可执行文件的文件名, 该文件名的扩展名只能是.EXE 或.COM, 即 DEBUG 只能对扩展名为.EXE 和.COM 的文件进行调试。

在启动 DEBUG 时, 如果输入文件名, 则 DEBUG 程序就把指定文件装入内存, 用户可以通过 DEBUG 的命令对指定文件进行调试。如果没有带文件名, 则可以当前内存的内容工作, 或者用命名命令 (Name) 或装入命令 (Load) 把需要的文件装入内存, 然后再通过 DEBUG 命令进行调试。

当启动 DEBUG 程序成功后, 屏幕上出现“-”提示符, 说明系统已进入 DEBUG 状态。

(2) DEBUG 程序对寄存器和标志位的初始化

在调入 DEBUG 程序后, 各寄存器和标志位置成为以下状态:

- 段寄存器 (CS、DS、ES、SS) 置于自由存储空间的底部, 也就是 DEBUG 程序结束后的第一个段。

- 指令指针 (IP) 置为 0100H。

- 堆栈指针置于段的结尾处或是装入程序的临时底部。哪一个更低, SP 就指向哪个。

- 余下的寄存器 (AX、BX、CX、DX、BP、SI 和 DI) 均置为 0。但是, 若调用 DEBUG 时包含一个要调试的可执行文件名, 则 CX 中包含以字节表示的文件长度, 若文件大于 64KB, 则文件长度包含在 BX 和 CX 中 (高位部分在 BX 中)。

- 标志位都置为清除状态。

- 默认磁盘的传送地址置成代码段的 80H。

注意 若由 DEBUG 调入的程序具有扩展名为.EXE 的文件, 则 DEBUG 必须进行再分配, 把段寄存器、堆栈指针置为程序中所规定的值。

2. DEBUG 命令的有关规定

① DEBUG 命令都是由一个英文字母, 后面跟着一个或多个有关参数组成的。

② 命令和参数可以用大写或小写或混合方式输入。

③ 命令和参数之间以及参数与参数之间可以用定界符 (逗号或空格) 分隔。

如下列命令是等效的:

```
des:100 110
```

```
d cs:100 110
```

```
d,cs:100,110
```

④ 每一个命令, 只有按了回车键以后才有效。

⑤ 参数中的地址和数据均用十六进制数表示, 但十六进制数据后面不跟“H”标志。

⑥ 可以用 Ctrl 和 Break 键来停止一个命令的执行, 返回到 DEBUG 的提示符“-”下。

⑦ 若一个命令产生相当多的输出行, 则为了能在屏幕上看清输出内容, 可在显示过程中用 Ctrl+Num Lock 键暂停上卷动作, 也可通过按任意键来继续上卷动作。

3. DEBUG 的主要命令

(1) 汇编命令 A

格式: A [address]

该命令允许输入汇编语言语句, 并能把它们汇编成机器代码, 相继地存放在从指定地址单元开始的存储区中。若在命令中没有指定地址, 但前面用过汇编命令, 则接着上一个汇编命令汇编结果存放的最后一个单元开始存放。若前面未用过汇编命令, 则从“CS: 100”单元开始存放。若输入的语句中有错误, 则 DEBUG 会显示“Error”信息, 然后重新显示现行的汇编地址, 等待新的输入。

注意 DEBUG 把输入的数字看成十六进制数, 如果要输入十进制数, 则应说明, 如: 100D。

(2) 显示内存单元内容的命令 D

格式: D [address] 或 D [range]

该命令能显示指定范围的内存单元的内容。显示的内容为两种形式: 一种为十六进制内容 (每一字节用两位 16 进制数显示); 另一种是用相应的 ASCII 码字符显示, 句号“.”表示不可显示的字符。

(3) 修改内存单元内容的命令 E

有两种格式:

① E [address] [list]

② E [address]

第①种格式可以用给定的内容表来代替指定存储单元的内容, 如:

-E DS:100 F3'xyz'8D

其中 F3、'x'、'y'、'z'和 8D 各占一个字节, 该命令可以用这 5 个字节来代替内存单元 DS:0100 到 0104 的原先内容。因为内容表中的内容可以是一个字节的十六进制数, 也可以是用单引号括起来的一串字符。

第②种格式则是采用逐个单元相继修改内存单元的内容的方法, 如:

-E CS:100

则可以显示为: 18E4:0100 89 -

如果想把该单元的内容修改为 78, 则可直接键入 78, 再按空格键, 可接着显示下一单元的内容如下:

18E4:0100 89 78 1B -

这样，可不断修改相继内存单元的内容，直至按回车键结束该命令为止。

(4) 检查和修改寄存器内容的命令 R

它有 3 种格式：

- ① R
- ② R [register name]
- ③ RF

第①种格式是显示 CPU 内所有寄存器内容和标志位状态。如：

```
-r
AX=0000 BX=0000 CX=010A DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=18E4 ES=18E4 SS=18E4 CS=18E4 IP=0100  NV UP DI PL NZ NA PO NC
18E4:0100C70604023801  MOV  WORD  PTR[024],0138  DS:0204=0000
```

第②种格式是显示和修改某个寄存器内容，如：

```
-r ax
```

则系统显示如下：

```
AX  F1F4
```

```
:
```

即 AX 寄存器中当前内容为 F1F4，如不修改，则按回车键，否则，可键入欲修改的内容，如：

```
-r bx
```

```
BX  0369
```

```
:059F
```

则把 BX 寄存器的内容修改为 059F。

第③种格式是显示和修改标志位状态，如：

```
-r F
```

则系统显示为：

```
OV  DN  EI  NG  ZR  AC  PE  CY
```

此时，如不修改其内容，则可按回车键，否则，可键入欲修改的内容。

注意 对于状态标志寄存器 FLAG 是以位的形式显示的，8 个状态标志的显示次序和符号见如表 13.1 所示。

表 13.1

标志位	状 态	显示形式（置位/复位）
溢出标志 OF	有/无	OV/NV
方向标志 DF	增/减	DN/UP
中断标志 IF	开/关	EI/DI
符号标志 SF	负/正	NG/PL

续表

标志位	状 态	显示形式 (置位/复位)
零标志 ZF	零/非零	ZR/NZ
辅助进位 AF	有/无	AC/NA
奇偶标志 PF	偶/奇	PE/PO
进位标志 CF	有/无	CY/NC

(5) 跟踪与显示命令 T

有两种格式:

① T [=address] ; 逐条指令跟踪

② T [=address] [value] ; 多条指令跟踪

第①种格式是执行一条指定地址处的指令后停下来, 显示 CPU 所有寄存器的内容和全部标志位的状态。如未指定地址, 则从当前 CS: IP 开始执行。

第②种格式是从指定地址起, 执行 n 条指令后停下来, n 由 value 指定。

(6) 反汇编命令 U

有两种格式:

① U [address]

② U [range]

第①种格式是从指定地址开始, 反汇编 32 个字节。如果命令中没有指定地址, 则从上一个 U 命令的最后一条指令的下一单元开始显示 32 个字节。

第②种格式是对指定范围的存储单元的内容进行反汇编。范围可以由起始地址、结束地址来规定, 也可以由起始地址与长度来规定。

(7) 运行命令 G

格式: G [=address1] [address2 [address3...]]

其中, 地址 1 指定了运行的起始地址, 如不指定, 则从当前 CS: IP 开始运行; 后面的地址均为断点地址, 当指令执行到断点时, 就停止执行并显示当前所有寄存器及标志位的内容和下一条将要执行的指令。

(8) 退出 DEBUG 命令 Q

格式: Q

程序调试完后退出 DEBUG, 返回到 DOS 状态下。

第 14 章 微机接口电路实验

基于 PC 总线微机的本试验装置可进行各种微机应用实验。本章安排了一系列的微机应用单元实验内容,旨在帮助实验者了解和掌握各种 I/O 接口芯片的功能、工作原理和硬件、软件接口技术,培养实验者分析和设计现代微机系统的动手能力,使实验者能做到理论与实践相结合,硬件与软件相结合。为方便实验者,对各实验基本原理及有关芯片作一简单介绍。详细原理见《微机系统与接口技术》等书籍。

需要说明的是:

- ① 实验中如果注明“利用通用插座”的,则说明该芯片不在实验台上,需要另外插上。
- ② 实验中只需要连接相关电路上的虚线部分。
- ③ 实验中不要带电插线。

14.1 可编程定时器/计数器8254(8253) 的原理及应用

14.1.1 实验目的

- ① 掌握 8254 (8253) 的基本工作原理。
- ② 熟悉 8254 (8253) 的各种工作方式。
- ③ 掌握 8254 (8253) 的编程方法,学会 8254 (8253) 的基本应用。

14.1.2 实验预备知识及学习要求

可编程定时器/计数器 8254 是一种在微机系统中实现定时和计数功能的 LSI 外围接口电路。它的内部有 3 个独立的 16 位定时器/计数器。每个计数器都有 6 种工作方式(方式 0~方式 5),由 8254 的控制字来控制。在写入控制字及计数初值后 8254 才能正常工作。由于计数过程中门控信号 GATE 的电平变化、计数初值的重新设置都会对实际计数造成影响,因此最终的 OUT 波形可能各种各样,实现的控

制关系各不相同。为了更好地完成本次实验,要求实验者必须对 8254 (8253) 的工作原理, 8254 (8253) 控制寄存器的格式及作用, 8254 (8253) 的 6 种工作方式的各自特点, 8254 控制字的读回命令与锁存命令的区别, 8254 与 8253 的差别等有清楚的认识。实验过程中, 门控信号 GATE 的电平高低变化可通过电平开关来控制。时钟脉冲 CLK 的发出可通过单脉冲发生器来实现。OUT 的电平变化可用逻辑笔来观察。OUT 的波形可通过其电平变化与时钟脉冲 CLK 的个数关系画出。

14.1.3 实验内容

① 按图 14.1 所示连接电路, 将计数器 0 设置为方式 0, 计数器初值为 N ($N \leq 0FH$), 用手动逐个输入单脉冲, 编程使计数值在屏幕上显示, 并观察在方式 0 下计数器初值写入后计数器的输出端 OUT 电平变化与时钟脉冲(单脉冲)个数的关系。

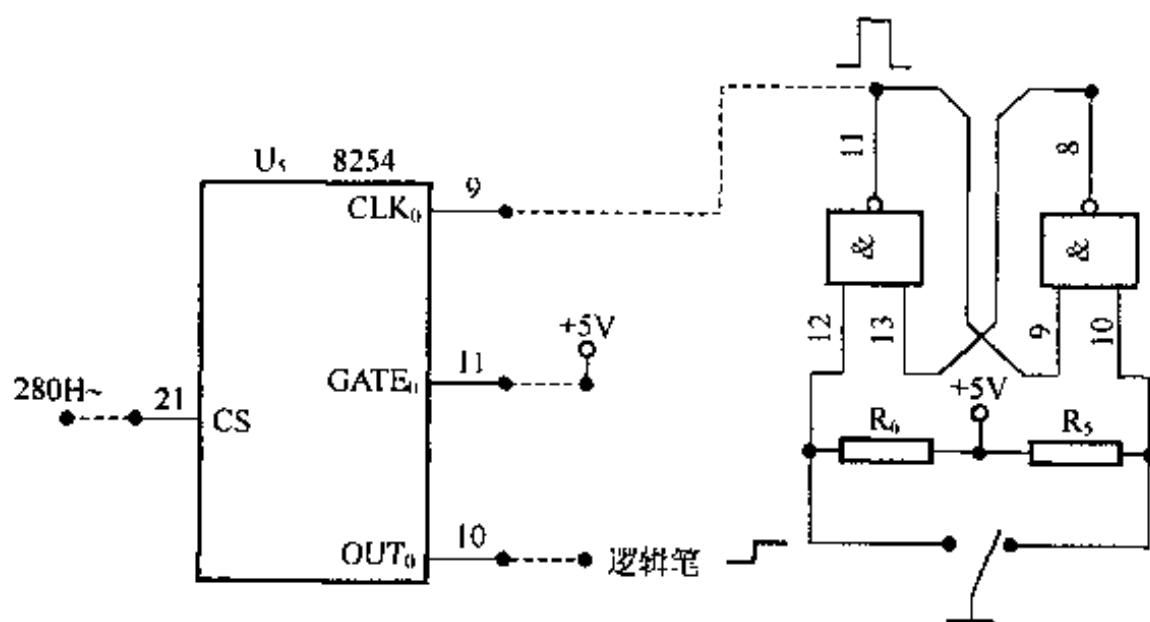


图 14.1 8254 (8253) 方式 0 计数的电路

② 按图 14.2 所示连接电路, 将计数器 0、计数器 1 分别设置为方式 3, 计数初值根据需要设定, 用逻辑笔观察通过计数器串联使用在 OUT_1 端获得的指定频率的电平信号 (定时实验)。

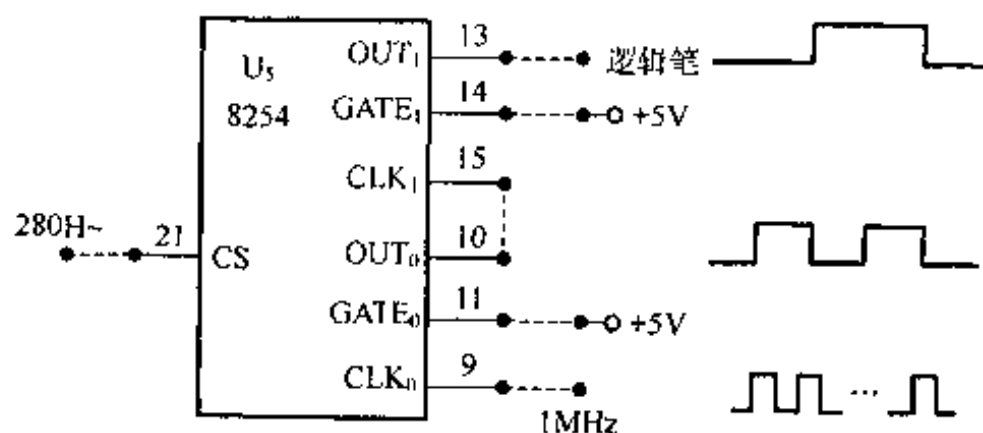


图 14.2 8254(8253)的级连方式电路图

③ 将图 14.1 中 GATE 端输入与电平开关的输出相接, 其他连接电路不变,

根据观察的结果理解在 6 种方式下 GATE 的高电平、低电平和上升沿对计数器工作的影响。

④ 按图 14.1 连接电路, 验证在其他 5 种工作方式下 OUT 端输出电平变化与计数值 N 之间的关系。

⑤ 按实验内容③连接电路, 验证在 6 种工作方式下计数过程中改变计数值的结果。

⑥ 控制字的锁存与读回命令的区别。

14.1.4 编程提示及参考程序流程框图

- | | |
|-----------------------|------|
| ① 8254 (8253) 控制寄存器地址 | 283H |
| 计数器 0 地址 | 280H |
| 计数器 1 地址 | 281H |
| CLK ₀ 连接时钟 | 1MHz |

② 编程中要注意当控制字中设置只读/写高字节或只读/写低字节时, 初始值为 1 字节; 当控制字中设置先读/写低字节, 后读/写高字节时, 初始值为 2 字节, 分两次传送。

③ 实验内容⑥的编程中要注意: (a) 锁存命令只能锁存一个计数器的计数值; (b) 读回命令能够锁存多个计数器的状态值和计数值。当同时锁存状态值和计数值时, 第一条输入指令读回的是状态值, 第二条输入指令读回的是计数值。

④ 参考程序流程如图 14.3、图 14.4、图 14.5 和图 14.6 所示。

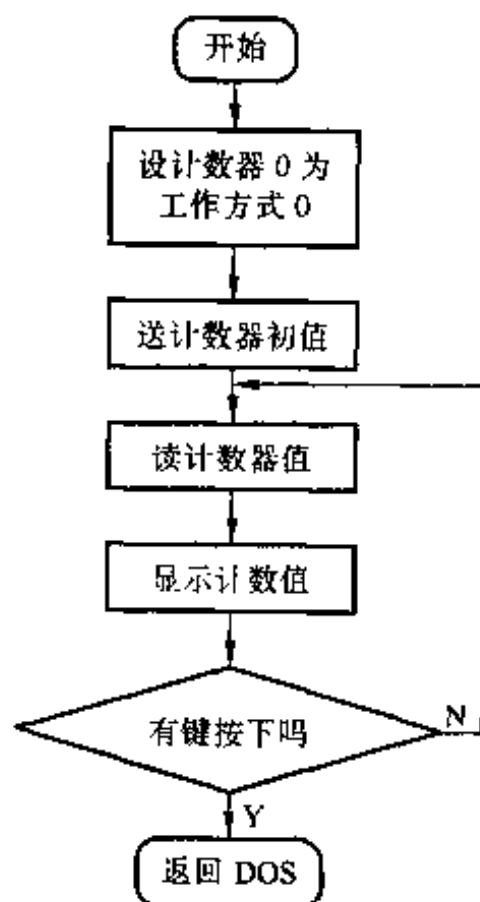


图 14.3 实验①程序流程图

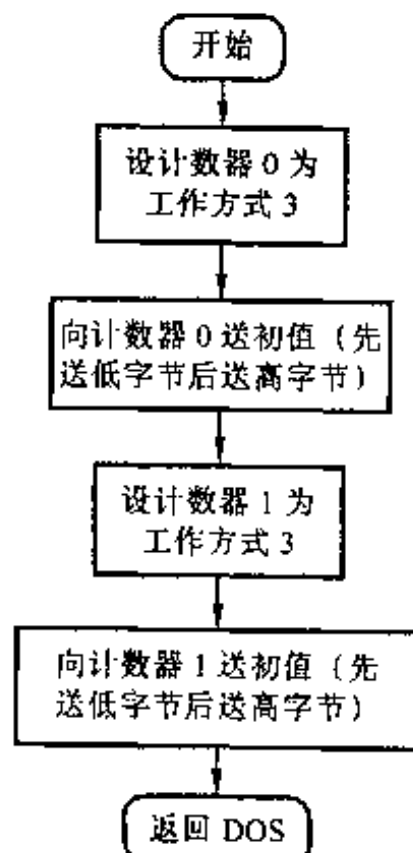


图 14.4 实验②的程序流程图

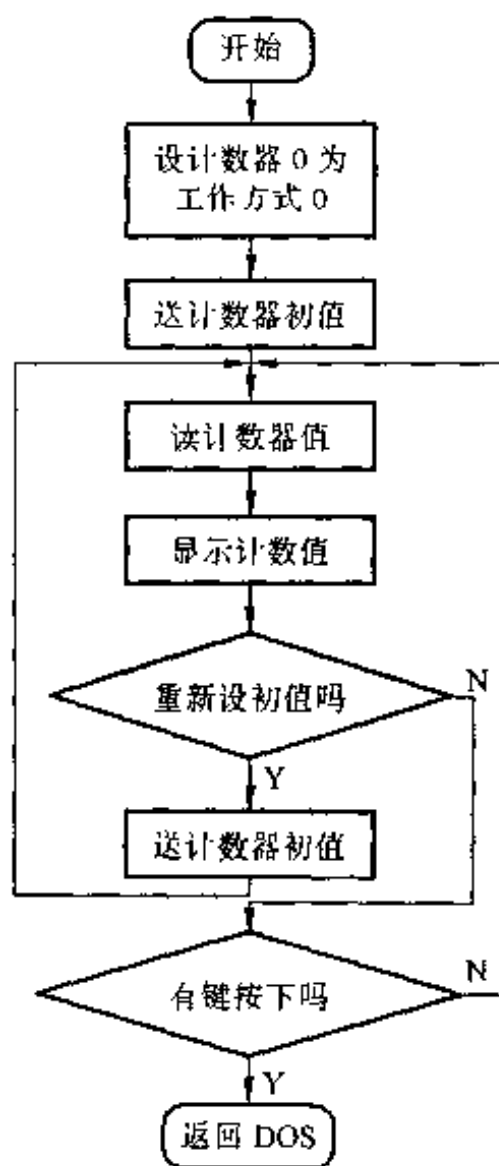
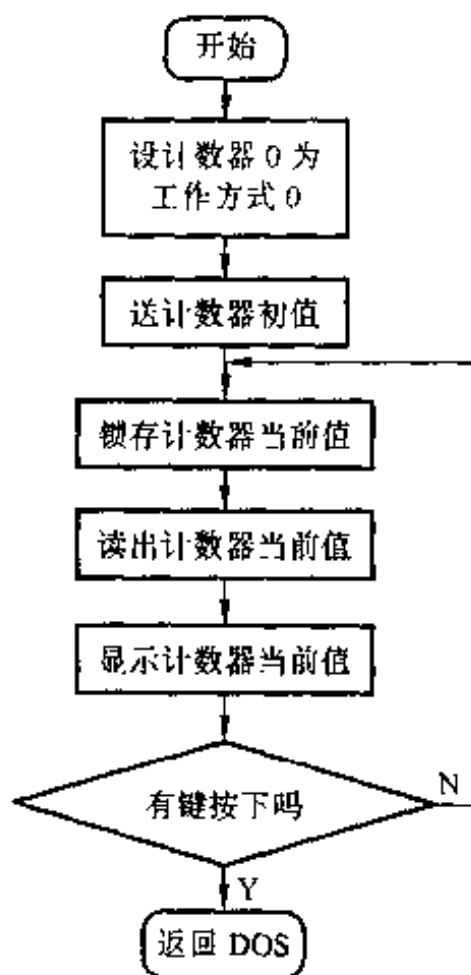
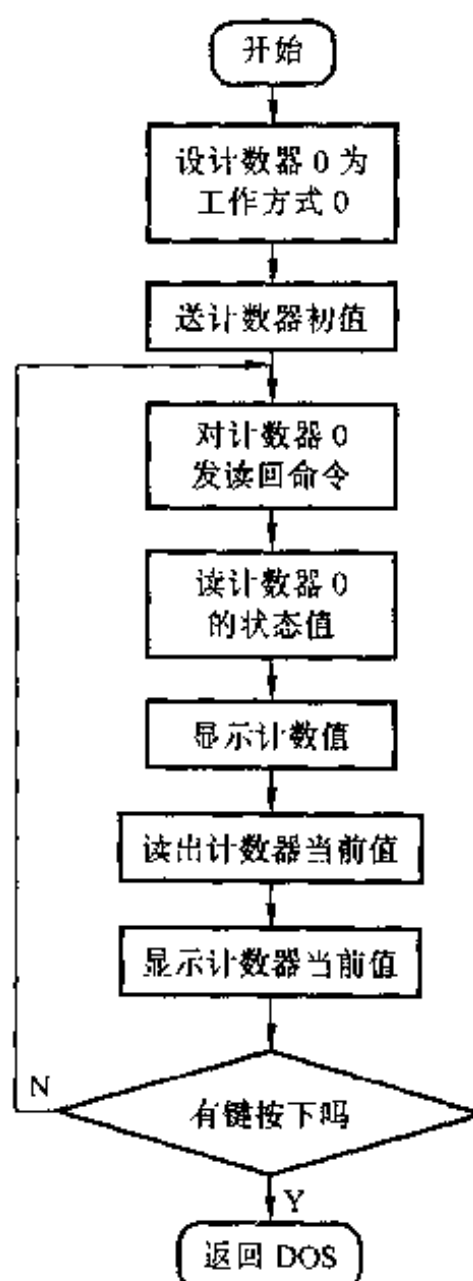


图 14.5



(a) 锁存命令



(b) 读回命令

图 14.6

14.1.5 实验报告要求

① 实验报告主要包括实验目的、实验原理、实验内容、编程实现、实验结果及效果分析等。

② 实验过程中如果出现问题或故障要在报告中说明。

③ 实验内容③和④的结果用表格形式列出。

④ 效果分析中要对实验所涉及到的 8254 (8253) 的各种计数方式的特点、GATE 端电平变化对计数过程的作用、OUT 端输出波形与计数值之间的关系、计数过程中改变计数值的结果及 8254 (8253) 级联方式的特点等进行必要的分析。

14.2 中断控制器8259的工作原理及应用

14.2.1 实验目的

- ① 掌握 PC 机中断处理系统的基本原理和工作过程。
- ② 学会编写中断服务程序。
- ③ 加深对 8259 芯片功能的了解。

14.2.2 实验预备知识及学习要求

可编程中断控制器 8259A 是配合 CPU 进行中断处理的芯片，是专为定时、中断驱动的微机应用系统进行中断控制而设计的。它能提供 8 级中断请求，并且可以方便地构成一个主 8259A，最多带 8 个从 8259A 的系统，可以控制 64 个优先级。

在 PC 机环境下，用户可使用的硬件中断只有可屏蔽中断，由 8259 中断控制器管理。中断控制器用于接受外部的中断请求信号，经过优先级判别等处理后向 CPU 发出可屏蔽中断请求。IBM PC、PC/XT 机内有一片 8259 中断控制器对外可以提供 8 个中断源，其中断类型号和功能如下：

中断源	中断类型号	中断功能
IRQ ₀	08H	时钟
IRQ ₁	09H	键盘
IRQ ₂	0AH	保留
IRQ ₃	0BH	串行口 2
IRQ ₄	0CH	串行口 1
IRQ ₅	0DH	硬盘
IRQ ₆	0EH	软盘
IRQ ₇	0FH	并行打印机

8 个中断源的中断请求信号线 IRQ₀~IRQ₇ 在主机的 62 线总线插座中可以引出。系统已设定中断请求信号为“边沿触发”，普通中断结束方式。对于 286 以上的微机又扩展了一片 8259 中断控制器，可以再提供 IRQ₈~IRQ₁₅ (INT 70H~INT 77H) 的 8 个中断源。此时 IRQ₂ 已用于两片 8259 之间级连。

在 8259A 进入正常工作之前，必须对系统中的 8259A 进行初始化。初始化是通过系统初始化程序设置的。初始化命令字已在微机系统初始化程序中设置完毕，实验者不用另行设置。

考虑到仪器通用性,本实验装置在接口卡上设有一个跳线开关(JP),可以根据不同的需要选择将 IRQ₂、IRQ₃、IRQ₄、IRQ₇引到实验台上的 IRQ 插座上,跳线方法前面已介绍(12.1 节),出厂设置的是 IRQ₇。

为了更好地完成本次实验,要求实验者对可编程中断控制器 8259 的 3 个操作命令字 OCW₁、OCW₂ 和 OCW₃ 的格式及含义,8259 的级联、中断类型号和中断向量的概念、中断优先级、中断屏蔽寄存器 IMR 及普通中断结束方式等有所了解。

14.2.3 实验内容

① 实验电路如图 14.7 所示,直接用单脉冲作为中断请求信号。要求每按一次单脉冲按钮产生一次中断并显示一串字符,中断 N 次后程序退出(中断次数自定)。

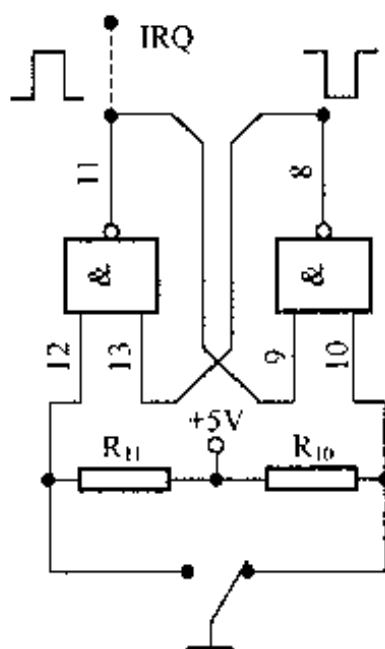


图 14.7 中断电路

② 实验电路如图 14.7 所示,直接用单脉冲作为中断请求信号,每按一次单脉冲按钮产生一次中断。要求在 IRQ₂ 中显示一次“THIS IS A IRQ₂ INTRUPT!”,并在 IRQ₂ 的中断服务程序中用特殊屏蔽方式产生 IRQ₇ 中断,中断的结果是在屏幕上显示一次“THIS IS A IRQ₇ INTRUPT!”。

说明:在实验开始前先将接口卡的跳线开关(JP)跳到 IRQ₂ 上,产生 IRQ₂ 中断后再将跳线开关跳到 IRQ₇ 上以便产生 IRQ₇ 中断。

③ 实验电路如图 14.7 所示,直接用单脉冲作为中断请求信号,每按一次单脉冲按钮产生一次中断。要求用查询中断方式检查请求中断的设备,并在屏幕上显示当前请求服务的最高中断优先级的中断源。

14.2.4 编程提示及参考程序流程框图

① 本实验采用的是 PC 机内的 8259A,其偶地址为 20H,奇地址为 21H。

② 实验者在进行实验内容①的编程时要根据中断类型号设置中断向量,在设置中断向量时要首先将原有的中断向量用功能 35H 读取并保存,在程序退出时,再用功能 25H 恢复。此外,8259A 中断屏蔽寄存器 IMR 对应位要清零(允许中断),中断服务结束返回前要使用中断结束命令:

```
MOV    AL, 20H
```

```
OUT    20H, AL
```

中断结束返回 DOS 时应将 IMR 对应位置 1,以关闭中断。

③ 编制实验内容②中 IRQ₂ 的中断服务程序。在程序中某处可先把 IRQ₂ 屏蔽,然后把 ESMM 和 SMM 都置“1”进入特殊屏蔽方式。在特殊屏蔽方式中显示这是 IRQ₇ 中断。在退出特殊屏蔽方式时先把 ESMM 置“1”、SMM 置“0”后允许 IRQ₂ 中断,然后退出特殊屏蔽方式回到一般屏蔽方式。程序中 IRQ₂ 的主程序流程可参考图 14.8 (a)。IRQ₇ 的中断控制流程如图 14.8 所示。

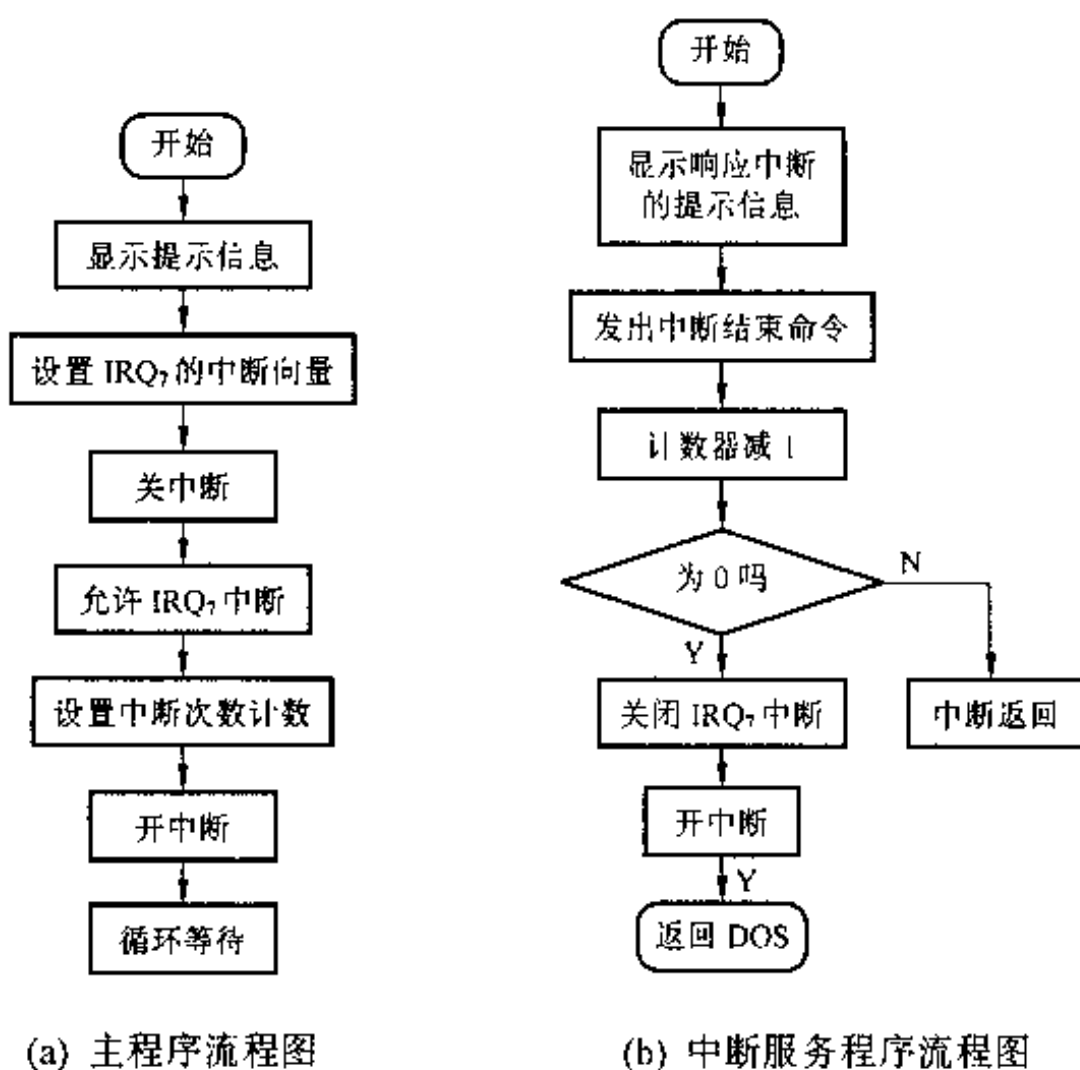


图 14.8 中断控制流程图

④ 对实验内容③的编程提示:先编制单脉冲产生中断程序,然后再将 OCW₃ 中的 P 位置“1”,用“IN 20H, AL”读出查询字,最后在屏幕上显示最高中断优先级的中断源。

⑤ 参考程序流程如图 14.8、图 14.9 和图 14.10 所示。

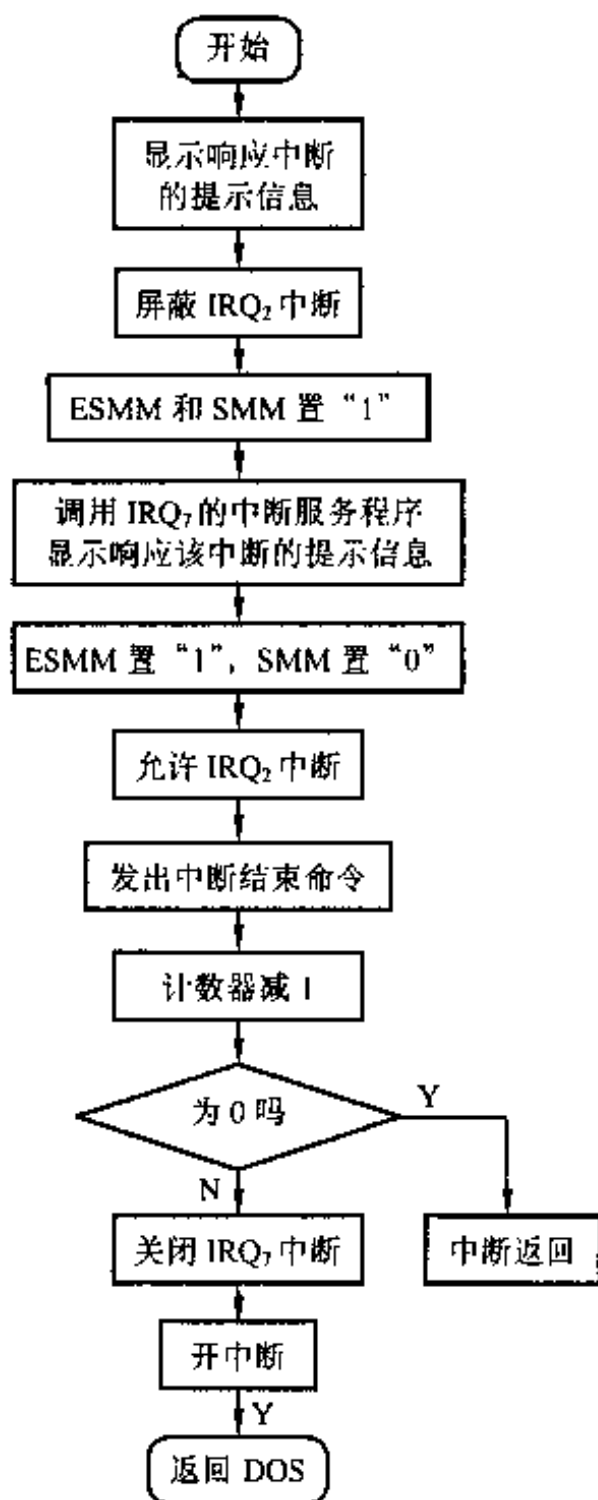


图 14.9 特殊屏蔽中断服务程序流程图

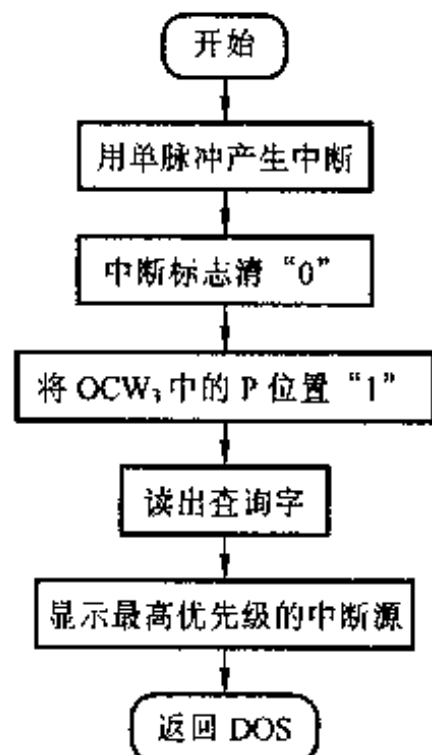


图 14.10 查询最高中断优先级的程序流程图

14.2.5 实验报告要求

①和②的要求同 14.1.5 小节中的①和②。

③ 在对实验进行分析和总结时，要力求全面地对 8259 的编程方法及实验所编程序（主程序和中断服务程序）进行比较分析。

14.3 可编程并行接口 8255 的原理及应用

14.3.1 实验目的

① 熟悉 8255 的各种工作方式及使用方法。

② 掌握 8255 方式 0 及方式 1 的工作原理及编程方法。

③ 进一步掌握中断处理程序的编写。

14.3.2 实验预备知识及学习要求

可编程并行接口 8255 是一个具有两个 8 位 (A 端口和 B 端口) 和两个 4 位 (C 端口) 并行 I/O 端口的芯片。在与外设进行数据传输时, 把 A、B、C 3 个端口分为两组。A 组由 A 端口和 C 端口的高 4 位组成。B 组由 B 端口和 C 端口的低 4 位组成。为了满足多种数据传输的要求, 可以通过对 8255 的编程用方式控制字设置 3 种工作方式来实现。这 3 种工作方式为: 方式 0 (基本 I/O 工作方式); 方式 1 (选通 I/O 工作方式); 方式 2 (双向传送方式)。8255 的控制字有工作方式控制字和 C 端口的位置位/复位控制字。工作方式控制字是必须要预先设定的, C 端口的位置位/复位控制字可视需要而定。一般来说, 在方式 0 中, C 端口除在特殊场合用作联络信号外, 如: 双机通信, 基本上都作为数据端口参与 I/O 操作。但在方式 1 或方式 2 下, C 端口的相应位可用来作为 I/O 操作的控制和同步信号, 也可用作对 CPU 的中断请求信号。

为了更好地完成本次实验, 要求实验者掌握 8255 控制寄存器, 8255 方式 0、方式 1 的工作原理及工作过程; 熟悉方式 1 (输入和输出) 下 A 端口的的方式字、C 端口的的位置位/复位控制字以及状态字的设置方法等。

14.3.3 实验内容

① 实验电路如图 14.11 所示, 8255 的 C 端口接逻辑电平开关 $K_0 \sim K_7$, A 端口接 LED 显示电路 $L_0 \sim L_7$ 。编程实现在方式 0 下从 8255 的 C 端口输入数据, 再从 A 端口输出的过程。

② 按图 14.12 (a) 所示的 8255 方式 1 的输出电路连好线路, 用每按动一次单脉冲按钮产生一个正脉冲使 8255 向 CPU 产生一次中断请求, 让 CPU 进行一次中断服务, 编程让每次中断输出一个值使对应的显示灯 ($L_0 \sim L_7$) 依次发光 (中断 8 次结束)。读 8255 的 A 端口方式 1 输出时的状态字。

③ 按图 14.12 (b) 所示的 8255 方式 1 的输入电路连好线路, 用每按动一次单脉冲按钮产生一个正脉冲使 8255 向 CPU 产生一次中断请求, 让 CPU 进行一次中断服务, 编程在屏幕上显示逻辑电平开关预置的 ASCII 码值的对应字符 (中断

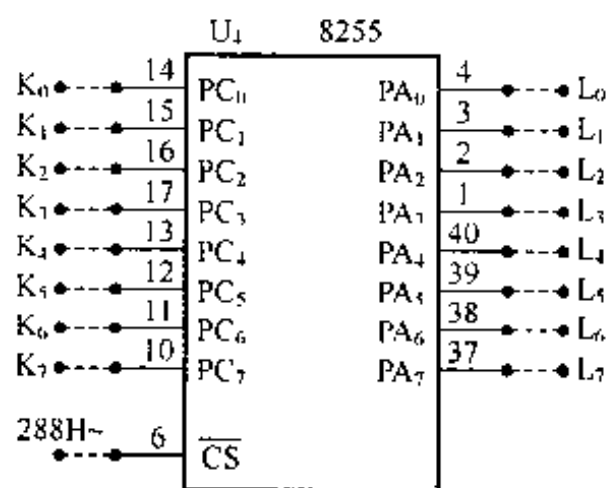


图 14.11 8255 方式 0 的输入/输出电路

次数自定)。读 8255 的 A 端口方式 1 输入时的状态字。

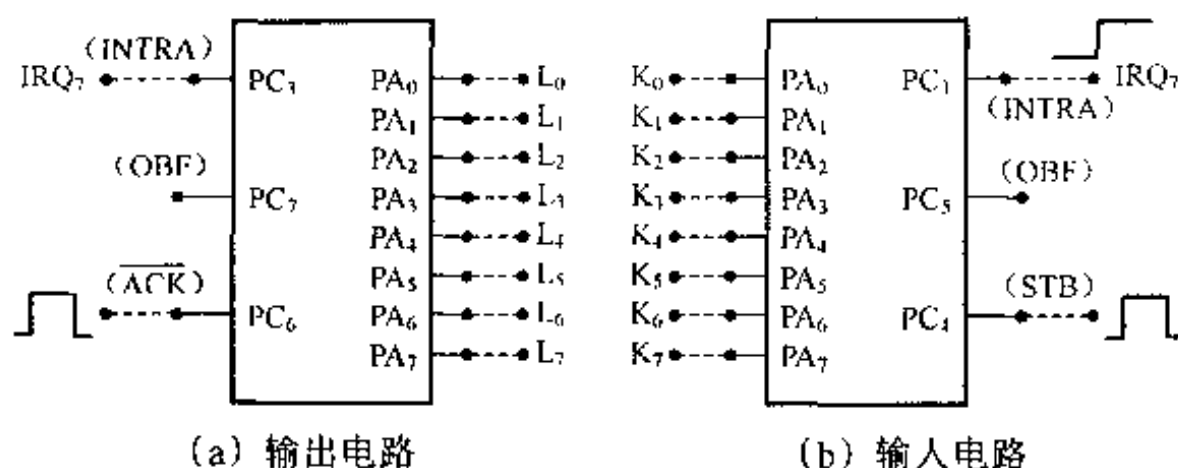


图 14.12 8255 方式 1 的输入/输出电路

14.3.4 编程提示及参考程序流程框图

- ① 8255 控制寄存器端口地址 28BH
 A 端口的地址 288H
 C 端口的地址 28AH

8255 的方式控制字和位置位/复位控制字共用一个端口地址。

② 本次实验内容②和③的编程可参照 8259 中断实验进行。8255 方式 1 输出和输入都分为主程序和中断服务程序两部分。

③ 参考程序流程如图 14.13、图 14.14 和图 14.15 所示。

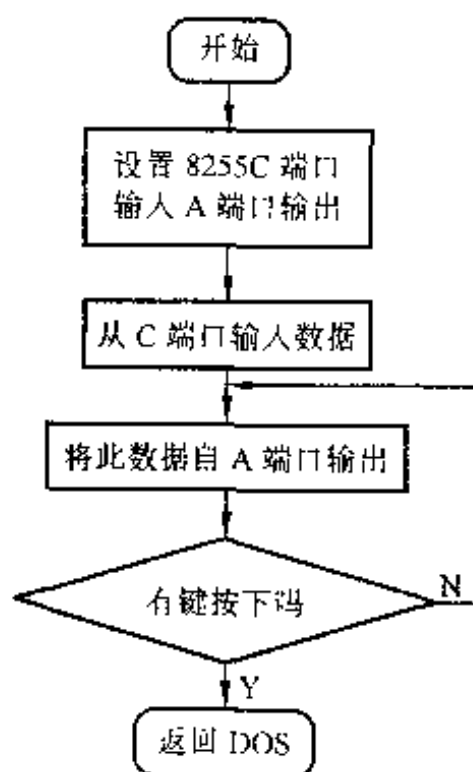


图 14.13 8255 方式 0 程序流程框图

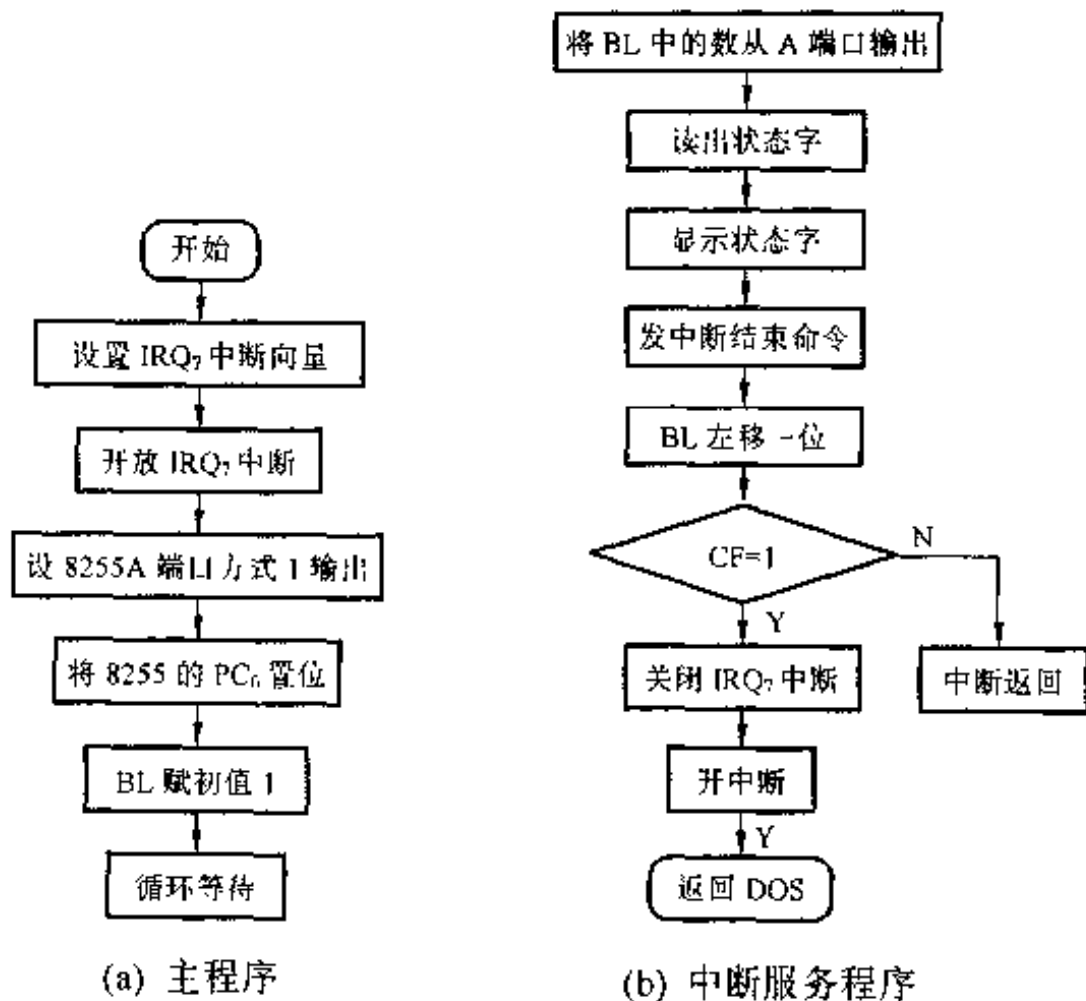


图 14.14 8255 方式 1 输出时的程序流程框图

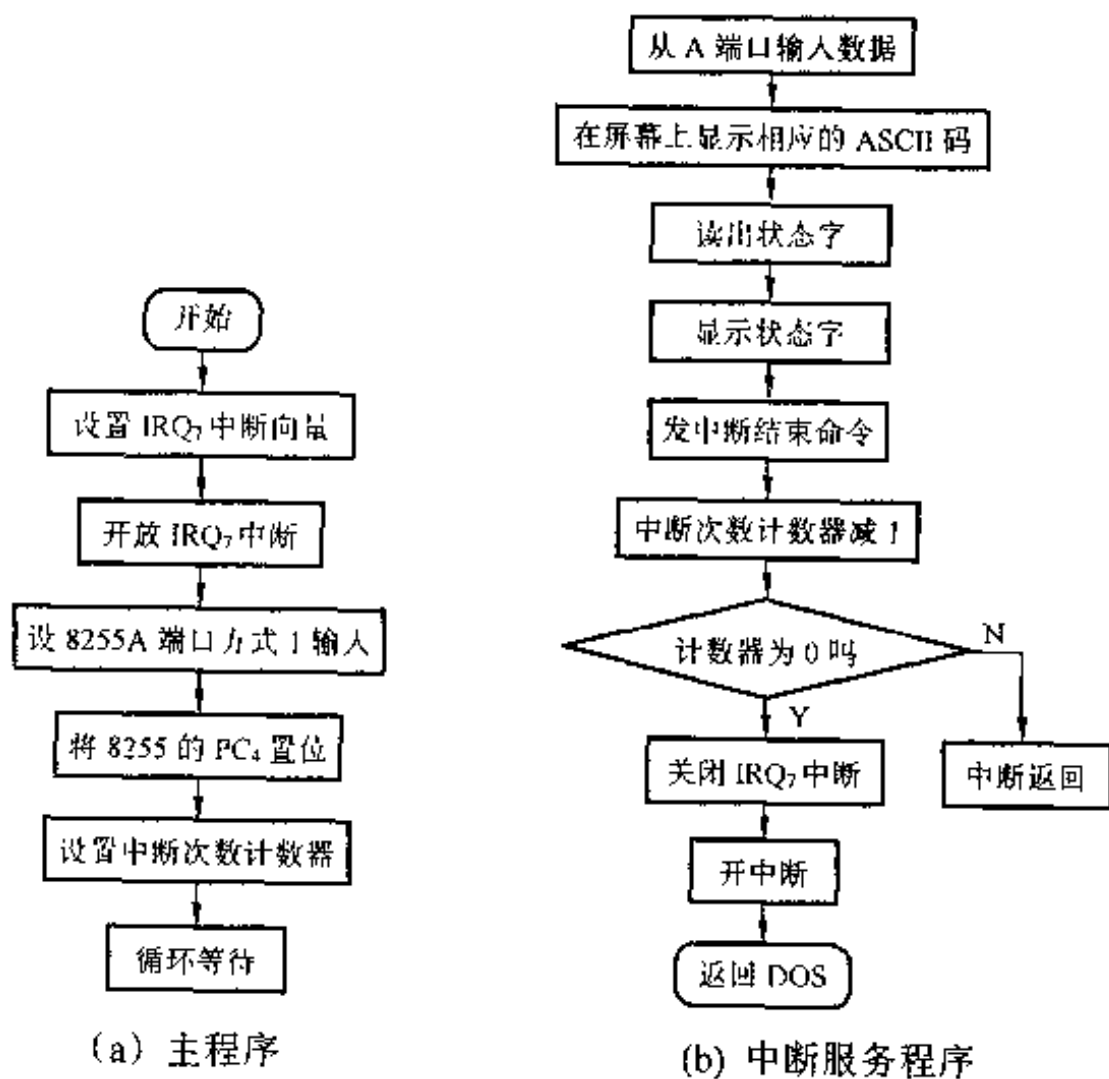


图 14.15 8255 方式 1 输入时的程序流程框图

14.3.5 实验报告要求

①和②的要求同14.1.5小节的实验报告要求①和②。

③ 分析和总结8255方式0的工作特点。

④ 分析和总结8255方式1的工作特点，特别是输入和输出时各信号之间的配合关系。分析8255方式1时通过C端口读回的状态字。

14.4 串行通信

14.4.1 实验目的

① 了解串行通信的基本原理。

② 掌握串行接口芯片8250的功能和接口方法。

③ 学会串行通信程序的编制。掌握8250查询和中断方式的编程方法。

④ 学会8250间的相互通信。

14.4.2 实验预备知识及学习要求

8250是一个可编程异步通信单元芯片，在微机系统中起串行数据的I/O接口作用。此外，它还包含有可程序波特率发生器，它可用1~65535的因子对输入时钟进行分频，以产生波特率16倍的用于进行8250传送或接收串行数据时所使用的时钟信号的频率。

8250片内有10个寄存器。其中两个波特率因子寄存器（也称为除数锁存器）和其他寄存器共用地址，在寻址除数锁存器时设置了特征位，使线路控制寄存器的最高位，即除数锁存器访问位DLAB的值为1。此时，执行的I/O指令将访问除数锁存器（DLL和DLH）。当DLAB为0时，寻址波特率因子寄存器以外的寄存器。8250内部寄存器的详细寻址情况如表14.1所示。表中还列出了IBM PC/XT系列机中异步串行通信口COM₁和COM₂所用8250各寄存器的物理地址。

从功能上分，8250的10个寄存器可分为以下两组。一组用于工作方式、通信参数的控制和设置，在8250初始化时用OUT指令设置。这包括：除数锁存器（DLL和DLH）、线路控制寄存器、Modem控制寄存器和中断允许寄存器。另一组用于实现通信传输，由接收缓冲器和发送保持寄存器、线路状态寄存器、Modem状态寄存器、中断识别寄存器等组成。

在数据传送的查询传送方式下，PC机通过读取线路状态寄存器的相应位的内容来决定是从8250输入字符还是向8250输出字符。如要从8250输入字符，则先测试线路状态寄存器的第0位是否为1，若为1，就表明数据已准备就绪，应尽快

表 14.1

地址信号 $A_2 A_1 A_0$	标志位 DLAB	COM ₁ /COM ₂ 地址 (H)	寄 存 器
0 0 0	0	3F8/2F8	写发送保持寄存器/读接收缓冲寄存器 (THR/RBR)
0 0 0	1	3F8/2F8	除数锁存器低 8 位 (DLL)
0 0 1	1	3F9/2F9	除数锁存器高 8 位 (DLH)
0 0 1	0	3F9/2F9	中断允许寄存器 (IER)
0 1 0	×	3FA/2FA	中断识别寄存器(IIR)
0 1 1	×	3FB/2FB	线路控制寄存器(LCR)
1 0 0	×	3FC/2FC	Modem 控制寄存器(MCR)
1 0 1	×	3FD/2FD	线路状态寄存器 (LSR)
1 1 0	×	3FE/2FE	Modem 状态寄存器(MSR)
1 1 1	×	3FF/2FF	专用

读走。当向 8250 输出字符时, 则先测试线路状态寄存器的第 5 位, 若为 1, 则表明可以发送数据了。

8250 除了用查询传送方式通信外, 还可用中断方式。因 PC 机中 8250 的中断输出受 OUT_2 信号控制, 所以要先对 Modem 控制寄存器的 OUT_2 位置“1”, 使 8250 可工作在中断方式下。这样, 8250 的中断请求信号通过系统总线作为 8259 的 IRQ_4 端输入。

8250 芯片可以处理 4 种类型的中断。中断允许寄存器的低 4 位分别对应这 4 种中断源。由于同时可能有多个中断源满足条件, 所以还要通过中断标识寄存器按优先级的高低来处理各个中断。

本实验是一个对 8250 各种功能进行测试的较齐全的实验, 有一定的复杂度。实验者必须对串行接口的异步通信方式、可编程异步串行接口 8250 的工作原理、8250 内部寄存器、8250 初始化、数据传送的查询传送方式、8250 收发通信、8250 的各种中断功能的实现等内容有正确的理解。

14.4.3 实验内容

① 按图 14.16 所示连接线路, 图中 8250 芯片插在 40 芯通用插座上。编程实现在回送方式下以查询状态自发自收的程序 (从键盘输入一个字符, 将其 ASCII 码发送再接收并在屏幕上显示)。

② 直接使用微机内的 8250 芯片进行本次实验, 编程实现在回送方式下 8250 的发送、接收、Modem 状态变化中断功能的测试。

③ 在两个实验台上分别将 8250 插入各自的通用插座 D 中的 40 芯通用插座上。分别按图 14.16 所示连接好电路, 其中 8250 的发送和接收时钟都接 2MHz 的输出插孔。最后将两个 8250 芯片之间相连实现相互通信 (连接方式自行考虑)。要求编程实现从主机方 (自定) 的键盘输入一个字符, 将其 ASCII 码加 1 后发送

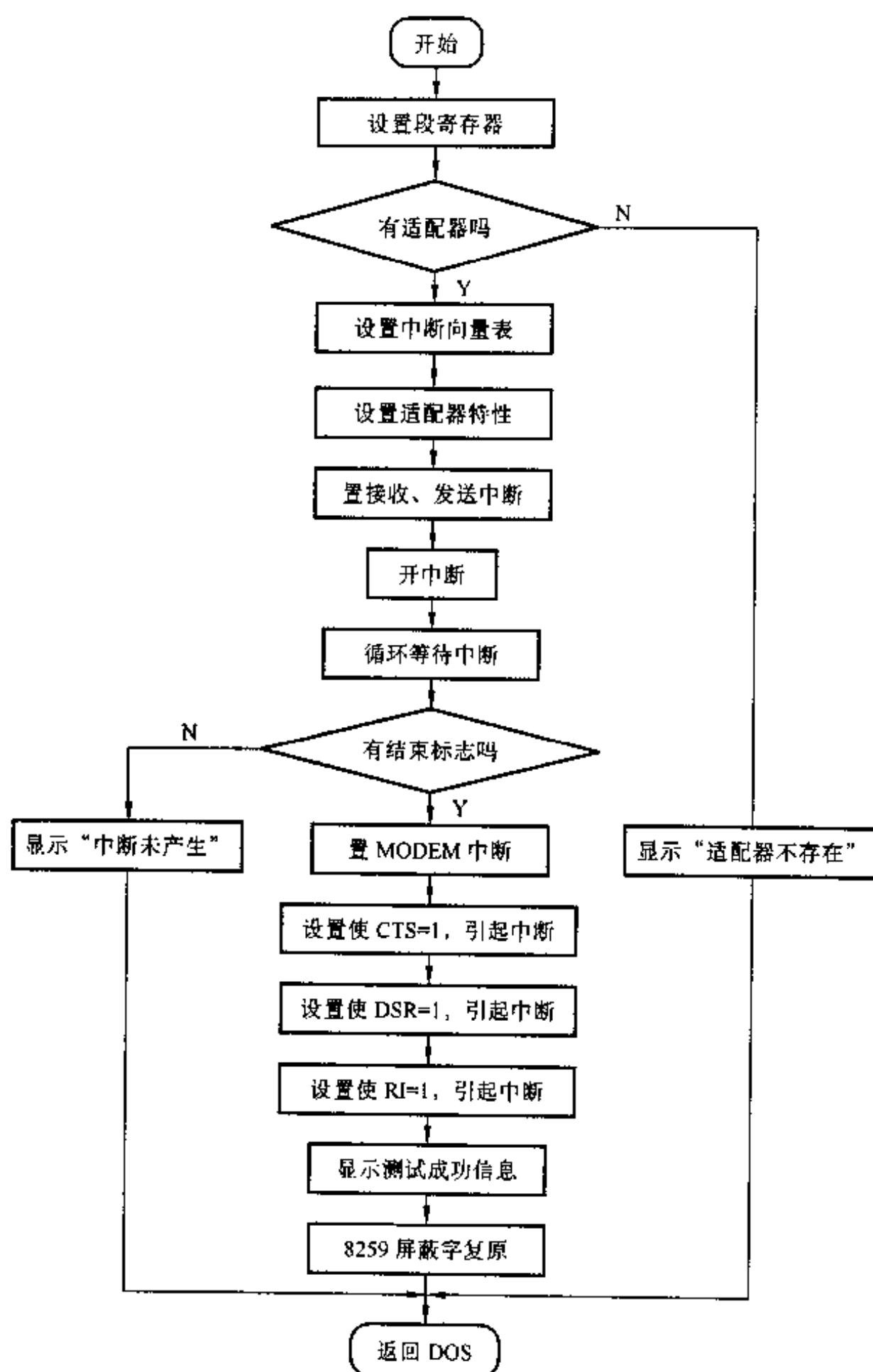


图 14.18 回送方式下 8250 各种中断功能的测试流程框图——主程序

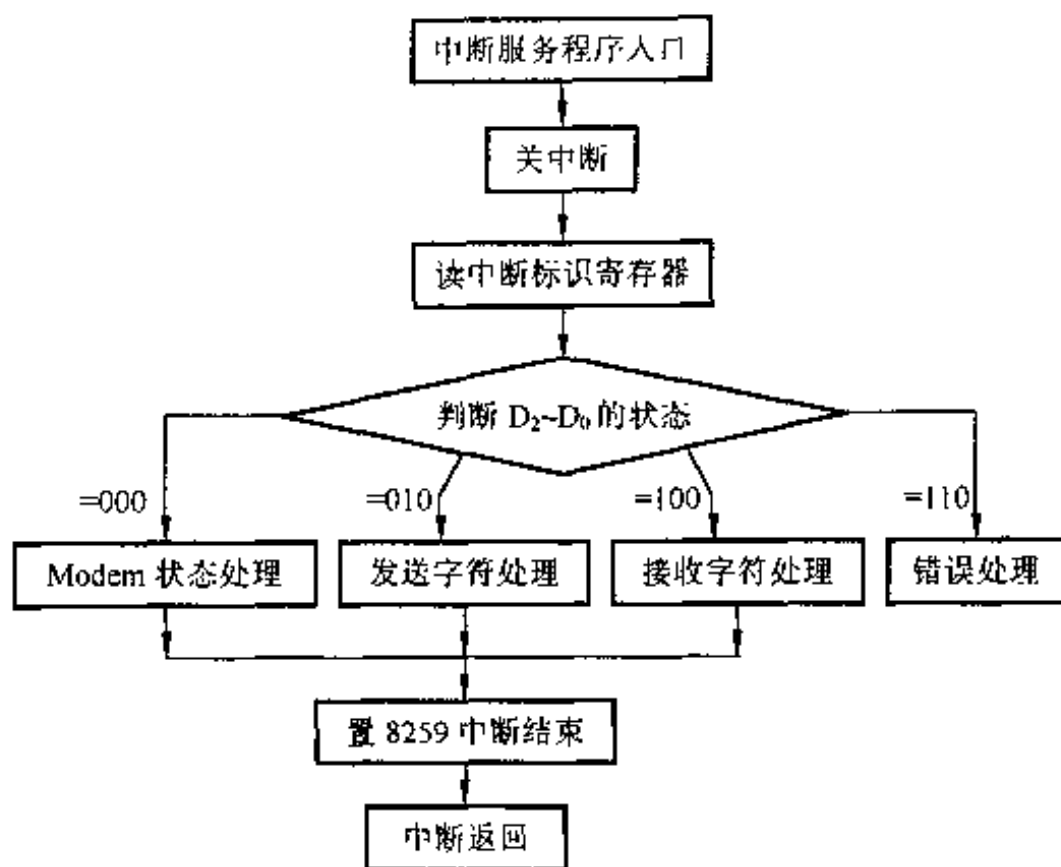


图 14.19 回送方式下 8255 各种中断功能的测试流程框图——中断服务程序

14.5 数/模 (D/A) 转换器及应用

14.5.1 实验目的

- ① 了解 D/A 转换器的基本原理及工作过程。
- ② 掌握 DAC 0832 芯片的使用方法。
- ③ 学会 D/A 转换的编程。

14.5.2 实验预备知识及学习要求

D/A 转换器的作用是将二进制数字量转换为相应的模拟量输出, 从而实现计算机对一个物理过程的控制。

DAC0832 芯片为 8 位电流 DAC 器件, 属于 CMOS 器件, 具有 8 位分辨率。该芯片可直接与微处理器相连。现将 DAC0832 芯片的引脚信号说明如下:

$DI_0 \sim DI_7$: 8 位数据输入端。

$\overline{CS}$: 片选信号, 用于选通输入寄存器。

$\overline{ILE}$: 输入数据允许锁存信号。

$\overline{XFER}$: 数据传送控制信号。

$\overline{WR}_1 / \overline{WR}_2$ ：输入寄存器/DAC 寄存器写选通信号。其中：信号 $\overline{WR}_1$ 、 $\overline{CS}$ 和 $\overline{ILE}$ 的逻辑组合产生输入寄存器的锁存信号； $\overline{XFER}$ 、 $\overline{WR}_2$ 的逻辑组合产生 DAC 寄存器的锁存信号。

U_{REF} ：外接参考电压。

R_{FB} ：内部反馈电阻。

I_{OUT1} ：电流输出端 1，它是为 1 的各位权电流的汇集输出端。

I_{OUT2} ：电流输出端 2，它是为 0 的各位权电流的汇集输出端。

$$I_{OUT1} + I_{OUT2} = \text{常数}$$

DAC 0832 有两级 8 位缓冲器，可以控制使其工作在 3 种方式下：①输入/输出直通方式；②单缓冲器方式；③双缓冲器同步方式。其中，②和③方式是 DAC 0832 与微机接口最常用的两种基本接口方式。其工作过程是：输入数据先通过 8 位输入寄存器，再送到 8 位 DAC 寄存器，最后经 8 位 D/A 转换器转换成差动的电流输出。DAC 0832 可通过外接运算放大器把转换后的电流变成电压输出，从而可工作在单/双极性输出情况下。

单缓冲器方式多用于只有一路 D/A 转换或虽是多路转换但不要求同步输出的情况。此时，两级缓冲器只有一级具有数据锁存功能。双缓冲器同步方式常用于多路 D/A 转换，而且要求同步进行 D/A 转换输出的情况。在这种方式下，双缓冲器分时锁存，当 DAC 寄存器锁存的数据被转换时，输入寄存器可锁存下次待转换的数据，从而加快了数据采集速度。

本次实验中， $\overline{ILE}$ 接高电平，当 $\overline{CS}$ 和 $\overline{WR}_1$ 信号到来时，数据总线送来的数据就直接输入寄存器； $\overline{WR}_2$ 和 $\overline{XFER}$ 接地，此时，DAC 寄存器为不锁存状态，输入数据可直接进行 D/A 转换。当 $\overline{WR}_1$ 信号变高时，此数据就被锁存到输入寄存器中，因此 D/A 转换的输出也保持不变。

为了更好地完成本次实验，要求实验者掌握 D/A 转换的基本原理，DAC 0832 芯片的特性，万用表、示波器的使用方法，debug 命令及应用等。

14.5.3 实验内容

① 实验电路如图 14.20 所示，DAC 0832 采用单缓冲方式，具有双极性输出端（图中的 U_a 和 U_b ），利用 debug 输出命令（O 290，数据）输出数据给 DAC 0832。用万用表测量单极性输出端 U_a 及双极性输出端 U_b 的电压，验证数字与电压的线性关系。

② 编程产生连续周期波形（从 U_b 输出，用示波器观察）：

锯齿波；

正弦波；

如此不断重复可得到该波形。

⑥ 三角波的产生可参照梯形波，但只需输出上升段和下降段。

⑦ 参考程序流程如图 14.21 和图 14.22 所示（只给出锯齿波和正弦波，其他波形可参照完成）。

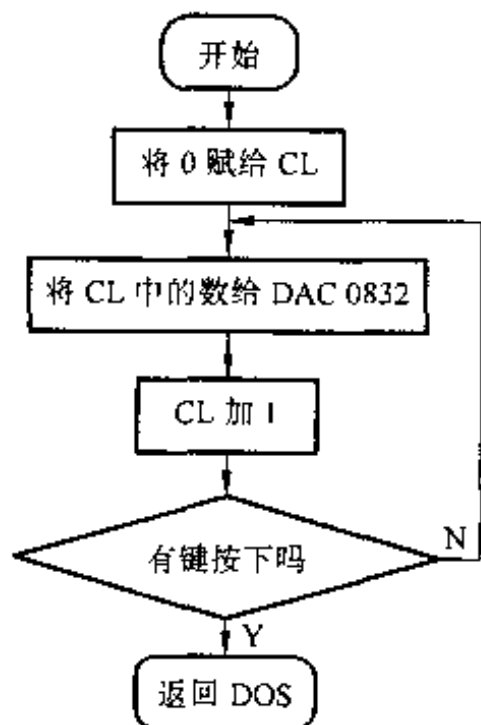


图 14.21 锯齿波流程图

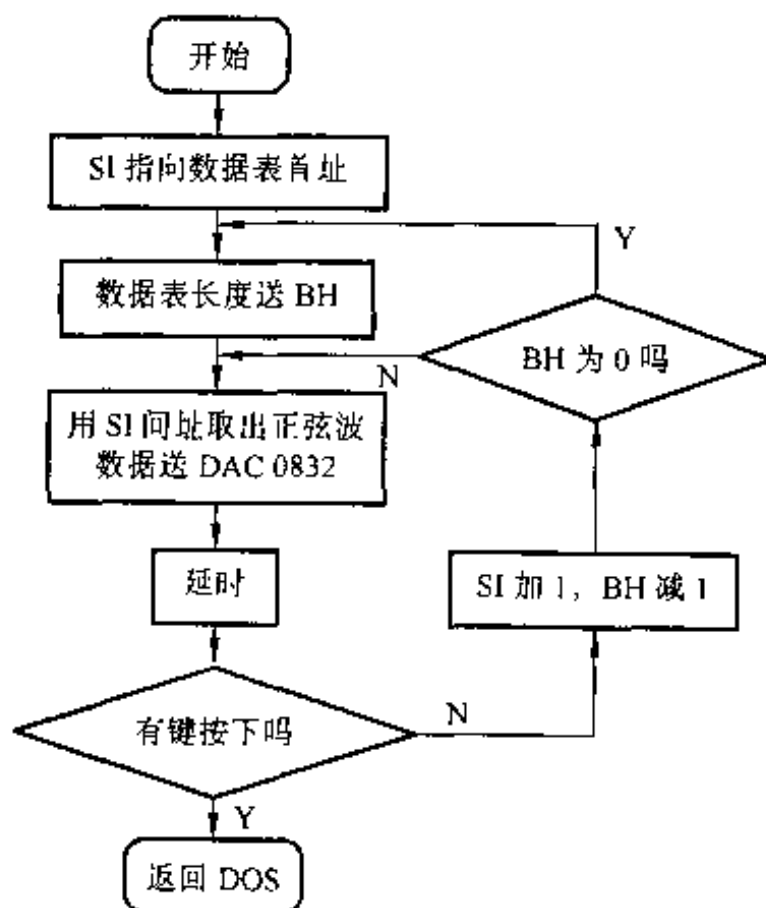


图 14.22 正弦波流程图

14.5.5 实验报告要求

①和②的要求同 14.1.5 小节的实验报告要求①和②。

③ 实验内容①的结果用表格表示，并根据此表画出数字和电压的关系图。分析栏中写出与理论公式的比较。

④ 分析与小结 DAC 0832 的应用编程。

14.6 模/数 (A/D) 转换器及应用

14.6.1 实验目的

① 了解 A/D 转换的基本原理及工作过程。

② 掌握 ADC 0809 的使用方法。

③ 学会 A/D 转换的编程。

14.6.2 实验预备知识及学习要求

由于微机只能处理数字化的信息，而在实际应用中被控对象常常是连续变换的物理量，因此，微机用于测控系统时需要有能把模拟信号转换成数字信号的接口，以便于能对被控对象进行处理和控制。A/D转换器就承担这样的任务，它适用于工业自动化控制、数据采集等许多领域。

ADC 0809 是一种 CMOS 单片 8 位 A/D 转换器，8 路模拟量输入以及地址锁存与译码。设有与微机数据总线相连的 TTL 三态输出锁存器。ADC 0809 可用单一的+5V 电源工作，转换时间约为 $100\mu\text{s}$ 。当用单一+5V 电源时，模拟量输入量程为 0~5V，对应的转换值为 00H~FFH。

ADC 0809 的主要引脚信号说明如下：

IN₀~IN₇：8 路模拟量输入端（此处只画出 IN₀~IN₂）。

ADD-A、ADD-B、ADD-C：三位地址线，通过地址译码选通 8 路模拟量输入端中的一路。

CLOCK：外部提供给 ADC 0809 工作的时钟信号。

EOC：A/D 转换结束信号。

ALE：通道地址锁存允许信号。

ENABLE：输出允许信号，用来打开三态输出的数据锁存器。

START：A/D 转换启动信号。

REF（+）、REF（-）：正的和负的参考电压。

ADC 0809 属于采用逐次逼近法的 A/D 转换类型的转换器。采用逐次逼近法的 A/D 转换的工作过程是：当启动脉冲到来后，控制逻辑首先使 N 位（这里 N=8）逐次逼近寄存器（SAR）的最高位置“1”，其余位清“0”。然后将该值送 D/A 转换器。经 D/A 转换后的输出电压即为满量程电压的 1/2（设为 V_h ）。将输入电压 V_i 与 V_h 比较，若 $V_i > V_h$ ，则最高位不变；若 $V_i < V_h$ ，则最高位清“0”。一次比较完成后，将 SAR 的次高位置“1”（前次高位的值不变）送 D/A 转换，新的 V_h 再同 V_i 比较来决定该位为“1”还是为“0”……上述过程重复进行直到最低位为止。经 N 次操作后，控制逻辑输出一个转换结束信号，控制缓冲寄存器接收 SAR 的内容，即本次 A/D 转换的结果。该缓冲寄存器的输出接数据总线。

ADC 0809 转换由 START 脉冲信号来启动，脉冲下降沿有效（转换开始）。当输入通道选择地址线状态稳定后，在 ALE 信号的上升沿将地址线的状态锁存到芯片的地址锁存器中。在转换操作过程中，信号 EOC 保持低电平，当转换结束时变为高电平。该信号主要用来查询 A/D 转换是否结束或者用来作为中断请求信号。当 ENABLE 被置为高电平时，三态门打开，将数据锁存器的内容输出到数据总线上。

为了更好地完成本次实验,要求实验者掌握 A/D 转换的基本原理,ADC 0809 芯片的特性和使用方法。

14.6.3 实验内容

① 实验电路如图 14.23 所示。通过实验台左下角电位器 RW_1 将输出的 0~5V 直流电压送入 ADC 0809 的 IN_0 通道,利用 debug 的输出命令启动 A/D 转换器 (O 298, 0), 输入命令读取转换结果 (I 298), 验证输入电压与转换后数字的关系。

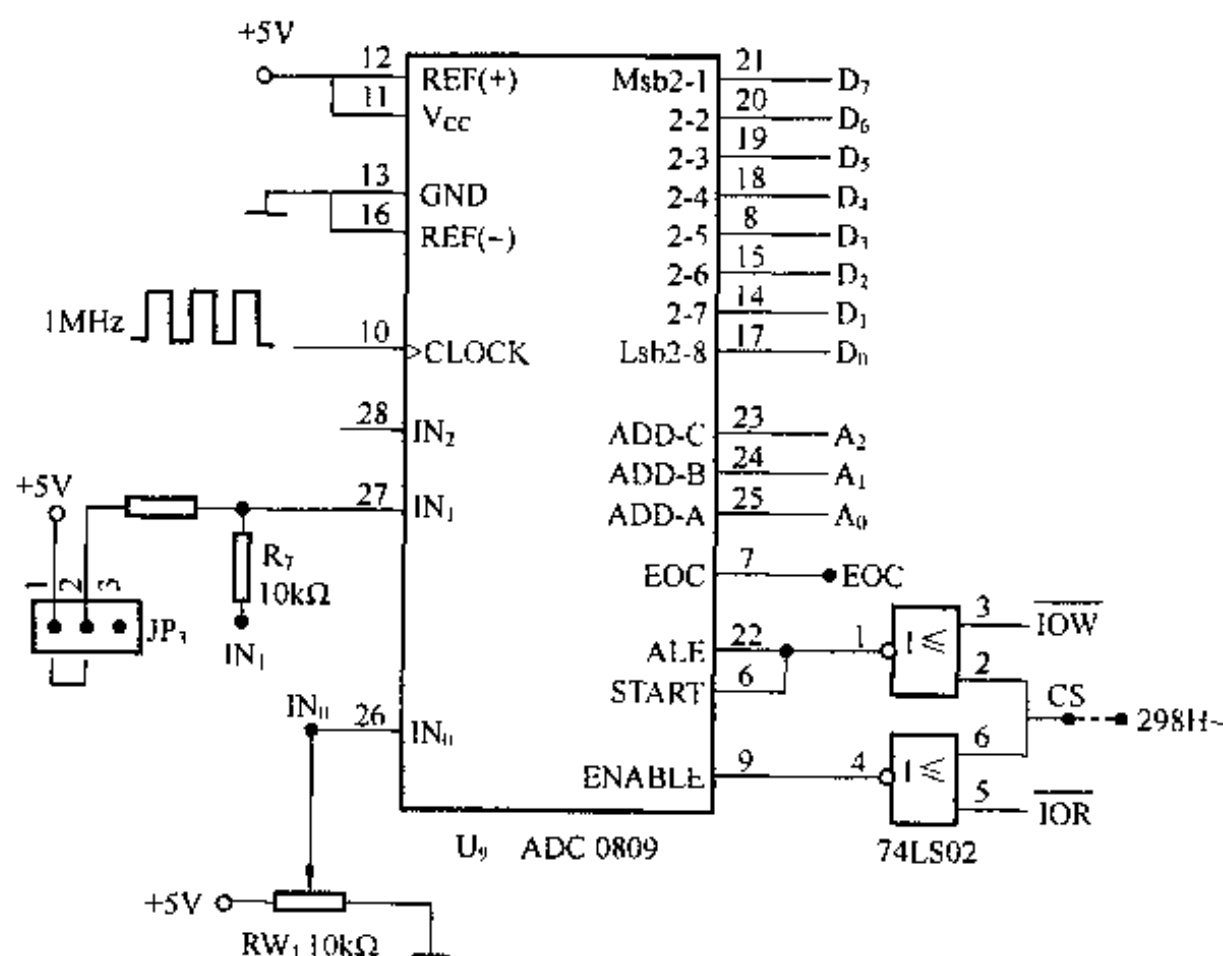


图 14.23 A/D 转换电路

② 编程采集 IN_0 输入的电压,在屏幕上显示经 A/D 转换后的数据(用十六进制数)。

③ 将 JP_3 的 1、2 短接,使 IN_2 处于双极性工作方式,并给 IN_1 输入一个低频交流信号(幅度为 -5~+5V),在屏幕上显示输入交流信号经采集后的数据波形。

14.6.4 编程提示及参考程序流程框图

① 参考程序流程如图 14.24 和图 14.25 所示。

② ADC 0809 的 IN_0 口地址为 298H, IN_1 口地址为 299H。

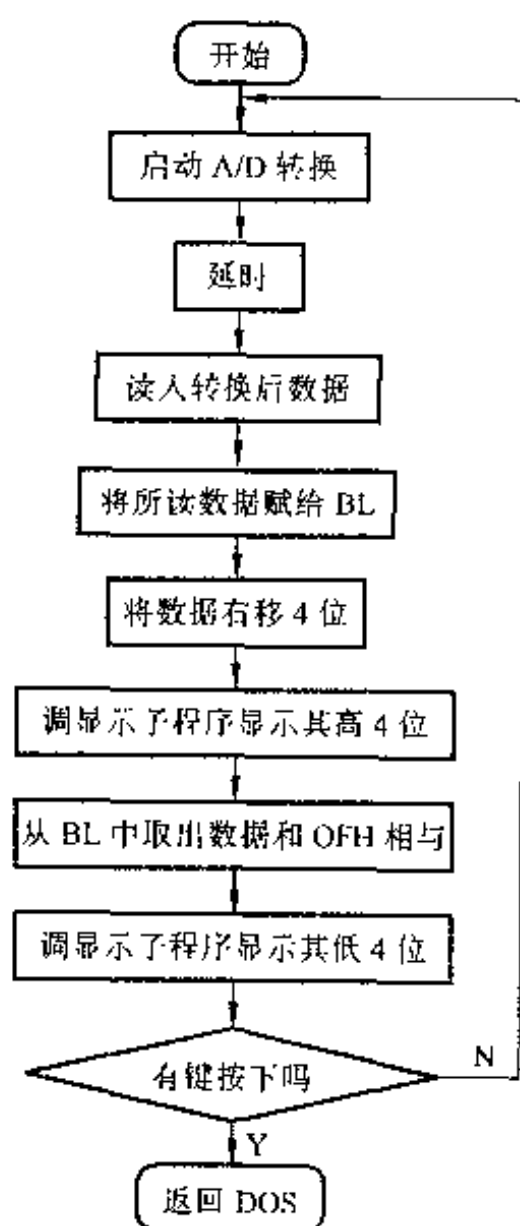
③ IN_0 单极性输入电压与转换后数字的关系为

$$N = V_i / V_{REF} \times 256$$

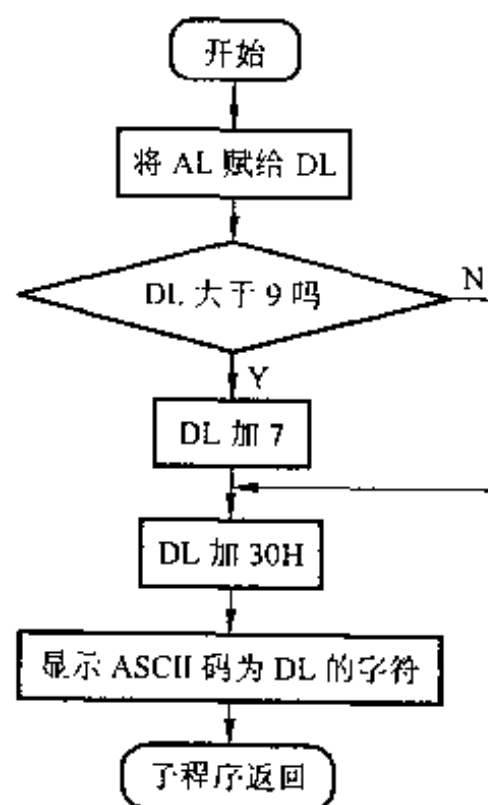
其中, V_i 为输入电压; V_{REF} 为参考电压, 这里的参考电压为 +5V 电源。

④ 一次 A/D 转换的程序可以为

```
MOV  DX, 端口地址
OUT  DX, AL;      启动转换
      延时
IN   AL, DX       读取转换结果
```



(a) 主程序



(b) 显示子程序

图 14.24 A/D 转换流程框图 (数字显示)

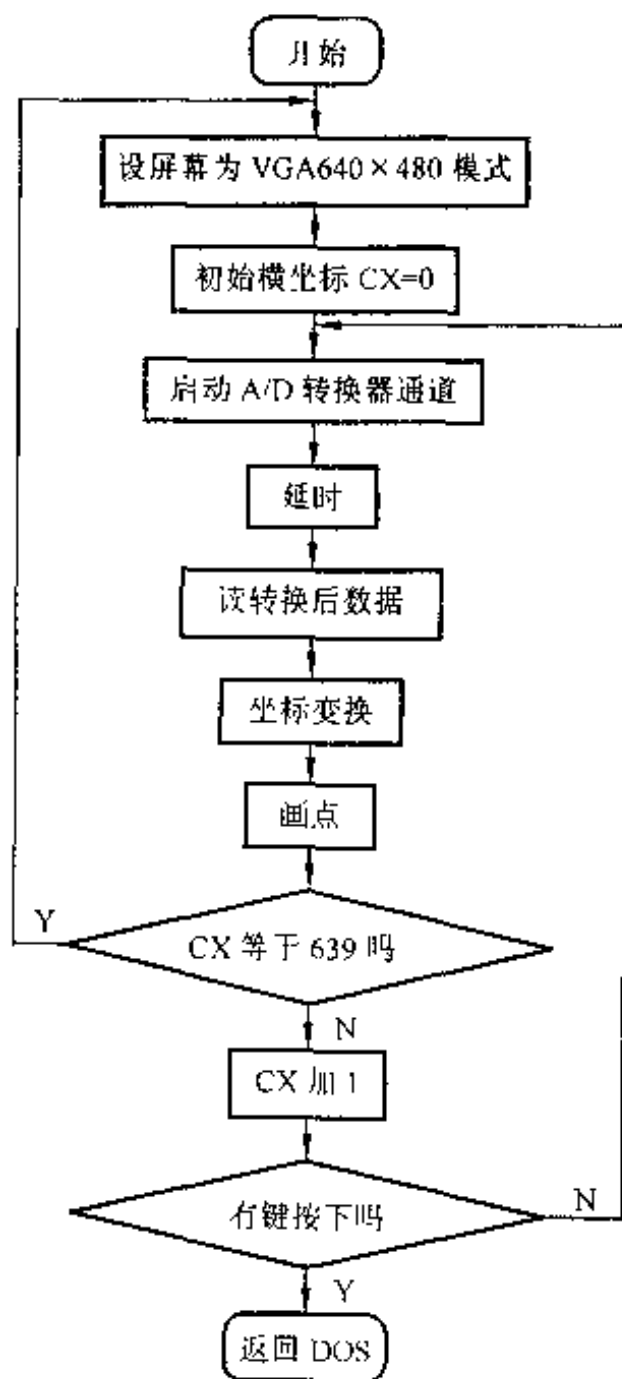


图 14.25 A/D 转换流程框图 (波形显示)

14.6.5 实验报告要求

- ①和②的要求同 14.1.5 小节的实验报告要求①和②。
- ③ 实验内容①的结果用表格表示,并根据此表画出输入电压和转换后数字的关系图。分析栏中写出与理论公式的比较。
- ④ 写出不用延时而用查询或中断方式来判断 A/D 转换是否结束的程序段。

14.7 DMA 传 送

14.7.1 实验目的

- ① 掌握 DMA 传输的基本原理。

- ② 掌握 PC 机工作环境下进行 DMA 数据传输的方法。
- ③ 学会对 DMA 控制器 8237 的编程。

14.7.2 实验预备知识及学习要求

I/O 设备与计算机之间进行数据传送除了通过 CPU 控制外,还可通过 DMA 控制。通过 CPU 进行的数据传送属于程序控制传送。在 DMA 控制下的数据传送方式称为 DMA 方式。DMA 方式下数据传送在外设与存储器之间直接进行而不需要通过 CPU。它通常用于一次要传送大量数据的场合。

DMA 方式传送的基本过程如下。

当外部接口中有数据要输入或向外部接口输出数据时,就向 DMA 控制器 (DMAC) 发一个 DMA 请求。DMA 控制器接到请求后向 CPU 发出总线请求。如果 CPU 允许让出总线,则发一个总线允许信号。DMA 控制器接到该信号后,就将地址寄存器的内容送到地址总线上,同时发 DMA 应答信号、I/O 读/写信号及内存写/读信号。当提出 DMA 请求的一方收到 DMA 应答信号后,撤除 DMA 请求信号并将数据送到数据总线上 (或从总线上接收数据),从而实现数据在外设与内存之间的传送。在 DMA 传送时,DMA 控制器的地址寄存器的内容要相应修改直至本次传送完毕为止。当 DMA 传送完成时,DMA 控制器撤销对 CPU 的总线请求,交还给 CPU 系统总线的管理和使用权。

8237 是 Intel 系列中高性能可编程 DMA 控制器,它允许 DMA 传输速度高达 1.6MB/s。8237 内有 4 个独立的通道,每个通道包含一对 16 位的地址初值寄存器和地址计数器 (占用同一个 I/O 地址) 以及一对 16 位的字节数初值寄存器和字节数计数器 (占用同一个 I/O 地址),还包括一个 8 位的方式寄存器。8237 中 4 个通道公用暂存寄存器 (PC 机不用)、命令寄存器和状态寄存器。此外,8237 内还有 4 个通道各占一位的 DMA 请求寄存器和屏蔽寄存器。

8237 内部共用到 16 个 I/O 端口,分别由上述各类寄存器占用。这些寄存器的寻址由地址码的 $A_3 \sim A_0$ (共 16 种组合) 及读/写命令来区分。其端口地址、软件命令与寄存器的对应关系如表 14.2 所示。

PC 机中的 DMA 控制器的寻址范围为 00H~0FH (可参见表 14.2)。4 个 DMA 通道中,通道 0 用来对动态 RAM 进行刷新;通道 1 为用户保留;通道 2 和通道 3 分别用来进行软盘驱动器和内存之间的数据传输以及硬盘驱动器和内存之间的数据传输。在 PC 机 BIOS 初始化过程中,控制寄存器的控制字被设置为 00H。因此,编程时不需要对控制寄存器进行设置。

实验前要求掌握 DMA 方式下数据传送的方法、可编程 DMA 控制器 8237 内部各寄存器的格式和含义、8237 方式字及 8237 的初始化等。

表 14.2 8237 内部寄存器和地址分配及软件命令

A ₃ A ₂ A ₁ A ₀	通道	读操作 ($\overline{\text{IOR}}$)	写操作 ($\overline{\text{IOW}}$)
0 0 0 0	CH ₀	读地址计数器	写地址初值寄存器和地址计数器
0 0 0 1		读字节数计数器	写字节数初值寄存器和字节数计数器
0 0 1 0	CH ₁	读地址计数器	写地址初值寄存器和地址计数器
0 0 1 1		读字节数计数器	写字节数初值寄存器和字节数计数器
0 1 0 0	CH ₂	读地址计数器	写地址初值寄存器和地址计数器
0 1 0 1		读字节数计数器	写字节数初值寄存器和字节数计数器
0 1 1 0	CH ₃	读地址计数器	写地址初值寄存器和地址计数器
0 1 1 1		读字节数计数器	写字节数初值寄存器和字节数计数器
1 0 0 0		读状态寄存器	写控制寄存器
1 0 0 1			写请求寄存器
1 0 1 0			写管理单通道的屏蔽字
1 0 1 1			写方式寄存器
1 1 0 0			清除先/后触发器命令
1 1 0 1		读暂存寄存器	主清除命令
1 1 1 0			清除屏蔽寄存器命令
1 1 1 1			写管理四通道的屏蔽字

14.7.3 实验内容

① 将 74LS273 芯片插入通用插座，按图 14.26 所示把电路连接好（74LS74 利用实验台上的 D 触发器）。在内存 6000H: 0 开始单元存放 10 个数据，对 DMA 控制器 8237 进行初始化，编程实现每一次 DMA 请求从内存向外设传送一字节数据。

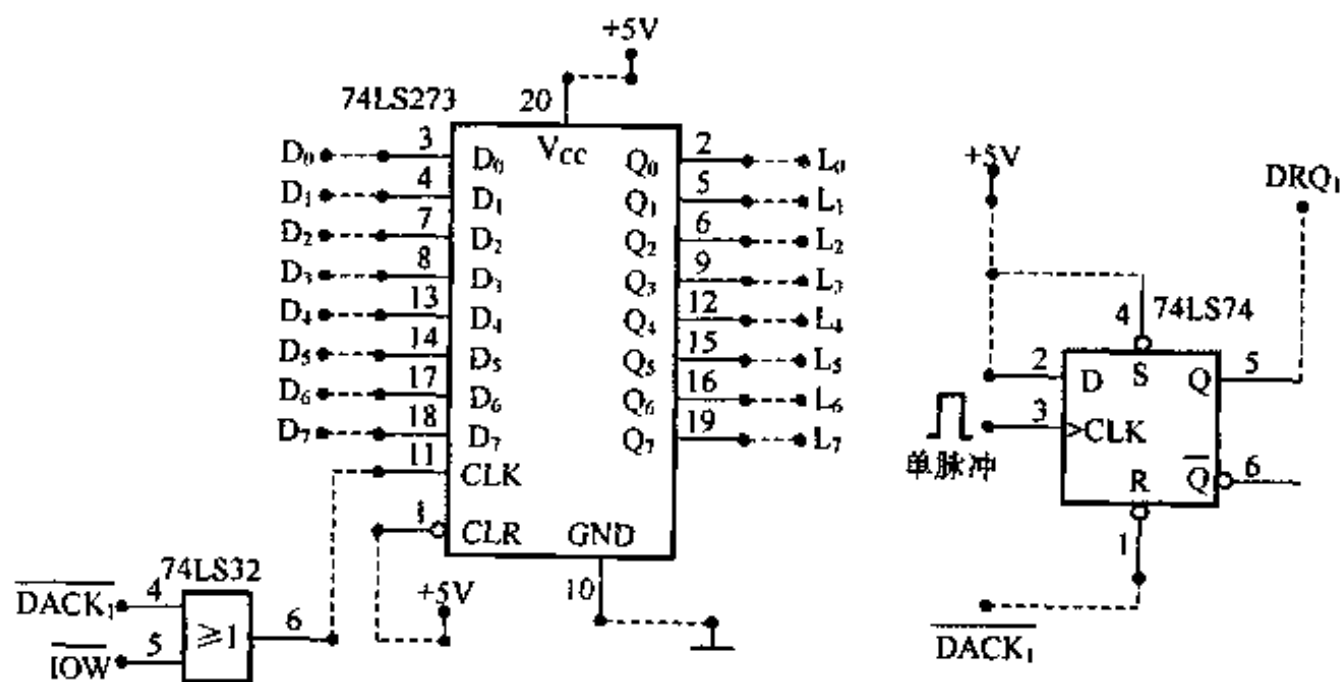


图 14.26 通过 DMA 输出数据

② 将 74LS244 芯片插入通用插座，按图 14.27 所示把电路连接好（74LS74 直接利用实验台上的 D 触发器）。在内存 6000H: 00 开辟一个 8 字节的数据缓冲区，对 DMA 控制器 8237 进行初始化，编程实现每一次 DMA 请求从外设向内存传送一个字符（ASCII 码），存入数据缓冲区，并实现在屏幕上不断显示该数据缓冲区的数据。

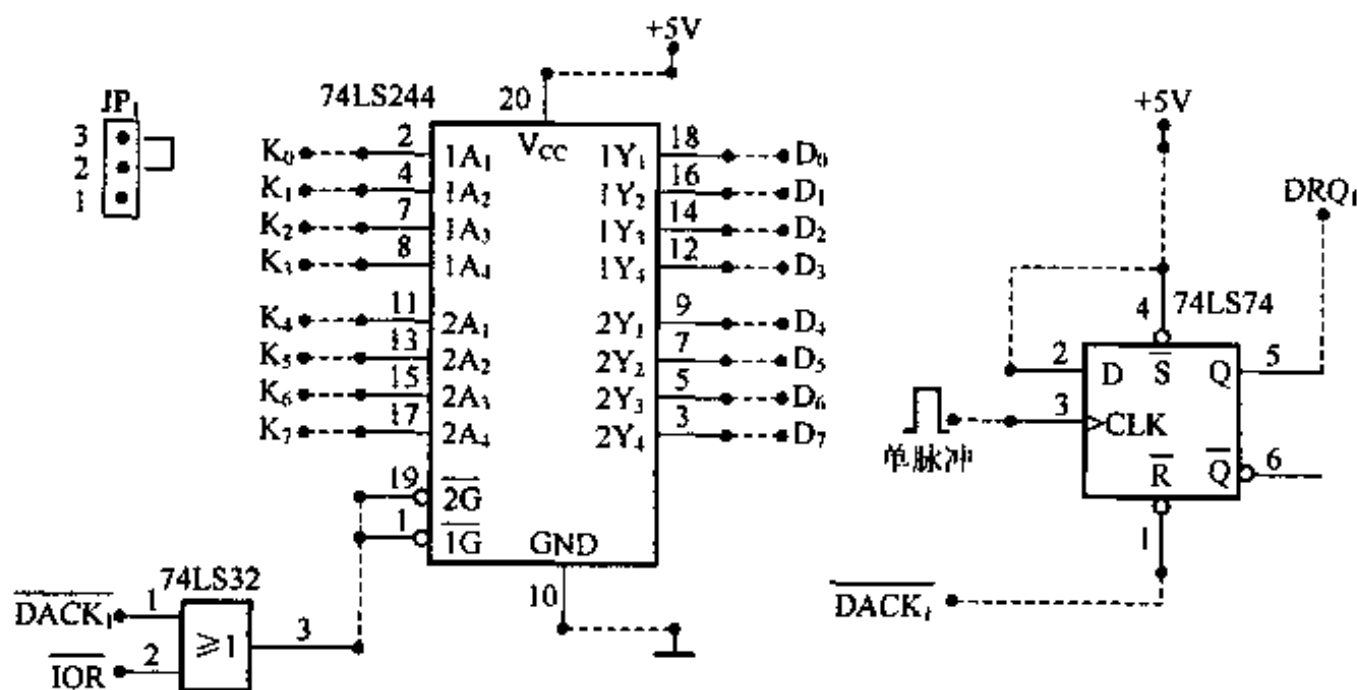


图 14.27 通过 DMA 输入数据

实验说明：

- ① DMA 实验应将实验台上跳线开关 JP₁ 的 2、3 短接。
- ② DMA 请求是由单脉冲发生器输入到 D 触发器，由触发器的 Q 端向 DRQ₁ 发出的。CPU 响应后发出 $\overline{DACK}_1$ ，将 D 触发器置成低电平以撤销请求。

14.7.4 编程提示及参考程序流程框图

① 8237 内端口地址分配如下：

通道 1 地址初值寄存器和地址计数器的端口地址为 02H。

通道 1 字节数初值计数器和字节数计数器的端口地址为 03H。

写屏蔽寄存器端口地址为 0AH。

写方式寄存器端口地址为 0BH。

清除字节指针寄存器端口地址为 0CH。

页面寄存器的端口地址为 83H。

② PC 机的 DMA 控制系统中只支持单字节传送方式。虽然软件请求方式是与块传送方式相对应的，但块的长度要严格限制，通常也设为单字节方式。因此，在传送多个字节时，要用循环一个一个来实现。

③ 对通道中的字节数计数器进行写入时要分两次进行，先写低字节，后写高

字节。

④ 对字节指针触发器的清“0”只用输出指令即可。

⑤ 参考程序流程如图 14.28 和图 14.29 所示。

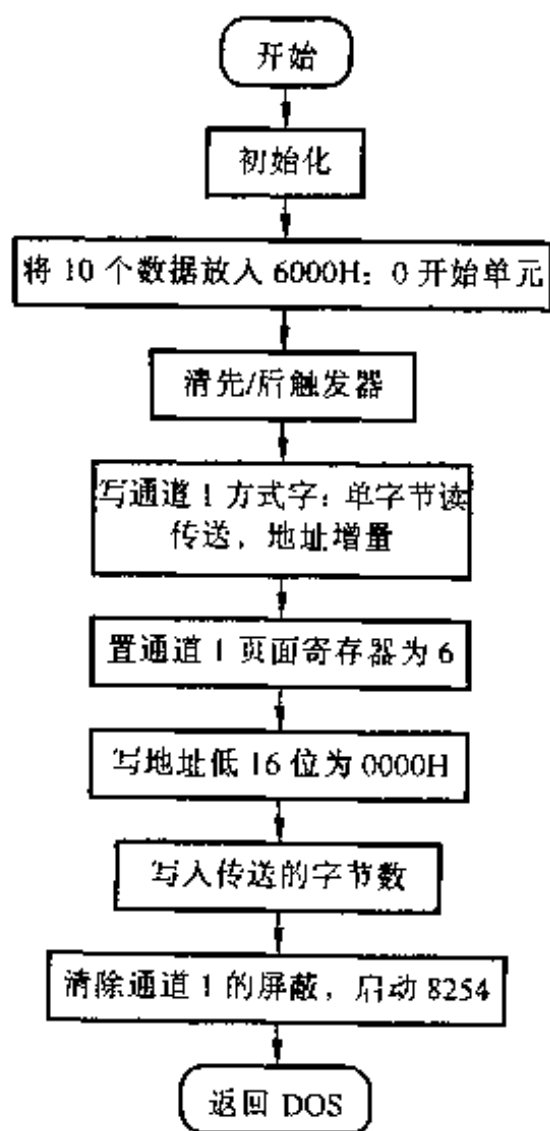


图 14.28 输出数据的流程框图

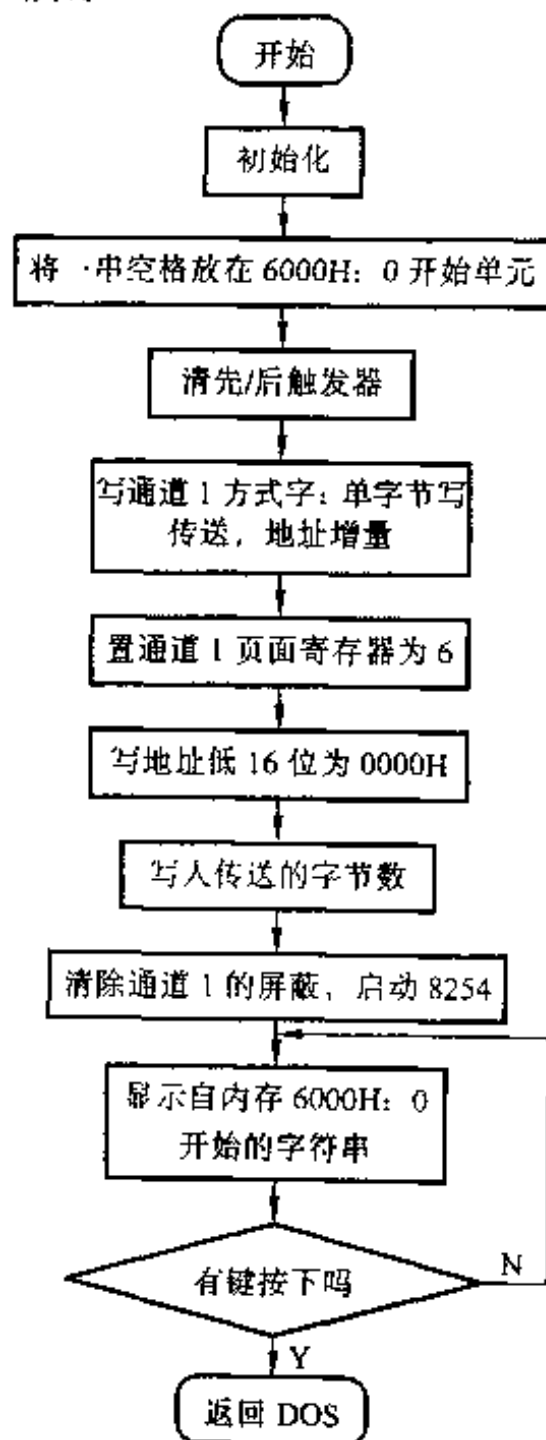


图 14.29 输入数据的流程框图

14.7.5 实验报告要求

①和②的要求同 14.1.5 小节的实验报告要求①和②。

③小结 8237 的初始化及使用编程。

④小结 DMA 传送方式的特点。

第 15 章 微机硬件应用综合设计

本章所列的实验实用性较强，需要利用多种微机接口和其他单元电路才能实现。目的是通过实验逐步提高运用理论知识设计一些综合实用的接口电路，自行设计、调试、分析问题和解决问题的独立工作能力，为将来开发和设计新型的微机系统打下基础。本章实验中实验预备知识及学习要求不再单独列出，有关知识可参看第 14 章的相关内容或相关书籍。

15.1 控制七段数码管进行静态与动态显示

15.1.1 实验目的

- ① 掌握数码管显示数字的原理。
- ② 进一步加深对 8255 方式 0 的理解。

15.1.2 实验内容

① 按图 15.1 所示连接好电路，将 8255 的 A 端口 $PA_0 \sim PA_6$ 分别与七段数码管的段码驱动输入端 a~g 相连，位码驱动输入端 S_1 接 +5V（选中）， S_0 、dp 接地（关闭）。编程实现在七段数码管上显示一位从键盘输入的十进制数（静态）。

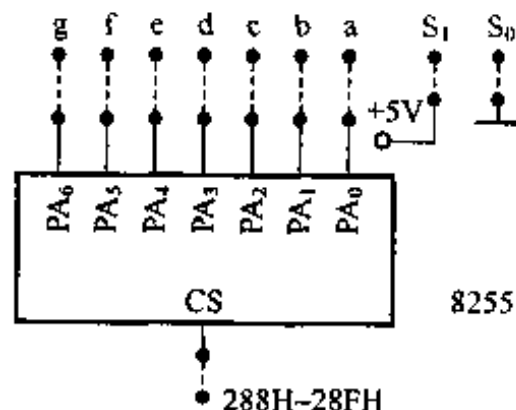


图 15.1 静态显示

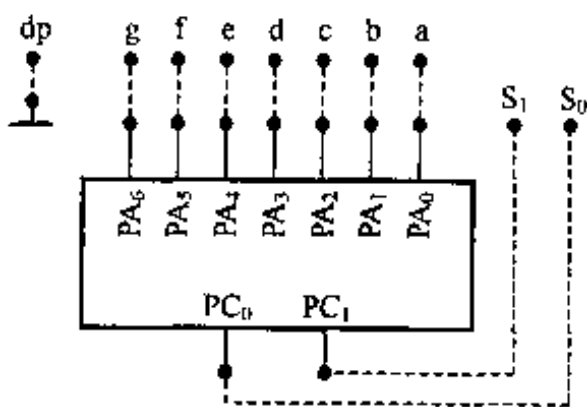


图 15.2 动态显示

② 按图 15.2 所示连接好电路，七段数码管段码连接不变，位码驱动输入端 S_1 、 S_0 接 8255C 端口的 PC_1 、 PC_0 ，编程实现在七段数码管上 2 位十进制数字的动态循环显示 00~99。

实验说明 实验台上的七段数码管为共阴性, 段码采用同相驱动, 输入端加高电平, 选中的数码管亮; 位码加反相驱动器, 位码输入端高电平选中。

15.1.3 编程提示及参考程序流程框图

① 七段数码管的字形代码如表 15.1 所示、字形如图 15.3 所示。

表 15.1 七段数码管的字形代码

显示字形	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3fh
1	0	0	0	0	1	1	0	06h
2	1	0	1	1	0	1	1	5bh
3	1	0	0	1	1	1	1	4fh
4	1	1	0	0	1	1	0	66h
5	1	1	0	1	1	0	1	6dh
6	1	1	1	1	1	0	1	7dh
7	0	0	0	0	1	1	1	07h
8	1	1	1	1	1	1	1	7fh
9	1	1	0	1	1	1	1	6fh

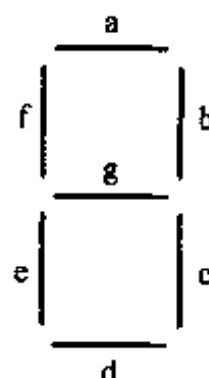


图 15.3 七段码管字形图

② 参考程序流程如图 15.4 和图 15.5 所示。

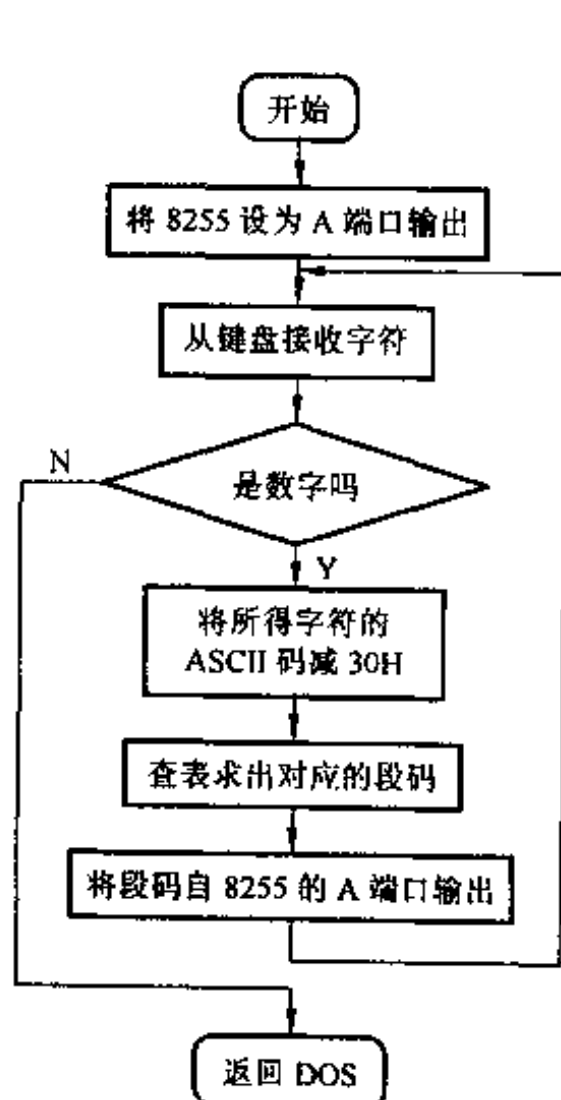


图 15.4 静态显示流程框图

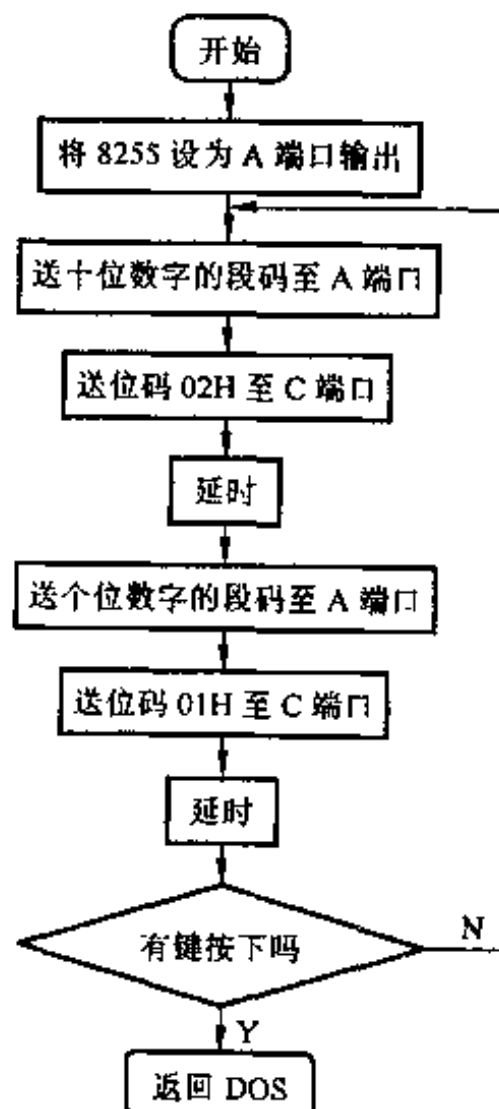


图 15.5 动态显示流程框图

15.1.4 实验报告要求

- ①和②的要求同 14.1.5 小节的实验报告要求①和②。
- ③ 小结七段数码管动态显示的编程方法。
- ④ 小结对 8255 方式 0 实际应用的体会。

15.2 继电器控制

15.2.1 实验目的

- ① 了解直流继电器的工作原理及微机控制直流继电器的一般方法。
- ② 进一步熟悉 8254 (8253) 和 8255 的使用方法。

15.2.2 实验内容

按图 15.6 所示连接电路。其中的继电器输出插头 J_4 接实验盒的继电器插头。编程使用 8254 (8253) 定时, 让继电器周而复始地闭合 n 秒钟, 断开 n 秒钟(n 自定)。

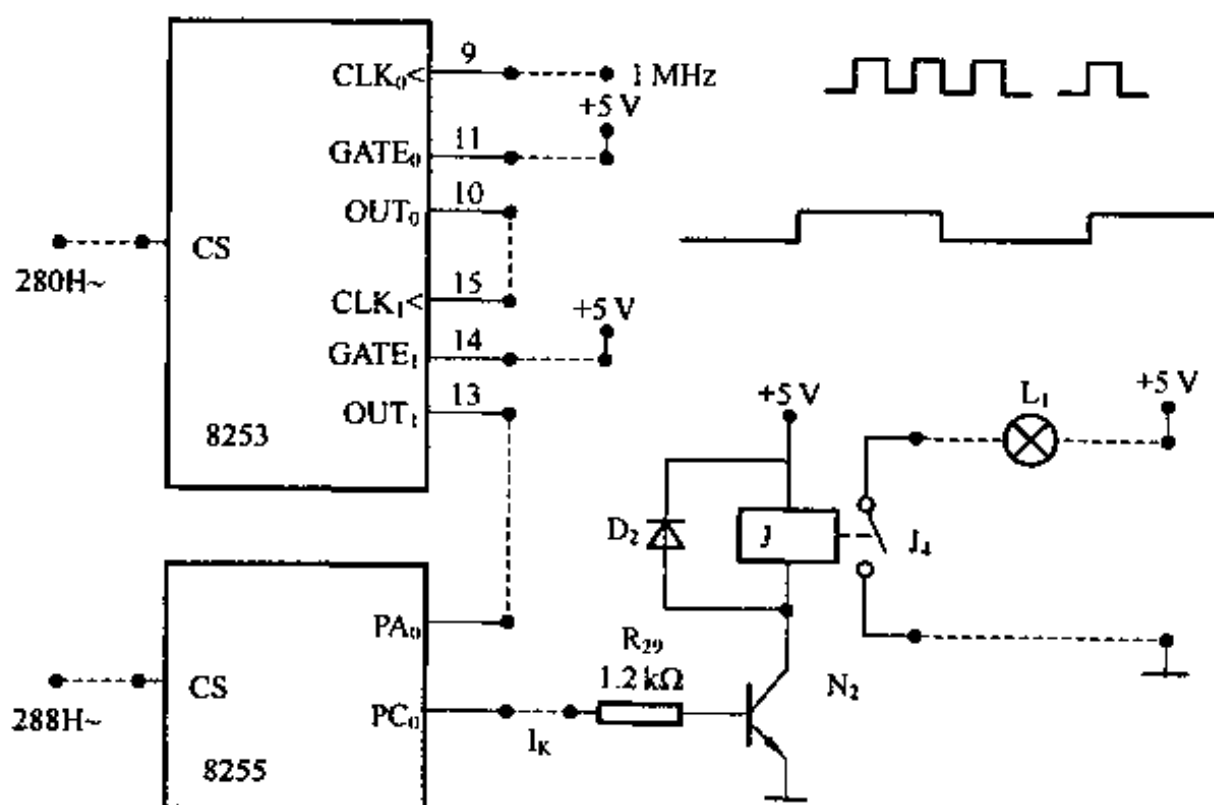


图 15.6 继电器控制电路

15.2.3 编程提示及参考程序流程框图

① 8254 (8253) 计数器 0 和计数器 1 初值的设置要根据继电器的闭合和断开的的时间而定。

② 启动计数器工作后, 经过 n 秒钟 OUT_1 输出高电平, 通过 8255 的 A 口查询 OUT_1 的输出电平, 用 C 口 PC_0 输出开关量控制继电器动作。

③ 当继电器开关量输入端输入“1”时, 继电器常开触点闭合, 电路接通, 指示灯亮, 输入“0”时断开, 指示灯熄灭。

④ 参考程序流程如图 15.7 所示。

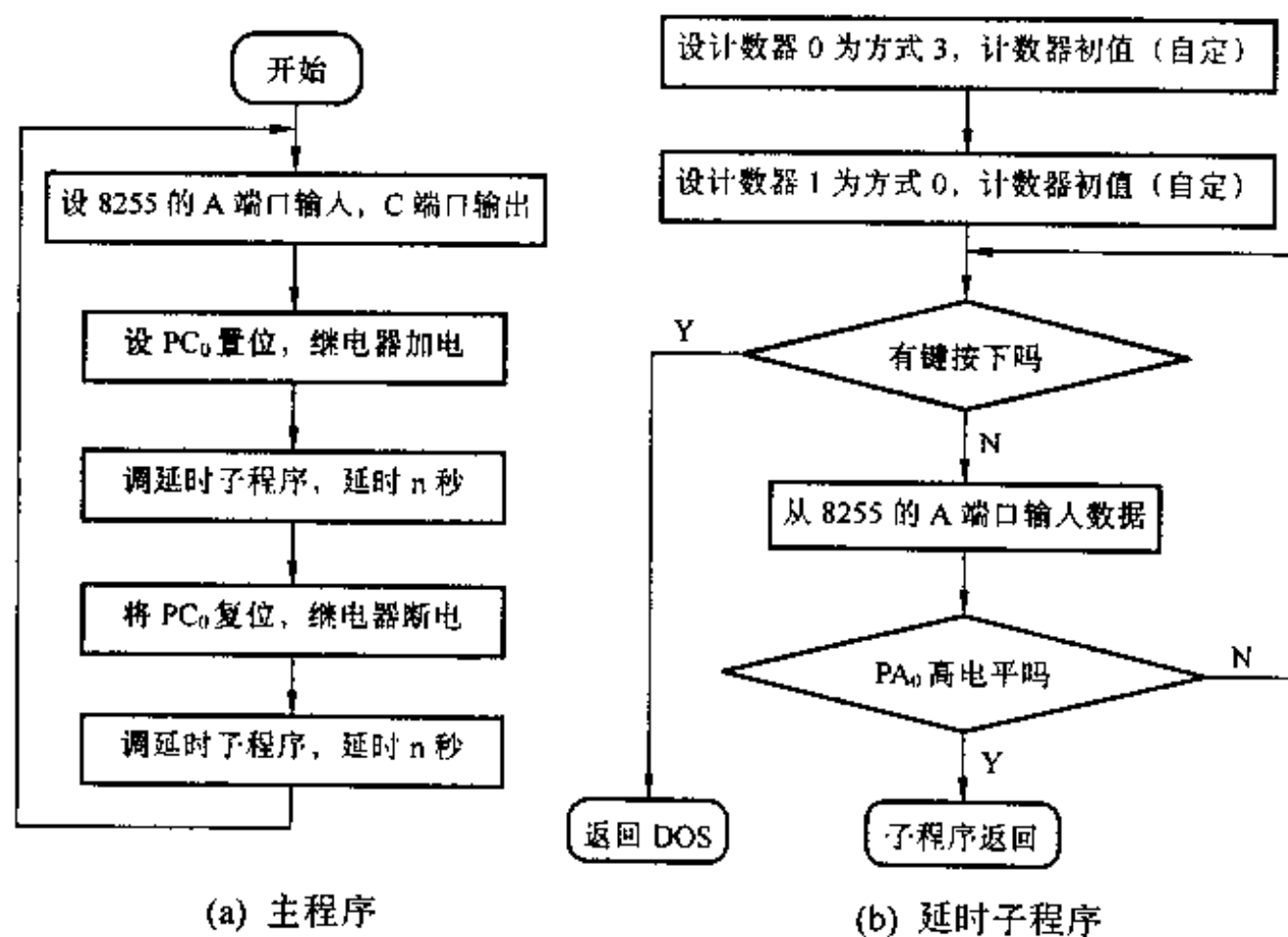


图 15.7 继电器控制程序流程图

15.2.4 实验报告要求

①和②的要求同 14.1.5 小节的实验报告要求①和②。

③ 重点分析 8254 (8253) 与 8255 连用实现定时控制的方法及 8255 方式 0 状态驱动接口的工作特点。

15.3 竞赛抢答器

15.3.1 实验目的

- ① 了解微机竞赛抢答器的基本原理。
- ② 进一步学习并行接口的使用方式。

15.3.2 实验内容

按图 15.8 所示连接电路。逻辑开关 $K_0 \sim K_7$ 代表竞赛抢答器按钮 0~7 号，当某个逻辑开关置“1”时，相当于某组抢答按钮按下，在七段数码管上将其组号(0~7)显示出来，并使喇叭响一下。

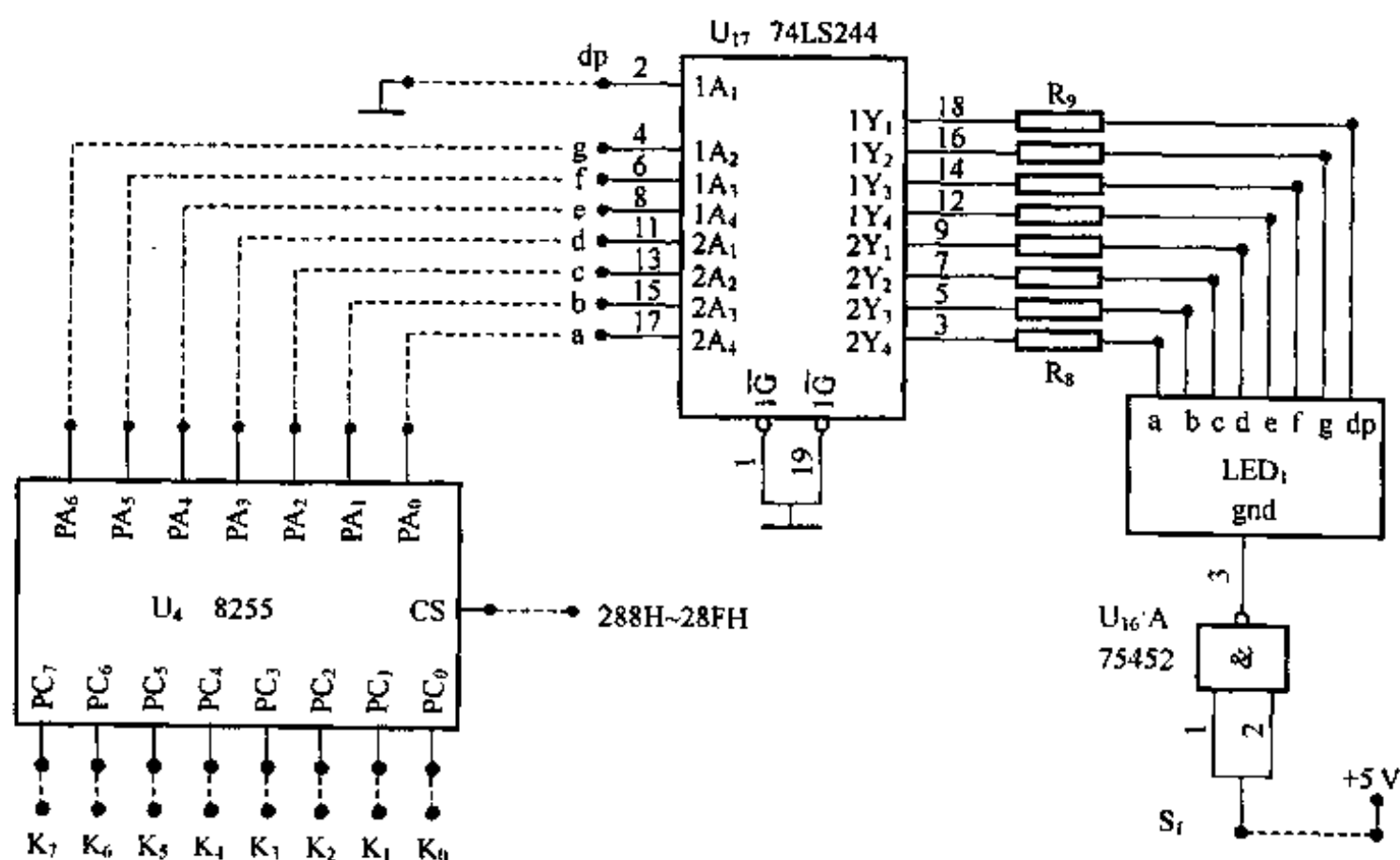


图 15.8 模拟竞赛抢答器原理图

15.3.3 编程提示及参考程序流程框图

① 设置 8255 为 C 端口输入、A 端口输出。读取 C 端口数据，若为 0，则表示无人抢答，若不为 0，则有人抢答。根据读取的数据可判断其组号。从键盘上按空格键开始下一轮抢答，按其他键程序退出。

② 参考程序流程如图 15.9 所示。

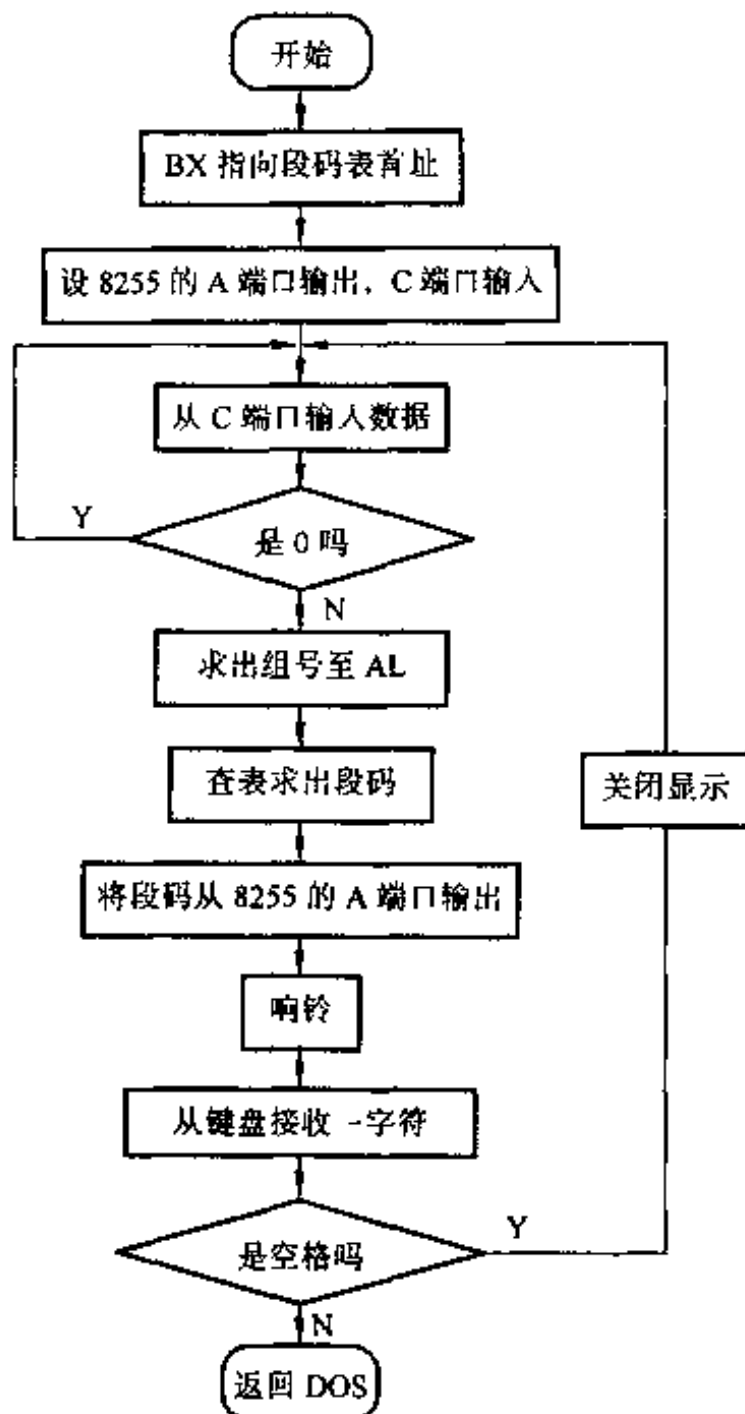


图 15.9 模拟竞赛抢答器流程框图

15.3.4 实验报告要求

- ①和②的要求同 14.1.5 小节的实验报告要求①和②。
 ③若要将其变成实用的竞赛抢答器, 需要进行哪些改进?

15.4 交通灯控制

15.4.1 实验目的

通过并行接口 8255 实现十字路口交通灯的模拟控制, 进一步掌握对并行口的

使用方法。

15.4.2 实验内容

按图 15.10 所示连接线路。L₇、L₆、L₅ 作为南北路口的交通灯与 PC₇、PC₆、PC₅ 相连，L₂、L₁、L₀ 作为东西路口的交通灯与 PC₂、PC₁、PC₀ 相连。编程使 6 个灯按交通灯的亮、灭变化规律进行控制。

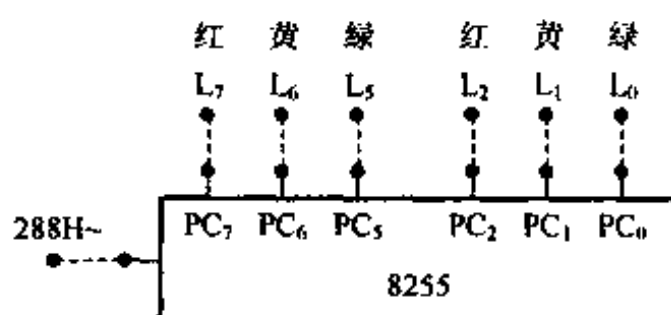


图 15.10 交通灯的模拟控制

15.4.3 编程提示及参考程序流程框图

编程提示及参考程序流程如图 15.11 所示。

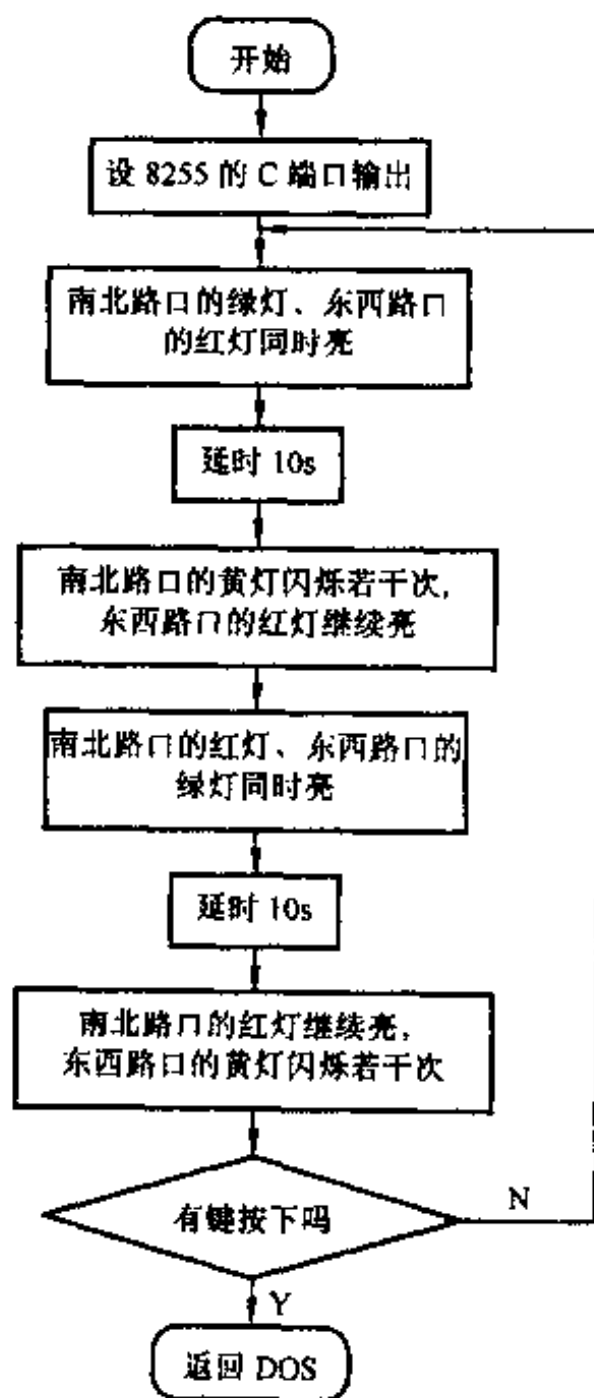


图 15.11 交通灯的模拟控制流程图

15.4.4 实验报告要求

①和②的要求同 14.1.5 小节的实验报告要求①和②。

③ 假设要根据大小不同路口及车流量的情况设置交通灯控制,谈谈你对本实验的改进意见(根据自己对条件的设定附上程序段)。

15.5 数字录音机

15.5.1 实验目的

- ① 了解数字录音技术的基本原理。
- ② 进一步掌握 A/D 转换器与 D/A 转换器的使用方法。

15.5.2 实验内容

按图 15.12 所示连接电路,将声传感器接 J_2 ,把代表语音的电信号送给 ADC 0809 通道 2 (IN_2); D/A 转换器的输出端 J_1 接喇叭。编程:以 5000 次/s 的速率采集 IN_2 输入的语音数据并存入内存,共采集 6 万个数据(录 12s),然后再以同样的速率将数据送 DAC 0832 使喇叭发声(放音)。

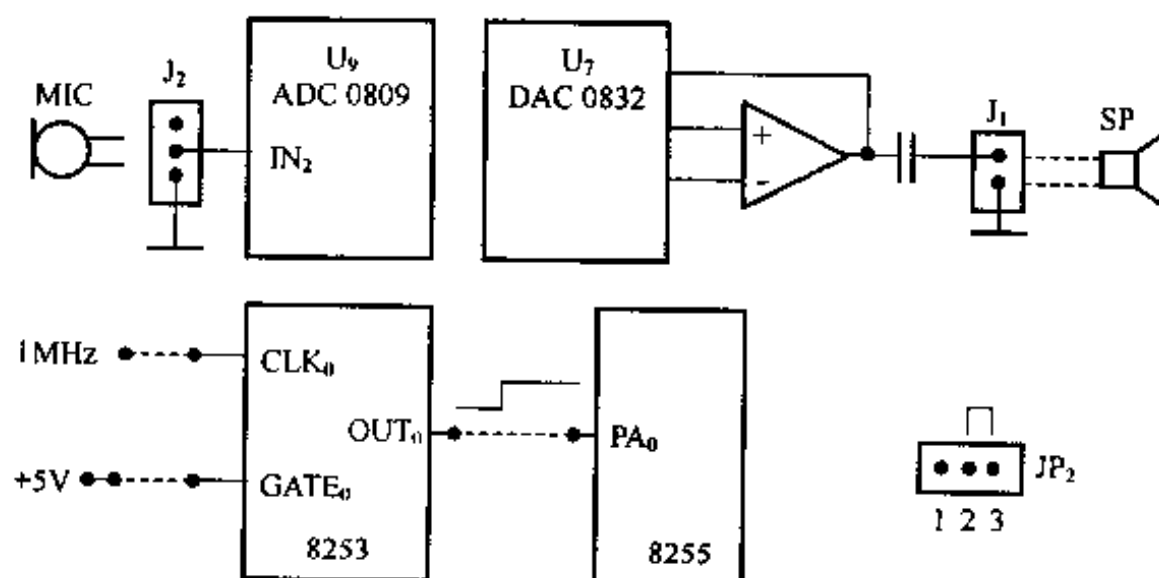
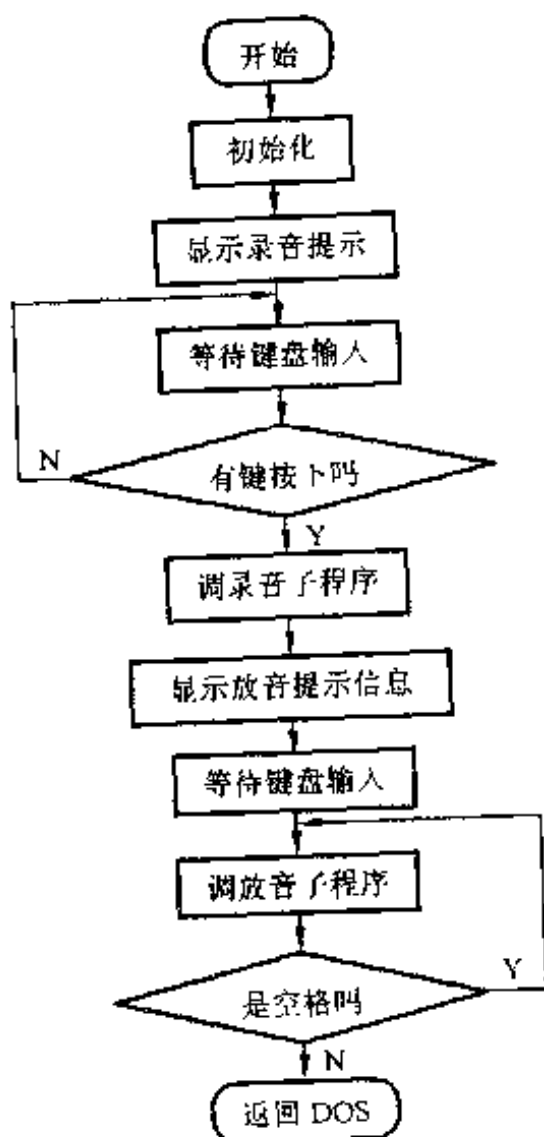


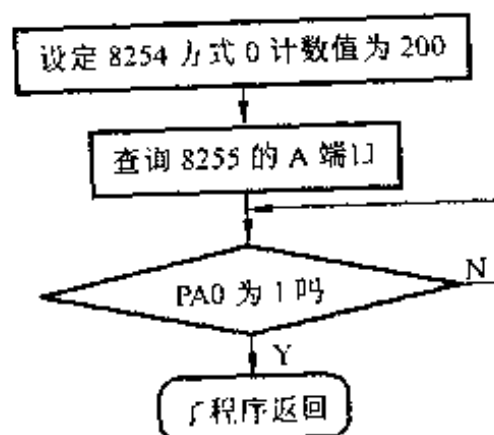
图 15.12 数字录音机电路图

15.5.3 编程提示及参考程序流程框图

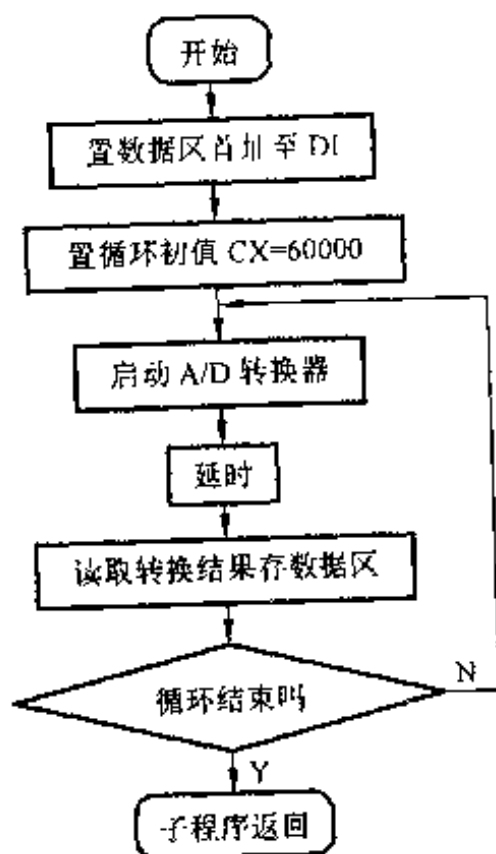
- ① 参考程序流程如图 15.13 所示。
- ② 将 8254 (8253) 设置成方式 0, 计数 200 个, 用 8255 的 PA_0 查询 OUT_0 电平, 高电平表示定时完成。
- ③ ADC 0809 通道 2 (IN_2) 的口地址为 298H。



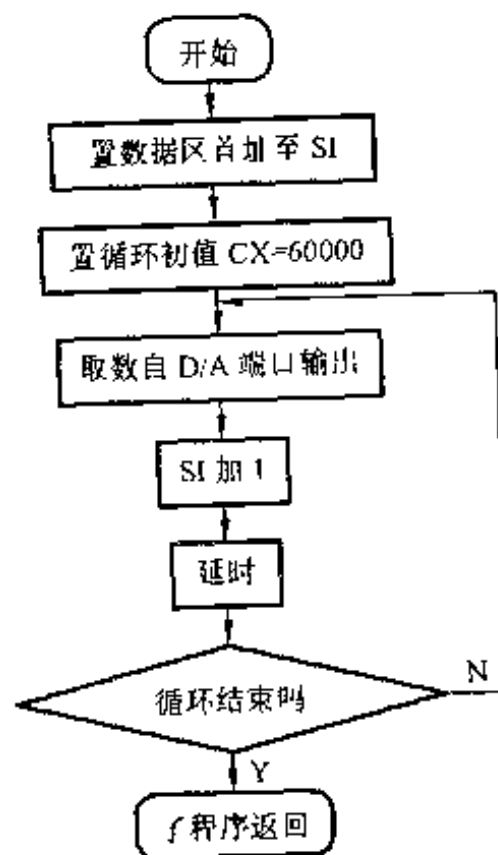
(a) 主程序



(b) 延时子程序



(c) 录音子程序



(d) 放音子程序

图 15.13 数字录音机程序流程图

15.5.4 实验报告要求

- ①和②的要求同 14.1.5 小节的实验报告要求①和②。
- ③ 分析与小结定时采样的方法。
- ④ 把延时子程序中的“定时”用其他方式实现（附上程序段）。

附录 A

《微机系统与接口技术》思考与练习题参考答案

第 1 章

1.1 微型计算机五个发展阶段。

第一代:4 位和低档 8 位微处理器。1971 年,英特尔公司推出了世界上第一款微处理器 4004,这是第一个可用于微型计算机的 4 位微处理器,它包含 2300 个晶体管。

第二代:8 位微处理器。Zilog 公司生产的 Z80、摩托罗拉公司生产的 6800,英特尔公司于 1976 年生产的增强型 8085 都属于第二代微处理器。它们均采用 NMOS 工艺,集成度约 9000 只晶体管,平均指令执行时间为 $1\sim 2\mu\text{s}$,采用汇编语言、BASIC、Fortran 编程,使用单用户操作系统。

第三代:16 位微处理器。1978 年英特尔公司生产的 8086 是第一个 16 位的微处理器。很快 Zilog 公司和摩托罗拉公司也宣布计划生产 Z8000 和 68000,这是第三代微处理器的起点。8086 微处理器最高主频速度为 8MHz,具有 16 位数据通道,内存寻址能力为 1MB。1979 年,英特尔公司又开发出了 8088。8086 和 8088 在芯片内部均采用 16 位数据传送,所以都称为 16 位微处理器,但 8086 每周期能传送或接收 16 位数据,而 8088 每周期只采用 8 位。8088 采用 40 针的 DIP 封装,工作频率为 6.66MHz、7.16MHz 或 8MHz,微处理器集成了大约 29000 个晶体管,采用 $1.5\mu\text{m}$ 技术。1982 年,英特尔公司在 8086 的基础上,研制出了 80286 微处理器,该微处理器的最大主频为 20MHz,内、外部数据传送均为 16 位,使用 24 位内存寄存器的寻址,内存寻址能力为 16MB。80286 的封装是一种被称为 PGA 的正方形包装,PGA 是源于 PLCC 的便宜封装,它有一块内部和外部固体插脚,在这个封装中,80286 集成了大约 134000 个晶体管。

第四代:32 位微处理器。1985 年 10 月 17 日,英特尔划时代的产品——80386DX 正式发布了,其内部包含 27.5 万个晶体管,时钟频率为 12.5MHz,后逐步提高到 20MHz、25MHz、33MHz,最后还有少量的 40MHz 产品。1989 年,80486 芯片由英特尔推出。这款经过四年、投入 3 亿美元资金开发的芯片的伟大之处在于它首次突破了集成 100 万个晶体管的界限,集成了 120 万个晶体管,使用 $1\mu\text{m}$ 以下的制造工艺。80486 的时钟频率从 25MHz 逐步提高到 33MHz、40MHz、50MHz。1993 年,全面超越 486 的奔腾微处理器问世。早期的奔腾 75~120MHz 使用 $0.5\mu\text{m}$ 的制造工艺,后期 120MHz 频率以上的奔腾则改用 $0.35\mu\text{m}$ 工艺。第一个有 MMX 技术(多媒体扩展技术)的多能奔腾,拥有 16KB 数据 L1 Cache,16KB 指令 L1 Cache,兼容 SMM,64 位总线,528MB/s 的频宽,2 时钟等待时间,450 万个晶体管,功耗 17 瓦。支持的工作频率有:133、150、166、200、233MHz。Pentium Pro(高能奔腾,686 级的 CPU)的核心架构代号为 P6 (也是未来 P II、P III 所使用的核心架构),这是第一代产品,二级 Cache 有 256KB 或 512KB,最大有 1MB 的二级 Cache。工作频率有:133/66MHz(工程样品),150/60、166/66、180/60、200/66MHz。AMD K6 拥有 32KB 数据 L1 Cache,32KB 指令 L1 Cache,集成了 880 万个晶体管,

采用 $0.35\mu\text{m}$ 技术,五层 CMOS,C4 工艺反装晶片,内核面积 168mm^2 (新产品为 68mm^2),使用 Socket7 架构。

第五代:64 位微处理器。以 Intel 推出的 Itanium 为代表,采用 6 层铜布线,22000 万个晶体管,主频达到 $900\text{MHz}\sim 1\text{GHz}$ 。

1.2 8086 由总线接口部件 BIU 和执行部件 EU 两部分组成,BIU 负责与存储器及 I/O 芯片接口,执行所有外部总线周期,提供总线控制信号,根据执行部件请求,完成取指和进行数据交换,EU 负责指令的执行,进行数据处理和有效地址计算,管理寄存器。

80286 由执行部件 EU、地址部件 AU、指令部件 IU 和总线部件 BU 等组成。AU 实现段式存储管理和虚拟存储管理,IU 包括指令译码器和已译码指令队列。指令部件将指令字节从预取队列中取出,送入指令译码器。指令译码器将每个指令字节译码变成 69 位的内部码形式,存入可容纳 3 条指令的已译码指令队列中。BU 负责处理 CPU 和系统总线之间的所有通信和数据传送,EU 用来执行指令、运算功能。

80386 由 6 个部分组成,它们是总线接口部件、指令预取部件、指令译码部件、执行部件、分段部件和分页部件。总线接口部件提供 80386 与其外部环境的接口以及与外部总线主控设备和协处理器的接口。指令预取部件负责执行 80386 程序的提前查看功能。指令译码部件从预取队列取出指令字节并将它们译成内部码存入指令缓冲的已译码指令队列中。执行部件执行来自已译码指令队列中指令,并且与要求完成这条指令的其他部件通信。执行部件还用专用硬件来加速某些操作的执行。分段部件将逻辑地址按执行部件要求变换成线性地址。当 80386 允许分页机制时,分页部件将分段部件产生的线性地址变换成物理地址。

1.3 参见本书知识要点 1.1.1 小节。

1.4 参见本书知识要点 1.1.2 小节。

1.5 8086 与 80186 微处理器内部均以 16 位工作,在外部输入/输出上 80186 采用 16 位。80186 内部集成了时钟发生器、片选和准备好电路、1 个中继控制器、2 个 DMA 控制器和 3 个计数器/定时器。

80286 在以下四个方面比 8086 有显著的改进:支持更大的内存,在实模式下,80286 可以访问的内存总量限制在 1MB;而在保护方式之下,80286 可直接访问 16MB 的内存;能够模拟内存空间;能同时运行多个任务;提高了处理速度。

1.6 参见本书知识要点 1.1.2 小节。

1.7 逻辑地址:由段基址和段内偏移地址组成的地址,段基址和段内偏移地址都是 16 位或 32 位的无符号二进制数,在程序设计时使用。

物理地址:存储器的绝对地址,是由 CPU 访问存储器时由地址总线发出的地址。

逻辑和物理地址是两种存储器地址,能在程序中编排的地址是逻辑地址,信息在存储器中实际存放的地址是物理地址。

逻辑地址经过分段单元处理后得到线性地址。如果系统不分页,线性地址就是物理地址;如果系统要分页,线性地址经过分页部件变换之后形成物理地址。

1.8 80386 有三种工作模式:实模式、保护模式、虚拟 8086 模式。

在实模式下,80386 可以处理 32 位数据,而且增加了 2 个段寄存器,在保护模式下,80386 提供一系列机制支持多任务,可以访问 4GB 的物理存储空间,段的最大长度在启动页功能时是 4GB,不启动页功能时是 1MB。虚拟 8086 模式是既有保护功能,又能执行 8086 代码的一种动态工作模式。

1.9 80386CPU 由总线接口部件、指令预取部件、指令译码部件、执行部件、分段部件和分页部件组成。总线接口部件主要用于 CPU 与外部总线的连接；指令预取部件通过总线接口部件按顺序提取指令流，并将其存入 16B 的预取队列中；指令译码部件从指令预取部件的预取队列中读取指令字节进行译码产生内部码，并将其存入缓冲区深度为 3 条指令的译码指令队列中。执行部件将内部码转换成按时间顺序排列的一系列控制信息，并发给处理器的其他处理部件；分段部件是把逻辑地址转换成线性地址；分页部件是将从分段部件得到的线性地址转换成物理地址。

80386DX 是 80386 的原型，为区别 80386SX 而命名，为全 32 位体系结构。80386SX 其内部结构与 80386DX 相同，外部数据总线是 16 位的，地址总线只有 24 位，寻址 16MB。

1.10 80386 有 8 个 32 位通用寄存器，6 个 16 位段寄存器，4 个 32 位的控制寄存器，以及标志寄存器，系统地址寄存器，测试寄存器和调试寄存器。

1.11 80386CPU 的标志寄存器是 32 位寄存器，定义了 14 位，其中有进位标志 CF，奇偶标志 PF，辅助进位标志 AF，零标志 ZF，符号标志 SF，溢出标志 OF 是状态标志；方向标志 DF，中断允许标志 IF，跟踪标志 TF，I/O 特权级 IOPL，任务嵌套标志 NT，恢复标志 RF，虚拟 8086 模式标志 VM。其中方向标志 DF，中断允许标志 IF，跟踪标志 TF 与 80286CPU 相兼容；I/O 特权级 IOPL，任务嵌套标志 NT，恢复标志 RF，虚拟 8086 模式标志 VM 是控制标志，是 80386CPU 新增的。

1.12 80486 的指令和数据共用 8KB 的 4 路组相连高速缓存与片外存储器间采用成组传送方式，还采用最近最少使用算法进行自动更新。80486 内部的浮点部件使其直接具有浮点处理能力，从而缩短了 CPU 与 FPU 之间的通信时间，它可以以极高速度进行单精度或双精度浮点运算，其处理能力是 80386 的 28 倍。

80486DX 是 80486 的原型，结构上 $80486DX = 80386 + 80387 + 8KB \text{ Cache}$ 及控制器； $80486SX = 80386 + 8KB \text{ Cache}$ 及控制器，没有浮点运算协处理器。

1.13 80486 是由总线接口部件、指令预取部件、指令译码部件、控制部件、整数部件、分段部件和分页部件以及浮点运算部件和高速缓存部件组成。

1.14 Pentium 采用 $0.8 \sim 0.6\mu m$ 的静态 BiCMOS 工艺制造，片内有 310 万晶体管，采用 273 引脚的针栅阵列封装，时钟频率由 60MHz 提高到 233MHz，它是 RISC 技术与 CISC 技术的混合产品。

Pentium Pro 又称高能 Pentium，采用 $0.6 \sim 0.25\mu m$ 的 4 层金属 BiCMOS 工艺，387 引脚的 PGA 封装，时钟频率为 155~433MHz。

Pentium MMX 采用 $0.35\mu m$ COMS 工艺，4 层金属配线，集成了 450 万晶体管，时钟频率为 160~233MHz，有 64 位外部总线和 32 位内部总线。MMX 是为了提高 PC 机处理多媒体和通信能力而推出的新一代处理器技术。

Pentium II 采用 $0.25\mu m$ 工艺，集成了 750 万晶体管，时钟频率为 233~450MHz。

1.15 Pentium III 采用了 $0.25\mu m$ 工艺，集成了 950 万晶体管，带有 70 条 SSE 指令。它的一级 Cache 为 32KB，二级 Cache 为 512KB，速度相当于 CPU 核心速度的一半。使用双重独立总线，且有动态执行功能。

Pentium 4 采用 $0.18\mu m$ 的 6 层互连铝金属层生产工艺，IA-32 体系结构，集成了 4200 万晶体管。Pentium 4 引入了全新的内核架构 NetBurst。

1.16 微处理器从诞生至今已经历了 1/4 个世纪，体系结构从 8 位、16 位向 32 位迁移。

这些数字清楚地表示出了各个时代的特点,以 x86 处理器为例,8 位处理器处于汇编时代,16 位处理器处于 DOS 和 Windows 3.x 时代,32 位处理器处于 Windows 9X 和 Windows NT/2000/XP 时代。

现在,32 位的体系结构虽然是主流,但在处理对象数据日益庞大的今天,在处理能力和地址空间方面,逐渐显得有点“捉襟见肘”。另外,在网络计算和数据库服务器中,存储 4GB 以上的数据已不足为奇,故而可以预计,在网络计算和数据库应用领域,地址空间富裕的 64 位体系结构将成为主流。

1.17 接口的分类方法有多种,按功能分,有内存储器接口、I/O 接口、键盘接口、鼠标接口、显示接口、外存储器接口和模拟接口;按应用分,有 3 种基本类型:运行辅助接口、用户交互接口和传感控制接口。

第 2 章

2.1 80486 微处理器内部包括总线接口部件,指令预取部件,指令译码部件,控制部件,整数部件,分段部件,分页部件以及浮点部件和 Cache 部件等。Pentium 包含了 80486 的全部特性,并增强和增加了许多功能,内部包含了指令 Cache 和数据 Cache,分支目标缓冲器和预取缓冲器,指令译码部件,控制 ROM 和控制部件,超标量体系结构的整数部件,流水线式浮点部件,分段和分页部件,数据整合和出错检测等。

2.2 32 位微处理器的通用寄存器在 16 位的基础上扩充为 32 位宽,仍然也兼容了 8 位和 16 位的寄存器,包含了累加器 EAX,基址寄存器 EBX,计数寄存器 ECX,数据寄存器 EDX,基址指针寄存器 EBP,源变址寄存器 ESI,目的变址寄存器 EDI,堆栈指针寄存器 ESP,它们的低 16 位可以用 16 位寄存器名 AX、BX、CX、DX、BP、SI、DI、SP 单独访问。段寄存器仍然为 16 位,但增加了两个,包括代码段寄存器 CS,数据段寄存器 DS,堆栈段寄存器 SS,附加段寄存器 FS、GS,在保护模式下用来装入 16 位选择符,在实地址和虚拟 8086 模式下装入段基址。

2.3 32 位微处理器标志寄存器 EFLAGS 包含三种标志:状态标志、控制标志和系统标志。状态标志报告算术指令执行后的状态;控制标志仅含有一个标志 DF,它用于控制串指令的执行方向;系统标志用于控制执行方式、I/O 权限、屏蔽中断、调试、任务转换和控制保护模式与虚拟 8086 模式之间的转换等微处理器的操作。

2.4 32 位微处理器控制寄存器包含了 $CR_0 \sim CR_3$,Pentium 扩充 CR_4 ,它们用于存放表示全局特性的机器状态。 CR_0 中含有控制或指示应用于整个系统条件的标志; CR_1 为将来的 Intel 处理器保留; CR_2 寄存器中存放页故障的线性地址; CR_3 为处理器提供当前任务的页目录表基址; CR_4 为 Pentium 扩充的控制寄存器,其中位 7~31 保留,低 7 位分别是虚拟 8086 模式扩展 (VME)、保护模式虚拟中断 (PVI)、禁止 RDTSC 指令 (TSD)、允许调试扩展 (DE)、允许页大小扩展 (PSE)、允许物理地址扩展 (PAE)、允许机器检查异常 (MCE)。

2.5 32 位微处理器系列用 4 个寄存器把在保护模式中常用的数据结构如基址、限长和其他属性保存起来,以确保其快速性,包含了全局描述符表寄存器 GDTR、局部描述符表寄存器 LDTR、中断描述符表寄存器 IDTR、任务寄存器 TR。GDTR 中存放操作系统使用的、各任务公用的全局描述符的 32 位线性基址和 16 位限长;LDTR 中存放用来检索当前任务的局部描述符表的选择符;IDTR 存放中断描述符表的基址和限长;TR 存放用来检索当前任务的任务状态段选择符。

2.6 32 位微处理器提供了 8 个 32 位的可编程寄存器来支持调试功能。其中, $DR_0 \sim DR_3$ 各存放一个断点的线性地址; DR_4 、 DR_5 保留 (Pentium 用它们作为 DR_6 、 DR_7 的别名); DR_6 是断点状态寄存器, 用以说明是哪一种性质的断点以及断点异常是否发生; DR_7 为断点控制寄存器, 它指明断点发生的条件及断点的类型。这些调试寄存器给 32 位微处理器带来了先进的调试功能, 如设置数据断点、代码断点 (包含 ROM 断点) 和对任务转换进行调试等。

2.7 Pentium 定义的模型专用寄存器, 用于控制可测试性、执行跟踪、性能监测和机器检查错误。Pentium 可以使用指令 RDMSR 和 WRMSR 读和写这些寄存器。

2.8 存储单元 13234H 中含 FFH, 下一个地址单元含 00H。指令执行后, $(13235H) = 11H$, $BX = 00FFH$ 。

2.9 52H。

2.10 DAA 把 AL 中和调整为紧缩 BCD 码; AAA 把 AL 和调整为非紧缩 BCD 码。

2.11 DAS 把 AL 中差调整为紧缩 BCD 码; AAS 把 AL 差调整为非紧缩 BCD 码。

2.12 0FD02H; 0002H。

2.13 AND EDX, 80H。

2.14 AND WORD PTR [0100H], 80H。

2.15 BSR EAX, DWORD PTR [SI]。

2.16 该指令将 DI 指向的双字存储单元的内容逻辑右移 CL 位, EAX 的右边 CL 位移入双字存储单元的高位中, 而 EAX 的内容保持不变。

2.17 DATA SEGMENT

FACT DW 0

N = 5

DATA ENDS

CODE SEGMENT

ASSUME CS: CODE, DS: DATA

START: MOV AX, DATA

MOV DS, AX

MOV CX, N

MOV AX, CX

LOOPn: CMP CX, 1

JNA EXIT

DEC CX

MUL CL

JMP LOOPn

EXIT: MOV FACT, AX

MOV AH, 4CH

INT 21H

CODE ENDS

END START

2.18 在数据段定义一个 48 位的存储器操作数, 低字是段长, 高双字是段基址。

INIT-GDTR DB 03000000FFFFH

```
2.19  MOV EAX, CR0
      AND EAX, FFFFFFFFH
      MOV CR0, EAX
```

```
2.20  MOV SI, 10011H
      MOV CX, 15
      MOV DX, 0A010H
L:    MOV AL, ES: [SI]
      OUT DX, AL
      NOP
      NOP
      INC SI
      LOOP L
```

2.21 IDTR 装入的地址是 00010000H；表内最大长度为 200H；该表提供 40H 个门。

2.22 ZF = 0；EAX = FFFFFFFFH，EBX 和 ECX 无变化。

2.23 使 TLB 中第 10H 项作废。

2.24 01H。

2.25 起始地址 00210000H；终止地址 002101FFH；表长 200H；存放 40H 个描述符。

2.26 00130040H。

2.27 段基址 0020000H；限长 00010110H；未装入物理存储器：代码段；00200226H。

2.28 实模式是早期的一种传统模式，功能与性能十分有限，Pentium 在开机或复位后首先进入实模式。保护模式是一种支持多任务多用户、保护各类程序高度可靠地运行的模式。虚拟 8086 模式的引入是要实现在保护模式下运行实模式的程序。多任务即是允许系统中同时存在多个任务，由 CPU 按一定管理方式进行任务转换。选择符用于检索 GDT、LDT 中描述符，具有 16 位的格式。描述符是描述一个段的属性，为 8B 长。GDT 含有操作系统以及各任务公用的描述符、各任务的 TSS 描述符、LDT 描述符、调用门和任务门。LDT 含有某一个任务专用的代码段和数据段描述符、进入更高特权级或操作系统的调用门以及进入本任务的子任务的任务门，IDT 含有整个系统的中断门、异常门和任务门。CPL 表示当前正在执行的代码段具有的访问特权级；DPL 表示段的被访问特权级；RPL 表示选择符的特权级；IOPL 表示 I/O 特权级。

2.29 32 位微处理器的保护模式和实模式在存储管理上有很大的区别，实模式只是简单地将段基址乘以 16，然后加上偏移量地址，而保护模式则要复杂很多，必须通过段管理部件将逻辑地址转换成线性地址和多种保护性检查。在允许分页的情况下，段部件转换后的线性地址还不是物理地址，还必须经过页变换。

2.30 特权级高的代码段可以访问同级或低级特权级的数据段；不允许特权级高的代码段向特权级低的代码段的控制转移；特权级低的代码段可以向特权级高的代码段控制转移。但堆栈段的特权级通常和程序执行时代码段特权级相同，当发生控制转移时，堆栈段的特权级应随之变化。

2.31 要更改，堆栈段的特权级将升高到代码段的现行特权级。

2.32 对于 IF 的置 1、清 0 可以用 STI、CLI 指令实现，对于 TF 可以用程序进行置 1 和清 0。


```
清 0:  PUSHFD
      POP EAX
      AND EAX, 0FFFFFFFh
      PUSH EAX
      POPFD
置 1:  PUSHFD
      POP EAX
      OR EAX, 00000100h
      PUSH EAX
      POPFD
```

隐操作包括中断响应时硬件的复位操作可以使 IF 和 TF 清 0。

2.33 由异常的外部事件引起的 CPU 改变程序执行流程的过程称为中断；由 CPU 内部原因而改变程序执行流程的过程称为异常。中断又分为可屏蔽中断 INTR 和非屏蔽中断 NMI。异常包括指令故障和 INT n 指令产生的软件中断。按照产生异常的指令可否重新执行和异常处理后的返回点不同，异常又可分为“Trap”、“Fault”、“Abort”。

2.34 在保护模式下，仍然使用 256 个中断，但它与实模式不同，中断处理不再以段：偏移量的双字形式寻址，而是通过保护选择符、偏移量、特权级等属性的门寻址，因为每次中断都可能对运行的程序产生很大的破坏作用，因此只有采用特权级措施才能提高安全性。CPL 转移通过门发生，且进入中断/异常服务程序或子程序时，CPL 不能降低。虽然 CPL 的代码段可以调用 $DPL \leq CPL$ 的证实代码段，但进入子程序时 CPL 并不改变。在中断/异常处理结果或转移返回时，应恢复转移前的 CPL，即 CPL 不能升高。

2.35 在实模式下，中断向量表的线性地址是由硬件设置的，占用内存最低 1024B 的地址空间。在保护模式下，中断描述符表可以放置于存储器的任意地址段，因为 IDT 的基地址以及表格大小的段界值是放在 IDTR 中，可通过指令灵活地进行设置。

2.36 IDT 的起始地址由 IDTR 控制。IDT 中表项有中断门、异常门和任务门三类；在保护模式下，通过门把程序的控制由源代码段转移到目标代码段。调用门用来改变特权级；任务门用来执行任务切换；中断门和异常门用来指定中断服务程序。

2.37 当 CPU 测试到了中断或异常时，先响应优先级高的中断或异常，通过中断类型码，获得中断服务程序的起始地址，然后保护现场，转去执行对应的中断服务程序。非屏蔽中断 NMI 通常是由严重的硬件错误引发的，当 NMI 引脚上接收到有效信号时就会立即停止当前操作，转而执行 2 号中断向量做相应的处理；可屏蔽中断 INTR 可通过复位标志寄存器 EFLAGS 中的中断标志 IF 实施封锁，在实模式下使用 CLI 指令即可达到此目的，这时处理器不会理睬该引脚上的信号变化；异常中断是指令执行过程中引起的内部异常操作处理，它不受中断允许标志 IE 的控制。

2.38 当某条指令产生异常或某条指令执行结束响应外部中断请求时，CPU 根据异常/中断向量号从 IDT 找到相应的异常门/中断门描述符。门描述符中的选择符指向一个在 GDT 或当前 LDT 中的可执行代码段描述符，此描述符内的段基址指向程序代码段的基地址；门描述符中偏移量指向相应处理程序的入口。由此 CPU 能将控制转移到处理程序。

2.39 任务切换的主要步骤：现行任务中段间 JMP 或 CALL 指令直接或通过任务门间接选择 GDT 中新任务 TSS 描述符，通过描述符（或门）特权级检查；进行新 TSS 存在性、限长

和可用性检查；保存现行任务的状态到其 TSS 中；新任务 TSS 的选择符装入 TR，选择 TSS 描述符，并装入段部件 Cache 中；从新任务 TSS 装入新任务整数部件运行环境（各种寄存器等），新任务开始运行。任务返回的主要步骤：将处理器当前状态存入当前 TSS；将返回链选择符（LINK）装入 TR 来访问前一个 TSS，使得微处理器可以恢复到前一个状态。返回到调用 TSS，由 IRET 指令来完成。

2.40 异常带或不带错误代码，产生异常指令的特权级低于或等于处理程序的特权级，这些差异使异常和中断后的堆栈结构有所差别。对于中断，只将 CS: EIP 和 EFLAGS 压入堆栈。对于异常，若两者特权级相同，则将 CS: EIP 和 EFLAGS 压入堆栈，若带错误代码，再压入错误代码；若特权级不同，还要将产生异常前的原 SS 和 ESP 压入堆栈。

2.41 实模式下的逻辑地址由 16 位段基址和 16 位偏移量构成，两个地址通过某种合并得到 20 位物理地址；保护模式下的虚拟地址也称为逻辑地址，它是程序设计所采用的地址，包括段选择符和偏移量两部分，在程序中用来指定指令和数据在存储器中的位置。在分段管理中，虚拟地址经过分段部件可转换为线性地址。若未采用分页管理，则此时的线性地址即为物理地址；在分页管理中，页式机构自动地将线性地址转换为物理地址；虚拟存储器可为用户提供一个比实际内存大得多的存储空间，当访问的程序或数据所在页不在内存时，页故障服务程序将所缺页从外存储器上调到内存，便能成功访问已调入页的程序或数据了。

2.42 保护模式与实模式的最主要区别是，提供了更完善的存储管理、保护机制与支持多任务系统。

2.43 全局地址空间即是存放系统中公用数据、代码的空间；局部地址空间即是存放某些供某些程序所应用到的数据和代码；GDT 含有操作系统以及各任务公用的描述符，各任务的 TSS 描述符，LDT 描述符，调用门和任务门；LDT 含有本任务专用的代码段和数据段描述符，进入更高特权级或操作系统的调用门以及进入本任务的子任务的任务门。

2.44 分段管理是将程序按逻辑功能或将数据按数据结构和使用情况分成若干逻辑段，并对这些段分别分配存储空间，段长可以不同，也不必连续分配，通过分段可以实现对段的保护；分页是将程序和数据分成一组称为页的容量大小相同的一些分区，并允许这些分区是非连续的，分页方法可减少碎片，并方便地实现虚拟存储技术。

2.45 由程序给出的虚拟地址包括选择符和偏移地址，选择符用作在 GDT 或 LDT 中索引段描述符，每当选择符装入段寄存器时，由索引指向的描述符自动从 T_1 位所选定的 GDT 或 LDT 中取出，装入相应的高速缓存器中。描述符中 32 位段基址与虚拟地址中 32 位偏移量相加即得到线性地址。

2.46 由 CR_3 中页目录表物理基址和线性地址中高 10 位页目录号乘以 4 确定页目录项；由页目录项找到对应页表的物理基址，再由线性地址中间 10 位页号乘以 4 确定页表项；由页表项找到对应物理页的基址；物理页基址加上线性地址中低 12 位页内偏移量得到实际物理地址。

2.47 由于选择符用 1 位的表指示和 13 位的索引值访问描述符，所以最多可以访问 2^{14} 个描述符。由于偏移量为 32 位，每个段最大为 $2^{32}B$ ，这就使得 32 位微处理器可以访问的虚拟存储器容量为 $2^{14} \times 2^{32}B = 64TB$ 。

2.48 段描述符高速缓冲寄存器对程序员而言是不可见的。每当把一个选择符装入到某个段寄存器时，处理器自动从某个描述符表中取出相应的描述符，把描述符中的信息保存到对应的高速缓冲寄存器中，此后对该段访问时，处理器都使用对应高速缓冲寄存器中的描述符信息，

而不用再从描述符表中取描述符。

2.49 段描述符，系统描述符，门描述符；GDTR，LDTR，IDTR，TR。

2.50 任务门即是在保护模式下，用来执行任务切换的门。任务门可以在 GDT、IDT 或 LDT 中，对于无权访问 TSS 描述符的代码段，通过任务门实行任务转换。任务寄存器 TR 存放当前任务 TSS 选择符和描述符，TSS 描述符可以通过 TR 寻址。

2.51 通过调用门的段间转移同当前任务与目标段的特权级有着密切的关系，此时调用或转移的目标段选择符不是指向目标段本身，而是首先指向调用门。

2.52 TSS 是一个特殊存储段，保存着一个任务的全部运行状态信息。它包含了所有寄存器值、LDT 选择符、页目录表基址、I/O 位图等。TSS 由 TR 中选择符指定的 TSS 描述符定位，它用于多任务环境中各任务的隔离、保护和转换。

2.53 TSS 描述符是位于 GDT 中的系统描述符，由 TR 中选择符选择。段基址给出了 TSS 的起始地址，它还包括了段限长，粒度位 G，存在位 P，DPL 等等。

2.54 CALL 指令使用调用门把控制权转移到比 CPL 特权级更高的子程序 NEW_ROUTINE，并且要对堆栈段进行选择符非空、描述符不越限、堆栈段的 DPL 等于代码段的 DPL、堆栈段为可写数据段、堆栈段的地址空间等合法性检查，以及一系列堆栈操作：从 TSS 中取出具有子程序特权级的堆栈指针装入 SS:ESP，将调用程序的低特权级堆栈段的选择符 0 扩展到 32 位压入具有更高特权级的新栈，将调用程序的低特权级堆栈段的 ESP 压入新栈，将调用门中字计数段中指定的几个参数从旧栈拷贝到新栈，将断点处 CS 选择符 0 扩展到 32 位后入栈，将断点 EIP 入栈，从调用门中取出子程序所在段选择符和偏移地址分别装入 CS 和 EIP，从而转到指定子程序去执行。

2.55 直接任务转换是利用段间 JMP 或 CALL 指令的选择符直接装入 TR 寄存器索引 GDT 中 TSS 描述符，间接任务转换是指令先索引 GDT/LDT 中的任务门，或发生了对应于 IDT 中任务门的中断或异常，通过任务门中的选择符修改 TR 寄存器引用新任务的 TSS 描述符实现。

2.56 80486/Pentium 的多任务系统允许系统同时存在多个任务，由 CPU 按一定管理方式进行任务转换，而每一个任务都有一个独自的局部空间，LDT 的数量与任务的多少相对应；每个任务还用了一个特殊的段，即任务状态段 TSS，它是保存任务现场条件的一个段，这样就起到了隔离作用，避免了任务间的相互干扰。对不同的任务实现不同级别的特权级保护和存储器保护，实现软件之间的隔离和禁止非法访问。

2.57 Cache 的命中率 = $[1340 / (1340 + 97)] \times 100\% = 93.25\%$

访存的总线周期包括 T_1 、 T_2 和 T 三个等待状态（另加 3 个 T_2 ），共 5 个时钟周期，故执行这个应用程序的平均等待状态数 = $(5 \times 97) / (1340 + 97) = 0.34$ 。

2.58 INVD 指令刷新片内 Cache，并分配一个总线周期刷新片外 Cache，使 Cache 的内容作废，但不会将片外 Cache 中数据写回内存；WBINVD 指令刷新片内 Cache，用一个专用总线周期将片外 Cache 内容写回内存，再用一个总线周期刷新片外 Cache，使 Cache 内容作废，但内存中保留了 Cache 的值；INVPBG 指令使转换后缓冲器 TLB 中某一项作废。如果 TLB 中含有一个存储器操作数映像的有效项，则该 TLB 项被标记为无效。

2.59 TLB 是处理器与内存之间的相联存储器，其中存放着 CPU 近期访问过页的有关信息，共有 32 页的信息。如果所访问的线性页号在 TLB 中（命中），则 TLB 可在半个时钟周期提供相应的物理地址，加快了页变换过程；如果要访问的线性页号不在 TLB 中（不命中），则按查页目标表、查页表、访问页的方法访问内存，同时将该线性地址所对应的线性页号、物理

页号、属性调入 TLB。当 TLB 满时，还要按 LRU 算法调出最近最少使用的页和调入刚未命中的线性地址所在的页。

2.60 从 80486 开始，其 CPU 芯片内都集成了数 KB 或数十 KB 容量的 Cache，称为第一级高速缓冲存储器，即 L1 Cache，而且芯片外或者芯片内部还可以接更大容量的 L2 Cache。80486 L1 Cache 是用 8KB 的指令和数据 Cache；Pentium 和 Pentium II 内部的 L1 Cache 有 8KB 的指令 Cache 和 8KB 的数据 Cache，而 L2 Cache 是外接的，只能安装在主板上；Pentium III 中 L1 Cache 的容量增加到代码 Cache 与数据 Cache 各为 16KB，而 L2 Cache 与 CPU 内核芯片封装在同一块基板上。英特尔的 32 位微处理器一般都配有 256KB 同速 L2 Cache。以 Thunderbird 为核心的 AMD Athlon，L1 Cache 有 64KB 指令 Cache 和 64KB 数据 Cache，L2 Cache 为 256KB。

2.61 第一，M 状态 Cache 行仅存在于本 CPU 的 Cache 中，它被修改过，和内存中相应内容不同；第二，E 状态 Cache 行仅存在于本 CPU 的 Cache 中，它未被修改过，和内存中相应内容相同；第三，S 状态 Cache 行同时存在于其他 CPU 的 Cache 和内存中；第四，I 状态 Cache 行表示其数据是无效的，即它们是空的，或者是未更新过的过时数据。对 I 状态进行 Cache 行的填充可以转移为 E 状态（当对应物理地址不存在于其他的 CPU 的 Cache 中时）或 S 状态（当对应的物理地址已存在于其他的 CPU 的 Cache 中并且其他 CPU 也处于 E 状态或 S 状态时，若其他 CPU 的 Cache 状态为 M，则还会在填充前引起其 Cache 行内容写入内存）。写 S 状态 Cache 行，将引起本 Cache 行转移到 E 状态，还会引起其他 CPU 的 Cache 行从 S 状态转移为 I 状态。写 E 状态 Cache 行，引起转移为 M 状态。

2.62 80486 内部 8KB 容量的 Cache 是一个 4 路组相联存储器；Pentium 内部每个 8KB 容量的 Cache 均为 2 路组相联存储器。

2.63 80486 内部有 8KB 的指令 Cache 和数据 Cache；Pentium 内部有独立的 8KB 指令 Cache 和 8KB 数据 Cache。

2.64 80486 CPU 的内部 Cache 采用通写方式，Pentium CPU 的内部 Cache 采用回写与通写相结合方式。

2.65 Pentium 和 80486 的 Cache 数据行（Line）分别为 256 位和 128 位。

2.66 Pentium 缓存式存储器读/写总线周期是突发式数据传送；可以是非流水线式总线周期，也可以是流水线式总线周期。

2.67 对片内数据 Cache 行采用通写方式。

2.68 $CD=0$ ，Cache 工作； $NW=0$ ，对内部 Cache 进行通写，即将数据写入 Cache，也写入内存。

```
2.69  MOV  EAX,CR0
      OR   EAX,60000000H
      MOV  CR0,EAX
```

2.70 依次为 20 位地址屏蔽输出信号；上个读周期数据奇偶错输出信号；8 位总线宽输入信号；16 位总线宽输入信号；地址输出信号；非突发周期就绪输入信号；突发周期的 $\overline{RDY}$ ；突发周期结束输出信号；强制 CPU 立即放弃系统总线输入信号；Cache 使能输入信号；清除片内数据 Cache 输入信号；浮点错误输出信号；忽略数字错误输入信号。

2.71 依次为询问周期地址奇偶错输出信号；地址 A20 屏蔽输入信号；上个读周期数据奇偶错输出信号；地址选通输出信号；奇偶校验允许输入信号；缓存式读/写输出信号；突发就绪信号；通知输出下一地址信号；强制 CPU 立即放弃系统总线输入信号；Cache 使能输入信号；

命中 Cache 行输出信号; 浮点错信号; 总线周期错输入信号; 忽略数字错误输入信号; 锁定信号; 系统管理模式工作输出信号。

2.72 在某些指令前加 LOCK 前缀, 使 CPU 在执行这些指令时发出 $\overline{\text{LOCK}}$ 信号; 在某些种类的存储器操作时, CPU 会自动发出 $\overline{\text{LOCK}}$ 信号。

2.73 当 CPU 收到 HOLD 高电平且未锁定时, 在当前总线周期完成后, 让出系统总线控制权, 其地址总线、数据总线、 $\text{DP}_3 \sim \text{DP}_0$ 、PCD、PWT、 $\text{W}/\overline{\text{R}}$ 、 $\text{D}/\overline{\text{C}}$ 、 $\text{M}/\overline{\text{IO}}$ 、 $\overline{\text{PLOCK}}$ 、 $\overline{\text{ADS}}$ 、 $\overline{\text{BLAST}}$ 等引脚呈高阻态, 并发出 HLDA 高电平通知其他总线主控设备可以使用系统总线。当 CPU 收到有效的 $\overline{\text{BOFF}}$ 信号后, 立即释放总线控制权, 且不用输出响应信号。

2.74 非流水线式数据输出周期。

2.75 发出 $\overline{\text{KEN}}$ 低电平有效信号和 $\overline{\text{CACHIE}}$ 低电平有效信号。

2.76 0; 0。

2.77 当信号为低电平时, 在流水线式总线周期中通知 CPU 将下一个地址输出到地址总线上。

2.78 80486 的总线周期可以分为单周期或多周期的、突发或非突发的、可高速缓存或不可高速缓存的、32 位或 16 位或 8 位的, 以及总线锁定、总线保持、行无效、中断、重启等周期; Pentium 在 80486 的基础上增加了流水线式的和非流水线式的总线周期。

2.79 见《微机系统与接口技术》教材图 2.56 和图 2.63。

2.80 Pentium 在 80486 的基础上增删了一些引脚, 见本书 2.1.4 小节的 (4)。

2.81 Pentium 采用 $\overline{\text{APCHK}}$ 信号进行地址奇偶位检测。Pentium 在地址总线增加了奇偶校验功能, 它输出的地址信号 $\text{A}_{31} \sim \text{A}_3$ 会产生一个偶校验位, 在 AP 信号线上输出, 存储器子系统可据此对地址进行校验; 在询问周期中, 要对请求的地址进行校验操作, 分别由 $\overline{\text{EADS}}$ 和 AP 输入外部地址有效和奇偶校验信息。如果发现奇偶校验错, 则在 $\overline{\text{APCHK}}$ 信号线上输出低电平。数据总线进出的数据奇偶性在 $\text{DP}_7 \sim \text{DP}_0$ 上反映, 每个字节 1 位, 如果奇偶出错, 输出低电平的 $\overline{\text{PCHK}}$ 信号; $\overline{\text{PEN}}$ 输入信号是规定在发生奇偶错时是否允许自动执行异常 18 的处理。

第 3 章

3.1 ROM; RAM。

3.2 一次可编程 ROM; 可擦除可编程 ROM; 电可擦除可编程 ROM; 闪存; 静态随机存储器; 动态随机存储器; 扩展数据输出 RAM; 同步动态随机存储器; 高速缓存动态随机存储器。

3.3 EPROM: 脱机状态下用紫外线照射, 所有电路中的悬浮硅栅上的电荷会形成光电流泄漏, 使电路恢复起始状态。EEPROM: 在某些引脚上加合适电压即可进行字节编程, 编程前自动擦除该单元内容。Flash Memory: 用电擦除, 但只能进行分块擦除而不能按字节擦除。

3.4 假定外部译码逻辑已经产生有效信号 $\overline{\text{CE}}$ 和 $\overline{\text{OE}}$, 接着将地址加到 EPROM 的地址输入端并在该电路内部译码以选择要访问的存储单元。在经过延迟时间 t_{ACC} 之后, 这个存储单元中的数据在数据输出端上有效。

3.5 SRAM 的基本存储电路为半导体双稳态触发器, 而 DRAM 是利用半导体 MOS 电容充电放电表示 1 和 0。因电容不断泄漏电荷, DRAM 需要定期刷新, 用读操作来实现, 故 DRAM 的存取速度比 SRAM 慢, 但 DRAM 的存储密度比 SRAM 高且成本低、功耗小、易于封装。

3.6 ①RAM 运行时既可以读出信息, 也可以写入信息, 但断电后将丢失存储内容; 而 ROM

在线运行时其存储内容是非易失性的、只读的。②RAM 可以在线写信息；EPROM 在每次写操作之前须用紫外线照射 15~20 分钟以擦去所有信息。

3.7 整体擦除结构的存储阵列是一个单一的块。自举块和快擦写文件结构的存储阵列是由多个独立的可擦/写块组成的。

3.8 需要的 RAM 芯片数	片内地址位数	片选地址位数
(1) 256	9	5
(2) 128	10	4
(3) 16	11	3
(4) 2	13	1

3.9 微机系统中往往要使用多个存储芯片，而 CPU 每次访问内存时只能选择构成一个字节所必须要访问的存储芯片。所以，CPU 要实现对存储单元的访问，首先要选择存储芯片，这称为片选；然后再从该芯片中依地址码选择出相应的存储单元以进行数据的存取，即所谓的字选。

3.10 刷新是指通过对 DRAM 进行“读出”来补充存储单元的 MOS 栅极电容电荷以保持原单元的内容。

刷新周期是指从上一次对整个存储器刷新结束到下一次对整个存储器全部刷新一遍为止的时间间隔。

动态 RAM 的刷新时间间隔不大于 2ms 时所存信息才不会丢失。

3.11 集中式刷新指在一个刷新周期内，利用一段固定的时间依次对存储器的所有行逐一再生，在此期间停止对存储器的读/写。缺点：在刷新期间不能访问存储器，有时会影响系统正常工作。

分散式刷新是把对每一行的再生分散到各个工作周期中去，使得整个存储器的刷新周期缩短了。缺点：频繁的刷新操作会影响 CPU 的工作，使存储器不能高速运作。

透明刷新不占用 CPU 时间，将刷新操作安排在机器执行某种内部操作或访存周期中的空闲时段进行。

3.12 略。

3.13 支持流水线式和突发式总线周期，可进行页式和存储体交替式访问，并支持对高速缓存的回写修改策略。

3.14 需要 32 片； $A_{11} \sim A_0$ 参与片内寻址； A_{13} 、 A_{12} 作片选信号。

3.15 存取速度(存取时间、读周期时间)、存储容量，价格/位。

3.16 参见自测练习问答题第 10 题。

3.17 采用字结构方式组织。

3.18 EDO RAM 技术假定下一个要访问的存储单元的地址和当前正在访问的地址是连续的，在当前的读/写周期中就启动下一个读/写周期，从而使 DRAM 的存取速度提高约 30%。EDO RAM 在工作时与 CPU 外频时钟不同步。

同步 DRAM 技术将 RAM 和 CPU 通过一个相同的时钟锁在一起，使 RAM 在工作时与 CPU 的外频时钟同步，从而解决了 CPU 和 RAM 之间速度不匹配的问题。从理论上讲，CPU 从这种内存读取数据时不需要等待周期。

高速缓存 DRAM 技术把高速的 SRAM 存储单元集成在 DRAM 芯片内部作为 DRAM 的内部 Cache，Cache 和 DRAM 单元之间通过片上总线连接。

第 4 章

4.1 CPU 与外设之间的连接电路及相关软件称为 I/O 接口。I/O 接口有以下主要功能：数据缓冲、接收和执行 CPU 命令、信号转换、设备选择、中断管理、数据宽度变换、可编程。

4.2 I/O 端口是指 I/O 接口中供 CPU 直接访问的那些寄存器或特定的硬件电路。它的编址方式有两种：端口统一编址方式和端口独立编址方式。

4.3 见《微机系统与接口技术》教材 4.2.3 小节。

4.4 定时提供的是时间基准，分为内部定时和外部定时。定时与计数本质上一致，只是计数的信号随机，定时的信号具有周期性。微机系统中需要定时信号，如系统的日历时钟，动态存储器的刷新，应用系统的定时中断、定时查询与检测等。定时方法有软件定时、不可编程硬件定时和可编程硬件定时。可编程定时/计数器本质上都是脉冲计数器，定时计的是内部基准源产生的脉冲，计数是计外部脉冲。

4.5 82C55A 有 3 个与外设相连的端口, 分别是 A 端口(PA)、B 端口(PB)和 C 端口(PC)。各端口可以独立工作, 由程序将它们设置成各种不同的工作方式。通常 A 端口或 B 端口作为 I/O 的数据端口, 而 C 端口作为控制或状态端口。

4.6 82C55A 的可编程控制字有两种。一是方式控制字 ($D_7=1$)，在用输出指令将方式控制字写入 82C55A 后，它将被分存于 A、B 两组控制寄存器中。 $D_6\sim D_3$ 这 4 位控制 A 组的工作方式以及 I/O， $D_2\sim D_0$ 这 3 位控制 B 组。二是端口 C 位设置字 ($D_7=0$)，端口 C 位主要起联络信号作用，对于每一位，可用程序来置位和复位，从而输出控制信号。

4.7 异步串行通信以字符为基本单位，传输时字符间异步，字符内部各位间同步。采用这种方式双方不需要严格同步，因而收发双方无需用同一时钟源，只需用同频率的收发时钟。同步串行通信则要求在传输时字符与字符之间、字符内部的位与位之间严格同步，所以收发双方需用同一时钟源，在通信开始时需要用同步字符或同步信号来实现收发双方的同步。异步方式用于数据量小、速度较慢的串行通信，同步方式用于数据量大、速度较快的串行通信。

4.8 参见《微机系统与接口技术》教材表 4.7 的说明。

4.9 参见《微机系统与接口技术》教材图 4.28。

4.10 数据发送部分包括,发送保持寄存器 (THIR) 和发送移位寄存器 (TSR) 两部分,而数据接收部分包括,接收移位寄存器 (RSR) 和接收缓冲寄存器 (RBR) 两部分;CPU 用 OUT 指令将一个 8 位的控制字写入线路控制寄存器 (LCR),以决定通信中数据的格式,CPU 读入线路状态寄存器 (LSR) 来了解数据发送和接收的情况。Modem 控制寄存器 (MCR) 和 Modem 状态寄存器 (MSR) 主要实现通信过程中的联络功能,包括联络信号生成及状态检测,ins 8250 具有可编程中断管理功能,用户可以通过对中断允许寄存器 (IER) 与中断识别寄存器 (IIR) 的读/写操作来对 8250 的 4 类共 10 种中断源设置允许禁止,识别最高优先级中断请求,依次响应及处理。

4.11	MOV	DX,	0A00H	;	寻址 I/O 端口 A00H
	MOV	DI,	1001H	;	用 DI 寻址目标存储块的首地址
	MOV	CX,	0FH	;	装入计数值
	CLD			;	地址自动加 1
	REP	INSB		;	字节串输入指令从 A00H 端口连续输入 16 个字 节的数据存入 ES:DI 指出的 1001H~1020H 存储单元

```
4.12  MOV SI, 1001H
      MOV CX, 0FH
      MOV DX, 0A010H
      CLD
      REP OUTSB
```

```
4.13  (1) MOV AL, 00110000B
      OUT 4CH, AL
      MOV AL, 34H
      OUT 40H, AL
      MOV AL, 12H
      OUT 40H, AL
```

```
(2) MOV AL, 01100101B
      OUT 4CH, AL
      MOV AL, 56H
      OUT 44H, AL
      MOV AL, 02H
      OUT 44H, AL
```

```
(3) MOV AL, 10111000B
      OUT 4CH, AL
      MOV AL, 0FFH
      OUT 48H, AL
      MOV AL, 1FH
      OUT 48H, AL
```

```
4.14  MOV AL, 80H
      OUT 4CH, AL
      IN AL, 48H
      MOV AH, AL
      IN AL, 48H
```

```
4.15   $11 \times \times \times 10 \times B$ 
```

4.16 A 端口: 方式 0 输出。B 端口: 方式 0 输入。C 端口: 方式 0 输出。
PA₇~PA₀ 输出, PB₇~PB₀ 输入, PC₇~PC₀ 输出。

4.17 一个数据占 10, 因此总共的数据位数为 $10 \times 5\,000 \times 2 = 100\,000$, 所以, 所需时间 = 数据位数/波特率 = $100\,000 \text{ 位} / 1200 \text{ 位/s} = 83.333 \text{ s}$ 。

```
4.18  0060H
```

第 5 章

5.1 中断方式与 DMA 方式都是数据交换方式的一种。与中断方式相比, DMA 方式可使数据传送响应时间大大缩短。因为 DMA 方式的数据传送不需要 CPU 介入, 不像中断方式那样

需要等一条指令执行结束才能进行中断响应。此外由于不利用 CPU 来控制数据传送,在响应 DMA 请求、进入 DMA 方式时,就不必保护 CPU 的现场。中断方式是利用硬件请求、软件传输的方式, DMA 是用硬件实现请求和传输的方式。

5.2 CPU 在执行当前程序的过程中,由于某种随机出现的外设请求或 CPU 内部的异常事件,使 CPU 暂停正在执行的程序而转去执行相应的处理程序;当处理程序运行完毕后, CPU 再返回到暂停处继续执行原来的程序。

5.3 为了解决 CPU 与外设之间的信息交换问题而引入了中断系统。引起中断的事件或发出中断请求的来源称为中断源。中断可分为硬中断和软中断两大类。32 位微处理器把中断分为中断和异常。中断分为可屏蔽中断和非屏蔽中断;异常分为自陷、故障和中止等 3 种。

5.4 当一个外设发出中断请求时,请求信号由相应的中断请求线 IR 输入到 8259A,使 IRR 相应位置 1,并且它在现行请求的中断中优先级是最高的。若 IMR 中相应位没有置位,且优先级高于正在服务的中断或没有正在服务的中断,则由控制逻辑向 CPU 发出高电平有效信号 INT,请求中断服务。CPU 响应该中断请求后,在第一个总线周期,向 8259A 发送第一个 $\overline{\text{INTA}}$ 脉冲。而优先权判决电路将允许中断的最高优先级请求位选通到 ISR,使 ISR 相应位置位。与此同时,IRR 中相应的请求位被清 0。在第二个总线周期, CPU 再次发出一个 $\overline{\text{INTA}}$ 脉冲, 8259A 接收第二个信号后,通过引脚 $D_7 \sim D_0$ 送出中断类型码。若 8259A 被设置成自动结束中断方式,则在这个脉冲开始时,ISR 相应位被复位。否则,将在中断服务结束时,由软件发出中断结束命令 EOI 使 ISR 相应位置位。

5.5 CPU 响应中断时根据中断源提供的信息,转向中断服务程序的中断称为向量中断。中断服务程序入口地址称为中断向量。16 位处理系统的中断向量表位于存储器的前 1024B 中,地址为 000000H~0003FFH。它包括 256 个不同类型的 4B 中断向量。中断向量包含中断服务程序的段地址和偏移量。

5.6 一般全嵌套:这是 8259A 最普通的工作方式。若在对 8259A 初始化以后,没有设置其他优先级方式,则自动进入。此时中断优先级固定为 IRQ_0 最高, IRQ_7 最低,较高级中断请求可中断低级中断服务,且屏蔽同级和低级请求。系统按这种方式工作需满足以下条件:主程序必须执行开中断指令 (STI),使 $\text{IF} = 1$,才有可能响应中断;因进入一个中断服务程序,系统会自动关中断,所以中断服务程序须再次开中断,才有可能嵌套更高级的中断;中断服务程序结束时,须发出 EOI 中断结束命令,使 ISR 中的对应位清“0”,解除对同级和低级中断请求的屏蔽。

特殊全嵌套:这种方式适用于多片 8259A 级联且响应的中断优先级保存在各从片中的大系统,此时特殊全嵌套仅设置在主片中。它和一般全嵌套方式基本相同,所不同的是在特殊全嵌套方式下,当主片处理某一级中断时,可响应同级的中断请求,从而实现对同级中断请求的从片的一般全嵌套。

5.7 一般屏蔽方式:该方式通过写 OCW_1 使 IMR 中某一位或者某几位置为 1,可将相应 IR 上中断请求屏蔽掉,且不影响其他 IR 上请求;当某一中断请求被响应时,将屏蔽同级和低级的中断请求。

特殊屏蔽方式:在某些场合,在执行某优先级中断服务程序时,又要允许某些优先级更低的中断请求被响应,此时可采用特殊屏蔽方式。这时先用 OCW_1 来置 1 IMR 中本级中断屏蔽位,再用 OCW_3 使 $\text{ESMM} = 1$ 、 $\text{SMM} = 1$ 。此时除正在服务的本级中断被屏蔽外,其余所有等级的中断都被开放。

5.8 如果一个中断服务程序被若干个中断请求级公用,则利用查询方式,CPU 就不需要执行中断响应周期,从而减少了程序量,节省了存储器空间。此外,当中断系统需要扩大到 64 级以上时,可以使级联增加到第三层,前两层仍以向量方式响应中断,而第三层以查询方式响应。CPU 可通过设置 OCW₃ 的 P=1,进入查询方式,以接着其后的一次输入操作读信号作为 CPU 的中断响应信号。

5.9 CPU 在响应外部 INTR 请求后,在第一个总线周期会向 8259A 发送第一个 $\overline{\text{INTA}}$ 脉冲。而优先权判决电路将允许中断的最高优先级请求位选通到 ISR,使 ISR 相应位置位。与此同时,IRR 中相应的请求位被清 0。在第二个总线周期,CPU 再发送一个 $\overline{\text{INTA}}$ 脉冲,8259A 接收第二个信号后,通过引脚 D₇~D₀ 送出中断类型码。若 8259A 被设置成自动结束中断方式,则这个脉冲使 ISR 相应位复位。否则,将在中断服务结束时,由软件发出 EOI 命令使 ISR 相应位复位。

5.10 对于级联的主从结构,CPU 发出的第一个 $\overline{\text{INTA}}$ 脉冲用来通知所有 8259A 有一个中断请求已被响应。主 8259A 在级联线 CAS₀~CAS₂ 上输出被响应的从 8259A 的标志码,在第二个 $\overline{\text{INTA}}$ 脉冲期间,只有本身的标志码和级联线上标志码相同的从 8259A 才能将其相应的中断类型码送上数据总线,供 CPU 读取。由于在中断响应过程中,主从 8259A 上相应的 ISR 位均会被置位,因此,在中断结束时,要分别向主从 8259A 发出中断结束命令 EOI。

5.11 用 DMA 方式进行主机与外设的数据交换时,数据在存储器与外设之间直接开辟的高速数据通路上传送,不需要 CPU 干预。这时用一个总线周期就能完成数据在存储器和外部设备之间的传送,数据交换速度仅受存储器的存取速度和外设传输特性的限制。对于快速、大数据量、多处理机和突发传送等应用,通常采用 DMA 方式进行数据交换。

5.12 DMAC 允许外部设备直接与系统存储器通信,有的 DMAC 也提供存储器到存储器、外设到外设的直接数据传送功能。它具有多种可编程的控制特性,可以在程序控制下对系统进行动态配置,实现系统最佳化,提高数据吞吐量。DMAC 可以工作在 CPU 的从属 I/O 接口芯片方式和系统总线主控设备的方式下。

5.13 DMAC 需要有定时和控制逻辑、优先权控制逻辑、命令控制逻辑、地址和数据总线缓冲器,此外还通过系统总线与 CPU 子系统接口或 DMAC 自己控制系统总线。DMAC 还需要有可编程的控制和状态寄存器,加 1 功能的地址寄存器和减 1 功能的字计数器等。定时和控制逻辑根据初始化编程时设置的工作方式寄存器的内容和命令,在输入时钟的定时控制下,产生 8237 内部定时信号和控制信号。优先权控制逻辑裁决各通道的优先权次序,解决多个通道同时请求 DMA 服务时可能出现的优先权竞争问题。命令控制逻辑对处理器送来的编程命令进行译码。

5.14 8237DMAC 的主要功能:①4 个独立的 DMA 通道,通过级联可扩充。②每个通道有 64KB 的寻址和计数功能、DMA 请求允许和禁止功能、可独立自动预置功能、软件设置 DMA 请求功能。③除了存储器与 I/O 传送外,还可以进行存储器之间的 DMA 传送。④具有 DMA 读、写和检验三种操作类型。⑤具有单次、成组、请求、级联四种传送方式。⑥在 DMA 结束时产生有效 $\overline{\text{EOP}}$ 终止 DMA 传送。

5.15 在单片 8237 系统中,HRQ 通常接到 CPU 的 HOLD 引脚,用来向 CPU 请求占用总线,当出现有效的 DREQ 信号,且请求没有被屏蔽时,8237 就会立即发出 HRQ 信号。在 HRQ 信号有效后,至少等待一个 DMA 时钟,HLDA 才会有效。当 HLDA 为高电平时,表示 CPU 已经将总线控制权让给 DMAC。DMAC 将在适当的时候发出有效的 DACK 信号通知外设 DMA

请求被响应。

5.16 见《微机系统与接口技术》教材 5.3.2 小节叙述及表 5.3。

5.17 见《微机系统与接口技术》教材 5.3.3 小节叙述及图 5.10。

5.18 由于 PC/XT 机器的系统地址总线为 20 位, 而 8237 只能提供 16 位地址。为了能访问所有存储单元, 必须加上高 4 位地址。为此, 在 PC/XT 系统 DMA 控制电路中设有一块 4 组 4 位寄存器堆 74LS670, 以存放 4 个 DMA 通道的高 4 位地址。当 $\overline{\text{DMAAEN}}$ 为低电平时, 页面寄存器的输出允许端 READ 有效, 送出选中通道的高 4 位地址, 与 8237 送出的低 16 位地址一起形成 20 位地址, 从而可以寻址 1MB 存储单元。PC/XT 系统有 16MB 内存空间, 每个通道及刷新电路都有一个 8 位页面寄存器来提供高 8 位地址, 这 8 位地址是在非 DMA 期间由 CPU 写入相应通道的页面寄存器中的。

5.19 参照 PC/XT 系统加电自检时对 DMA 系统进行检测的方法对 PC/AT 系统级联的 2 片 8237 芯片所有通道的寄存器进行读/写测试。

5.20 82380 是 32 位微机系统设计的高性能、多功能 I/O 接口芯片, 采用 132 个引脚 PGA 封装。它有一个 32 位 8 通道高性能 DMA 控制器, 一个 20 级可编程中断控制器及 4 个 16 位可编程定时器/计数器。此外, 芯片内部还包含多个系统支持电路, 如可编程等待状态控制电路、DRAM 刷新控制电路、内部总线仲裁和控制电路以及系统复位控制电路等。采用此芯片可大大减少组成计算机系统所需外部接口电路的数量。由于采用 32 位接口, 系统 I/O 速度得到很大提高。

第 6 章

6.1 总线是计算机中各个模块间传递信息的一组信号线的集合。它为模块内和模块间的各部件提供了标准的信息通路。利用不同的总线, 可以实现芯片之间、部件之间、主机与外设之间、系统与系统之间的连接与通信。接口是介于主机和外设之间的缓冲电路, 解决由于信息格式不同、数据总线不允许被某个设备长期占用、CPU 与外设之间速度不匹配等问题。

6.2 总线可以按照其功能、性能和级别分类。

6.3 每种总线标准都有详尽的规范说明, 一般包括下列内容: ①机械结构规范, ②功能结构规范, ③电气规范。

6.4 参考例 6.5。

6.5 标准局部总线目前有两种: 一种是 1993 年 Intel 公司发布的 PCI 总线。目前该总线可分为 PCI 1.0 和 PCI 2.0。另一种是 VL-BUS。局部总线是 32 位高速总线, 具有 132MB/s 的数据传送率。其优点是采用了开放性结构、协议简单、传输率高、价格低廉、支持多种硬器件。

6.6 USB 是康柏、DEC、IBM、Intel、微软、NEC、NorTel 等 7 大公司联合推出的基于 PC 平台的即插即用型的串行总线标准。特点: 即插即用总线, 支持热拔插功能, 传输带宽可以从几 kb/s 到几百 Mb/s, 速率最高可达 480Mb/s, 支持准同步数据传输和异步消息传输的混合模式, 允许多台设备同时工作, 协议开销小, 总线利用率高, 支持实时的语音、音频和视频数据传送, 可以向 USB 总线上的设备供电, 总线上的设备也可以自备电源。

6.7 ① 有极高的数据传送速度; ② 支持硬盘和其他设备。

6.8 STD 总线适用于工业现场监测与控制, 工作环境适应范围宽, 实时性高, 组态灵活, 开发手段简易。

6.9 IEEE 488 总线的主要规定有如下几方面。

- (1) 数据传送速度不超过 1MB/s。
- (2) 连接在总线上的设备（包括作为主控器的微计算机）不超过 15 个。
- (3) 设备间的最大距离不超过 20m。
- (4) 整个系统的电缆总长不超过 220m，若电缆长度超过 220m，则会因延时而改变定时关系、降低可靠性，需通过附加 Modem 来解决。
- (5) 所有数据交换都必须数字化。
- (6) 使用 24 线的组合插头座，并且采用负逻辑，即用小于 0.8V 的电平表示逻辑“1”，用大于 2V 的电平表示逻辑“0”。

6.10 IEEE 488 总线上数据传送采用异步“握手”（挂钩）联络方式，即用 DAV、NRFD 和 NDAC 这 3 根线进行“握手”联络

第 7 章

7.1 系统板上的键盘接口电路按照键盘代码串行传送的应答约定，接收键盘发送的扫描码，同时向键盘发送主机的控制信号。

一旦键盘上有键被按下，键盘就将按键的扫描码通过键盘数据线和键盘时钟以串行方式送到键盘接口电路。

键盘输入与主机 CPU 是异步的，主机系统以中断请求方式支持键盘的随机输入。每当键盘接口电路从键盘收到串行的按键扫描码，并转换为并行的按键扫描码时，就向主机 CPU 发出硬件中断请求，主机响应该中断并在该中断程序的控制下从键盘接口电路读取按键扫描码，转换为相应的 ASCII 码，保存在键盘缓冲区或仅设置某种键盘状态。

7.2 命令译码器用来对输入信号译码，对数据锁存器进行相应的操作；数据收发器处于系统和内部的数据总线之间，数据的输入/输出、控制信号的输出和状态信号的输入都经过数据收发器。

7.3 用作打印机接口。

7.4 MC 6845CRTC 具有的特点：

- ① 字母数字和图形显示。
- ② 可以通过处理器数据总线进行全部编程，可以为任一种字符、图形产生定时信号，也可以产生水平同步和垂直同步信号。
- ③ 有硬件滚屏能力，并提供光笔接口。
- ④ 有光标寄存器和比较电路，光标的格式和闪烁可以编程。
- ⑤ 可以通过编程选用逐行扫描方式或隔行扫描方式。
- ⑥ 能够提供 14 位刷新地址和 5 位扫描线地址。

7.5 显示方式有：①字符显示方式；②图形显示方式。应用较为广泛的标准显示系统有：①MDA(Monochrome Display Adapter)，②CGA(Color Graphics Adapter)，③EGA(Enhanced Graphics Adapter)，④VGA(Video Graphics Array)，⑤TVGA。

7.6 CGA 的各种图形和字符显示功能是通过其卡上的许多寄存器来控制的，这些寄存器的长度为一个字节，通过 I/O 指令进行访问。直接对这些寄存器编程，可以大大加快图形的显示速度。寄存器的具体功能见教材。

7.7 ①CPU 在显示器水平和垂直回扫期间，把要显示的信息写入显示缓存。②CRTC 6845 一方面产生水平同步、垂直同步信号；另一方面按时产生显示缓存地址、读出显示信息。在一帧显示时间内，遍读显示缓存的 1 页或全部图形。③在字符模式下，读出的字符代码经过字符

发生器,按扫描线序号转换为点阵信息,再经过并/串转换,成为和扫描线同步的点阵串行信息进入彩色编码器。字符属性信息再经过转换进入彩色编码器,产生红、绿、蓝和亮度信号。在图形方式下,读出像素信息,经过并/串转换变成和扫描线同步的串行图形信息,进入彩色编码器。根据彩色选择寄存器选定色组,彩色编码器产生红、绿、蓝和亮度信号。④信号以及水平、垂直同步信号经过插座与显示器相连。

7.8 见例题精析。

7.9 见例题精析。

7.10 当“A”被按下时,检测电路产生一个高电平有效的按键状态信号 KEY DEPRESSED 送 8048 的测试端口 T_1 , 8048 停止扫描并以当前计数值作为按键的位置码(即按键扫描码)。在 8048 P_{21} 端输出的键盘时钟 KBD CLOCK 的同步下,将 8048 P_{22} 端串行输出按键的扫描码,送往键盘接口电路。键盘接口电路用键盘时钟同步接收键盘数据。当接收了整个串行序列,并经奇偶检验正确时,将键盘的按键扫描码转换为并行的与标准键盘兼容的系统扫描码,作为一个字节保存到单片机的输出缓存中。系统扫描码经过 CRT 处理后,由显示器显示输出。

7.11 光笔的输出接到 CGA 接口,如果有光信号,CRTC 6845 就把与该点对应的显示器缓存单元的偏移地址放在 R_{16} 、 R_{17} 寄存器组内。由 CGA 接口内的一个光笔触发器控制 R_{16} 、 R_{17} 寄存器组内存放的地址。

7.12 在 EGA/VGA 卡中,VRAM 被分成 4 个位平面,这 4 个位平面占有相同的地址空间。显示字符时,字符的 ASCII 码放在位平面 0,属性码放在位平面 1,字符集调入位平面 2,并可以根据需要调入不同的字符集(EGA 卡有 4 套,VGA 卡有 8 套)。

图形显示时,将 VRAM 中 4 个位平面上相同位置的 1 位组合起来,形成一个 4 位的二进制数来表示对应的像素的颜色。

7.13 见知识要点。

7.14 见知识要点。

7.15 EGA/VGA 中,VRAM 被分成 4 个占有相同地址空间的位平面,显示字符时,字符的 ASCII 码放在位平面 0,属性码放在位平面 1,字符集调入位平面 2,并可以根据需要调入不同的字符集。显示图形时,屏幕上的每一个像素直接同 VRAM 中的二进制信息相对应,以确定一个像素的颜色。

像素值作为一个指针指向 16 个调色寄存器(又称为彩色索引寄存器,编号为 0~15)之一。对 EGA 卡,调色寄存器长度为 6 位,其中存放的就是彩色值;对 VGA 卡,调色寄存器长度为 8 位,且这 8 位并不作为彩色值,而是作为一个指针,指向 256 个彩色寄存器(又称为 DAC 寄存器,编号位为 0~255)之一,彩色寄存器的长度为 18 位,存放的是彩色值。

7.16 见教材 7.3.5 EGA/VGA 编程中关于这 6 类寄存器组的说明。

第 8 章

8.1 见例题精析例 8.1。

8.2 EIDE 对于不用 BIOS 存取磁盘的操作系统,采用自动配置法,在访问磁盘驱动器时旁路 BIOS,使用标准的驱动器参数表将 IDE 驱动器容量信息直接传送给操作系统;对于 DOS 和 Windows 操作系统,采用自动转换法,在上电自检时,BIOS 建立和填写增强的驱动器参数表,表中包含标识驱动器命令返回并经转换得到的信息和 BIOS 传给操作系统的信息。

8.3 系统初启总是先进入总线空闲阶段,然后进入仲裁阶段,在此阶段,各总线设备将

自己的 ID 号通过对应的数据线送往 SCSI 适配器, 争用总线使用权, ID 号越大优先级越高。仲裁获准的设备控制总线选择与之通信的设备, 总线进入选择阶段, 在该阶段, 占用总线的启动器将目标器的 ID 号编码送数据线与目标器联系, 被选的目标器接管总线控制权, 在信息传送阶段完成 I/O 传送。

多台设备复用 I/O 通道时要进入重选阶段。一个启动器发送命令给目标器后就可释放总线, 把占用权让给其他启动器和目标器。执行命令的目标器在命令执行后要返回数据和状态时还要争用总线, 若获准则进入重选阶段。

8.4 软盘驱动器工作时, 压紧机构把盘片压紧在主轴上, 主轴电机拖动主轴, 盘片随之一起旋转, 磁头加载时, 电磁铁动作, 加载臂把磁头压向盘面, 磁头在步进电机驱动下在盘上做径向运动, 在控制器送来的方向信号和步进脉冲控制下, 步进电机使磁头定位到寻址的磁道和扇区。读操作时, 磁头的微弱信号经前置放大器进行线性放大, 然后由滤波器滤除干扰。微分后的波形再经放大、限幅、鉴零、整形, 送到控制器分离出数据和时钟信号。写操作时, 写电流由恒流源提供, 以免读信号时产生峰点漂移, 由于内圈磁道密度高于外圈的, 所以内、外圈磁道采用不同大小的写电流。

8.5 磁头、小车、导轨、盘片等封装在一个腔体内, 采用质量轻、浮力小的磁头, 接触启停工作方式, 主轴电机上装有缩短摩擦过程的制动机构, 盘片上涂有润滑剂。

8.6 RAID-0 通过两个以上的硬盘组成一个磁盘阵列, 数据被均匀等量地分成数据块, 分别放在几个硬盘中; RAID-1 系统会有一个主硬盘和至少一个做实时备份用的副硬盘, 不管存入什么数据, 系统都要将数据完整、实时地保存在副硬盘中; RAID-10 阵列中一半的驱动器采取 RAID-0 结构, 通过并行传送来提高数据传输率, 另一半采用镜像磁盘存储技术, 提高可靠性, 这样两组 RAID-0 磁盘阵列间又成为一个 RAID-1 阵列; RAID-2 在 RAID-0 的基础上, 采用海明码错误校验和位交叉技术将数据进行编码后按位交叉写到若干硬盘中; RAID-3 阵列中有一台驱动器用作奇偶校验盘, 通过编码将数据按位或字节交叉存在几个数据盘中, 而将奇偶校验位单独存在奇偶校验盘中; RAID-4 也用一个校验盘, 与 RAID-3 不同的是数据以扇区为单位, 交叉存放在数据盘中; RAID-5 将奇偶校验位按要求写到阵列中的数据盘上; RAID-6 采用分块交叉技术和两个磁盘驱动器容错技术; RAID-7 可称为存储计算机, 自身带有智能化的实时操作系统和用于存储管理的软件工具, 独立于主机, 分担计算机在存储方面的工作负担, 性能更高。

8.7 采用螺旋扫描读/写方式的磁带机, 磁头作旋转运动, 运动的轨迹与磁带长度方向成一角度; 采用多磁头平行读/写方式的磁带机, 磁头不运动, 读/写的轨迹与磁带运动方向平行。

8.8 EVENODD 结构用两个冗余盘存放检错、纠错信息, 一个盘存放水平冗余信息, 一个盘存放对角线冗余信息, 是一种双磁盘容错的阵列结构, 冗余方案优于 RAID; Crosshatch 结构的磁盘驱动器均为双端口, 每个磁盘驱动器同时挂在水平串和垂直串上, 当一个串控制器出错时, 系统降级运行, 不影响整个阵列的正常工作, 当两个串控制器出错时, 最多只影响一个驱动器访问, 其数据可通过校验组重构得到, 系统仍可运行, 是一种综合考虑驱动器容错、控制器容错和电缆容错的 RAID。

8.9 QIC 磁带机采用多磁头部件, 用 GCR 成组编码记录方式在直线磁道上串行逐行记录数据, 记录信息时从 0 磁道开始, 偶数磁道从磁带首端到磁带末端, 奇数磁道从磁带末端到磁道首端, 使用 1/4 in 盒式磁带; DAT 磁带机采用螺旋扫描方式记录数据, 两个读磁头和两个写磁头位于一个倾斜安装的旋转鼓上, 磁带宽度为 4mm; 视频盒式磁带机也采用螺旋扫描技术, 但在旋转鼓周围安装了 3 个磁头——读磁头、写磁头和伺服磁头, 此外还有一个抹除磁头, 使

用 8mm 视频盒式磁带: DLT 磁带机采用将数据置于纵向磁道上的线性、蛇形记录方式, 磁头固定, 能同时读/写多个通道。

8.10 光磁软盘将光学技术和磁记录技术结合在一起, 采用磁记录技术读/写数据, 光学技术伺服定位; 磁光盘是在光盘片基上镀上一层矫顽力很强且具有垂直磁化特性的磁性相变介质, 利用激光在磁光盘的磁性薄膜上产生热效应和磁光相互作用的磁性克尔效应或法拉第效应的原理来进行读/写的。

8.11 CD-ROM 在写入前要进行预处理, 采用 EFM 编码把 8 位数据编码成 14 位光轨道位, 这些光轨道位再采用 (2, 10) RLL 编码, 然后在磁带、硬盘或 CD-R 盘上生成光盘映像, 在文件和数据中插入同步、地址、数据类型、检错码和校正码, 最后按光盘格式顺序存储它们; 读出时, 利用激光聚焦成亚微米级激光束对光轨道上模压形成的凹坑进行扫描, 扫到边缘时, 反射率发生变化, 则表示数字“1”, 在坑内或岸上则为数字“0”。

CD-R 一般采用丙烯酸酯作为盘基材料, 在它上面蒸附或溅射碲系合金类非晶态材料薄膜或涂布其他记录介质, 将聚焦成直径不到 $1\mu\text{m}$ 的激光束照射到光盘介质上, 对微小区域加热, 烧蚀出 μm 级凹坑形微孔, 或产生结晶状态变化, 或引起物理变形, 这些不可逆的物理化学变化主要改变了记录介质的反射率。信息的写入使介质加热部分与未加热部分呈现不同的反射率, 利用此性质就可以读出所记录的信息。

在对磁光盘记录或抹除信息时, 预先把作为记录介质的磁化薄膜相对于膜面进行取向一致的垂直磁化, 然后将该磁光盘置于小于其矫顽力的弱小记录磁场中, 采用半导体激光束在几 ns 内把直径为 $1\mu\text{m}$ 左右的记录介质加热到居里点, 于是该区域的磁化将在取向相反的记录磁场方向上发生翻转, 记录下代表数据位信息的柱状磁畴, 激光关掉后, 该区域立即冷却, 磁畴方向也固定; 读出信息时则利用磁光相互作用的磁光效应, 将磁化方向的变化, 变成偏振光旋转方向的变化, 再由检偏器转换为输出光强弱的变化, 最后由光电探测器通过光电转换检出写入的信息。

相变光盘是利用一种记录材料的同素异构体中的互相转变, 即晶态和非晶态间的可逆转变完成写入和擦除的。写入信息时, 用高功率密度、脉宽为 100ns 及以下的激光脉冲照射记录介质, 使其温度上升到材料结晶熔点, 光点移动后又骤然冷却, 使之非晶化, 反射率变低。抹除信息时, 用椭圆形低功率密度的激光照射, 对介质的非晶化部分进行缓慢加热, 使之变成结晶态。读出信息时, 利用相变介质在晶态和非晶态间存在激光反射率差别的性质, 检测出这种差别, 并将记录的标志变为信息流。

8.12 高密度, 高画质, 高兼容性, 高可靠性。

第 9 章

9.1 存储器数据总线(MD): ROM 和 RAM 存储体共用的数据通路。扩展数据总线(XD): 所有 I/O 数据设备的数据通路。

9.2 IBM PC/AT 系统的中断请求 INTR 由中断控制(PIC)模块输出给 80286, 系统中使用两个 8259A 中断控制器, 每个 8259A 提供 8 个优先级的中断控制。

非屏蔽中断 NMI 逻辑模块为微处理器提供非屏蔽中断, 中断源为读/写存储器奇偶校验(PCK)和 I/O 通道检查(IO CHCK)2 个信号。

9.3 PC/AT 系统板上有 2 个 8237A DMA 控制器。DMA 控制器 1 的 I/O 地址为 000H~01FH, 含通道 0~3, 分别用于 SDLC 同步通信或 ISA 总线与外设传送数据的 I/O 通道; DMA 控制器 2 的 I/O 地址为 0C0H~0DFH, 含通道 4~7, 其中通道 4 作为 DMA 控制器 1 的级联, 通道 5~

7 用于 ISA 总线。DMA 页面寄存器的地址范围是 080H~09FH。

9.4 使用 82340 芯片组的 80386 微机主板,其核心是 80386DX CPU、80387DX 数字协处理器、82385DX 高速缓存控制器、82345 数据缓冲器、82346 系统控制器和 82344ISA 控制器一端连接在 CPU 的局部总线,另一端通过系统总线与 ISA 总线相连;82341 外围组合芯片没有直接与 80386DX CPU 相连,而通过外围扩展总线与 ISA 总线相连。

9.5 SD 总线多路转换器的输入数据源是 $XD_7 \sim XD_0$ 、 $SD_{15} \sim SD_0$ 、 $D_{31} \sim D_0$ 、 $MD_{31} \sim MD_0$;输出为其中之一。

9.6 CPU 数据总线: $D_{31} \sim D_0$ (32 位), TTL 电平输入/输出信号(I/O-TTL)。存储器数据总线: $MD_{31} \sim MD_0$ (32 位), TTL 电平输入/输出信号(I/O-TTL)。系统数据总线: $SD_{15} \sim SD_0$ (16 位), TTL 电平输入/输出信号(I/O-TTL)。外围电路数据总线: $XD_7 \sim XD_0$ (8 位), TTL 电平输入/输出信号(I/O-TTL)。

9.7 82345 芯片中奇偶校验码生成电路和奇偶校验电路仅对存储器数据总线(MD)的数据进行校验。①在读总线周期中,保存在 DRAM 中的奇偶校验信息经奇偶校验位传送线($PAR_3 \sim PAR_0$)输入到 82345 中,奇偶校验信息收发器经过锁存器后成为奇偶校验电路的输入,同时字节允许信号($LBE_3 \sim LBE_0$)也锁存在 82345 中成为另一组输入。最后,经 $MD_{31} \sim MD_0$ 输入的存储器数据转换到 CPU 数据总线上并锁存在 MD 锁存器中,MD 锁存器的输出成为奇偶校验电路的第三组输入。根据这三组输入,奇偶校验电路开始对 32 位长的输入字进行奇偶校验,如果发现错误,则 D 型触发器被复位,在 $\overline{PARERROR}$ 输出线上输出奇偶校验错信号。②在写操作中,CPU 数据总线上的 32 位数据转换到 MD 总线收发器的输入口。这些数据同时也送入通向奇偶校验生成电路的多路转换器的 B 端口,奇偶校验码生成电路收到数据后,生成 1 个 4 位长的奇偶校验码(每个字节生成 1 位校验码),奇偶校验码送入校验码收发器的输入口,在 $PAR_3 \sim PAR_0$ 输出,并与 MD 数据总线上的数据字 $MD_{31} \sim MD_0$ 一起写入存储器。

9.8 80386 通过 $XD_7 \sim XD_0$ 访问 82346 的内部配置寄存器,其地址是在 I/O 地址范围内。

9.9 82344 芯片内部电路主要包括 4 个部分:①数据转换及等待状态控制电路,将 80386 DX 产生的字节允许信号变为 80286 兼容的奇偶字节选择信号,用于 ISA 扩展总线中。② 82284/82288 准备好信号及总线控制电路,为 ISA 扩展总线产生许多控制信号。③外围控制电路,集成了中断控制器、DMA 控制器、页面地址寄存器、可编程定时/计数器、扬声器驱动电路、实时时钟和刷新计数器等,用于控制 PC/AT 机的外围设备。④地址译码器,根据输入的地址信号产生片选信号,用于启动 8042 键盘控制器及访问主板上的 BIOS EPROM 芯片。

9.10 参见教材 9.3.5 82341 高集成度外围组合芯片说明及图 9.9。

9.11 EISA 微机系统的外围芯片组 82350 包含 3 种芯片: EISA 总线缓冲器 82352(用于对 CPU 引脚上数据线及地址的缓冲)、ISA 总线控制器 82358(用于对 CPU 引脚上控制总线的控制)、集成系统外围电路 82357(集成中断控制器、定时/计数器、DMA 控制器、NMI 信号产生电路 4 种 I/O 外围电路)。

9.12 Pentium 微机系统由主板及其部件和硬盘、软盘、光盘驱动器、显示器、键盘等外设组成。CHIPS M520(VX)主板的 CPU 外围芯片组由三个组件构成。PCMC 的功能是管理 PCI 总线、外部 Cache,控制 DRAM 的刷新和读/写操作等。局部总线加速器芯片 LBX 为 CPU 与内存、CPU 与 PCI 总线、内存与 PCI 总线进行数据读/写操作提供缓冲功能。系统 I/O 芯片 SIO 用于将 PCI 总线变为标准 ISA 总线,并负责多个 ISA 设备的仲裁,此外还集成了中断控制、DMA 控制等诸多功能。

附录 B

ASCII 码表及控制符号定义

B.1 ASCII码表

	列	0	1	2	3	4	5	6	7
行	高位 低位	000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	,	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*		J	Z	j	z
B	1011	VT	ESC	+	,	K	[	k	{
C	1100	FF	FS	.	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS		>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

注：表中 00H 至 1FH 以及 7FH 为控制符，不可显示；其余的为可显示字符。

B.2 ASCII码表中控制符号的定义

NUL	Null	空白	DLE	Data Link Escape	转义
SOH	Start of Heading	标题开始	DC1	Device Control 1	设备控制 1
STX	Start of Text	正文开始	DC2	Device Control 2	设备控制 2
ETX	End of Text	正文结束	DC3	Device Control 3	设备控制 3
EOT	End of Transmit	传输结束	DC4	Device Control 4	设备控制 4

续表

ENQ Enquiry	询问	NAK Negative Acknowledge	否定
ACK Acknowledge	承认	SYN Synchronize	同步
BEL Bell	响铃	ETB End of Transmitted Block	信息组结束
BS Backspace	退格	CAN Cancel	作废
HT Horizontal Tab	横向制表	EM End of Medium	纸尽
LF Line Feed	换行	SUB Substitute	取代
VT Vertical Tab	纵向制表	ESC Escape	换码
FF Form Feed	换页	FS File Separator	文件分隔符
CR Carriage Return	回车	GS Group Separator	组分隔符
SO Shift Out	移出	RS Record Separator	记录分隔符
SI Shift In	移入	US Unit Separator	单元分隔符
SP Space	空格	DEL Delete	删除

附录 C

中断类型分配

类 别	中断向量地址	中断类型	功 能
80X86 中断	0~3	0	除法错
	4~7	1	单步 (用于 DEBUG)
	8~B	2	非屏蔽中断 (NMI)
	C~F	3	断点中断 (用于 DEBUG)
	10~13	4	溢出中断
	14~17	5	打印屏幕
	18~1F	6、7	保留
8259 中断	20~23	8	定时器 (IRQ ₀)
	24~27	9	键盘 (IRQ ₁)
	28~2B	A	彩色/图形 (IRQ ₂)
	2C~2F	B	串行通信 COM2 (IRQ ₃)
	30~33	C	串行通信 COM1 (IRQ ₄)
	34~37	D	并行通信 LPT2 (IRQ ₅)
	38~3B	E	磁盘 (IRQ ₆)
	3C~3F	F	并行通信 LPT1 (IRQ ₇)
BIOS 软中断	40~43	10	屏幕显示
	44~47	11	设备检验
	48~4B	12	检测存储器容量
	4C~4F	13	磁盘 I/O
	50~53	14	串行通信口 RS-232C I/O
	54~57	15	盒式磁带、I/O 系统扩展
	58~5B	16	键盘输入
	5C~5F	17	打印机输出
	60~63	18	ROM-BASIC 入口
	64~67	19	引导装入程序
	68~6B	1A	门时钟
提供给用户的中断	6C~6F	1B	键盘 Ctrl-Break 控制的软中断
	70~73	1C	定时器控制的软中断
数据表指针	74~77	1D	显示器参数表
	78~7B	1E	软盘参数表
	7C~7F	1F	图形表

续表

DOS 软中断	80~83	20	程序正常结束
	84~87	21	DOS 系统功能调用
	88~8B	22	程序结束退出
	8C~8F	23	Ctrl-Break 退出
	90~93	24	严重错误处理
	94~97	25	绝对磁盘读功能
	98~9B	26	绝对磁盘写功能
	9C~9F	27	程序驻留并退出
	A0~A3	28	DOS 安全使用
	A4~A7	29	快速写字符
	A8~AB	2A	Microsoft 网络接口
	B8~BB	2E	基本 SHELL 程序装入
	BC~BF	2F	多路服务中断
	CC~CF	33	鼠标中断
	104~107	41	硬盘参数块
	118~11B	46	第二硬盘参数表
	11C~3FF	47~FF	BASIC 中断

附录 D

MS-DOS 系统功能调用

功能号 (AH)	功 能	调用参数	返回参数
00H	程序终止 (同 INT ₂₀)	CS = 程序段前缀 PSP	
01H	键盘输入并回显		AL = 输入字符
02H	显示器输出	DL = 输出字符	
03H	串行通信 COM1 输入		AL = 输入数据
04H	串行通信 COM1 输出	DL = 输出字符	
05H	打印机输出	DL = 输出字符	
06H	直接控制台 I/O	DL = FFH (输入) DL = 字符 (输出)	AL = 输入字符
07H	直接控制台输入 (无回显, 不检测 Ctrl-C)		AL = 输入字符
08H	键盘输入 (无回显, 检测 Ctrl-C)		AL = 输入字符
09H	显示字符串	DS: DX = 串地址, 字符串以 “\$” 结尾	
0AH	带缓冲区的键盘输入 (输入字符串)	DS DX = 缓冲器首址 (DS DX = 缓冲区最大字符数)	DS: DX+1 = 实际输入的字符数
0BH	检查键盘状态		AL = FF, 有输入, AL = 00, 无输入
0CH	清除键盘缓冲区, 并执 行指定输入功能	AL = 输入功能号 (1, 6, 7, 8 或 A)	
0DH	磁盘复位		清除文件缓冲区
0EH	指定当前默认的磁盘驱 动器	DL = 驱动器号 (0 = A, 1 = B, 2 = C, ...)	AL = 驱动器数目
0FH	打开文件 (FCB)	DS: DX = FCB 首地址	AL = 00 文件找到 AL = FF 文件未找到
10H	关闭文件 (FCB)	DS: DX = FCB 首地址	AL = 00 目录修改成功 AL = FF 目录中未找到文件
11H	查找第一个目录项	DS: DX = FCB 首地址	AL = 00 找到, AL = FF 未找到
12H	查找下一个目录项	DS: DX = FCB 首地址	AL = 00 找到, AL = FF 未找到
13H	删除文件 (FCB)	DS: DX = FCB 首地址	AL = 00 删除成功 AL = FF 未找到

续表

功能号 (AH)	功 能	调用参数	返回参数
14H	顺序读文件 (FCB)	DS: DX = FCB 首地址	AL = 00 读成功 AL = 01 文件结束, 未读到数据 AL = 02 DTA 空间不够 AL = 03 文件结束, 记录不完整
15H	顺序写文件 (FCB)	DS: DX = FCB 首地址	AL = 00 写成功 AL = 01 盘满 AL = 02 DTA 空间不够
16H	建文件 (FCB)	DS: DX = FCB 首地址	AL = 00 建文件成功 AL = FF 无磁盘空间
17H	更改文件名	DS: DX = FCB 首地址	AL = 00 文件被改名 AL = FF 文件未改名
19H	取当前缺省磁盘驱动器		AL = 缺省的驱动器号 0 = A, 1 = B, 2 = C, ...
1AH	设置磁盘传送缓冲区 (DTA)	DS: DX = DTA 地址	
1BH	取当前盘文件分配表 (FAT) 信息		AL = 每簇扇区数 DS: DX = FTA 标识字节 CX = 物理扇区的字节数 DX = 每磁盘的簇数
1CH	取指定盘文件分配表 (FAT) 信息	DL = 驱动器号	同上
21H	随机读文件 (FCB)	DS: DX = FCB 首地址	AL = 00 读成功 AL = 01 文件结束 AL = 02 缓冲区溢出 AL = 03 缓冲区不满
22H	随机写文件 (FCB)	DS: DX = FCB 首地址	AL = 00 写成功 AL = 01 磁盘满 AL = 02 缓冲区溢出
23H	测定文件大小 (FCB)	DS: DX = FCB 首地址	AL = 00 成功, 文件长度填入 FCB AL = FF 未找到匹配文件
24H	设置随机记录号	DS: DX = FCB 首地址	
25H	设置中断向量	DS: DX = 中断向量 AL = 中断类型号	
26H	建立程序段前缀 PSP	DX = 新 PSP 段地址	
27H	随机读多个记录 (随机分块读)	DS: DX = FCB 首地址 CX = 记录数	AL = 00 读成功 AL = 01 文件结束 AL = 02 缓冲区太小, 传输结束 AL = 03 缓冲区不满

续表

功能号 (AH)	功 能	调用参数	返回参数
28H	随机写多个记录 (随机分块写)	DS: DX = FCB 首地址 CX = 记录数	AL = 00 写成功 AL = 01 磁盘满 AL = 02 缓冲区溢出
29H	分析文件名	ES: DI = FCB 首地址 DS: SI = 字符串地址 AL = 分析控制标志	AL = 00 标准文件 AL = 01 多义文件 AL = FF 非法盘符
2A	取系统日期		CX = 年 (1980—2099) DH = 月 (1—12) DL = 日 (1—31) AL = 星期 (0—6)
2B	置系统日期	CX = 年 (1980—2099) DH = 月 (1—12) DL = 日 (1—31)	AL = 00 成功 AL = FF 无效
2C	取系统时间		CH: CL = 时: 分 DH: DL = 秒: 1/100 秒
2D	置系统时间	CH: CL = 时: 分 DH: DL = 秒: 1/100 秒	AL = 00 成功 AL = FF 无效
2E	置磁盘自动读写标志	AL = 00 关闭标志 AL = FF 打开标志	
2F	取磁盘缓冲区首址		ES: BX = 缓冲区首址
30	取 DOS 版本号		AH = 发行号 AL = 版本号 BH = DOS 版本标志 BL: CX = 序号 (24 位)
31H	结束并驻留在内存	AL = 返回码 DX = 驻留区大小	
33H	置/取 Ctrl+Break 检测状态	AL = 00 取状态 AL = 01 置状态 (DL)	DL = 00 关闭检测 DL = 01 打开检测
35H	取中断向量	AL = 中断类型号	ES: BX = 中断入口地址
36H	取空闲磁盘空间	DL = 驱动器号 (0 = 默认, 1 = A, 2 = B, ...)	成功: AX = 每簇扇区数 BX = 可用簇数 CX = 每扇区字节数 DX = 总簇数 失败: AX = FFFFH
38H	置/取国别信息	DS: DX = 信息区首址 AL = 00 取当前国别信息 AL = FF 国别代码在 BX DX = FFFF 置国别代码	BX = 国别代码 (国际电话前缀码) DS: DX = 返回的信息区首址 AX = 错误代码

续表

功能号 (AH)	功 能	调用参数	返回参数
39H	建立子目录 (MKDIR)	DS: DX=目录路径中首地址	CF=0 成功, 否则失败
3AH	删除子目录 (RMDIR)	DS: DX=目录路径中首地址	CF=0 成功, 否则失败
3BH	改变子目录 (CHDIR)	DS: DX=目录路径中首地址	CF=0 成功, 否则失败
3CH	建立文件	DS: DX=文件名首址 CX=文件属性	成功: AX=文件号 (CF=0) 失败: AX=错误码 (CF=1)
3DH	打开文件	DS: DX=文件名首址 AL=0 只读 AL=1 只写 AL=2 读/写	成功: AX=文件号 (CF=0) 失败: AX=错误码 (CF=1)
3EH	关闭文件	BX=文件号	CF=0 成功, 否则失败
3FH	读文件或设备	DS: DX=缓冲区首址 BX=文件号 CX=读取的字节数	成功: AX=读入的字节数 (CF=0) 失败: AX=错误代码 (CF=1)
40H	写文件或设备	DS: DX=缓冲区首址 BX=文件号 CX=写入的字节数	成功: AX=写入的字节数 (CF=0) 失败: AX=错误代码 (CF=1)
41H	删除文件	DS: DX=文件名首址	成功: AX=0 失败: AX=错误码 (CF=1)
42H	移动文件读写指针	BX=文件号 CX: DX=位移量 AL=移动方式 AL=0 从文件头开始移动 AL=1 从当前位置移动 AL=2 从文件尾倒移	成功: DX: AX=新的指针位置 失败: AX=错误代码 (CF=1)
43H	置/取文件属性	DS: DX=文件名首址 AL=0 取文件属性 AL=1 置文件属性 CX=文件属性	成功: CX=文件属性 (读时) 失败: AX=错误代码 (CF=1)
44H	设备文件 I/O 控制	BX=文件号 AL=0 取状态 AL=1 置状态 DX AL=2 读字符设备 AL=3 写字符设备 AL=4 读块设备 AL=5 写块设备 AL=6 取输入状态 AL=7 取输出状态 CX=读/写的字节数	成功: DX=设备信息 AX=传送的字节数 失败: AX=错误代码 (CF=1)

续表

功能号 (AH)	功 能	调用参数	返回参数
45H	复制文件号	BX = 文件号 1	成功: AX = 文件号 2 失败: AX = 错误代码 (CF = 1)
46H	强行复制文件号	BX = 文件号 2 CX = 文件号 1	成功: CX = 文件号 1 失败: AX = 错误代码 (CF = 1)
47H	取当前目录路径名	DL = 盘号, DS:SI = 字符串地址 (从根目录开始的路径名)	成功: DS:SI = 当前目录路径名 失败: AX = 错误代码 (CF = 1)
48H	分配内存空间	BX = 申请内存数量	成功: AX = 分配内存首址 失败: AX = 错误代码 (CF = 1) BX = 最大可用空间
49H	释放已分配内存空间	ES = 内存起始段地址	失败: AX = 错误代码
4AH	修改已分配内存空间	ES = 原内存起始段地址 BX = 新申请内存字节数	失败: AX = 错误代码 (CF = 1) BX = 最大可用空间
4BH	装入/执行程序	DS:DX = 字符串地址 ES:BX = 参数区首址 AL = 0 装入并执行 AL = 3 仅装入	CF = 0, 成功 CF = 1, 失败
4CH	终止当前程序, 返回调用程序	AL = 返回码	
4DH	取返回代码		AX = 返回代码
4EH	查找第一个匹配文件	DS:DX = 字符串地址 CX = 文件属性	失败: AX = 错误代码 (CF = 1)
4FH	查找下一个匹配文件	DTA 中有 4EH 得到的信息	失败: AX = 错误代码 (CF = 1)
50H	置 PSP 段地址	BX = 新 PSP 段地址	
51H	取 PSP 段地址		BX = 当前运行进程的 PSP
52H	取磁盘参数块		ES:BX = 参数块链表指针
53H	把 BIOS 参数块(BPB)转换为 DOS 的驱动器参数块(DPB)	DS:SI = BPB 的指针 ES:BP = DPB 的指针	
54H	取写盘后读盘的校验状态		AL = 0 检验关闭 AL = 1 检验打开
55H	建立 PSP	DX = 建立 PSP 的段地址	
56H	文件改名	DS:DX = 老文件名地址, ES:DI = 新文件名地址	失败: AX = 错误代码 (CF = 1)
57H	置/取文件日期和时间	BX = 文件号 AL = 0 读取日期和时间 AL = 1 设置日期和时间 DX: CX = 日期、时间	成功: DX: CX = 日期和时间 (CF = 0) 失败: AX = 错误代码 (CF = 1)

续表

功能号 (AH)	功 能	调用参数	返回参数
58H	取/置内存分配策略码	AL = 0 取策略代码 AL = 1 置策略代码 BX = 策略代码	成功: AX = 策略代码 失败: AX = 错误代码 (CF = 1)
59H	取扩充错误码	BX = 00	AX = 错误扩充码 BH = 错误类型 BL = 建议的操作 CH = 出错设备代码
5AH	建立临时文件	CX = 文件属性, DS:DX = 文件名地址(以\结束)	成功: AX = 文件代号 失败: AX = 错误代码 (CF = 1)
5BH	建立新文件	CX = 文件属性 DS: DX = 文件名地址	成功: AX = 文件代号 失败: AX = 错误代码 (CF = 1)
5CH	控制文件存取	AL = 0 封锁 AL = 1 开锁 BX = 文件代号 CX: DX = 文件偏移值 SI: DI = 文件长度	失败: AX = 错误代码 (CF = 1)
62H	取程序段前缀地址		BX = PSP 地址

附录 E

ROM BIOS 功能调用

INT	AH	功 能	调用参数	返回参数
10H	0	置显示方式	AL=00: 40×25 黑白方式 01: 40×25 彩色方式 (16 色) 02: 80×25 黑白方式 03: 80×25 彩色方式 (16 色) 04: 320×200 彩色图形 (4 色) 05: 320×200 黑白图形 06: 640×200 黑白图形 07: 80×25 单色文本 08: 160×200 16 色图形 (PCjr) 09: 320×200 16 色图形 (PCjr) 0A: 640×200 16 色图形 (PCjr) 0B: 保留 0C: 保留 0D: 320×200 彩色图形 (16 色 EGA) 0E: 640×200 彩色图形 (16 色 EGA) 0F: 640×350 黑白图形 (EGA) 10: 640×350 彩色图形 (EGA/VGA) 11: 640×480 单色图形 (EGA) 12: 640×480 16 色图形 (EGA) 13: 320×200 256 色图形 (EGA) 14: 360×480 256 色图形 40: 80×30 彩色文本 (CGE400) 41: 80×50 彩色文本 (CGE400) 42: 640×400 彩色文本 (CGE400) 56: 132×60(16 色) super VGA 5B: 800×600(16 色) super VGA 5C: 640×400(256 色) super VGA 5D: 640×480(256 色) TVGA 5E: 800×600(16 色) TVGA 5F: 1024×768(16 色) super VGA 60: 1024×768(4 色) super VGA 62: 1024×768(256 色) super VGA	

续表

INT	AH	功 能	调用参数	返回参数
10H	1	置光标类型	(CH) 0~3 = 光标起始行 (CH) 0~3 = 光标结束行	
10H	2	置光标位置	BH = 页号, DH/DL = 行/列	
10H	3	读光标位置	BH = 页号	CH = 光标起始行 CL = 光标结束行 DH/DL = 行/列
10H	4	读光笔位置		AX = 0 光笔未触发 AX = 1 光笔触发 CH/BX = 像素行/列 DH/DL = 字符行/列
10H	5	置当前显示页	AL = 页号	
10H	6	当前页上滚	AL = 上滚行数, AL = 0 整个窗口空白 BH = 卷入行属性 CH、CL = 滚动区域左上角的行、列号 DH、DL = 滚动区域右下角的行、列号	
10H	7	当前页下滚	AL = 下滚行数, AL = 0 整个窗口空白 BH = 卷入行属性 CH、CL = 滚动区域左上角的行、列号 DH、DL = 滚动区域右下角的行、列号	
10H	8	读光标位置的字符和属性	BH = 显示页号	AH = 属性 AL = 字符
10H	9	在光标位置显示字符和属性	BH = 显示页号 AL/BL = 字符/属性 CX = 字符重复次数	
10H	A	在光标位置显示字符	BH = 显示页号, AL = 字符 CX = 字符重复次数	
10H	B	置彩色调色板	BH = 彩色调色板 ID BL = 和 ID 配套使用的颜色	
10H	C	写像素	AL = 像素值 DX = 行 (0~199) CX = 列 (0~639)	
10H	D	读像素	DX = 行 (0~199) CX = 列 (0~639)	AL = 像素值
10H	E	显示字符(光标前移)	AL = 字符 BL = 前景色, BH = 页号	
10H	F	取当前显示方式		AL = 显示方式 AH = 字符列数
10H	10	置调色板寄存器 (EGA/VGA)	AL = 0, BL = 调色板号, BH = 颜色值	

续表

INT	AH	功 能	调用参数	返回参数
10H	11	装入字符发生器 (EGA/VGA)	AL=0~4 全部或部分装入字符点阵集 AL=20~24 置图形方式显示字符集 AL=30 读当前字符集信息	
10H	12	返回当前适配器 设置的信息 (EGA/VGA)	BL=10H 子功能	BH=0 单色方式, BH =1 彩色方式, BL= VRAM 容量 (0=64KB, 1=128KB, ...) CH=特征位设置 CL=EGA 的开关设置
10H	13	显示字符串	ES: BP=字符串地址 AL=写方式 (0~3), CX=字符串长度 DH/DL=起始行/列, BH/BL=页号/属性	
11H		取系统设备信息		AX=返回值 (位映像) 0: 对应设备未安装 1: 对应设备已安装
12H		取内存容量		AX=内存容量 (单位 KB)
13H	0	磁盘复位	DL=驱动器号 (00, 01 为软盘; 80, 81, ... 为硬盘)	AH=磁盘状态 失败: AB=错误代码
13H	1	读磁盘驱动器状 态		AH=磁盘状态字节
13H	2	读磁盘扇区	AL=扇区数 CH, CL=磁道号, 扇区号 DH, DL=磁头号, 驱动器号 ES: BX=数据缓冲区地址	成功 AH=0, AL=读取 的扇区数 失败 AH=错误代码 (CF=1)
13H	3	写磁盘扇区	AL=扇区数 CH, CL=磁道号, 扇区号 DH, DL=磁头号, 驱动器号 ES: BX=数据缓冲区地址	成功 AH=0, AL=写入 的扇区数 失败 AH=错误代码 (CF=1)
13H	4	检验磁盘扇区	AL=扇区数 CH, CL=磁道号, 扇区号 DH, DL=磁头号, 驱动器号	成功 AH=0, AL=检验 的扇区数 失败 AH=错误代码 (CF=1)
13H	5	格式化磁盘磁道	AL=扇区数 CH, CL=磁道号, 扇区号 DH, DL=磁头号, 驱动器号 ES: BX=格式化参数表指针	成功: AH=0 失败: AH=错误代码

续表

INT	AH	功 能	调用参数	返回参数
14H	0	初始化串行口	AL = 初始化参数 DX = 串行口号	AH = 通信口状态 AL = 调制解调器状态
14H	1	向通信口写字符	AL = 字符 DX = 通信口号	AH0-6 = 通信口状态 AH7=0, 表示成功 AH7=1, 表示失败
14H	2	从通信口读字符	DX = 通信口号	AH7=0, 表示成功, AL = 字符 AH7=1, 表示失败
14H	3	取通信口状态	DX = 通信口号	AH = 通信口状态 AL = 调制解调器状态
	4	初始化扩展 COM		
15H	0	启动盒式磁带机		
15H	1	停止盒式磁带机		
15H	2	磁带分块读	EX: BX = 数据传输区地址 CX = 字节数	AH = 状态字节 AH = 00 读成功 AH = 01 冗余检验错 AH = 02 无数据传输 AH = 04 无引导 AH = 80 非法命令
15H	3	磁带分块写	EX: BX = 数据传输区地址 CX = 字节数	AH = 状态字节 (同上)
16H	0	从键盘读字符		AH = 键入字符扫描码 AL = 键入字符 ASCII 码
16H	1	判有无键入(读键盘缓冲区字符)		ZF=0 键盘有输入, AL = 字符码, AH = 扫描码, ZF=1 键盘无输入, 缓冲区空, 等待
16H	2	取键盘状态字节		AL = 键盘状态字节
17H	0	打印字符	AL = 字符, DX = 打印机号	AH = 打印机状态字节
17H	1	初始化打印机	DX = 打印机号	AH = 打印机状态字节
17H	2	取打印机状态	DX = 打印机号	AH = 打印机状态字节
18H		ROM BASIC 语言		
19H		引导装入程序		
1AH	0	读时钟		CH: CL = 时: 分 DH: DL = 秒: 1/100 秒
1AH	1	置时钟	CH: CL = 时: 分 DH: DL = 秒: 1/100 秒	
1AH	6	置报警时间	CH: CL = 时: 分 (BCD) DH: DL = 秒: 1/100 秒 (BCD)	

续表

INT	AH	功 能	调用参数	返回参数
1AH	7	清除报警		
33H	0	鼠标复位	AL = 00	BX = 鼠标的键数
33H	0	显示鼠标光标	AL = 01	显示鼠标光标
33H	0	隐藏鼠标光标	AL = 02	隐藏鼠标光标
33H	0	读鼠标状态	AL = 03	BX = 键状态 CX/DX = 鼠标水平/垂直位置
33H	0	设置鼠标位置	AL = 04, CX/DX = 鼠标水平/垂直位置	
33H	0	设置图形光标	AL = 09, BX/CX = 鼠标水平/垂直中心 ES: DX = 16 × 16 光标映像地址	
33H	0	设置文本光标	AL = 0A, BX = 光标类型 CX = 像素位掩码或起始扫描线 DX = 光标掩码或结束扫描线	设置的文本光标
33H	0	读移动计数器	AL = 0B	CX/DX = 鼠标水平/垂直距离
33H	0	设置中断子程序	AL = 0C, CX = 中断掩码 ES: DX = 中断服务程序的地址	

F.1 指令集中所用符号说明

符 号	说 明
acc	AL/AX/EAX 累加器
reg	通用寄存器
r8/r16/r32	8 位/16 位/32 位通用寄存器
seg	段寄存器
mm	整数 MMX 寄存器: MMX ₀ ~MMX ₇
xmm	128 位的浮点 SIMD 寄存器: XMM ₀ ~XMM ₇
m8/m16/m32/m64/m128	8 位/16 位/32 位/64 位/128 位存储器操作数
mem	8 位或 16 位或 32 位存储器操作数
i8/i16/i32	8 位/16 位/32 位立即操作数
imm	8 位或 16 位或 32 位立即操作数
dst	目的操作数
src	源操作数
label	标号
m16&32	16 位段限和 32 位段基地址
d8/d16/d32	8 位/16 位/32 位偏移地址
EA	有效地址

F.2 整数操作指令

格 式	功 能	备 注
AAA	加法运算后用 ASCII 码调整 AL	
AAD	除法运算后用 ASCII 码调整 AX	
AAM	乘法运算后用 ASCII 码调整 AX	
AAS	减法运算后用 ASCII 码调整 AL	
ADC reg, mem/imm/reg mem, reg/imm acc, imm	带进位加法 (dst) ← (src)+(dst)+CF	

续表

格 式	功 能	备 注
ADD reg, mem/imm/reg mem, reg/imm acc, imm	加法 $(dst) \leftarrow (src) + (dst)$	
AND reg, mem/imm/reg mem, reg/imm acc, imm	逻辑乘 $(dst) \leftarrow (src) \wedge (dst)$	
ARPL dst, src	调整选择符的 RPL 字段	286 起有, 系统指令
BOUND reg, mem	检查数组下标是否越界, 越界则产生 INT5	286 起有
BSF r16, r16/m16 r32, r32/m32	自右向左位扫描(src), 遇第一个为 1 的位, 则 $ZF \leftarrow 0$, 该位位置装入 reg; 若 $(src) = 0$, 则 $ZF \leftarrow 1$	386 起有
BSR r16, r16/m16 r32, r32/m32	自左向右位扫描(src), 遇第一个为 1 的位, 则 $ZF \leftarrow 0$, 该位位置装入 reg; 若 $(src) = 0$, 则 $ZF \leftarrow 1$	386 起有
BSWAP r32	(r32)字节次序变反	486 起有
BT reg, reg/i8 mem, reg/i8	位测试	386 起有
BTC reg, reg/i8 mem, reg/i8	位测试并求反	386 起有
BTR reg, reg/i8 mem, reg/i8	位测试并清零	386 起有
BTS reg, reg/i8 mem, reg/i8	位测试并置位	386 起有
CALL reg/mem	调用过程 (子程序) 段内直接: push (IP 或 EIP), $(IP) \leftarrow (IP) + d16$ 或 $(EIP) \leftarrow (EIP) + d32$ 段内间接: push (IP 或 EIP), $(IP \text{ 或 } EIP) \leftarrow (EA)/reg$ 段间直接: push CS, push (IP 或 EIP), $(CS) \leftarrow dst$ 指定的段地址, $(IP \text{ 或 } EIP) \leftarrow dst$ 指定的偏移地址 段间间接: push CS, push (IP 或 EIP), $(IP \text{ 或 } EIP) \leftarrow (EA)$, $(CS) \leftarrow (EA + 2)$ 或 $(EA + 4)$	
CBW	字节转换成字, (AL)符号扩展到(AH)	
CDQ	双字转换成四字, (EAX)符号扩展到(EDX)	386 起有
CLC	进位标志清零 ($CF \leftarrow 0$)	
CLD	方向标志清零 ($DF \leftarrow 0$)	
CLI	中断标志清零 ($IF \leftarrow 0$)	
CLTS	CR ₀ 中的任务切换标志清零	386 起有, 系统指令
CMC	进位标志求反	
CMP reg, reg/mem/imm mem, reg/imm	比较两个操作数, $(dst) - (src)$, 结果影响标志位	

续表

格 式	功 能	备 注
CMPS/CMP SB/CMP SW/ CMP SD	数据中比较	
CMPXCHG reg/mem, reg	比较并交换 (acc ← dst), 相等: ZF ← 1, (dst) ← (src) 不相等: ZF ← 0, (acc) ← (src)	486 起有
CMPXCHG8B dst	比较并交换 8 字节 (EDX, EAX) ← (dst), 相等: ZF ← 1, (dst) ← (ECX, EBX), 不相等: ZF ← 0, (EDX, EAX) ← (dst)	586 起有
CWD	字转换成双字 (AX) 符号扩展到 (DX)	
CWDE	字转换成双字 (AX) 符号扩展到 (EAX)	386 起有
DAA	加法后对 AL 作十进制调整	
DAS	减法后对 AL 作十进制调整	
DEC reg/mem	减 1 (dst) ← (dst) - 1	
DIV r8/m8 r16/m16 r32/m32	无符号数除法	386 起有
ENTER i16, i8	建立堆栈帧	386 起有
HLT	暂停	
IDIV r8/m8 r16/m16 r32/m32	带符号整数除法	386 起有
IMUL r8/m8 r16/m16 r32/m32	带符号整数乘法	386 起有
IN acc, i8/DX	自端口输入数据 (acc) ← (i8) 或 (DX)	
INC reg/mem	加 1 (dst) ← (dst) + 1	
INS/INSB/INSW/INSD	串输入	286 起有
INT i8	中断	
INTO	溢出中断	
INVD	清高速缓存	486 起有, 系统指令
IRET/IRETD IRET IRETD	中断返回 (IP) ← POP(), (FLAGS) ← POP() (EIP) ← POP(), (CS) ← POP() < (EFLAGS) ← POP()	386 起有
JZ/JE d8/d16/132	零标志 ZF = 1, 则转移 (等于或为零转移)	d16/d32 从 386 起有
JNZ/JNE d8/d16/132	零标志 ZF = 0, 则转移 (不等于或不为零转移)	同上
JS d8/d16/132	符号标志 SF = 1, 则转移 (结果为负转移)	同上
JNS d8/d16/132	符号标志 SF = 0, 则转移 (结果为正转移)	同上
JO d8/d16/132	溢出标志 OF = 1, 则转移 (溢出转移)	同上
JNO d8/d16/132	溢出标志 OF = 0, 则转移 (无溢出转移)	同上

续表

格 式	功 能	备 注
JP/JPE d8/d16/32	奇偶标志 PF=1, 则转移 (结果为偶数个 1 转移)	同上
JNP/JPO d8/d16/32	奇偶标志 PF=0, 则转移 (结果为奇数个 1 转移)	同上
JC d8/d16/32	进位标志 CF=1, 则转移 (有进位)	同上
JNC d8/d16/32	进位标志 CF=0, 则转移 (无进位)	同上
JB/JNAE d8/d16/32	低于或不高于等于转移	同上
JNB/JAE d8/d16/32	不低于或高于等于转移	同上
JBE/JNA d8/d16/32	低于等于或不高于转移	同上
JNBE/JA d8/d16/32	不低于等于或高于转移	同上
JL/JNGE d8/d16/32	小于或不大于等于转移	同上
JNL/JGE d8/d16/32	不小于或大于等于转移	同上
JLE/JNG d8/d16/32	小于等于或不大于转移	同上
JNLE/JG d8/d16/32	不小于等于或大于转移	同上
JCXZ/JECXZ d8/d16/d32	当 CX 或 ECX 等于零时转移	386 起有
JMP label/mem/reg	无条件转移(段内/段间直接或间接转移)	
LAHF	把标志寄存器 (FLAGS) 的低字节装入 AH 中	
LAR reg, mem/reg	装入访问权限字节	286 起有, 系统指令
LDS reg, mem	把指针装入 DS	
LEA reg, mem	装入有效地址	
LEAVE	高级过程退出	286 起有
LES reg, mem	把指针装入 ES	
LFS reg, mem	把指针装入 FS	386 起有
LGDT mem	装入全局描述符表寄存器(GDTR)←(mem)	286 起有, 系统指令
LGS reg, mem	把指针装入 GS	386 起有
LIDT mem	装入中断描述符表寄存器(IDTR)←(mem)	286 起有, 系统指令
LLDT reg/mem	装入局部描述符表寄存器(LDTR)←(reg/mem)	286 起有, 系统指令
LMSW reg/mem	装入机器状态字 (在 CR ₀ 寄存器中)	286 起有, 系统指令
LOCK	插入 LOCK # 信号前缀	
LODS/LODSB/LODSW/ LODSD	装入字符串操作数 (AL/AX/EAX) ← (SI 或 ESI) (SI 或 ESI) ← (SI 或 ESI) + (1 或 2 或 4)	ESI 自 386 起有
LOOP label	(CX/ECX) ≠ 0 时循环	ECX 自 386 起有
LOOPE/LOOPZ label	(CX/ECX) ≠ 0 且 ZF=1 则循环 (相等/为 0 时)	同上
LOOPNE/LOOPNZ label	(CX/ECX) ≠ 0 且 ZF=0 则循环 (不相等/不为 0 时)	同上
LSL reg, reg/mem	装入段界	286 起有, 系统指令
LSS reg, mem	把指针装入 SS	386 起有
LTR reg/mem	装入任务寄存器	286 起有, 系统指令

续表

格 式	功 能	备 注
MOV reg, reg/mem/imm mem, reg/imm Reg, CR ₀ ~CR ₃ CR ₀ ~CR ₃ , reg reg, DR DR, reg reg, SR SR, reg	传送数据 (reg) ← (reg/mem/imm) (mem) ← (reg/imm) (reg) ← (CR ₀ ~CR ₃) (CR ₀ ~CR ₃) ← (reg) (reg) ← (调试寄存器 DR) (DR) ← (reg) (reg) ← (段寄存器 SR) (SR) ← (reg)	386 起有, 系统指令 386 起有, 系统指令 386 起有, 系统指令 386 起有, 系统指令
MOVS/MOVSb/MOVSd/ MOVSD	字符串之间的数据传送	(MOVSD)386 起有
MOVSX reg, reg/mem	带符号扩展的数据传送	386 起有
MOVZX reg, reg/mem	带零扩展的数据传送	386 起有
MUL reg/mem	无符号数乘法 (AX) ← (AL)*(r8/m8) (DX,AX) ← (AX)*(r16/m16) (EDX,EAX) ← (EAX)*(r32/m32)	386 起有
NEG reg/mem	求补	
NOP	空操作	
NOT reg/mem	求反	
OR reg, reg/mem/imm mem, reg/imm	逻辑或 (reg) ← (reg) ∨ (reg/mem/imm) (mem) ← (mem) ∨ (reg/imm)	
OUT i8, acc DX, acc	输出数据到端口	
OUTS/OUTSB/OUTW/ OUTD	输出数据串到端口	386 起有
POP reg/mem/SR POPA/POPAD POPF/POPFD	从堆栈弹出数据(reg/mem/SR) ← ((SP 或 ESP)) (SP 或 ESP) ← (SP 或 ESP)+2 或 +4 出栈送 16 位/32 位通用寄存器 出栈送标志寄存器(FLAGS/EFLAGS)	286/386 起有 386 起有
PUSH reg/mem/SR/imm	数据压入堆栈(SP 或 ESP) ← (SP 或 ESP)-2 或 -4, ((SP 或 ESP)) ← (reg/mem/imm)	
PUSHA/PUSHAD PUSHF/PUSHFD	16 位/32 位通用寄存器压入堆栈 标志寄存器 FLAGS/EFLAGS 压入堆栈	286/386 起有 386 起有
RCL/RCR reg/mem, 1/CL/18	通过进位循环左移/右移	18 自 386 起有
RDMSR	从模式指定寄存器读	586 起有
REP	重复前缀(CX 或 ECX) ← (CX 或 ECX)-1, 当(CX 或 ECX) ≠ 0, 重复操作, REP 可加在 MOVS、STOS、LODS、INS、OUTS 指令前	

续表

格 式	功 能	备 注
REPE/REPZ	相等/为零时重复, 即当(CX 或 ECX $\neq$ 0 且 ZF=1 时重复, 可加在 CMPS、SCAS 指令前	
REPNE/REPZ	不相等/不为零时重复, 即当(CX 或 ECX $\neq$ 0 且 ZF $\neq$ 0 时重复, 可加在 CMPS、SCAS 指令前	
RET	从过程(子程序)返回	
ROL/ROR reg/mem, 1/CL/r8	不通过进位循环左移/右移	r8 自 386 起有
RSM	从系统管理方式恢复	586 起有, 系统指令
SAHF	把 AH 存到标志寄存器低字节中	
SAL/SAR reg/mem, 1/CL/r8	算术左移/算术右移	r8 自 386 起有
SBB reg, reg/mem/imm mem, reg/imm	带借位的整数减法	
SCAS/SCASB/SCASW/ SCASD	比较字符串数据	386 起有
SETcc r8/m8	按条件设置字节	386 起有
SGDT mem	保存全局描述符表寄存器	386 起有, 系统指令
SHL/SHR reg/mem, 1/CL/r8	逻辑左移/逻辑右移	
SHLD/SHRD reg/mem, reg, r8/CL	双精度左移/双精度右移	386 起有
SIDT	存储中断描述符表寄存器	286 起有, 系统指令
SLDT	存储局部描述符表寄存器	286 起有, 系统指令
SMSW	存储机器状态字	286 起有, 系统指令
STC	进位位置 1	
STD	方向标志置 1	
STI	中断标志置 1	
STOS/STOSB/STOSW/ STOSD	存储字符串数据	386 起有
STR reg/mem	存储任务寄存器 (reg/mem) $\leftarrow$ (IR)	286 起有, 系统指令
SUB reg, mem/imm/reg mem, reg/imm acc, imm	整数减法 (dst) $\leftarrow$ (dst)-(src)	
TEST ref, mem/imm/reg mem, reg/imm acc, imm	测试(逻辑比较) (dst) $\wedge$ (src), 结果影响标志位	
VERR/VERW reg/mem	读/写段检验	286 起有, 系统指令
WAIT	等待	
WBINVD	写回并使高速缓存无效	486 起有, 系统指令
WRMSR	写型号特定寄存器 MSR(ECX) $\leftarrow$ (EDX, ECX)	586 起有, 系统指令

续表

格 式	功 能	备 注
XADD reg/mem, reg	交换并相加 $TEMP \leftarrow (src) + (dst)$, $(src) \leftarrow (dst), (dst) \leftarrow TEMP$	486 起有
XCHG reg/acc/mem, reg	寄存器/存储器数据交换 $(dst) \leftrightarrow (src)$	
XLAT/XLATB	表查找翻译	
XOR reg, mem/imm/reg mem, reg/imm acc, imm	逻辑异或 $(dst) \oplus (src)$	

F.3 浮点操作指令

指 令	功 能
F2XM1	计算 $2^x - 1$ (计算 $2^{ST(0)} - 1$)
FABS	求绝对值
FADD/FADDP/FIADD	加/加并弹出栈/加整数
FBLD	装入 BCD 数
FBSTP	存储 BCD 数并弹出
FCHS	改变符号
FCLEX	清除异常
FCOM/FCOMP/FCOMPP	比较实数/比较实数并弹出栈/比较实数并弹出栈两次
FCOS	余弦计算 (ST(0)的余弦)
FDECSTP	堆栈指针减 1
FDIV/FDIVP/FIDIV	除法
FDIVR/FDIVRP/FIDIVR	反向除法
FFREE	释放 ST(i)寄存器
FICOM/FICOMP	比较整数/比较整数并弹出
FILD	装入整数
FINCSTP	堆栈指针增值
FINIT	初始化浮点单元
FIST/FISTP	存储整数/存储整数并弹出栈
FLD	装入实数
FLD1	把 +1.0 装入 ST(0)
FLDCW	装入控制字
FLDENV	装入 FPU 环境
FLDL2E	把 $\log_2(e)$ 装入 ST(0)
FLDL2T	把 $\log_2(10)$ 装入 ST(0)
FLDLG2	把 $\log_{10}(2)$ 装入 ST(0)
FLDLN2	把 $\log_e(2)$ 装入 ST(0)

续表

指 令	功 能
FLDPI	把 π 装入 ST(0)
FLDZ	把 +0.0 装入 ST(0)
FMUL/FMULP/FIMUL	乘法操作
FNOP	空操作
FPANAT	部分反正切
FPREM/FPREMI	部分余数
FPTAN	部分正切
FRNDINT	舍入为整数
FRSTOR	恢复 FPU 状态 (实方式和虚拟 86 方式, 16 位/32 位地址; 保护方式, 16 位/32 位地址)
FSAVE	存储 FPU 状态 (实方式和虚拟 86 方式, 16 位/32 位地址; 保护方式, 16 位/32 位地址)
FSCALE	比例换算
FSIN	计算正弦
FSINCOS	计算正弦与余弦
FSQRT	计算平方根
FST/FSTP	存储实数/存储实数并弹出栈 (32 位或 64 位存储器或 ST(i)/32 位或 64 位或 80 位存储器或 ST(i))
FSTCW	存储控制字
FSTENV	存储 FPU 环境 (实方式和虚拟 86 方式, 16 位/32 位地址; 保护方式, 16 位/32 位地址)
FSTSW/FNSTSW	存储状态字到 AX/存储状态字到存储器
FSUB/FSUBP/FISUB	减法
FSUBR/FSUBRP/FISUBR	反向减法
FTST	测试
FUCOM/FUCOMP/FUCOMPP	无序比较实数/无序比较并弹出栈/无序比较并两次弹出栈
FWAIT	等待直到 FPU 就绪
FXAM	检验
FXCH	寄存器内容互换 (交换 ST(0)和 ST(1))
EXTRACT	抽取指数和有效值
FYL2X	计算 $y \times \log_2 x$ ($ST(1) \times \log_2(ST(0))$)
FYL2XP1	计算 $y \times \log_2(x+1)$ ($ST(1) \times \log_2(ST(0)+1.0)$)

参 考 文 献

- 1 吴产乐主编. 微机系统与接口技术. 武汉: 华中科技大学出版社, 2003
- 2 Walter A. Trieble 著. 80X86/Pentium 处理器硬件、软件及接口技术教程. 王克义, 王均, 方晖等译. 北京: 清华大学出版社, 1998
- 3 曾家智, 周小华, 黄克军. 现代微机系统与接口. 成都: 电子科技大学出版社, 1999
- 4 姚燕南, 薛均义, 欧文等. 微型计算机原理. 西安: 电子科技大学出版社, 2000
- 5 Intel Corp. Microprocessors Volume I、II、III, 1993
- 6 Intel Corp. Pentium Processor Family User's Manual Volume1: Pentium Processor Family Data Book, 1994
- 7 Intel Corp. Pentium Processor Family User's Manual Volume3: Architecture and Programming Manual, 1994
- 8 Intel Corp. Pentium 4 Processor Manuals, 2000